

Ingeniería de Software

GUILLERMO PANTALEO - LUDMILA RINAUDO



libroWeb

 **Alfaomega**



Ingeniería de Software

Guillermo Pantaleo – Ludmila Rinaudo



Pantaleo, Guillermo

Ingeniería de software / Guillermo Pantaleo y Ludmila Rinaudo. - 1a ed. - Ciudad Autónoma de Buenos Aires : Alfaomega Grupo Editor Argentino, 2015.

480 p. ; 24x21 cm.

ISBN 978-987-1609-78-9

1. Informática. 2. Ingeniería de Software. I. Rinaudo, Ludmila II. Título
CDD 005.1

Queda prohibida la reproducción total o parcial de esta obra, su tratamiento informático y/o la transmisión por cualquier otra forma o medio sin autorización escrita de Alfaomega Grupo Editor Argentino S.A.

Edición: Héctor Germán Asenjo

Revisión de estilo: Romina Yael Ryzenberg

Diseño de interiores y portada: Iris Biaggini

Internet: <http://www.alfaomega.com.mx>

Todos los derechos reservados © 2015, por Alfaomega Grupo Editor Argentino S.A.

Paraguay 1307, PB, oficina 11, Buenos Aires

ISBN 978-958-778-044-4

Queda hecho el depósito que prevé la ley 11.723

NOTA IMPORTANTE: La información contenida en esta obra tiene un fin exclusivamente didáctico y, por lo tanto, no está previsto su aprovechamiento a nivel profesional o industrial. Las indicaciones técnicas y programas incluidos han sido elaborados con gran cuidado por el autor y reproducidos bajo estrictas normas de control. Alfaomega Grupo Editor Argentino S.A. no será jurídicamente responsable por: errores u omisiones; daños y perjuicios que se pudieran atribuir al uso de la información comprendida en este libro, ni por la utilización indebida que pudiera dársele.

Los nombres comerciales que aparecen en este libro son marcas registradas de sus propietarios y se mencionan únicamente con fines didácticos, por lo que Alfaomega Grupo Editor Argentino S.A. no asume ninguna responsabilidad por el uso que se dé a esta información, ya que no infringe ningún derecho de registro de marca. Los datos de los ejemplos y pantallas son ficticios, a no ser que se especifique lo contrario.

Los hipervínculos a los que se hace referencia no necesariamente son administrados por la editorial, por lo que no somos responsables de sus contenidos o de su disponibilidad en línea.

Empresas del grupo:

Argentina: Alfaomega Grupo Editor Argentino, S.A.

Paraguay 1307 P.B. "11", Buenos Aires, Argentina, C.P. 1057

Tel.: (54-11) 4811-7183 / 0887 - E-mail: ventas@alfaomegaeditor.com.ar

México: Alfaomega Grupo Editor, S.A. de C.V.

Pitágoras 1139, Col. Del Valle, México, D.F., México, C.P. 03100

Tel.: (52-55) 5575-5022 - Fax: (52-55) 5575-2420 / 2490. Sin costo: 01-800-020-4396

E-mail: atencionalcliente@alfaomega.com.mx

Colombia: Alfaomega Colombiana S.A.

Calle 62 No. 20-46 Tel: (571) 7460102 Fax:(571) 2100122 Bogotá, D.C.

cliente@alfaomegacolombiana.com

Chile: Alfaomega Grupo Editor, S.A.

Av. Providencia 1443. Oficina 24. Santiago de Chile. Tel. (562) 2235 4248 Tel. (562) 2947-9351 Fax. (562) 2235 5786

E-mail: agechile@alfaomega.cl

Agradecimientos

Deseamos agradecer a los señores Diego Montaldo,
Guillermo Rugilo y Nicolás Páez
por su colaboración en el inicio de este proyecto
y por su aporte en los temas relacionados
a las metodologías ágiles.

También agradecemos al Profesor Carlos Fontela (Universidad de Buenos Aires)
por el prólogo y la revisión de los capítulos 6, 7, 8 y 9
así como a la Profesora Elvira Quiroga (Universidad Tecnológica Nacional)
por la revisión de los capítulos 1, 2 y 3.

Quiero agradecer a las siguientes personas:

A Guillermo, por haber confiado en mí y
darme la oportunidad de participar en este proyecto.

A mi mamá, por ese apoyo incondicional en todo lo que emprendo.

A mi familia y amigos, por su ayuda y apoyo.

Ludmila

Gracias a mi esposa Marta por su paciencia e incondicionalidad
y gracias a mis padres por haberme indicado el camino.

Guillermo

Mensaje del Editor

Los conocimientos son esenciales en el desempeño profesional. Sin ellos es imposible lograr las habilidades para competir laboralmente. La universidad o las instituciones de formación para el trabajo ofrecen la oportunidad de adquirir conocimientos que serán aprovechados más adelante en beneficio propio y de la sociedad. El avance de la ciencia y de la técnica hace necesario actualizar continuamente esos conocimientos. Cuando se toma la decisión de embarcarse en una vida profesional, se adquiere un compromiso de por vida: mantenerse al día en los conocimientos del área u oficio que se ha decidido desempeñar. Alfaomega tiene por misión ofrecerles a estudiantes y profesionales conocimientos actualizados dentro de lineamientos pedagógicos que faciliten su utilización y permitan desarrollar las competencias requeridas por una profesión determinada. Alfaomega espera ser su compañera profesional en este viaje de por vida por el mundo del conocimiento.

Alfaomega hace uso de los medios impresos tradicionales en combinación con las tecnologías de la información y las comunicaciones (IT) para facilitar el aprendizaje. Libros como este tienen su complemento en una página Web, en donde el alumno y su profesor encontrarán materiales adicionales, información actualizada, pruebas (test) de autoevaluación, diapositivas y vínculos con otros sitios Web relacionados.

Esta obra contiene numerosos gráficos, cuadros y otros recursos para despertar el interés del estudiante, y facilitarle la comprensión y apropiación del conocimiento.

Cada capítulo se desarrolla con argumentos presentados en forma sencilla y estructurada claramente hacia los objetivos y metas propuestas. Los libros de Alfaomega están diseñados para ser utilizados dentro de los procesos de enseñanza-aprendizaje, y pueden ser usados como textos guía en diversos cursos o como apoyo para reforzar el desarrollo profesional.

Alfaomega espera contribuir así a la formación y el desarrollo de profesionales exitosos para beneficio de la sociedad.

Los autores

Guillermo Pantaleo

Ingeniero en comunicaciones (Universidad Nacional de La Plata).

Investigador y docente en la UBA (Universidad de Buenos Aires).

Ha participado en numerosos seminarios nacionales e internacionales y es autor de numerosas publicaciones científicas.

Actividad docente: profesor de la materia de grado “Calidad en el desarrollo de sistemas” en el Departamento de Computación, Facultad de Ingeniería, Universidad de Buenos Aires. Profesor de la materia de grado “Técnicas de diseño” en el Departamento de Computación, Facultad de Ingeniería, Universidad de Buenos Aires.

Libros publicados: “Calidad en el desarrollo del software”, Alfaomega, 2010.

Ludmila Lis Rinaudo

Ingeniería en informática (Universidad de Buenos Aires).

Desarrolladora y auditora de sistemas en el ámbito privado, en diferentes países.

Actividad docente: profesora de las clases prácticas en la materia de grado “Calidad en el desarrollo de sistemas” en el Departamento de Computación, Facultad de Ingeniería, Universidad de Buenos Aires.

Contenidos

Prólogo	XIX	1.4.2 Formación de un ingeniero de software	11
Prefacio	XXI	1.5 Nuestro caso testigo.....	11
Guía de lectura.....	XXIII	1.5.1 Descripción del problema a resolver.....	11
		1.5.2 Observaciones obtenidas en la empresa cliente	12
		1.5.3 Característica de la empresa desarrolladora	13
		1.6 Conclusión.....	14
		1.7 Contenido de la página web de apoyo.....	14
		1.7.1 Mapa conceptual	14
		1.7.2 Autoevaluaciones	14
		1.7.3 Presentaciones en Power Point (exclusivo para el docente).....	14
		1.8 Referencias	14
Parte I - Introducción		Capítulo 2 – Condiciones de trabajo en el Desarrollo de Software	
Capítulo 1 - Evolución			
1.1 Introducción.....	2	2.1 Proyectos de Software en diferentes ámbitos.....	18
1.2 Los hitos en la evolución histórica del desarrollo de software	2	2.1.1 Desarrollo de sistemas	18
1.2.1 Los sistemas integrados hardware-software	2	2.1.2 Desarrollo de productos	18
1.2.2 Los primeros sistemas de software independientes	2	2.1.3 Desarrollo remoto.....	20
1.2.3 La crisis del software	4	2.1.4 Software factory	22
1.2.4 La creación de la Ingeniería de Software	4	2.1.5 Proyectos de código abierto	23
1.2.5 La aparición de la PC	5	2.1.6 Servicios para clientes internos	25
1.2.6 La interconexión de las PCs	5	2.2 Proyectos de Software y las personas que participan	26
1.2.7 El surgimiento de la Internet	6	2.2.1 Trabajo en equipo	27
1.2.8 La evolución de Internet y las arquitecturas corporativas	6	2.3 Proyectos de software y las condiciones de trabajo.....	28
1.2.9 El futuro cercano.....	6	2.3.1 Condiciones versus procesos.....	28
1.3 Problemas y soluciones.....	7	2.3.2 Condiciones y metodologías	29
1.3.1 Fallas, malas estimaciones y metodologías	7	2.3.3 Manifestación en la gestión de los proyectos.....	29
1.3.2 Herramientas de desarrollo.....	8	2.4 Buenas prácticas	30
1.3.3 El concepto de calidad en el software.....	8	2.4.1 Trabajo iterativo	30
1.3.4 Venta de proyectos de desarrollo de software	8	2.4.2 Comunicación fluida y cerrada	31
1.3.5 Visibilidad de los proyectos de desarrollo	9	2.4.3 Entregables frecuentes.....	31
1.3.6 Calidad versus velocidad de desarrollo ..	9	2.4.4 Reflexión y mejoras	31
1.4 Incumbencias de la nueva ingeniería	10		
1.4.1 Conocimientos de procesos versus conocimientos tecnológicos	10		

2.4.5 Seguridad personal	32
2.4.6 Focalización en tareas de los proyectos	32
2.4.7 Fácil acceso a los clientes	32
2.4.8 Ambiente de desarrollo con integración frecuente.....	32
2.4.9 Colaboración de la organización a partir del compromiso con los proyectos.....	32
2.4.10 Alta visibilidad de los proyectos.....	33
2.4.11 Inducción a los proyectos	33
2.4.12 Asignación de tareas clausurando funcionalidades	33
2.4.13 Arquitecto programador	34
2.4.14 Programación compartida	34
2.5 Conclusión.....	34
2.6 Contenido de la página web de apoyo.....	34
2.6.1 Mapa conceptual	34
2.6.2 Autoevaluaciones	34
2.6.3 Presentaciones en Power Point (exclusivo para el docente).....	34
2.7 Referencias	35

Capítulo 3 – Paradigmas y lenguajes de programación

3.1 Paradigmas de programación	38
3.2 Programación con Procedimientos.....	39
3.2.1 Historia.....	39
3.2.2 Conceptos	39
3.2.3 Su uso en la actualidad.....	39
3.3 Programación Funcional.....	40
3.3.1 Historia.....	40
3.3.2 Conceptos	40
3.3.3 Su uso en la actualidad.....	40
3.4 Programación Lógica.....	40
3.4.1 Historia.....	40
3.4.2 Conceptos	41
3.4.3 Su uso en la actualidad.....	41
3.5 Programación Orientada a Objetos	42
3.5.1 Historia.....	42
3.5.2 Conceptos	42
3.5.3 Su uso en la actualidad.....	42
3.6 Programación Orientada a Aspectos.....	43
3.6.1 Historia.....	43
3.6.2 Conceptos	43
3.6.3 Su uso en la actualidad.....	43

3.7 Programación Genérica.....	43
3.7.1 Historia.....	43
3.7.2 Conceptos	44
3.7.3 Su uso en la actualidad.....	44
3.8 El modelo de múltiple paradigma	45
3.8.1 Historia.....	45
3.8.2 Conceptos	45
3.8.3 Su uso en la actualidad.....	45
3.9 Ejemplo	45
3.9.1 Especificación	45
3.9.2 Solución	46
3.10 Lenguajes de programación.....	46
3.10.1 Clasificación	46
3.11 Métricas del grado de utilización	49
3.12 Conclusiones.....	50
3.13 Contenido de la página web de apoyo.....	51
3.13.1 Mapa conceptual	51
3.13.2 Autoevaluaciones	51
3.13.3 Presentaciones en Power Point (exclusivo para el docente).....	51
3.14 Referencias	51

Parte II – Proceso de desarrollo de Software / Metodologías

Capítulo 4 – Metodologías de desarrollo de software

4.1 Metodologías.....	54
4.1.1 Historia.....	54
4.1.2 Clasificación	54
4.2 Metodologías conducidas por los planes	55
4.2.1 Cascada.....	55
4.2.2 Prototipado	57
4.2.3 DRA (Desarrollo Rápido de Aplicaciones).....	58
4.2.4 Incremental	61
4.2.5 Espiral	62
4.3 Metodologías ágiles.....	64
4.4 Diferencias de enfoque.....	65
4.5 Diferencias de aplicabilidad	68
4.6 Conclusión.....	69
4.7 Contenido de la página web de apoyo.....	70
4.7.1 Mapa conceptual	70
4.7.2 Autoevaluaciones	70

4.7.3 Presentaciones en Power Point (exclusivo para el docente).....	70
4.8 Referencias.....	70

Capítulo 5 – Metodologías conducidas por los planes

5.1 El Proceso Unificado de Desarrollo de Software.....	72
5.2 Conducido por los planes	74
5.3 El flujo de trabajo	75
5.3.1 Las fases y los hitos	75
5.4 Las iteraciones	77
5.5 El organigrama, los roles y actividades	78
5.6 Roles asociados a la organización	80
5.6.1 Roles derivados de la forma de trabajo	80
5.7 Los activos como nexos entre los roles	82
5.7.1 Los hitos y entregables	82
5.8 Planes	86
5.8.1 Preventa.....	86
5.9 Plan de proyecto.....	87
5.10 Otros planes	88
5.10.1 Plan de administración de la configuración	88
5.10.2 Plan de administración de la calidad	88
5.11 Activos de trabajo	89
5.11.1 Especificación de arquitectura de software	89
5.12 Conclusión.....	89
5.13 Contenido de la página web de apoyo.....	90
5.13.1 Mapa conceptual	90
5.13.2 Autoevaluaciones	90
5.13.3 Presentaciones en Power Point (exclusivo para el docente).....	90
5.14 Referencias	90

Capítulo 6 – Metodologías ágiles

6.1 Extreme Programming y Scrum	92
6.1.1 Procesos de gestión y procesos técnicos	92
6.2 Extreme Programming (XP)	93
6.2.1 Valores	93
6.2.2 Principios	93

6.2.3 Prácticas	94
6.2.4 Flujo de trabajo	97
6.3 Scrum.....	98
6.4 Integración Xp – Scrum	100
6.5 Ágiles	101
6.6 El flujo de trabajo	102
6.6.1 Ciclo de vida	102
6.7 Organigrama y seguimiento del proyecto	105
6.8 Conclusión.....	107
6.9 Contenido de la página web de apoyo.....	107
6.9.1 Mapa conceptual	107
6.9.2 Autoevaluaciones	107
6.9.3 Presentaciones en Power Point (exclusivo para el docente).....	107
6.10 Referencias	107

Parte III – Modelos y Documentación

Capítulo 7 – El lenguaje UML y sus modos de utilización

7.1 Introducción.....	110
7.1.1 ¿Qué es UML?.....	110
7.1.2 Preguntas frecuentes	110
7.2 Evolución histórica	111
7.3 ¿Cómo es utilizado en la comunidad informática?.....	113
7.3.1 Dibujo (<i>Sketch</i>)	113
7.3.2 Plan (<i>Blueprint</i>)	113
7.3.3 Lenguaje de programación	113
7.4 Perspectivas.....	114
7.4.1 Problema (Concepción y Análisis).....	114
7.4.2 Solución (Diseño).....	114
7.4.3 Respuestas a las preguntas frecuentes	114
7.4.4 Madurez y nivel de conocimiento de los desarrolladores	117
7.5 Diagramas	117
7.5.1 Diagramas Estáticos.....	117
7.5.2 Diagramas Dinámicos.....	118
7.6 Relación con el proceso de desarrollo	118
7.7 Selección del modo de uso	119
7.8 Contenido de la página web de apoyo.....	119
7.8.1 Mapa conceptual	119
7.8.2 Autoevaluaciones	119

7.8.3 Presentaciones en Power Point (exclusivo para el docente).....	119
7.9 Referencias.....	119

Capítulo 8 – El lenguaje UML - Diagramas estáticos

8.1 Introducción.....	122
8.2 Diagramas	122
8.3 Clases	123
8.4 Paquetes.....	124
8.5 Componentes.....	125
8.6 Instalación	126
8.7 Ejemplo	127
8.8 Conclusión.....	131
8.9 Contenido de la página web de apoyo.....	131
8.9.1 Mapa conceptual	131
8.9.2 Autoevaluaciones	131
8.9.3 Presentaciones en Power Point (exclusivo para el docente).....	131
8.10 Referencias.....	131

Capítulo 9 – El lenguaje UML - Diagramas dinámicos

9.1 Introducción.....	134
9.2 Actividades.....	134
9.3 Casos de uso.....	135
9.4 Interacción.....	137
9.5 Estado.....	138
9.6 Ejemplo	138
9.7 Conclusión.....	141
9.8 Contenido de la página web de apoyo.....	141
9.8.1 Mapa conceptual	141
9.8.2 Autoevaluaciones	141
9.8.3 Presentaciones en Power Point (exclusivo para el docente).....	141
9.9 Referencias.....	141

Parte IV – Relevamiento, modelado y análisis de requerimientos

Capítulo 10 – Relevamiento de requerimientos

10.1 Relevamiento de requerimientos.....	144
---	------------

10.1.1 Objetivos	144
10.1.2 Técnicas de relevamiento	148
10.2 Casos de uso.....	150
10.2.1 Paquetes funcionales	150
10.3 Organización del trabajo con requerimientos.....	161
10.3.1 Planificación	161
10.4 Priorización de requerimientos.....	162
10.5 Conclusión.....	166
10.6 Contenido de la página web de apoyo...	166
10.6.1 Mapa conceptual	166
10.6.2 Autoevaluaciones	166
10.6.3 Presentaciones en Power Point (exclusivo para el docente).....	166
10.7 Referencias	166

Capítulo 11 – Análisis de requerimientos

11.1 Análisis de requerimientos	168
11.1.1 Objetivos	168
11.1.2 Modelo del comportamiento y control	171
11.1.3 Trazabilidad de los requerimientos.....	175
11.2 Conclusión	176
11.3 Contenido de la página web de apoyo...	177
11.3.1 Mapa conceptual	177
11.3.2 Autoevaluaciones	177
11.3.3 Presentaciones en Power Point (exclusivo para el docente).....	177
11.4 Referencias	177

Capítulo 12 – Pruebas a los requerimientos

12.1 Pruebas a los requerimientos	180
12.2 Validación	180
12.2.1 Validación de la aplicación.....	180
12.2.2 Validación del modelo de negocio.....	182
12.3 Verificación	187
12.3.1 Casos de pruebas.....	187
12.3.2 Ciclo de vida y herramientas....	194
12.4 Conclusión	196
12.5 Contenido de la página web de apoyo...	197
12.5.1 Mapa conceptual	197
12.5.2 Autoevaluaciones	197

12.5.3 Presentaciones en Power Point (exclusivo para el docente).....	197
12.6 Referencias	197

Parte V – Arquitectura y diseño de software

Capítulo 13 – Arquitectura de software

13.1 Definiciones	200
13.1.1 El concepto de arquitectura	200
13.1.2 Documentación de la arquitectura	208
13.1.3 Vista estática.....	209
13.2 Selección de componentes	211
13.2.1 Plataformas marco (<i>Framework</i>)	211
13.2.2 Comunicación con sistemas externos.....	213
13.2.3 Atributos de calidad	217
13.2.4 Pruebas a la arquitectura	218
13.2.5 Herramientas de desarrollo.....	218
13.3 Conclusión	219
13.4 Contenido de la página web de apoyo...	219
13.4.1 Mapa conceptual	219
13.4.2 Autoevaluaciones	219
13.4.3 Presentaciones en Power Point (exclusivo para el docente).....	219
13.5 Referencias	219

Capítulo 14 – Diseño de software

14.1 El proceso de diseño	222
14.1.1 Criterios de buen diseño	222
14.1.2 Patrones de diseño	224
14.2 Desarrollo del Caso Testigo.....	224
14.2.1 Presentación	224
14.2.2 Acceso al negocio	226
14.2.3 Negocio	230
14.2.4 Persistencia	230
14.2.5 Conexión con sistemas externos.....	233
14.2.6 Web Service.....	234
14.3 Conclusión	237
14.4 Contenido de la página web de apoyo...	237
14.4.1 Mapa conceptual	237
14.4.2 Autoevaluaciones	237
14.4.3 Presentaciones en Power Point (exclusivo para el docente).....	237
14.5 Referencias	237

Capítulo 15 – Métricas de software

15.1 El proceso de evaluación	240
15.1.1 Construyendo un sistema.....	240
15.1.2 Manteniendo un sistema	240
15.2 Primera aproximación al diseño de un sistema.....	240
15.3 Adentrándonos en los subsistemas.....	245
15.3.1 Métricas	245
15.3.2 Polimétricas.....	248
15.3.3 Desarmonías	250
15.4 Buenas prácticas de codificación.....	251
15.4.1 Estándares	252
15.4.2 Chequeos de estilo	252
15.5 Conclusión	257
15.6 Contenido de la página web de apoyo...	257
15.6.1 Mapa conceptual	257
15.6.2 Autoevaluaciones	257
15.6.3 Presentaciones en Power Point (exclusivo para el docente).....	257
15.7 Referencias	258

Parte VI – Codificación y pruebas

Capítulo 16 – Pruebas de software

16.1 Definiciones.....	260
16.1.1 Objetivos y filosofía.....	261
16.1.2 Principios	262
16.2 Niveles y Tipos de pruebas	264
16.2.1 Métodos de prueba.....	266
16.2.2 Caja Negra	266
16.2.3 Caja Blanca	267
16.3 Estrategias de prueba	270
16.3.1 Propiedades básicas	271
16.3.2 Evolución de las pruebas con el desarrollo.....	271
16.4 Tácticas.....	273
16.5 Claves del cambio en la forma de trabajo.....	275
16.5.1 Razones para automatizar las pruebas	276
16.5.2 Pruebas sistemáticas	278
16.6 Ejemplo.....	281
16.7 Conclusión	283
16.8 Contenido de la página web de apoyo...	284

16.8.1 Mapa conceptual	284
16.8.2 Autoevaluaciones	284
16.8.3 Presentaciones en Power Point (exclusivo para el docente).....	284
16.9 Referencias	284

Capítulo 17 – Proceso de pruebas de software

17.1 El proceso de prueba de software.....	286
17.1.1 Plan de pruebas.....	286
17.1.2 Qué probar	287
17.1.3 Hasta cuándo probar	289
17.2 Automatización de las pruebas.....	291
17.2.1 Por qué automatizar las pruebas.....	291
17.2.2 Qué debería automatizarse	292
17.3 Integración continua	293
17.3.1 Fases del flujo de trabajo	293
17.3.2 Infraestructura	297
17.3.3 Resultados	298
17.4 Conclusión	301
17.5 Contenido de la página web de apoyo... 301	
17.5.1 Mapa conceptual	301
17.5.2 Autoevaluaciones	301
17.5.3 Presentaciones en Power Point (exclusivo para el docente).....	301
17.6 Referencias	301

Parte VII – Gestión de Proyectos

Capítulo 18 – Gestión de proyectos – Estimación y planificación

18.1 Gestión de proyectos de desarrollo de software	304
18.2 Estimación	304
18.2.1 Venta de proyectos de desarrollo de software.....	304
18.2.2 Metodología conducida por los planes.....	306
18.2.3 Metodologías ágiles.....	307
18.3 Planificación	307
18.3.1 Metodología conducida por los planes.....	308
18.3.2 Metodologías ágiles.....	319
18.4 Conclusión.....	322

18.5 Contenido de la página web de apoyo... 322	
18.5.1 Mapa conceptual	322
18.5.2 Autoevaluaciones	322
18.5.3 Presentaciones en Power Point (exclusivo para el docente).....	322
18.6 Referencias.....	322
18.7 Apéndice A – Cuestionario guía de relevamiento preliminar.....	323
18.7.1 Aspectos vinculados al contexto	323
18.7.2 Reingeniería de sistemas (Sistemas a ser reemplazados)	323
18.7.3 Sistemas a desarrollar en proyectos integrales	325

Capítulo 19 – Gestión de proyectos - Monitoreo y control

19.1 Monitoreo y control de proyectos.....	330
19.1.1 Metodologías conducidas por los planes.....	330
19.1.2 Metodologías ágiles	335
19.2 Conclusión.....	337
19.3 Contenido de la página web de apoyo... 338	
19.3.1 Mapa conceptual	338
19.3.2 Autoevaluaciones	338
19.3.3 Presentaciones en Power Point (exclusivo para el docente).....	338
19.4 Referencias.....	338

Capítulo 20 – Gestión del proyecto del caso testigo - Planificación

20.1 Preventa.....	340
20.1.1 Propuesta técnico económica.....	340
20.2 Metodología conducida por los planes .. 345	
20.2.1 Selección de la metodología	345
20.2.2 Estimación y planificación del proyecto	346
20.3 Metodología ágil.....	353
20.3.1 Selección de la metodología	353
20.3.2 Estimación y planificación del proyecto	354
20.4 Conclusión.....	358
20.5 Contenido de la página web de apoyo... 359	
20.5.1 Mapa conceptual	359
20.5.2 Autoevaluaciones	359

20.5.3 Presentaciones en Power Point (exclusivo para el docente).....	359
20.6 Referencias.....	359
20.7 Apéndice - Visión del proyecto	359
20.7.1 Requerimientos del negocio	359
20.7.2 Visión de la solución.....	360
20.7.3 Características salientes.....	361
20.7.4 Alcance y limitaciones	361
20.7.5 Contexto del proyecto.....	362

Capítulo 21 – Gestión del proyecto del caso testigo - Seguimiento

21.1 Metodología conducida por los planes	364
21.1.1 Monitoreo y control del proyecto	364
21.1.2 Cierre del Proyecto.....	370
21.2 Metodología ágil.....	371
21.2.1 Monitoreo y control del proyecto	371
21.2.2 Cierre del Proyecto.....	374
21.3 Conclusión.....	375
21.4 Contenido de la página web de apoyo...	375
21.4.1 Mapa conceptual	375
21.4.2 Autoevaluaciones	375
21.4.3 Presentaciones en Power Point (exclusivo para el docente).....	375
21.5 Referencias.....	376

Parte VIII – Calidad de Software

Capítulo 22 – Calidad de procesos y productos de software

22.1 Calidad de procesos y productos de software	378
22.1.1 El concepto de calidad aplicado al software.....	378
22.1.2 Validación y verificación	379
22.1.3 Ejemplo de uso	380
22.2 Métricas en el software	385
22.2.1 Las medidas como mecanismo de aseguramiento y control de calidad	385
22.2.2 Medidas de la calidad de productos de software.....	387
22.2.3 Medidas de la calidad de procesos de software.....	389

22.3 Conclusiones.....	394
22.4 Contenido de la página web de apoyo	394
22.4.1 Mapa conceptual	394
22.4.2 Autoevaluaciones	394
22.4.3 Presentaciones en Power Point (exclusivo para el docente).....	394
22.5 Referencias.....	395

Capítulo 23 – Organización del área de calidad de una empresa

23.1 Introducción.....	398
23.2 Origen	398
23.2.1 Primeros pasos	398
23.3 Implementación del área de calidad.....	399
23.3.1 Alternativas de organigrama	401
23.3.2 Roles.....	403
23.3.3 Actividades.....	405
23.3.4 Recursos.....	409
23.3.5 Ejemplo de uso	410
23.4 Conclusiones.....	414
23.5 Contenido de la página web de apoyo...	415
23.5.1 Mapa conceptual	415
23.5.2 Autoevaluaciones	415
23.5.3 Presentaciones en Power Point (exclusivo para el docente).....	415
23.6 Referencias.....	415

Capítulo 24 – Planificación de las actividades de control de calidad de un proyecto de desarrollo

24.1 Introducción.....	418
24.2 Plan de calidad.....	420
24.2.1 Ejemplo del caso testigo	422
24.2.2 Objetivos	431
24.2.3 Flujo de trabajo	431
24.3 Conclusiones.....	432
24.4 Contenido de la página web de apoyo	432
24.4.1 Mapa conceptual	432
24.4.2 Autoevaluaciones	432
24.4.3 Presentaciones en Power Point (exclusivo para el docente).....	432
24.5 Referencias.....	432

24.6 Apéndice – Ejemplo del caso testigo	432	25.8 Referencias.....	450
--	-----	-----------------------	-----

Capítulo 25 – Estándares de calidad de software

25.1 Introducción.....	440
25.2 Normas y modelos de referencia	440
25.3 Norma ISO 9001-2000.....	444
25.4 Modelo CMMi.....	446
25.5 Enfoque y alcance de aplicación	448
25.6 Conclusiones.....	448
25.7 Contenido de la página web de apoyo...	450
25.7.1 Mapa conceptual	450
25.7.2 Autoevaluaciones	450
25.7.3 Presentaciones en Power Point (exclusivo para el docente).....	450

Parte IX – Cierre
Capítulo 26 – Conclusiones

26.1 Acerca de la tecnología	452
26.2 Acerca de la forma de trabajo.....	452
26.3 Acerca de la calidad	453
26.4 Acerca de la Ingeniería de Software	453
26.5 Contenido de la página web de apoyo...	454
26.5.1 Mapa conceptual	454
26.5.2 Autoevaluaciones	454
26.5.3 Presentaciones en Power Point (exclusivo para el docente).....	454
26.6 Referencias.....	454

Registro en la Web de apoyo

Para tener acceso al material de la página Web de apoyo del libro:

1. Ir a la página <http://virtual.alfaomega.com.mx>
2. Registrarse como usuario del sitio y propietario del libro.
3. Ingresar al apartado de inscripción de libros y registrar la siguiente clave de acceso
4. Para navegar en la plataforma virtual de recursos del libro, usar los nombres de Usuario y Contraseña definidos en el punto número dos. El acceso a estos recursos es limitado. Si quiere un número extra de accesos, escriba a webmaster@alfaomega.com.mx

Estimado profesor: Si desea acceder a los contenidos exclusivos para docentes, por favor contacte al representante de la editorial que lo suele visitar o escribanos a:

webmaster@alfaomega.com.mx



Contenidos interactivos: indica la presencia de contenidos extra en la Web.

Prólogo

El software nos rodea. Cada vez hay más máquinas de todo tipo controladas por software y más dispositivos que se comportan como computadoras programables. En el transcurso de la década vamos a asistir a la proliferación casi indiscriminada de dispositivos móviles que corren aplicaciones y acceden a Internet, además de un creciente número de artefactos del hogar, medios de transporte y procesos industriales que son controlados por software, en mayor grado día a día. Esto hace que la humanidad dependa cada vez más del software, y que su calidad se torne crítica.

La ingeniería de software es la disciplina que estudia el desarrollo, la operación y el mantenimiento del software. Así, abarca desde el descubrimiento de las necesidades de los clientes y usuarios hasta la construcción, control de calidad y puesta en marcha; desde la administración de proyectos de desarrollo hasta el estudio de métodos de desarrollo. Hoy la ingeniería de software se ve apremiada por los deseos de una inmensa cantidad de usuarios finales que pretenden cada vez menores tiempos de desarrollo, más disponibilidad, mayor adaptabilidad y personalización, menos problemas, más facilidad de aprendizaje, más usabilidad, entre otras tantas características.

Hay algunos libros de ingeniería de software, y hasta una guía de la IEEE que busca definir el cuerpo de conocimiento de la disciplina y de sus áreas de conocimiento. Sin embargo, no ha adquirido aún el nivel de reconocimiento que sí han conseguido otras ingenierías. Las razones para este déficit pueden ser varias: falta de madurez relativa, la percepción de que es una disciplina a la que le cuesta cumplir con plazos y presupuestos y no deja satisfechos a sus clientes, la intangibilidad del producto resultante, entre otras.

Por todo lo dicho, hace ya mucho que quiero escribir un libro de ingeniería de software. No obstante, los años van pasando y la tarea me ha parecido tan titánica que no la he abordado nunca. Guillermo y Ludmila han venido a ocupar ese lugar y saludo este logro que otros no hemos podido alcanzar.

El libro de Guillermo Pantaleo y Ludmila Rinaudo ha conseguido encarar todas las áreas de conocimiento de la ingeniería de software, sin perder la simplicidad y la concisión que lo hacen legible sin tener la sensación de estar leyendo una enciclopedia.

El libro está orientado a estudiantes universitarios. Sin embargo, es útil también para profesionales informáticos, e incluso como texto de divulgación entre otros profesionales que deban interactuar con equipos de desarrollo de software.

Asimismo, no hay una pretensión de cubrir los temas a fondo. De hecho, cada capítulo incluye abundante bibliografía para quien quiera profundizar en cada tema específico. Esto logra, entre otras virtudes, que el libro se pueda leer de corrido de una manera amena y en un tiempo razonable.

Además, los autores han incluido un caso testigo que hace las veces de hilo conductor a través de los distintos capítulos del libro. Este caso testigo ha sido elegido cuidadosamente, siendo suficientemente complejo como para poder cubrir todos los temas y suficientemente simple para no necesitar conocimiento del dominio en cuestión.

Los temas se abordan desde una perspectiva actual, y en este sentido la obra logra estar mucho más al día que otros textos ya arraigados. Sin embargo, no cae en la tentación fácil de estar a la moda en cuanto a técnicas, herramientas y metodologías: tiene la virtud de hacernos ver el panorama completo, de mostrarnos que hay situaciones que requieren un enfoque y que las hay también para otros. En este mismo sentido, es muy interesante la reseña histórica que los autores presentan en el capítulo 1, con el objetivo declarado de no perder la perspectiva cronológica. Igualmente, resulta invaluable la comparación entre métodos ágiles y los más orientados a planes en cuanto a gestión de proyectos, entendiendo que los enfoques son diferentes pero ambos tienen valores que vale la pena analizar.

Además de lo que el lector puede encontrar en otros textos de ingeniería de software, resulta gratamente sorprendente ver el tratamiento de temas que otros autores han abordado menos. Entre ellos el tema de las pruebas, que incluye un tratamiento de la automatización en varios niveles; la comparación entre los tipos de contratos de desarrollo y su relación con el proceso de venta; los temas de desarrollo remoto, a través de software factory y los proyectos de código abierto, tan relegados en la literatura habitual; y hasta el análisis de modelos de calidad con sus dos variantes más usuales: ISO 9001 y CMMi.

Tal vez haya lectores a los que extrañe la ausencia de un capítulo específico de personas y roles u otro de comunicación. Quizá sea mucho pedir para un libro introductorio.

Lo cierto es que Guillermo y Ludmila han decidido compartir con los lectores mucho más que lo que suele encontrarse en otros libros de la disciplina. Toda su experiencia académica y en la industria, sumada a una admirable calidad didáctica expresada por escrito, se ha puesto de relieve en la obra que usted tiene entre sus manos.

Esperemos que haya muchos más libros de ingeniería de software, que nos ayuden a hacer seguir creciendo el cuerpo de conocimiento de esta disciplina tan vigente y con tanto futuro por delante.

Carlos Fontela

Prefacio

Este libro fue pensado como un texto para los estudiantes de las carreras de Ingeniería Informática, Licenciatura en Sistemas e Ingeniería de Software. Hace unos treinta años era más fácil escribir un libro de estas características que hoy día. En las tres últimas décadas ha habido muchos progresos, algunos fundamentales, otros no tanto.

Ante este panorama nos preguntamos si un libro de texto orientado a transmitir conceptos básicos ¿debe incluirlos todos? ¿Con qué profundidad deben ser tratados? ¿Cómo debe relacionar los nuevos conceptos y nuevas técnicas con los anteriores ya superados pero que sirvieron de sustento y fueron generadores de experiencias? ¿Alcanza un paradigma por sí solo para entender los diferentes caminos en la concepción de software o es necesario apelar a varios de ellos para transmitir estas ideas?

Un libro de texto de Ingeniería de Software ¿qué temas debe incluir? ¿Con qué profundidad deben ser tratados ya que cada uno de ellos podría servir de objeto de un libro en sí mismo?

Todas estas preguntas nos las hacemos los autores al momento de emprender este proyecto. Por todas estas razones lo consideramos todo un desafío.

Además de construir un texto que balancee todas las cuestiones mencionadas, nos hemos impuesto los siguientes objetivos:

- El libro debe ser entretenido, de forma que los lectores una vez comenzada su lectura deseen continuarla.
- Debe enseñar a partir de su lectura pero además debe servir como referencia.
- Servir como herramienta de formación de estudiantes de grado, por lo tanto debe ser una puerta de entrada a los diferentes temas.
- Reflejar la tendencia actual del desarrollo de software en el sentido de ser ésta una disciplina sinérgica

en la cual participan diferentes roles que realizan diversas actividades. Por esta razón no puede ser una colección de capítulos sobre los distintos temas, sino que debe tener un hilo conductor que permita ver el proceso de desarrollo desde la realización de las diferentes tareas sin perder de vista el vínculo con el resto. Elegimos como hilo conductor del libro un caso testigo sobre el cual se trabaja en la presentación de cada uno de los temas.

- Tratar conceptos centrales y no debe convertirse en el recetario de la tecnología de moda.

Por todo esto el lector encontrará en estas páginas un conjunto de temas que a criterio de los autores son los fundamentales en la formación de un ingeniero de software y que por lo expresado más arriba son tratados a un nivel que busca balancear la inclusión y el detalle de las exposiciones. Para cada uno de los temas tratados recomendaremos bibliografía en la cual el lector podrá especializarse en los temas de su interés.

La estrategia seguida por los autores es presentar todos los temas según el estado actual de la tecnología (estado del arte en la materia) mencionando los vínculos con métodos y técnicas superadas pero que ayudan a comprender la evolución de los conceptos y el porqué de los cambios experimentados a lo largo de los años. Cada una de las actividades tratadas se ejemplifican sobre un mismo proyecto de desarrollo a lo largo del libro, éste será nuestro hilo conductor.

Presentamos el problema a resolver en el primer capítulo y trabajamos en cada uno de los aspectos del proyecto (Requerimientos, Diseño, Código, Pruebas, Calidad y Gestión) en los capítulos correspondientes donde se tratan dichos temas. De esta forma el lector podrá relacionar de manera fácil las actividades realizadas y los productos generados en cada momento del ciclo de vida del proyecto.

El libro debe leerse dos veces. En una primera lectura se aprenden los conceptos básicos y los objetivos de

cada tarea. En la segunda, el lector debe asumir los diferentes roles y profundizar los detalles fundamentalmente en el análisis del caso testigo presentado.

Este es un libro introductorio y por lo tanto todos los temas son expuestos con un determinado nivel de complejidad, esto es así ya que se busca establecer las bases para que a partir de aquí los lectores se especialicen en los temas de su interés. Sin embargo no es un libro básico ya que a entender de los autores los conceptos expuestos son fundamentales, simples en esencia pero que necesitan experimentación para terminar de ser aprendidos.

Podríamos comparar el aprendizaje del desarrollo de software al de conducir una bicicleta. Si preguntamos a alguien que hace tiempo lo aprendió responderá que es muy fácil, sin embargo si tratamos de explicar cómo hacerlo a partir de la descomposición y

descripción de cada uno de los movimientos y las acciones a realizar resulta complicado. El desarrollo de software tiene un ingrediente adicional: cambia. Las bicicletas desde hace décadas presentan casi las mismas dificultades a un aprendiz. Siguiendo con esta metáfora, podríamos fundamentar las dos lecturas en ella. En la primera buscamos conocer los movimientos que haremos y las acciones asociadas, en la segunda nos despreocupamos de cada una y las desarrollamos como si fuera un único movimiento. Con esto queremos reforzar el concepto de que las diferentes actividades del proceso de desarrollo están intrínsecamente vinculadas y funcionan de forma sinérgica.

Este libro de texto es concebido por profesionales que han compartido y comparten actividades académicas y laborales a partir de la visión compartida que acabamos de expresar.

Los autores

Considero que la educación es crucial para el desarrollo intelectual y la motivación de los seres humanos; incentivada por ese móvil intento brindar un pequeño aporte en el aprendizaje de las personas con inquietudes en el mundo del software. Anhele lograr la motivación de cada uno de los lectores en uno o más de los contenidos del presente libro por medio de la transmisión de sus conceptos troncales. La amplitud de cuestiones y puntos de vista expuestos intentan cubrir gran parte de los contextos planteados en las distintas áreas relacionadas con el desarrollo de software procurando contribuir en cada una de ellas.

Ludmila Rinaudo

Guía de lectura

Este libro consta de ocho partes que agrupan capítulos de temas afines y fuertemente relacionados.

Para aquellos lectores estudiantes - Si bien algunos de los temas se pueden abordar directamente ingresando al libro por el capítulo correspondiente, la sucesión de temas sigue de alguna manera el mismo orden en que son necesarios a lo largo del ciclo de vida de un proyecto de desarrollo. Por esta razón se aconseja su lectura en orden, es decir según la sucesión en que son presentados los diferentes capítulos.

Para lectores profesionales – Aquellos lectores que utilicen el libro como consulta se aconseja una búsqueda en el índice siguiendo las Partes y luego los Capítulos en particular.

Parte I – Introducción: es general e incluye capítulos que tratan aspectos históricos, de condiciones de trabajo y enfoque de las distintas actividades que comprenden la Ingeniería de Software en general y el desarrollo de software en particular.

En el primer capítulo, **Capítulo 1: Evolución**, se presenta una reseña histórica de las diferentes etapas por las que pasó la Ingeniería de Software. Fue incluido porque consideramos fundamental no perder la perspectiva cronológica cuando se leen los capítulos restantes. Se tratan además algunos temas relacionados a la formación de los profesionales de esta área. Por último se presenta el caso testigo sobre el cual se ejemplificarán los diversos temas técnicos y de gestión en cada uno de los capítulos.

En el segundo capítulo, **Capítulo 2: Condiciones de trabajo en el desarrollo de software**, se describen los diferentes escenarios del desarrollo de software y los aspectos claves de éstos que influyen en la forma de trabajo de las distintas actividades. A continuación se tratan las condiciones de trabajo que afectan a los proyectos y personas. Se listan y describen una serie de propiedades deseables que las organizaciones deben establecer para lograr el buen desarrollo pro-

fesional y personal de sus miembros en los trabajos de los proyectos.

El tercer capítulo es una visión general acerca de los diferentes enfoques del desarrollo de software, **Capítulo 3: Paradigmas y lenguajes del desarrollo de software**. Se presenta una revisión histórica y un ejemplo que incluye los paradigmas descriptos. Se analiza una clasificación de los lenguajes de programación asociados a estos paradigmas y se presenta una estadística actualizada de su utilización en los proyectos de desarrollo.

Parte II - Proceso de Desarrollo de Software / Metodologías

En esta parte se presentan los conceptos asociados a las diferentes formas de trabajo o metodologías. Se revisan las más conocidas en el **Capítulo 4: Proceso de desarrollo y metodologías** y se analizan sus aspectos salientes. Además se hace una comparación de las llamadas ágiles y de las conducidas por los planes basada en la diferencia de enfoque y en la aplicabilidad. En los otros dos capítulos, **Capítulo 5: Metodologías conducidas por los planes** y **Capítulo 6: Metodologías ágiles**, se presenta un análisis detallado de los activos, los roles y la dinámica de los dos tipos de metodologías.

Parte III - Modelos y documentación

Para los modelos asociados al desarrollo de software se definió un estándar conocido como UML (Unified Modeling Language), el cual pasó a ser un estándar de hecho a un estándar soportado por una organización y un comité de estudio y actualización. En esta parte se presenta en el **Capítulo 7: El lenguaje UML y sus modos de utilización** las diferentes maneras de hacer uso del lenguaje. En el **Capítulo 8: Diagramas estáticos** se describe la utilización de UML para la construcción de modelos estáticos y en el **Capítulo 9: Diagramas dinámicos** se hace lo propio con los modelos dinámicos. Estos modelos no dependen de ninguna metodología y se

pueden utilizar en la documentación de cualquier tipo de proyecto.

Parte IV - Relevamiento, modelado y análisis de requerimientos

En esta parte se comienza con la presentación de aspectos técnicos como lo son las diferentes tareas del trabajo con los requerimientos de un proyecto de desarrollo de software. Presentamos en el **Capítulo 10: Relevamiento de requerimientos** el desarrollo de las diferentes actividades relacionadas con el relevamiento de los requerimientos. En los capítulos siguientes, **Capítulo 11: Análisis de requerimientos** y **Capítulo 12: Pruebas a los requerimientos**, completamos el tratamiento del trabajo con los requerimientos que va mucho más allá del relevamiento ya que cubre la construcción de modelos de especificación, modelos de dominio y de pruebas. Todos estos temas están ejemplificados utilizando el caso testigo presentado en el Capítulo 1: Evolución.

Parte V - Arquitectura y diseño de software

Los capítulos incluidos en esta parte tratan los temas relacionados a las definiciones técnicas de los proyectos de software. Por un lado en el **Capítulo 13: Arquitectura de software** se describe el proceso de definición de la arquitectura a partir de los requerimientos del proyecto. Luego en el **Capítulo 14: Diseño de software** se desarrollan los temas de diseño de los diferentes módulos con la utilización de criterios de diseños y patrones, por último en el **Capítulo 15: Métricas del diseño** se muestra cómo utilizar métricas aplicadas sobre el diseño y el código a efectos de realizar revisiones guiadas por estos números y así mejorar la calidad de los productos generados. Todos estos temas se ejemplifican sobre el proyecto del caso testigo del libro.

Parte VI - Codificación y pruebas

En esta parte se desarrollan los temas de pruebas de software y el proceso de pruebas. Se presentan estrategias y tácticas de pruebas orientadas a su automatización mediante la utilización de herramien-

tas del tipo xUnit. Esto es presentado en el **Capítulo 16: Pruebas de software**. En el **Capítulo 17: El proceso de prueba de software** se describe el proceso de pruebas, su planificación y la implementación de la forma de trabajo conocida como *Integración continua*. Se ejemplifican estos temas sobre el proyecto del caso testigo del libro.

Parte VII - Gestión de proyectos

Los temas concernientes a la administración de los proyectos son incluidos en esta parte. Separamos la estimación y planificación de los proyectos por un lado y el seguimiento, es decir, monitoreo y control por otro. Además presentamos los temas en capítulos separados a los ejemplos. Estos últimos incluyen la utilización de metodologías ágiles y conducidas por los planes para el proyecto del caso testigo del libro.

- **Capítulo 18: Gestión de proyectos – Estimación y planificación**
- **Capítulo 19: Gestión de proyectos – Monitoreo y control**
- **Capítulo 20: Gestión del proyecto del caso testigo – Planificación**
- **Capítulo 21: Gestión del proyecto del caso testigo - Seguimiento**

Hemos presentado en los ejemplos dos escenarios distintos, uno con un cierre de proyecto exitoso y otro con un cierre fallido para que el lector pueda evaluar ambos contextos y las causas que determinan los dos casos.

Parte VIII - Calidad de software

Un área de la Ingeniería de Software que se ha desarrollado mucho es la de calidad de software. Tratamos estos temas en los capítulos:

- **Capítulo 22: Calidad de procesos y productos de software**

- **Capítulo 23: Organización del área de calidad de una empresa**
- **Capítulo 24: Planificación de las actividades de control de calidad de un proyecto de desarrollo,**
- **Capítulo 25: Estándares de calidad de software.**

En ellos presentamos los conceptos básicos pero además algunos avanzados como las métricas apli-

cadadas a la mejora de procesos, los modelos de desarrollo basados en la calidad del software y la forma en que una empresa debe afrontar su organización para crear productos de calidad en sus proyectos de software.

Parte IX – Cierre

Capítulo 26: Conclusiones

Contenido

1.1 Introducción	2
1.2 Los hitos en la evolución histórica del desarrollo de software	2
1.3 Problemas y soluciones	7
1.4 Incumbencias de la nueva ingeniería	10
1.5 Nuestro caso testigo	11
1.6 Conclusión	14
1.7 Contenido de la página web de apoyo	14
1.8 Referencias	14

Objetivos

- Conocer qué avances se lograron en la Ingeniería de Software
- Vincular estos avances con la época cuándo se lograron
- Relacionarlos con las necesidades que los motivaron
- Conocer la forma en que se trabajó para lograrlos

Competencias específicas

- Reconocer las diferentes tecnologías de desarrollo de software a partir de las necesidades que motivaron su concepción e implementación

1.1 Introducción

Este libro, como fue explicado en el prefacio, tiene como objetivo convertirse en un texto de estudio y referencia para aquellas personas que desarrollarán su actividad profesional en el área de Ingeniería de Software. Si bien es importante una clara y didáctica presentación de cada uno de los temas, es opinión de los autores que es de suma importancia conocer la cronología de los avances de la tecnología y cómo estos fueron dando forma a las técnicas y herramientas que se presentan en cada capítulo. Por esta razón hemos decidido realizar una introducción en la que se relevan cuáles fueron las necesidades en cada momento y cuáles las limitaciones de la época, ya que de esta manera se comprenderá el cabal significado que cada innovación tuvo y cómo se vinculó al resto de los avances que fueron modificando el estado del arte en materia de desarrollo de software. Esta presentación está orientada a establecer el contexto en el cual serán expuestos todos los temas del libro. Para los lectores interesados en una cronología detallada citamos y recomendamos la lectura de la referencia [1].

1.2 Los hitos en la evolución histórica del desarrollo de software

En esta sección mencionaremos los grandes hitos de la evolución del desarrollo de software, las necesidades emergentes de cada uno de esos momentos y los avances tecnológicos que surgieron como solución. Se pone énfasis en la necesidad presente desde el inicio de contar con estrategias y técnicas que permitieran el desarrollo de productos confiables en el marco de proyectos predecibles. Esta necesidad fue la que motivó la creación de la Ingeniería de Software. En la figura 1.1 se muestra un esquema de las diferentes etapas de esta evolución.

1.2.1 Los sistemas integrados hardware-software

En los años posteriores a la Segunda Guerra Mundial (1945 – 1950) el desarrollo del software consiste en la integración del software y el hardware dedicado sobre el cual se ejecuta. Estos desarrollos estaban orientados principalmente a aplicaciones militares tales como el control de tiro y radares. Estos desarrollos, como los de cualquier industria, presentaban fallas que eran corregidas, y luego eran reensayados.

La necesidad de estos días; herramientas de desarrollo. La necesidad en esta época es contar con herramientas de desarrollo, que no existen como se las conoce hoy en día, ya que los sistemas consisten en dispositivos construidos especialmente para cada caso.

1.2.2 Los primeros sistemas de software independientes

Con el advenimiento de los lenguajes de alto nivel y los progresos en el desarrollo de hardware comienza la producción de sistemas independientes del hardware sobre el cual se ejecutan. Estos sistemas fueron comerciales y tuvieron en los de reserva de pasajes a sus precursores. Hacia la década del 50 varias empresas en Estados Unidos producen computadoras de propósito general. Los sistemas de esta etapa resuelven problemas en las áreas de explotación de petróleo, compañías de seguros y ventas en grandes mercados minoristas. IBM construye hacia 1960 el sistema operativo IBM OS/ 360, el desarrollo más sofisticado de la época y el que genera una serie de lecciones aprendidas que son compiladas por Frederick Phillips Brooks, Jr. [2].

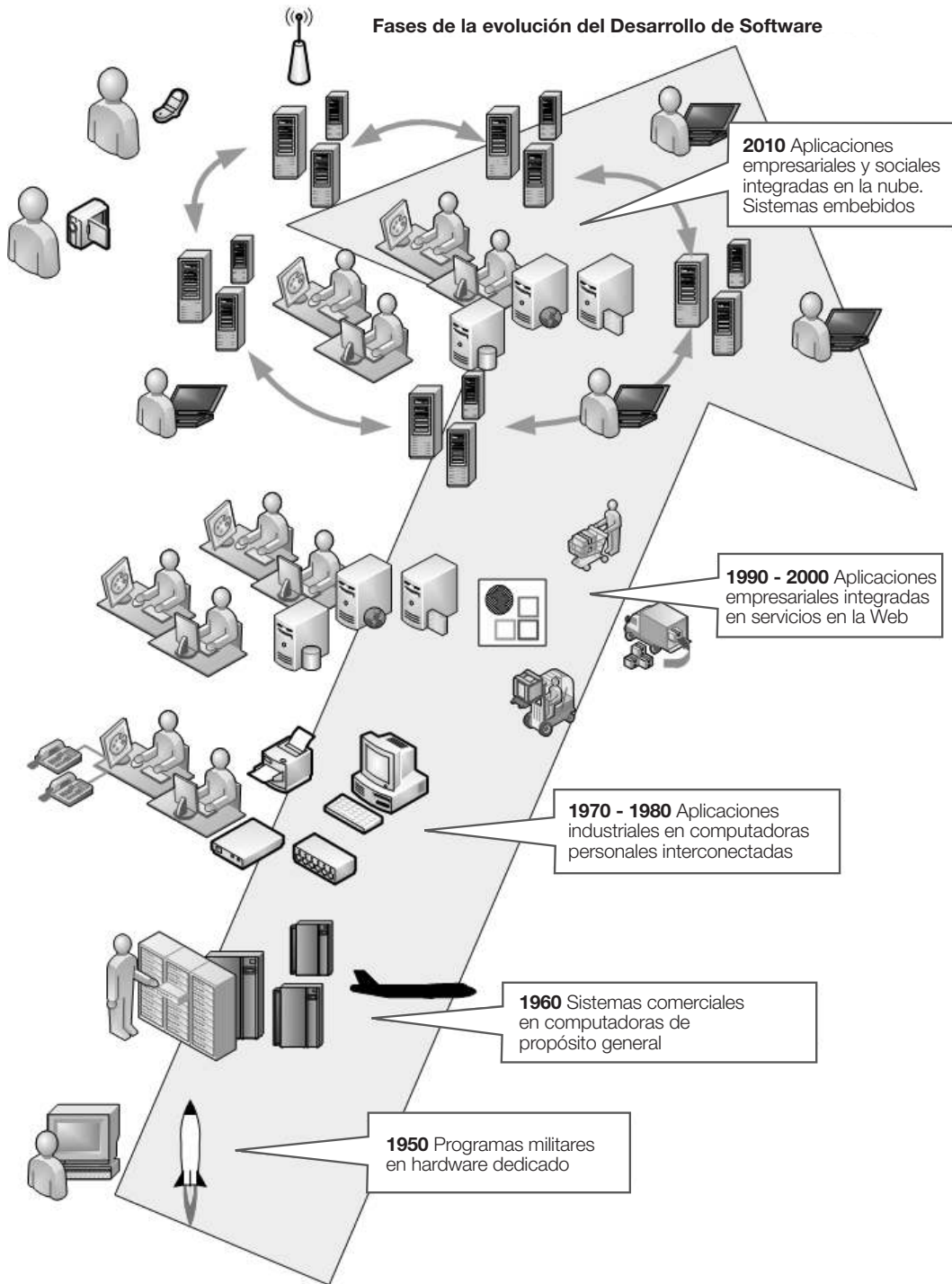


Figura 1.1. Fases de la evolución del desarrollo de software

La necesidad de estos días; contar con **estándares de desarrollo y herramientas de propósito general**. Los lenguajes con los cuales se trabaja son Fortran, Algol y Cobol [3]. Por entonces no se cuenta tampoco con un proceso definido de desarrollo que posibilite una evolución predecible de los proyectos.

1.2.3 La crisis del software

El tamaño de los sistemas, la complejidad de los problemas a resolver, la falta de madurez de las herramientas y de los procesos de desarrollo hacen que se instale en los primeros años de la década de 1970 la llamada *crisis del software*. En esta época, al desarrollar un sistema se prefiere implementar los componentes usando hardware, debido a la escasa confiabilidad que puede ofrecer el código y a las dificultades para depurarlo y ponerlo a punto.

La necesidad de estos días es contar con herramientas robustas y procesos maduros. Aparece el lenguaje de programación Pascal y el concepto de programación estructurada. El lenguaje de programación C se convierte en el estándar de la industria [4]. Se afianza el concepto de módulo en los lenguajes mencionados y se instala el paradigma Desarrollo Estructurado de Sistemas con las metodologías en cascada como forma de trabajo.

1.2.4 La creación de la Ingeniería de Software

Los acontecimientos de esos años generan preocupación y así es como hacia 1980 se concluye que es necesaria la creación de una disciplina que al igual que en otras ramas de la tecnología se ocupe sistemáticamente del diseño y la construcción de productos de software. Se la llama Ingeniería de Software, nombre cuyos primeros usos datan del 7 al 11 de octubre de 1968, cuando diversos referentes participaron en el Comité de Tecnología de la Organización del Tratado del Atlántico Norte (OTAN). En dicha conferencia se acuñan importantes expresiones como **Ingeniería de Software** que trata de agrupar las disciplinas relacionadas a la construcción de productos de software; arquitectura de software, que se refiere a la estructura misma de las aplicaciones y otros conceptos relacionados.

Un hecho asociado al anterior es la creación del Instituto de Ingeniería del Software (SEI) de la Universidad Carnegie Mellon como centro de investigación y desarrollo del gobierno federal de los Estados Unidos de América. Se crea en 1984 con el propósito de promover el avance de la Ingeniería de Software y de las disciplinas relacionadas para asegurar que el desarrollo y la operación de los sistemas se llevan a cabo en forma predecible, mejorando el costo, los plazos de entrega y la calidad. En la Universidad de Carnegie Mellon, reconocida por sus programas de Ciencias Tecnológicas e Ingeniería, que opera a la vanguardia de la innovación técnica, en el año 1991 se publica la primera versión del marco de referencia CMM (*Capability Maturity Model*).

En 1990 IEEE publica el estándar 610.12 [5] que establece el glosario de términos para la Ingeniería del Software, en el que se la define como **la aplicación de un enfoque sistemático, disciplinado y cuantificable para el desarrollo, operación y mantenimiento del software**.

Desde entonces esta disciplina se estudia en las universidades de distintas partes del mundo presentando orientaciones y concepciones diversas acerca de la forma de enseñar los conceptos relacionados pero coincidiendo en sus incumbencias. En la figura 1.2 se muestra una gráfica, generada por el buscador de Google,

de la distribución cronológica de referencias a “Ingeniería de Software”. Es de notar que previamente a la fecha mencionada de acuñación del término este par de palabras se usaban pero no existían como concepto integrado.

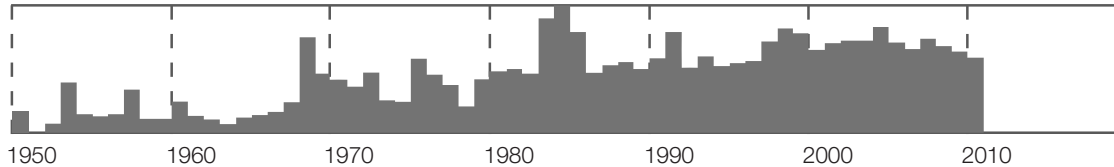


Figura 1.2. Referencias a “Ingeniería de Software” generada por Google

La necesidad de estos días conduce la búsqueda de un paradigma superador que permita a partir del encapsulamiento evitar la propagación de cambios y así resolver los diferentes problemas en forma iterativa. Aparecen lenguajes orientados a objetos como Smalltalk y Ada. Unos años después C++ reemplaza a C como lenguaje estándar en la industria y se trabaja en la concepción de una metodología para este nuevo paradigma [3], [4], [6], [7], [9].

1.2.5 La aparición de la PC

Mientras esto sucede la tecnología sigue avanzando y se cuenta con plataformas de bajo volumen y bajo costo que ofrecen la posibilidad de desarrollar software como una oportunidad de negocio a gran escala. En esta etapa, como consecuencia del avance alcanzado en el desarrollo de microprocesadores, aparecen las primeras computadoras personales. En 1981, IBM presenta su *Personal Computer* (PC), que cambia definitivamente el modo de entender los sistemas de información. Asoma al mundo una empresa llamada Microsoft, productora del sistema operativo para estas plataformas llamado DOS. En la década de 1990 se acentúa el crecimiento de los sistemas con protagonistas como Microsoft, ya convertida en líder mundial, Netscape, Oracle y otros. Se desarrollan sistemas como el sistema operativo NT, uno de los desarrollos más importantes del momento. En esta etapa se consolidan las metodologías de desarrollo de tipo iterativas las cuales van suplantando a las conocidas como cascada. Aparecen algunas metodologías llamadas ágiles así como el concepto de integración continua y se continúa trabajando en IVV (*Independ Verification Validation*) [1].

La necesidad de esos días está dada por la **posibilidad de desarrollar software en estaciones de trabajo de bajo costo para un mercado masivo**. Para esto era necesario contar con formas estándares de especificación y desarrollo. Se define un lenguaje estándar de especificación de software al que se llamó UML (*Unified Modeling Language*) [9].

1.2.6 La interconexión de las PCs

La necesidad de intercambiar información y compartir datos entre computadoras hace que en la década del 70 se desarrollara lo que se llamó LAN (*Local Area Network*). El estándar Ethernet fue desarrollado en Xerox PARC en 1973–1975. A mediados de los 90 aparecen sistemas operativos orientados a funcionar en red (Microsoft NT, Windows for Workgroups). Las primeras instalaciones fueron realizadas en corporaciones y uni-

versidades, y con el paso del tiempo y la baja de los costos en ambientes de pequeñas empresas. Además del acceso a la información en forma distribuida este avance permitió realizar una mejor utilización de los recursos.

1.2.7 El surgimiento de la Internet

A mitad de la década siguiente (1995) se convierte en realidad el proyecto Internet y hacia el comienzo del nuevo siglo ya se cuenta con la red de redes WEB establecida como el contexto en que mayoritariamente el software construido funcionará.

La necesidad de estos días es consolidar un proceso de desarrollo que permita llevar adelante proyectos de gran volumen y métodos de prueba que garanticen la calidad del software. Aparece el lenguaje Java de programación y el UP (*Unified Process*) se promueve como metodología de desarrollo [10].

1.2.8 La evolución de Internet y las arquitecturas corporativas

Al comienzo del nuevo siglo (2000) el escenario establecido para el desarrollo de software está determinado por hardware cada vez más poderoso, software de última generación (lenguajes orientados a objetos, lenguajes deductivos, interpretados, intermedios, multiplataformas, arquitecturas orientadas a servicios), modelos de desarrollo (CMMi, ITIL) [11], [12] y metodologías ágiles (XP, Scrum) [13], [14], [15].

La necesidad de estos días es asegurar la calidad de procesos y productos de software a partir del establecimiento de estándares de desarrollo.

Otra iniciativa importante es SWEBOK (*Software Engineering Body of Knowledge*), un documento creado por la Software Engineering Coordinating Committee, promovido por la IEEE Computer Society, que se define como una guía al conocimiento presente en el área de la Ingeniería de Software. Constituye un avance de importancia en el desarrollo de la profesión porque representa un amplio consenso respecto a los contenidos de la disciplina [29].

Además de marcos de referencia del tipo CMMi (*Capability Maturity Model Integration*) las organizaciones necesitan marcos de referencia para la administración de la tecnología informática por lo que se afianza el modelo ITIL (*Information Technology Infrastructure Library*) [30].

1.2.9 El futuro cercano

Al momento de escribir este libro los desarrollos de software más novedosos se orientan a los sistemas embebidos [16], cloud computing [17] y las redes sociales [18].

La necesidad es contar con herramientas que faciliten el desarrollo y la prueba de estos sistemas y la adecuación de formas de trabajo sistemáticas que permitan llevar adelante en forma predecible los proyectos asociados.

La presentación cronológica anterior muestra que la evolución en el desarrollo de software comenzó con aplicaciones construidas a medida con herramientas propietarias y un número reducido de personas; que arri-

baron hoy día al uso de tecnologías estándares por desarrolladores que trabajan en grupos con varios miembros generando productos que son utilizados por organizaciones y / o comunidades de personas en diferentes ámbitos. **Esta evolución generó y se realizó de la sistematización del conocimiento de esta área de la tecnología, la cual es conocida como Ingeniería de Software.**

1.3 Problemas y soluciones

Si bien esta nueva ingeniería es muy reciente y lleva un corto camino recorrido, los cambios que la han afectado han sido de significativa importancia. Algunos de los problemas mencionados fueron resueltos completamente y otros solo parcialmente. Mencionaremos cuáles son los que aún permanecen abiertos a criterio de los autores a efectos de establecer el contexto dentro del cual serán presentados los temas tratados en el libro.

1.3.1 Fallas, malas estimaciones y metodologías

Los problemas presentes que afectan a los proyectos de desarrollo de software son la falta de predictibilidad en su realización dentro del tiempo y presupuesto planificado. Gran parte de estos casos son generados por errores en las estimaciones de esfuerzo de dichos proyectos o la utilización de algunas metodologías de desarrollo y gestión con criterios equivocados.

El desarrollo de software ha pasado, según lo describimos más arriba, por diferentes etapas en su evolución y la comunidad informática ha capitalizado la experiencia de proyectos cuyo desarrollo no resultó ser como lo planificado. A veces el análisis de estas lecciones aprendidas contribuyó a esta evolución que generó cambios e influyó en el rumbo seguido por esta disciplina. Otras veces las necesidades derivadas de un negocio cada vez más dinámico y con mayor competencia tuvo una influencia negativa. Como mencionamos, algunas de las falencias más notorias fueron la falta de pruebas suficientes, falta de rigor y realismo en las estimaciones de esfuerzo y falta de decisión para cambiar formas de trabajo que ya habían mostrado no ser las adecuadas. Estas falencias tuvieron sus causas en diferentes razones. Por un lado, la visión de que las pruebas son un gasto y que todos los recursos orientados a ellas no son visibles para el cliente fundamenta la primera. Por suerte esta visión ha cambiado y hoy día hay una corriente o escuela de desarrollo, mayoritariamente soportada por las metodologías ágiles, que basa su evolución en las pruebas [19].

Por otro lado, todavía es común realizar estimaciones de esfuerzo sobre escenarios ideales sin afectar a las estimaciones por factores determinantes, como la madurez del proceso de desarrollo definido para el grupo, la capacidad y experiencia de sus miembros, la complejidad y antecedentes en el negocio, la habilidad del trabajo con los requerimientos, el compromiso de los involucrados, etcétera [20].

Por último, la demora en la adaptación de la metodología elegida a la cultura de la organización ocasionada por temor a equivocarse y falta de experiencia hacen que muchas empresas aún hoy día no hayan seleccionado y ajustado una forma de trabajo para desarrollar sus proyectos [20].

Como veremos en las diferentes partes del libro, buena parte de los capítulos trata acerca de las soluciones a los problemas mencionados.

1.3.2 Herramientas de desarrollo

Las *herramientas de desarrollo* son programas de computación utilizados para generar otros programas con su ayuda. Este es un aspecto que en los últimos años se ha vuelto muy importante debido al volumen y la complejidad del software que se desarrolla. Cada vez más la eficiencia depende de las características de estas herramientas. Hoy día las hay de marcas [22] y gratuitas de código abierto [23]. Sin estas, por ejemplo, la instalación de una aplicación Enterprise distribuida se convierte en un conjunto de tareas tediosas, repetitivas y muy propensas a errores. Por esta razón, además de las que implementan ambientes de desarrollo, son muy valoradas aquellas orientadas a la administración de repositorios [24], compilación y empaquetamiento de binarios [25], pruebas sistemáticas y automáticas [26].

En los diferentes capítulos listaremos herramientas con sus características, bondades y modos de uso utilizadas tanto para desarrollar y probar productos así como para gestionar los proyectos.

1.3.3 El concepto de calidad en el software

Otro aspecto importante es el de calidad en el desarrollo de software. En el análisis cronológico presentado en *Calidad en el Desarrollo de Software* [27] se cita que para el año 1984 cuando se fundó el SEI (Software Engineering Institute):

Por ese entonces estaba claro que el desarrollo de software consistía en administrar el proceso de construir un producto o un sistema que cubra las necesidades de los usuarios, probarlo, instalarlo en el ambiente productivo, mantenerlo y hacerlo evolucionar con los cambios del negocio. Por lo tanto un aspecto de la calidad vinculado al software consiste en llevar adelante este proceso de la mejor forma que permita al proyecto asociado terminar en el tiempo planificado y dentro del presupuesto asignado.

Si bien este concepto y las actividades relacionadas se han instalado como parte de las incumbencias de la nueva ingeniería, aún es percibido como un gasto en algunas empresas. No obstante se han logrado importantes acuerdos en la comunidad acerca del aseguramiento de la calidad como factor determinante de los proyectos. Existen estándares de trabajo y modelos asociados que constituyen una referencia en la mejora de procesos asociados al desarrollo de software [27]. En los capítulos de la Parte VIII: Calidad de Software, trataremos la problemática de la calidad de software y los diferentes modelos de referencia utilizados, además de cómo las empresas se benefician cuando adquieren madurez trabajando en estos marcos.

1.3.4 Venta de proyectos de desarrollo de software

La visión de que los proyectos de desarrollo comienzan antes de su contratación ha modificado la venta de estos como productos cerrados. Muchos proyectos en el pasado han fracasado por haber sido acordados con requerimientos, precio y tiempo prefijados a partir de información preliminar escasa. Cuando analicemos la gestión de los proyectos mencionaremos algunos tipos de contratos en los que se privilegian los acuerdos,

la alta visibilidad y la participación de todos los involucrados como forma de trabajo. Analizaremos cómo esto contrasta con los casos de contratos orientados a la protección de los intereses de una parte contra los de la otra. El aspecto de la venta está directamente asociado a las estimaciones de esfuerzo con las cuales se cotizan los proyectos. En los capítulos en los que tratemos los temas de planificación y seguimiento de los proyectos nos referiremos a esta problemática y su solución.

1.3.5 Visibilidad de los proyectos de desarrollo

La difusión de buenas prácticas de desarrollo independientemente de la diversidad de las metodologías enfatiza el alto grado de visibilidad de los proyectos como una propiedad fundamental. De hecho todas ellas promueven la gran participación de los usuarios y clientes en el proceso de desarrollo así como la de todos los involucrados. Esta característica posibilita la toma de decisiones tanto técnicas como de gestión y contribuyen al establecimiento de acuerdos a lo largo de todo el ciclo de vida. Un ejemplo de esto es la utilización de una intranet donde se comparten los activos de los proyectos. En los capítulos en los que tratemos la gestión de los proyectos y dentro del ejemplo del caso testigo del libro mostraremos la importancia de este aspecto.

1.3.6 Calidad versus velocidad de desarrollo

Una forma de trabajo empleada a lo largo del tiempo para hacer predecibles los proyectos fue el establecimiento de estándares. También se promovió la reutilización de componentes. El uso repetitivo de estos, una vez probados, aumentaron la confiabilidad de los productos en los cuales se utilizaban.

De este modo se generaron activos y procedimientos para su construcción en disciplinas como el trabajo con requerimientos, diseño y código. Estos estándares fueron propuestos como una forma de garantizar la calidad de procesos y productos.

Con el tiempo, este aumento en la predictibilidad de los proyectos los hizo más estables. Pero esta estabilidad no era la dinámica apropiada cuando surgió la Internet como plataforma de comunicación, oferta y consumo de servicios. Los tiempos de salida al mercado de los productos se acortaron y la estabilidad mencionada era percibida como un obstáculo; fue así como aparecieron las metodologías ágiles.

Fue necesario ponderar la creatividad y velocidad de adaptación a los nuevos escenarios por sobre el control y la previsibilidad en los proyectos. Este fenómeno de estabilidad de procesos versus creatividad ya se había percibido décadas antes entre las software factory de Japón y la industria del software norteamericana. La calidad de excelencia de la producción oriental se logró a expensas de la ausencia de nuevos productos los cuales eran ofrecidos por las empresas de EE. UU. que trabajaban en un ambiente mucho menos controlado.

Como muchas veces a lo largo de la historia de esta industria se instaló hace algunos años una idea de moda. Esta vez consiste en pensar que cuanto menos estructuradas las empresas sean, mejor estarán posicionadas para enfrentar cambios y ganar nuevos negocios. Sin embargo, como Cosummano [1] muestra en su libro, también están en condiciones mucho más vulnerables para perder todo lo realizado en solo unos meses.

La razón de esto sigue siendo explicada por W. Demming y su frase acerca de cuál es el objetivo de las empresas, perdurar en el tiempo y generar empleo.

El desafío de hoy día con respecto a garantizar la calidad en las organizaciones dedicadas al desarrollo de software es organizarse para que sus proyectos sean ordenados y predecibles; aunque deben a su vez tener la capacidad de dejar de lado estos procesos rápidamente para reorganizarse y adaptarse a los cambios. Por eso es muy importante lograr madurez en estos procesos de manera de seguirlos cómodamente para que no se constituyan en una carga que aumente los costos e impida contar con agilidad para cambiar. Nos dedicaremos a tratar los temas relacionados con los modelos de referencia como CMMi en un capítulo de la última parte del libro.

1.4 Incumbencias de la nueva ingeniería

Existe diversidad de opiniones acerca de las incumbencias de la Ingeniería de Software. Estas diferentes opiniones tienen gran influencia en la definición de las incumbencias mencionadas y aún en el establecimiento de la curricula de la carrera de Ingeniería de Software de las universidades más importantes del mundo. Dentro de esa diversidad hay ofertas como Ingeniería Informática, Ingeniería en Sistemas, Ingeniería en Computación, Ingeniería de Software y algunas otras por el estilo. Una persona que desea obtener un título universitario en el área de desarrollo de software en cuál de estas carreras se inscribiría.

1.4.1 Conocimientos de procesos versus conocimientos tecnológicos

A efectos de hacer una separación simple establecemos las siguientes definiciones [28].

Ciencias de la Computación: esta rama de la ciencia se ocupa de la teoría de autómatas, el diseño de lenguajes y sistemas operativos, análisis de la estructura de datos y algoritmos, entre otros temas.

Ingeniería de Software: es el área donde se desenvuelven los profesionales utilizando matemáticas, física, química y conocimientos tecnológicos para resolver problemas y / o generar productos. Hace seis décadas, como se describió en una sección anterior, cada vez más productos incluyen partes de software. Sin embargo, de la misma manera que los físicos contribuyen a una ciencia que sirve de soporte a la Ingeniería Eléctrica, la ciencia de la computación debe servir de soporte a la Ingeniería de Software y no mezclarse con ella. El objetivo es formar a los ingenieros para que estén capacitados en la **utilización de los conocimientos tecnológicos** generados por la ciencia de la computación **para construir productos de software**.

A la luz de las definiciones anteriores deseamos respondernos entonces algunas preguntas como:

¿Deberían ocupar los ingenieros de software el rol de programadores?

¿Son el diseño y la escritura de código una incumbencia de esta ingeniería, o solo deben ocuparse de la definición de procesos, procedimientos, estándares y gestionar a las organizaciones en relación con estas?

Estas inquietudes surgen debido a la confusión instalada que contribuyó a descuidar en algunos casos la formación de los estudiantes de ciencia de la computación y en otros a difundir la idea de que los ingenieros de software no se ocupan de tareas técnicas. La respuesta es sí, los ingenieros deben programar como parte de sus tareas.

1.4.2 Formación de un ingeniero de software

Hay dos corrientes bien marcadas que poseen visiones diferentes. Por un lado unos afirman que los ingenieros deben formarse en conocimientos “duros” (programación y diseño, por ejemplo) y los conocimientos “blandos” (procesos de desarrollo, gestión de proyectos y gestión de la calidad) se adquieren con el tiempo, la experiencia y la sistematización de conocimientos a posteriori. Tiempo de vida profesional, experiencia adquirida a partir de la participación en varios proyectos y sistematización lograda con la madurez alcanzada al integrar las anteriores.

Por otro lado existe otra corriente de opinión que expresa la necesidad de una formación sistemática temprana en conocimientos “blandos”, que dejan la posibilidad de que los profesionales elijan la orientación de su formación hacia lo técnico o la gestión.

Esta discusión hizo que en muchas ofertas de instituciones universitarias se presenten programas de Ciencias de la Computación bajo el nombre de Ingeniería de Software. Más allá de estas confusiones, la experiencia nos muestra que cuanto mayor es la experiencia de un profesional en el campo de los conocimientos duros, mejor es su desempeño cuando decide orientar su carrera a la administración de procesos y la gestión. Sumado esto al hecho de que un ingeniero de software debe ocuparse de aspectos técnicos como no técnicos, podemos concluir que es importante una formación profesional en Ingeniería de Software abarcadora de todas las actividades que se realizan a lo largo de todo el ciclo de vida de los proyectos, tanto técnicos como de procesos. Parece ser que la madurez obtenida en el desempeño de roles de programación, diseño y prueba de software establecen las bases para un futuro bien cometido como líder de proyecto, gerente de desarrollo y responsable de calidad.

1.5 Nuestro caso testigo

Daremos a este texto un hilo conductor basado en un caso testigo y el trabajo realizado en el proyecto de desarrollo asociado a él a lo largo de los distintos momentos de su ciclo de vida. Ejemplificaremos cada una de las tareas relacionadas al trabajo con requerimientos, arquitectura, diseño y pruebas; así como las actividades de la gestión del proyecto y la administración de su calidad con este mismo caso. Lo hacemos así porque creemos que esta forma de presentación ayudará al lector a relacionar los diferentes temas.

1.5.1 Descripción del problema a resolver

El sistema a desarrollar debe administrar el servicio de transporte de cargas internacionales por barco. La funcionalidad básica es la programación con antelación de solicitudes de las cargas, el seguimiento de las cargas despachadas y la facturación del servicio a los clientes. Actualmente este negocio se administra en forma manual, con el soporte de utilitarios de oficina. El servicio que se ofrece incluye la recepción de los bultos en los depósitos, el transporte hasta el destino y la entrega en los depósitos del destino. Las cargas son clasificadas por tamaño y peso, y deben ser asignadas a contenedores según sean alimentos (refrigerados), tóxicos (aislados) y neutras (estándares). Estas asignaciones se realizan al momento del alta de las cargas. En función del origen y destino se especifica una ruta. Los itinerarios posibles (ruta) para conectar el origen y destino se obtienen de un subsistema tal que dada una especificación de origen/destino y un criterio de

optimización devuelve un itinerario. Si el cliente no seleccionara ningún criterio, el sistema debe asignar a la carga la ruta que lleve al destino correspondiente con transporte programado y que todavía dispone de lugar en cada uno de los viajes. Este subsistema implementará un algoritmo para cada criterio de optimización. Estos criterios son: menor tiempo, mejor clima y menor costo. Fue encargado a una empresa de desarrollo que solo realizó el trabajo de relevamiento y análisis, solo realizaron un diseño muy preliminar y el cliente cuenta con la documentación generada. Se desea aprovechar el trabajo ya realizado. Debe presupuestarse la terminación de dicho subsistema. Cada posible itinerario está compuesto por una o más etapas ordenadas. Cada una posee un puerto de carga y uno de descarga. El viaje entre los puertos de las etapas posee una fecha de partida y de llegada respectivamente, y una tarifa asociada a la carga transportada en dicho viaje. La tarifa final se obtiene como sumatoria de las tarifas por viaje. Por cada día de almacenamiento en los puertos intermedios se paga una tarifa porcentual del viaje correspondiente al trayecto de llegada a dicho puerto. Por cada día de depósito en el puerto de destino se cobra al cliente una tarifa adicional porcentual de la tarifa total al día de arribo. El negocio de transporte está funcionando hace 5 años y es administrado por un sistema propio desarrollado para tal fin. Para implementar su funcionalidad el sistema debe interactuar con un módulo de otro sistema, ya desarrollado, que memoriza las cargas programadas a efectos de conocer al momento de las altas de las solicitudes cuántas cargas de tal tipo ya fueron programadas. Para no exceder la capacidad de la empresa. Además debe interactuar con un sistema externo a efectos de tercerizar las cargas que la empresa no puede transportar. Actualmente se tercerizan del orden del 10% de los servicios de transporte de cargas totales. Ambos sistemas externos resuelven problemas del mismo negocio, sin embargo el modelo de estos es diferente. Cuando las cargas son descargadas en el puerto de destino de la ruta especificada, se le envía al cliente un mail con una notificación, cronología del itinerario y la factura. En el futuro puede ser que se desee enviar alguna otra información a través de este mail. Se desea contar con reportes de seguimiento por fecha de partida, de alta de solicitudes, tipo de carga, carga por puerto y facturación diaria. Al momento de esta especificación no se han terminado de definir todos los reportes. Las terminales de altas de cargas y cada uno de los puntos intermedios de carga, descarga y almacenamiento en depósito tendrán interfaces para los operadores de los puertos intermedios y de destino. En estos puntos de la ruta son registrados todos los movimientos de las cargas (carga, descarga, almacenamiento). Los operadores de venta del servicio de cargas deben poder dar de alta nuevos pedidos de servicios de despacho de cargas y consultar el seguimiento de las cargas despachadas. Los clientes deben poder realizar por la Web el seguimiento de sus cargas. La empresa cuenta con un registro de los clientes de los últimos cuatro años (14000 aproximadamente), los cuales deberían cargarse al nuevo sistema.

Actualmente poseen 20 oficinas en distintas partes del mundo donde se realiza venta de servicios y 200 puestos intermedios de supervisión de cargas distribuidos a lo largo de las rutas utilizadas. Se requiere una prestación de servicio de 7 días x 24 horas x 365. Para clientes frecuentes se cuenta con un programa de beneficios que incluye tarifas reducidas. Las categorías de clientes son tres: clientes que transportan entre 500 kilogramos y una tonelada mensual, clientes entre una y cinco toneladas y clientes que superan las cinco toneladas.

1.5.2 Observaciones obtenidas en la empresa cliente

La empresa tiene una trayectoria de 5 años en el mercado. Está constituida por un directorio de seis personas, que son los accionistas. De ellos uno es el presidente, otro es el gerente de operaciones y otro es el gerente de logística. El presidente se ocupa del marketing y la gestión comercial, posee buen conocimiento del negocio de alto nivel y una muy buena agenda de contactos. Suele comprometerse con objetivos de negocio que en

ocasiones la empresa no está en condiciones de soportar (según miembros de empresas con quienes han trabajado en otros proyectos). El gerente de operaciones (a cargo del área de transporte) es el miembro que conoce el negocio desde la práctica diaria y toda su problemática, pero no conoce bien la estrategia que se planificó para llevarlo adelante. El gerente de logística es el que conoce a los *partners*, conoce los problemas asociados a la logística y tiene una buena visión del negocio en general. Los empleados (usuarios del sistema a desarrollar) tienen una formación media, en general solo conocen el aspecto del negocio en que les toca trabajar. Un supervisor (Juan Rodríguez) es el que mejor conoce los problemas de la operatoria en detalle, tiene buena relación con los directivos y no tan buena con el resto de los usuarios. En la actualidad existen problemas asociados con el seguimiento de las cargas ya que el sistema de las empresas transportadoras con las que se trabaja no posee un seguimiento que pueda ser consultado en forma externa por los clientes y esta se realiza en forma telefónica. Los precios de los servicios de carga de terceros no están disponibles online, por lo cual la solicitud de carga se realiza en dos pasos: uno al contratar el servicio y el otro a las horas, al contar con los valores provistos por las transportadoras. La empresa aseguradora no tiene acceso a los contratos en forma online, por lo cual se demora un mínimo de 24 horas para asegurar las solicitudes registradas. La planificación de las rutas se realiza actualmente en forma manual, por lo que se debe recibir la información de precios y clima antes de poder sugerir alternativas a los clientes. A pesar de estos inconvenientes de operación, algunos de los miembros de la empresa no piensan que un sistema los vaya a resolver. La empresa está detrás de un contrato con una empresa multinacional de equipos de computación por un tiempo de un año. La aseguradora les está imponiendo estándares de calidad del servicio para hacerse cargo de esas cargas que actualmente no cumplen. El sistema en desarrollo debería mejorar sustancialmente este nivel, una característica importante es el seguimiento de las cargas.

La empresa multinacional, de acceder al negocio, exigirá comenzar a operar dentro de seis meses (ya pasaron 15 días). Un comercial de la empresa que trabaja bajo el presidente contactó a la empresa desarrolladora y trabajaron para relevar esta información con el gerente de operaciones y el de logística. También se reunieron con el supervisor.

En los últimos tiempos el gremio de los empleados portuarios, al cual están afiliados los usuarios operadores, mantuvo reclamos salariales por lo que se desarrollaron trabajos a reglamento y hubo algunos días de paro. Actualmente se trabaja con tres empresas externas a las cuales se les tercerizan cargas. Todas poseen sistemas para administrar su negocio. Dos los compraron y tienen un área de sistemas que lo administra. La tercera posee un área de sistemas que desarrolló dicho sistema.

El sistema propio de consultas de volumen de cargas, el cliente lo compró hace tres años y es un producto enlatado.

1.5.3 Característica de la empresa desarrolladora

Este sistema resolverá la problemática de un negocio para el cual existen, además de este, otros clientes potenciales. Ya se han tenido algunas conversaciones y reuniones de trabajo con algunos de ellos. Muy probablemente este sistema se le instalará a distintas empresas de este negocio, y cada una de estas ha especificado diferentes interfaces de usuario para su administración y diferentes formatos de reportes. Algunas de ellas han definido nuevas interfaces para algunas de las funcionalidades ya existentes en el sistema original. Actualmente la empresa cuenta con proyectos que ocupan todos nuestros desarrolladores, podríamos contar con tiempo de un analista para relevar mayor información y trabajar en la propuesta junto a un Líder de Proyec-

to y un Líder técnico. De los proyectos en marcha, el primero en terminar será en un mes (dos desarrolladores, uno senior y uno junior) y el próximo en dos meses (tres desarrolladores, semiseniors y analista). Ambos están a cargo del mismo Líder de Proyecto. Cuentan con un proyecto acordado que comienza en mes y medio y tiene una duración de seis meses, para lo que se necesitan tres desarrolladores, analista y líder de proyecto. Esta empresa posee dos áreas de desarrollo, una que trabaja con metodologías conducidas por los planes y otra que utiliza metodologías ágiles.

La empresa cliente solicitó una propuesta técnico / económica para dentro de diez días a efectos de analizarla en el directorio que incluya una planificación de todos los aspectos del proyecto (gestión y técnicos) donde el aseguramiento de la calidad debe ser uno de los objetivos de la implementación del sistema. En ambas áreas de desarrollo se trabaja con tecnologías .Net y Java. Se han desarrollado sistemas de tipo Enterprise utilizando los siguientes frameworks: Tomcat, Axis2, BPEL, Struts, Hibernate, Spring, VisualStudio 3 y 3.5. En ambas áreas se cuenta con recursos de desarrollo que cubren todos los roles del proceso de desarrollo, tanto técnicos como de gestión. Ambas áreas comparten un conjunto de prácticas de desarrollo entre las que se pueden mencionar: pruebas con herramientas de tipo xUnit, trabajo en forma iterativa, utilización de un servidor de integración continua, estándares de codificación y sesiones de revisión de activos (modelos y código).

1.6 Conclusión

La presentación cronológica anterior muestra que la evolución en el desarrollo de software comenzó con aplicaciones construidas a medida con herramientas propietarias y un número reducido de personas; y que hoy día se arribó al uso de tecnologías estándares por desarrolladores que trabajan en grupos con varios miembros elaborando productos que son utilizados por organizaciones y / o comunidades de personas en diferentes ámbitos. Esta evolución generó y se realimentó de la sistematización del conocimiento de esta área de la tecnología, la cual es conocida como Ingeniería de Software.

1.7 Contenido de la página web de apoyo



1.7.1 Mapa conceptual

1.7.2 Autoevaluaciones

1.7.3 Presentaciones en Power Point (exclusivo para el docente)

1.8 Referencias

- [1] *The Business of Software: What Every Manager, Programmer, and Entrepreneur Must Know to Thrive and Survive in Good Times and Bad*, Michael Cosumano, Reed Business Information - Reed Elsevier Inc, 2004.

- [2] *The Mythical Man-Month: Essays on Software Engineering*, Frederick Brooks, Addison Wesley, 1975.
- [3] *Teoría de Lenguajes*, http://es.wikipedia.org/wiki/Teoría_de_lenguajes_de_programación
- [4] *Historia de los lenguajes de programación*, <http://www.levenez.com/lang/>
- [5] <http://standards.ieee.org/findstds/standard/610.12-1990.html>
- [6] *Metodologías de desarrollo de software*, http://es.wikipedia.org/wiki/Metodología_de_desarrollo_de_software
- [7] *Smalltalk-80, The Interactive Programming Environment*, Adele Goldberg. Addison-Wesley Series in Computer Science, 1983.
- [8] *The design and evolution of C++*, Bjarne Stroustrup, 1994.
- [9] *Unified Modeling Language-User's Guide*, Grady Booch, Jim Rumbaugh and Ivar Jacobson, Addison-Wesley, 1999.
- [10] *Unified Software Development Process*, Ivar Jacobson, Grady Booch and Jim Rumbaugh, Addison-Wesley, 1999.
- [11] *CMMi, Guidelines for Process Integration and Product Improvement*, M. B. Chrissis, M. Konrad, S. Shrum, 2003.
- [12] *Art Of Service - Itil v3 Foundation Complete Certification Kit - 2009 Edition*.
- [13] *Extreme Programming Explained*, Kent Beck, Addison-Wesley, 2000.
- [14] *Agile Development: Lessons Learned From the First Scrum*, Jeff Sutherland, 2004.
- [15] *Agile Project Management with Scrum*, Ken Schwaber, Microsoft Press, January 2004, 163pp, ISBN 0-7356-1993-X.
- [16] *Sistemas embebidos*, http://es.wikipedia.org/wiki/Sistema_embebido
- [17] *Cloud Computing and SOA Convergence in Your Enterprise*, David S. Linthicum, 2009.
- [18] *10 Must-Know Topics For Software Architects In 2009*, <http://hinchcliffe.org/archive/2009/03/17/16712.aspx>
- [19] *Continuos Integration*, Martin Fowler, <http://martinfowler.com/articles/continuousIntegration.html>
- [20] *Software Estimation: Demystifying the Black Art*. Redmond, Wa.: Microsoft Press, 2006.
- [21] *La nueva metodología*, Martin Fowler, <http://martinfowler.com/articles/newMethodology.html>
- [22] *Visual Studio*, Microsoft, <http://msdn.microsoft.com/es-ar/vstudio/default.aspx>
- [23] *Proyectos Eclipse*, <http://www.eclipse.org/>
- [24] *Subversion*, <http://subversion.tigris.org/>
- [25] *Proyecto Apache Maven*, <http://maven.apache.org/>
- [26] *Proyecto Apache Continuum*, <http://continuum.apache.org/>
- [27] *Calidad en el Desarrollo de Software*, Guillermo Pantaleo, Alfaomega, 2011.
- [28] *Software Engineering Programs Are Not Computer Science Programs*, David Lorge Parnas, IEEE Software, Nov, Dec 1999.
- [29] <http://www.computer.org/portal/web/swebok>
- [30] <http://www.itiil-officialsite.com/>

Condiciones de trabajo en
el Desarrollo de Software

Contenido

2.1 Proyectos de Software en diferentes ámbitos	18
2.2 Proyectos de Software y las personas que participan	26
2.3 Proyectos de software y las condiciones de trabajo	28
2.4 Buenas prácticas	30
2.5 Conclusión	34
2.6 Contenido de la página web de apoyo	34
2.7 Referencias	35

Objetivos

- Conocer cuáles son los diferentes escenarios en que se desarrolla software hoy día
- Estudiar cómo influyen éstos en la forma en que se realizan las distintas disciplinas
- Conocer cómo afectan las condiciones de trabajo a los proyectos
- Conocer cómo afectan las condiciones de trabajo a las personas
- Analizar con qué propiedades deberían contar las organizaciones para facilitar el desarrollo de todas las actividades

Competencias específicas

- Analizar los diferentes contextos en que se llevan adelante los proyectos a efectos de tener en cuenta su influencia al momento de planificar y gestionar proyectos de desarrollo

En este capítulo describiremos el contexto en el cual se llevan adelante las tareas de la Ingeniería de Software tanto para las empresas como para las personas que participan de ellas.

Está dedicado a los estudiantes sin experiencia laboral y orientado a su inserción en el ambiente de trabajo, seguramente alguno de los escenarios descriptos caracterizará su primer trabajo.

Trataremos algunas condiciones que deben reunirse para hacer del desarrollo de software una actividad eficiente y lograr como resultado productos de alta calidad. También mencionaremos distintos ámbitos comunes a esta actividad y analizaremos cómo estos influyen en la realización de las tareas del proceso de desarrollo.

Por ser el desarrollo de software una industria de mano de obra intensiva, emplea una gran cantidad de personas que trabajan en grupos de desarrollo en las organizaciones. Por esto consideramos importante la inclusión de los temas que siguen ya que en toda actividad humana realizada por personas que comparten un ambiente laboral hay condiciones de este trabajo que deben establecerse para garantizar una organización productiva.

2.1 Proyectos de Software en diferentes ámbitos

Además de las típicas empresas dedicadas al desarrollo de productos y / o sistemas existen algunos otros ámbitos en los cuales las actividades realizadas en los proyectos son llevadas adelante en condiciones diferentes. A continuación analizamos estos ámbitos y nos ocupamos de las condiciones de los trabajos para los grupos de desarrollo y las personas que los integran.

2.1.1 Desarrollo de sistemas

Entendemos como *desarrollo de sistemas* al trabajo realizado por empresas que desarrollan software a medida para sus clientes. Algunas veces estas empresas son especialistas en algún mercado vertical, como ser telefonía, control industrial o sistemas embebidos. También pueden contar con especialidades tecnológicas o dominar varias tecnologías. Otras no se especializan en ningún área de negocio y se dedican a desarrollar software de administración de información en general. Los miembros de los grupos de desarrollo se agrupan en torno a un proyecto de acuerdo a sus necesidades y a los roles que cada persona puede desempeñar. Los proyectos son de duración variada y los grupos pueden o no permanecer unidos por más de un proyecto. Cada proyecto es diferente en relación a la naturaleza de los riesgos que se presentan, sean estos generados por la empresa cliente, la complejidad del negocio o la propia empresa. Existe siempre una gran incertidumbre en las estimaciones de alcance y esfuerzo, lo que contribuye a amplificar los riesgos [22].

2.1.2 Desarrollo de productos

Las organizaciones que poseen un producto propio y que lo hacen evolucionar a lo largo del tiempo componen este caso, así como las empresas que se dedican al desarrollo de productos para terceros. Esto hace que los miembros técnicos de la organización posean conocimientos de un segmento de mercado y sean especialistas en él. La dinámica y los riesgos de los proyectos son diferentes al del escenario de desarrollo de

sistemas. Cuando el tiempo pasa y se trabaja en el desarrollo de versiones evolucionadas los riesgos están más controlados y el conocimiento del producto es mayor, lo que contribuye a achicar la incertidumbre que afecta a los proyectos de software.

Se entiende por *Línea de Producto* [21] al conjunto de componentes de software que comparten un conjunto común y administrado de características que satisfacen necesidades específicas de un segmento de mercado y son construidos a partir de un conjunto de activos y de un modo predefinido.

En la figura 2.1 se muestra un esquema de los procesos centrales en la implementación de una política de línea de producto en una organización productora.



Figura 2.1. Esquema de las tres actividades esenciales del trabajo en líneas de productos, adaptada de Northrop [21]

Los procesos de Desarrollo de Activos Básicos y Desarrollo de Producto están íntimamente ligados. La relación entre ellos se basa en la construcción de nuevos productos a partir de componentes genéricos pre-existentes y abstractos que a su vez en ocasiones son extraídos y generalizados desde los nuevos productos construidos. Estos productos comparten una arquitectura y son construidos por componentes básicos. Entre los atributos buscados con esta forma de trabajo se encuentran:

- Robustez
- Menor nivel de fallas
- Menor "Time to Market"
- Bajo costo de producción
- Bajo costo de mantenimiento
- Consistencia de los productos.

Estos dos procesos además son coordinados y gestionados a partir de un tercer proceso de Administración, el cual abarca la provisión de recursos, el planeamiento estratégico y provisión de capacitación, así como la promoción de la política asociada a la línea de producto.

Algunas líneas de productos han sido implementadas con éxito en diferentes mercados tales como los listados a continuación:

- Telefonía móvil
- Sistemas de control de aviones y barcos
- Estaciones de satélites
- Almacenamientos masivos de datos
- Control de máquinas
- Sistemas de facturación
- Sistemas Web de ventas
- Electrodomésticos
- Sistemas financieros y de impuestos
- Dispositivos médicos

Claramente la reutilización de componentes de software es lo que caracteriza a este ámbito y determina una forma de trabajo en la que las tareas de diseño son enfocadas en este requerimiento. En la figura 2.2 se presenta en forma esquemática la diferencia entre el desarrollo de un sistema único y una línea de producto.

2.1.3 Desarrollo remoto

Se conoce como *desarrollo remoto* a aquel que es realizado por un grupo de programadores en un sitio físico diferente al de los clientes y usuarios. Estos últimos son accedidos a través de diferentes medios de comunicación (teléfono, mail, chat, video conferencia y visitas esporádicas).

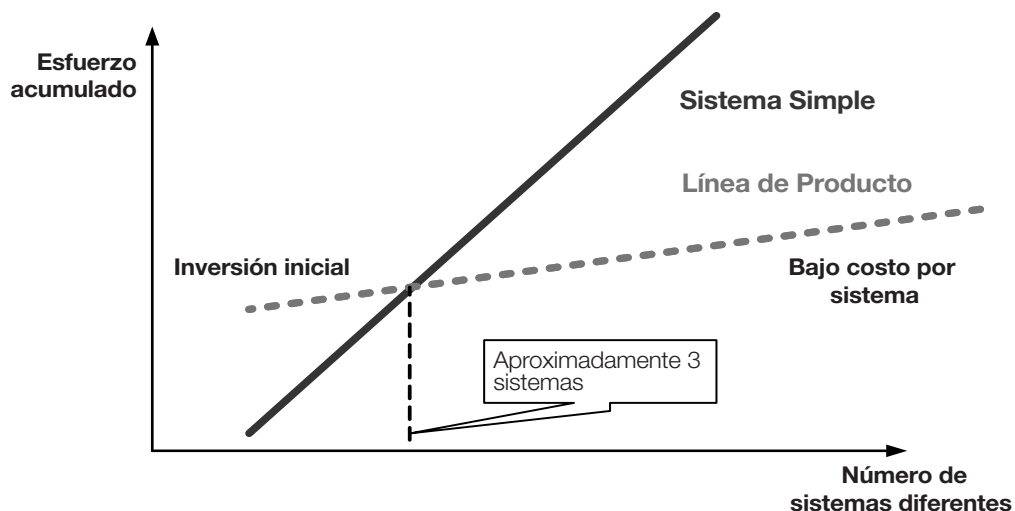


Figura 2.2. Diferencia entre el desarrollo de un sistema simple y una línea de productos

Esta modalidad se difundió con el crecimiento de polos de desarrollo en algunos lugares del mundo debido a la baja de costos asociada a ellos. El caso típico es el fenómeno de la India [10], que a partir del crecimiento de su infraestructura y la calidad de sus recursos humanos hizo que las más importantes empresas consumidoras de software del mundo encargaran desarrollos parciales o totales a centros como Bangalore. Este ámbito está relacionado al de Software Factory ya que, como analizaremos, la forma de trabajo seleccionada para estos proyectos está basada en esa modalidad. Al promediar la década de 1990 ya había experiencia acumulada en este tipo de desarrollo y se conocían algunos de los obstáculos con que se debía trabajar. Estos fueron: la diferencia horaria, de idioma y algunos aspectos culturales.

Algunos de los problemas operativos se resolvieron con la utilización de herramientas que facilitaban la realización de las diferentes tareas de un proyecto. Por ejemplo, los repositorios globales compartidos, como Google Docs, que permiten compartir documentación; herramientas que funcionan sobre Internet, como Subversion, que permiten compartir repositorios de código; y el uso de software de comunicaciones, como Spyke, que permiten mantener conferencias entre los miembros de grupos dispersos.

Sin embargo, la experiencia indica que el mayor problema no se plantea por estas dificultades operativas sino por las diferencias culturales. De qué manera afecta este ítem al desarrollo de proyectos con grupos remotos, cómo puede definirse la visión de un proyecto y realizar el seguimiento conjuntamente con personas que hablan otro idioma, piensan, razonan y actúan según otros valores. En la figura 2.3 se esquematiza el trabajo remoto. En estudios realizados [15] se evidenciaron las siguientes dimensiones de la cultura como influyentes:

- Reverenciar jerarquías
- Individualismo versus colectivismo
- Enfoque en las tareas o en las relaciones
- Evitar riesgos
- Visión a corto o largo plazo
- Espacio
- Bienes materiales
- Amistad
- Tiempo
- Compromiso

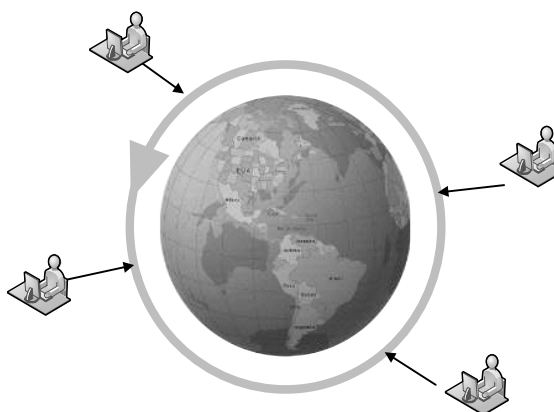


Figura 2.3. Representación esquemática del trabajo remoto

Estas dimensiones determinan culturas de bajo y alto contexto. Las de bajo contexto requieren conversaciones explícitas con gran cantidad de palabras. Las de alto, conversaciones con conceptos implícitos y menos palabras. En las culturas de alto contexto las relaciones son más importantes que las tareas, a la inversa en las de bajo contexto. Las culturas occidentales son de bajo contexto, las orientales de alto. Entonces, más allá del idioma, ¿cómo deben comunicarse dos profesionales del software para que esto no se convierta en un obstáculo? Además, estas dimensiones también afectan el trabajo en grupo, en cómo estos se organizan y cómo desarrollan sus actividades. A la hora de organizar proyectos remotos con grupos cuyos miembros pertenecen a diferentes culturas deben ser tenidas en cuenta todas estas cuestiones [16].

2.1.4 Software factory

Esta modalidad de desarrollo nació con la idea de evitar los errores que se cometían en el desarrollo alrededor del año 1970 en Japón [9]. Por esos días se cuestionaba la falta de procedimientos definidos que guiaban el desarrollo de las distintas actividades. Estas primeras implementaciones encontraron el ambiente propicio brindado por la disciplina de los ingenieros orientales. Con el tiempo se difundió en otras partes del mundo y alcanzó notoriedad hacia la década de 1990 con los emprendimientos de la India basados en el modelo CMM [10]. La esencia de este escenario es la aplicación al desarrollo de software de prácticas de construcción de productos utilizados con éxito en otras ingenierías. Básicamente metodologías conducidas por los planes y una fuerte disciplina para con los procesos asociados. En este escenario se privilegia la reutilización y el uso de componentes estándares que ensamblados de diferentes formas permiten la construcción eficiente de los diferentes productos que los clientes encargan. Esta forma de organización, que nació como una posible respuesta a la crisis del software, en la década de 1990, sustenta su crecimiento en cuestiones económicas vinculadas a la baja de costos. Su adopción generó distintos resultados. Fue exitosa en aquellos lugares donde la idiosincrasia de la gente tenía rasgos de disciplina más arraigados. En algunos países asiáticos y la India aún se emplea esta forma de organización. La idea original fue la implementación del concepto de fábrica de software como un entorno configurado para soportar el desarrollo rápido de un tipo específico de aplicaciones [17] con lo que se buscó incorporar patrones de industrialización.

El esquema de una fábrica consiste en una plataforma de desarrollo que permite adaptarse a los productos a construir. Este esquema se obtiene a partir del mapeo entre un modelo de desarrollo y la fábrica. Los activos y procesos para construirlos se mapean a microprocesos que determinan la estructura y dinámica de la fábrica.

En la figura 2.4 se representa esquemáticamente este mapeo. Cada tipo de producto determina un mapeo diferente y especializado que requiere la adaptación del modelo general y por lo tanto la configuración de la organización de la fábrica. Los proyectos pueden abarcar el ciclo completo de vida de los productos o solo algunas fases, como por ejemplo diseño, construcción y prueba, desarrollándose los requerimientos y la arquitectura en la organización cliente que es la que conserva el conocimiento del negocio.

Esta organización está basada en la reutilización de componentes y en líneas de productos. Algunos de los mercados que mejor adoptaron este modelo fueron los de la industria de software embebido para automóviles y telecomunicaciones. Para su buen funcionamiento requiere un alto grado de madurez, lo cual no siempre fue alcanzado.

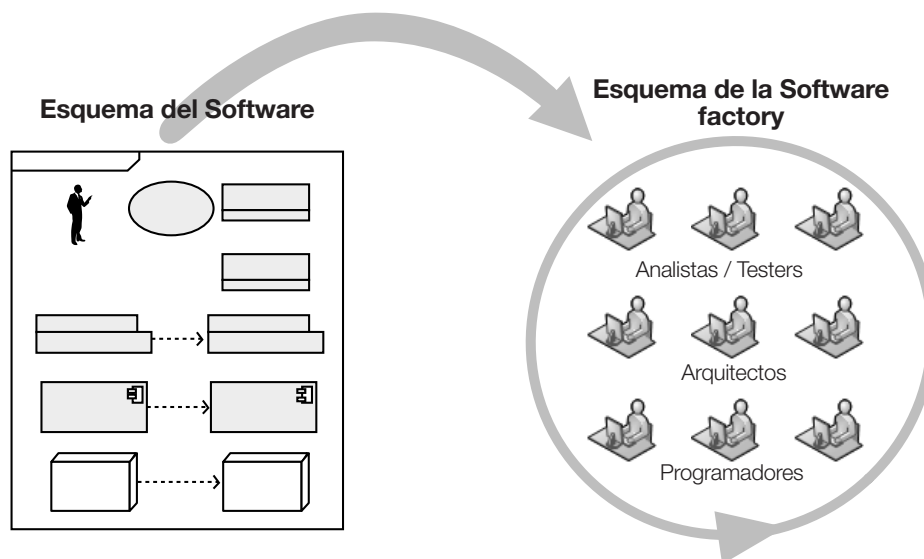


Figura 2.4. Esquema en el cual se basa la organización de una software factory

2.1.5 Proyectos de código abierto

Esta traducción de *open source* significa “software que se puede editar, modificar y redistribuir con libertad”. Este ámbito de desarrollo se estableció como tal hacia fines de la década de 1990. Originalmente surgió con el nombre de *free software*, ya que se deseaba enfatizar el aspecto práctico de compartir el código. Si bien existe cierta ambivalencia entre ambos términos, y aún se reconocen como dos movimientos diferentes, ambos reconocen el mismo conjunto de licencias y mantienen principios equivalentes. En la actualidad el código abierto se utiliza para definir un movimiento nuevo de software, diferente al movimiento del software libre, incompatible con este último desde el punto de vista filosófico y completamente equivalente desde el punto de vista práctico. De hecho, ambos movimientos trabajan juntos en el desarrollo práctico de proyectos [11], [18]. En estos movimientos subyace el concepto de considerar antiético y en contra del sentido común la prohibición de distribución de software propietario.

La esencia de esta forma de organización es que al ser compartido a través de la Web por gran cantidad de desarrolladores, el código mejora con el aporte de todos de una forma más eficiente que en un área de desarrollo cerrada y convencional. Entre los requisitos que debe cumplir el desarrollo de este escenario podemos citar:

- Libre distribución.
- Libre acceso al código fuente.
- Las modificaciones deben ser redistribuidas de igual forma.
- Las licencias igualan los derechos de todos los poseedores del software.

Entre los proyectos más conocidos realizados en estos escenarios podemos citar a los del grupo Jakarta auspiciada por la fundación Apache Software [12], el sistema operativo Linux, navegador Mozilla Firefox, Google Chrome y OpenOffice.org Suite.

En un inicio esta modalidad de desarrollo no tenía un proceso de creación de software asociado y cada miembro se administraba de una forma propia. Sin embargo, hoy día existe una forma común de trabajo.

En la figura 2.5 se presenta un esquema de las fases del desarrollo de un proyecto en esta modalidad.

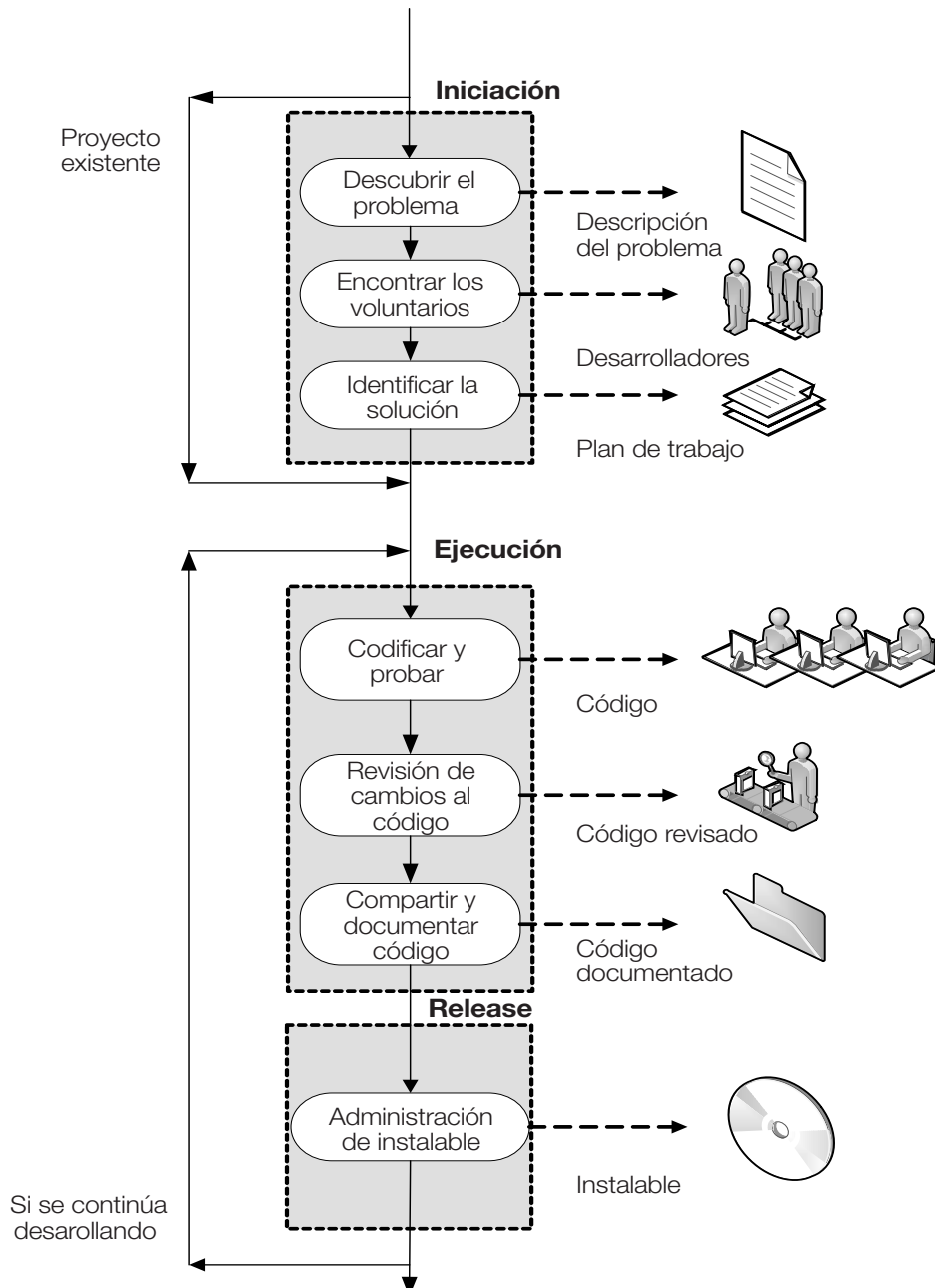


Figura 2.5. Fases del desarrollo de software en el escenario de código abierto

Entre las características salientes de esta forma de desarrollo podemos mencionar:

- Los involucrados en los proyectos son programadores.
- Los proyectos nacen como necesidades de los mismos involucrados.
- No existe un proceso de análisis de negocio y relevamiento de requerimientos, generalmente se comienza con pruebas de concepto y prototipos.
- Se trabaja en forma iterativa.
- Se utilizan muchas de las prácticas de desarrollo surgidas en algunas metodologías ágiles tales como refactorización e integración continua.
- La arquitectura del software evoluciona al igual que el diseño mientras el proyecto avanza, no habiendo una definición inicial.
- La implementación y la integración ha ido mejorando con el uso de herramientas de administración de proyectos como Maven, xUnit y Continuum.
- La comunicación se realiza vía mail, conferencias y herramientas de seguimiento de proyectos del estilo Jira [25] y SourceForge [26].
- No existen limitaciones de tiempo.

En la figura 2.6 se observa la organización de un proyecto de código abierto en lo que respecta a sus roles.

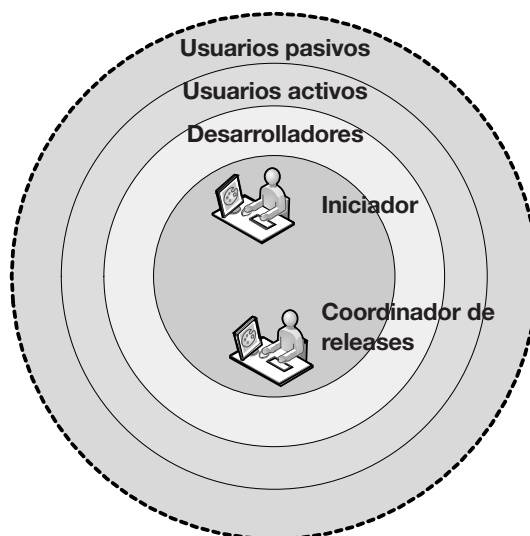


Figura 2.6. Organización de los roles de un proyecto de código abierto, adaptada de Crowston, K. y Howison, J. [19]

2.1.6 Servicios para clientes internos

Además de las empresas cuyo negocio es el desarrollo de software asociado a algún producto o de sistemas a medida, un escenario muy común son las organizaciones con áreas de desarrollo de software para la prestación de sus propios servicios. Ejemplos comunes de este son empresas de telecomunicaciones, bancos, hipermercados, entre otras. Hoy en día estos escenarios son los indicados para la adopción de arquitecturas

de tipo SOA (*Service Oriented Architecture*) como analizaremos en un próximo capítulo. En esta modalidad de desarrollo se mezclan las tareas de desarrollo y operativas de los servicios. Esta problemática se constituye en uno de los obstáculos comunes debido a que el conocimiento necesario para realizar tareas de mantenimiento y operación así como los tiempos asignados a estas actividades son diferentes a los de un proyecto de desarrollo. En general, las organizaciones que desarrollan software en este escenario además administran la tecnología informática (IT) donde el software es un componente más. En estos casos la integración de los diferentes componentes conforman los servicios de IT que se presta al negocio en cuestión. Un modelo orientado a organizar la gestión de estos ámbitos es ITIL (*Information Technology Infrastructure Library*) [20] donde los servicios en su evolución pasan por las siguientes fases:

- **Estrategia:** dentro de las actividades de administración de portafolio, capacidad, continuidad del negocio y proveedores se planifican los proyectos donde están incluidos los componentes de software.
- **Diseño:** esta fase incluye las actividades de diseño, construcción y pruebas en ambientes de prueba. El trabajo es focalizado en el nivel de prestación de servicio acordado.
- **Transición:** esta fase incluye la administración de la configuración y las pruebas en ambientes productivos así como el proceso de instalación de los componentes de software.
- **Operación:** esta fase se ocupa de la administración de incidentes y problemas que alimentan el registro que luego dará lugar a proyectos de modificación o mantenimiento de los componentes de software. Incluye también la administración de las aplicaciones de software.
- **Mejora continua:** esta fase se ocupa de la planificación de mejoras a partir del relevamiento y análisis de métricas.

De la misma forma que los ámbitos analizados determinan y condicionan el funcionamiento de las organizaciones, hay otros aspectos que lo hacen con sus miembros. A continuación trataremos la problemática de las personas que cumplen los diferentes roles en los grupos de trabajo. Estos temas son transversales a todos los ámbitos analizados y constituyen un aspecto organizativo muchas veces ausente en las empresas.

2.2 Proyectos de Software y las personas que participan

La Ingeniería de Software comprende un conjunto de actividades que en su mayoría son de mano de obra intensiva, es decir realizada por personas. La mayor parte de ellas se realizan en grupos organizados de trabajo especializado, razón por la cual se ven enfatizados y cobran gran importancia los aspectos relacionados al llamado *factor humano* [8], [23]. Los grupos de trabajo están compuestos por personas que deben comunicarse todo el tiempo para lograr acuerdos que posibiliten lograr los objetivos de los proyectos. Basados en esta premisa, en las secciones que siguen se presentan una serie de condiciones recomendadas de trabajo y buenas prácticas recogidas de la experiencia. Estas son transversales a las metodologías y los tipos de organización aunque su establecimiento e implementación dependen del contexto.

Los integrantes de un grupo de desarrollo de software, de la misma manera que las personas que viven en sociedad, tienen necesidades de distinto tipo [1]. No se puede pedir a un individuo que se desarrolle como persona

si sus necesidades básicas de alimento y abrigo no están satisfechas. No se le puede pedir a una persona que se desarrolle como persona plena en algún ámbito de la sociedad si sus necesidades de seguridad no están satisfechas, si sus necesidades de estima por parte de sus semejantes y su autoestima no son satisfechas. Si trazamos un paralelo entre la realización como ser humano de una persona y el desarrollo exitoso de un proyecto, veremos que existen necesidades de distinto nivel que deben ser atendidas. En la figura 2.7 se representa a la derecha una adaptación de este concepto tomado de Steve McConnell [1] y a la izquierda, a modo de comparación, lo que llamaremos *condiciones* como aspectos básicos y *procesos* como aspectos de mayor nivel en lo que hace a la organización de una empresa productora de software. Steve McConnell [1] define a las necesidades como:

Necesidades de supervivencia: proyecto sano (no cancelado, con miembros estables y ambiente adecuado).

Necesidades de seguridad: compromiso personal con los tiempos y requerimientos.

Pertenencia y autoestima: aporte a la dinámica del grupo de desarrollo.

Autoestima: sentimiento de productividad, asumir la importancia del proyecto.

Desarrollo profesional: desarrollo integrado al aprendizaje organizacional.

A continuación analizamos cuáles son las condiciones que una organización debe generar en sus grupos de desarrollo para que estas necesidades sean satisfechas y por lo tanto pueda aspirarse a la implementación de un ambiente de desarrollo que garantice el progreso exitoso de los proyectos.

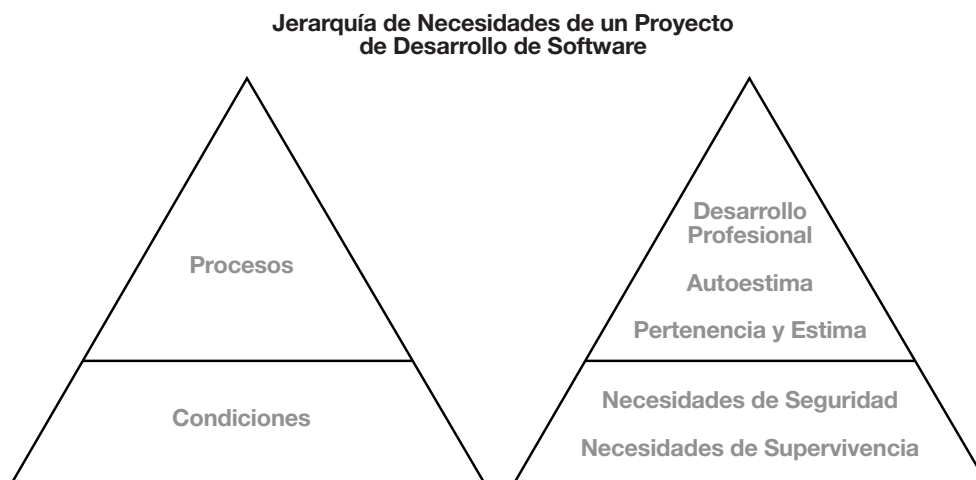


Figura 2.7. Comparación entre las necesidades básicas, las condiciones y los procesos de una organización

2.2.1 Trabajo en equipo

Desde hace algunos años se ha aprendido que la eficiencia en el desarrollo de software no debe medirse por la productividad de los programadores como individuos sino por el trabajo realizado por los grupos que

funcionan como equipos. **Un equipo es un grupo de desarrolladores en el que los proyectos pasados compartidos y las afinidades personales hacen que el trabajo del grupo sea mucho más que la suma del trabajo de sus miembros.** Para que esto suceda deben darse una serie de condiciones entre las cuales podemos mencionar:

- Colaboración entre todos los miembros para lograr los objetivos comunes.
- Complementación de conocimientos y tareas asociadas a los roles.
- Entendimiento logrado a partir de la madurez de la obtención de acuerdos.
- Visión compartida de los proyectos.
- Motivación a partir de un propósito moral.
- Ambiente de desarrollo (lugar físico) compartido.
- Identidad de grupo fijando sus propias reglas de funcionamiento.
- Excelencia técnica a partir de promover el conocimiento compartido.
- Disponibilidad de herramientas para trabajo colaborativo.

Las metodologías ágiles se han preocupado por estas condiciones y las han incluido como uno de sus componentes de importancia al definir su forma de trabajo en relación con estas condiciones [13].

El logro de constituir un equipo de desarrollo es una de las capitalizaciones más preciadas. Para obtenerla y gozar de los beneficios de contar con equipos de desarrollo una de las estrategias a seguir es la conservación de los grupos unidos más allá de los proyectos. Estos aspectos vinculados al factor humano relacionado a la organización fueron estudiados y pueden leerse en detalle en Alistair Cockburn [14].

2.3 Proyectos de software y las condiciones de trabajo

Cuando una empresa trabaja en su organización siempre debe ocuparse de la definición, el acuerdo, la publicación, inducción e institucionalización de los procesos que harán posible su funcionamiento como tal. Para que estas definiciones se realicen en un ámbito en el que los miembros desarrollen sus actividades motivados y colaborando, es decir alineados con los objetivos planteados, es muy importante que previamente se haya trabajado en asegurar un conjunto de condiciones [14] que sienten las bases para el desarrollo profesional y faciliten el desarrollo humano de los miembros en relación con la actividad de la empresa.

2.3.1 Condiciones versus procesos

Las primeras son las más importantes y garantizan una alta probabilidad de un buen proceso de desarrollo. Sin embargo, el seguimiento de los procesos no garantiza la obtención de las primeras.

Cualquiera de las metodologías conocidas que lograron instalar estas condiciones en cientos de proyectos relevantes tuvieron éxito. Son la base sobre la que se asienta la realización de cualquier proceso, los cuales son llevados adelante por personas que deben contar con condiciones adecuadas para desarrollarlos. Consideramos los temas tratados en este capítulo de singular importancia ya que es un error común de las empresas organizarse sin tenerlos en cuenta y es además una fuente frecuente de malos desempeños en muchos de los proyectos.

2.3.2 Condiciones y metodologías

Las condiciones que listamos y analizamos a continuación no son propiedad de ninguna metodología en particular, sin embargo las metodologías ágiles son las que se ocuparon mayormente de ellas incluyéndolas en sus definiciones. Algunas de estas basan su esencia en la autonomía de los miembros de los grupos y su mayor decisión en asuntos vinculados al ambiente de trabajo y al contexto del proyecto, cuestión que caracteriza a estas últimas. No obstante, esto, es opinión del autor que deberían procurarse en cualquier grupo de desarrollo independientemente de la forma de trabajo.

Un relevamiento de alrededor de 12 proyectos en 20 años muestra que [8] un aspecto ausente en la elaboración de las distintas metodologías es: ¿Cuáles son las características de las personas que afectan el desarrollo de software y cuáles son sus implicancias en el diseño de las metodologías?

Algunos aspectos de los que las metodologías no se ocupan son:

- Las personas se comunican hablando cara a cara y a partir de diálogos con preguntas y respuestas.
- Las personas no son consistentes en su forma de pensar continuamente en el tiempo.
- Las personas son variables, día a día y de lugar en lugar.
- Las personas generalmente desean sentirse integrados a las empresas, son curiosos, toman iniciativas y están dispuestos a realizar esfuerzos para que los proyectos tengan éxito.
- Las personas necesitan tiempo para pensar y oportunidades para comunicarse.
- Las personas trabajan mejor a partir de ejemplos.
- Las personas prefieren fallar en forma conservativa que tener éxito asumiendo riesgos.
- La gente tiene rechazo a los cambios.
- Personalidades individuales dominan los grupos.
- Carácter personal que afecta la habilidad para desarrollar tareas específicas.

Además, en general las descripciones de las distintas metodologías son:

- Largas y complejas.
- Mezclan descripción de procesos con productos generados.
- No tratan pequeñas reglas que producen grandes efectos.
- No tratan de las personas y su ambiente de trabajo, sino de roles.

Estas observaciones contribuyen a sacar algunas conclusiones acerca de la importancia de la existencia de ciertas condiciones en el ambiente de trabajo que contemplen estos comportamientos. Sin estas condiciones ninguna metodología es asumida como la mejor por todos, ya que están ausentes las bases que aseguran al ser humano desarrollarse como profesional.

2.3.3 Manifestación en la gestión de los proyectos

El planeamiento y seguimiento de los proyectos en distintos niveles de responsabilidad debe ser llevado adelante con distinta información. La dinámica de los procesos en cascada es más lenta que los iterativos, por lo tanto es posible una planificación y un seguimiento con mayor información y en forma más abierta. El plan es único y la gestión, aún de los niveles gerenciales, posee información detallada de la evolución de los proyectos.

En los procesos iterativos, en cada iteración se decide la continuidad del proyecto. La dinámica es más veloz, se realizan todas las tareas para generar un entregable. Esta dinámica es posible si se organizan los grupos de desarrollo, se les otorga independencia a partir de una gestión con información relevante no detallada y se deja al líder del proyecto la conducción con decisión de primera línea.

Sin embargo esto no significa que el líder de proyecto trabaje solo, es necesario un apoyo constante al grupo, con recursos, intervenciones frente a problemas escalados y presencia continua en las reuniones de seguimiento por parte de la gerencia.

Parte de la independencia del grupo de desarrollo es la libertad para fijar las políticas de su funcionamiento. Esto es en gran medida garantizado a partir de facilitar y generar las buenas prácticas que se listan y describen a continuación. La implementación de estas contribuye a establecer condiciones que facilitan el trabajo en los proyectos.

2.4 Buenas prácticas

Algunas de las buenas prácticas propuestas las compartimos con Alistair Cockburn [2] y otras son derivadas de nuestra experiencia.

2.4.1 Trabajo iterativo

Es válida si la organización ha decidido trabajar en forma iterativa. Está compuesta de los siguientes ingredientes:

Cultura organizacional: debe ser consistente la forma iterativa de desarrollo con el modo en que los proyectos son vendidos. No pueden venderse proyectos de alcance, presupuesto y planificación fijos. Los acuerdos deben explicarse y ser entendidos por todos los involucrados. Estos aspectos relacionados a la forma de trabajo deben ser el elemento de cohesión del trabajo compartido por la gerencia, el departamento de ventas, marketing y área de desarrollo.

Grupo de desarrollo: de parte del grupo de desarrollo se espera actitud y compromiso para con el proyecto. Esto se traduce en el líder planeando y siguiendo según los riesgos, analistas y desarrolladores conduciendo su trabajo con los requerimientos, una definición de la arquitectura como basamento y monitoreo de la calidad de procesos y productos en cada momento del proyecto.

Organización cliente: dispuesta a establecer acuerdos centrados en la confianza que genera el conocimiento de la forma de trabajo y el trabajo compartido más que en conflicto de intereses y multas por incumplimientos. Actitud para participar del proceso de desarrollo además de soportar el proyecto.

Si bien son factibles diferentes formas de trabajo con distintas dinámicas, como veremos en los capítulos próximos, el desarrollo de software ha obtenido una madurez que indica a la iterativa como la forma más adecuada. Los ingredientes anteriores son los necesarios para que esta forma de trabajo se convierta en una propiedad deseable.

2.4.2 Comunicación fluida y cerrada

La información debe fluir en el ambiente a través de las conversaciones de los miembros del grupo. Este intercambio trata acerca de temas e intereses comunes. Una condición para lograrlo es una habitación común para el grupo. Esto implica no mezclarlos, por ejemplo, con profesionales de marketing o ventas, los cuales tienen otros tiempos, otros intereses, otras conversaciones. Si bien es fundamental y más fácil de lograr en proyectos chicos, es importante para todo tipo de proyectos y debe facilitarse aunque sea por subgrupos. Las ventajas de esto son:

- Facilita la comunicación entre desarrolladores menos y más experimentados.
- Genera cohesión del grupo.
- Genera identidad del grupo.

Las condiciones para lograrla:

- Habitación propia.
- El grupo distribuye el mobiliario, el lugar es propio.
- Un lugar apartado con pizarrón para reuniones.
- Un lugar apartado para conversaciones entre miembros.

2.4.3 Entregables frecuentes

Los entregables no deben superar el mes entre ellos. Esto presenta las siguientes ventajas:

- Mayor visibilidad del avance del proyecto para la organización lo cual ayuda al grupo a mostrar su trabajo y a la organización a medir el funcionamiento del grupo.
- Gran visibilidad del avance a los clientes lo cual ayuda a medir la dinámica del proyecto y a realizar pruebas conjuntas y obtener realimentación.
- Ayuda a un desarrollo más dinámico y a la posibilidad de rehacer y mejorar a partir de la realimentación obtenida.
- Ayuda al grupo en el aprendizaje del recorrido del camino completo del desarrollo (análisis detallado, diseño detallado, codificación, pruebas e instalación).
- Garantiza la satisfacción y el relajamiento de un objetivo alcanzado para el grupo de desarrollo.
- Provee visibilidad temprana que genera un ambiente de confianza ante la percepción de todos los involucrados del avance del proyecto.
- Los entregables con períodos fijados de tiempo ayudan a medir la eficiencia y velocidad de desarrollo del grupo.

2.4.4 Reflexión y mejoras

La esencia de esta propiedad es reflexionar y mejorar. Una buena forma de hacerlo es a través de reuniones cada 15 días o mensuales orientadas a mejorar prácticas, documentación, estimación de tiempos, manejo de tecnologías, formas de generar test unitarios, mejorar diseños, etc. Para la instrumentación de esta propiedad se hacen revisiones formales e informales y sesiones de lecciones aprendidas. El objetivo es repensar el comportamiento profesional desarrollado con miras a mejorarlo.

2.4.5 Seguridad personal

Para asegurar la seguridad personal es necesario un ámbito abierto donde los miembros expresen opiniones acerca de cronograma, diseño, código y procedimientos. Un ámbito construido a partir del respeto personal, de la honestidad profesional e identidad de grupo. Una clave para la instalación de esta propiedad es el uso de la libertad con responsabilidad.

Ventajas:

- Abundancia de opiniones que enriquecen al grupo.
- Transparencia.
- Reconocimiento de aciertos y errores.
- Visibilidad de fortalezas, limitaciones y debilidades.

2.4.6 Focalización en tareas de los proyectos

Esta propiedad busca fortalecer la dedicación al proyecto que a veces se pierde en organizaciones que distraen a sus miembros en tareas ajenas al proyecto. Los líderes deben ocupar un porcentaje de su tiempo al proyecto, tanto en su gestión como en el acompañamiento de otros roles (ejemplo: análisis, diseño, pruebas). Los líderes técnicos no deben participar en más de un proyecto y eventualmente asesorar en otro. Las tareas fuera del proyecto deben ser programadas para no contribuir a la pérdida del foco en el proyecto a los miembros del grupo de desarrollo.

2.4.7 Fácil acceso a los clientes

La necesidad de la incorporación del cliente al proyecto tiene dos razones importantes. Una es el rol de usuario experto que define y aclara aspectos del sistema en desarrollo, la otra tiene que ver con la visibilidad de la evolución y las decisiones que se toman con el fin de que haya en todo momento un conocimiento detallado del estado del proyecto. Esto facilita acuerdos y acelera decisiones cuando hay que realizar cambios. La gestión de un proyecto de desarrollo de software es un proceso de cambios continuos determinados por las decisiones que se toman día a día. Esta propiedad busca también facilitar la relación con los involucrados por parte del cliente, aspecto de fundamental importancia.

2.4.8 Ambiente de desarrollo con integración frecuente

Esto permite el desarrollo conducido por pruebas y la integración en forma continua, la fijación de errores previamente a su corrección, la administración consistente del código compartido y el ordenamiento de entregables de todo tipo generados en el proyecto. Esta propiedad busca generar un ambiente de seguridad grupal a partir de la organización de las tareas de generación de código robusto.

2.4.9 Colaboración de la organización a partir del compromiso con los proyectos

El apoyo de la organización a los proyectos debe darse a través de los recursos materiales, en el reconocimiento al trabajo del grupo, al trabajo conjunto del área de ventas, técnica, calidad y recursos humanos

a efectos de mantener un contacto estrecho con los clientes y atender a la resolución de todo tipo de conflictos. Desarrolladores apoyados por su empresa son más productivos y generan productos de mejor calidad.

2.4.10 Alta visibilidad de los proyectos

Los proyectos poseen alta visibilidad cuando todos los involucrados están informados de su estado. Para esto se utilizan diagramas de seguimiento, informes de estado y avance, actualización colaborativa de los indicadores de progreso, etc. Un ambiente de alta visibilidad genera confianza a través de la valoración de una actitud sincera y de la certeza de la no existencia de condiciones ocultas o dobles mensajes.

2.4.11 Inducción a los proyectos

Esta propiedad se consigue con reuniones informales en las cuales los desarrolladores experimentados toman su primer contacto con el sistema a desarrollar. Eric Evans [24] lo llama *el establecimiento del lenguaje ubicuo*. Entre otras cosas se comienza con:

- Modelo de negocio.
- Requerimientos en general.
- Modelo de dominio.
- Alternativas tecnológicas de la solución.
- Formación del grupo de acuerdo a necesidades.
- Metodología a utilizar en el proyecto.
- Identificación de riesgos relacionados al desarrollo.
- Revisión y adaptación del Plan de Proyecto.
- Participación de los desarrolladores en las estimaciones de esfuerzo.

Esta propiedad está orientada a aprender el negocio, y concebir y entender la solución que aportará el proyecto.

2.4.12 Asignación de tareas clausurando funcionalidades

Consiste en organizar al grupo de desarrollo de acuerdo a los niveles de experiencia a efectos de la realización de tareas en forma paralela. Además, en el caso de tratarse de un grupo nuevo, tiene en cuenta cómo serán integrados los miembros con menos conocimiento y experiencia, de qué manera se realizará el *coaching* correspondiente. Esta organización también es importante a la hora de armar el cronograma del proyecto ya que permite la asignación de tareas y su distribución en el tiempo de forma más realista. Las tareas deben ser asignadas de forma que cada módulo (conjunto de funcionalidades) tenga un dueño dentro del grupo de desarrollo. Si la asignación se realiza por área técnica de conocimiento, se garantiza la claridad y el buen diseño, pero genera problemas de integración y falta de compromiso con el sistema en desarrollo desde la perspectiva del problema de negocio a resolver. Las tareas distribuidas por unidades funcionales motivan a los desarrolladores y generan compromiso al sentirse dueños del módulo correspondiente [3].

2.4.13 Arquitecto programador

La asignación de arquitectos con actividades transversales a los proyectos es una práctica seguida en estos días. Las consecuencias de esto son los grupos de trabajo con desarrolladores sin *coaching* y con definiciones que nunca terminan de hacerse propias debido a que no existe el conductor que coopere codo a codo en la codificación siguiendo estas definiciones. También conduce a definiciones técnicas desligadas del negocio, lo que se convierte a la larga en arquitecturas interesantes pero que no son las que el sistema en desarrollo necesita. La clave es reemplazar ese rol por el de líder técnico, que es un desarrollador experimentado con conocimiento y experiencia al frente de las definiciones de la arquitectura, pero que codifica las partes críticas haciendo *coaching* día a día, embebido junto al analista en los detalles funcionales del sistema. Las definiciones de arquitectura deben ser elaboradas, explicadas y transmitidas por los desarrolladores experimentados del grupo. Estableciendo un paralelo con los arquitectos de edificios, la deformación en esa área tiene que ver con los hacedores de planos que nunca visitan la obra y conducen la construcción [6]. Esta propiedad da seguridad y facilita el trabajo de los desarrolladores menos experimentados.

2.4.14 Programación compartida

Se puede ver como una versión relajada de la característica “Pair Programming” de XP [5]. Puede ser realizada en una misma computadora por personas sentadas en lugares adyacentes. Está relacionada con la propiedad Comunicación Fluida y Cerrada. En general permite la realización de *coaching* a los miembros del grupo por parte de los desarrolladores con mayor conocimiento y experiencia. Pueden consistir en tareas compartidas o independientes. Busca salvar los inconvenientes planteados por “Pair Programming” debido a velocidades diferentes de los desarrolladores, diferencias de conocimientos, personalidades, etcétera. Permite el trabajo conjunto con mayor libertad e independencia.

2.5 Conclusión

Hemos presentado y analizado diferentes ámbitos en los que se llevan adelante proyectos de desarrollo de software. Cada uno posee características diferentes que condicionan las actividades de esta ingeniería.

Independientemente de la forma de trabajo utilizada y el modelo de referencia en el cual se elija enmarcar el trabajo de una empresa de desarrollo de software, existe un conjunto de condiciones que deben garantizarse para facilitar el trabajo de sus miembros y el completo logro de los objetivos de dicha empresa. Hemos listado y analizado estas condiciones y un conjunto de buenas prácticas y cómo estas influyen en el proceso de desarrollo. Cuando ejemplifiquemos en los próximos capítulos la realización de las diferentes actividades, lo haremos utilizando estos escenarios y nos referiremos a las influencias mencionadas en este capítulo.

2.6 Contenido de la página web de apoyo



2.6.1 Mapa conceptual

2.6.2 Autoevaluaciones

2.6.3 Presentaciones en Power Point (exclusivo para el docente)

2.7 Referencias

- [1] *Software Project survival guide*, Steve McConnell, Microsoft Press, 1998.
- [2] *Crystal Clear – A Human – Powered Methodology for small teams*, Alistair Cockburn, 2004.
- [3] *The Interaction of Social Issues and Software Architecture architectural decisions*, Alistair Cockburn, 1996, ACM Communications.
- [4] *Unified Software Development Process*, Ivar Jacobson, Grady Booch and Jim Rumbaugh, Addison-Wesley, 1999.
- [5] *Extreme Programming Explained*, Kent Beck, Addison-Wesley, 2000.
- [6] *Who needs an architect?* Martin Fowler, IEEE Software, 2003.
- [7] *The new Methodology*, Martin Fowler, www.martinfowler.com.
- [8] *Characterizing People as Non-Linear, First-Order Components in Software Development*, Alistair Cockburn, 1996.
- [9] *Calidad en el Desarrollo de Software*, Guillermo Pantaleo, Alfaomega, 2011.
- [10] *My Job Went to India: 52 Ways to Save Your Job*, Chad Fowler, 2005.
- [11] *A framework for creating hybrid-open source software communities*, Sharma, S., Sugumaran, V. & Rajagopalan, B. Information, Systems Journal 12 (1), 7-25, (2002).
- [12] *The Apache Software Foundation*, <http://www.apache.org/>.
- [13] *El manifiesto para el desarrollo ágil de software*, <http://agilemanifesto.org/>.
- [14] *Agile Software Development: The Cooperative Game*, Second Edition, Alistair Cockburn, Addison Wesley Professional, 2006.
- [15] *Culture Surprises in Remote Software Development Teams*, Judith S. Olson, Gary M. Olson, Queue - Distributed Development Queue Homepage archive, Volume 1 Issue 9, December/January 2003-2004.
- [16] *Global Software Teams: Collaborating Across Borders and Time Zones*, Carmel E., Upper Saddle River, NJ: Prentice Hall, 1999.
- [17] *Software Factories: Assembling Applications with Patterns, Models, Frameworks, and Tools*, Jack Greenfield, Keith Short, John Wiley and Sons, November 2004.
- [18] *The Cathedral & the Bazaar, Musings on Linux and Open Source by an Accidental Revolutionary*, By Eric S. Raymond, O'Reilly Media Released, October 1999.
- [19] *The Social Structure of Free and Open Source Software Development*, Crowston, K. and Howison, J. (2004). First Monday, 10 (2).
- [20] *Sitio oficial de ITIL*, <http://www.itil-officialsite.com/>.
- [21] *Software Product Line Essentials*, Linda M. Northrop, SEI, <http://www.sei.cmu.edu>.
- [22] *The Business of Software: What Every Manager, Programmer, and Entrepreneur Must Know to Thrive and Survive in Good Times and Bad*, Michael Cosumano, Reed Business Information - Reed Elsevier Inc, 2004.
- [23] *Peopleware: Productive Projects and Teams*, Tom DeMarco, Timothy Lister, Dorset House Publishing, 1987.
- [24] *Domain Driven Design*, Eric Evans, Addison Wesley, 2003.
- [25] <http://www.atlassian.com/es/software/jira/overview>.
- [26] <http://sourceforge.net/>.

Paradigmas y Lenguajes de Programación

Contenido

3.1 Paradigmas de programación	38
3.2 Programación con Procedimientos	39
3.3 Programación Funcional	40
3.4 Programación Lógica	40
3.5 Programación Orientada a Objetos	42
3.6 Programación Orientada a Aspectos	43
3.7 Programación Genérica	43
3.8 El modelo de múltiple paradigma	45
3.9 Ejemplo	45
3.10 Lenguajes de programación	46
3.11 Métricas del grado de utilización	49
3.12 Conclusiones	50
3.13 Contenido de la página web de apoyo	51
3.14 Referencias	51

Objetivos

- Aprender el concepto de paradigma como enfoque del desarrollo
- Aprender la importancia del enfoque desde el cual se analizan los problemas y se les da una solución
- Aprender conceptualmente los distintos paradigmas
- Relacionar los lenguajes de programación con los paradigmas que los concibieron y en los cuales se basan

Competencias específicas

- Seleccionar el enfoque más adecuado para la solución a un problema resuelto con herramientas de software

En este capítulo presentamos el concepto de *paradigma* en el desarrollo de software y su relación con los diferentes lenguajes de programación.

Para esto nos haremos y responderemos las siguientes preguntas:

¿Qué es un paradigma? ¿Cuáles son los paradigmas en el desarrollo de software, cómo evolucionaron a lo largo del tiempo? ¿Cómo se clasifican los lenguajes de programación, cómo se relacionan con los paradigmas, cómo evoluciona su utilización en la industria?

Consideramos de importancia este tema debido a las alternativas de análisis y solución que aporta a un profesional de la Ingeniería de Software frente a un determinado problema, a la posibilidad de pensar un problema desde diferentes perspectivas y seleccionar la tecnología más adecuada para resolverlo.

3.1 Paradigmas de programación

Las tareas y actividades que componen los procesos de la Ingeniería de Software se agrupan en metodologías (formas de trabajo) de gestión y desarrollo de proyectos, que trataremos a lo largo de los capítulos de este libro. Algunas de estas actividades son comunes a todas las formas de trabajo y otras son particulares. Cada proyecto de desarrollo genera un sistema o producto de software que fue concebido e implementado según un enfoque determinado. Llamamos a este enfoque *paradigma*. En este capítulo nos ocuparemos de los distintos paradigmas de programación.

La palabra *paradigma* proviene de la palabra griega *parádeigma*, la cual a su vez es una composición de los vocablos *para* (junto) y *déigma* (modelo). Podríamos decir que su significado aplicado al software define el **enfoque** con que se piensa una solución a un problema que se resolverá con un programa de computación. Esta diferencia de enfoque hizo que a lo largo de los años de evolución del desarrollo de software surgieran distintos lenguajes de programación basados en los diferentes paradigmas que fueron surgiendo. A continuación analizaremos los más importantes y mencionaremos cuál fue la influencia en la evolución de la Ingeniería de Software.

Cuando decimos un *enfoque diferente* nos referimos al punto de vista desde el cual el desarrollador analiza el problema y diseña una solución. Este enfoque determina cómo se usan los mecanismos de descomposición, abstracción y elaboración de jerarquías; los cuales conducen el análisis del problema y el posterior diseño de la solución. Algunas veces este enfoque fue seleccionado y / o concebido con libertad, como parece haber sido el caso de la Programación Orientada a Aspectos. Otras fue condicionado por el estado de avance de la tecnología del momento, como fue la Programación Procedural. Consideramos también que la madurez de algunos conceptos necesitó de años para proveer una visión que permitiera evolucionar de un paradigma a otro. A continuación presentamos los diferentes paradigmas y sus características salientes.

Los *lenguajes de programación* son lenguajes formales que permiten describir, definir y controlar el comportamiento de los procesadores al manipular datos. Estos lenguajes poseen una sintaxis que determina la precisión de sus expresiones y una semántica que define el significado de cada uno de sus elementos. Un *paradigma* es un enfoque que el programador puede adoptar para resolver un problema basado en las construcciones y los métodos de evaluación del lenguaje utilizado. De allí que en este capítulo tratemos una clasificación de los lenguajes de programación según los paradigmas que soportan. En las secciones que siguen

se presentan los paradigmas Imperativo, Funcional, Lógico, Orientación a Objetos, Orientación a Aspectos, Genérico y Multiparadigma. Se lo hace a partir de una reseña histórica, los conceptos centrales, su utilización en la actualidad y un ejemplo conceptual.

3.2 Programación con Procedimientos

3.2.1 Historia

La Programación con Procedimientos (PP) o Imperativa consiste en una secuencia de sentencias que se ejecutan en orden y van cambiando el estado del programa del cual forman parte. Estas sentencias indican cómo realizar una tarea al procesador. Esta estructura de los lenguajes fue fuertemente influenciada y condicionada por el hardware de las computadoras sobre el cual este código se ejecutaba. Este paradigma o modelo de programación fue, de hecho, el estándar de la industria entre los años 1960 y 1980. Ejemplos de este tipo de lenguajes son Fortran [1], Pascal [2] y C [3].

3.2.2 Conceptos

El componente básico de estos lenguajes es la secuencia de sentencias que pueden estar agrupadas formando procedimientos que se invocan. Una característica saliente es el escaso grado de encapsulamiento que presentan. Los sistemas desarrollados con esta tecnología eran pensados como sucesivas descomposiciones de funciones cada vez más simples, flujos y repositorios de datos. Este enfoque fue nombrado *Desarrollo Estructurado de Sistemas* y las metodologías en cascada fueron la forma de trabajo asociada a este modelo.

3.2.3 Su uso en la actualidad

Una separación es necesaria al momento de analizar la vigencia de estos lenguajes en la actualidad. Por un lado, están los lenguajes orientados a implementar sistemas y, por el otro, los orientados a implementar aplicaciones para algún negocio. El lenguaje C de programación se sigue utilizando en la implementación de sistemas.

Ejemplo

Se incluye un código muy simple que permite observar la estructura de procedimientos del lenguaje C. Este programa imprime un mensaje en la consola invocando a la función (procedimiento) **printf** de la librería de C. Aún para introducir una nueva línea (\n) es necesario indicarlo, hecho que ilustra el carácter imperativo.

```
# include <stdio.h>
int main(void)
{
    printf("Hola Mundo\n");
    return 0;
}
```

3.3 Programación Funcional

3.3.1 Historia

Este modelo, Programación Funcional (PF), tiene características declarativas y está basado en el uso de funciones aritméticas. Los programas escritos en un lenguaje funcional están constituidos únicamente por definiciones de funciones, entendiendo estas no como subprogramas clásicos de un lenguaje imperativo, sino como funciones puramente matemáticas, en las que se verifican ciertas propiedades. Ejemplos de este tipo de lenguajes son Haskell [4], Lisp [5], Erlang [6] y Scala [7].

3.3.2 Conceptos

Otras características propias de estos lenguajes son la no existencia de asignaciones de variables y la falta de construcciones estructuradas como la secuencia o la iteración.

3.3.3 Su uso en la actualidad

En la actualidad son todos utilizados y además han influenciado la construcción de otros lenguajes que, si bien no son funcionales, han copiado algunas de sus características como Python y Ruby.

Ejemplo

Se muestra la implementación en Lisp de la función factorial y se indica el modo de utilización.

```
; ---FACTORIAL---  
;Definición matemática  
; Factorial(x) = 1          si x=0 caso base  
;          x*factorial(x-1) si x>0 caso recursivo  
  
;Función factorial hecha con recursividad no final  
(defun factorial (n)  
  (if (= 0 n)  
      1          ; caso base  
      (* n (factorial (- n 1))))) ; caso recursivo  
  
(factorial 4)          ;esto nos devolvería 24=4*3*2*1
```

3.4 Programación Lógica

3.4.1 Historia

Un programa de computación construido en base a este paradigma está constituido por un conjunto de axiomas o reglas que definen relaciones entre objetos. Las construcciones básicas son los términos y las declaraciones que provienen de la lógica. Hay tres declaraciones básicas: hechos, reglas y consultas.

El cálculo de un programa lógico es una deducción de consecuencias del programa [25]. El conjunto de consecuencias constituye su significado. La evolución de un programa es conducido por los objetivos de una consulta. Permiten expresar formalmente problemas complejos y resolverlos mediante la aplicación de las reglas asociadas al dominio. Para esto se especifican hechos, se definen reglas y se realizan consultas. Ejemplo de este modelo es el lenguaje Prolog [5], [8].

3.4.2 Conceptos

Su característica saliente es la declaración en predicados y la inferencia lógica. Un *predicado* es la funcionalidad proporcionada por un procedimiento, el cual está constituido por un conjunto de *cláusulas* acerca de la misma relación. Las cláusulas son unidades de información compuestas por hechos y/o reglas.

3.4.3 Su uso en la actualidad

Actualmente se usan en aplicaciones de inteligencia artificial, como el reconocimiento de lenguaje natural, sistemas expertos y demostración automática de teoremas.

Ejemplo

En este ejemplo de programación lógica se muestra la inferencia realizada a partir de las declaraciones de parentesco entre personas realizadas. Se ejecutan dos consultas a efectos de mostrar los resultados de las inferencias mencionadas.

```
%%
%% declaraciones
%%
padrede('juan', 'maria'). % juan es padre de maria
padrede('pablo', 'juan'). % pablo es padre de juan

% A es hijo de B si B es padre de A
hijode(A,B) :- padrede(B,A).

% A es abuelo de B si A es padre de C y C es padre B
abuelode(A,B) :- padrede(A,C), padrede(C,B).

%%
%% consultas
%%
% ¿pablo es abuelo de maria?
?- abuelode('pablo', 'maria').
yes

% ¿maria es abuela de pablo?
?- abuelode('maria', 'pablo').
No
```

3.5 Programación Orientada a Objetos

3.5.1 Historia

La Programación Orientada a Objetos (POO) surgió como una alternativa a la Programación Procedural y se basa en la descomposición de los problemas a partir de conceptos extraídos del dominio. Estos conceptos a la larga terminarán siendo objetos del modelo asociado al problema. La descomposición de los problemas en forma ortogonal a como lo hace la programación procedural es la diferencia esencial, en lugar de enfocarse en funcionalidades cada vez más simples, lo hace mediante los conceptos mencionados y la interacción entre ellos. Los otros dos mecanismos conductores de este paradigma son la abstracción en la determinación del alcance de los objetos y el establecimiento de jerarquías al generalizar sus comportamientos [26]. Existen varios lenguajes de programación que se basan y soportan este paradigma. Entre ellos Smalltalk [9], Eiffel [10], Ada [11], CLOS [12], Delphi [13], PHP [14], C++ [15], Java [16], C# [17] y otros. Este paradigma surgió con el lenguaje Simula67.

3.5.2 Conceptos

Los conceptos fundamentales son las clases, sus instancias (objetos), el mecanismo de herencia entre clases, el mecanismo de polimorfismo y el encapsulamiento. Es a partir de esta propiedad de encapsulamiento que evolucionó el concepto de arquitectura de software, se logró la inversión en la cadena de dependencia del software y se elaboraron formas de trabajo (metodologías) que permitieron desarrollar software de manera iterativa e incremental. Otros conceptos importantes son la identidad de los objetos dada por el medio con que se accede, su estado dado por el valor de sus atributos y su comportamiento dado por las acciones desarrolladas por sus métodos o funciones.

3.5.3 Su uso en la actualidad

Hasta la década de 1990 el estándar de programación en la industria fue el lenguaje C++. En la actualidad, si bien C++ se sigue utilizando para el desarrollo de sistemas, Java y C# son los lenguajes de mayor uso en el desarrollo de aplicaciones.

Ejemplo

El código Java del ejemplo implementa la impresión de un saludo en la consola a partir de la utilización de la clase Saludo. En el método main se crea un objeto de dicha clase y se imprime el mensaje devuelto por el método getSaludo.

```
public class Saludo {
    private String texto;
    public Saludo(String texto){ this.texto = texto;}
    public String getSaludo(){ return texto;}

    public static void main(String[] args) {
        Saludo saludo = new Saludo("Hola Mundo");
        System.out.println(saludo.getSaludo());
    }
}
```

3.6 Programación Orientada a Aspectos

3.6.1 Historia

La Programación Orientada a Aspectos (POA) permite encarar de manera efectiva los aspectos (*concerns*), transversales, tales como el *logging*, la seguridad, monitoreo de performance o manejo de excepciones, entre otros. A diferencia de las técnicas de desarrollo convencional que “dispersa” la implementación de cada aspecto entre varias clases, la POA los localiza, encapsula y los mantiene separados de las clases que manejan la funcionalidad. Es un paradigma de programación relativamente reciente que plantea separar y encapsular los conceptos de una aplicación y del código transversal que implementa servicios a efectos de evitar la dispersión de este código a lo largo de todo el código de la aplicación. Estos conceptos son clasificados en aspectos y la estrategia es que los componentes que implementan un aspecto no sean visibles desde los otros. De esta forma se logra que el código sea más legible y reutilizable. AspectJ [18] es una extensión Java del proyecto Eclipse para ayudar en el desarrollo orientado a aspectos. También existen implementaciones en Perl, PHP, C# y Python.

3.6.2 Conceptos

Aspect (aspecto) es una funcionalidad transversal (*cross-cutting*) que se implementa de forma modular y separada del resto del sistema. Un ejemplo simple de un aspecto es el *logging* (registro de sucesos) dentro del sistema, ya que necesariamente afecta a todas las partes del sistema que generan un suceso. Este código estaría disperso por el código de los diferentes módulos de negocio y se convertiría así en un obstáculo para su lectura y reutilización.

Joint Point (punto de cruce o de unión) es un punto de ejecución dentro del sistema donde un aspecto puede ser conectado. El código del aspecto es insertado en el flujo de ejecución del código de negocio para añadir su funcionalidad transversal.

Advice (consejo) es la implementación del aspecto, es decir, contiene el código que implementa la nueva funcionalidad transversal. Se insertan en la aplicación en los Puntos de Cruce.

Pointcut (puntos de corte) define los **Advice** que se aplicarán a cada **Joint Point**.

3.6.3 Su uso en la actualidad

Su utilización se ha centrado en la implementación de servicios en frameworks. Ejemplo de ello es la implementación del aspecto de control de transaccionalidad en el framework Spring [19].

3.7 Programación Genérica

3.7.1 Historia

La Programación Genérica (PG) fue popularizada por la implementación de la librería STL (*Standard Template Library*) de C++ [20]. Consiste en la separación de los datos de los algoritmos con el objetivo de lograr

desacoplar los tipos de los datos, los contenedores de datos y los algoritmos que sobre ellos operan. Estos conceptos fueron desarrollados por Stepanov y Lee. Su implementación se basa en los *templates*, mecanismo que permite generalizar un comportamiento independiente del tipo de datos. En las implementaciones de Java 1.5 y 1.6 fue incorporado este mecanismo bajo el nombre de *Generics*.

3.7.2 Conceptos

Los componentes claves son los contenedores, iteradores y algoritmos. Los *contenedores* son las estructuras que contienen los datos, los *iteradores* sirven como nexo entre los datos y los *algoritmos* permiten el acceso a los datos para su procesamiento independientemente del contenedor de donde proceden. Otros componentes importantes son los Functión Objects que son utilizados como parámetros de métodos, Allocators para diferentes esquemas de memoria, Locales y Facets orientados a la internacionalización, IOStreams para la administración de entradas y salidas de texto y binarios, estructuras matriciales complejas y técnicas de diseño como Concepts, Traits, Adaptors y Archetypes.

Si bien su implementación es en C++, un lenguaje orientado a objetos implementado en base a clases, el paradigma es diferente. La especificación se realiza en términos funcionales. Por ejemplo, un contenedor debe proveer cierto iterador, un iterador debe proveer ciertas operaciones, un algoritmo debe operar sobre tal rango de datos.

Las diferencias importantes con la POO son: las clases no siempre están relacionadas, no todos los componentes son clases ya que existen las funciones que implementan algoritmos y las interfaces no siempre reflejan todas las operaciones posibles de realizar sobre un componente genérico.

3.7.3 Su uso en la actualidad

Un ejemplo de su extendida utilización es el sitio Boost, donde se concentra la mayor actividad de esta área de la tecnología y se publican algunas de las librerías más utilizadas [21].

Ejemplo

Este ejemplo de programación genérica está escrito en C++. Se observa la definición de listas de diferentes tipos a partir de la misma clase genérica List. En este caso la implementación es a través del mecanismo de template que provee el lenguaje.

```
template<typename T>
class List
{
    /* métodos y atributos de la clase */
};

List<Animal> lista_de_animales;
List<Auto> lista_de_autos;
```

3.8 El modelo de múltiple paradigma

3.8.1 Historia

Las aplicaciones y sistemas que se desarrollan presentan un conjunto de problemas de diferentes características. El enfoque con que se analizan y resuelven no siempre está alineado a un único paradigma. La POO fue adoptada masivamente en el desarrollo de aplicaciones, sin embargo cuando los modelos de los dominios presentan reglas de negocio complejas es más claramente analizado y más fácilmente resuelto a partir del uso de la PL. Esto conduce a la necesidad de contar con herramientas (lenguajes) que provean propiedades de múltiple paradigma y permitan enfocar el desarrollo del software de cada módulo de un sistema según el paradigma más apropiado. La idea es que un lenguaje de programación pueda soportar distintos paradigmas de programación con el objetivo de que el programador utilice el más conveniente a la hora de resolver un problema. Ningún paradigma es capaz de resolver todos los problemas de forma sencilla y eficiente, por lo tanto es útil poder elegir entre distintos estilos de programación dependiendo del tipo de problema. Como fue visto, C++ es uno de estos lenguajes, Scala es otro de reciente concepción que provee PO más PF.

3.8.2 Conceptos

Lenguajes como C++ permiten combinar el paradigma procedural, el orientado a objetos y la programación genérica [23]. Scala provee PO más PF, que permite obtener concurrencia de forma eficiente.

3.8.3 Su uso en la actualidad

Programación genérica utilizando POO en C++, Scala en el desarrollo de aplicaciones de negocios diversos potenciado por su interoperabilidad con Java y .Net.

3.9 Ejemplo

3.9.1 Especificación

El sistema del ejemplo está asociado con una empresa que administra proyectos. Por cada proyecto deben calcularse y asociarse los pagos, los cuales dependen del monto total del proyecto. Si el monto supera una determinada cantidad, el proyecto debe ser abonado por el cliente para quien se desarrolló en tres pagos, si el monto está entre dos cantidades los pagos serán dos y si no uno solo. Siempre un pago se realiza cuando finaliza el proyecto y de corresponder más de un pago, el primero se realiza al inicio. Además se aplican descuentos cuando el monto supera una determinada cantidad. Se desean obtener reportes ordenados por distintos criterios a definir y los proyectos y sus pagos deben persistirse en una base de datos relacional. El sistema debe soportar multiusuarios en forma concurrente. El problema a resolver es de gran volumen, y este aspecto solo es una parte de este.

3.9.2 Solución

Este sistema podría ser implementado utilizando, por ejemplo, POO en C++ o Java. Si se elige este enfoque se dispondrá de un modelo de dominio con toda la riqueza que brindan los objetos, sin embargo los servicios de administración de la persistencia y de las transacciones deberán ser invocados desde alguna parte del modelo de dominio. Estas líneas de código estarán dispersas por diferentes clases degradando la legibilidad del código al introducir cuestiones que son ajenas al dominio. Si en cambio elegimos implementar estos requerimientos utilizando POA, el código de los servicios mencionados residirá en sus módulos correspondientes. De esta manera el código de dominio permanecerá limpio de estos artefactos, como se muestra en forma esquemática en la figura 3.1. La validación de las reglas de negocio asociadas con los pagos, de ser implementada en el mismo lenguaje orientado a objetos, es posible que conduzca al uso de malas prácticas de programación. Esto es, la violación de criterios de buen diseño, como el código clausurado ante cambios, hace que cuando se desee cambiar por ejemplo algunos de los parámetros vinculados a las reglas se deberá cambiar el código ya implementado y probado. Esto decrecerá la posibilidad de que haya reutilización de código y diseño. El enfoque más apropiado es la PF. En la figura 3.1 se indica la implementación de la validación de las reglas utilizando PL en Prolog. Esta implementación brinda la posibilidad de conservar las reglas ligadas a las clases a las cuales se asignaron y además permite sacar fuera del código la validación y sus parámetros. También se muestra la utilización de PG en la implementación de otro de los requerimientos, el manejo de colecciones ordenadas según el enfoque de los contenedores, iteradores y algoritmos genéricos. Hacemos notar que en el caso mostrado la POA y la PG están siendo implementadas utilizando POO. Sin embargo este enfoque no corresponde al problema original planteado sino a subsistemas que prestan servicios. No hay plataformas que permitan resolver problemas utilizando múltiples paradigmas, sí los hay que implementan algunos de ellos. Por ejemplo, la programación en lenguaje Java permite POO, POA (AspectJ) y PG (Generics). La solución en la mayoría de los casos se obtiene complementando las plataformas con diferentes tecnologías, lo cual presenta la dificultad de adaptación de algunas para el buen funcionamiento con las otras.

3.10 Lenguajes de programación

Los *lenguajes de programación* son el medio a través del cual los programadores producen programas. Escribiendo sentencias en estos lenguajes se construyen los programas que al ejecutarse hacen que las aplicaciones posean un determinado comportamiento.

Estos lenguajes pueden clasificarse según diferentes criterios o categorías. Históricamente evolucionaron junto a alguno de los paradigmas que analizamos anteriormente; de esta forma permiten a los programadores escribir programas con distintos enfoques [24].

3.10.1 Clasificación

Los lenguajes de programación poseen un conjunto de características que hace factible su clasificación con diferentes criterios.

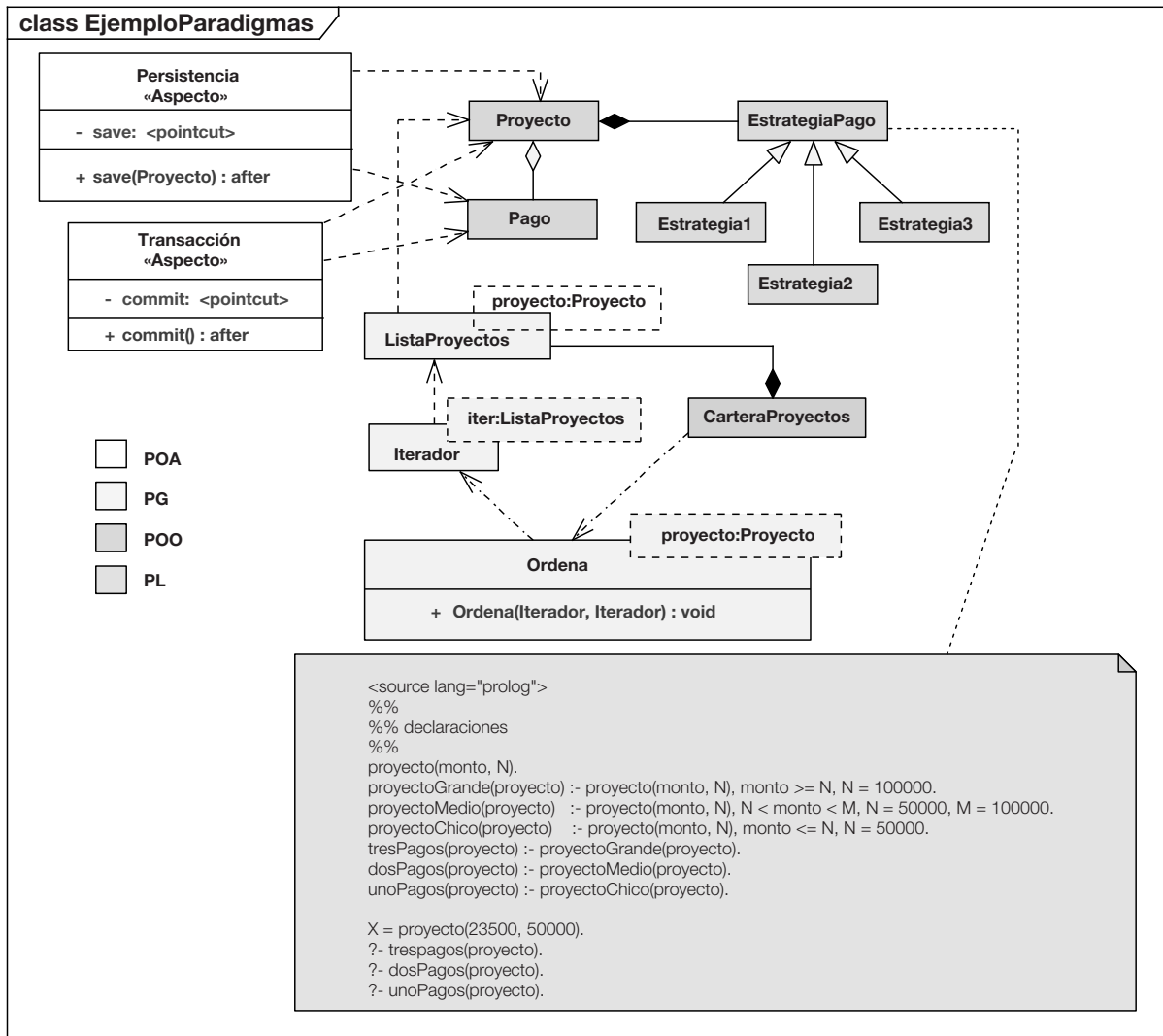


Figura 3.1. Esquema del ejemplo, el sistema implementado con diferentes paradigmas según el enfoque más apropiado para cada aspecto del problema

En la tabla 3.1 se presentan algunos de los lenguajes de programación. Se puede ver que se han listado además las características según las cuales podrán clasificarse. Estas características son:

Validación de tipos – Los chequeos de la semántica de los lenguajes pueden ser estáticos o dinámicos. Se dice que un lenguaje tiene tipos si la especificación de cada operación define tipos de datos para los cuales la operación es aplicable. En algunos lenguajes, las operaciones inválidas son detectadas cuando el programa es compilado (validación de tipos “estática”) y son rechazadas por el compilador. En otros, son detectadas cuando el programa es ejecutado (validación de tipos “dinámica”) y se genera una excepción en tiempo de ejecución.

Propósito – A los lenguajes concebidos como herramientas de desarrollo de sistemas no especializados, pertenecientes a una gran gama de aplicaciones posibles se los llamaron de *Propósito General*. A aquellos pensados para el desarrollo de sistemas especializados y que por consiguiente poseen características especiales orientadas a la realización de tareas específicas o a ámbitos de ejecución particulares fueron catalogados como *Dedicados* o *Específicos*.

Implementación – La transición entre el código que los programadores escriben al código que la computadora ejecuta puede ser de diferentes formas. Una es la de los lenguajes Compilados, cuyo compilador genera código binario directamente. Otra forma es la de los lenguajes cuyo compilador genera Código intermedio el cual es cargado en el contexto de ejecución y un proceso asociado a este lo ejecuta. Por último existen los lenguajes Interpretados, cuyo código no es compilado sino que al momento de ser ejecutado un proceso asociado al contexto de ejecución lo interpreta y ejecuta.

Estilo – En los lenguajes Imperativos las sentencias que el programador escribe definen “qué” se hace y “cómo se hace”. En los lenguajes Declarativos solo se enuncia “qué” se hace y la implementación queda oculta.

Paradigma – Como fue presentado en la primera parte del capítulo, cada paradigma fue acompañado en su evolución por uno o más lenguajes. Estos poseen el enfoque de desarrollo que plantea el paradigma que los sustenta.

Tabla 3.1 Lenguajes de programación y sus características					
Lenguaje	Validación de Tipos	Propósito	Implementación	Estilo	Paradigma
Java	Estático	General	Código intermedio	Imperativo	P00
C++	Estático	General	Compilado	Imperativo	P00
C	Estático	General	Compilado	Imperativo	PP
C#	Estático	General	Código intermedio	Imperativo	P00
Prolog	Dinámico	General	Interpretado	Declarativo	PL
Scala	Estático	General	Código intermedio	Imperativo	P00, PF
Java Script	Dinámico	General	Interpretado	Imperativo	P00
PHP	Dinámico	General	Interpretado	Imperativo	PP
Haskell	Estático	General	Compilado	Imperativo	PF
Lisp	Dinámico	General	Interpretado	Declarativo	PF
Smalltalk	Dinámico	General	Interpretado	Imperativo	P00
Objective C	Estático	General	Compilado	Imperativo	P00
Erlang	Dinámico	General	Código intermedio	Declarativo	PF
SQL	Dinámico	Específico	Interpretado	Declarativo	-
Cobol	Estático	General	Compilado	Imperativo	PP

Tabla 3.1 Lenguajes de programación y sus características					
Lenguaje	Validación de Tipos	Propósito	Implementación	Estilo	Paradigma
Python	Dinámico	General	Código intermedio	Imperativo	P00
Ruby	Dinámico	General	Interpretado	Imperativo	P00
Perl	Dinámico	General	Interpretado	Imperativo	PP
Self	Dinámico	General	Interpretado	Imperativo	P00

3.11 Métricas del grado de utilización

A partir de la cantidad de reuniones, seminarios, licencias, publicaciones y foros se ha relevado evidencia acerca del uso de los lenguajes de programación. Con esta se han elaborado métricas que muestran el grado de utilización de los diferentes lenguajes a través del tiempo.

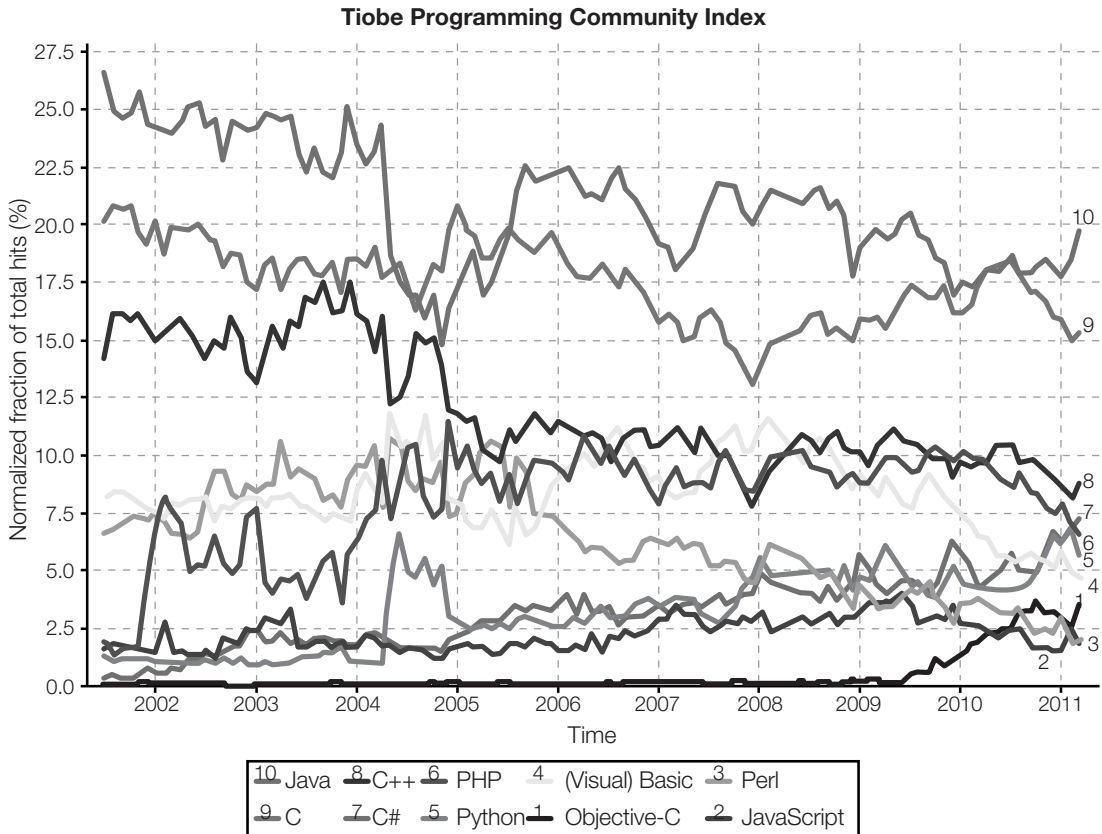


Figura 3.2. Tendencia en la evolución del uso de los lenguajes de programación en la última década. Fuente www.tiobe.com

Puede verse, en la figura 3.2, que los lenguajes de propósito general tales como C++ y Java presentan una declinación. Estos lenguajes son utilizados para el desarrollo de **aplicaciones de volumen importante**, lo cual puede ser interpretado como que en parte fueron reemplazados por algún otro lenguaje, como por ejemplo C# que muestra un sesgo creciente en el desarrollo de este tipo de aplicaciones. La tendencia de C es más estable debido seguramente a que no ha sido reemplazado como herramienta de desarrollo de **sistemas**.

Entre los lenguajes que presentan un crecimiento pueden citarse Java Script, el cual se produjo a partir del año 2006 de manera más marcada con el advenimiento de la **Web 2**, lo cual impuso requerimientos de mayor dinámica a las interfaces de usuario.

Otro crecimiento marcado se puede ver en la tendencia de Objective C, herramienta de desarrollo del software de los dispositivos de teléfono y tabletas digitales desarrolladas por **Apple** a partir del año 2008.

Puede observarse también que Php mantiene su nivel de uso, Perl decrece al igual que Visual Basic.

Estas estadísticas brindan además información de la utilización de los lenguajes por diferentes categorías. En la tabla 3.2 se muestran estos números según el paradigma. Los lenguajes orientados a objetos lideran la preferencia de los desarrolladores, seguidos por los de procedimiento, funcionales y lógicos.

Tabla 3.2. Porcentajes de utilización de los lenguajes de programación según el paradigma al cual pertenecen y su variación relativa entre los años 2010 y 2011		
Categoría	Marzo 2011	Variación marzo 2010
Lenguajes orientados a objetos	58,7%	5,3%
Lenguajes con procedimientos	36,4%	-5,6%
Lenguajes funcionales	3,6%	0,5%
Lenguajes lógicos	1,2%	-0,3%

Respecto de los lenguajes con tipos estáticos y dinámicos, continúan siendo de mayor utilización los primeros, como se observa en la tabla 3.3.

Tabla 3.3. Porcentajes de utilización de los lenguajes según el tratamiento que hacen de los tipos		
Categoría	Marzo 2011	Variación marzo 2010
Lenguajes con tipos estático	64,2%	4,2%
Lenguajes con tipos dinámico	35,8%	-4,2%

3.12 Conclusiones

Si bien los diferentes paradigmas acompañaron la evolución del desarrollo de software, no siempre fueron enfoques superadores sino diferentes. Con el aumento del tamaño de los sistemas de software de hoy día y la complejidad de los problemas que se buscan resolver se ha puesto de manifiesto la necesidad de contar con herramientas que soporten múltiples paradigmas. Una muestra de ello es el lenguaje Scala, de cortísima vida, que combina como vimos POO y PF, también los avances de más vieja data como la PG y la inclusión de la POA en el diseño e implementación de diferentes plataformas de software.

3.13 Contenido de la página web de apoyo



3.13.1 Mapa conceptual

3.13.2 Autoevaluaciones

3.13.3 Presentaciones en Power Point (exclusivo para el docente)

3.14 Referencias

- [1] *FORTTRAN for Scientists and Engineers: 1995-2003*, Stephen J. Chapman, McGraw-Hill Science/Engineering/Math, 2003.
- [2] *Programación con el lenguaje Pascal*, Llorca, Lozano, Paraninfo, 1980.
- [3] *El entorno de programación UNIX*, Kernighan, Pike, Prentice Hall, 1987.
- [4] *Página oficial de Haskell*, <http://www.haskell.org/>.
- [5] *Functional and Logic Programming Languages*, Peter Salus, Macmillan Technical Publishing, Macmillan Computer Publishing, 1998.
- [6] *Página oficial del proyecto Erlang*, <http://www.erlang.org/>.
- [7] *Página oficial de Scala*, <http://www.scala-lang.org/>.
- [8] *PROLOG Programming for Artificial Intelligence*, Ivan Bratko, third edition, 2001.
- [9] *Smalltalk 80: The Language*, Adele Goldberg and David Robson, 1989.
- [10] *Object Oriented Software Construction*, Bertrand Meyer, 1988.
- [11] *Página con información*, <http://www.adaworld.com/>.
- [12] *Object-oriented Programming in Common Lisp: A Programmer's Guide to CLOS*, Keene, Sonya, 1988, Addison-Wesley.
- [13] *Página oficial de Borland*, <http://www.embarcadero.com/products/delphi>.
- [14] *Página oficial de PHP*, <http://www.php.net/>.
- [15] *The C++ Programming Language*, Bjarne Stroustrup, Addison-Wesley Pub Co; Tercera edición, 2000.
- [16] *Thinking in Java*, Bruce Ekel, Pearson Education, 2000.
- [17] *C# 4.0 in a Nutshell: The Definitive Reference*, Joseph Albahari, Ben Albahari, O'Reilly, 2010.
- [18] *AspectJ*, <http://www.eclipse.org/aspectj/>.
- [19] *SpringSource*, <http://www.springsource.org/>.
- [20] *C++ modern design: Generic Programming and Design Patterns Applied*, Andrei Alexandrescu, Addison Wesley, 2001.
- [21] *Boost C++ libraries*, http://www.boost.org/community/generic_programming.html.
- [22] *Generic Programming*, <http://www.generic-programming.org/>.
- [23] *Multi Paradigm Design*, James Coplien, tesis doctoral, 2000.
- [24] *Programming Language Pragmatics, second edition*, Michael L. Scott, Elsevier, 2006.
- [25] *The art of Prolog*, Leon Sterling, Ehud Shapiro, The MIT Press, 1999.
- [26] *Object Oriented Analysis and Design with Applications*, Grady Booch, The Benjamin/Cummings Publishing Company, 1993.

Metodologías de desarrollo de software

Contenido

4.1 Metodologías	54
4.2 Metodologías conducidas por los planes	55
4.3 Metodologías ágiles	64
4.4 Diferencias de enfoque	65
4.5 Diferencias de aplicabilidad	68
4.6 Conclusión	69
4.7 Contenido de la página web de apoyo	70
4.8 Referencias	70

Objetivos

- Aprender el concepto de proceso de desarrollo y metodología
- Aprender las diferentes alternativas de organización de un proyecto de acuerdo al tipo de sistema a construir y al tipo de organización que encarga el sistema en desarrollo
- Aprender las diferentes alternativas de organización de un proyecto de acuerdo a las capacidades de los profesionales participantes y al tipo de organización en la cual se lleve adelante el proyecto

Competencias específicas

- Seleccionar la forma de trabajo en la organización de un proyecto de desarrollo de software

4.1 Metodologías

Una *metodología* es un marco de trabajo que puede ser utilizado como guía de las actividades a llevar a cabo. Por lo tanto, una *metodología de desarrollo de software* no es más que una forma de trabajo para desarrollar software, donde se especifica las tareas a llevar a cabo, los artefactos a generar y las relaciones entre ambos.

Este marco o forma de trabajo permite organizar el proceso de desarrollo de software a través de la definición de pautas a seguir y restricciones a cumplir. Las reglas y limitaciones especificadas son propias de cada metodología. En consecuencia, existe una amplia variedad de metodologías definidas, cada una con ventajas y limitaciones inherentes al contexto de aplicación. Es decir que no existe una metodología que pueda ser aplicada perfectamente en todos los contextos de trabajo.

4.1.1 Historia

En la década del 50 el software se desarrolla a medida para cada aplicación y es utilizado por la misma persona u organización que lo desarrolla. No se realiza ningún tipo de planificación ni documentación. La distribución del software es ínfima. No existe una metodología de desarrollo definida: se codifican y se corrigen los errores encontrados.

Desde mediados de los 60 el avance de la tecnología, junto con la multiprogramación y los sistemas multiusuario, aumentan el desarrollo de software y traen aparejado su distribución. De esta forma el software se establece como producto. La distribución del software requiere del mantenimiento y desarrollos propios para cada cliente. En consecuencia se busca el establecimiento de procedimientos para llevar a cabo el desarrollo de software. Así surge la programación estructurada en 1969 y en 1975 la programación estructurada de Jackson.

A partir de allí las metodologías evolucionan con el concepto de los sistemas distribuidos de la siguiente forma en los 80: Structured System Analysis and Design Methodology (SSADM), Structured Analysis and Design Technique (SADT) e Ingeniería de la información (IE/IEM).

En la década del 90, con el empleo masivo de los microprocesadores, de las redes y del software, las metodologías utilizadas son: Rapid Application Development (RAD), Programación Orientadas a Objetos (OOP), Dynamic System Development Method y Rational Unified Process (RUP) en 2003.

A finales de la década del 90 nace Scrum como parte de las metodologías ágiles y ante la rápida volatilidad de los cambios en el software. Este nuevo contexto hace nacer otra serie de metodologías ágiles en el nuevo milenio: eXtreme Programming (XP), Cristal, Desarrollo manejado por rasgos (FDD), Método de Desarrollo de Sistema Dinámico (DSDM) y más recientemente Kanban.

4.1.2 Clasificación

Hasta aquí hemos realizado la clasificación de las metodologías en aquellas conducidas por los planes y ágiles. Veamos ahora las diferencias más significativas entre ellas; como se muestra en la tabla 4.1.

Tabla 4.1

Metodologías conducidas por los planes	Metodologías ágiles
Conducidas por los planes	Orientadas al código (dado que la parte importante de la documentación es el código)
Planificación predictiva	Planificación adaptable
Resistencia a los cambios	Los cambios son bienvenidos
Orientadas a los procesos	Orientadas a las personas
Negociación de contratos	Colaboración con el cliente

4.2 Metodologías conducidas por los planes

Previamente hemos mencionado un listado cronológico de las metodologías conducidas por los planes. A continuación describiremos los modelos genéricos en los cuales estas se basan.

- Cascada
- Prototipado
- DRA (Desarrollo Rápido de Aplicaciones)
- Incremental
- Espiral

4.2.1 Cascada

Es un **modelo de desarrollo lineal secuencial**. El proyecto de software es dividido en fases que deben procederse en forma secuencial. Estas fases han servido de base para la definición de otras metodologías, como por ejemplo el modelo incremental descrito posteriormente en este capítulo.

El proceso incluye una serie de etapas en el siguiente orden [Roger Pressman]:

1. Definición del software: corresponde a la visión del producto, a sus aspectos desde el punto de vista comercial.
2. Análisis de requerimientos: implica el entendimiento del dominio del producto a ser desarrollado; esto es, funciones, comportamiento y relación con sistemas externos. El principal objetivo de esta fase es achicar los riesgos de negocio.
3. Diseño de la arquitectura: es la forma en que la solución se implementará.
4. Codificación: corresponde a la implementación de la solución de acuerdo a cómo se ha estipulado durante el diseño de la arquitectura.
5. Pruebas: deben asegurar que el producto satisfaga los requerimientos, es decir que cumplan con el comportamiento esperado. Los principales objetivos de esta fase son asegurar la calidad del producto y reducir los riesgos de falla de la aplicación en el entorno final de implantación.

Cuando finaliza cada una de las fases mencionadas, se realiza una revisión con el fin de determinar si está en condiciones de avanzar a la siguiente. Las fases no se solapan unas con otras.

Podríamos decir que la *metodología en cascada* es un modelo guiado por los documentos, dado que los elementos principales generados por cada una de las fases mencionadas son los documentos. De esta manera, el resultado obtenido en la fase de definición del software, que podríamos denominar “Documento de Visión”, es la entrada para el análisis de requerimientos. A su vez, el análisis de requerimientos genera un documento que podríamos denominar “Especificación de requerimientos”, el cual representa el ingreso a la etapa de diseño de la arquitectura. La arquitectura de la aplicación se plasma en un documento que podríamos nombrar “Documento de Arquitectura”, este elemento será la puerta de entrada a la codificación. Finalmente, la codificación genera el producto de software en sí y es la entrada para la fase de pruebas. Esta secuencia se observa en la figura 4.1.

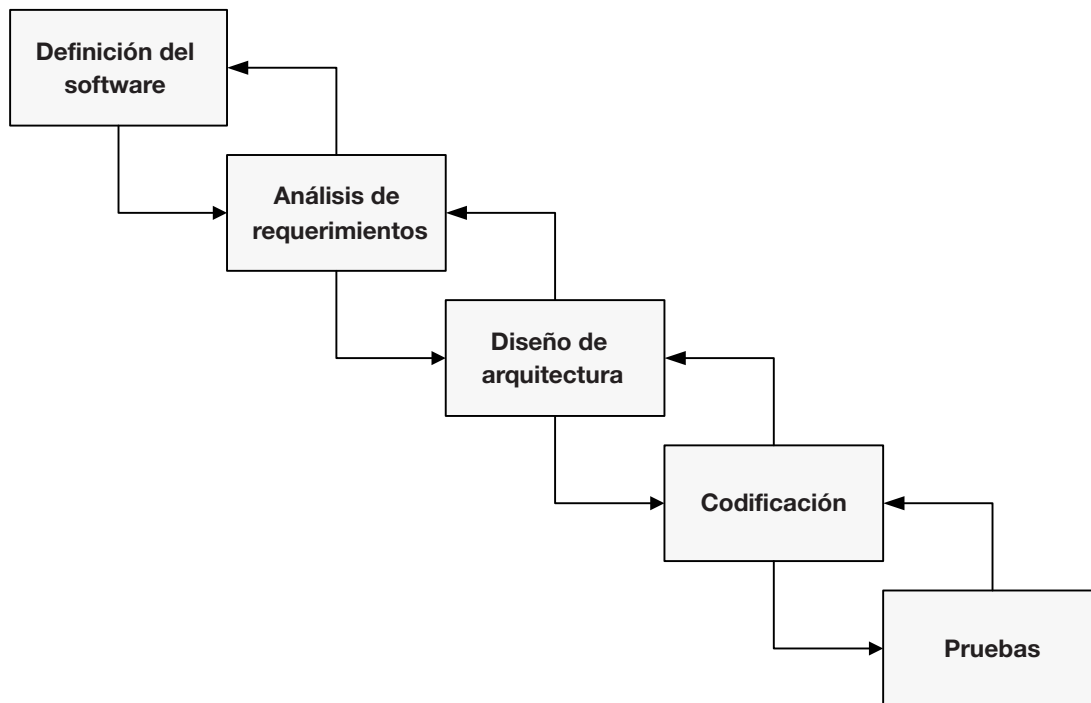


Figura 4.1. Metodología en cascada

Para poder aplicar esta metodología es conveniente que los requerimientos del cliente sean conocidos de antemano. Asimismo es preferente un número reducido de requerimientos. Tengamos en cuenta que las pruebas de la aplicación serán llevadas a cabo cuando todo el desarrollo se encuentre finalizado. Encontrar un error en esta etapa provoca su corrección y la ejecución de las pruebas de regresión pertinentes. Por último, cuando el equipo de trabajo no tenga experiencia, la aplicación de esta metodología puede ser de gran utilidad dado que ayuda a minimizar el esfuerzo en vano.

4.2.1.1 Roles

El modelo de cascada, así como cualquier proceso de desarrollo, se apoya sobre los recursos para llevar a cabo cada una de las etapas. Podríamos decir más específicamente que los modelos se valen de roles para ejecutar las distintas fases que los componen.

Para cada etapa del modelo cascada se puede asociar un rol y dado que las fases se siguen en forma ordenada, cada uno de los roles formará parte del proyecto en la fase correspondiente. Los roles asociados a cada una de las fases podrían ser:

- | | | |
|-------------------------------|---|--|
| 1. Definición del software | → | Responsable de la parte comercial del proyecto |
| 2. Análisis de requerimientos | → | Analista |
| 3. Diseño de la arquitectura | → | Arquitecto |
| 4. Codificación | → | Desarrollador |
| 5. Pruebas | → | Responsable de las pruebas del sistema |

No hemos mencionado un rol crucial para la ejecución de los proyectos: el líder de proyecto. El rol de líder de proyecto tiene una participación activa durante todas las etapas. Es el responsable de la administración del proyecto, de su éxito o su fracaso, sobre él recaen todas las responsabilidades.

4.2.1.2 Ventajas

La ventaja de emplear el modelo en cascada es que los riesgos son detectados en forma temprana dado que los requerimientos se congelan inicialmente. Esto permite tener una estimación considerablemente buena sobre los tiempos y costos requeridos.

4.2.1.3 Desventajas

A su vez, el hecho de contar con los requerimientos congelados impide o dificulta cualquier tipo de cambio que deba ser incorporado al producto. Por ejemplo, supongamos que en la etapa de pruebas el cliente detecta que se ha olvidado de solicitar un requerimiento. Esto impacta directamente sobre cada una de las fases. La magnitud del impacto estará directamente relacionada con el nuevo requisito. Siendo incluso, en algunos casos, imposible satisfacer la nueva necesidad. Además, dada la naturaleza de dependencia entre las distintas fases, el impacto en una de ellas provocará muy probablemente el impacto sobre el resto de las fases.

4.2.2 Prototipado

El *prototipado* es **un modelo de desarrollo iterativo** en el cual se desarrolla una maqueta del producto. Esta maqueta o prototipo desarrollada por el equipo de proyecto y refinada junto al cliente permite idealmente especificar los requerimientos del producto. Inicialmente se comienza con la definición de alto nivel de los requisitos junto al cliente. En base al entendimiento de los requerimientos se desarrolla un prototipo que será presentado al cliente, quien probará la maqueta y emitirá su opinión, de acuerdo a la cual se iterará sobre el proceso descrito o se procederá a la implementación definitiva del producto. Así el prototipo evolucionará refinando los requerimientos en cada iteración hasta lograr la aprobación del cliente.

El objetivo principal del proceso descrito es disminuir los riesgos que podría traer aparejada la falta de claridad de los requerimientos. Con esto se busca orientar al cliente/usuario con la definición de los requisitos.

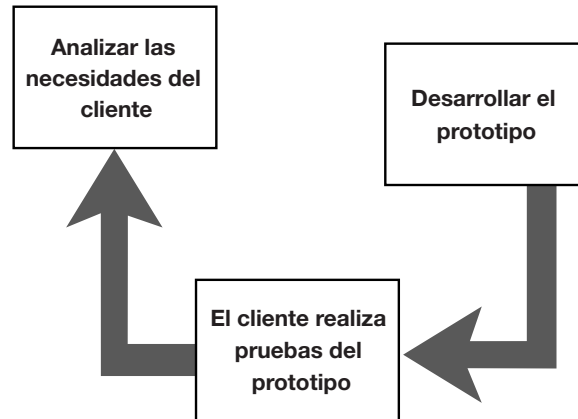


Figura 4.2. Metodología de Prototipo

Cuando se tiene una necesidad concreta pero no se tiene claridad sobre los detalles, es un caso que amerita el empleo del prototipo.

4.2.2.1 Roles

Los roles involucrados durante la etapa de definición del prototipo variarán de acuerdo al contexto pero podríamos mencionar al analista y/o desarrollador. Una vez que el prototipo se encuentre acordado con el cliente, aparecerán los roles de arquitecto, desarrollador y responsables de pruebas. Al igual que para el modelo en cascada, el líder de proyecto participará durante todo el ciclo de vida del proyecto.

4.2.2.2 Ventajas

Este modelo permite obtener una respuesta rápida por parte del cliente sobre el producto que desarrollaremos. Consecuentemente podremos corregir y/o modificar el requisito que no se adecue con las necesidades del cliente.

4.2.2.3 Desventajas

Dado que se presenta la aplicación “funcionando” al cliente, éste puede pensar que sólo restan algunos ajustes y cuando se le informa que debe construirse el producto puede ser que no logre comprenderlo y surjan conflictos. Para mitigar este riesgo es importante aclarar la forma de trabajo desde el comienzo. Puede suceder que con el fin de definir rápidamente el prototipo se lleve a cabo flujos o elementos que no pueden ser implementados posteriormente cuando se construya el producto final.

4.2.3 DRA (Desarrollo Rápido de Aplicaciones)

Este modelo, también conocido como RAD (en inglés *Rapid Application Development*), implica un **proceso de desarrollo lineal secuencial** llevado a cabo en paralelo por distintos equipos de trabajo. Cada uno

de estos equipos de trabajo será responsable por una parte de las funcionalidades del proyecto. El objetivo es crear en un período corto de tiempo un producto final. Asimismo, el modelo se basa fuertemente en la reutilización de componentes y en caso de que esto no sea posible, en desarrollar componentes reutilizables.

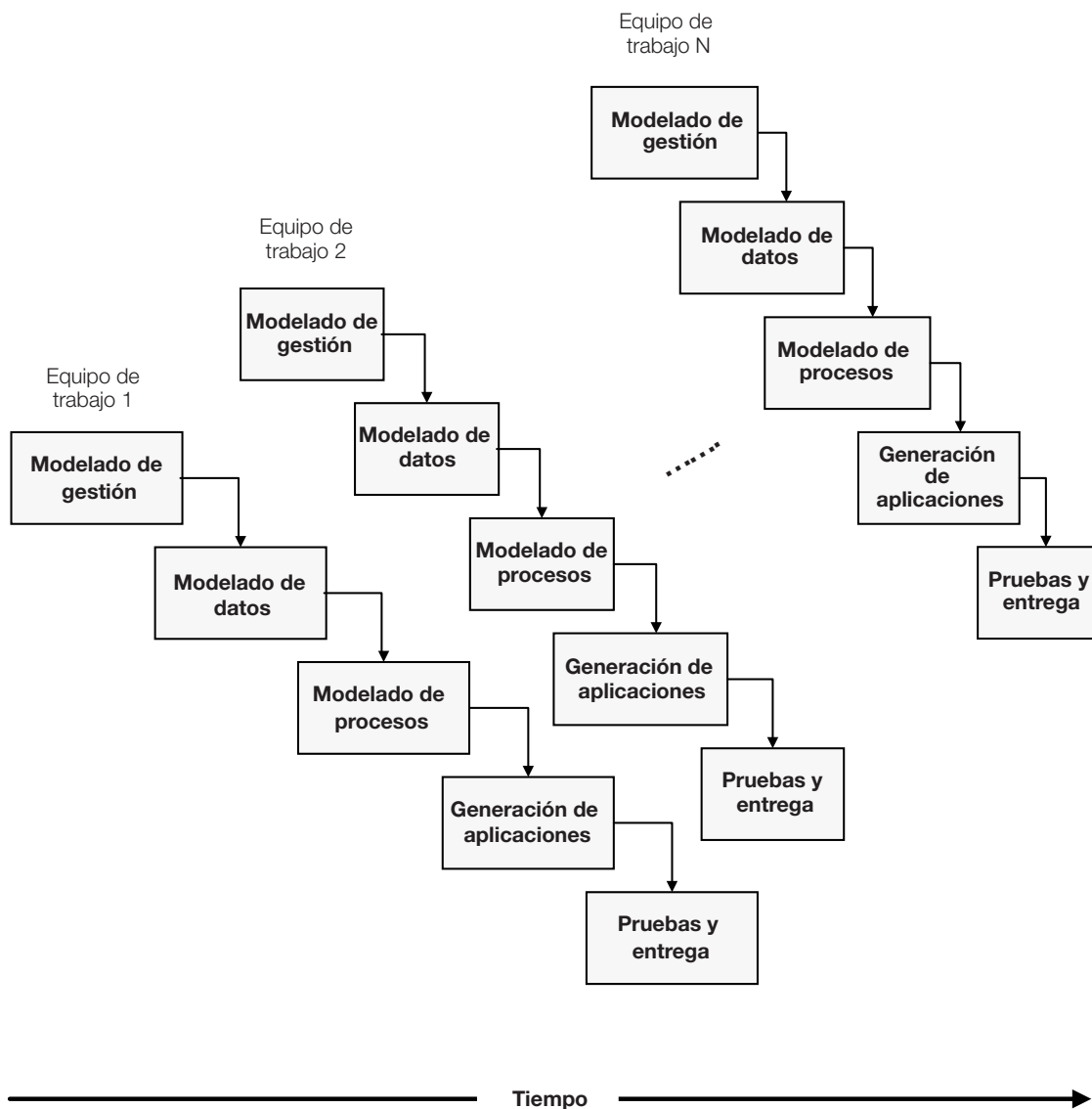


Figura 4.3. Metodología de desarrollo rápido de aplicaciones

Por lo tanto, el modelo requiere la división de los requerimientos en los equipos de trabajo. Cada equipo trabajará sobre los requerimientos asignados empleando las siguientes etapas [Roger Pressman]:

- **Modelado de gestión**
El proceso de gestión se modela teniendo en cuenta los siguientes asuntos: la información que conduce la gestión, la información generada y el responsable de haberla generado, el destino de la información y el responsable de su procesamiento.
- **Modelado de datos**
El modelado de los datos se resuelve teniendo en consideración: los objetos de datos básicos que el sistema procesará, los atributos y componentes de ese objeto y la relación entre los objetos de datos y los procesos que los impactan. Básicamente se busca obtener una abstracción del dominio relevante al producto en cuestión.
- **Modelado de proceso**
Lo que se busca modelar es el proceso de agregar, eliminar, modificar o recuperar los objetos de datos obtenidos en la etapa de modelado de datos.
- **Generación de aplicaciones**
La generación de aplicaciones corresponde al desarrollo del código necesario en un lenguaje de programación predefinido de los requerimientos estipulados. Como se ha comentado previamente, se buscará reutilizar componentes ya existentes, siempre que sea posible. Caso contrario se intentará crear componentes que puedan ser reutilizados posteriormente.
- **Prueba y entrega**
Las pruebas a realizar dependerán del tipo de componente utilizado. En el caso de que se reutilicen componentes, muchas veces puede suponerse que estos ya han sido probados y restaría solamente llevar a cabo una prueba integral entre los componentes. Cuando los componentes usados sean nuevos se requerirá llevar a cabo un conjunto de pruebas que logren asegurar el correcto funcionamiento del componente en cuestión.

Las etapas mencionadas previamente pueden variar según el autor.

4.2.3.1 Roles

Los roles involucrados coinciden con el modelo en cascada. Haciendo la distinción que se requerirá el desempeño de roles simultáneamente. Por ejemplo, inicialmente el equipo 1 comenzará a trabajar sobre los requerimientos correspondientes. Podría suceder que antes de finalizar con la etapa de modelado de gestión, el equipo 2 emprenda su fase de modelado de gestión en base a los requisitos asignados. Es decir que paralelamente se estarán ejecutando dos fases de modelado de gestión disjuntas. El mismo caso podría suceder para las etapas posteriores y/o con más etapas simultáneas. Esto implica contar con una cierta cantidad de recursos disponibles.

4.2.3.2 Ventajas

Cuando los requerimientos se encuentran especificados claramente desde el inicio es posible la construcción de un producto en un lapso de tiempo considerablemente pequeño.

4.2.3.3 Desventajas

El modelo DRA precisa de la cooperación del cliente para resolver temas que puedan presentarse en relación a él. Esta cooperación permitirá la rápida resolución de los eventuales inconvenientes facilitando una veloz evolución del proceso.

Asimismo, requiere contar con un gran número de recursos simultáneamente. Este número de recursos será proporcional a la magnitud del producto a desarrollar y al tiempo comprometido con el cliente.

El modelo exige que el producto a desarrollar pueda ser modularizado con el fin de llevar a cabo el proceso en paralelo. Por lo tanto, la metodología no puede ser aplicada a cualquier tipo de sistema.

4.2.4 Incremental

Las metodologías mencionadas hasta aquí no consideran la evolución del producto debido al tiempo. En contraposición, el modelo incremental permite el aumento iterativo de la funcionalidad del producto.

La base de esta metodología es el modelo lineal secuencial que se repetirá iterativamente. Cada iteración incrementará funcionalidad al producto hasta haber implementado todos los requerimientos. Por lo general cada una de las iteraciones se construye sobre el producto que se tiene hasta el momento, es decir sobre el resultado obtenido de las iteraciones precedentes. Queremos recalcar que como resultado de cada iteración se obtendrá un producto operacional y no un prototipo como en el caso del modelo prototipo.

Será conveniente utilizar este modelo cuando el producto a construir no esté definido completamente, cuando haya un gran número de requerimientos.

4.2.4.1 Roles

Dada la base en el modelo lineal secuencial, los roles participantes coincidirán. Con la gran diferencia que los roles participarán en forma simultánea unos con otros. Veamos un ejemplo para comprenderlo mejor. Supongamos el rol que desempeña el analista. Inicialmente comenzará el análisis de una cierta cantidad de requerimientos correspondientes a la primera iteración. Una vez finalizada esta tarea podrá comenzar el arquitecto con el diseño de la aplicación. Simultáneamente, el analista continuará con el análisis de los requerimientos para la próxima iteración. Cuando el analista haya terminado con su tarea, el arquitecto podrá iniciar su diseño para la segunda iteración y así sucesivamente para cada una de las iteraciones hasta finalizar el producto. De esta manera, el analista finalizará en primera instancia el trabajo en el proyecto y le seguirán el arquitecto, los desarrolladores y los responsables de las pruebas.

Recordemos nuevamente que el rol del líder de proyecto se llevará a cabo a lo largo de todo el ciclo de vida del proyecto.

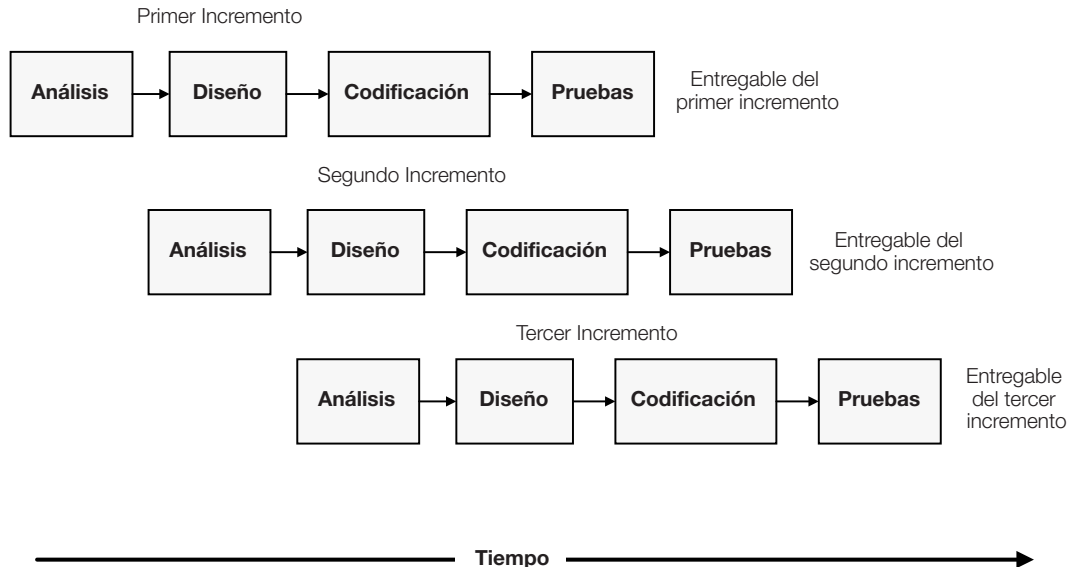


Figura 4.4. Metodología Incremental

4.2.4.2 Ventajas

El modelo iterativo permite adaptarse a la evolución que puede sufrir el producto con el pasar del tiempo. Al igual que para el modelo de prototipado, da la posibilidad de obtener una pronta respuesta por parte del cliente sobre el producto que se está desarrollando.

4.2.4.3 Desventajas

Previamente dijimos que cada una de las iteraciones da como resultado un producto operativo. Armar este entregable requiere de un trabajo extra por cada iteración. Este tiempo extra representa tiempo y costo al proyecto. Por lo tanto, debe ponderarse la importancia de estos entregables con el fin de reducir los tiempos y costos que siempre son tiranos en los proyectos.

4.2.5 Espiral

Este modelo combina los modelos de cascada y la esencia iterativa del prototipado descriptos previamente. Por lo tanto es una **metodología iterativa incremental**.

El proyecto de software es dividido en “mini-proyectos”. Cada uno de los cuales evoluciona el producto y busca minimizar los riesgos.

Como se puede desprender a partir del gráfico del modelo, el proyecto comienza desde el origen del espiral con una planificación de los requerimientos, continúa con el análisis de los riesgos del proyecto y así sigue. Es

importante observar que las primeras iteraciones son más pequeñas en costo y en tiempo. Por ejemplo, podría pensarse en un primer prototipo en papel, el cual en las próximas iteraciones podría convertirse en código. Los “mini-proyectos” están compuestos de las siguientes fases [Steve Mc Connell]:

1. Determinar los objetivos
Define los objetivos a ser alcanzados por el mini-proyecto correspondiente.
2. Identificar y resolver los riesgos
Identifica y resuelve los riesgos técnicos y de gestión del proyecto.
3. Evaluar alternativas
Involucra el análisis y prototipado de distintas alternativas posibles para los requerimientos correspondientes al mini-proyecto en cuestión.
4. Codificación y pruebas
Tareas necesarias para la implementación y las pruebas de los requerimientos pactados para el “mini-proyecto” correspondiente. La funcionalidad será implementada de acuerdo al resultado obtenido del análisis de las distintas alternativas planteadas previamente.
5. Planificar la próxima iteración
Involucra las acciones pertinentes para planificar los recursos, tiempos y costos necesarios para llevar a cabo la próxima iteración.

Con respecto a las etapas, es importante aclarar que el orden mencionado previamente es un buen proceso a seguir, pero que las fases no necesariamente deben continuarse como se han enumerado. El avance de las etapas estará asociado al contexto del proyecto.

Asimismo es posible combinar este modelo con otro, como podría ser la metodología en cascada. Es decir, sería posible realizar en primera instancia algunas iteraciones en espiral hasta reducir los riesgos y posteriormente continuar con el modelo en cascada hasta lograr el producto final.

4.2.5.1 Roles

Los roles involucrados son los mencionados en los modelos descriptos previamente. Cada uno de ellos participará en forma simultánea en el proyecto dada la naturaleza iterativa de la metodología.

4.2.5.2 Ventajas

La mayor ventaja del método es el control de los riesgos que lleva a cabo. A medida que se avanza en las iteraciones los riesgos del proyecto decrecen.

4.2.5.3 Desventajas

El modelo en espiral es difícil de implementar dado que requiere de una administración minuciosa. En relación a esto, es importante remarcar que si el principal riesgo del proyecto no es descubierto y mitigado puede ser complicada su administración. Asimismo, puede resultar complicado la definición de objetivos y su evaluación para decidir el momento de avanzar en las iteraciones [Steve Mc Connell].

4.3 Metodologías ágiles

Las metodologías ágiles surgen en el nuevo milenio como resultado de algo que se venía gestando hacia fines de la década del 90 y que se concretó en el año 2001. En este año se reunieron algunos representantes de los modelos ágiles y dieron origen a lo que se denomina el manifiesto ágil. Este acontecimiento y el manifiesto ágil se encuentran detallados en el capítulo 6 de “Metodologías ágiles”. De todas maneras es necesario aquí introducir un pequeño fragmento del manifiesto ágil que presentaremos en el capítulo mencionado con el fin de comprender el espíritu de estas metodologías. En el mismo capítulo podrá encontrar además un detalle de cada uno de los puntos principales.

El manifiesto ágil

“Estamos descubriendo formas mejores de desarrollar software tanto por nuestra propia experiencia como ayudando a terceros. A través de este trabajo hemos aprendido a valorar:

- Individuos e interacciones sobre procesos y herramientas.
- Software funcionando sobre documentación extensiva.
- Colaboración con el cliente sobre negociación contractual.
- Respuesta ante el cambio sobre seguir un plan.

Esto es, aunque valoramos los elementos de la derecha, valoramos más los de la izquierda.”

En las metodologías conducidas a los planes los procesos definidos son estrictos y burocráticos. En contraste, las metodologías ágiles, como su nombre lo indica, buscan brindar flexibilidad al proceso de desarrollo de software. De esta forma, intentan lograr un punto que sea útil entre las restricciones de los modelos orientados a los planes y la libertad absoluta.

Hay una gran variedad de modelos de metodologías ágiles que mencionaremos en el capítulo 6 dedicado exclusivamente a ellas. Dada esta variedad, nos es difícil practicar una definición específica. Sin embargo, podemos decir, en línea general, que el modelo de desarrollo de las metodologías ágiles es evolutivo, iterativo y trabaja con timeboxed. Cuando decimos que el modelo de desarrollo trabaja con timeboxed nos referimos a que las iteraciones poseen tiempos fijos. La planificación es adaptativa y los entregables evolutivos. Esto permite brindar valor al negocio en forma rápida y adaptarse fácilmente a los cambios. Además de estas prácticas existen otras. Todas se basan en valores ágiles presentados previamente.

La agilidad y adaptabilidad de estas metodologías requieren que el cliente pueda comprender y aceptar lo que esto implica. Es decir que es necesario un tipo de relación distinta con el cliente. Los contratos fijos que generalmente exigen los clientes no encajan en este modelo. Esto no significa que no se pueda fijar un presupuesto para el producto por adelantado. Lo que queremos decir es que no se puede fijar tiempo, precio y alcance. La manera ágil usual es fijar tiempo y precio, y permitir que el alcance varíe de manera controlada. La modificación del alcance le brinda la posibilidad al cliente de definir durante la vida del proyecto sus requerimientos, entonces puede apreciar a lo largo del proceso qué elementos le dan mayor valor a su negocio. Generalmente estos rasgos son difíciles de apreciar al principio del proyecto.

Con el fin de abstraer la esencia de las metodologías ágiles véase el siguiente gráfico donde se han incorporado los conceptos cruciales.

Entre las metodologías ágiles más conocidas podemos mencionar Extreme Programming (XP), Scrum, Lean Software Development, Crystal y Feature Driven Development.

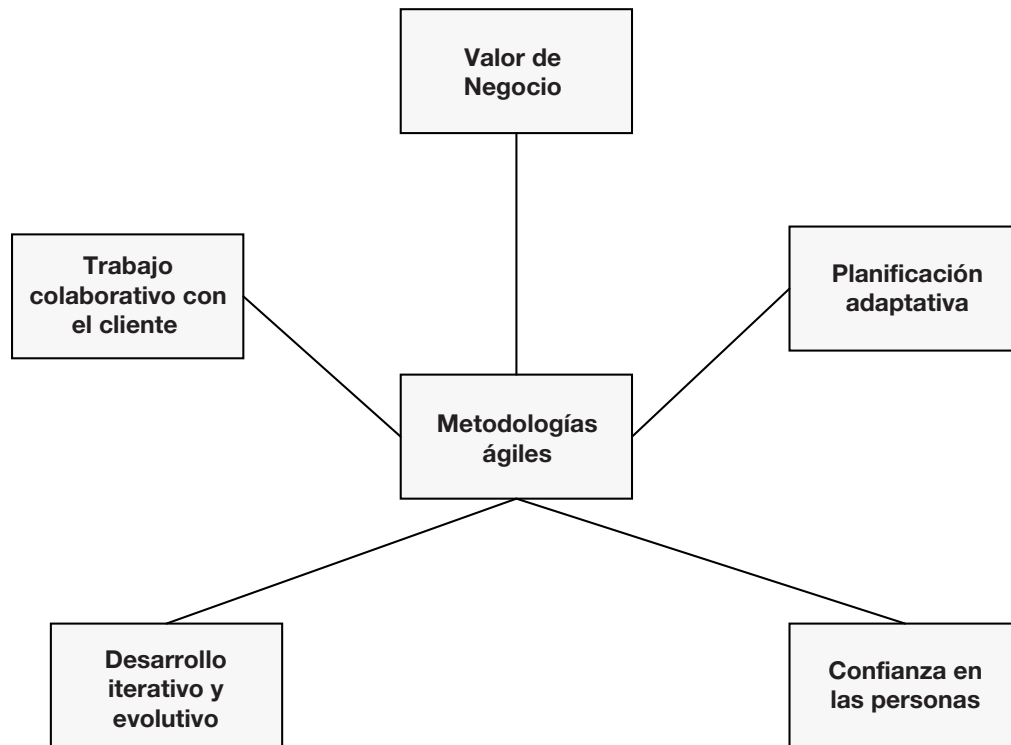


Figura 4.5. Esquema mostrando la esencia de las metodologías ágiles

4.4 Diferencias de enfoque

Una perspectiva interesante para hacer una comparación, entre las metodologías conducidas por los planes y las ágiles, es desde la teoría de las restricciones [10].

Para ello es necesario establecer un paralelo entre un proyecto de desarrollo de software y una organización. La gestión de las organizaciones tradicionalmente se administraron según la reducción de costos. Sin embargo no todas las áreas de una organización necesitan seguramente reducirlos para lograr un buen funcionamiento. Otras quizás sí lo necesitan. Esta visión local es reemplazada por una visión sistémica en la teoría de las restricciones, en la que en lugar de enfocarse en los costos se propone centrarse en la producción. Se trata entonces de maximizar el valor producido al negocio.

Como en toda organización, en un proyecto hay una inversión (I), un gasto operacional (GO) y una producción (P), como se muestra en la figura 4.6. La gestión del proyecto según esta estrategia es aumentar la producción (P) manteniendo la inversión (I) controlada y reduciendo el gasto operacional (GO) y el inventario (V).

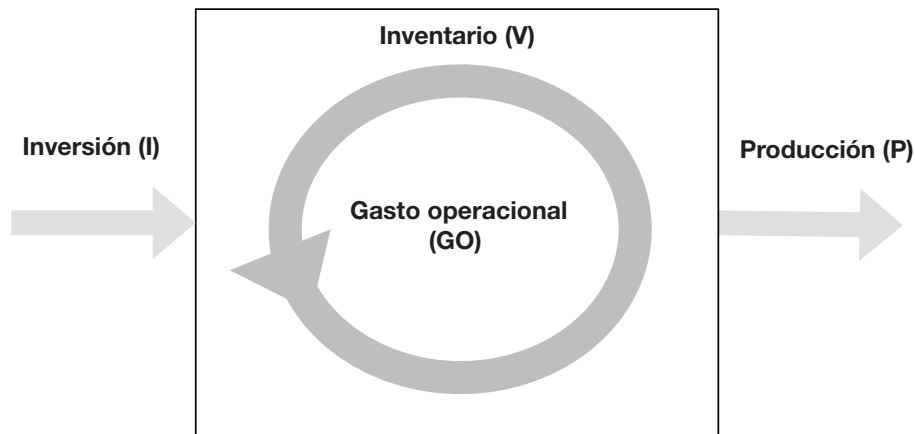


Figura 4.6. Diagrama esquemático que muestra la inversión (I), el gasto operacional (GO), el inventario (V) y la producción (P) en una organización

La diferencia de enfoque trata de priorizar una de las magnitudes frente a las otras. En la figura 4.7 se observa un esquema que muestra esta diferencia entre el enfoque que prioriza los costos y el que lo hace con la producción.



Figura 4.7. Esquema que muestra la diferencia de prioridades entre los enfoques basada en los costos y en la producción

El paralelo entre una organización y un proyecto nos lleva a la siguiente analogía:

Inversión (I): está representada por las ideas relevadas del cliente acerca del negocio y a implementar en el sistema. El alcance del proyecto está dado por este conjunto de ideas.

Costo operacional (GO): es el costo de transformar estas ideas en código productivo.

Inventario (V): está dado por las ideas elaboradas sin cerrar en el proyecto.

Producción (P): valor agregado al negocio a partir del código funcionando generado.

El comportamiento de estos enfoques aplicado a los proyectos de desarrollo y a la metodología utilizada es el siguiente:

Metodologías Conducidas por los Planes (Centrada en los Costos) que planifican los proyectos a partir de estimaciones de alcance y esfuerzo. A partir de estas se estima el tiempo de duración del proyecto y el presupuesto. La falencia de esto radica en que el alcance es el componente de mayor incertidumbre y todo este proceso se basa en él. La gestión del proyecto se desarrolla tratando de cumplir con los hitos planificados y reducir así la incertidumbre. Por lo tanto es equivalente a cumplir con la premisa de implementar los requerimientos que refinados exceden los originalmente estimados a un precio prefijado, o sea que se trata de minimizar costos. La gestión transcurre en negociaciones que permiten avanzar y terminar el proyecto utilizando alguna de las dos alternativas a saber: a) quitar requerimientos achicando el alcance y así cumplir con el presupuesto y tiempo planificados o b) alargar el tiempo y por lo tanto el presupuesto para implementar los requerimientos originales. Esta prioridad de alcance frente al resto de los otros componentes se muestra en la parte superior de la figura 4.8. Además, debido a su esencia, las metodologías están centradas en la documentación, el inventario muchas veces crece más de lo necesario y por lo tanto también el gasto operacional. Este incremento del inventario es visto como un activo y no como un compromiso.

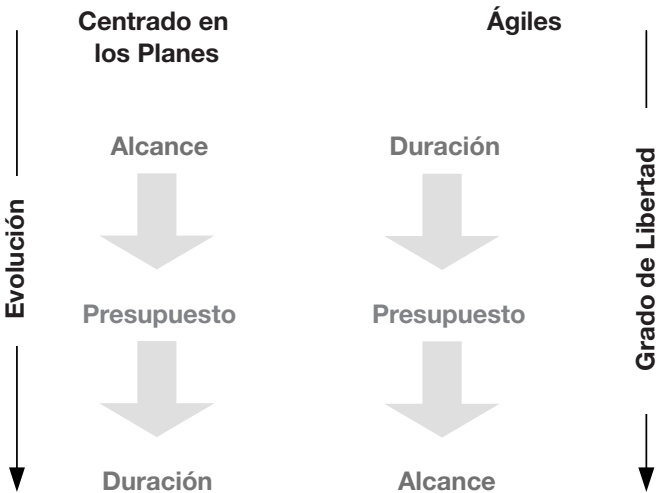


Figura 4.8. Esquema de la diferencia de los procesos de gestión de los proyectos con enfoques conducidos por los planes y ágiles

Metodologías Ágiles (Centrada en la Producción) que planifican sus proyectos en base a estimaciones preliminares de la misma forma que las anteriores pero no atan sus compromisos a estas. A diferencia de las

anteriores se fija un tiempo de duración del proyecto y un presupuesto, y se comprometen a implementar la mayor cantidad de requerimientos a partir de prioridades establecidas por el negocio. Esto es equivalente a maximizar la producción (P). Como se indica en la parte inferior de la figura 4.8, en este caso el alcance que es el componente de mayor incertidumbre ha pasado a ser el dependiente. La clave del cambio está en subordinar el elemento de mayor incertidumbre a los de menor.

4.5 Diferencias de aplicabilidad

Es necesario examinar la aplicabilidad de los métodos ágiles y considerar el espectro de dominios de problemas en los que podrían ser aplicables y aquellos en los que otros métodos serían mejores.

La evidencia de los datos relevados estadísticamente indica que deben tenerse en cuenta las siguientes variables y ayuda a sacar algunas conclusiones las cuales fueron condensadas en la tabla 4.2 que sigue:

Tabla 4.2. Guía de uso de los métodos ágiles y conducidos por los planes			
Según sea		Ágil	Conducida por los Planes
Tipo de proyecto	Maduro		V
	Inmaduro	V	
Tipo de sistema	Monolítico		V
	Divisible	V	
Madurez del proceso de desarrollo	Maduro		V
	En inducción	V	
Madurez del modelo de desarrollo	Definido		V
	Indefinido	V	
Capacidad / experiencia de desarrolladores	Autónomos	V	
	Dependientes		V

Se observa que las metodologías ágiles son más adecuadas cuando los dominios son inmaduros o desconocidos ya que permiten una mayor adaptación a los cambios, cuando es factible el desarrollo incremental y se cuenta con desarrolladores con buena formación y experiencia ya que esto permite organizar el grupo de proyecto en un trabajo autónomo. Cuando se cuenta con procesos y modelos de desarrollo definidos y maduros se está en condiciones de afrontar los proyectos utilizando otras metodologías con el previo análisis del resto de las variables [10].

En la figura 4.9 se muestra en forma esquemática una guía con el mismo sesgo que el presentado en la tabla anterior.

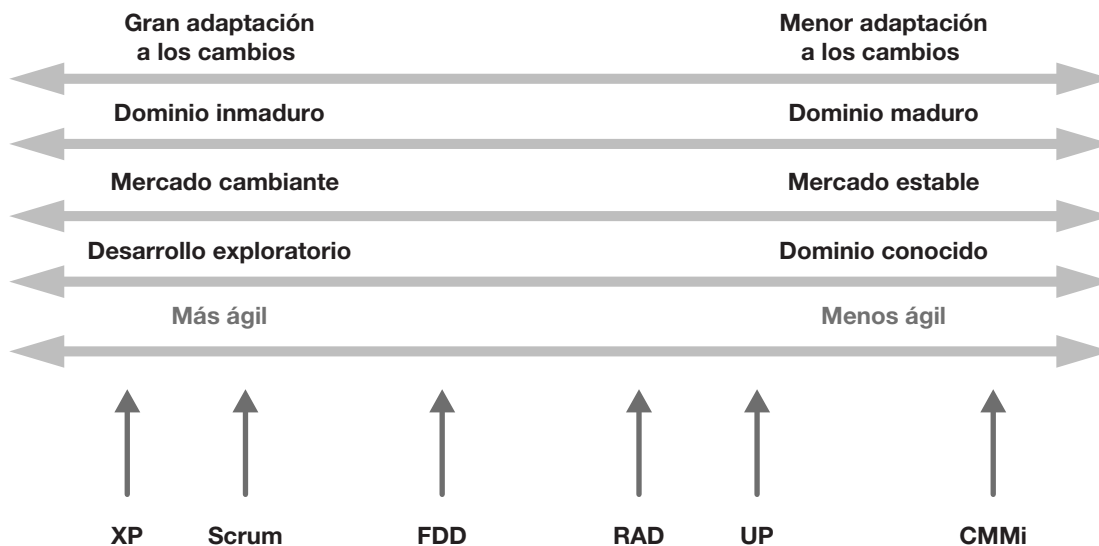


Figura 4.9. Indicadores de aplicabilidad de diferentes metodologías, adaptada de David J. Anderson, Eli Schragenheim [10]

4.6 Conclusión

Hasta aquí hemos descrito a grandes rasgos las metodologías conducidas por los planes y las ágiles. Es importante remarcar que el éxito de un proyecto no depende exclusivamente de la metodología utilizada, sino de otro conjunto de factores que oportunamente mencionaremos.

A continuación listamos los contextos en los que sería más apropiado utilizar metodologías conducidas a los planes:

- Un equipo de trabajo de gran magnitud
- Un alcance o contrato fijo.

La previsibilidad es una característica deseada, pero no siempre puede darse en los proyectos de desarrollo de software. Creer que tenemos previsibilidad y luego esto no se cumple puede provocar importantes desvíos sobre el proyecto. En contraposición a un contexto de previsibilidad mencionamos una serie de situaciones en las cuales sería más conveniente aplicar metodologías ágiles:

- Requerimientos inciertos o volátiles
- Desarrolladores responsables y motivados
- Cliente comprometido, es decir que entienda y se involucre.

Con esto no queremos decir que si estamos en un proyecto en el cual el alcance está cerrado tenemos que aplicar metodologías conducidas a los planes indefectiblemente. Sino que si se cumple esta condición y

estamos ante un cierto contexto en particular sería conveniente la aplicación de alguna metodología orientada a los planes.

De acuerdo a nuestro criterio, el hecho de forzar una forma de trabajo hace que no se defina ninguna metodología o se defina una metodología inapropiada. Por lo tanto, creemos que es conveniente adaptar los modelos en función de la realidad de la empresa o equipo de trabajo que la aplicación de una metodología aplicada al pie de la letra o, peor aún, la ausencia de una forma de trabajo.

A las conclusiones anteriores agregamos las siguientes:

1. Las metodologías ágiles no presuponen la no existencia de planes; por el contrario, promueven el ejercicio de planificación continua.
2. Las metodologías ágiles no se ocupan del modelo de desarrollo de software en los aspectos técnicos (modelo de requerimientos, modelo de dominio, modelo de diseño); los grupos que elijan una metodología ágil deben definir su propio modelo de desarrollo.
3. La forma en que se venden y comparan los proyectos de desarrollo de software son determinantes al momento de utilizar una metodología ágil o una conducida por los planes.

4.7 Contenido de la página web de apoyo



4.7.1 Mapa conceptual

4.7.2 Autoevaluaciones

4.7.3 Presentaciones en Power Point (exclusivo para el docente)

4.8 Referencias

- [1] Software Engineering Method and theory. A vision Statement, Ivar, Bertrand and Richard, <http://www.semat.org/pub/Main/WebHome/SEMAT-vision.pdf>.
- [2] *Kanban y Scrum. Obteniendo lo mejor de ambos*, Henrik Kniberg & Mattias Skarin, 2010 C4Media Inc.
- [3] *Patterns of Agile Practice Adoption. The technical cluster*, Amr Elssamadisy, forewords by Ron Jeffries, Craig Larman, 2007 C4Media Inc.
- [4] *Managing the development of large Software Systems*, Dr. Winston W. Royce, 1970, Los Angeles, EE.UU.
- [5] *Rapid Development*, Steve McConnell, Microsoft Press, Copyright 1996.
- [6] *Software Project Survival Guide* - Steve McConnell, Microsoft Press, Copyright 1998.
- [7] *Ingeniería de Software. Un enfoque práctico*, 5ta edición, Roger S. Presuman, McGraw Hill, 2002.
- [8] *Paper: "The new methodology"*, Martin Fowler, 13 de diciembre de 2005.
- [9] *Agile and Iterative Development: A Manager's Guide*, Craig Larman, Addison Wesley, 2004.
- [10] *Agile Management for Software Engineering: Applying the Theory of Constraints for Business Results*, David J. Anderson, Eli Schragenheim, Prentice Hall PTR, 2003.

Metodologías conducidas por los planes

Contenido

5.1 El Proceso Unificado de Desarrollo de Software	72
5.2 Conducido por los planes	74
5.3 El flujo de trabajo	75
5.4 Las iteraciones	77
5.5 El organigrama, los roles y actividades	78
5.6 Roles asociados a la organización	80
5.7 Los activos como nexo entre los roles	82
5.8 Planes	86
5.9 Plan de proyecto	87
5.10 Otros planes	88
5.11 Activos de trabajo	89
5.12 Conclusión	89
5.13 Contenido de la página web de apoyo	90
5.14 Referencias	90

Objetivos

- Qué es una metodología conducida por los planes
- Qué son los planes
- Cuáles son las características centrales de este tipo de metodologías
- El rol de la documentación en el ciclo de vida de un proyecto
- Cuáles son los activos de documentación más importantes de un proyecto

Competencias específicas

- Organizar un proyecto de desarrollo de software utilizando una metodología conducida por los planes

La Ingeniería de Software, como analizamos en capítulos anteriores, si bien tiene una vida mucho más corta que otras disciplinas ha tenido una evolución acelerada. Los cambios que han sucedido se podrían separar en dos tipos. Por un lado, los generados por la tecnología y así podemos llamarlos *técnicos*, y otros cuya esencia radica en la forma de trabajar para llevar adelante un proyecto de desarrollo. No estamos diciendo que ambos no estén relacionados, lo están. Se expresan por separado y en este sentido podemos tratarlos como tal, pero su vínculo es más complejo.

En este capítulo analizaremos un aspecto de la Ingeniería de Software que consideramos de importancia: las metodologías conducidas por los planes.

Como explicamos en el capítulo anterior, una metodología no es más que una forma de trabajo. Sin embargo es clave a la hora de organizar las tareas de un grupo de desarrollo. Remarcamos que forma de trabajo de los miembros de un grupo significa en este contexto; las actividades que cada uno realiza, los productos que cada uno genera y en qué momento y cómo estos productos son utilizados por el resto de los miembros. Todo esto conducido por el objetivo de construir un sistema que resuelva un problema determinado. Esta definición es amplia y un tanto ambigua pero nos sirve para introducir diferentes variantes que podrían ser comprendidas en esta definición y así reforzar las diferencias.

Al momento de escribir este libro y con la intención de simplificar esta presentación, podríamos decir que existen dos tipos de metodologías. Unas las conducidas por los planes y otras las ágiles. En este capítulo trataremos las primeras. Cuando decimos que una metodología es conducida por los planes lo que estamos expresando es que la disciplina de los miembros del grupo de desarrollo está basada en la elaboración y seguimiento de planes y documentación en forma previa a desarrollar todas y cada una de las tareas comprendidas en el desarrollo del proyecto.

5.1 El Proceso Unificado de Desarrollo de Software

Tomaremos como ejemplo de este tipo de metodología al Proceso Unificado de Desarrollo de Software [1]. En la década de 1970 existían en la industria del desarrollo de software dos estándares instalados de hecho, uno era el lenguaje “C” de programación y el otro el Desarrollo Estructurado de Sistemas [2] como paradigma de las metodologías llamadas *trabajo en Cascada* [3] [4]. Estas metodologías, como ya analizamos, consistían en el desarrollo del proceso en una cascada de tareas. Las dificultades que presentaban estas metodologías radicaban en:

1. Falta de una relación estrecha con los requerimientos en todo el ciclo de vida del proyecto de desarrollo.
2. Falta de una especificación estable de la arquitectura del software.
3. Una dinámica de trabajo con poca posibilidad de realimentación entre las distintas etapas del proceso de desarrollo.

Tanto las metodologías en Cascada como su transición al Proceso Unificado de Desarrollo de Software son un claro ejemplo de la estrecha relación entre los conceptos técnicos y los relacionados con la forma de trabajo. Afirmamos esto debido a que las falencias mencionadas fueron resueltas en parte por la introducción de un nuevo paradigma basado en la programación orientada a objetos y los lenguajes que la implementaron.

Hacia el año 1990 ya existían lenguajes orientados a objetos y una forma de trabajo que no se terminaba de estandarizar pero que era utilizada por los desarrolladores de la época [5] [6]. No fue hasta el año 1999 en que fue publicado el libro citado como [1] cuando comenzó a popularizarse esta nueva forma de trabajo, la cual fue acompañada por la mayor madurez de los compiladores de código C++ [7], lenguaje que se convertiría en estándar y reemplazaría a “C”.

Esta metodología propuso resolver las falencias con tres pilares básicos:

1. Proceso conducido por los casos de uso
2. Proceso centrado en la arquitectura
3. Proceso iterativo e incremental

Los casos de uso, como veremos con más detalle en un próximo capítulo, son una forma de especificar requerimientos funcionales. Esta forma tiene una expresión gráfica que permite realizar diagramas y así abarcar un grupo de funcionalidades asociadas, es decir cohesivas desde el punto de vista del negocio. Además constan de una forma textual que permite detallar dicha especificación. Más aún, las pruebas funcionales basan sus casos de prueba en los casos de uso. Además, verificamos el sistema construido en función de la implementación de la totalidad de los requerimientos basándonos en los casos de uso. Por esta razón se dice que el proceso es conducido por los casos de uso, ya que están presentes en todo el ciclo de vida.

Al estar inspirada esta nueva metodología en la programación orientada a objetos, toma fuerza y es más fácil de incorporar el concepto de arquitectura de software. Esto es debido a que los objetos como concepto poseen identidad y alcance, lo que facilita el reconocimiento y agrupamiento de componentes en partes. Estas partes y la manera en que se comunican son lo que llamaremos *arquitectura de software*. Los lenguajes orientados a objetos a partir de su propiedad de encapsulamiento hicieron posible definir arquitecturas estables para los sistemas en desarrollo. La nueva metodología puso énfasis en la definición temprana de la arquitectura y su prueba con relación a la funcionalidad derivada de los casos de uso que debía soportar.

Por último, y por la misma razón, el encapsulamiento aportado por los lenguajes, fue posible trabajar en forma iterativa e incremental. Esto es, ir construyendo pedazos cerrados del software de manera progresiva a lo largo del ciclo de vida del proyecto. De esta manera se pudo analizar en detalle, diseñar, construir, probar e integrar el código que implementa una dada funcionalidad al sistema en desarrollo sin modificar lo ya construido. Esto permite contar con implementaciones parciales y tempranas del sistema a efectos de aprender el negocio a medida que se desarrolla el sistema. Otra ventaja es achicar los riesgos de forma también temprana, lo cual no era posible en las metodologías previas. Esta imposibilidad era lo que generaba grandes inestabilidades en los proyectos en etapas tardías, al tratar de mitigar algunos riesgos cuando el proyecto estaba cerca de su fecha de finalización (cambios en la arquitectura por cambios en alguna funcionalidad, pruebas integrales a la arquitectura, integración de funcionalidades).

Como veremos en una próxima sección, esta metodología posee una dinámica expresada en fases y un conjunto de productos que los distintos roles generarán en cada momento.

Pero nos interesa analizar el aspecto vinculado a los planes de esta forma de trabajo, porque ese es el objetivo del capítulo.

5.2 Conducido por los planes

Cada vez que necesitamos realizar un propósito, conciente o inconcientemente, prolija o desprolijamente, documentado o no documentado, realizamos un plan. Este plan no es más que una ayuda o guía en el desarrollo del propósito mencionado. Seguramente incluimos el listado de actividades a realizar y el orden en que las haremos, los recursos y objetos que utilizaremos, quiénes nos ayudarán, cuándo haremos cada actividad, cómo las haremos y qué obtendremos como resultado, qué entregaremos y a quiénes.

Este plan tiene dos objetivos, el primero es realizar el ejercicio de planificación y el segundo es contar con un documento de referencia para todos los involucrados en el proyecto asociado a nuestro propósito. Si las tareas a realizar son complejas, podemos incluir qué acciones y alternativas evaluaremos seguir ante determinados obstáculos a los cuales llamaremos *riesgos*.

Este proyecto bien podría ser un viaje, una reunión, una expedición, la construcción de una casa o el desarrollo de un proyecto de software. El plan en cualquiera de estos casos sería la guía, que consultaríamos al comienzo y durante el tiempo que dure el proyecto. Cada vez que algo no pueda ser llevado adelante como establecimos en el plan, replanificaremos justificando las razones y modificaremos el plan que a partir de ese momento seguiremos. El plan es el que conduce la evolución del proyecto como se esquematiza en la figura 5.1.

No solo el plan se utiliza para llevar adelante el proyecto sino que **los participantes elaboran una serie de documentos que junto al plan constituyen los elementos conductores**. Todos los involucrados utilizan estos documentos como medio para respaldar, técnicamente y desde la gestión, todos y cada uno de los aspectos relacionados al proyecto. También son utilizados para construir acuerdos de alcances, pruebas de aceptación y formas de trabajo.

Los documentos son vistos como registros válidos de la realización de las actividades asociadas y son la mayoría de las veces entregados a los clientes, de allí su nombre genérico de *entregables*.

La disciplina de trabajo en esta metodología está representada por la elaboración de dichos documentos y su calidad, entendiéndose por calidad su adherencia a estándares predefinidos.

Estos documentos están ligados a los momentos del proyecto y a las actividades realizadas en esos momentos. Estas relaciones las analizaremos en una próxima sección.

Para elaborar el plan que conducirá el proyecto es necesario predecir de antemano los acontecimientos que sucederán durante el desarrollo. Con este fin es que se detectan los posibles riesgos y luego se administran de modo que no afecten el desarrollo predicho y no debemos apartarnos del plan. Por eso se realizan estimaciones de esfuerzo con tiempos de resguardos incluidos de modo que la incertidumbre de estas estimaciones no afecte la planificación y debemos modificar el plan. Este gran trabajo realizado al comienzo de nuestro proyecto con información no del todo certera es una de las críticas que se le hace a estas metodologías y es una de las razones por las que surgieron un par de décadas después las metodologías ágiles que presentaremos en el próximo capítulo.

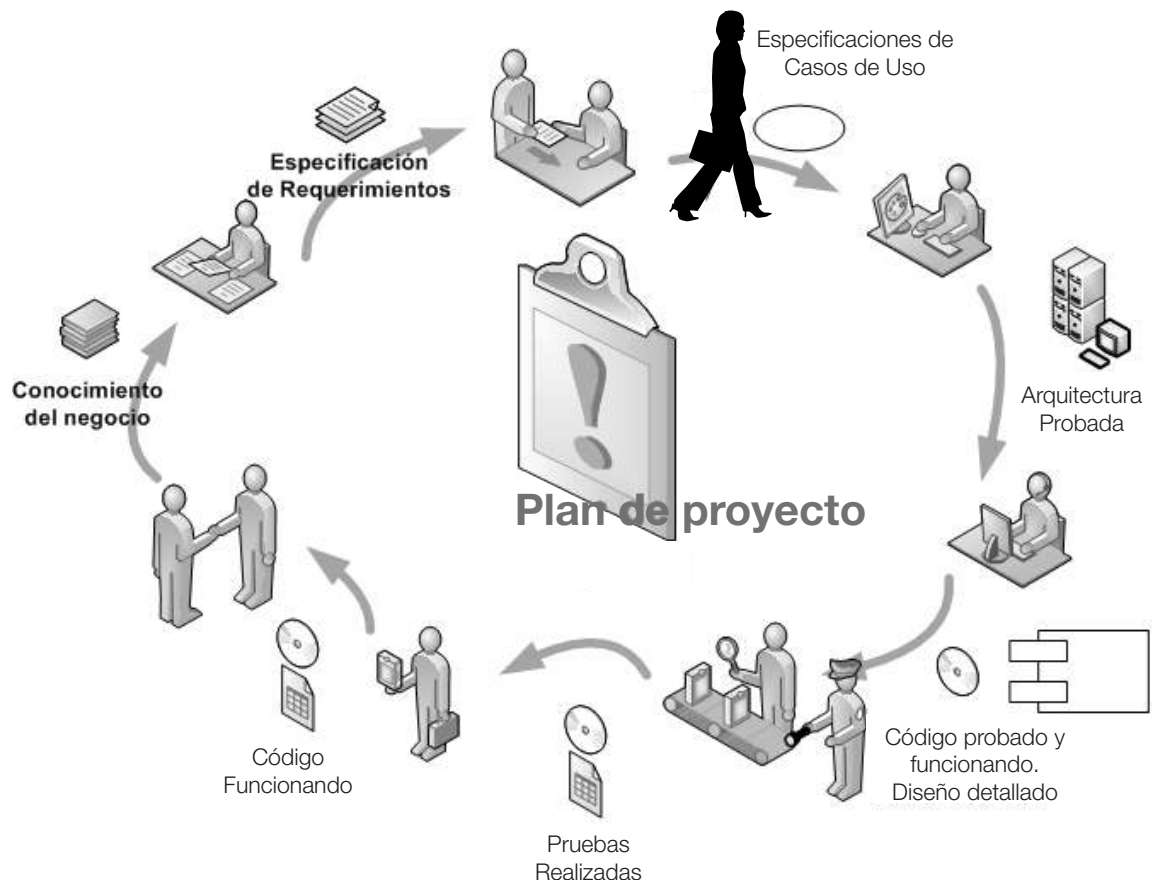


Figura 5.1. Flujo de trabajo en una metodología conducida por los planes

5.3 El flujo de trabajo

5.3.1 Las fases y los hitos

El Proceso Unificado de Desarrollo de Software posee cuatro fases. Cada una de ellas termina dando lugar a un hito importante en el ciclo de vida del proyecto. Como se muestra en la figura 5.2 estas fases se dan en secuencia, una termina y comienza la próxima. Hemos agregado a las fases originales de esta metodología una fase inicial de preventa en la cual se trabaja a efectos de vender el proyecto de desarrollo. A esta fase nos referiremos cuando tratemos, en detalle, la gestión de los proyectos en un capítulo más adelante. La diferencia sustancial con la secuencia de tareas de las metodologías en cascada es que aquí en cada una de las fases se llevan adelante un conjunto de tareas que son listadas verticalmente en la figura. Las partes de la figura donde se observan áreas sombreadas de la figura representa la intensidad con que se realizan cada una de estas tareas a lo largo del ciclo de vida del proyecto. En la parte inferior de la figura se indican las iteraciones de cada fase. Podemos ver que en este caso hubo una única iteración durante la fase de concepción y dos durante la fase de elaboración.

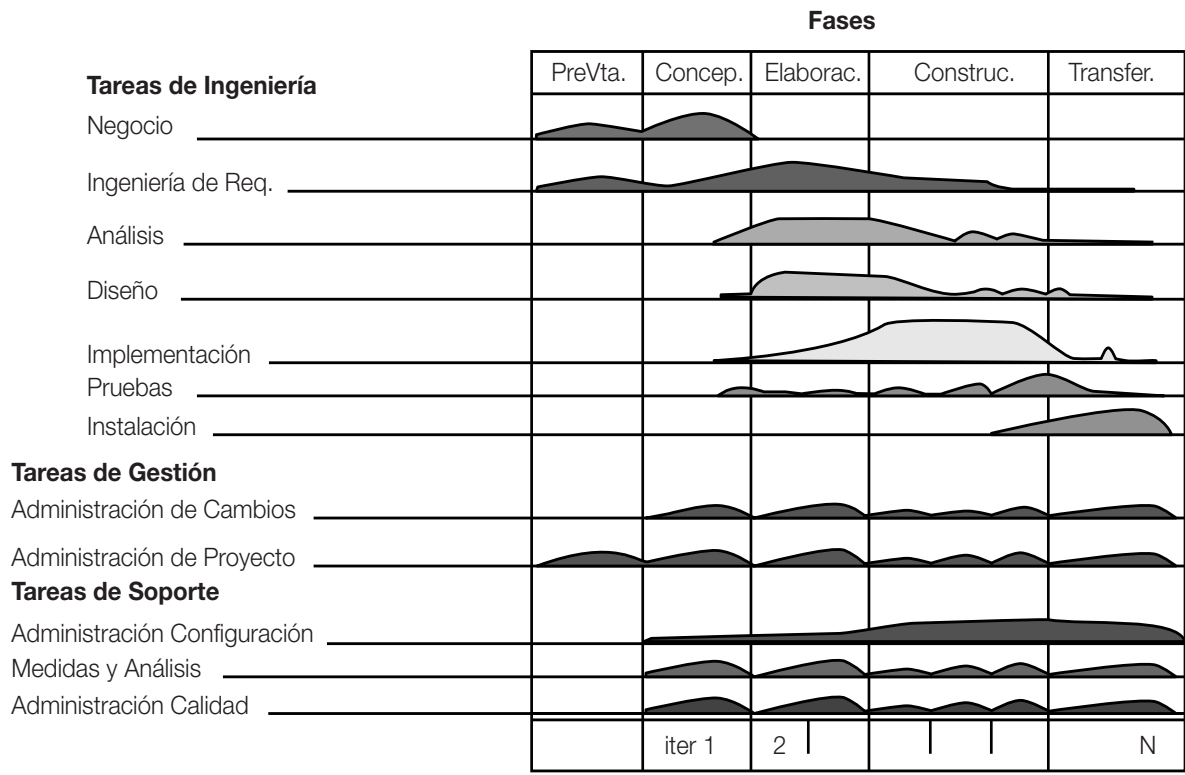


Figura 5.2. Las fases y tareas del Proceso Unificado de Desarrollo

Nos referiremos luego en detalle a las iteraciones.

Los hitos de finalización de cada fase están ligados al logro de los objetivos de cada una de ellas.

Como se muestra en la tabla 5.1, adaptada de Kurt Bittner y Ian Spence [8], el primer hito es establecer la factibilidad acordada del proyecto a partir de los requerimientos y la información relevada durante la fase de concepción. El objetivo para esta fase es achicar los riesgos del negocio, en relación a su comprensión.

El segundo hito, vinculado con el fin de la fase de elaboración, es una alternativa de solución probada. Nos referimos a ella como *arquitectura probada*. El objetivo para esta fase es achicar los riesgos derivados de la tecnología en relación con la arquitectura definida para el sistema en desarrollo.

El tercer hito, ligado a la terminación de la fase de construcción, es contar con una solución disponible. Se trata aquí de achicar los riesgos asociados a la implementación en tiempo y con los recursos disponibles.

Por último, el hito final está asociado a la terminación de la fase de transferencia que consiste en cerrar el proyecto de desarrollo. El riesgo que se busca achicar es el de la instalación en el ambiente productivo.

Tabla 5.1. Las fases y los hitos del Proceso Unificado de Desarrollo de Software			
Concepción	Elaboración	Construcción	Transferencia
Foco del riesgo Negocio	Foco del riesgo Arquitectura	Foco del riesgo Logística	Foco del riesgo Ambiente Productivo
Preguntas ¿Estamos construyendo la cosa correcta para el cliente? ¿Es la solución factible? ¿Cuánto esfuerzo costará?	Preguntas ¿Sabemos qué estamos construyendo? ¿Sabemos cómo construirlo? ¿Estamos de acuerdo en qué construir? ¿Cuánto dinero costará? ¿Cuáles son los riesgos técnicos? ¿Podemos mitigarlos?	Preguntas ¿Lo estamos construyendo? ¿Estará terminado a tiempo? ¿Es suficientemente bueno? ¿Mantenemos nuestros supuestos y decisiones? ¿Están los usuarios listos?	Preguntas ¿Es aceptable? ¿Está siendo usado? ¿Hemos terminado?
Artefactos claves - Visión. - Riesgos. - Plan de proyecto. - Listado de casos de uso críticos. - Modelo de negocios.	Artefactos claves - Modelo de casos de uso. - Especificación de los casos de uso principales. - Especificación de la arquitectura. - Prototipo de arquitectura. - Prueba de arquitectura.	Artefactos claves - Especificación de casos de uso. - Diseño. - Código. - Pruebas. - Resultado de pruebas. - Material de capacitación y documentación de usuario.	Artefactos claves - Instaladores. - Convertidores de datos. - Listado de últimos defectos y resoluciones.
Salidas Acuerdo de proyecto.	Salidas Arquitectura probada.	Salidas Solución probada, documentada y lista para instalar.	Salidas Solución instalada y funcionando en forma productiva.

5.4 Las iteraciones

Como ya mencionamos, una de las claves de esta metodología es su dinámica iterativa e incremental. En la figura 5.3, adaptada de Kurt Bittner y Ian Spence [8], se muestra este comportamiento. Puede verse la realización de todas las tareas asociadas a la implementación de los casos de uso seleccionados para las iteraciones. Al término de cada una se obtiene un incremento del producto en desarrollo, el cual es evaluado por todos los involucrados en el proyecto.

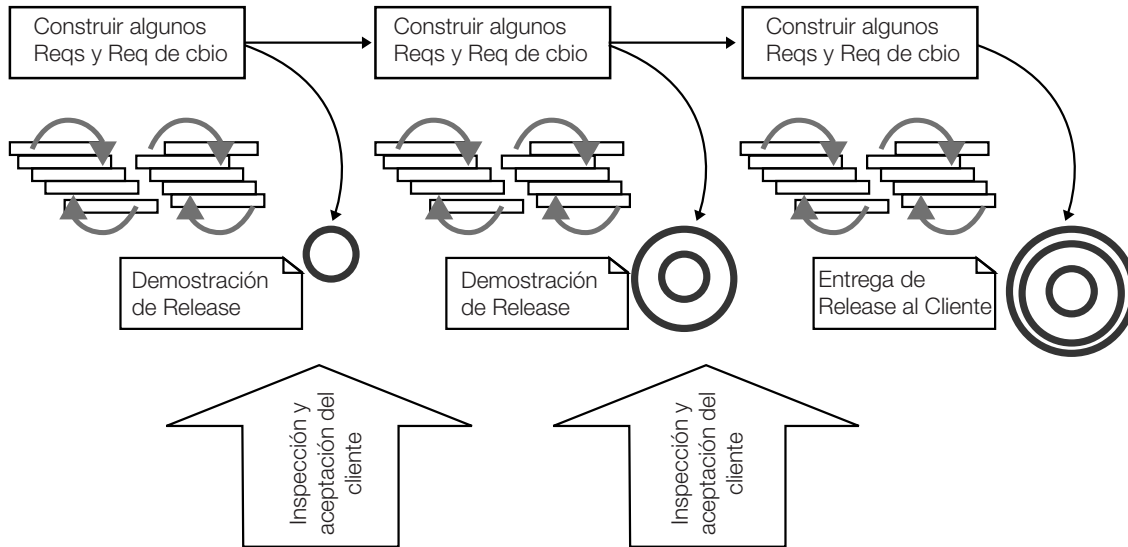


Figura 5.3. Esquema de la dinámica iterativa e incremental

Se pueden desarrollar iteraciones que incluyan la realización de un número fijado de casos de uso o desarrollar iteraciones de duración fija en las cuales se implementarán la mayor cantidad de casos de uso posible. Esta selección deberá ser priorizada como trataremos, con más profundidad, en un capítulo próximo. Existen ventajas en esta última alternativa que están dadas por la posibilidad de aprender por parte del grupo de desarrollo la propia velocidad de desarrollo y además mantener bajo control la planificación. Es decir, fijar un presupuesto y una planificación y trabajar todo el tiempo en maximizar la cantidad de funcionalidad priorizada que se implementa. En los capítulos dedicados a la gestión de los proyectos nos volveremos a referir a este tema.

5.5 El organigrama, los roles y actividades

Hasta los días en que las metodologías de trabajo en cascada fueron el estándar, los miembros de los grupos de desarrollo ocupaban cargos más que desarrollar roles. El Proceso Unificado de Desarrollo de Software fue innovador y precursor de la visión de los grupos formados por personas que en distintos momentos del ciclo de vida del proyecto pudiesen desarrollar distintos roles [1]. El aspecto de la dinámica iterativa de la metodología hace que se obtengan mejores resultados con miembros especialistas en determinados conocimientos que conducen la agenda del proyecto en una determinada fase. En la fase siguiente este miembro puede desarrollar otro rol o el mismo pero con otro grado de protagonismo. Esta rotación de roles impone un gran conocimiento por parte de todos los miembros de los pormenores del proyecto. También hace que todos los miembros permanezcan en el proyecto durante todo el ciclo de vida y con una carga de trabajo intensa. A diferencia de esto, era común ver en las metodologías en cascada a personas que participaban de una fase de un proyecto como analistas, por ejemplo, para luego irse a otro proyecto. Esta característica de las metodologías iterativas hizo que se deba modificar la formación profesional de los desarrolladores, fue necesaria una formación muy buena en todos los aspectos del desarrollo de software con especialización en alguna. Como conclusión, se cambiaron de una metodología a otra, del trabajo en cascada realizado por cargos al trabajo iterativo llevado adelante por roles cambiantes. En la figura 5.4 que sigue se esquematiza esta forma de organización de los grupos de desarrollo.

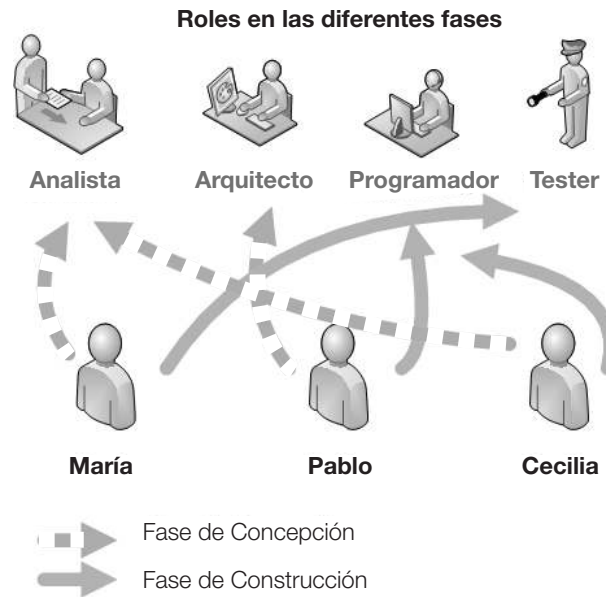


Figura 5.4. Roles intercambiables en las diferentes fases

En la figura 5.5 que sigue se muestra un organigrama típico de una organización dedicada al desarrollo de software. En ella aparecen los diferentes roles que hacen al funcionamiento de cualquier empresa de este tipo.

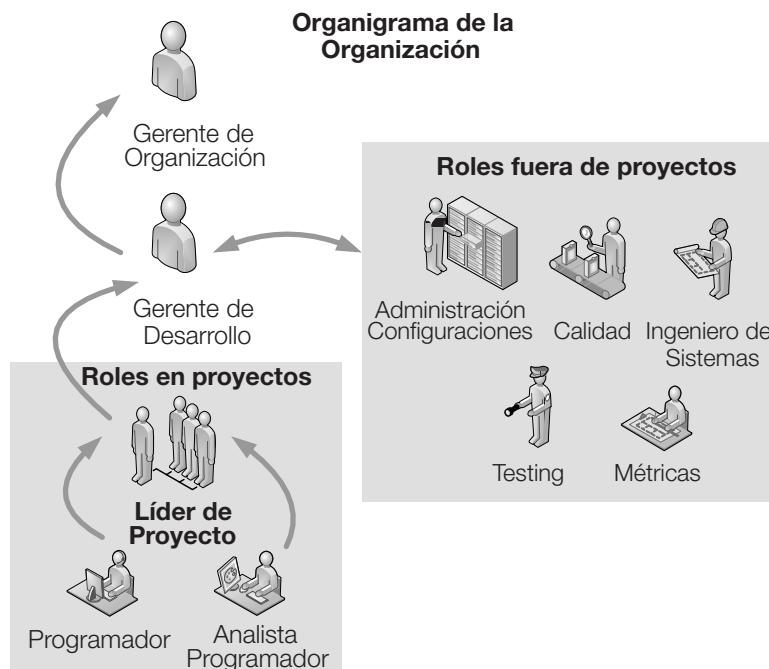


Figura 5.5. Organigrama de una organización típica de desarrollo de software

Como muestra la figura 5.5 hay roles en una organización de este tipo que funcionan a nivel de la organización y otros a nivel de los proyectos. A continuación los enumeramos y listamos sus actividades típicas.

5.6 Roles asociados a la organización

Gerente de Desarrollo: administra el área de desarrollo proveyendo recursos a los proyectos y participando en el seguimiento de todos ellos. Es responsable del área de desarrollo y del cumplimiento de las políticas de la organización.

Gerente de Calidad: gestiona el área de calidad reportando al CEO de la organización. Administra los recursos para el control de calidad en los proyectos de desarrollo. Participa de las definiciones de estándares de calidad.

Responsable de Calidad: lleva adelante el control de calidad en los proyectos de desarrollo.

5.6.1 Roles derivados de la forma de trabajo

Líder de Proyecto: gestiona un proyecto de desarrollo acordando con el cliente y facilitando el trabajo al grupo de desarrollo. Es responsable del proyecto y del grupo de desarrollo.

Analista Programador: posee una doble interfaz, una hacia los clientes en el relevamiento de requerimientos y otra hacia el grupo de desarrollo en el análisis de los requerimientos, planificación de pruebas funcionales y participación en la definición de la arquitectura.

Arquitecto: es el responsable técnico del proyecto, define la tecnología a utilizar, el ambiente de desarrollo y la arquitectura del sistema en desarrollo del proyecto.

Programador: realiza diseño de software, codifica y prueba.

Tester: ejecuta y recolecta resultados de las pruebas.

Los roles listados participan en los distintos momentos del ciclo de vida del proyecto realizando diferentes actividades. En la tabla 5.2 que sigue adaptada de Kurt Bittner y Ian Spence [8] se muestran las actividades realizadas en cada fase. En esta tabla dichas actividades están separadas de acuerdo a cómo los roles se relacionan con el desarrollo. Estos enfoques son el problema, la solución y el proyecto.

Si bien estos enfoques vinculan directamente al analista con el aspecto del problema, a los roles técnicos con la solución y al líder con el proyecto, todos ellos comparten su visión con los otros roles aunque con diferente protagonismo.

Tabla 5.2. Las fases y las actividades del Proceso Unificado de Desarrollo de Software

Concepción		Elaboración		Construcción		Transferencia	
Problema Solución Proyecto	• Identificar los requerimientos principales (arquitectura). • Entender el ambiente operacional. • Definir el problema. • Determinar el valor de resolver el problema.	Factibilidad Acordada	• Estabilizar los requerimientos. • Detallar los requerimientos críticos. • Refinar la visión.	Alternativa Probada	• Completar los requerimientos. • Construir la documentación. • Administrar los cambios de requerimientos. • Facilitar la disponibilidad de los usuarios. • Solicitar requerimientos de cambios.	Solución Disponible	• Llevar adelante las pruebas de aceptación. • Entrenar a los usuarios. • “Market” la solución. • Administrar los cambios en la comunidad de los usuarios. • Sugerir mejoras. • Informar defectos y deficiencias.
	• Explorar soluciones posibles. • Evaluar soluciones alternativas. • Proponer la arquitectura.		• Probar la arquitectura. • Desarrollar prototipos. • Describir la arquitectura. • Describir las interfaces entre subsistemas y sistemas externos. • Seleccionar componentes.		• Desarrollar componentes. • Probar y evaluar. • Refinar la arquitectura. • Optimizar el diseño de componentes.		• Instalar a los usuarios finales. • Prepararse para el mantenimiento. • Corregir defectos. • Ajustar la aplicación. • Contrastar funcionamiento y migrar datos.
	• Establecer el alcance del proyecto. • Planificar el ciclo de vida. • Establecer los costos. • Construir el modelo de negocio. • Identificar los riesgos críticos.		• Monitorear el progreso. • Mejorar las estimaciones. • Planear el desarrollo. • Controlar los riesgos. • Establecer la infraestructura del proyecto. • Definir las métricas. • Dar recursos al proyecto. • Refinar el proceso.		• Analizar el impacto. • Monitorear y controlar el desarrollo. • Planear la instalación. • Optimizar los procesos. • Monitorear los riesgos. • Tomar las métricas. • Optimizar el uso de recursos. • Controlar los costos.		• Evaluar el impacto del cambio. • Programar cambios. • Asignar soporte. • Monitorear y controlar instalación. • Cerrar el proyecto.
						Proyecto Terminado	

5.7 Los activos como nexo entre los roles

5.7.1 Los hitos y entregables

Según dijimos en una sección anterior esta metodología basa su disciplina en los planes y la documentación. Analizaremos entonces cómo se relacionan los hitos del ciclo de vida con los productos / documentos generados y entregados y cómo están vinculados los involucrados a estos entregables. En la figura 5.6 se presenta un esquema correspondiente a la fase de concepción del proyecto.

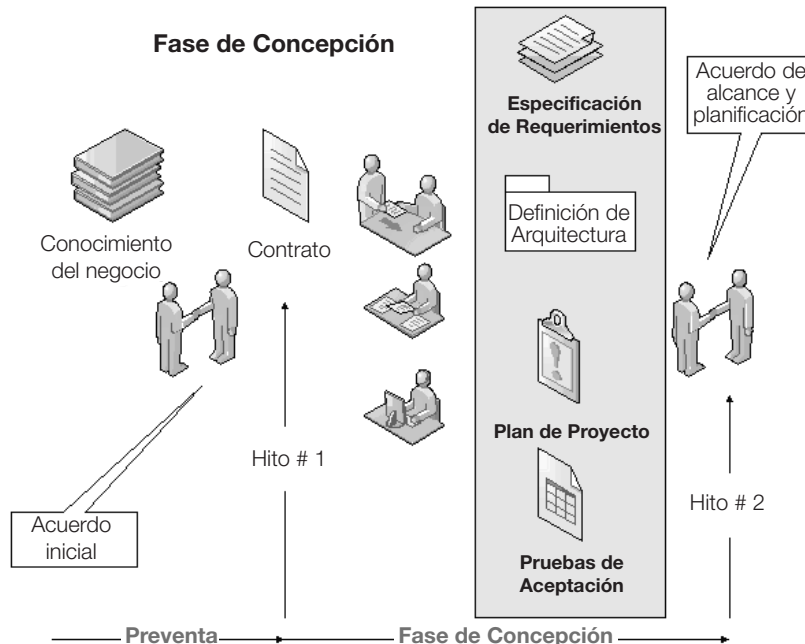


Figura 5.6. Productos generados y entregables de la fase de concepción

El primer hito, el que determina el comienzo del proyecto, está dado por la firma del contrato entre la organización cliente y la desarrolladora. El documento ligado a este hito es el contrato correspondiente. Los miembros del grupo de desarrollo más el resto de los involucrados desarrollando las actividades que se analizan en una próxima sección construyen los siguientes productos con el objetivo que se indica:

- ERS (Especificación de Requerimientos de Software): definición y **acuerdo del alcance del proyecto**.
- EDA (Especificación de Arquitectura): **definición y listado de alternativas** de la arquitectura y ambiente de desarrollo.
- Plan de Proyecto: **establecimiento de una estrategia** para el desarrollo del proyecto (estimación de esfuerzo, listado de tareas, recursos necesarios, riesgos a mitigar, cronograma, infraestructura necesaria, capacitaciones necesarias, organigrama y responsabilidades del proyecto, hitos y entregables).
- Otros planes: **administración de la configuración y de la calidad del proyecto**.

La aprobación y el acuerdo de lo definido en estos documentos determinan el hito de finalización de esta primera fase y el comienzo de la segunda.

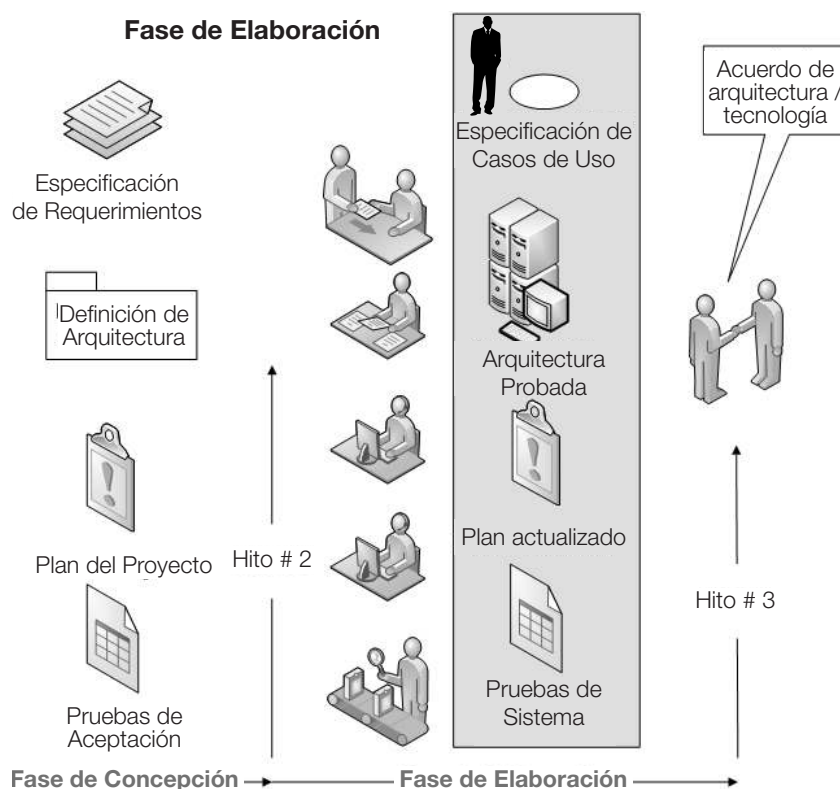


Figura 5.7. Productos generados y entregables de la fase de concepción

En la figura 5.7 se presenta un esquema similar correspondiente a la fase de elaboración del proyecto. Los miembros del grupo de desarrollo más el resto de los involucrados desarrollando las actividades que se analizan en una próxima sección construyen los siguientes productos con el objetivo que se indica:

- ERS (Especificación de Requerimientos de Software): **especificación de los casos de uso principales (que afectan a la arquitectura).**
- EDA (Especificación de Arquitectura): **alternativa seleccionada probada.**
- Plan de Proyecto: **estimaciones de esfuerzo adaptadas, iteraciones definidas, cronograma adaptado, riesgos actualizados.**
- Plan de Pruebas: **pruebas de sistema realizadas y a realizar.**

Fase de Construcción

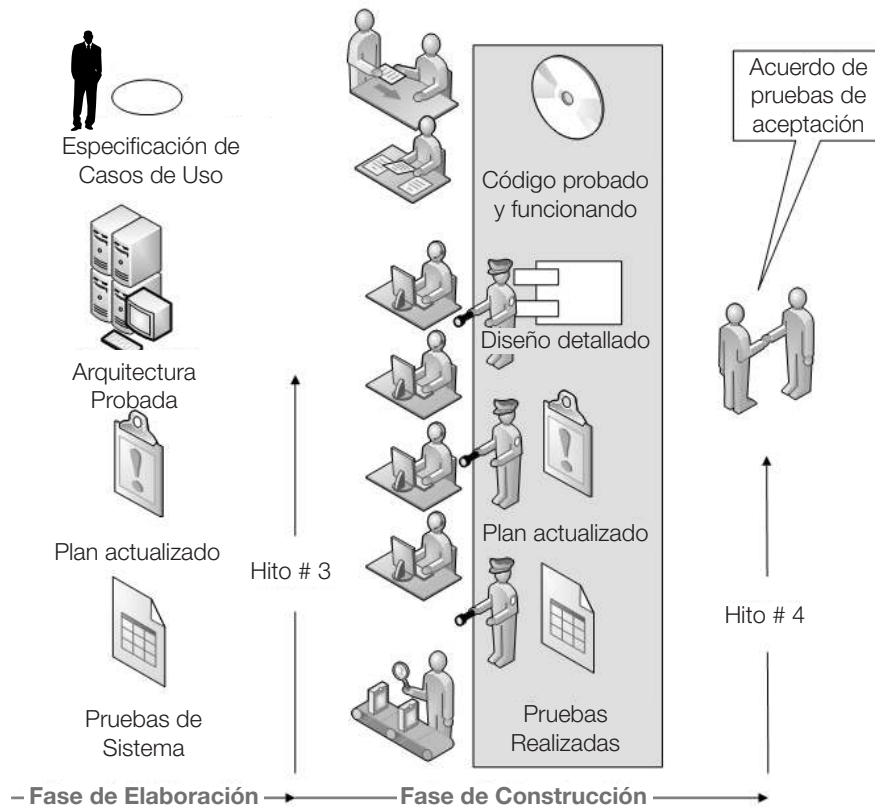


Figura 5.8. Productos generados y entregables de la fase de elaboración

En la figura 5.8 se presenta un esquema similar correspondiente a la fase de construcción del proyecto. Los miembros del grupo de desarrollo más el resto de los involucrados desarrollando las actividades que se analizan en una próxima sección construyen los siguientes productos con el objetivo que se indica:

- ERS (Especificación de Requerimientos de Software): **especificación de todos los casos de uso.**
- EDD (Especificación de Diseño detallado): **diseño detallado de los distintos paquetes.**
- Plan de Proyecto: **estimaciones de esfuerzo adaptadas, informes de las iteraciones realizadas, cronograma adaptado, riesgos actualizados.**
- Plan de Pruebas: **resultados de las pruebas realizadas.**

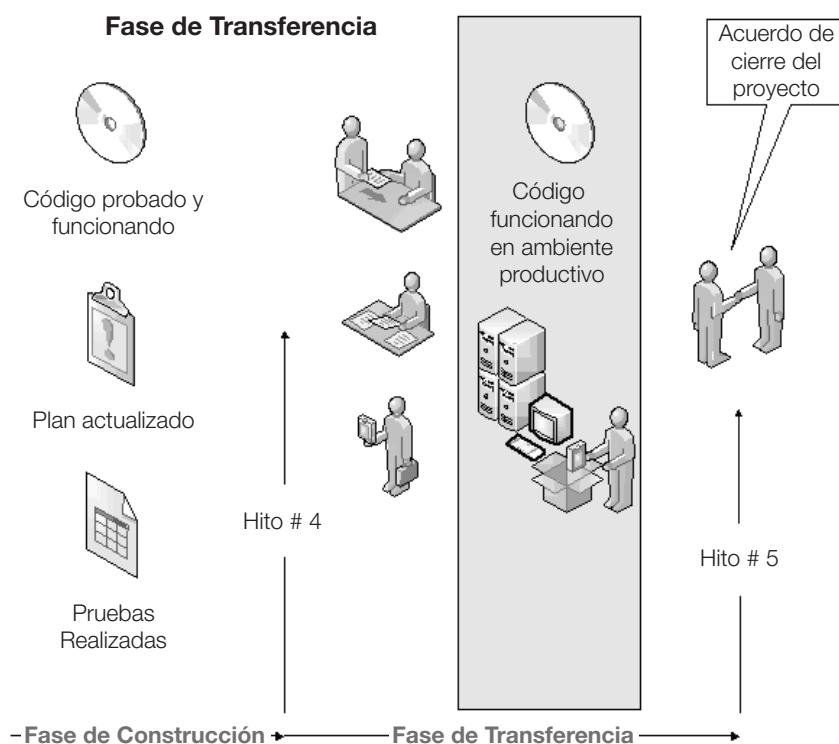


Figura 5.9. Productos generados y entregables de la fase de transferencia

En la figura 5.9 se presenta un esquema similar correspondiente a la fase de transferencia del proyecto. Los miembros del grupo de desarrollo más el resto de los involucrados desarrollando las actividades que se analizan en una próxima sección construyen los siguientes productos con el objetivo que se indica:

- Plan de Proyecto: **informes de seguimiento y cierre del proyecto.**
- Plan de Pruebas: resultados de las **pruebas de aceptación.**
- Guía de instalación y manual de usuario.
- Copia de código binario.

A partir de esta secuencia de imágenes que muestran la forma en que los roles participantes desarrollan sus actividades y generan productos que son adaptados y/o consumidos en las fases que siguen, queremos enfatizar la disciplina de esta metodología basada en estos productos / documentos intercambiados.

El otro aspecto que caracteriza a esta metodología, como ya lo mencionamos, es que todo este flujo de trabajo es planeado de antemano en los inicios del ciclo de vida del proyecto.

5.8 Planes

Como enunciamos en el inicio del capítulo, cuando decimos que esta metodología es conducida por los planes lo que estamos expresando es que la disciplina de los miembros del grupo de desarrollo está basada en la elaboración y seguimiento de planes y documentación en forma previa a desarrollar todas y cada una de las tareas comprendidas en el desarrollo del proyecto.

5.8.1 Preventa

Analicemos la planificación de un proyecto en el que se ha decidido la utilización del Proceso Unificado de Desarrollo de Software como metodología. Todos los temas que siguen, asociados a la planificación, los analizaremos en detalle en los capítulos que tratan la gestión de los proyectos.

Al momento de concretar el acuerdo entre el cliente y la empresa desarrolladora y firmar el contrato correspondiente se contará con la siguiente información:

- Una visión del proyecto.
- Una definición preliminar (con información sin relevar y otra sin analizar) del alcance del sistema que se va a desarrollar.
- Una estimación preliminar (por basarse en un alcance preliminar) de esfuerzo.
- Una definición preliminar de la tecnología.
- Una definición preliminar del grupo de desarrollo.
- Un presupuesto.
- Una definición de la forma de trabajo.
- Una planificación preliminar (duración del proyecto).

Como puede verse, esta información es en su mayor parte de carácter preliminar, ya que se ha trabajado un tiempo acotado para elaborar una propuesta técnico / económica por parte de la empresa desarrolladora hacia el cliente. Este corto tiempo, dependiendo del proyecto, puede ser un par de semanas, con un par de personas trabajando. Esta propuesta se conoce con el nombre de *anteproyecto* en otras ingenierías. Es de fundamental importancia esta fase del proyecto ya que como Steve M'Connell dice en *Software Project, Survival Guide* [9] esta es una de las oportunidades en que se evalúa la factibilidad del proyecto. El hecho de acordar su realización implica que ambas partes consideran factible el desarrollo en los términos y condiciones que se expresan en la correspondiente propuesta. No habrá sido trabajo en vano obtener como conclusión de esta tarea que el proyecto no es factible, ya que de no haberse concluido en este momento las pérdidas, para ambas partes, cuando el proyecto estuviese en estado avanzado hubiesen sido mucho más importantes.

Es en este punto donde las metodologías se bifurcan, por un lado las conducidas por los planes y las ágiles por el otro. Básicamente se diferencian en cómo avanzan con el desarrollo del sistema del proyecto en base a esta información preliminar, con bajo grado de certeza, con poca granularidad, sin detalle, relevada en poco tiempo y casi sin analizar.

Frente a esta incertidumbre las metodologías conducidas por los planes eligen relevar y analizar los requerimientos del proyecto en una primera fase de concepción y a partir de esta información dar forma definitiva a la planificación que es volcada en detalle en el plan del proyecto que se elabora al mismo tiempo.

Las metodologías ágiles eligen elaborar una visión de la evolución del proyecto muy poco detallada e ir descubriendo, aprendiendo, adaptando esta visión mientras se desarrollan las iteraciones correspondientes y se va aprendiendo el negocio para el cual el sistema en desarrollo se construye. Cuando en un próximo capítulo tratemos estas metodologías volveremos sobre este tema.

5.9 Plan de proyecto

Un plan de proyecto incluye los siguientes puntos:

1. Introducción

- a. Descripción del proyecto
- b. Objetivos
- c. Documentos relacionados
 - i. Entregados por el cliente
 - ii. Generados por el proyecto
- d. Destinatarios
- e. Entregables

2. Organización del Proyecto

- a. Planificación de la gestión
 - i. Estrategia general
 - ii. Supuestos
 - iii. Restricciones
 - iv. Administración de riesgos
 - v. Seguimiento

3. Planificación Técnica

- a. Ciclo de vida
- b. Requerimientos
- c. Infraestructura
- d. Organigrama
- e. Documentación a generar
- f. Capacitación a implementar
- g. Administración de datos

4. Relación con otros planes

- a. Plan de CM
- b. Plan de QA
- c. Plan de MA
- d. Acuerdo con proveedores

5. Productos de trabajo, cronograma y presupuesto

- a. Cronograma
 - i. Estimaciones de alcance y esfuerzo
 - ii. WBS
- b. Presupuesto

6. Replanificación

- a. Causas y fundamentación
- b. Alcance y esfuerzo
- c. Cronograma
- d. Presupuesto

7. Aprobación del plan**8. Acuerdos**

- a. Pruebas de aceptación
- b. Acuerdo del plan

9. Control de cambios

Esta plantilla modelo muestra los aspectos que se documentan del ejercicio de planificación mencionado. En un próximo capítulo trataremos la gestión de los proyectos analizando cada uno de estos aspectos y presentaremos un ejemplo de un proyecto de nuestro caso testigo.

5.10 Otros planes

5.10.1 Plan de administración de la configuración

Como en todo plan, en este se planean necesidad y disponibilidad de los recursos materiales, infraestructura, roles involucrados y tiempos del proyecto. En este caso son cubiertos los siguientes aspectos:

Recursos: software de administración del repositorio de software, software de backup y restauración del repositorio, almacenamiento de backup, etcétera.

Infraestructura: servidor del repositorio de software, conectividad al servidor, etcétera.

Roles y asignación: administrador de la configuración, administrador de la infraestructura.

Procesos y procedimientos: descripción de los procesos y procedimientos para cada tarea (agregar, cambiar y borrar ítems a la configuración, mantenimiento, auditorías, backup, etc.).

Definición de los ítems de configuración: alcance y modo de identificación de los ítems.

Tiempos: vínculo de las tareas con los hitos del proyecto de desarrollo.

5.10.2 Plan de administración de la calidad

Como en todo plan, en este se planean la necesidad y disponibilidad de los recursos materiales, infraestructura, roles involucrados y tiempos del proyecto. En este caso son cubiertos los siguientes aspectos:

Recursos: software de cálculo de métricas, software de oficina, etcétera.

Infraestructura: repositorio de documentación de procesos y procedimientos, computadoras personales de uso diario, etcétera.

Roles y asignación: responsable del área, controladores de la calidad y testers.

Procesos y procedimientos: descripción de los procesos y procedimientos para cada tarea (revisiones de activos y procesos, evaluación y escalamiento de no adherencias, etc.).

Definición de los procesos y procedimientos: definiciones de la organización para su proceso de desarrollo.

Tiempos: vínculo de las tareas con los hitos del proyecto de desarrollo.

5.11 Activos de trabajo

Entre los activos de trabajo típicos de esta metodología se encuentran la Especificación de Requerimientos de Software (ERS) y la Especificación de la Arquitectura (EDA). Estos activos relacionados con aspectos técnicos del proyecto se construyen y son utilizados en esta metodología con el objetivo de definir y acordar el alcance del sistema en desarrollo por un lado y los aspectos vinculados a la tecnología por el otro.

5.11.1 Especificación de requerimientos de software

Este activo tiene por objetivo documentar qué se construirá. Por esta razón se utiliza para definir y acordar el alcance con el cliente a partir de los requerimientos y casos de uso asociados. Documenta aspectos funcionales, no funcionales y del proceso de desarrollo que necesitan ser comunicados a todos los involucrados en el proyecto. Cuando tratemos en un próximo capítulo el trabajo con los requerimientos lo analizaremos.

5.11.2 Especificación de arquitectura de software

Este activo tiene por objetivo documentar cómo se construirá el sistema en desarrollo. Por esta razón se utiliza para definir y acordar su arquitectura. Documenta cómo los requerimientos funcionales y no funcionales serán implementados utilizando la alternativa tecnológica elegida. Cuando tratemos en un próximo capítulo el trabajo con el diseño de la arquitectura lo analizaremos.

5.12 Conclusión

En las metodologías conducidas por los planes, no solo el plan se utiliza para llevar adelante el proyecto sino que los participantes elaboran una serie de documentos que junto al plan constituyen los elementos conductores. Todos los involucrados utilizan estos documentos como medio para respaldar, técnicamente y desde la gestión, todos y cada uno de los aspectos relacionados al proyecto. También son utilizados para construir acuerdos de alcances, pruebas de aceptación y formas de trabajo.

Los documentos son vistos como registros válidos de la realización de las actividades asociadas y son la mayoría de las veces entregados a los clientes, de allí su nombre genérico de *entregables*.

La disciplina de trabajo en esta metodología está representada por la elaboración de dichos documentos y su calidad, entendiéndose por calidad su adherencia a estándares predefinidos.

Estos documentos están ligados a los momentos del proyecto y a las actividades realizadas en ellos. Para elaborar el plan que conducirá el proyecto es necesario predecir de antemano los acontecimientos que sucederán durante el desarrollo.

5.13 Contenido de la página web de apoyo



5.13.1 Mapa conceptual

5.13.2 Autoevaluaciones

5.13.3 Presentaciones en Power Point (exclusivo para el docente)

5.14 Referencias

- [1] *The Unified Software Development Process*, I. Jacobson, G. Booch, J. Rumbaugh, Addison Wesley, 1999.
- [2] *Structured Design*, L. Constantine, E. Yourdon, Yourdon Press, 1975.
- [3] *Structured Analysis and System Specification*, Tom Demarco, Prentice Hall, 1979.
- [4] *Structured System Analysis: Tools and Techniques*, Gane C. and Sarson T., New York: IST, Inc., 1977.
- [5] *Object Oriented Analysis and Design with Applications*, Grady Booch, Cumming Press, 1992.
- [6] *Object oriented Modeling and Design*, J. Rumbaugh et al., Prentice hall, 1991.
- [7] *The Design and Evolution of C++*, Bjarne Stroustrup, Addison Wesley, 1994.
- [8] *Managing Iterative Software Development Projects*, Kurt Bittner, Ian Spence, Addison-Wesley, 2006.
- [9] *Software Project, Survival Guide*, Steve McConnell, Microsoft Press, 1998.

Metodologías ágiles

Contenido

6.1 Extreme Programming y Scrum	92
6.2 Extreme Programming (XP)	93
6.3 Scrum	98
6.4 Integración Xp – Scrum	100
6.5 Ágiles	101
6.6 El flujo de trabajo	102
6.7 Organigrama y seguimiento del proyecto	105
6.8 Conclusión	107
6.9 Contenido de la página web de apoyo	107
6.10 Referencias	107

Objetivos

- Qué es una metodología ágil
- Cómo se diferencia de las metodologías conducidas por los planes
- Dinámica de la forma de trabajo de una metodología ágil
- El rol del cliente en el grupo de desarrollo
- Desarrollo y gestión ágil de un proyecto

Competencias

- Organizar un proyecto de desarrollo de software utilizando una metodología ágiles

Cuando en el capítulo anterior nos referíamos a las metodologías conducidas por los planes, enfatizamos dos aspectos de fundamental importancia. El primero fue el objetivo y el segundo el enfoque.

Cuando decimos que una metodología es ágil lo que estamos expresando es que la disciplina de los miembros del grupo se enfoca en un conjunto de prácticas definidas, lo que determina la disciplina del proyecto. Con relación a su objetivo decimos que este es maximizar la cantidad de requerimientos implementados en un tiempo y presupuesto fijados.

De las metodologías de este tipo mencionadas en el capítulo 4 elegimos desarrollar este capítulo basados en Xp y Scrum [4] [5] por ser las más utilizadas.

6.1 Extreme Programming y Scrum

En los años de 1995 a 2003 existían en la industria del desarrollo de software dos estándares instalados de hecho, uno eran los lenguajes “C++/Java” de programación y el otro era el Proceso Unificado de Desarrollo de Software como paradigma de las metodologías llamadas *orientadas a objetos*. Estas metodologías, como analizamos en el capítulo anterior, consistían en una evolución en forma iterativa e incremental del proceso, soportado por la arquitectura y conducido por los casos de uso.

Las dificultades que presentaron la utilización de estas metodologías radicarón en:

1. Carencia de un criterio para la adaptación de la forma de trabajo a los diferentes tipos de proyectos.
2. Generación excesiva de la documentación propuesta para los modelos de desarrollo.
3. Inconsistencias en la gestión de los proyectos con requerimientos, tiempo y presupuesto fijados.

Hacia el año 2001 se fundó el movimiento Ágil de metodologías de desarrollo a partir de la definición de un conjunto de principios comunes expresados en el famoso manifiesto citado en el capítulo 4.

De todos los referentes que confluyeron a esta fundación, hubo algunos que aportaron algunas de las metodologías conocidas en la época. Entre ellas se destacan Scrum como un **marco de gestión de los proyectos de desarrollo** y XP como un **conjunto de prácticas que configuran aspectos técnicos del desarrollo**. En este capítulo utilizaremos la complementación entre ambas para referirnos a las metodologías ágiles.

6.1.1 Procesos de gestión y procesos técnicos

De la misma manera que en las metodologías ya vistas, es necesario para que un proceso de desarrollo evolucione procesos definidos en el ámbito de las tareas técnicas así como en las que se ocupan de la gestión. Con esta premisa, la combinación de XP y Scrum es la forma de trabajo más utilizada por aquellos quienes eligen una metodología ágil [4].

6.2 Extreme Programming (XP)

XP fue concebida por Kent Beck y popularizada por su aplicación a un famoso proyecto en el cual las conducidas por los planes habían fracasado [3]. En realidad se debería decir por desarrolladores que aplicaron otras metodologías con un criterio erróneo que hizo que el proyecto no prosperara.

Su nombre se debe a que promueve la disciplina extrema de cada una de las personas que componen el grupo de desarrollo en la realización de un conjunto de prácticas que son consideradas esenciales.

En su fundamentación se mencionan valores, prácticas y principios.

Los *valores* constituyen la filosofía del método, las *prácticas* son expresiones de estos valores y los *principios* son la guía que ayuda a trasladar los valores a prácticas específicas.

6.2.1 Valores

Los valores de las personas, miembros del grupo de proyecto en este caso, determinan las decisiones que toman ante una situación y su actitud frente a esta. Estos valores son cinco y se describen a continuación.

Comunicación: se sitúa a la comunicación como una de las claves de un grupo que trabaja para lograr un objetivo común. Basada en esta comunicación se promueve la colaboración y el aprendizaje en equipo.

Simplicidad: debido a las múltiples experiencias de fracasos anteriores y al exceso que acompañaron por lo general a todos aquellos casos se eligió ponderar la simplicidad como resultado de lecciones aprendidas. Este concepto es aplicado al diseño y codificación así como al resto de los aspectos de un proyecto.

Retroalimentación: la corrección o adaptación ante los cambios hizo que se decidiera incorporar el aprendizaje a partir de lo ya realizado como mecanismo. En este se sustenta, como veremos, una de las prácticas emblemáticas llamada *refactorización*.

Coraje: se promueve la toma de decisiones ante la inacción. Este principio fomenta una actitud positiva y de empuje frente a cualquiera de los problemas que se plantean durante el ciclo de vida de un proyecto.

Respeto: se apela al respeto entre las personas como mecanismo de cohesión del grupo, lo cual implica la mayor parte de las veces una mayor efectividad.

6.2.2 Principios

Los valores no explican cómo realizar las tareas de desarrollo de un proyecto, son las prácticas las que proveen detalles específicos. Sin embargo, dado que hay una gran distancia entre las prácticas y los valores, XP también se conforma de principios, los cuales funcionan como mecanismos que permiten generar prácticas concretas a partir de los valores abstractos.

Los principios, a diferencia de los valores, son específicos del dominio del desarrollo de software y ayudan a entender y aplicar mejor las prácticas de la metodología.

La siguiente tabla 6.1 muestra los catorce principios y una breve descripción que sintetiza el espíritu de cada uno de ellos.

Claramente se observan aspectos relacionados al factor humano que hasta entonces no había sido incluido en la definición de otras formas de trabajo anteriores. Esto más el traslado de la disciplina centrada en los miembros del grupo en lugar de la documentación generada hizo que se hablara en la época de su publicación de una metodología basada en las personas.

Tabla 6.1. Principios de XP y sus descripciones	
Principios de XP	
Principio	Descripción
Beneficio mutuo	Todas las actividades deben beneficiar a todos los miembros.
Diversidad	Deben ser evaluadas todas las opiniones de los participantes.
Humanidad	El proceso debe contemplar necesidades humanas.
Responsabilidad aceptada	La responsabilidad debe aceptarse, no asignarse.
Economía	Todas las tareas deben sumar valor al negocio.
Calidad	No utilizar como variable de control del proyecto a la calidad.
Mejora	Trabajar de manera de asegurar una mejora continua de los procesos y productos.
Flujo	Asegurar la entrega continua de productos con valor.
Reflexión	Autointerrogarse acerca del trabajo realizado, cómo y por qué.
Pequeños pasos	Avanzar siempre de a pequeños incrementos.
Autosemejanza	Reutilizar todo lo usado en experiencias anteriores exitosas.
Falla	Utilizar el caso fallido como mecanismo de aprendizaje.
Oportunidad	Capitalizar los problemas como oportunidades de aprendizaje.
Redundancia	Atacar los problemas complejos con más de una alternativa a la vez.

6.2.3 Prácticas

Las prácticas son las actividades llevadas adelante por los miembros del grupo del proyecto. Se dividen en dos categorías:

Prácticas Primarias: son aquellas aplicables a cualquier actividad del proyecto y por las que se recomienda empezar. Estas se listan en la tabla 6.2.

Prácticas Corolarias: prácticas que se recomiendan adoptar una vez que se haya adquirido madurez en las Prácticas Primarias. Estas se listan en la tabla 6.3.

Se recomienda la implementación gradual, que favorece la madurez en el desarrollo de todas y cada una de las prácticas.

Tabla 6.2. Prácticas primarias de XP y sus descripciones

Prácticas Primarias			
Práctica Primaria	Descripción	Principios que la generan	Valores relacionados
Sentarse cerca	Disponer de un espacio donde trabaje el equipo completo.	Humanidad, Beneficio Mutuo, Diversidad	Comunicación
Equipo entero	Disponer de las personas con las habilidades necesarias para el éxito del proyecto.	Diversidad, Economía, Responsabilidad aceptada	Respeto
Espacio de trabajo informativo	El espacio de trabajo debe mostrar información acerca del estado del proyecto.	Mejora, Reflexión	Comunicación, Retroalimentación
Trabajo energizado	Trabajar sólo si uno está saludable y durante una cantidad de horas que no afecten a la persona ni a su productividad.	Humanidad	Respeto
Programación de a pares	Siempre programar cada funcionalidad de a dos, de forma tal que dos personas sentadas en la misma máquina analicen, diseñen, prueben y desarrollen.	Diversidad, Calidad, Reflexión, Beneficio Mutuo	Comunicación, Retroalimentación, Respeto
Historias de usuario (Stories)	Planificar utilizando historias de usuario: funcionalidades de valor visible para el cliente que son estimadas por el equipo.	Economía, Flujo	Comunicación, Simplicidad
Ciclo semanal	Planificar el trabajo de a una semana, en una reunión en la que se revise el progreso de la semana anterior, el cliente seleccione las historias de la semana y el equipo desglose las historias en tareas.	Pequeños incrementos, Economía, Reflexión	Retroalimentación, Comunicación
Ciclo cuatrimestral	Planificar el trabajo de a cuatrimestres para alinearlos a los objetivos de alto nivel.	Pequeños incrementos, Economía, Reflexión	Retroalimentación, Comunicación
Slack	En toda planificación que se haga, incluir algunas pequeñas tareas que puedan ser descartadas en caso de necesidad.	Flujo	Comunicación
Build de diez minutos	Compilar y ejecutar de forma automática los tests de todo el sistema en menos de diez minutos.	Flujo, Oportunidad, Falla	Retroalimentación
Integración Continua	Al producirse cambios sobre el sistema, integrar todas las partes y probarlo lo antes posible.	Flujo, Oportunidad. Pequeños incrementos	Retroalimentación

Tabla 6.2. Prácticas primarias de XP y sus descripciones			
Prácticas Primarias			
Práctica Primaria	Descripción	Principios que la generan	Valores relacionados
Desarrollo conducido por las pruebas	Antes de hacer un cambio, escribir el código que pruebe automáticamente lo que se va a modificar.	Falla, Pequeños incrementos, Calidad	Coraje, Retroalimentación
Diseño incremental	Diseñar el sistema todos los días y de forma gradual.	Pequeños incrementos, Calidad	Coraje, Retroalimentación

Tabla 6.3. Prácticas Corolarias de XP y sus descripciones			
Prácticas Corolarias			
Práctica Corolaria	Descripción	Principios que la generan	Valores relacionados
Involucramiento del Cliente	Hacer que la gente que se vea afectada por el sistema sea parte del equipo.	Flujo, Economía	Comunicación, Retroalimentación
Entregas incrementales	Al reemplazar un sistema existente, hacerlo por partes.	Flujo, Pasos de bebé	Retroalimentación, Coraje, Simplicidad
Continuidad del equipo	Intentar mantener juntos a los equipos efectivos.	Humanidad	Comunicación, Simplicidad
Contraer equipos	A medida que el equipo crece en productividad, mantener la carga de trabajo constante reduciendo su tamaño.	Mejora	Simplicidad
Análisis de causas	No sólo eliminar los problemas, sino también sus causas.	Flujo, Reflexión, Mejora	Comunicación
Código compartido	Cualquier miembro del equipo puede mejorar cualquier parte del sistema en cualquier momento.	Reflexión, Mejora	Retroalimentación
Código y pruebas	El código y sus pruebas deben ser el único artefacto permanente, el resto se extrae a partir de ellos.	Economía, Calidad	Simplicidad
Línea Base única del repositorio de código	Intentar desarrollar siempre una única línea de desarrollo. Sólo abrir ramas cuando sea estrictamente necesario y por el menor tiempo posible.	Flujo	Simplicidad, Comunicación
Instalación diaria	Instalar nuevas versiones del sistema en producción todos los días.	Flujo, Pasos de bebé, Economía, Calidad	Retroalimentación, Coraje, Simplicidad
Contrato de alcance negociable	Escribir contratos que propicien la negociación de alcance mientras el proyecto esté en curso.	Flujo, Economía	Coraje, Simplicidad
Pagar por usar	Intentar cobrar por el uso del sistema conectando directamente el flujo de dinero con el desarrollo del software.	Economía	Retroalimentación

Deseamos hacer notar el fuerte énfasis en el proceso de desarrollo a partir de las prácticas vistas y la ausencia de definiciones acerca del modelo de desarrollo que guíe la transformación de las historias (requerimientos) en código ejecutable.

6.2.4 Flujo de trabajo

La inclusión del cliente establece en XP una comunicación continua dentro del grupo de proyecto que determina la evolución en la dirección de crecimiento de los conceptos esenciales del negocio y la solución. Esto facilita el análisis de las historias, la definición de su alcance y la elaboración de las pruebas que permitirán su verificación [3], [7].

En el aspecto estrictamente técnico se le suma a este proceso el diseño incremental y conducido por las pruebas [10], [11], los cuales hacen que los desarrolladores estén inmersos en un proceso de aprendizaje continuo del negocio y elaboren incrementalmente la solución. Esto consiste en la construcción de tests que plantean el modo de uso de los diferentes componentes previamente a su construcción. A partir del desarrollo de los tests, según este enfoque, se desarrolla el software de manera que las pruebas sean ejecutadas en forma exitosa.

A medida que las iteraciones se suceden y el software escrito crece se van detectando necesidades de cambio y adaptaciones que permitan agregar nuevos componentes derivados de las nuevas funcionalidades. Para que esto sea factible se incorporan las actividades de refactorización, base del diseño incremental. Esta refactorización consiste en cambiar el diseño en determinadas direcciones utilizando criterios y patrones de diseño en pequeños pasos. En adelante utilizaremos de forma indistinta los términos test y prueba.



Figura 6.1. Esquema conceptual que muestra las prácticas emblemáticas de XP

En la figura 6.1 se muestran en forma esquemática las diferentes prácticas de XP. Cuando en los capítulos venideros nos dediquemos al diseño del software nos extenderemos en los temas mencionados de criterios y patrones de diseño.

6.3 Scrum

Esta metodología debe su nombre a la formación de rugby donde los jugadores miembros de un equipo se agrupan y suman sus fuerzas empujando al obstáculo que en ese caso es el equipo adversario [2]. Esta forma de trabajo ataca los problemas asociados a la gestión de proyectos y como tal necesita de un complemento que se ocupe de los aspectos técnicos [5].

En la tabla 6.4 se presentan los componentes de Scrum y su descripción.

Tabla 6.4. Listado de componentes de Scrum	
Componentes de Scrum	
Roles	Descripción
Product Owner	Involucrado por parte del cliente con conocimientos del negocio. Define el producto / sistema a construir.
Scrum Master	Coordinador y facilitador del grupo, acuerda con el Product Owner la operatividad del desarrollo.
Team	Desarrolladores, entre 3 y 7 personas.
Project Manager	Responsable de integrar el proyecto al flujo de trabajo de la organización. Provee recursos.
Activos	Descripción
Product Backlog	Listado priorizado de requerimientos del producto / sistema a construir.
Sprint Backlog	Subconjunto de requerimientos provenientes del Product Backlog a implementar por el team en una iteración (Sprint).
Burndown char	Diagrama que muestra el trabajo sin terminar, se aplica al sprint y también al proyecto completo.
Flujo de trabajo	Descripción
Project Planning	Actividad de planificación del proyecto.
Sprint Planning	Actividad de planificación de una iteración (Sprint).
Sprint	Iteración de duración fija entre 15 y 30 días.
Release	Código funcionando entregado al cliente.
Daily Scrum	Reunión diaria del Team con el Scrum Master de unos minutos de duración en la que se revisan las actividades del día.
Demo Meeting	Culminación del Sprint donde se muestra al Product Owner las funcionalidades incorporadas al producto.
Sprint Retrospective	Reunión del Team y el Scrum Master en la que se reflexiona sobre el Sprint anterior y cuáles son las oportunidades de mejoras.

En la figura 6.2 se muestra en forma esquemática la relación entre los principales activos de la metodología. Se observa que a partir del refinamiento de las tareas necesarias para la implementación de los diferentes requerimientos del Sprint Backlog se arma el contenido de dicho Sprint. La realización de estas tareas durante la ejecución del Sprint y su correspondiente estado se muestra en el Burndown Chart.

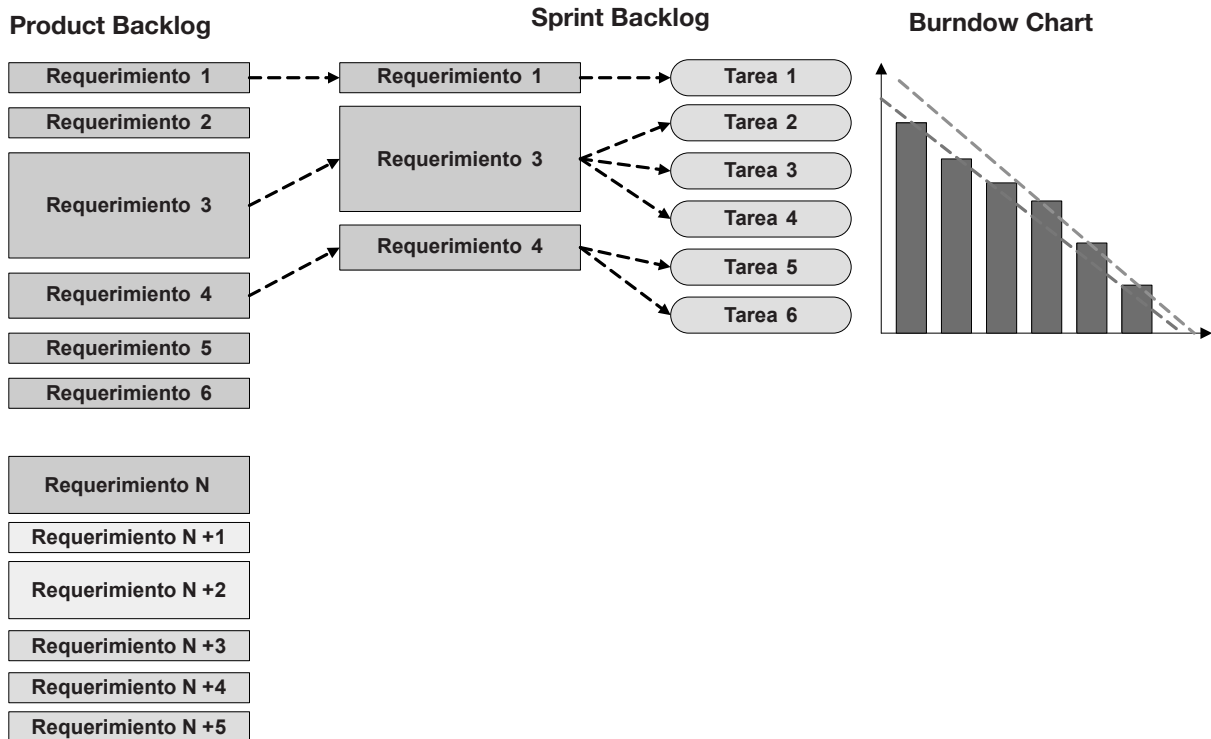


Figura 6.2. Diagrama esquemático de los activos principales de Scrum y sus relaciones

En la figura 6.3 se presenta un esquema de la dinámica de Scrum entre todos sus componentes. Hemos conservado los nombres en inglés de estos componentes, y así lo haremos a lo largo del libro.

La clave del flujo de trabajo es el enfoque en el Sprint Backlog y el seguimiento cercano de su evolución.

El alcance del Sprint Backlog no debe cambiar, sus requerimientos y las tareas asociadas no deben cambiarse durante su ejecución. Esto es así ya que la planificación fue realizada sobre una duración fija de tiempo para el cual se tuvo en cuenta el esfuerzo de cada una de las tareas incluidas y la velocidad de desarrollo del grupo. A diferencia de esto, el alcance del Product Backlog cambia y se adapta a medida que el proyecto avanza. El Product Owner saca y /o agrega nuevos requerimientos, siempre priorizándolos.

Las iteraciones al igual que el proyecto tienen duración fija; cada una debe tener un objetivo y una forma de verificar su logro.

El Scrum Master publica el estado de los Sprint en un Burndown Chart para dar visibilidad al proyecto.

El Team aprende y ajusta su velocidad de producción a medida que el proyecto avanza, lo cual permitirá realizar estimaciones más certeras en futuros Sprints.

Al final de cada Sprint se revisa lo hecho y se trata de mejorar en todos los aspectos.

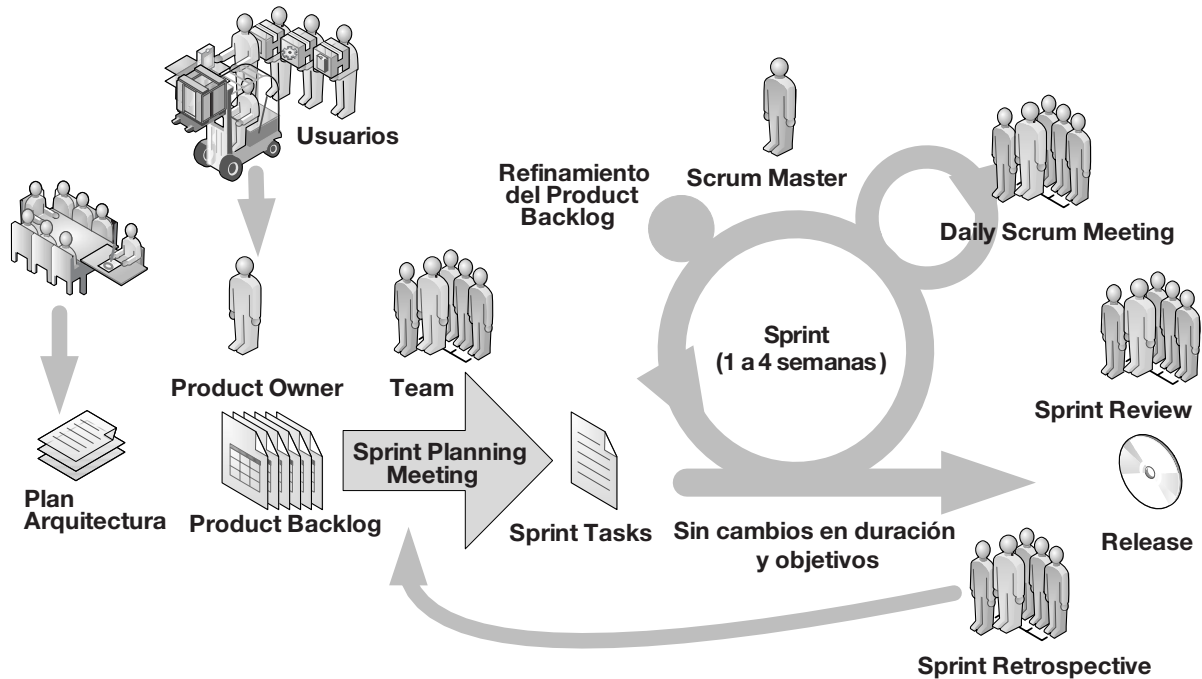


Figura 6.3. Esquema de los diferentes componentes de Scrum y su dinámica

6.4 Integración XP – Scrum

A efectos de implementar una metodología ágil, muchas empresas eligen utilizar Scrum para su gestión y XP para las tareas técnicas del proceso de desarrollo [4], [5].

Es importante mencionar que esta complementación requiere la adaptación de algunas prácticas de alguna de las metodologías. Un ejemplo de ello es la duración de las iteraciones, ambos proponen diferentes duraciones.

En la figura 6.4 se presenta un diagrama conceptual donde se sintetiza el aporte de cada una de las dos formas de trabajo al complemento propuesto.

En la próxima sección se describe un ámbito donde ambas metodologías funcionan juntas.

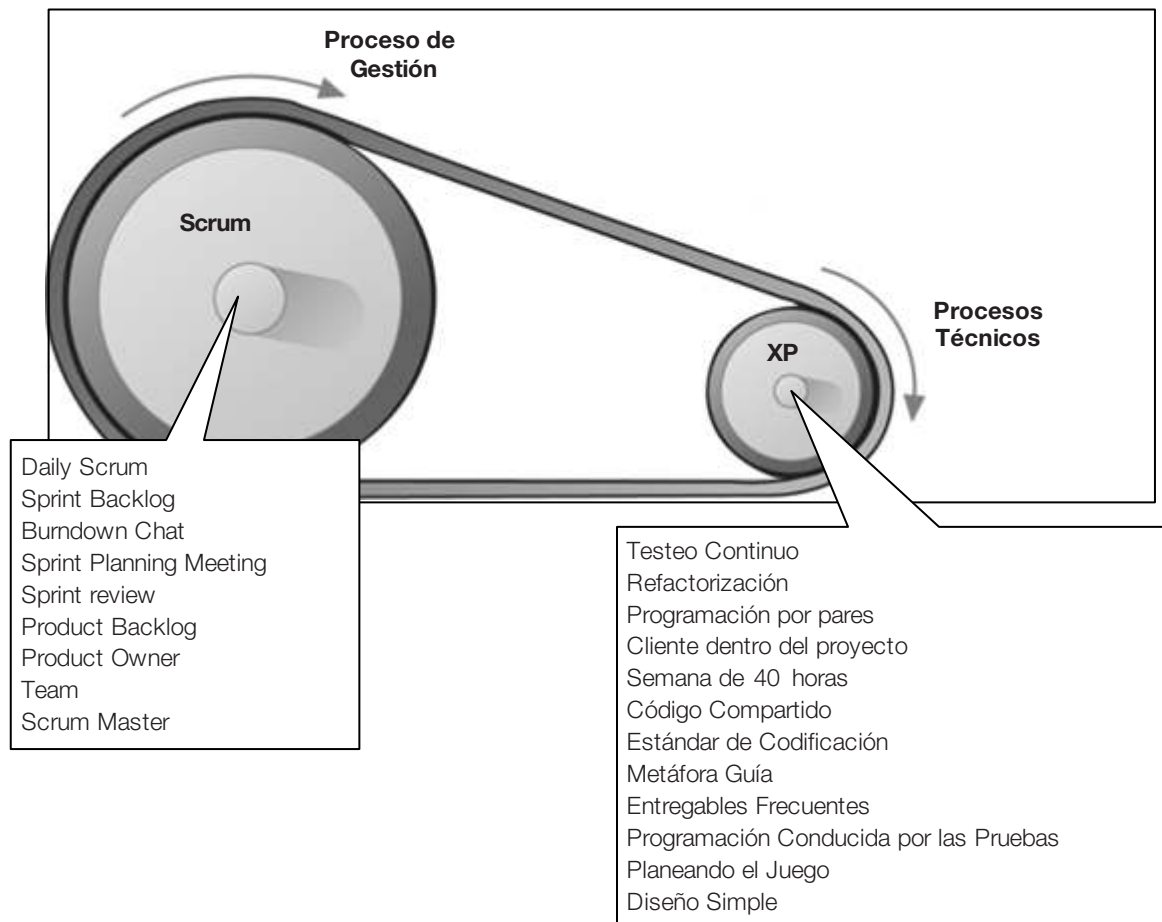


Figura 6.4. Integración para la complementación de Xp y Scrum

6.5 Ágiles

La lectura del manifiesto ágil permite sacar algunas conclusiones acerca del origen y la motivación de la fundación de este movimiento [5]. Cada una de las declaraciones del manifiesto presenta una razón de ser común, la cual está dada por la experiencia fallida en la utilización de las metodologías conducidas por los planes. Esencialmente los factores que provocaron este emergente fueron los proyectos retrasados y con sobrepresupuesto. Situación que no resolvieron las metodologías como el Proceso Unificado de Desarrollo ni siquiera con la incorporación del trabajo iterativo e incremental. Los errores más comunes fueron mencionados en la sección anterior. Se privilegiaron los planes ante el ejercicio de planificación. Se hizo uso de modelos en forma excesiva, es decir, en situaciones donde no eran necesarios. Se enfatizó la realización de las diferentes actividades en lugar de trabajar para lograr objetivos. No se resolvió el problema de la incertidumbre establecida por las estimaciones de alcance. En este escenario fue que surgió el manifiesto ágil.

Por lo tanto es comprensible que se antepongan las personas a los procesos y herramientas, cuando las metodologías establecidas redundaban en definiciones de procesos y procedimientos, y algunas fueron utilizadas por vendedores de herramientas para dar forma a sus productos.

Se entiende también que se valore el software funcionando antes que una extensa documentación.

Es claro que se trató de atacar una de las razones claves del fracaso de los proyectos al anteponer la colaboración del cliente a la negociación contractual.

Por último, promover el cambio ante planes rígidos fue la respuesta a la forma de trabajo establecida.

Sin embargo, deseamos notar que el estado de cosas descrito no se deriva de la estructura y dinámica de las metodologías, sino del criterio con que estas son implementadas en los grupos de desarrollo. Esto último es fácilmente visible en los practicantes de las metodologías ágiles al exacerbar algunos de sus rasgos y desarrollar un criterio erróneo para su utilización. En los cierres fallidos del proyecto del caso testigo hemos tratado de introducir para ambos tipos de metodologías estas malas interpretaciones, como será analizado en la séptima parte del libro.

De la declaración del manifiesto “Software funcionando sobre documentación extensa” podemos extraer la esencia de la forma de trabajo ágil. Se enfoca el proceso en los requerimientos que el sistema en desarrollo debe cumplir. Como se muestra en la figura 6.5 el plan fue reemplazado por el llamado Product Backlog (pila de producto) el cual está constituido por el conjunto de requerimientos a implementar y constituye el alcance del proyecto. De este activo se nutre el proyecto en cada iteración (Sprint). Además es el lugar donde el Product Owner introduce cambios, extrae pedidos e inserta nuevos. Fundamentalmente se trata de implementar en un tiempo y presupuesto preestablecidos la mayor cantidad de estos requerimientos.

6.6 El flujo de trabajo

Cuando un proyecto se inicia ya han sido realizadas tareas que son fundamentales para su vida. Una es por supuesto la venta de los servicios de desarrollo por parte de la desarrolladora y su adquisición por la parte cliente. Debido al rol que en las metodologías ágiles debe asumir uno de los involucrados por parte de esta última organización, los proyectos deben venderse teniendo esto en cuenta. En el capítulo donde trataremos el desarrollo del proyecto del caso testigo del libro profundizaremos estas cuestiones. Por ahora deseamos presentar las metodologías en sus aspectos técnicos y de gestión desde el momento en el ciclo de vida cuando el proyecto se inicia.

6.6.1 Ciclo de vida

El ciclo de vida de un proyecto administrado de esta forma posee una primera etapa de planificación, un conjunto de iteraciones y uno o más entregables.

Planificación del proyecto - En la primera etapa del proyecto se planifican los diferentes aspectos del proyecto tales como:

- Ambiente de desarrollo
- Recursos humanos
- Asignación de roles a los miembros del grupo del proyecto
- Alcance y prioridades de los requerimientos del Product Backlog
- Arquitectura del sistema en desarrollo
- Entregables del proyecto.

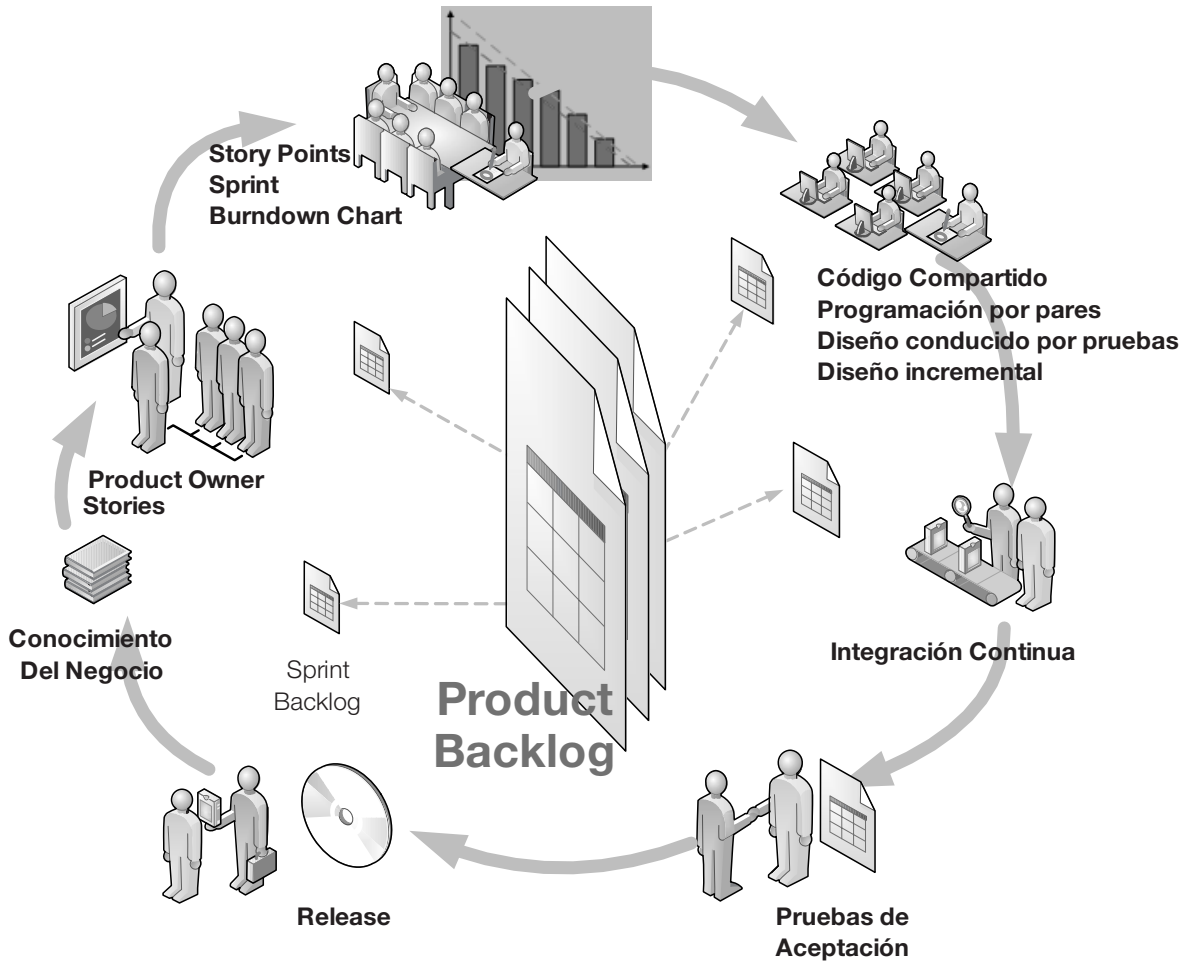


Figura 6.5. Flujo de trabajo en una metodología ágil en la que se observa el enfoque a partir de un Product Backlog

En la figura 6.5 se muestra la realización de estas actividades en relación con el ciclo de vida del proyecto. En la realización de estas tareas participan:

Product Owner: definiendo la funcionalidad y estableciendo prioridades.

Team (desarrolladores): relevando los requerimientos y aprendiendo el negocio construyen las Stories.

Scrum Master: coordinando las tareas y planificando el desarrollo.

Project Manager: integrando el proyecto a la organización.

Con la revisión de las prioridades y el acuerdo del Product Backlog se está en condiciones de planear un Sprint (iteración).

Planificación de los Sprints - Consiste en extraer del Product Backlog un subconjunto de requerimientos a implementar en el Sprint que se planea y refinar las tareas necesarias para la implementación de las Stories. La participación de los roles en estas actividades está enfocada a:

Product Owner: definiendo el alcance en base a las prioridades, fijando el objetivo y participando en el establecimiento de la verificación del Sprint.

Team (desarrolladores): realizando una estimación del esfuerzo (Story Points) en base a las tareas necesarias para la implementación de cada requerimiento (Story) incluido y colaborando en el establecimiento de la verificación del Sprint. Esto se realizará hasta que el Sprint tenga una carga de tareas que el grupo pueda manejar en el intervalo de tiempo fijado para la iteración de acuerdo a la velocidad de trabajo del equipo.

Scrum Master: coordinando las tareas, balanceando las restricciones, administrando los riesgos y planificando el desarrollo.

Desarrollo de los Sprints – Una vez definido el Sprint, se comienza a trabajar sobre él. Para ello cada miembro del equipo se asigna una tarea, marcándola en desarrollo, de esta manera es visible qué tareas no están iniciadas y cuáles sí, además de quién la está resolviendo. Una vez resuelta, el desarrollador la marca como resuelta y vuelve a ver qué tareas están pendientes para asignarse una nueva. Originalmente solía utilizarse una pizarra, como se muestra en la Figura 6.6, donde se adjuntaban tarjetas vinculadas a las Stories. Hoy día la mayoría de las herramientas orientadas a facilitar la gestión de este flujo de trabajo presentan vistas que emulan dicha pizarra. El estado “resuelta” significa que están listas para ser verificadas al término del Sprint.

El Team trabaja realizando programación por pares, diseño conducido por las pruebas, diseño incremental y en un contexto brindado por el código compartido y la integración continua.

Evaluación de los Sprints – Finalizado el Sprint, se prepara un entregable y se hace una reunión de demostración en la cual se le muestra al Product Owner las funcionalidades incorporadas, resultado del Sprint. El Product Owner puede aprobarlo o rechazarlo. También puede decidir continuar o no con los Sprint dando fin al proyecto.

Además, al finalizar un Sprint, el equipo mantiene una reunión de revisión (Retrospective Meeting) en la cual hace una evaluación y acuerda qué mantener y qué mejorar en el próximo Sprint.

En esta instancia ya se está en condiciones de avanzar con la planificación de un nuevo Sprint.

Entregables - Un proyecto incluirá en su desarrollo un conjunto de entregables a los que en Scrum se llaman Release. Los productos generados a lo largo del ciclo de vida del proyecto son entregados y a veces

instalados en el ambiente productivo. Es una práctica común vincular los pagos a estos hitos. Cada Release comprende entre dos y cinco Sprints.

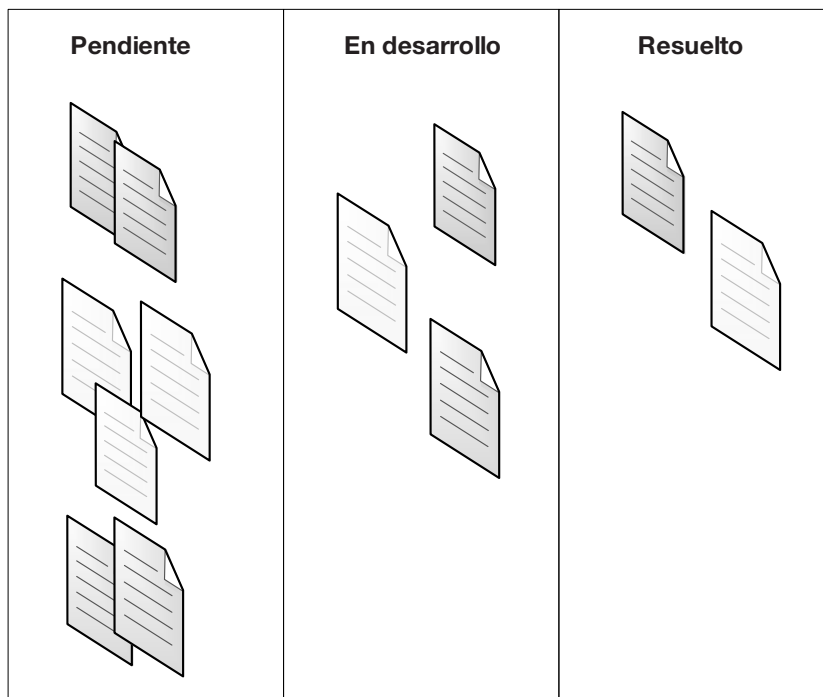


Figura 6.6. Visualización del estado de las tareas de un Sprint en desarrollo

6.7 Organigrama y seguimiento del proyecto

Un aspecto clave de las metodologías ágiles es la gran visibilidad de todos los sucesos en la evolución de los proyectos. Por esta razón el seguimiento se basa fuertemente en realizar en forma disciplinada las actividades que generen la visibilidad necesaria.

Para esto se gestiona el proyecto utilizando los componentes provistos por Scrum, como los son:

- Daily Scrum
- Sprint Review Meeting
- Sprint Retrospectives
- Product Backlog
- Burn down Chart.

Estas actividades fueron descriptas en la tabla 6.4 de este capítulo y serán tratadas cuando se apliquen al seguimiento del caso testigo. En la figura 6.7 se presenta un diagrama esquemático donde se muestran las reuniones mencionadas y los roles participantes. El ámbito marcado con puntos corresponde a las actividades

6.8 Conclusión

En las metodologías ágiles los participantes adhieren a un conjunto de principios y buenas prácticas que constituyen los elementos conductores. Todos los involucrados desarrollan estas prácticas como forma de garantizar la evolución del proyecto. Se trabaja con la inclusión del cliente en el grupo de proyecto con el fin de facilitar la transmisión de conocimientos y los acuerdos.

La esencia de estas formas de trabajo se basa en la maximización del valor agregado entregado a los clientes en forma de software funcionando.

Se enfatizan los procesos por sobre los activos, con excepción del código y se promueve la rápida adaptación a los cambios [5], [6], [7].

6.9 Contenido de la página web de apoyo



6.9.1 Mapa conceptual

6.9.2 Autoevaluaciones

6.9.3 Presentaciones en Power Point (exclusivo para el docente)

6.10 Referencias

- [1] *Agile Software Development: The Cooperative Game*, Second Edition, Alistair Cockburn, Addison Wesley, 2006.
- [2] *Agile Project Management with Scrum*, Ken Schwaber, Microsoft Press, 2004.
- [3] *Planning Extreme Programming*, Kent Beck, Martin Fowler, Addison Wesley, 2000.
- [4] *Agile Management for Software Engineering: Applying the Theory of Constraints for Business Results*, David J. Anderson, Eli Schragenheim, Prentice Hall PTR, 2003.
- [5] *SDLC 3.0 – Beyond a tacit understanding of Agile*, Mark Kennaley, Fourth Medium Press, 2010.
- [6] *Balancing Agility and Discipline: A Guide for the Perplexed*, Barry Boehm, Richard Turner, Addison Wesley, 2003.
- [7] *Agile Estimating and Planning*, Mike Cohn, Prentice hall, 2005.
- [8] *Calidad en el Desarrollo de Software*, Guillermo Pantaleo, Alfaomega, 2011.
- [9] *Lean Software Development: An Agile Toolkit*, Mary Poppendieck, Tom Poppendieck, Addison Wesley, 2002.
- [10] *Refactoring, improving the design of existing code*, Martin Fowler, Addison Wesley, 2002.
- [11] *Test-Driven Development by Example*, Beck K., Addison Wesley, 2003.

El lenguaje UML y sus modos de utilización

Contenido

7.1 Introducción	110
7.2 Evolución histórica	111
7.3 ¿Cómo es utilizado en la comunidad informática?	113
7.4 Perspectivas	114
7.5 Diagramas	117
7.6 Relación con el proceso de desarrollo	118
7.7 Selección del modo de uso	119
7.8 Contenido de la página web de apoyo	119
7.9 Referencias	119

Objetivos

- Qué es el lenguaje UML (Unified Modeling Language)
- Cuáles son los modos de su utilización en el desarrollo de software
- Qué aportan los diagramas de UML a la documentación de un proyecto

Competencias

- Definir la forma de utilización del lenguaje UML como herramienta de relevamiento, análisis, diseño y documentación en un proyecto de desarrollo de software

7.1 Introducción

Hemos seleccionado como medio de documentación de los aspectos técnicos desarrollados a lo largo del libro al lenguaje UML (*Unified Modeling Language*). Lo elegimos por ser el estándar en la industria y los modelos que lo componen serán los que utilizemos por ejemplo en la presentación del proyecto del caso testigo que comenzaremos a desarrollar en el capítulo 10.

7.1.1 ¿Qué es UML?

UML es un lenguaje que está compuesto por un conjunto de diagramas agrupados por un metamodelo que ayuda a especificar y diseñar el software de sistemas; particularmente software orientado a objetos. Por muchos años fue el estándar de hecho de la comunidad informática aunque después de 6 años de su primera versión fue adoptado como tal.

Es un estándar abierto controlado por *Object Management Group* (OMG) [1]. Desde el año 2005 el lenguaje UML es un estándar aprobado por la ISO como ISO/IEC 19501:2005 Information technology — Open Distributed Processing— Unified Modeling Language (UML) Version 1.4.2.

De la misma manera que un constructor de casas o edificios antes de apilar ladrillos elabora un plano, un desarrollador de software antes de escribir código hace un plano que le permita evaluar alternativas de su diseño. Igual que una modista utilizando hojas de papel construye un molde para sus prendas en forma previa a cortar la tela y coserla, un desarrollador de software necesita elaborar un modelo que describa los diferentes aspectos del problema a resolver y de la solución propuesta. UML es un lenguaje de especificación y diseño que permite a los desarrolladores construir modelos para estudiar, analizar y decidir acerca de la estructura y el comportamiento que propondrá como solución y en definitiva cómo estará compuesto y cómo se comportará el código que implementará dicha solución.

7.1.2 Preguntas frecuentes

Las preguntas que deseamos responder son las que con frecuencia hemos escuchado en personas sin experiencia en la utilización de los diagramas de UML. Basaremos nuestra presentación en las necesidades derivadas de la complejidad del problema a resolver y en la experiencia del desarrollador.

Estas preguntas son:

1. ¿Debo construir un Diagrama/Modelo para cada aspecto del problema?
2. ¿Debo construir un Diagrama/Modelo para cada aspecto de la solución?
3. ¿Cuáles diagramas son necesarios construir?
4. ¿Los diagramas que debo construir dependen de la metodología de desarrollo que utilizamos?
5. ¿En qué momento del ciclo de vida del proyecto debo construirlos?
6. ¿Quién construye cada modelo? ¿Qué rol?
7. ¿Cuándo se utilizan los diagramas a lo largo del ciclo de vida del proyecto?
8. ¿Qué debo incluir en cada diagrama? ¿Cuán detallado deben ser?
9. ¿Qué aporta cada diagrama?
10. ¿Cómo relaciono los diagramas?
11. ¿Cómo los mantenemos actualizados ante cambios?
12. ¿Qué herramientas puedo utilizar?

7.2 Evolución histórica

Durante muchos años no existió un estándar que permitiera documentar la especificación y el diseño del software. Los miembros de distintos grupos de desarrollo tenían un obstáculo cada vez que debían intercambiar información. Alrededor del año 1995 ya se había instalado la programación orientada a objetos como estándar en la industria a través del uso del lenguaje C++. Sin embargo no había un estándar de especificación y diseño, lo que sí había eran distintas propuestas en tal sentido. Entre las más conocidas podemos mencionar los diagramas de Booch [2], [3] y Rumbaugh [4]. Tampoco había una metodología estándar definida de desarrollo de software esencialmente inspirada en el nuevo paradigma de entonces, la programación orientada a objetos. Por esos días ambas propuestas fueron una mezcla de lenguaje y forma de usarlo, es decir una metodología.

En la figura 7.1.a se muestra un diagrama de clases utilizando la notación de Booch [2], el mismo diagrama se muestra en la figura 7.1.b utilizando la notación de Rumbaugh [4] y en la figura 7.1.c utilizando UML.

Años después estas propuestas fueron fundidas con la inclusión de los casos de uso aportados por Ivar Jacobson y sus colegas [6], en lo que se llamó UML. Fueron popularizados como “los tres amigos” y la herramienta que produjo la empresa que los reunió, Rational, fue la primera en soportar dicho lenguaje. De esta unión además fueron publicados dos libros, uno que describía el lenguaje [7] y otro la metodología propuesta que se llamó *Proceso Unificado de Desarrollo de Software* [8].

El lenguaje ha tenido algunos cambios desde su versión 1.1 hasta la 1.5. Hoy día la vigente es la 2.0, la cual ha incorporado nuevos diagramas orientados a construir especificaciones de procesos de negocio. Estos diagramas son los utilizados para hacer modelado de flujo de trabajo y procesos en lo que se llama BPM (*Business Process Modeling*). También ha incorporado diagramas temporales en los que se pueden especificar restricciones de tiempo en diagramas de objetos.

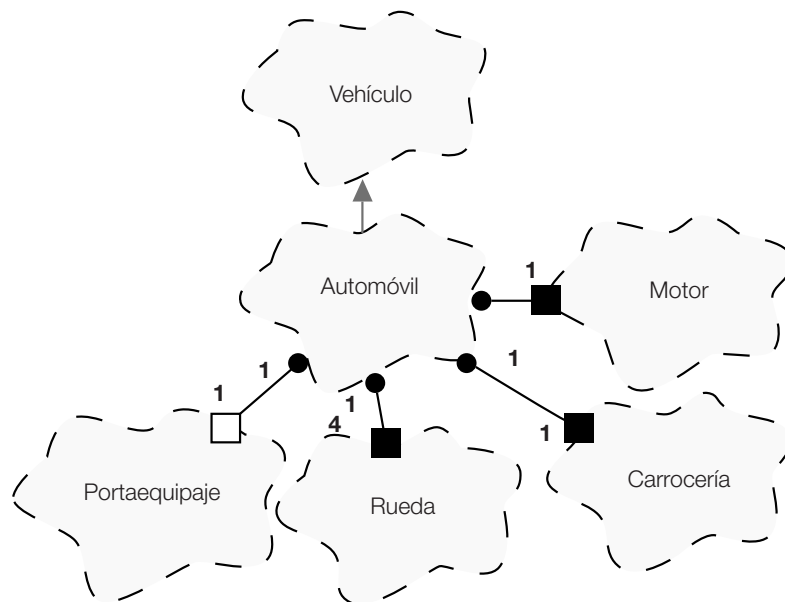


Figura 7.1.a Diagrama de clases en notación de Booch

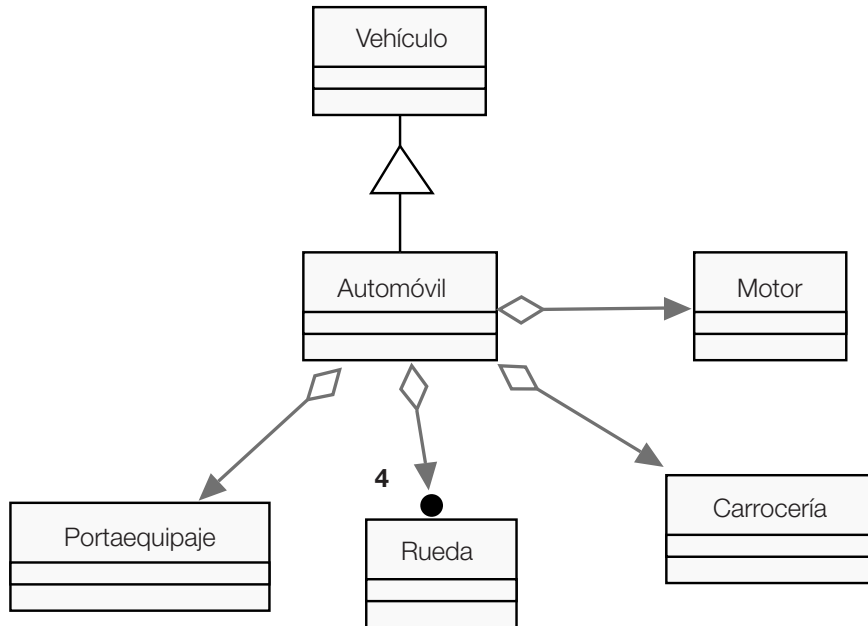


Figura 7.1.b Diagrama de clases en notación de Rumbaugh (OMT)

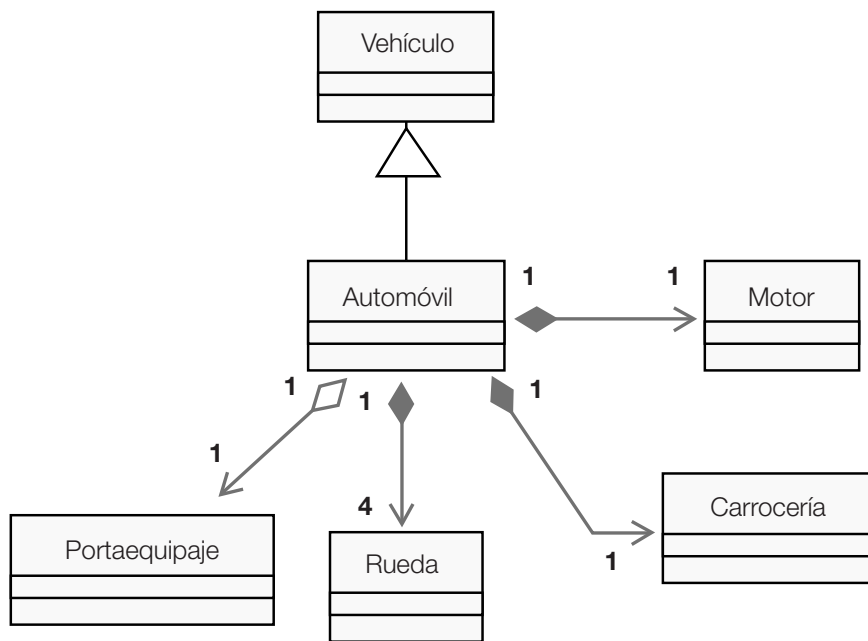


Figura 7.1.c Diagrama de clases en UML

7.3 ¿Cómo es utilizado en la comunidad informática?

Antes de responder las preguntas planteadas, haremos un repaso de cómo es utilizado UML [5] y a partir de este ensayaremos las respuestas.

7.3.1 Dibujo (*Sketch*)

Usado por los desarrolladores para comunicar diferentes aspectos de un sistema. Definición de aspectos de diseño en reuniones colaborativas. Se utilizan herramientas o se hacen a mano alzada. No interesan detalles aportados por el lenguaje. Sólo se incorporan aquellos que resulten relevantes al objetivo con que se construyó el diagrama.

7.3.2 Plan (*Blueprint*)

Usado como herramienta desde la cual se comprende un sistema a desarrollar (diagramas previos al código) o ya desarrollado (documentan código ya existente - ingeniería inversa). Interesan los detalles ya que de allí se guiarán los desarrolladores para escribir el código casi mecánicamente.

Se utilizan herramientas de tipo CASE (*Computer Aided Software Engineering*) es decir, asistida por computadoras.

7.3.3 Lenguaje de programación

Usado como lenguaje de programación a partir del uso de herramientas sofisticadas. Interesan todos los detalles. Generadores de código sustituyen a los programadores. Ejemplo: *Model Driven Architecture* (MDA), *Platform Independent Model* (PIM), *Platform Specific Model* (PSM).

Es importante para el desarrollador no confundir el modo en que está haciendo uso de UML y ubicarse así en la perspectiva correcta. La mayor cantidad de fallas en su utilización que hemos observado se debió a que en el mismo proyecto se usaba en la modalidad Dibujo para algunos diagramas y se pretendía utilizar como Plan para otros. Los diagramas así contruidos generan desconcierto en los miembros de los grupos de desarrollo que ante esta situación desestiman su uso y padecen su carencia.

Su uso más difundido es en la modalidad Dibujo. Esto es debido al objetivo con que se construyen los diagramas y en general toda documentación. En el seno del grupo de desarrollo el objetivo está dado por la necesidad de comunicación entre los miembros, para lo cual alcanza su uso en esta modalidad. También alcanzan los diagramas así contruidos como documentación a futuro, para realizar mantenimiento del sistema durante su vida útil en servicio. Sin embargo deseamos refinar este concepto y para ello distinguiremos dos perspectivas distintas que determinarán dentro de esta modalidad de uso la información a incorporar en los diferentes diagramas.

7.4 Perspectivas

Las dos perspectivas que mencionamos están motivadas por las dos diferentes vistas que los desarrolladores necesitan en dos momentos del ciclo de vida del proyecto de desarrollo [5]. Estas son independientes de la metodología utilizada. Están determinadas por las diferentes preocupaciones de los desarrolladores en los momentos mencionados. Estos son la comprensión del problema a resolver y la propuesta de una solución. Martin Fowler [5] llama a estas perspectivas *conceptual* y *de software*. Nosotros compartimos esta visión pero las llamaremos *problema* y *solución*.

7.4.1 Problema (Concepción y Análisis)

La preocupación y el objetivo de los primeros momentos de cualquier proyecto son entender cómo los usuarios interactuarán con el sistema, entender las reglas del negocio, definir un lenguaje común y determinar el alcance del proyecto. Para la realización de estas tareas la utilización de UML para expresar conceptos es muy importante. Más que la precisión se necesita la posibilidad de concebir y analizar conceptos.

UML ofrece diagramas de casos de uso para mostrar la interacción de los actores con el sistema, diagramas de clases para la perspectiva conceptual, diagramas de actividad para modelar flujos de trabajo y diagramas de estado para mostrar el ciclo de vida de los conceptos extraídos del dominio del problema.

7.4.2 Solución (Diseño)

La otra perspectiva está dada por la elaboración de una solución. La preocupación es elaborar un diseño que sea quizás escalable, flexible de cambiar, fácil de probar, etc. Para definir este diseño queremos ser más precisos y nos ayudamos de diagramas de clases con mayor cantidad de adornos (detalles) para definir la estructura de nuestro sistema, diagramas de secuencia para mostrar su comportamiento, diagramas de paquetes para realizar particiones y separar incumbencias, diagramas de estado para mostrar ciclos de vida y de componentes para documentar instalación en distintos ambientes. Todos estos diagramas definen la estructura y el comportamiento del código, por esta razón en *UML Distilled: A Brief Guide to the Standard Object Modeling Language* [5] la llaman *software*. Si bien proponemos la utilización en la modalidad Dibujo, queda claro que en esta perspectiva detallaremos aspectos que en la anterior no era necesario.

7.4.3 Respuestas a las preguntas frecuentes

A continuación respondemos las preguntas que nos formulamos en una sección anterior.

1. ¿Debo construir un Diagrama/Modelo para cada aspecto del problema?
Por lo general los sistemas de software, por las características del problema a resolver y de su solución, enfatizan algunos aspectos en desmedro de otros. Esto implica que no siempre será necesario construir todos los diagramas. Por ejemplo, para un sistema complejo con varias y diferentes áreas de negocio seguramente será útil construir un diagrama de paquetes que muestre la distribución de las funcionalidades en cada uno de ellos. Además son indicados los diagramas de actividad en estos

casos, en los que puede documentarse el flujo de información entre los diferentes roles. En el caso de un sistema pequeño estos modelos no aportan mayor información.

2. ¿Debo construir un Diagrama/Modelo para cada aspecto de la solución?
Siguiendo el mismo razonamiento que en la pregunta anterior, no siempre los diagramas aportan información de utilidad. Por ejemplo, en modelos de clases simples, los diagramas de clases son suficientes. Sin embargo, cuando estas clases participan de relaciones con otras clases, y estas son complejas, es necesario entender cómo interactúan las instancias de estas clases en memoria. En estos casos son de utilidad los diagramas de secuencia y / o comunicación que muestran los aspectos dinámicos.
3. ¿Cuáles diagramas son necesarios construir?
Siguiendo el criterio de que la documentación a construir es aquella que se necesita y que el objetivo de su construcción es la comunicación, los diagramas que deben construirse son aquellos que comunican información acerca del problema o la solución. Muchas veces dos o más diagramas muestran la misma información, en estos casos debemos optar por aquel que lo haga de forma más clara.
4. ¿Los diagramas que debo construir dependen de la metodología de desarrollo que utilizamos?
La respuesta a esta pregunta podría ser no. Sin embargo necesita una aclaración. Algunas metodologías proponen, además de la forma de trabajo, una forma de documentar el proyecto. Por ejemplo, el Proceso Unificado de Desarrollo basa el trabajo con los requerimientos funcionales en los casos de uso. Otras no lo hacen; por ejemplo, algunas de las metodologías ágiles. Nuestra experiencia es que se puede elegir una forma de trabajo y una forma de documentación de manera que ambas sean independientes. De hecho, actualmente estamos trabajando con una empresa que eligió como su metodología Scrum y documenta los requerimientos utilizando casos de uso en lugar de tarjetas con storyboards.
5. ¿En qué momento del ciclo de vida del proyecto debo construirlos?
Independientemente de la metodología elegida, cualquiera sea, en los primeros momentos de un proyecto de desarrollo se busca como objetivo achicar los riesgos del negocio. Esto es, obtener un conocimiento del negocio o dominio que el sistema a desarrollar debe resolver. Este conocimiento no es técnico, sino conceptual. Construiremos entonces los diagramas desde la perspectiva del problema, conceptual según Martin Fowler [5]. Documentaremos las entidades más importantes como clases, las reglas de negocio como responsabilidades a validar por estas clases, las funcionalidades más importantes como casos de uso, los flujos de información como diagramas de actividad. Cuando el proyecto avance y contemos con un modelo del negocio, vendrá entonces el momento de definir la arquitectura del sistema y construiremos para ello diagramas de paquetes, componentes y nodos. También diagramas de secuencia que muestren las interacciones entre los diferentes componentes. Luego diseñaremos el interior de cada paquete y entonces agregaremos nuevos diagramas de clases, ahora con el objetivo de hacer al sistema escalable, flexible de cambiar y mantener.
6. ¿Quién construye cada modelo? ¿Qué rol?
La construcción de los diferentes modelos está a cargo de los diferentes roles dentro de un grupo de desarrollo. Por lo general los analistas construyen diagramas de casos de uso, actividad y clases de dominio para sus tareas de relevar y analizar requerimientos; el arquitecto los relacionados a documentar la arquitectura y los diagramas de diseño el resto de los desarrolladores.

7. ¿Cuándo se utilizan los diagramas a lo largo del ciclo de vida del proyecto?
Los diagramas se utilizan desde que son construidos hasta que el proyecto finaliza. Se construyen cuando surge la necesidad de hacerlo y luego se consultan durante el ciclo de vida del proyecto.
8. ¿Qué debo incluir en cada diagrama? ¿Cuán detallado deben ser?
Esta es la pregunta cuya respuesta está directamente alineada con el uso de UML en la modalidad Dibujo. Si deseamos mostrar que una determinada clase complementa a otra en un diseño, basta con mostrar sus relaciones, cardinalidad y los métodos que tienen la responsabilidad de la implementación de dicho complemento. Seguramente ambas clases tendrán una cantidad de atributos y otros métodos que no son determinantes y no agregan información acerca del aspecto que se busca comunicar. Por lo tanto esos detalles (adornos) no se incluyen. De igual forma es muy saludable particionar los diagramas por contextos a efectos de mantenerlos manejables, por lo tanto en el diagrama anterior no incluiríamos otras clases asociadas a las dos anteriores.
9. ¿Qué aporta cada diagrama?
Cada diagrama es una vista diferente del problema y la solución, algunos muestran aspectos dinámicos, otros estáticos, y en general si se construyen por contextos muestran las distintas partes que componen a ambos. En un próximo capítulo, cuando presentemos cada diagrama, nos referimos al aporte de información de cada uno en detalle.
10. ¿Cómo relaciono los diagramas?
Al constituir diferentes vistas del mismo sistema, todos los diagramas pueden y deben relacionarse. Por ejemplo, las actividades de un diagrama de actividad seguramente son casos de uso de un diagrama de tal tipo; los objetos de un diagrama de secuencia seguramente se pueden relacionar con las clases de las cuales proviene; las clases pueden relacionarse con los casos de uso que implementan, etc. Este tipo de relaciones se denominan *trazas* y por lo general las herramientas CASE permiten administraras. Son de gran utilidad al momento de evaluar el impacto de cambios en un sistema grande. La trazabilidad entre los diferentes diagramas es una ayuda importante durante el ciclo de vida del desarrollo que permite establecer relaciones entre los componentes.
11. ¿Cómo los mantenemos actualizados ante cambios?
Esta es la pregunta que se hacen muchos desarrolladores y como no hallan respuesta deciden no documentar sus diseños. Creemos que es un síntoma de no haberse decidido por uno de los modos de uso de UML. Si hubiesen elegido por alguno de los otros dos, deberían mantener actualizada todo el tiempo la documentación, ya que de allí se genera el código. En la modalidad Dibujo, por incluir solo lo relevante, es raro que la documentación quede en algún momento desactualizada. Si sucede esto significa que ha habido un cambio importante y por lo tanto se justifica el esfuerzo de actualizarla. En los casos en que además se necesite contar con documentación que refleje detalles de la implementación, se puede construir utilizando herramientas como Doxygen [11], que la elaboran en forma inversa a partir del código ya escrito.
12. ¿Qué herramientas puedo utilizar?
Existen en el mercado una gran variedad de herramientas para la construcción de diagramas de UML. Estas varían en funcionalidad y por supuesto en precio. Las más elaboradas además están integradas con otras que permiten realizar tareas de gestión de los proyectos. También las hay de tipo open

source y libres. Citamos algunas muy conocidas como Modelio [13], ArgoUML [14], Rational Rose [9] y Enterprise Architect [10].

7.4.4 Madurez y nivel de conocimiento de los desarrolladores

Es importante mencionar que un aspecto a considerar al momento de decidir qué documentar y qué no en un proyecto es la experiencia de los miembros del grupo de desarrollo. Seguramente cuando en él haya personas con poca experiencia y menor conocimiento deberá documentarse con más detalle algunos aspectos del sistema en desarrollo. De esta forma la documentación cumplirá acabadamente con uno de los objetivos mencionados en el inicio del capítulo que es comunicar. Esta comunicación contará con más información y más detallada a efectos de soportar la falta de experiencia y conocimiento.

7.5 Diagramas

En la figura 7.2 se muestran los diferentes diagramas que componen el conjunto de diagramas de UML.

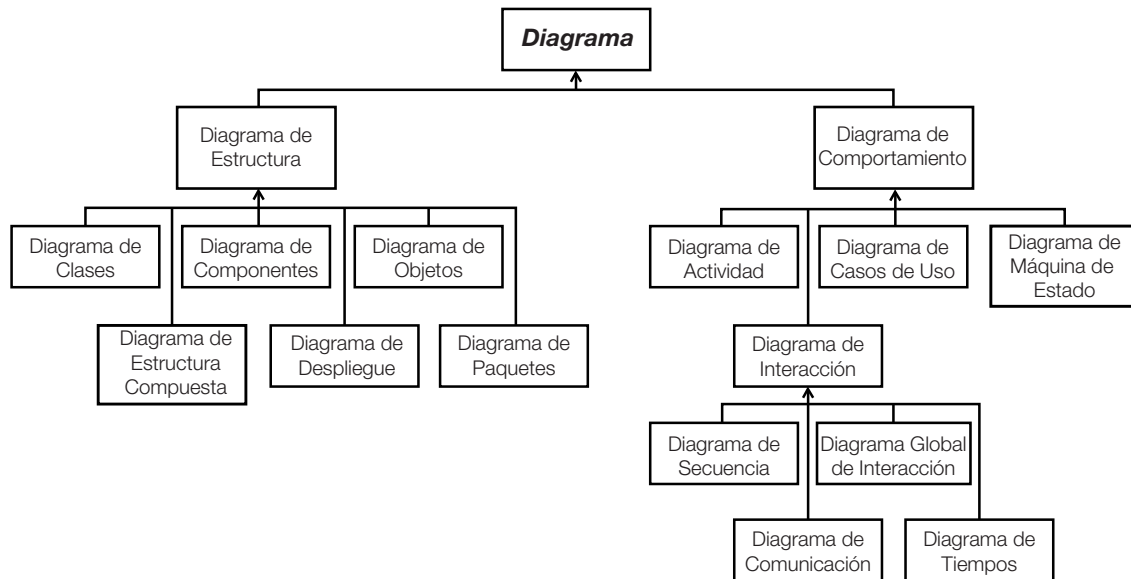


Figura 7.2. Diagramas del lenguaje UML (tomado de Wikipedia)

En un próximo capítulo presentamos cada diagrama en detalle, a continuación describimos sus propósitos.

7.5.1 Diagramas Estáticos

- **Diagramas de Clases:** Un diagrama de clases describe la estructura de un sistema. Muestra las clases, sus atributos y métodos, y las relaciones entre ellas.

- **Diagramas de Objetos:** Un diagrama de objetos muestra una vista parcial o completa de la estructura de un sistema en un momento determinado a partir de las instancias de las clases del sistema.
- **Diagramas de Paquetes:** Un diagrama de paquetes muestra cómo un sistema se divide en agrupaciones lógicas mostrando las dependencias entre ellas. El criterio de agrupamiento está dado para los paquetes que encapsulan funcionalidad por la cohesión desde el punto de vista del negocio y para los paquetes que encapsulan aspectos de la tecnología por la cohesión de la prestación de servicios.
- **Diagramas de Componentes:** Un diagrama de componentes en el Lenguaje de Modelado Unificado representa cómo un sistema de software se divide en componentes y muestra las dependencias entre estos. Estos componentes pueden ser librerías estáticas, librerías dinámicas, etcétera.
- **Diagramas de Instalación:** Un diagrama de nodos sirve para modelar el hardware utilizado en las implementaciones del sistema, los componentes desplegados en el hardware, y las asociaciones entre ellos. Cabe mencionar que otros nombres para este mismo diagrama que suelen usarse son "despliegue" y "nodos".

7.5.2 Diagramas Dinámicos

- **Diagramas de Actividad:** Un diagrama de actividad muestra un flujo de información entre roles de negocio o entre componentes de la tecnología.
- **Diagramas de Estado:** Un diagrama de estados muestra los estados posibles de las diferentes entidades, los eventos por ellas recibidos y su transición entre ellos.
- **Diagramas de Casos de Uso:** Un diagrama de casos de uso muestra la funcionalidad ofrecida por el sistema desde la perspectiva de los actores externos y la relación entre estas funcionalidades.
- **Diagramas de Comunicación:** Un diagrama de comunicación expresa la interacción entre componentes en términos de mensajes intercambiados.
- **Diagramas de Secuencia:** Un diagrama de secuencia muestra la interacción entre componentes y el orden temporal en que estos mensajes son enviados y recibidos.

7.6 Relación con el proceso de desarrollo

Con el advenimiento de las metodologías ágiles que enfatizan, como se mencionó en capítulos anteriores, la comunicación verbal in situ frente a la documentación, se generó en la comunidad un falso conflicto.

Se creó la imagen de que si la metodología era ágil no era necesario documentar. Esta situación es comentada en *Is Design Dead?* [12] donde Martin Fowler cuenta que hasta Kent Beck hace un borrador de parte de sus diseños aunque sin ninguna notación, aun cuando este es una de las dos personas que concibieron Extreme Programming, la primera metodología no conducida por los planes.

Es verdad que algunas metodologías conducidas por los planes enfatizan parte de su flujo de trabajo e información en la documentación generada y otras no lo hacen, como acabamos de comentar. Pero también es cierto que en todas ellas, estén basadas en lo que fuere, cuando participa un grupo de personas es necesario comunicar y acordar, y para esto es que la documentación debe ser construida. Es verdad que existen formas alternativas de documentación a UML, pero también es verdad que cuanto más estándar sean los modelos que se utilicen en la construcción del software más robusto este será. UML es hoy el estándar de la industria, tiene detractores pero también un conjunto de organizaciones que trabajan para mejorarlo, como lo han hecho a lo largo de sus varias versiones.

7.7 Selección del modo de uso

Hemos enfatizado la utilización de UML en el modo Dibujo (*Sketch*) frente a la modalidad Plan (*Blueprint*) y Lenguaje de Programación porque seguramente será la primera forma y la más directa en que los lectores de este libro lo utilizarán. Aún falta tiempo a criterio de estos autores para que el concepto de UML como Lenguaje de Programación en el diseño de arquitecturas conducidas por los modelos (*Model Driven Architecture* - MDA) se utilice como herramienta corriente de desarrollo. Respecto de la utilización como Plan (*Blueprint*) lo hemos observado en algunos proyectos de gran volumen donde habían sido definidos roles de diseñadores y programadores de forma separada en el contexto de proyectos remotos. El resultado de su utilización de esta forma no es eficiente y se convierte a la larga en una restricción para los desarrolladores.

7.8 Contenido de la página web de apoyo



7.8.1 Mapa conceptual

7.8.2 Autoevaluaciones

7.8.3 Presentaciones en Power Point (exclusivo para el docente)

7.9 Referencias

- [1] Object Management Group (OMG), <http://www.uml.org/>.
- [2] *Object-oriented Analysis and Design with Applications*, 2nd ed., Redwood City: Benjamin Cummings, Grady Booch, 1993.
- [3] *Designing Object-Oriented C++ Applications using the Booch Method*, Robert C. Martin, Prentice-Hall, 1995.
- [4] *Object Oriented Modeling and Design*, James Rumbaugh, Prentice Hall, 1991.
- [5] *UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd Edition)*, Martin Fowler, Addison-Wesley, 2005.
- [6] *Object-Oriented Software Engineering: A Use Case Driven Approach*, Ivar Jacobson, Magnus Christerson, Patrik Jonsson, Gunnar Overgaard, Addison-Wesley, 1992.

- [7] *Unified Modeling Language-User's Guide*, Grady Booch, Jim Rumbaugh, and Ivar Jacobson, Addison-Wesley, 1999.
- [8] *The Unified Software Development Process*, Ivar Jacobson, Grady Booch, James Rumbaugh, Addison-Wesley Professional, 1999.
- [9] *Rational Rose*, <http://www-01.ibm.com/software/rational/>.
- [10] *Enterprise Architect*, <http://www.sparxsystems.com.ar/>.
- [11] *Doxygen*, <http://www.stack.nl/~dimitri/doxygen/>.
- [12] *Is Design Dead?*, Martin Fowler, <http://martinfowler.com/articles/designDead.html>.
- [13] *Modelio*, <http://www.modelio.org/>.
- [14] *ArgoUML*, <http://argouml.tigris.org/>.

El lenguaje UML Diagramas estáticos

Contenido

8.1 Introducción	122
8.2 Diagramas	122
8.3 Clases	123
8.4 Paquetes	124
8.5 Componentes	125
8.6 Instalación	126
8.7 Ejemplo	127
8.8 Conclusión	131
8.9 Contenido de la página web de apoyo	131
8.10 Referencias	131

Objetivos

- Aprender qué aspectos de un sistema documentan los diagramas estáticos de UML
- Cuáles son los diagramas estáticos de UML
- Qué relación tienen estos diagramas entre ellos
- Qué relación tienen estos diagramas con las actividades de un proyecto
- Cuándo, en qué momento del ciclo de vida de un proyecto se construyen los diferentes diagramas

Competencias

- Establecer un criterio de utilización del lenguaje UML como medio de documentación de los aspectos estáticos de un proyecto de desarrollo de software

8.1 Introducción

Por ser este un libro de texto de Ingeniería de Software, los autores hemos analizado con detenimiento si el lenguaje UML debía o no ser incluido como tema del texto. De ser incluido y por ser este un lenguaje, ¿por qué entonces no incluir otros de los muchos lenguajes de software conocidos? ¿Con qué detalle debía ser presentado? Las respuestas a estas preguntas son los tres capítulos del tema en el contenido del libro.

Hemos decidido tratar el tema no desde el lado técnico, tal como detalles de la definición del lenguaje, sino cuáles son las actividades que son realizadas a lo largo del ciclo de vida de un proyecto que utiliza alguno de los diagramas que componen el lenguaje UML. Una lectura de los aspectos técnicos en cuanto a su definición puede verse en [2] y [4].

Deseamos enfatizar las diferencias entre un proceso de desarrollo que utiliza UML y el lenguaje UML en sí mismo. Esta es una confusión común en mucha gente del ambiente del desarrollo de software. Se pueden utilizar los diagramas de UML como lenguaje de especificación y diseño de software con cualquier metodología, conducida por los planes o ágiles. Desde esta perspectiva es que desarrollamos los capítulos que siguen. La relación entre los diferentes diagramas y su uso a lo largo del ciclo de vida del proyecto puede verse en [1], [3] y [5]. Una herramienta para construir diagramas UML puede consultarse en [6].

8.2 Diagramas

Los diagramas estáticos de UML son los que se listan a continuación:

- Paquetes
- Clases
- Componentes
- Instalación

Estos diagramas se utilizan en diferentes momentos del proyecto y con diferentes objetivos. Todos ellos muestran vistas estáticas del sistema a resolver.

De cada uno de ellos y dependiendo del tamaño y complejidad del proyecto podremos contar con varios, cada uno mostrando distintas vistas de interés.

Cubren aspectos relacionados a los primeros momentos del ciclo de vida del proyecto como aquellos vinculados a las últimas tareas que se realizan.

Permiten relacionar desde los requerimientos funcionales, pasando por la arquitectura, el diseño de los módulos y la distribución de los componentes asociados hasta la configuración del hardware en el cual serán instalados y ejecutados.

8.3 Clases

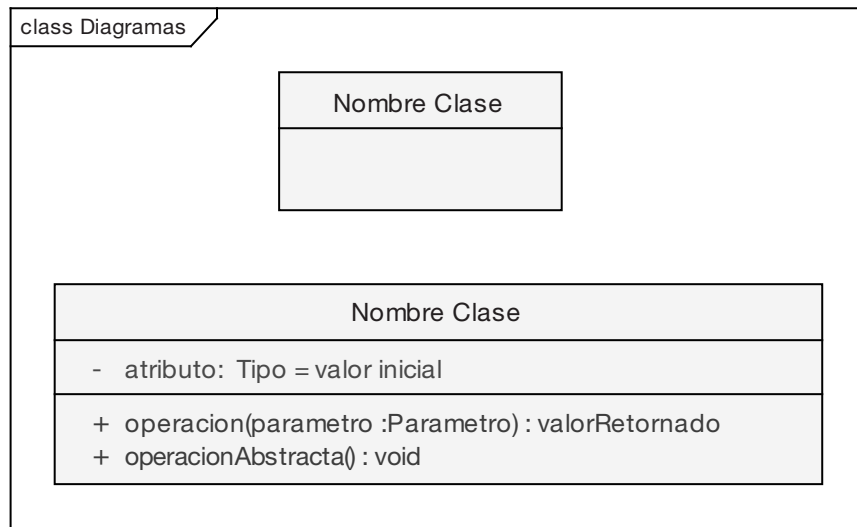


Figura 8.1. Notación de UML para las clases

El diagrama de clases muestra los componentes esenciales de un modelo orientado a objetos. En ellos se muestran las clases con algunos de sus atributos y métodos más importantes, utilizados como definimos en el capítulo 7 en modo Dibujo (*Sketch*). Seguramente contaremos en el diseño de un sistema con varios gráficos en los cuales puede aparecer la misma clase. Cada uno de los diagramas debe haber sido construido con objetivos diferentes, y la clase en cuestión estará presente en cada uno de ellos jugando los diferentes roles que se le asignaron. La estrategia recomendada en la documentación de las clases es construir un diagrama por cada contexto del modelo y así serán manejables y legibles. En la figura 8.1 se muestra la notación de UML para una clase y en la figura 8.2 un diagrama de clases donde se muestran las relaciones entre clases.

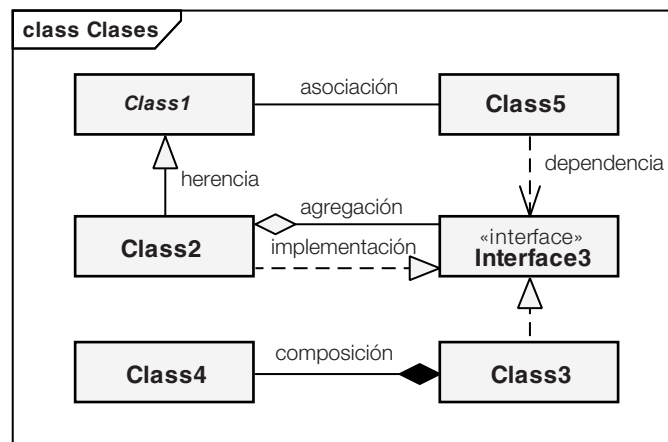


Figura 8.2 . Diagrama de clases

En la figura 8.2 se muestran los diferentes tipos de relaciones entre clases. Puede verse que la clase Class2 está relacionada a la clase Class1 mediante una relación de herencia o extensión. Esto indica, para los lenguajes orientados a objetos con tipos como C++ y Java, que una instancia (objeto) de esta clase hija **es de tipo** Class1 como su clase padre. Es decir que especializa parte del comportamiento de la clase padre.

La clase Class2, de la misma figura, tiene una referencia a una interfaz de tipo Interface3, lo cual indica que en algún momento de su existencia a las instancias de esta clase le será provisto un objeto que implementará dicha interfaz. Por ejemplo, esta referencia podría ser un objeto de tipo Class3 que, como se muestra en la figura, **implementa** dicha interfaz.

La clase Class5 posee una dependencia del tipo Interface3, lo que indica que, por ejemplo, un método de esta usa un objeto implementando ese tipo el cual le es provisto por parámetro.

Las instancias de la clase Class3 **tienen** (forman parte de) un objeto de tipo Class2.

Estos diagramas son de utilidad en las fases tempranas del ciclo de vida cuando se construye un modelo de dominio que representa los conceptos extraídos del dominio del problema y su comportamiento, es decir un modelo de negocio. También son los diagramas por excelencia para la documentación del diseño que soportará el modelo de negocio del sistema. Esta vista estática frecuentemente es acompañada por una vista dinámica que muestra cómo interactúan las instancias de estas clases, tema que trataremos en el capítulo 9.

8.4 Paquetes

El mecanismo de descomposición es uno de los tres que necesitamos utilizar en el proceso de análisis y construcción de un sistema, y nos permite descomponer la complejidad en problemas tratables. El producto de la aplicación de este mecanismo son los subsistemas que componen el sistema en desarrollo. Un artefacto de utilidad para empaquetar estas partes del desarrollo son los paquetes. En la figura 8.3 se muestra cómo se indican en UML.

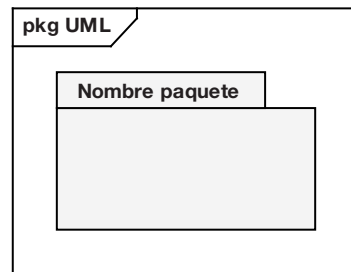


Figura 8.3. Notación de paquete en UML

Los paquetes los utilizamos por primera vez en la concepción de un sistema al momento de separar el modelo de negocio en diferentes áreas o contextos. En ese momento son de utilidad porque nos permiten visualizar el primer esbozo de la arquitectura del sistema. En ellos empaquetamos los casos de uso con el criterio de cohesión derivado de la pertenencia al área de negocio o contexto asociado.

En momentos posteriores y cuando estemos elaborando el diseño de nuestros subsistemas, los paquetes pasarán a ser los contenedores de las clases del diseño.

Definiremos interfaces a partir de las cuales estos paquetes se relacionarán y unos usarán a los otros del modo que lo defina la arquitectura seleccionada.

En la figura 8.4 se muestra un paquete empaquetando casos de uso y relacionado con el paquete que lo implementa.

En la figura 8.5 se muestran dos paquetes relacionados a partir de una interfaz, el paquete C usa al B importando a su definición e implementando la interfaz correspondiente. Estos dos paquetes podrían representar dos subsistemas del mismo sistema.

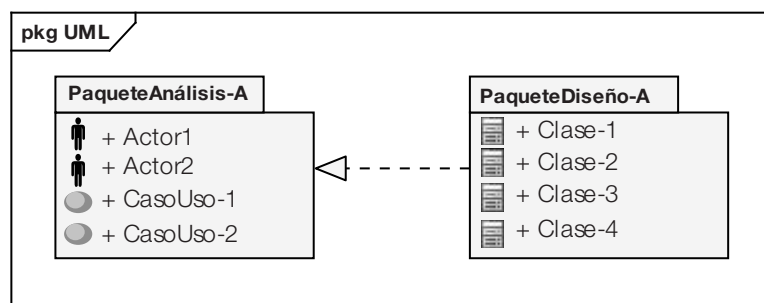


Figura 8.4. Relación entre paquetes, análisis empaquetando casos de uso y de diseño empaquetando clases. PaqueteDiseño-A implementa al PaqueteAnálisis-A

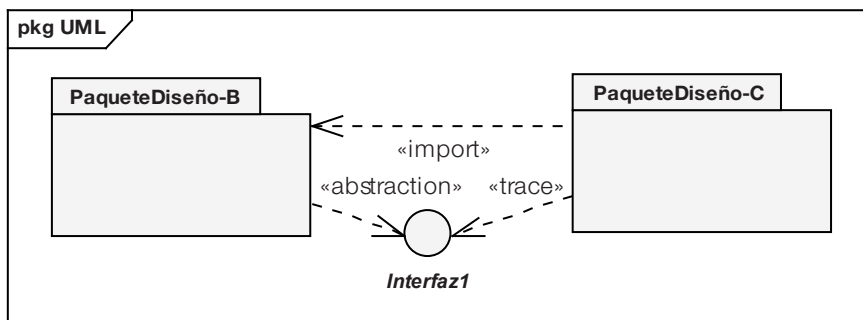


Figura 8.5. Relación entre paquetes, C usa a B e implementa la Interfaz1

8.5 Componentes

UML también permite documentar los componentes en que terminan implementados los diferentes subsistemas. Esta vista física del sistema en desarrollo es de utilidad al momento de la distribución del sistema de manera completa. En la figura 8.6 se muestra un diagrama de componentes, el cual permite visualizar las dependencias entre los diferentes componentes de un sistema (ejecutables, base de datos, archivos, librerías).

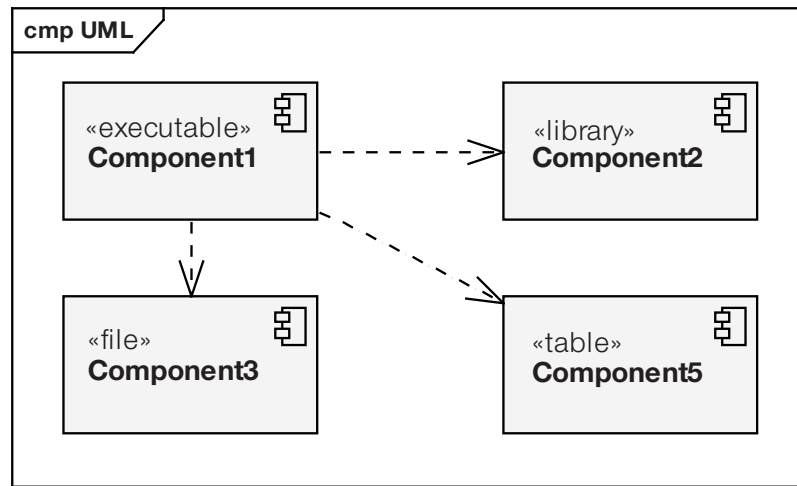


Figura 8.6. Diagrama de componentes

La trazabilidad, concepto introducido en el capítulo 7, posibilita establecer relaciones entre las diferentes partes del desarrollo y cómo estos son empaquetados para su distribución. Un ejemplo de este tipo de vista se muestra en la figura 8.7.

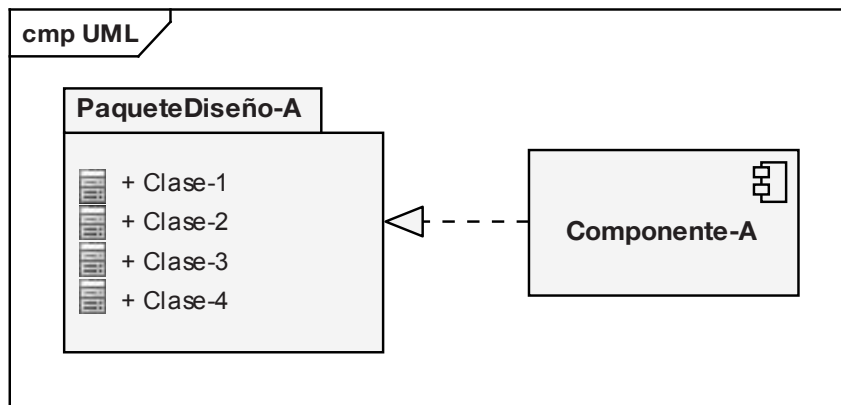


Figura 8.7. Relación entre un paquete y el componente que lo implementa

8.6 Instalación

Todo software necesita un contexto de ejecución, este está dado por el hardware donde el software será instalado y será ejecutado. UML permite documentar esto a partir de sus diagramas de instalación. En ellos se muestran los diferentes componentes y su relación con los nodos en los cuales este software residirá. En la figura 8.8 se muestra un ejemplo de este tipo de diagramas.

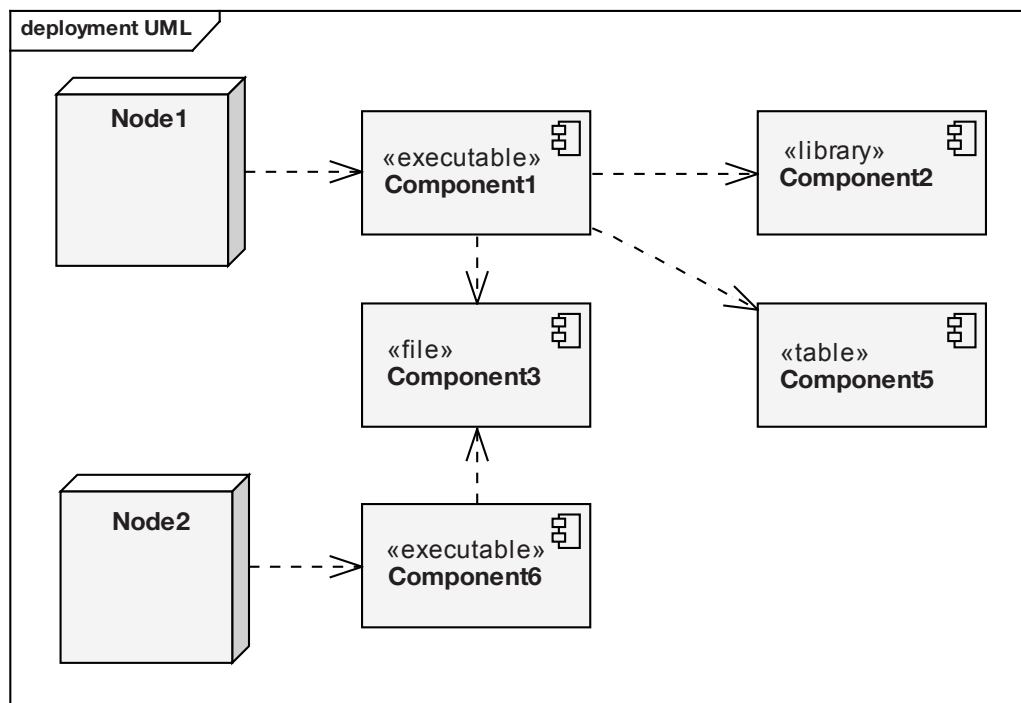


Figura 8.8. Diagrama de instalación de los componentes en sus respectivos nodos

8.7 Ejemplo

A continuación presentamos un ejemplo que proviene de un proyecto real, el cual fue adaptado para que se ajuste a la presentación en este libro.

El negocio del cual proviene este ejemplo es el conocido como “locutorios”. Este es una sala donde llegan N líneas telefónicas las cuales son provistas por una TELCO (empresa de telecomunicaciones) para ser utilizadas por el locutorio. El locutorio obtiene una ganancia a partir de la diferencia entre lo que paga a la TELCO y lo que cobra a los usuarios por realizar llamadas telefónicas por dichas líneas. En el locutorio hay N cabinas, cada una asociada a una línea. Las líneas son conectadas a una computadora personal (PC) la cual controla el acceso a estas, detecta los eventos electrónicos de señalización y tarifa las llamadas. El software a desarrollar es el de este sistema. Cada día la TELCO envía por una comunicación a través de una línea la guía con los datos para rutear las llamadas y tarifarlas; además recibe de parte del locutorio el movimiento del día anterior compuesto por el registro de cada llamada realizada. El sistema debe administrar una contabilidad por turno, día y mes. Deben administrarse los roles y permisos de acceso para los operadores y supervisores. El sistema debe presentar una vista al operador del conjunto de cabinas y sus estados. Cada vez que un usuario solicita realizar una llamada, el operador habilita a través del teclado una cabina y le indica al usuario que pase por ella. El usuario realiza una o más llamadas. Al finalizar el usuario recibe un *ticket* impreso en papel por los servicios prestados y abona el precio correspondiente.

En la figura 8.9 se muestra un diagrama de paquetes en los cuales se pueden ver los casos de uso correspondientes a cada uno de ellos. La relación entre los paquetes es consecuencia de la relación entre los casos de uso.

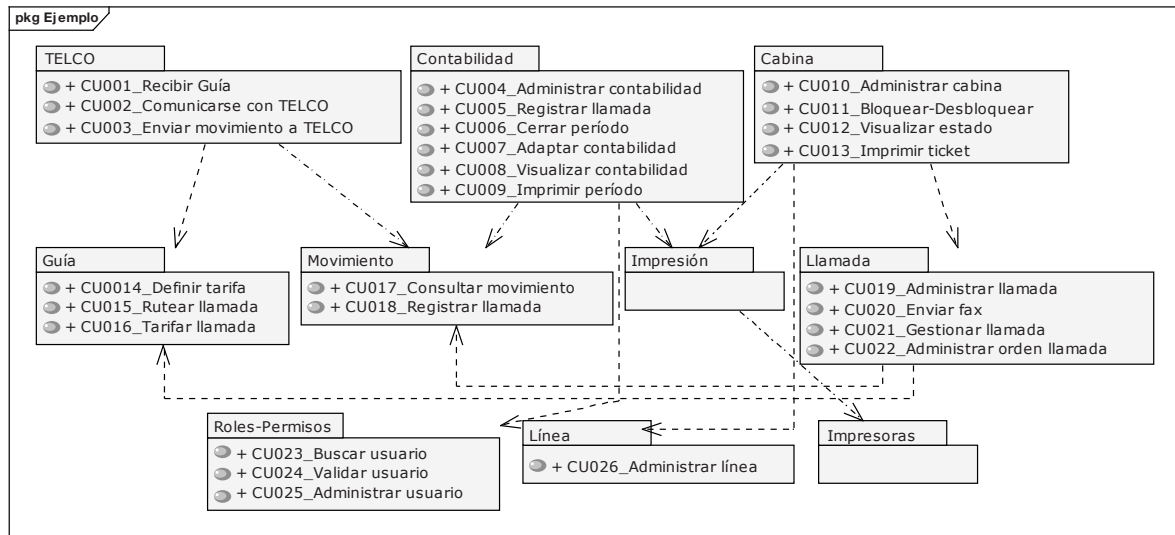


Figura 8.9. Diagrama de paquetes que contienen a los casos de uso

La funcionalidad deseada expresada más arriba está distribuida en estos paquetes que por ahora son lógicos. Puede observarse también que esta distribución y sus relaciones expresan una primera visión de la arquitectura del sistema que es del estilo estratificada en capas.

El diseño detallado de cada parte de los paquetes anteriores fue documentada utilizando diagramas de clases como el que se muestra en la figura 8.10. En ella se ven las clases que implementan la visualización del estado de las cabinas.

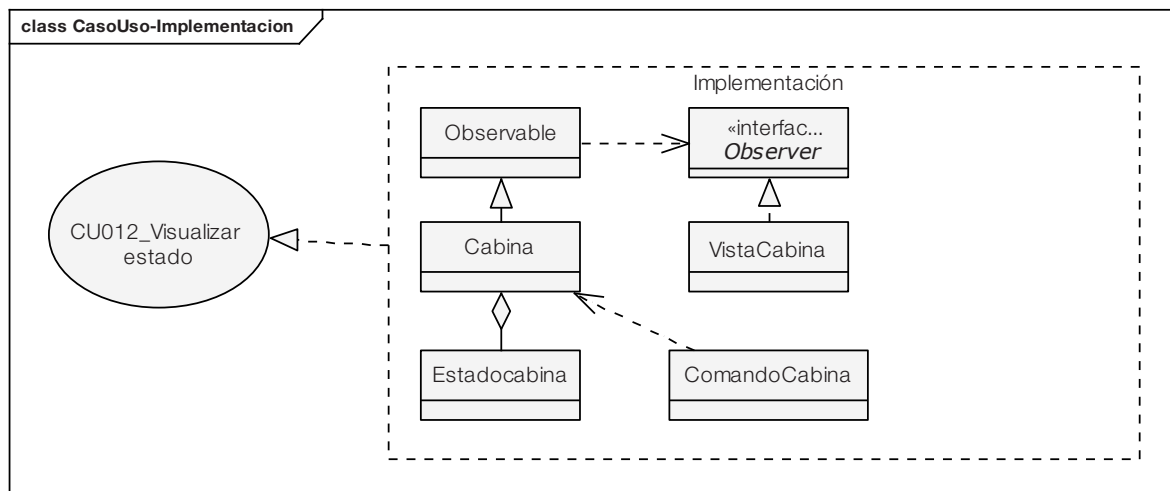


Figura 8.10. Diagrama de clases de la implementación del caso de uso visualizar cabina

El proyecto evoluciona y para cada paquete realizamos un diseño que implementa la funcionalidad de cada uno. En la figura 8.11 se muestra para el paquete que encapsula los aspectos del sistema vinculados a la cabina las clases correspondientes. Se ha decidido que la mayoría de los paquetes sean implementados en librerías dinámicas, cuyo componente se muestra en la misma figura.

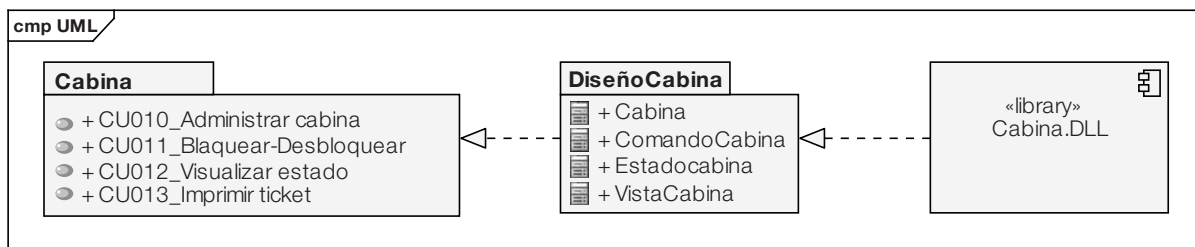


Figura 8.11. Diagrama de UML que muestra la implementación de la funcionalidad del paquete correspondiente a las cabinas

El ejecutable que prestará el servicio de administración del locutorio dependerá para su funcionamiento de una serie de librerías, de la base de datos del sistema y de un archivo de configuración. Este aspecto del sistema se muestra en el diagrama de componentes de la figura 8.12.

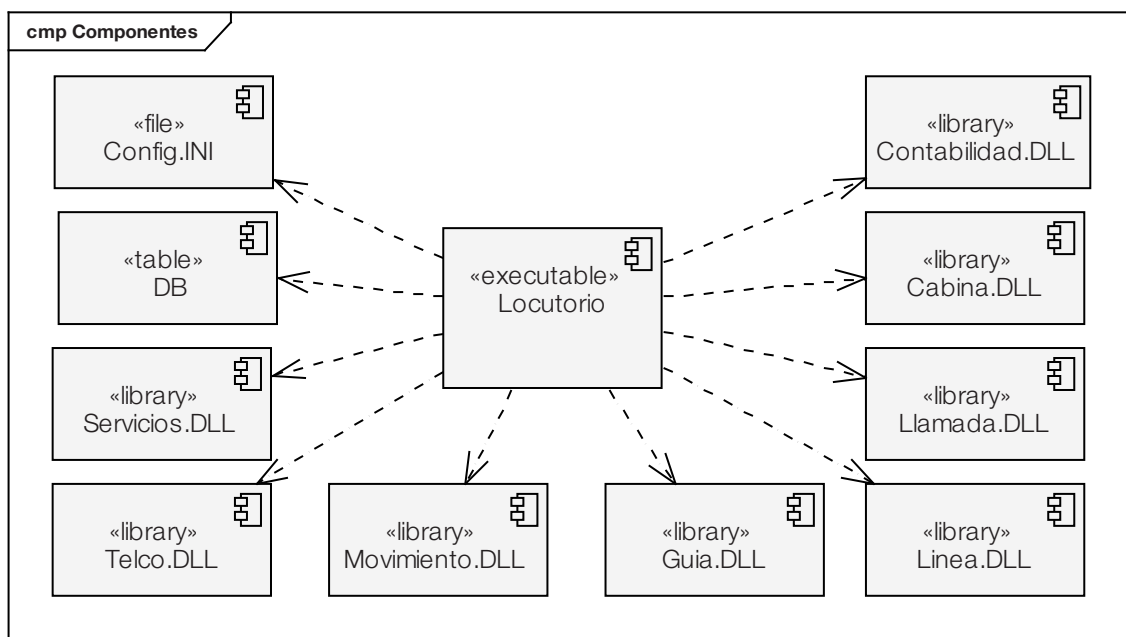


Figura 8.12. Diagrama de componentes del sistema

Debido a los requerimientos impuestos por la interacción con el API (*Application Programmer Interface*) de la placa de hardware que conecta las líneas telefónicas a la PC en relación al intercambio de mensajes en tiempo real, se decidió distribuir las tareas de impresión y comunicación con la TELCO en otro nodo (PC) dado que estas tareas son lentas y consumen tiempo de procesador. Esta decisión se

muestra en la figura 8.13. La comunicación entre los tres procesos se hará, como puede verse, a través de archivos. Además para el control de las líneas se asignó un thread independiente del que administra la interfaz gráfica de usuario para que el sistema tenga una buena respuesta.

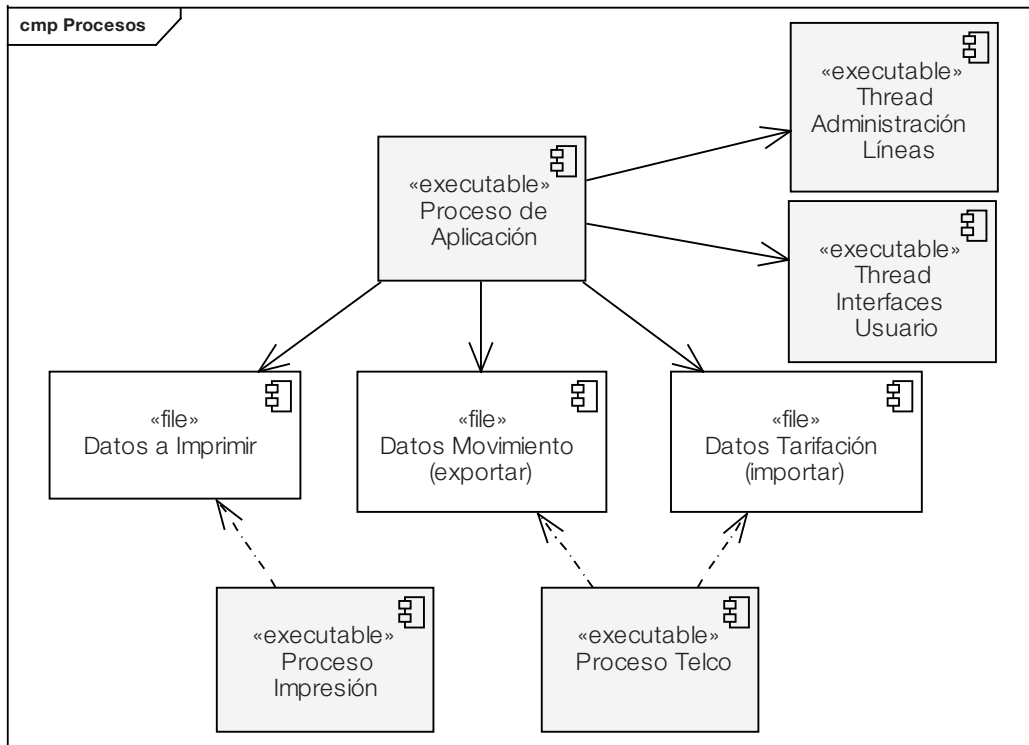


Figura 8.13. Diagrama de componentes que muestran los procesos del sistema

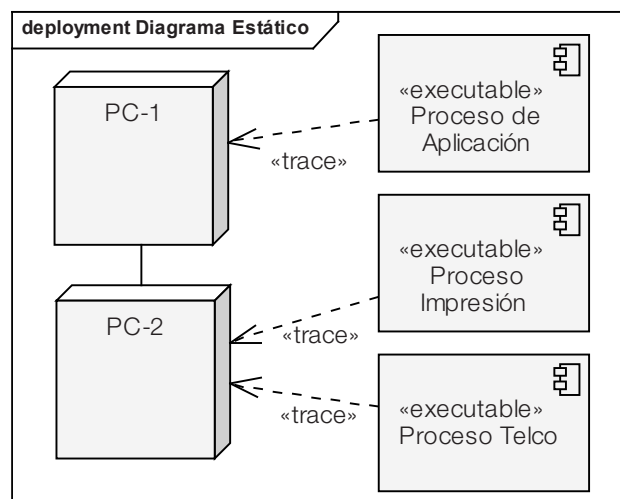


Figura 8.14. Diagrama de instalación de los componentes del sistema

Finalmente el diagrama de la figura 8.14 muestra la distribución de los distintos componentes en los nodos que forman parte del sistema.

8.8 Conclusión

Hemos presentado los diferentes diagramas estáticos de UML. El modo de uso que se muestra en el ejemplo es el descrito como Dibujo (*Sketch*) en el capítulo 7. Una presentación detallada de los diagramas puede leerse en [3] y [4]. En el próximo capítulo haremos una presentación similar para los diagramas dinámicos, los cuales muestran el comportamiento de las diferentes partes del sistema en los distintos momentos del ciclo de vida del proyecto.

8.9 Contenido de la página web de apoyo



8.9.1 Mapa conceptual

8.9.2 Autoevaluaciones

8.9.3 Presentaciones en Power Point (exclusivo para el docente)

8.10 Referencias

- [1] *Architectural Blueprints — The “4+1” View Model of Software Architecture*, Philippe Kruchten, IEEE Software 12 (6), November 1995, pp. 42-50.
- [2] Object Management Group (OMG), <http://www.uml.org/>.
- [3] *UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd Edition)*, Martin Fowler, Addison-Wesley, 2005.
- [4] *Unified Modeling Language-User’s Guide*, Grady Booch, Jim Rumbaugh, and Ivar Jacobson, Addison-Wesley, 1999.
- [5] *The Unified Software Development Process*, Ivar Jacobson, Grady Booch, James Rumbaugh, Addison-Wesley Professional, 1999.
- [6] *Enterprise Architect*, <http://www.sparxsystems.com.ar/>.

El lenguaje UML Diagramas dinámicos

Contenido

9.1 Introducción	134
9.2 Actividades	134
9.3 Casos de uso	135
9.4 Interacción	137
9.5 Estado	138
9.6 Ejemplo	138
9.7 Conclusión	141
9.8 Contenido de la página web de apoyo	141
9.9 Referencias	141

Objetivos

- Aprender qué aspectos de un sistema documentan los diagramas dinámicos de UML
- Cuáles son los diagramas dinámicos de UML
- Qué relación tienen estos diagramas entre ellos
- Qué relación tienen estos diagramas con las actividades de un proyecto
- Cuándo, en qué momento del ciclo de vida de un proyecto se construyen los diferentes diagramas

Competencias

- Establecer un criterio de utilización del lenguaje UML como medio de documentación de los aspectos dinámicos de un proyecto de desarrollo de software

9.1 Introducción

Los diagramas de UML presentados en el capítulo anterior serán complementados por los que presentemos en este capítulo. Aquellos eran estáticos, estos son dinámicos.

Los diagramas dinámicos de UML son los que se listan a continuación [1]:

- Actividades
- Casos de uso
- Interacción
- Estado.

Al igual que los diagramas estáticos, estos se utilizan en diferentes momentos del proyecto y con diferentes objetivos [4]. Todos ellos muestran vistas dinámicas, comportamiento del sistema a resolver.

Según sea la complejidad de los proyectos y de su tamaño se puede contar con varios de estos diagramas los cuales muestran distintas vistas de interés.

Cubren aspectos relacionados a los primeros momentos del ciclo de vida del proyecto como aquellos vinculados a las últimas tareas que se realizan.

Permiten relacionar diferentes aspectos del negocio con los requerimientos funcionales, así como la dinámica y la forma de comunicación entre las diferentes partes de la arquitectura y del diseño de los módulos.

9.2 Actividades

Los diagramas de actividad son ideales para modelar el flujo de información entre roles participantes de un negocio. En ellos se indican las actividades realizadas por cada uno, la relación entre ellas y los productos generados por estas. También muestran los estados de los productos mencionados y cómo estos cambian al ser afectados por las actividades. En la figura 9.1 puede verse un diagrama de este tipo. En ella puede observarse un flujo de trabajo que comienza con la realización de la Actividad_1, el cual al finalizar su realización presenta la bifurcación de las actividades siguientes en dos flujos simultáneos. Esta bifurcación es indicada con la barra horizontal (*fork*). El flujo de la izquierda continúa con una decisión tomada a partir de la respuesta a una condición indicada por un rombo. Dependiendo de esta condición se desarrollará la Actividad_2 o la Actividad_3. El flujo de la derecha continúa con el desarrollo de la Actividad_4, la cual genera como producto de su ejecución el Objeto_1 en el estado inicial. Sigue esta dinámica con el desarrollo de la Actividad_2, la cual es una actividad compleja que está descompuesta en un conjunto de actividades como se indica en el diagrama con el símbolo “¥”. Como resultado de esta ejecución cambió el estado del Objeto_1 a estado final. Ambos flujos son independientes y se desarrollan simultáneamente, cada uno a su ritmo, sin embargo ambos se juntan (sincronizan) para que luego el flujo continúe siendo único nuevamente y finalice. La sincronización de ambos flujos se indica con una barra horizontal (*join*) como se muestra en la figura. Una presentación detallada de estos diagramas puede leerse en [2] y [3].

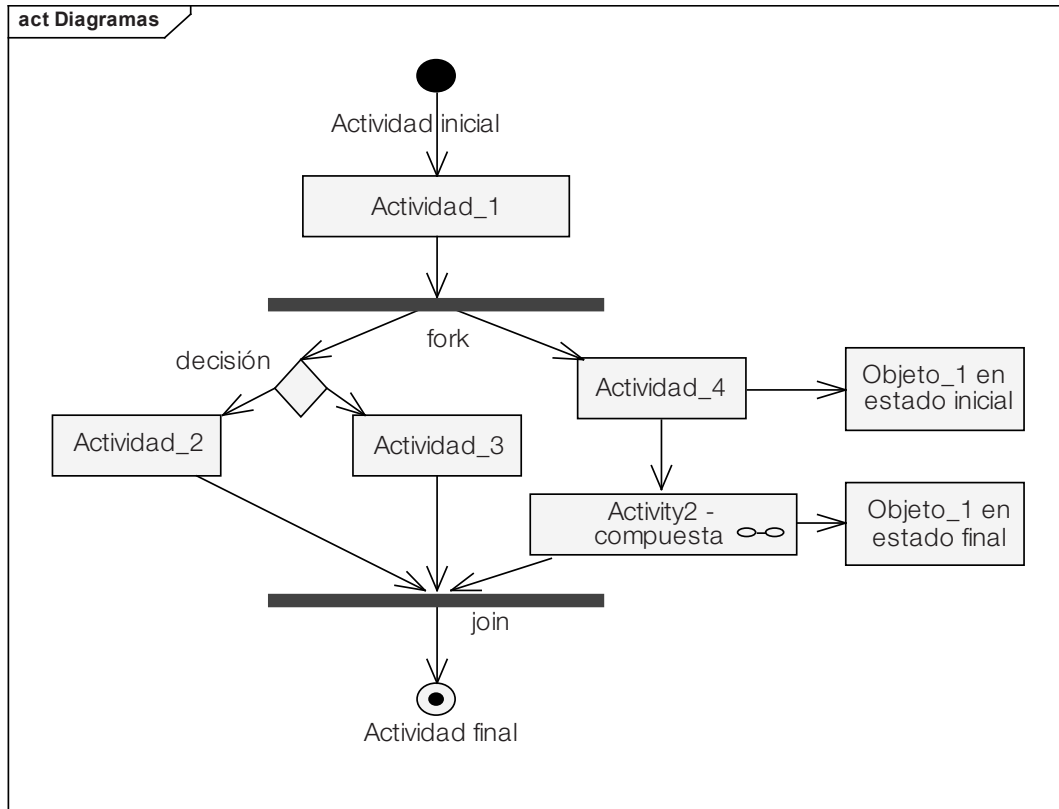


Figura 9.1. Diagrama de actividad

Estos diagramas también pueden utilizarse para documentar la interacción y las actividades realizadas por los diferentes componentes de un sistema, así como para modelar la dinámica de casos de uso. En este caso los roles son reemplazados por los subsistemas [1], [4], [5].

En un ejemplo al final de este capítulo ejemplificamos su uso en relación con los otros diagramas dinámicos y mostramos cómo estos se complementan para establecer una vista completa de los diferentes aspectos de un desarrollo.

9.3 Casos de uso

Cuando en etapas tempranas de un proyecto se busca definir su alcance, a partir de un listado de requerimientos funcionales se construye un diagrama de casos de uso. Estos modelan funcionalidades y sus relaciones; y su especificación textual especifica cómo serán implementados los requerimientos asociados [1], [5]. Asociamos cada funcionalidad a un actor, quien es el que a partir del uso del sistema genera el evento que ejecuta dicha funcionalidad. En la figura 9.2 se muestra un diagrama de este tipo donde aparecen dos actores y la funcionalidad del sistema que estos ejecutarán. El Actor1 participa de dos funcionalidades y el Actor2 de una única. Puede verse que los casos de uso están unívocamente identificados y además tienen un nombre

asociado al objetivo que los actores buscan lograr con su ejecución. Puede verse también que el caso de uso CU_002 tiene una relación de tipo “include” con el caso de uso CU_004. Esto implica que alguno de los pasos que el actor debe realizar interactuando con el sistema en busca del objetivo 2 incluye la realización de los pasos comprendidos en el caso de uso CU_004. Debido a que la funcionalidad representada por el caso de uso CU_002 también debe ser ejecutada como parte de los pasos del caso de uso CU_001 es que se extrajo para ser reutilizada. Siempre que se ejecuten los casos de uso CU_001 o CU_002 se ejecutará como parte de ellos el caso de uso CU_004. Existe por lo tanto un conocimiento desde el caso de uso que incluye hacia el incluido.

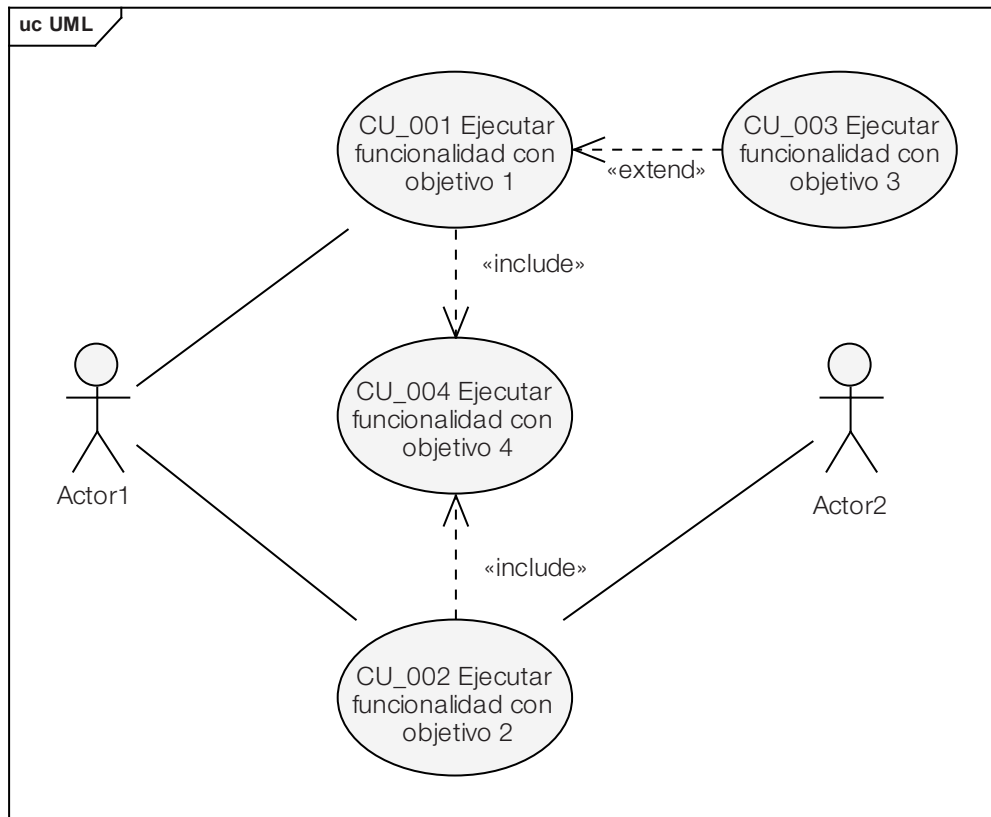


Figura 9.2. Diagrama de casos de uso

En la figura se observa otro tipo de relación entre casos de uso, la que existe entre los casos CU_001 y CU_003. Esta relación llamada *extend* vincula ambos casos de diferente manera que la anterior. En este proceso durante la ejecución del caso de uso CU_001, solo si se cumple una determinada condición, el caso de uso CU_003 será ejecutado. En esta ocasión existe un conocimiento desde el caso de uso que extiende hacia el extendido, es decir que el caso CU_003 conoce al CU_001, de la condición que debe cumplirse para ejecutarse y del punto de extensión (paso del CU_001 donde debe insertarse).

Estos diagramas son acompañados con la especificación textual de los pasos del caso de uso y prototipos de interfaces de usuario. En el capítulo donde tratamos el trabajo con los requerimientos ejemplificaremos su utilización sobre el caso testigo que presentamos en la Introducción.

9.4 Interacción

Los diagramas de secuencia y su variante de comunicación están orientados a mostrar la interacción en el tiempo entre entidades, en general objetos, instancias de las clases que componen el sistema en desarrollo [1], [2]. En la figura 9.3.a se puede ver la secuencia de mensajes que intercambian dos objetos.

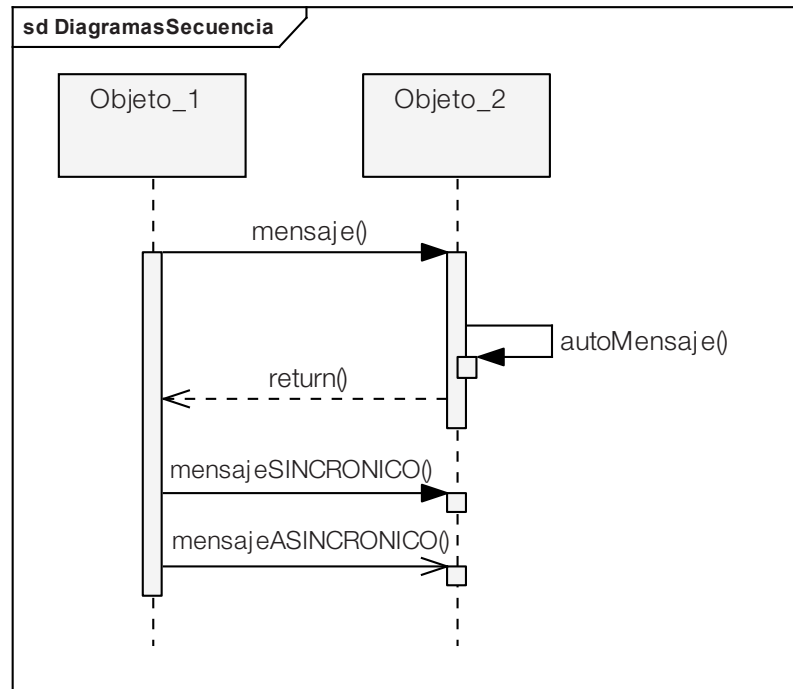


Figura 9.3.a. Diagrama de secuencia que muestra la interacción entre dos objetos

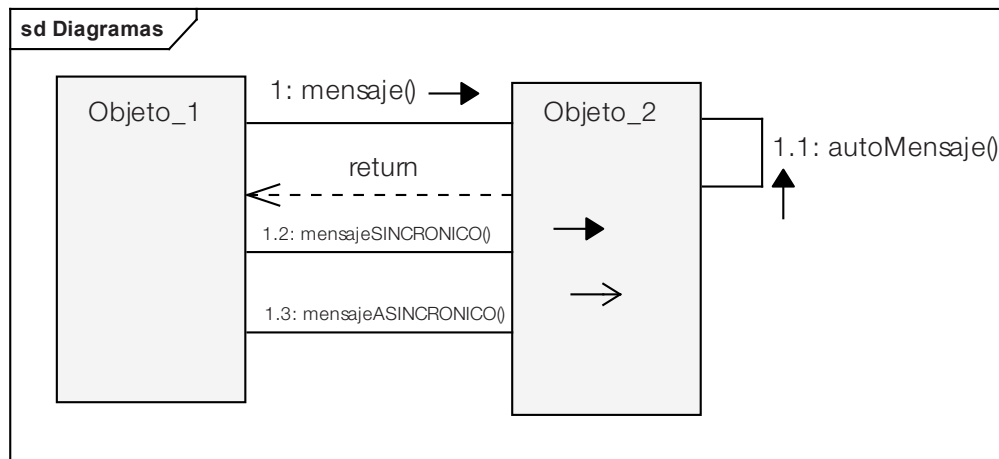


Figura 9.3.b. Diagrama de comunicación correspondiente al de secuencia de la figura 9.3.a

En la figura 9.3.b se muestra el diagrama de comunicación correspondiente al mismo escenario de interacción entre los objetos de la figura 9.3.a. En este tipo de diagramas no aparece en forma explícita el tiempo como en el anterior. Ambos son equivalentes y se puede pasar de uno al otro según sea de mayor utilidad las vistas que estos ofrecen. Muestran la misma información de manera distinta.

9.5 Estado

Los diagramas de estado están orientados a mostrar los estados y la transición entre ellos de cualquier entidad que a partir de recibir eventos cambia su estado [1], [2]. Ejemplo típico de esto son las órdenes de compra, los tickets de facturación de servicios, las líneas telefónicas, entre otros.

Son de utilidad ya que seguramente estas entidades tienen un comportamiento diferente según estén en un estado o en otro. Ejemplo de esto es la línea telefónica ya que al estar el teléfono asociado a ella colgado no pueden discarse dígitos o al estar discando dígitos sobre ella el abonado no puede comunicarse a través de ella.

En la figura 9.4 se muestra un diagrama de este tipo. En el ejemplo del final del capítulo se muestra cómo pueden estos diagramas complementar a los otros diagramas dinámicos mostrando los diferentes estados de las entidades afectadas por las actividades de un flujo de negocio.

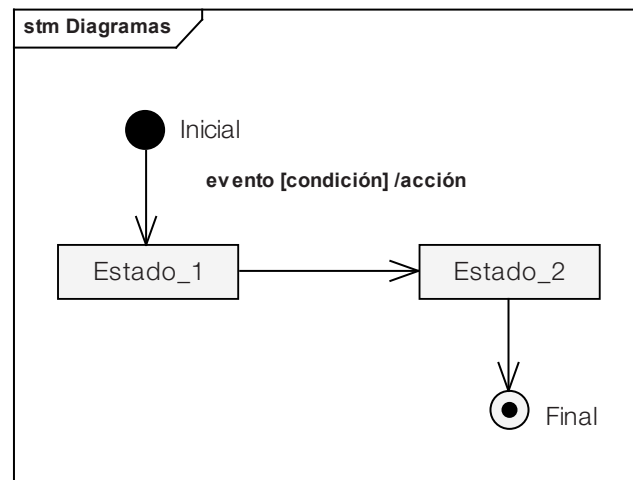


Figura 9.4. Diagrama de estados

9.6 Ejemplo

El ejemplo que elegimos trata acerca de un negocio de compras y ventas de productos. El comprador, ya registrado como cliente, realiza un pedido a partir del cual se genera una orden de compra que es "ingresada". Esta actividad es la inicial y se muestra en la figura 9.5. A esta actividad le sigue la recepción

de dicha orden por parte del vendedor que registra el pedido. Como consecuencia de esta actividad, la orden de compra está ahora en un nuevo estado, es decir “registrada”. A partir de allí se ejecutan dos actividades simultáneas, las cuales están orientadas a facturar la venta asociada a la orden y preparar el envío de los productos pedidos. Estas actividades son desarrolladas por dos roles diferentes, el distribuidor y el contador. Cuando la distribución está terminada, la orden original pasa al estado “preparada para envío”. Cuando se recibe el pago, cambia nuevamente de estado, esta vez a “facturada”. Hasta que ambas actividades, la recepción del pago y el despacho no se realicen efectivamente no se podrá continuar.

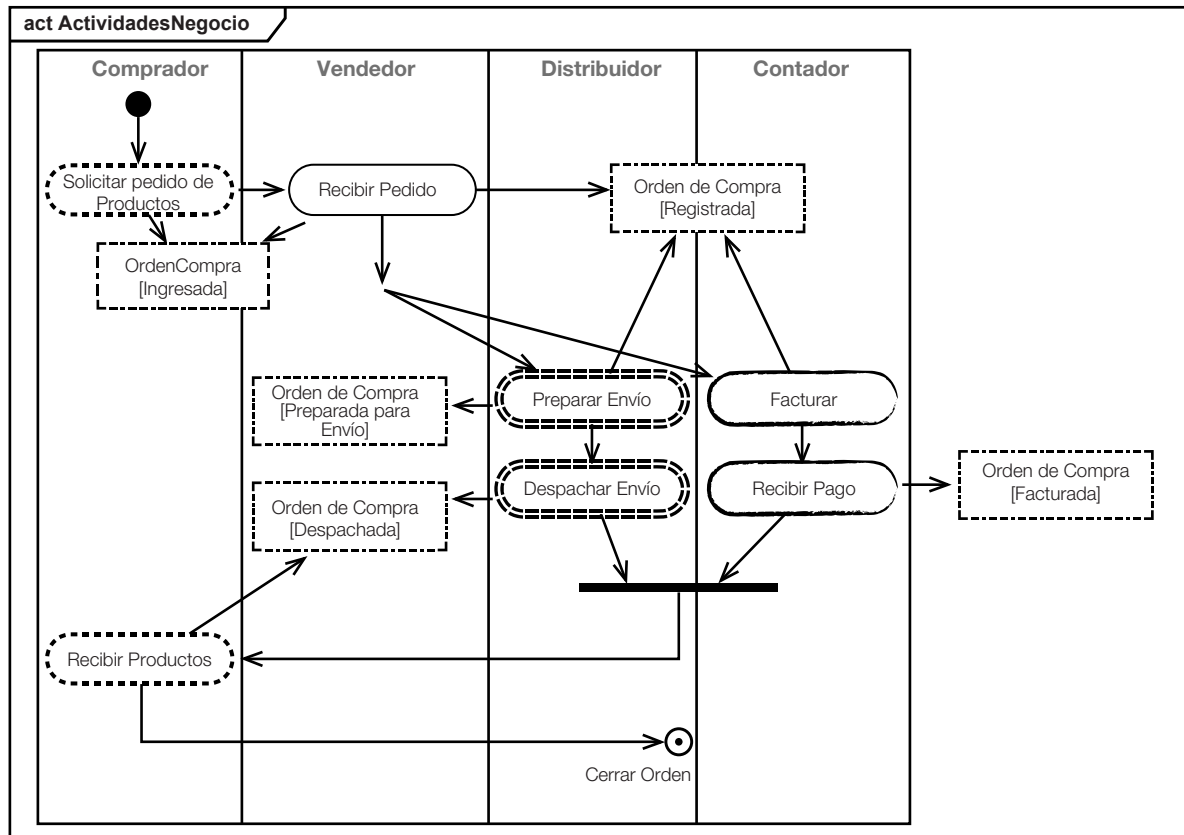


Figura 9.5. Diagrama de actividad del flujo de negocio de compra y venta

La situación anterior se muestra en la figura 9.5 con un Joint de ambas actividades. Es decir que la orden pasará por los estados “facturada” y “despachada” antes de que los productos puedan ser distribuidos. Finalmente, los productos son entregados al comprador y el flujo finaliza.

Las actividades mostradas en el diagrama anterior serán llevadas adelante por los diferentes roles, actores del sistema en desarrollo, y constituyen casos de uso asociados a la funcionalidad solicitada como parte del alcance del desarrollo en los requerimientos funcionales. En la figura 9.6 se muestra un diagrama de casos de uso en la que se señalaron los casos con las mismas tramas de las actividades relacionadas del diagrama-

ma de actividad de la figura 9.5. Puede verse que los actores del diagrama son los roles que ejecutaban las actividades.

De la misma forma podemos visualizar en un diagrama de estados los correspondientes a la orden de compra a medida que esta es manipulada por las diferentes actividades. En la figura 9.7 se muestran los estados correspondientes, que son los mismos esquematizados en la figura 9.5. Estas tres figuras muestran la forma en que estos diagramas se complementan para mostrar en diferentes niveles el comportamiento deseado de un módulo de un sistema en desarrollo.

En [6], [7] y [8] pueden verse herramientas orientadas a la construcción de estos diagramas.

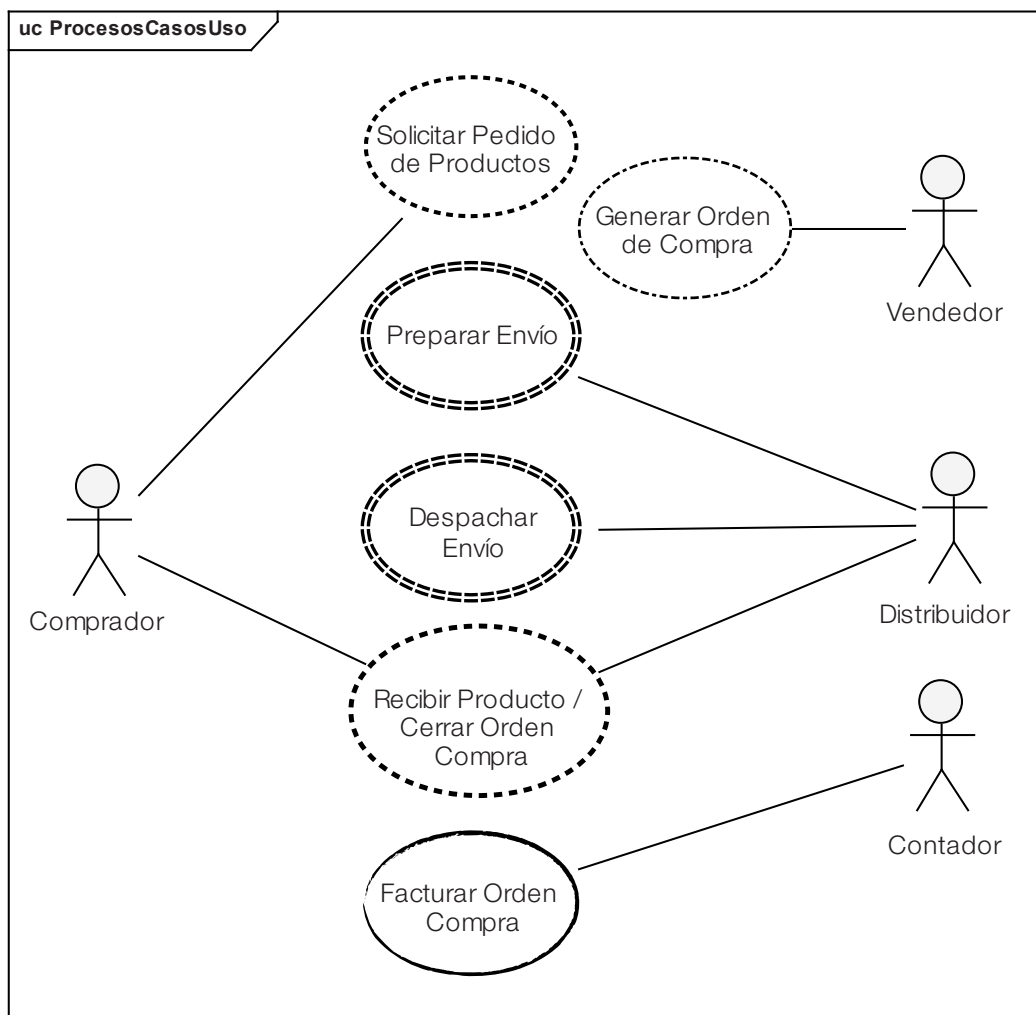


Figura 9.6. Diagrama de casos de uso del sistema en desarrollo

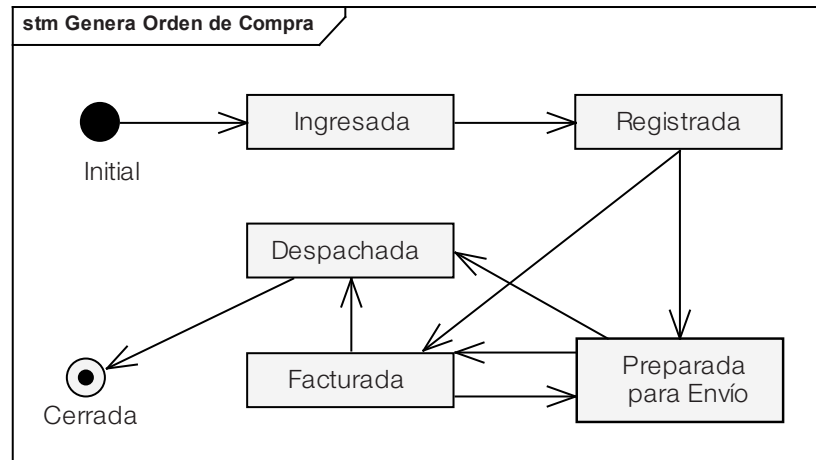


Figura 9.7. Diagrama de estados de la entidad Orden de Compra

9.7 Conclusión

Hemos presentado los diferentes diagramas dinámicos de UML. El modo de uso que se muestra en el ejemplo es el descrito como Dibujo (*Sketch*) en el capítulo 7. En los capítulos donde trataremos temas de especificación y diseño del software del caso testigo del libro volveremos a utilizarlos.

9.8 Contenido de la página web de apoyo



9.8.1 Mapa conceptual

9.8.2 Autoevaluaciones

9.8.3 Presentaciones en Power Point (exclusivo para el docente)

9.9 Referencias

- [1] Object Management Group (OMG), <http://www.uml.org/>.
- [2] *UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd Edition)*, Martin Fowler, Addison-Wesley, 2005.
- [3] *Unified Modeling Language-User's Guide*, Grady Booch, Jim Rumbaugh, and Ivar Jacobson, Addison-Wesley, 1999.
- [4] *The Unified Software Development Process*, Ivar Jacobson, Grady Booch, James Rumbaugh, Addison-Wesley Professional, 1999.

- [5] *Applying Use cases, a practical guide*, G. Schneider, J. Winters, Addison-Wesley, 1997.
- [6] *Rational Rose*, <http://www-01.ibm.com/software/rational/>.
- [7] *Enterprise Architect*, <http://www.sparxsystems.com.ar/>.
- [8] *Doxygen*, <http://www.stack.nl/~dimitri/doxygen/>.

Relevamiento de requerimientos

Contenido

10.1 Relevamiento de requerimientos	144
10.2 Casos de uso	150
10.3 Organización del trabajo con requerimientos	161
10.4 Priorización de requerimientos	162
10.5 Conclusión	166
10.6 Contenido de la página web de apoyo	166
10.7 Referencias	166

Objetivos

- Entender para qué sirven los requerimientos y su importancia en un proyecto de desarrollo
- Aprender cómo se relevan durante el trabajo con todos los involucrados en el proyecto
- Aprender cómo construir modelos de requerimientos
- Cómo contribuyen los requerimientos a la planificación del proyecto
- Cómo se priorizan los requerimientos de un proyecto durante su planificación

Competencias

- Definir la forma de trabajo con los requerimientos funcionales y no funcionales en un proyecto de desarrollo de software
- Facilitar la planificación del trabajo con los requerimientos en los proyectos de desarrollo

10.1 Relevamiento de requerimientos

10.1.1 Objetivos

10.1.1.1 Especificación de qué desarrollar

La primera actividad en un proyecto de software, independientemente de la metodología a utilizar, consiste en construir un listado de requerimientos constituido por aquello “qué” debe permitir hacer el sistema en desarrollo a los usuarios. Estos son los *requerimientos funcionales*. “Cómo” debe funcionar dicho sistema en relación a su ambiente operacional será definido por los *requerimientos no funcionales*. Unos expresan la funcionalidad a implementar, los otros las restricciones vinculadas a su funcionamiento [1]. Estos requerimientos deben ser escritos en frases simples, claras, directas y atómicas de manera que posean ciertas propiedades tales como: no ambigüedad, completitud y ser correctos. Para el sistema del caso testigo se construyó previamente un diagrama del contexto, figura 10.1, el cual muestra la primera aproximación al sistema.

En la figura 10.2 se muestra un listado de los requerimientos del sistema a construir para el caso testigo del libro, el cual fue presentado en la sección “Descripción del problema a resolver” del capítulo 1. En la figura 10.3 se muestran los requerimientos no funcionales. En ambos listados puede verse que cada requerimiento está identificado, esto es para poder establecer una traza desde y hacia ellos con los otros artefactos del desarrollo en el futuro.

Estos requerimientos fueron relevados en reuniones planificadas con los usuarios y otros involucrados por parte de la empresa cliente. Para ello se utilizaron algunas de las técnicas que describimos en la próxima sección.

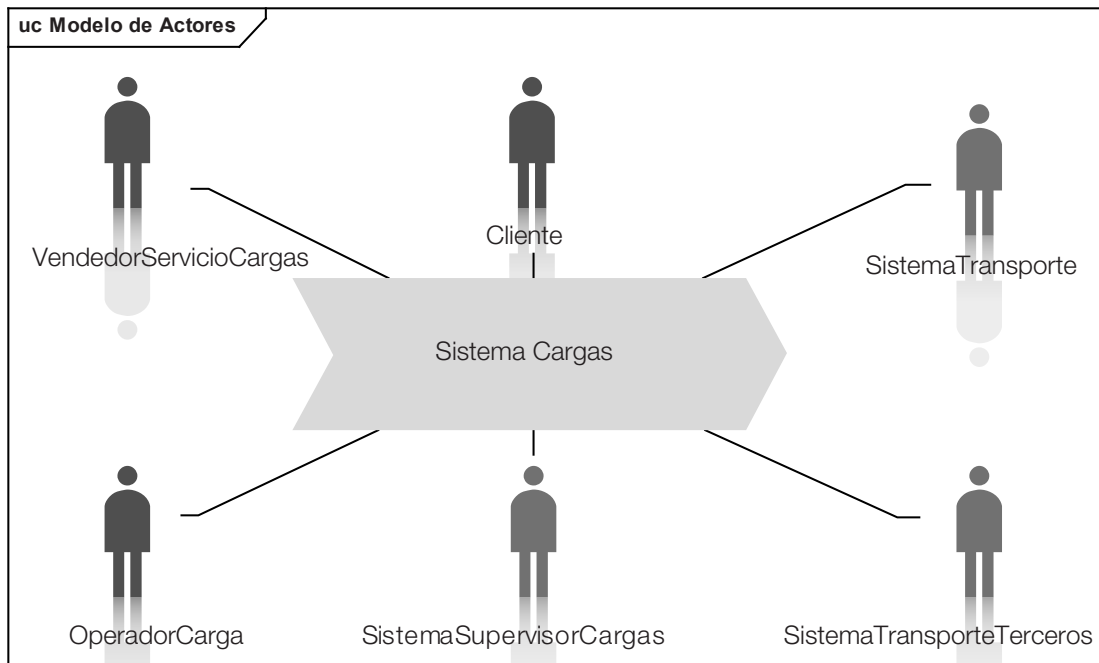


Figura 10.1. Diagrama de contexto del sistema del caso testigo

En este relevamiento se buscó cumplir con el objetivo de obtener los requerimientos que permitieran definir el alcance del sistema, para lo cual se utilizó el material relevado en forma preliminar cuando se elaboró la propuesta técnico económica a efectos de acordar la realización del proyecto, en la fase de preventa. En esa oportunidad se relevaron con el propósito de determinar la intensión del sistema. El foco fue puesto en diferentes aspectos de manera de guiar el relevamiento a partir de responder a las siguientes preguntas: qué, quién, cuándo, dónde, por qué y cómo [2]. La vista que aportan los listados de requerimientos ayuda a definir la estructura del sistema en desarrollo. Para trabajar en estas tareas del proyecto el grupo se organizó de la forma que describimos en una próxima sección.

Es una práctica común en las metodologías conducidas por los planes volcar estos listados en un documento denominado *Especificación de requerimientos de Software* (ERS), como fue presentado en el capítulo 3.

En las metodologías ágiles estos listados aportarán al “product backlog”.

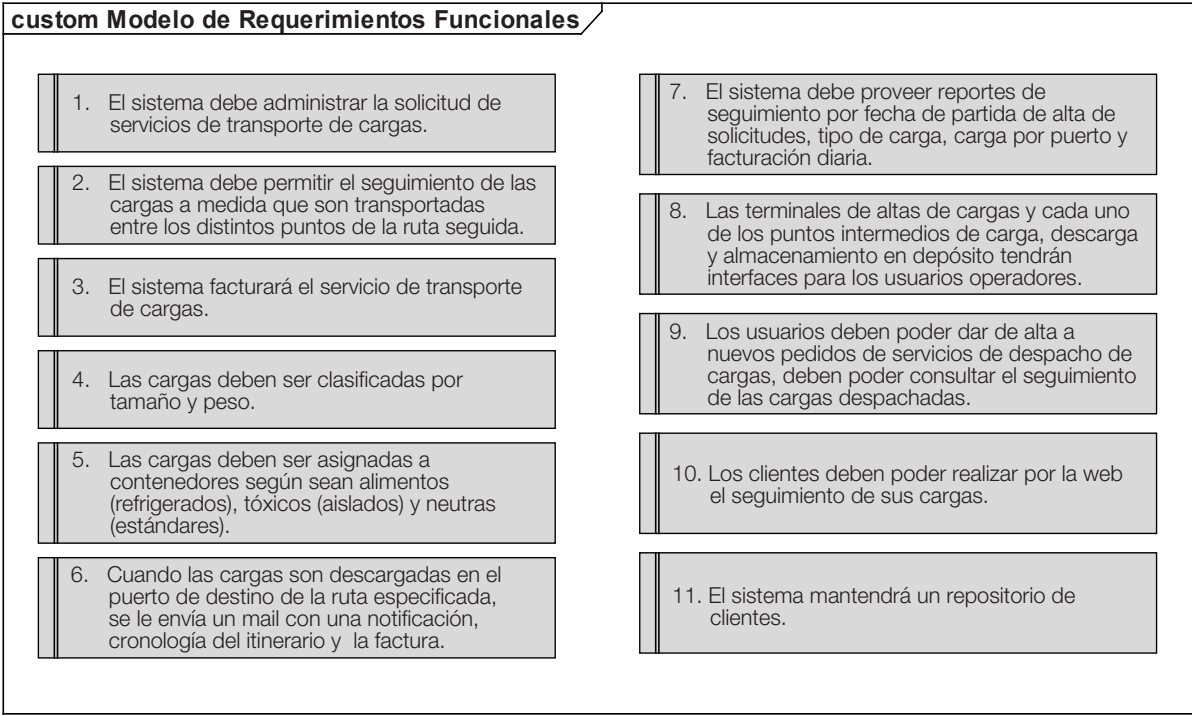


Figura 10.2. Listado de requerimientos del sistema del caso testigo

Otra vista de interés y de fundamental importancia es la del control, aportada por las reglas del negocio. En la figura 10.4 se presenta el listado de reglas que serán asignadas a las clases del modelo de dominio que elaboraremos en el próximo capítulo, para obtener así un modelo que describa en estructura y comportamiento al negocio que busca resolver el sistema en desarrollo.

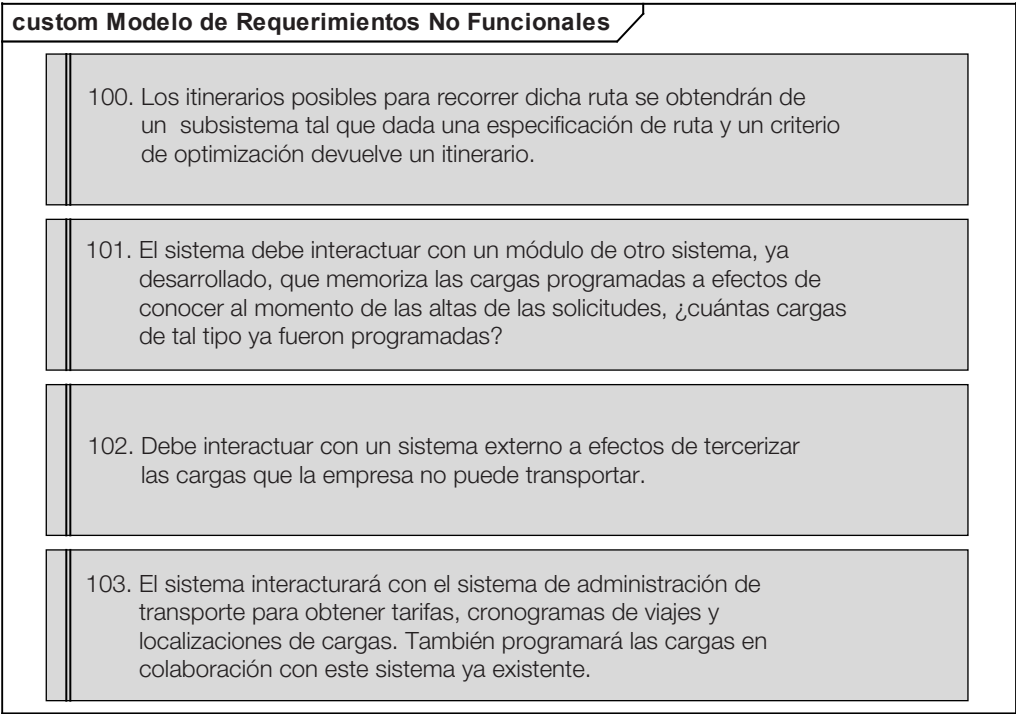


Figura 10.3. Listado de requerimientos no funcionales del sistema del caso testigo

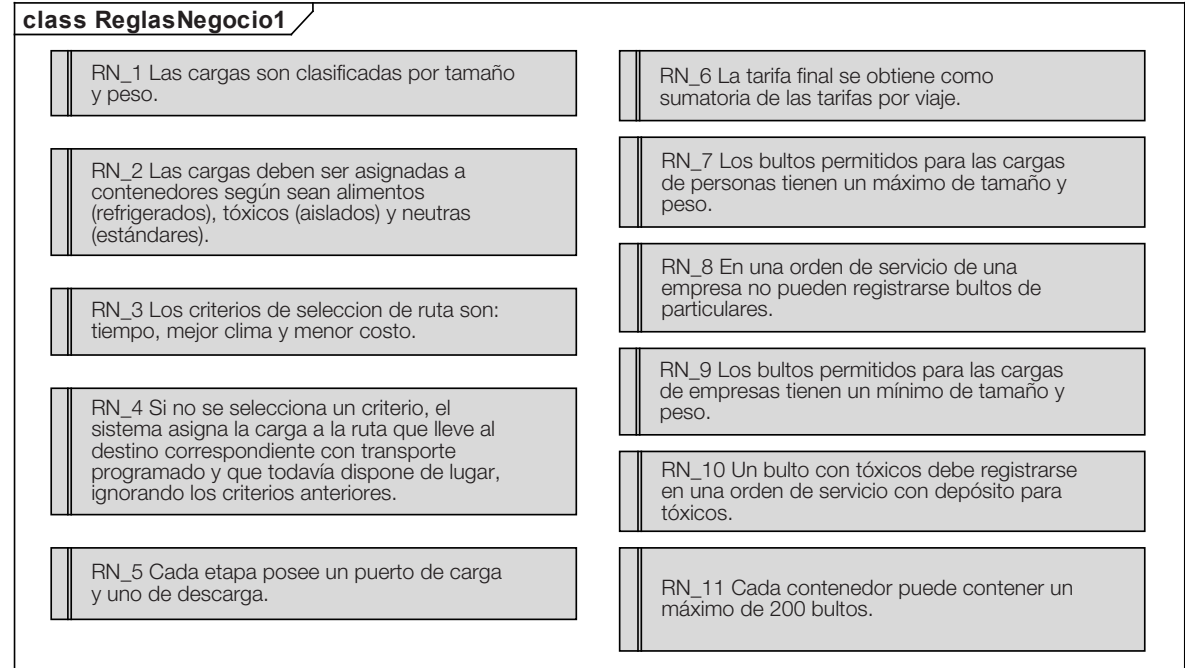


Figura 10.4. Listado de las reglas del negocio del sistema del caso testigo

10.1.1.2 Definición del alcance

A efectos de definir el alcance a la información ya relevada sumamos otra vista de importancia, la del comportamiento y la dinámica aportada por los casos de uso. En la figura 10.5 se muestra un diagrama de los casos de uso obtenido a partir de los requerimientos funcionales y los actores del sistema mostrados en la figura 10.1. Estos casos de uso no son detallados, lo que implica que cuando se refinan aparecerán otros no mostrados en este diagrama. Se obtienen a partir de cada uno de los actores y el uso que estos hacen del sistema.

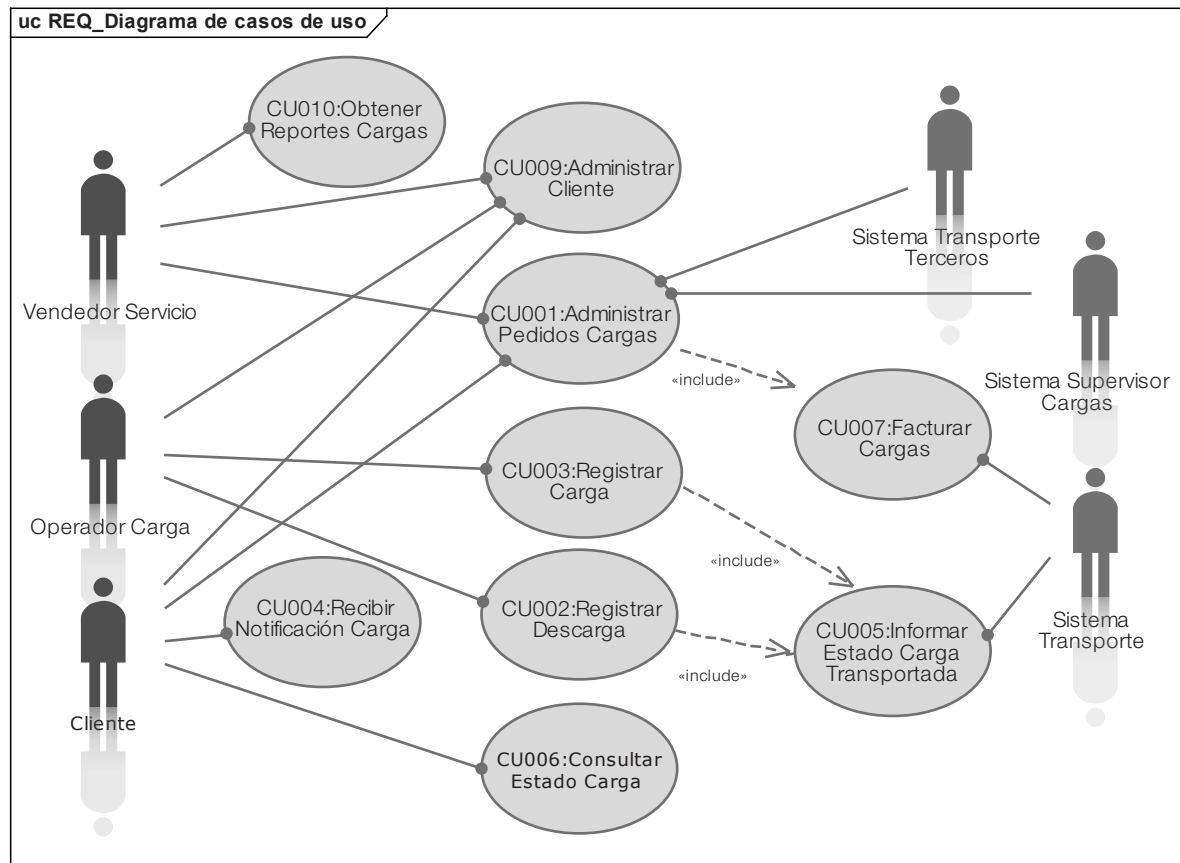


Figura 10.5. Diagrama de casos de uso preliminares del sistema del caso testigo

A partir del alcance que se determina de la información relevada estamos en condiciones de establecer una estimación de esfuerzo que permita realizar una planificación más precisa que la expresada en la fase de preventa. Estos temas serán abordados en detalle en los capítulos de gestión de proyectos.

10.1.1.3 Acuerdo del proyecto

A partir del alcance definido y de la estimación de esfuerzo basada en este, se acuerda el proyecto. Existe una forma de desambiguar los acuerdos respecto del alcance y es la inclusión en la ERS de una sección que

exprese en forma explícita “qué” no se construirá. La experiencia nos muestra que la mayor parte de los casos en que son incluidas se convierte en un tema de discusión entre las partes, lo cual indica que existió alguna ambigüedad en el alcance definido. Esta sección de no inclusiones funciona como un disparador al momento de establecer el acuerdo del proyecto y ayuda a cerrar y acordar el alcance definitivo.

Cuando se trabaje en cada una de las iteraciones del ciclo de vida del proyecto, se refinarán los casos de uso y se especificarán a efectos de elaborar un diseño detallado orientado a su implementación. También se elaborarán casos de prueba en base a los casos de uso refinados. Todas estas cuestiones serán tratadas a lo largo del libro en próximos capítulos.

10.1.2 Técnicas de relevamiento

Para lograr obtener el análisis detallado presentado en las secciones anteriores y que se verán en las subsiguientes existen diferentes técnicas que pueden utilizarse. El principal objetivo de estas técnicas, denominadas *de relevamiento*, es entender las necesidades del cliente; es decir, comprender los requisitos y el porqué de estos. Además buscan obtener las propiedades del dominio del sistema actual, si es que existe, interfaces, escenarios posibles, potenciales conflictos y riesgos. No nos olvidemos que el cliente utiliza un lenguaje ubicuo que intentamos comprender para plasmar sus necesidades implícitas y explícitas como requerimientos.

Por último, es importante destacar que no existe una técnica de relevamiento mejor que otra, depende el contexto del proyecto es conveniente utilizar una u otra. Cuando nos referimos al contexto del proyecto hacemos referencia al tiempo, a los recursos disponibles y de la clase de información que se precisa elicitar (relevar).

Entre las técnicas de relevamiento podemos mencionar las siguientes:

Entrevistas

Es una técnica simple y directa. Puede ser muy útil para descubrir requerimientos ocultos por medio de la comunicación directa y la exploración de distintas soluciones. En primera instancia, las entrevistas requieren la identificación de los candidatos a ser entrevistados. Generalmente se toman en cuenta a los *stakeholders* (interesados en el desarrollo a llevar a cabo, puede ser el cliente y/o usuario) como participantes de estas reuniones. Además debe tenerse en cuenta como participantes a los representantes del equipo que desarrollará el sistema (analista, líder de proyecto, desarrollador, arquitecto). Para lograr obtener los requerimientos, el equipo que desarrollará la aplicación guiará la reunión buscando establecer el perfil del cliente, evaluar el problema, comprender el entorno, evaluar la oportunidad, la fiabilidad y el desempeño requerido. Luego analizar lo recolectado para comprenderlo y continuar con las entrevistas focalizándolas en entender el problema real del cliente y evaluar la/s posible/s solución/es. Examinar los problemas del cliente y las soluciones que podríamos brindarle a cada uno de ellos. Cuando la reunión haya finalizado, es conveniente resumir los puntos más importantes e identificar los problemas en forma prioritaria para el cliente.

Cabe mencionar que es crucial llevar a cabo la reunión. Es la forma de entender su contexto y sus necesidades en forma directa. No es posible sustituir el contacto personal por medio del envío de un mail con el cuestionario.[1]

Simulación de usuario

Es una técnica de relevamiento de requerimientos que facilita el estudio y la comprensión del ámbito del usuario. El analista toma la posición del usuario y así se integra al ambiente de trabajo para el cual se hará el desarrollo. Este método le permite comprender sus necesidades.

La desventaja que presenta esta técnica es que consume un tiempo considerable para llevar a cabo la simulación. Es decir que es una técnica costosa.

Historietas (Storyboard)

Esta técnica permite analizar la reacción del usuario ante la propuesta que tiene el equipo que desarrollará el sistema. Asimismo proporciona una respuesta temprana sobre el sistema por parte del usuario y/o cliente, a la vez que es fácil de crear y modificar.

Generalmente se definen tres tipos de historietas, aunque en la práctica no hay reglas definidas, lo que permite utilizar la imaginación para su elaboración. Las clases de historietas las podemos agrupar en función del modo de interacción con el usuario y/o cliente como: historietas pasivas, activas e interactivas.

Las historietas pasivas narran una historia al usuario por medio de imágenes, capturas de pantallas o presentaciones. Las historietas activas son una película que no ha sido producida aún, lo cual le permite al usuario/cliente una perspectiva del uso típico de la aplicación. Por último, las historietas interactivas requieren la participación activa del cliente. Pueden ser implementadas utilizando simuladores o una presentación interactiva.

Como podrá suponerse, las historietas interactivas son más costosas que las pasivas. Por lo tanto la utilización de cada una de estas técnicas dependerá de la complejidad y los riesgos del sistema. Es posible, si se considera útil, comenzar por la técnica pasiva, continuar por la activa y por último implementar las historietas interactivas.

Durante el progreso de las historietas es importante detectar los actores que interactúan con el sistema que estamos definiendo, los elementos que representan el comportamiento de los usuarios cuando interactúan con la aplicación y viceversa; y, por último, cómo ocurre la interacción entre el usuario y el sistema.

Finalmente, queremos destacar que no es conveniente invertir mucho tiempo en la elaboración de las historietas, es altamente probable que el usuario/cliente desee hacer cambios. [1].

Prototipado

En esta técnica se elabora un modelo funcional del sistema. Esto le permite al usuario tener una idea del flujo y del estilo de la aplicación. Se utiliza cuando existe cierta incertidumbre sobre los requerimientos o cuando es necesaria una respuesta rápida por parte de los usuarios y/o clientes.

A partir del uso del prototipo el usuario emitirá su opinión para refinar los requerimientos. Lo más conveniente es desarrollar un prototipo de las funcionalidades más complejas o de aquellas que más se desconozca.

Existe una amplia variedad de técnicas de prototipado que abarcan desde versiones beta de los productos hasta esquemas de diseños de pantallas en papel. Para el caso de las versiones beta, permiten ser reutilizadas como base para el posterior desarrollo. En otros casos los prototipos pueden descartarse.

Esta técnica puede emplearse en combinación con otras técnicas de elicitación de requerimientos. También puede ser utilizada para el relevamiento de requerimientos como para su validación, como se verá en el capítulo 12.

10.2 Casos de uso

10.2.1 Paquetes funcionales

10.2.1.1 Conceptos extraídos del dominio del problema

Hasta aquí contamos con un conjunto de casos de uso preliminares del sistema a desarrollar. Para poder administrar de mejor forma el conjunto, agruparemos los casos de uso por medio de paquetes funcionales teniendo en cuenta el modelo de dominio de la figura 10.6.

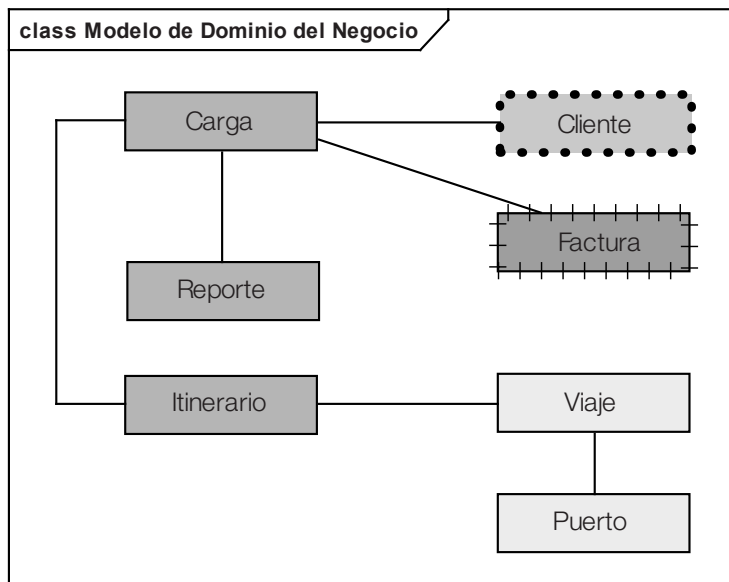


Figura 10.6. Diagrama del modelo de dominio de negocio

Los paquetes fueron elegidos a partir de la separación de incumbencias/aspectos y centrados en las entidades más representativas de los diferentes aspectos. En la figura 10.7 se presenta un diagrama de paquetes centrados en carga, factura, cliente, roles y viaje. A este último se lo denominó **Logística** ya que abarca más que el concepto de viaje. Puede verse en la misma figura que los casos de uso fueron dispuestos en los paquetes. La relación entre los paquetes se muestra como un vínculo de dependencia.

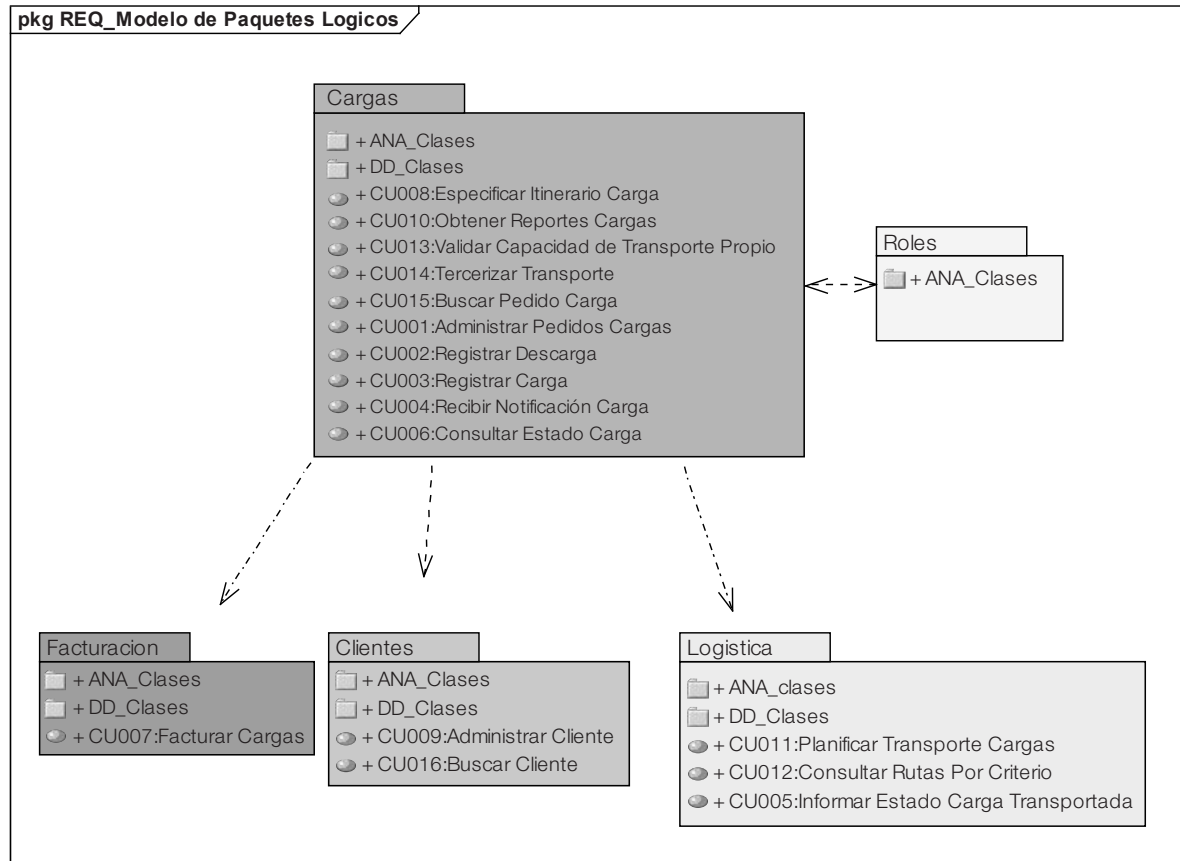


Figura 10.7. Diagrama del modelo de paquetes lógicos

Cada uno de los paquetes funcionales contiene un conjunto de casos que permiten obtener un diagrama de casos de uso más detallados. Para el paquete de Cargas diagramamos dos escenarios diferentes.

En este momento de la actividad de refinar los casos de uso surgen nuevos como resultado de este refinamiento. Para cada uno de los dos escenarios pueden verse casos de uso que no estaban presentes en el diagrama preliminar mostrado en la figura 10.5.

En la figura 10.8 se muestra el diagrama de casos de uso correspondiente a la administración de las cargas y su interacción con los sistemas externos de transporte, transporte de terceros y supervisor de cargas.

En la figura 10.9 se presenta el diagrama de casos de uso correspondiente a las consultas del estado de las cargas.

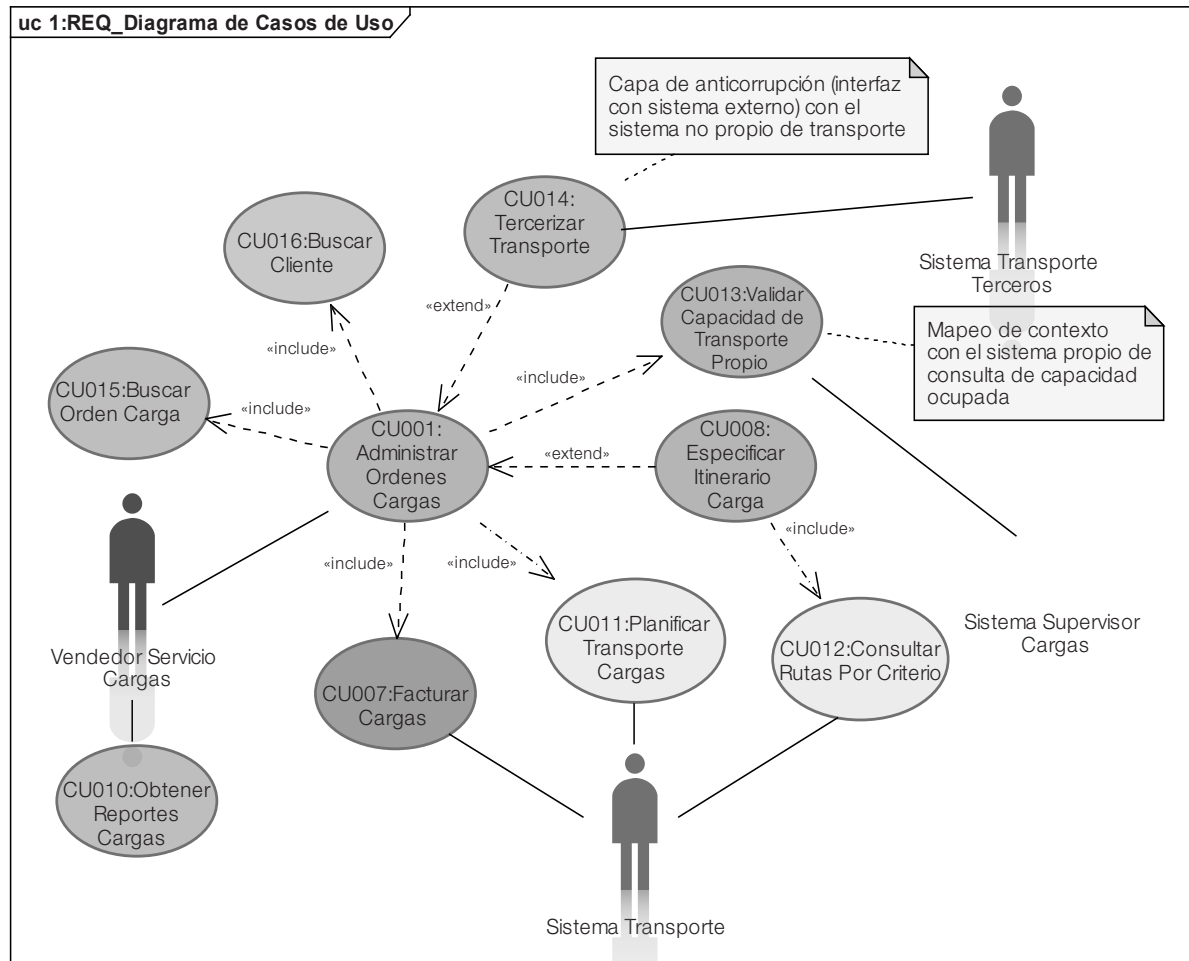


Figura 10.8. Diagrama de casos de uso de Cargas, relación con otros sistemas

Cuando arribamos a este punto, a partir del análisis realizado sobre los requerimientos y casos de uso, y de la información obtenida del relevamiento, estamos con condiciones de especificar los detalles de los casos de uso del sistema. A modo de guía comenzaremos detallando el relevamiento efectuado, por medio de la técnica de prototipado, sobre el caso de uso “CU044: Buscar Pedido de Carga”.

La primera pantalla presentada en la figura 10.10 despliega los campos del dominio requeridos por el sistema para lograr obtener la carga solicitada por el usuario. Los datos que son demandados surgen del negocio de la aplicación y del refinamiento que se lleva a cabo junto al cliente. Todos ellos (identificador, fecha de partida, puerto origen, puerto destino, tipo de carga y nombre de cliente) están directamente relacionados con la entidad OrdenServicio del modelo de dominio que se presenta en el próximo capítulo. Es decir, son atributos de la solicitud del servicio de carga y es necesario que el sistema los conozca para poder identificar una carga específica.

La segunda pantalla es el resultado de haber ingresado los datos de la solicitud de servicio carga solicitada en el primer paso de la búsqueda. Es decir que la aplicación utilizará todos los datos ingresados para buscar la carga y presentar toda la información al usuario. El resultado de la búsqueda se muestra en la segunda pantalla de la figura 10.10.

uc 2:REQ_Diagrama de Casos de Uso

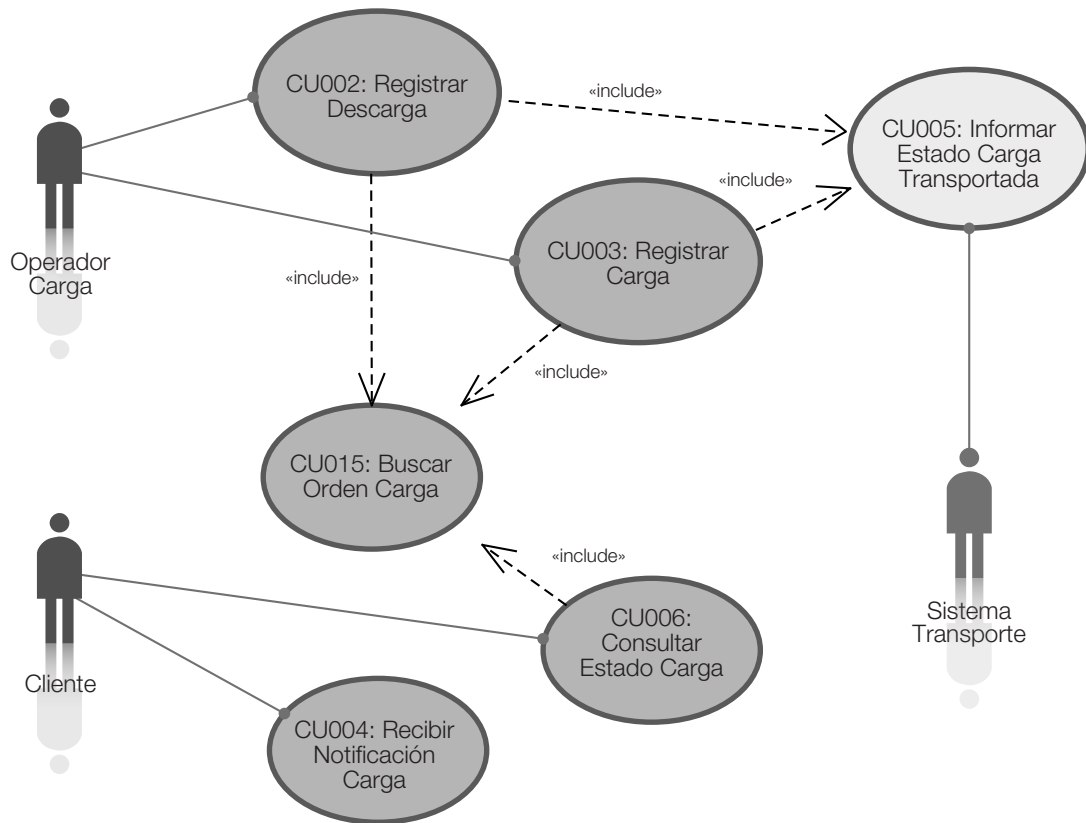


Figura 10.9. Diagrama de casos de uso de Cargas, consultas de estado de cargas

La navegación y los datos presentados en el prototipo definen el flujo de la aplicación y nos permiten especificar el caso de uso que se muestra a continuación.

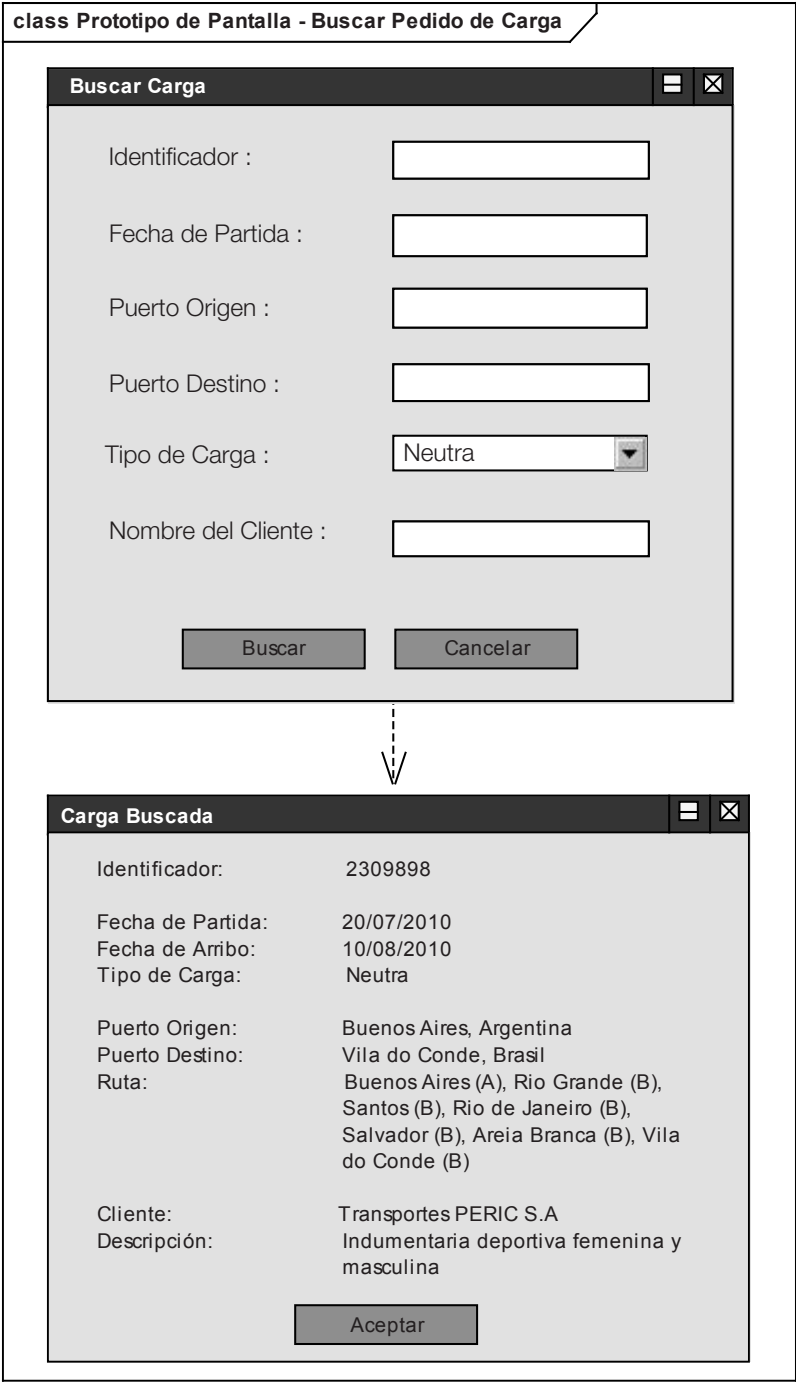


Figura 10.10. Prototipo de pantalla del caso de uso “CU015: Buscar Orden de Carga”

Caso de Uso: CU015 Buscar Orden de Carga

Descripción del caso de uso

“Buscar pedido de carga” permite llevar a cabo la búsqueda y visualización de un pedido de carga específico.

Actores

El actor involucrado en el caso de uso es el vendedor del servicio de cargas.

Precondiciones

No hay precondiciones para el caso de uso.

Activación

Se activará cuando el vendedor del servicio de cargas solicite buscar un pedido de cargas.

Poscondiciones

Se presenta al usuario el detalle del pedido de carga buscado.

Campo	Tipo	Obligatorio	Valor defecto	Solo lectura	Tipo componente
Identificador	Númérico	Sí	-	No	Caja de texto
Fecha de partida	Fecha	No	-	No	Caja de texto
Puerto origen	Texto	No	-	No	Caja de texto
Puerto destino	Texto	No	-	No	Caja de texto
Tipo de carga	Texto	No	Neutra	No	Combo
Nombre del cliente	Texto	No	-	No	Caja de texto

Flujo principal

1. El vendedor de servicio de cargas requiere buscar un pedido de carga.
2. El sistema le solicitará ingresar los datos (identificador de carga, fecha de partida, puerto origen, puerto destino, tipo de carga, cliente) pertinentes para poder identificar el pedido de carga.
3. El vendedor ingresa los datos válidos solicitados y presiona “Buscar”.
4. La aplicación despliega toda la información relativa al pedido de carga solicitado (identificador de carga, fecha de partida, puerto origen, puerto destino, ruta, datos del cliente, tipo de carga, notas especiales).

Flujos alternativos

A) No existen pedidos de carga que satisfagan la búsqueda

4. La aplicación despliega un mensaje informando que no existe ningún pedido de carga que satisfaga la búsqueda.

B) El usuario cancela la búsqueda

3. El vendedor selecciona “Cancelar” y anula la búsqueda.
4. La aplicación despliega la pantalla principal.

Flujos de validación

A) Los datos ingresados no son correctos o son escasos

3. El vendedor ingresa datos erróneos o los datos ingresados no son suficientes para lograr la búsqueda.
4. La aplicación despliega un mensaje informando que algún dato es erróneo o que no se han encontrado resultados que apliquen a la búsqueda.

Figura 10.11. Especificación del caso de uso “CU015: Buscar Orden de Carga”

Es interesante destacar que en el paso 2 del flujo principal de la especificación se menciona el ingreso de los siguientes datos: identificador de carga, fecha de partida, puerto origen, puerto destino, tipo de carga y cliente. Esta información ha surgido del prototipo correspondiente a este caso de uso presentado previamente. Lo mismo ocurre con el paso 4 del mismo flujo.

A continuación presentaremos el prototipado para un caso de uso que está relacionado con otros: “CU003: Registrar Carga”.

La primera pantalla le solicita al usuario ingresar los datos necesarios para poder registrar la carga. Nuevamente la información solicitada son atributos de la entidad OrdenCarga. Una vez que el usuario ingresa todos los datos y presiona el botón “Registrar” la aplicación procede a almacenarlos en forma permanente informando el número que ha utilizado para identificar la carga. Así el usuario podrá emplear este número para buscar la carga como se ha visto en el caso de uso descrito previamente.

class Prototipo de Pantalla - Registrar Carga

Registrar Carga

Fecha de Partida :

Puerto Origen :

Puerto Destino :

Nombre del Cliente :

Tipo de Carga :

Neutra

Descripción :

Registrar

Cancelar

↓

Carga Registrada

Los datos de la carga han sido
almacenados satisfactoriamente con
el siguiente número de identificación:
10067890928

Aceptar

Figura 10.12. Prototipo de pantalla correspondiente al caso de uso “Registrar Carga”

En este punto nos encontramos en condiciones de especificar el caso de uso “CU003: Registrar Carga”.

Caso de Uso: CU 003 Registrar Carga

Descripción del caso de uso

“Registrar carga” permite registrar las cargas para su posterior administración.

Actores

Los actores involucrados son el operador de carga y el sistema de transporte.

Precondiciones

No hay precondiciones para este caso de uso.

Activación

Se activará cuando el operador de carga necesite registrar una nueva carga.

Poscondiciones

Los datos de la carga han sido almacenados en el sistema.

Definición de datos

Campo	Tipo	Obligatorio	Valor defecto	Solo lectura	Tipo componente
Fecha de partida	Texto	Sí	Fecha actual	No	Caja de texto
Puerto origen	Texto	Sí	-	No	Caja de texto
Puerto destino	Texto	Sí	Fecha actual	No	Caja de texto
Tipo de carga	Texto	Sí	-	No	Combo
Nombre del cliente	Texto	Sí	-	No	Caja de texto
Descripción de la carga	Texto	No	-	No	Caja de texto

Flujo principal

- El operador de carga requiere registrar una nueva carga.
- El sistema le solicitará ingresar los datos de la nueva carga (fecha de partida, puerto origen, puerto destino, tipo de carga, cliente, descripción de la carga) para poder registrar la carga correctamente.
- El operador ingresa los datos válidos solicitados y presiona “Guardar”.
- La aplicación buscará la carga registrada (CU044: Buscar Orden de Carga) para verificar que no haya sido registrada previamente y luego procederá a almacenar en forma permanente la información ingresada por el operador. Por último, desplegará un mensaje informando que los datos de la carga (CU005: Informar Estado de Carga Transportada) se han guardado satisfactoriamente con número de identificación respectivo.

Flujos alternativos

A) Ocurre un error durante el proceso de almacenamiento de los datos

4. La aplicación despliega un mensaje informando que ha ocurrido un error durante el almacenamiento de los datos.
5. Continúa en el paso 2 del flujo principal.

B) El operador cancela la registración

3. El operador selecciona “Cancelar” y anula el proceso de registración.
4. La aplicación despliega la pantalla principal.

Flujos de validación

A) Los datos ingresados son inválidos

3. El vendedor ingresa uno o más datos inválidos y selecciona “Guardar”.
4. La aplicación despliega un mensaje informando que se ha ingresado un dato inválido.
5. Continúa en el paso 3 del flujo principal.

B) Los datos ingresados ya se encuentran registrados en la aplicación

3. El vendedor ingresa los datos de una carga que ha sido registrada en la aplicación previamente y presiona “Guardar”.
4. La aplicación despliega un mensaje informando que los datos ya han sido registrados en el sistema.

Figura 10.13. Especificación del caso de uso “Registrar Carga”

Cabe remarcar que los datos requeridos y presentados por la aplicación surgen del dominio del negocio. Asimismo se puede observar que la información que ingresa el usuario para registrar una carga coincide con la que se presenta cuando el usuario busca una carga.

A continuación se presentan algunos diagramas más de casos de uso que serán tenidos en cuenta para la especificación de los casos de uso de logística (Figura 10.14), facturación de cargas (Figura de 10.15) y de clientes (Figura 10.16).

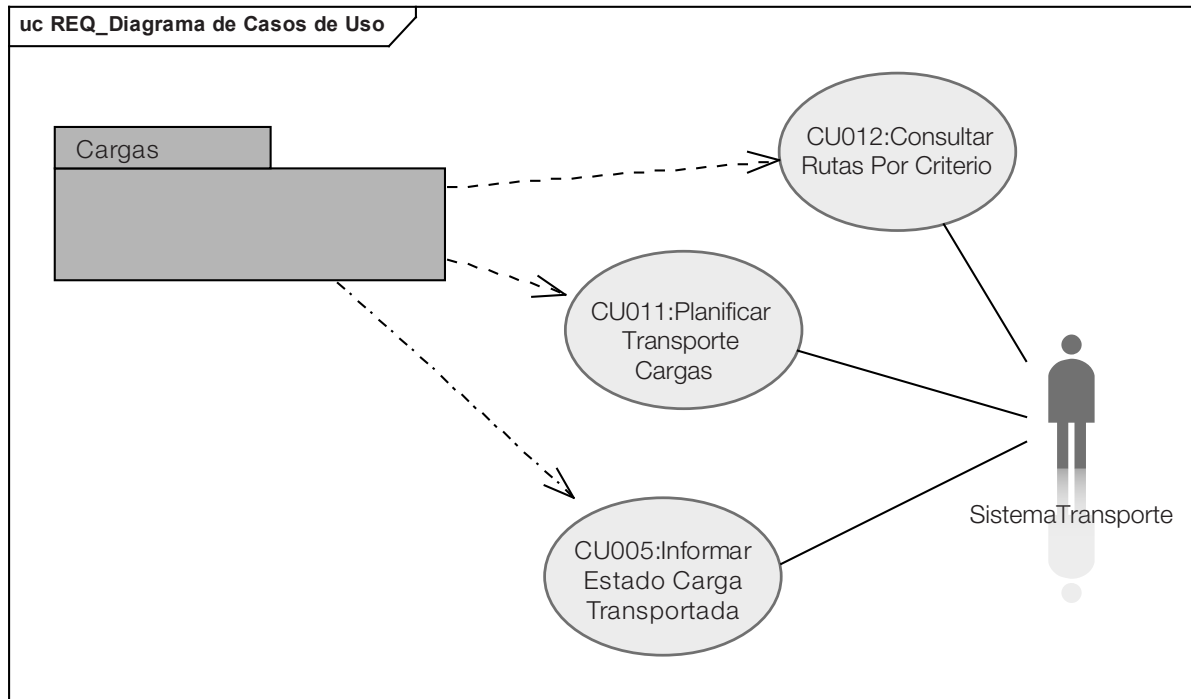


Figura 10.14. Diagrama de casos de uso de la logística de las cargas

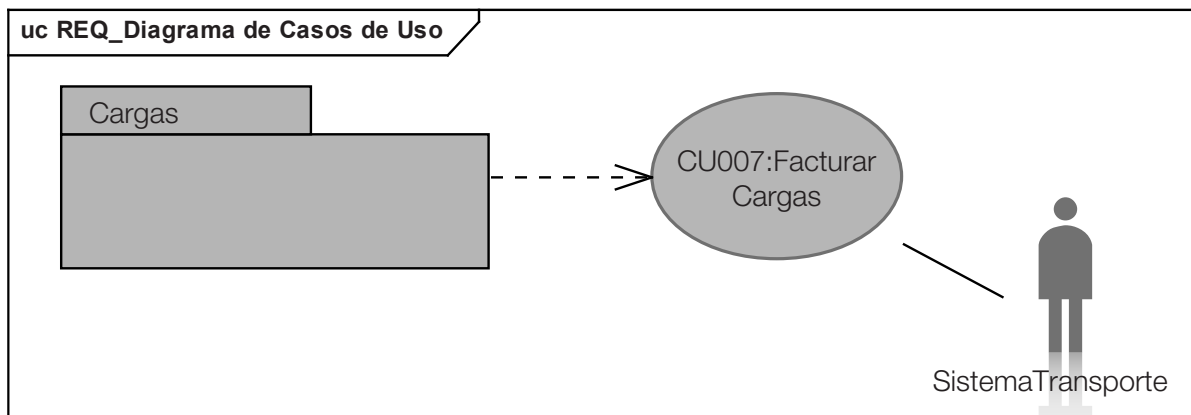


Figura 10.15. Diagrama de casos de uso de Facturación de Cargas

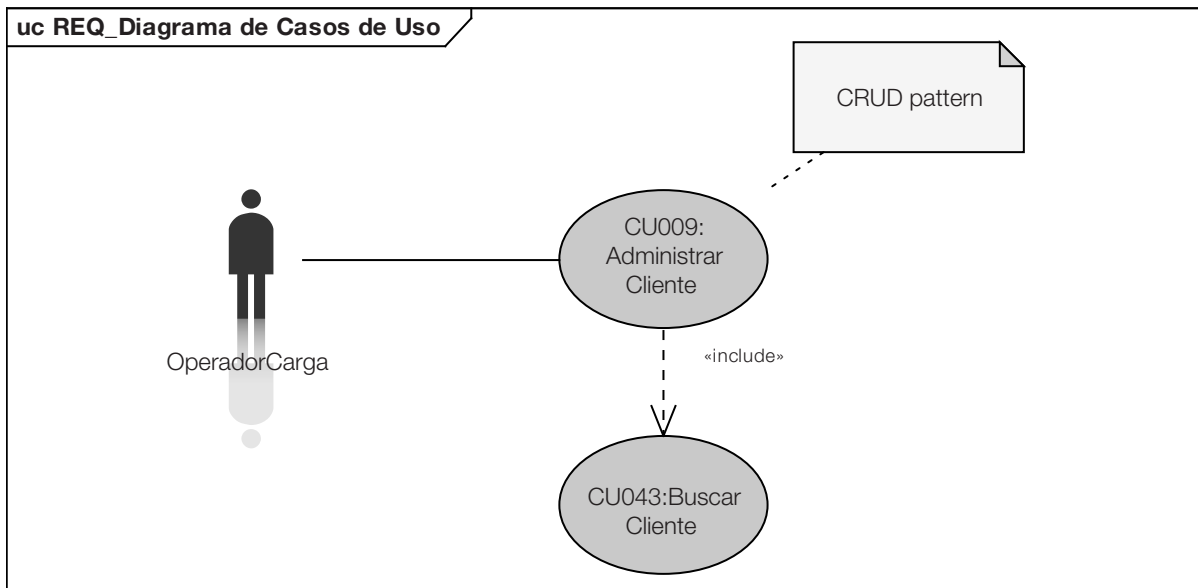


Figura 10.16. Diagrama de casos de uso de Clientes

10.3 Organización del trabajo con requerimientos

10.3.1 Planificación

Al momento de la planificación del proyecto es importante organizar el trabajo con los requerimientos ya que con la realización de estas tareas se generarán los primeros productos que permitirán definir el alcance y determinar el esfuerzo necesario para su implementación. Algunos de los temas a tener en cuenta son: quiénes son los involucrados por parte de la empresa que encarga el desarrollo, el lugar donde se realizarán estas tareas, los días y horarios de trabajo, material necesario a disponer, como por ejemplo documentos o bases de datos, sistemas existentes en funcionamiento, etcétera.

10.3.1.1 Forma de trabajo

La forma de trabajo que se adopta en cada proyecto depende de sus características, de la complejidad del negocio a relevar, de la aptitud y actitud de los clientes y usuarios y de la organización del grupo de desarrollo. Las técnicas de relevamiento descriptas anteriormente deben ser seleccionadas a partir de estas características. Las sesiones de trabajo con referentes del negocio deben ser planificadas de manera de cubrir todos los aspectos del negocio con el objetivo de obtener un entendimiento completo de las necesidades de los usuarios y del ambiente operacional donde deberá funcionar el sistema en desarrollo. La concepción temprana de módulos que agrupen funcionalidades es de importancia ya que facilita un enfoque ordenado.

10.3.1.2 Productos generados

El modelo de requerimientos incluye los siguientes artefactos que son vistas obtenidas en diferentes momentos del ciclo de vida del proyecto de desarrollo del software.

Listado de requerimientos: incluye los requerimientos funcionales, no funcionales y del proceso de desarrollo (Ej. documentación requerida).

Diagramas de casos de uso: incluye diagramas, especificación, prototipos de interfaces y diseño conceptual de interfaces.

Modelo de negocio: incluye diagrama de clases del dominio con reglas de negocio asignadas.

A veces, cuando los negocios son complejos es útil construir diagramas de actividad que describan los flujos de trabajo de las diferentes áreas de negocio.

10.4 Priorización de requerimientos

Un aspecto importante al momento de realizar la planificación de los proyectos es el establecimiento de las prioridades de los requerimientos, las cuales permitirán ordenar el desarrollo del sistema. Estas prioridades deben ser analizadas por la organización cliente debido a que las necesidades del negocio son las que las fijan y deben ser acordadas con los responsables del proyecto por parte de la empresa desarrolladora que elabora la planificación. Para ayudarse en esta tarea existen pautas derivadas desde el valor agregado y evolución financiera del proyecto, deseo de los usuarios y riesgos de implementación.

Para ejemplificar estas tareas presentaremos el análisis realizado en el siguiente proyecto, cuya descripción abreviada hacemos a continuación.

A partir del relevamiento preliminar surgieron una serie de problemas a resolver que determinan la visión del proyecto.

Claramente existen dos necesidades a satisfacer, por un lado el ordenamiento de la **administración de las cargas** en su fase de contratación, programación de ruta, definición de transporte y **facturación**. Esta problemática afecta al área de la organización que trata directamente con los clientes y además genera la información para que la empresa aseguradora determine su participación en cada contrato. Los inconvenientes derivados de no contar con un sistema que permita generar las contrataciones en línea y programar el transporte sin retrasos afectan al negocio desde siempre. Por otro lado se ha presentado una oportunidad de negocio que generaría ingresos adicionales importantes a la empresa. El contrato que lo permitiría, con una empresa fabricante de computadoras, exige una mejora de la calidad del servicio, fundamentalmente en lo referido al **seguimiento de las cargas durante los viajes**. La concreción de dicho contrato está condicionada a la posibilidad de contar con un sistema que permita el seguimiento.

Realizamos un análisis de flujo financiero del proyecto para cada módulo según las variables que se muestran en el encabezamiento de la tabla 10.1, la cual sirve de modelo y no contiene datos.

Módulo de Seguimiento de Cargas: para este módulo se requerían 4 meses de desarrollo con el equipo de proyecto, por lo que por ese lapso de tiempo no ingresarían sino egresarían recursos. Además es necesario instalar en cada terminal de carga y descarga equipos para interactuar con el sistema. A partir del quinto mes se debe gastar en mantenimiento del sistema.

A partir del sexto mes se contaría con **nuevos ingresos** provenientes del nuevo contrato. Se especuló que los **ingresos nuevos** se captarían a partir del incremento en el volumen del 10% de las actuales cargas mensuales por contar ahora con seguimiento en línea y que el 15% de las actuales cargas mensuales que no se repiten por falta de seguimiento serían **retenidas**. No se detectaron mejoras en la eficiencia operacional. El flujo financiero del proyecto para este módulo se muestra en la figura 10.17 en la curva inferior. Puede verse que a partir del cuarto mes, de ser posible instalar el sistema por partes, el desarrollo de este módulo generaría ingresos.

Un análisis similar se realizó para los otros módulos con los resultados mostrados en la figura 10.17 con diferentes curvas.

Tabla 10.1. Análisis del flujo de dinero						
Módulo						
Meses	Ingresos nuevos	Ingresos incrementados	Ingresos retenidos	Eficiencia operacional	Costo desarrollo	Flujo dinero

Si bien el módulo de Seguimiento de Cargas es el que generará mayor ingreso genuino también es el que necesita una mayor inversión inicial. Debido a compromisos financieros varios, la empresa duda en afrontarla. El módulo de Administración de Cargas, mostrado en la figura con curva de bolitas, también generará sus beneficios a partir, en este caso, de ingresos retenidos por la prolijidad al momento de las contrataciones y de la mejora en la eficiencia operacional al no realizar tareas de facturación y otras actividades manuales así como ahorro en gastos de comunicaciones telefónicas.

A efectos de balancear los beneficios con las erogaciones, primero se pensó en desarrollar el módulo de Administración de Cargas y con los beneficios generados soportar los gastos del desarrollo del módulo de Seguimiento de Cargas. Sin embargo se determinó que esto no era posible ya que el nuevo contrato debía firmarse y comenzar a prestar los servicios asociados en 6 meses. Cuando presentemos en próximos capítulos los temas que tratan la gestión de los proyectos retomaremos este tema y contaremos las decisiones que se tomaron.

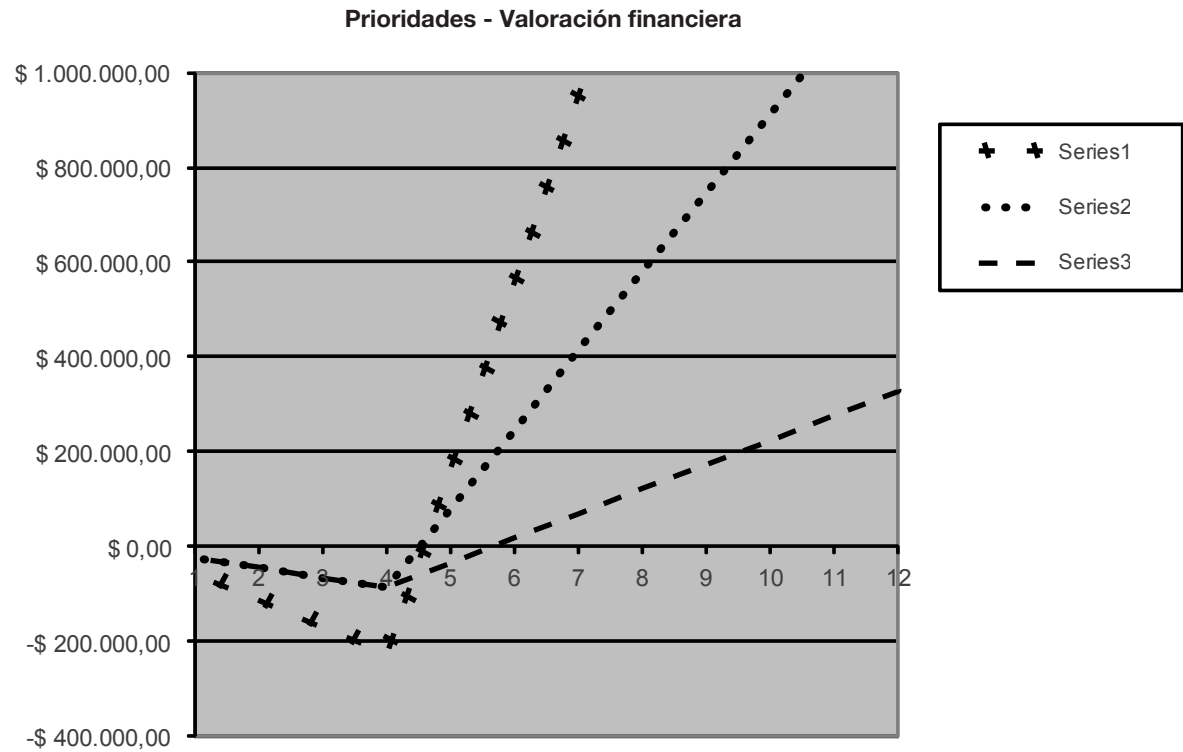


Figura 10.17. Análisis del flujo de dinero del proyecto

Otro análisis que se realizó a efectos de fijar las prioridades del proyecto se basó en contemplar el deseo de contar con determinadas funcionalidades por parte de los miembros de la empresa. Se segmentó a la población en dos grupos, uno formado por la gerencia, cuyo resultado se muestra en la tabla 10.2, y otro por los empleados que realizan las tareas de administración, los resultados se presentan en la tabla 10.3.

Tabla 10.2. Prioridades determinadas a partir de la opinión del grupo de gerentes							
Prioridad según deseo de miembros gerenciales							
Requerimiento	Beneficio Relativo	Penalidad Relativa	Valor Total	% Valor	Esfuerzo [horas]	% Costo	Prioridad
Seguimiento	9	6	15	41,67	536	12,57	3,31
Administración	6	3	9	25,00	1448	33,96	0,74
Cientes Facturación	7	5	12	33,33	2280	53,47	0,62
Total	22	14	36	100,00	4264	100	

Es interesante mencionar que debido a los diferentes intereses de ambos grupos, las prioridades resultaron diferentes.

Tabla 10.3. Prioridades determinadas a partir de la opinión del grupo de administrativos							
Prioridad según deseo de miembros empleados administrativos							
Requerimiento	Beneficio Relativo	Penalidad Relativa	Valor Total	% Valor	Esfuerzo [horas]	% Costo	Prioridad
Seguimiento	4	3	7	20,59	536	12,57	1,64
Administración	8	6	14	41,18	1448	33,96	1,21
Cientes	7	6	13	38,24	2280	53,47	0,72
Facturación							
Total	19	15	34	100,00	4264	100	

Los empleados priorizaron contar con la funcionalidad asociada al módulo de Administración de Cargas que sería el que les facilitaría su trabajo diario, mientras la prioridad para el otro grupo fue coincidente con el resultante del flujo de dinero.

Tabla 10.4. Prioridades determinadas a partir de evaluar conocimiento del negocio, valor agregado y riesgo de implementación				
Prioridad valorando conocimiento, valor y riesgo				
Requerimiento	Conocimiento del Negocio	Valor Agregado al Negocio	Riesgos	Prioridad
Seguimiento	Medio	Alto	Medio	Media
Administración	Alto	Medio	Alto	Alta
Cientes				
Facturación	Bajo	Bajo	Medio	Baja

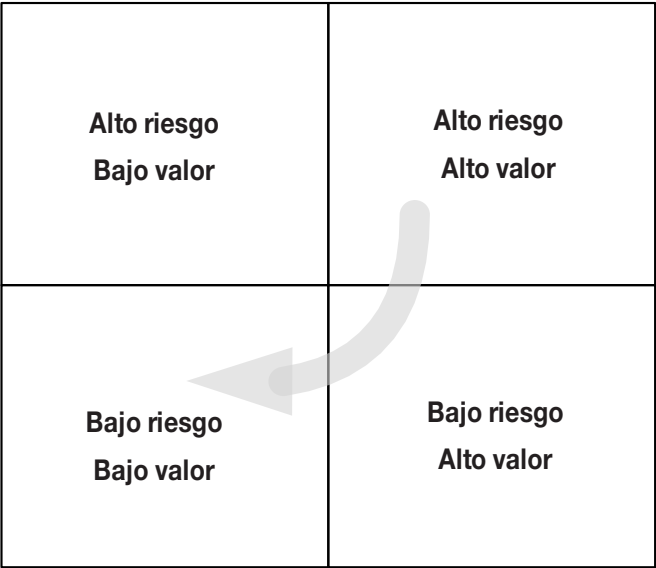


Figura 10.18. Estrategia de implementación según el valor agregado y el riesgo

Por último, en la tabla 10.4 se muestra el análisis realizado en el que se incorporó el conocimiento de negocio que el grupo de desarrollo obtendría por abocarse al desarrollo de cada módulo. Es deseable desarrollar aquellos de los cuales se obtendrá mayor conocimiento, debido a que estos facilitarán el desarrollo de los posteriores. También se incluyó el valor agregado al negocio y el riesgo de implementación. Puede verse que en este caso debido a su riesgo mayor la prioridad del módulo de administración resultó mayor. Esto es debido a que la estrategia a seguir, mostrada en la figura 10.14, expresa que se debe comenzar por los componentes de alto riesgo y alto valor agregado, y continuar priorizando el valor con menor riesgo. Siempre evitando aquello altamente riesgoso y de poco valor agregado.

Este ejercicio de priorización es de fundamental importancia. Los criterios aplicados se derivan de la visión del proyecto y están ligados a los objetivos buscados con la construcción del sistema en estudio. Cuando tratemos los aspectos relacionados a la planificación de los proyectos volveremos sobre este tema.

10.5 Conclusión

Se presentaron dos de los artefactos más importantes del trabajo con los requerimientos, como son los requerimientos y los casos de uso asociados.

Se indicó la importancia de priorizar estos requerimientos y se mostró una forma de hacerlo. Asimismo se mencionaron las técnicas tradicionales para relevar los requerimientos, de forma tal de brindar las herramientas más útiles en función de los distintos contextos que puedan presentarse en los desarrollos de software.

Se puntualizaron los aspectos salientes de la planificación de las tareas asociadas a la construcción de estos activos.

En el próximo capítulo se analizarán estos requerimientos buscando capturar las reglas del negocio en el modelo de dominio refinado generando lo que llamaremos un *modelo de negocio*.

10.6 Contenido de la página web de apoyo



10.6.1 Mapa conceptual

10.6.2 Autoevaluaciones

10.6.3 Presentaciones en Power Point (exclusivo para el docente)

10.7 Referencias

- [1] *Managing Software Requirements: A Use Case Approach*, 2/E, Dean Leffingwell, Don Widrig, Addison-Wesley Professional.
- [2] *Requirements by Collaboration: Workshops for Defining Needs*, Ellen Gottesdiener, Pearson Education, 2002.
- [3] *Agile Estimating and Planning*, Mike Cohn, Prentice hall, 2005.

Análisis de requerimientos

Contenido

11.1 Análisis de requerimientos	168
11.2 Conclusión	176
11.3 Contenido de la página web de apoyo	177
11.4 Referencias	177

Objetivos

- Entender el negocio sobre el cual operará el sistema en desarrollo a través de los requerimientos
- Construir modelos de dominio
- Asignar las reglas de negocio a entidades de los modelos de dominio
- Estudiar la relación entre los requerimientos y los artefactos de los modelos

Competencias

- Definir la forma en que serán utilizados los requerimientos para la construcción de un modelo de dominio que represente, en el software, el negocio del sistema en desarrollo

11.1 Análisis de requerimientos

11.1.1 Objetivos

11.1.1.1 Entendimiento del negocio

Hay dos aspectos bien diferenciados en el desarrollo de un sistema de software: la *aplicación* y el *negocio*. La aplicación está constituida básicamente por el conjunto de operaciones que los usuarios pueden llevar adelante utilizando el software. Nos referimos a estas en el capítulo anterior cuando tratamos el tema de los requerimientos. Los requerimientos funcionales y los casos de uso asociados a estos requerimientos son los que expresan el alcance de la aplicación.

Sin embargo, esta aplicación permite realizar operaciones sobre los objetos de un modelo que representa el dominio del problema del cual forma parte este sistema.

La importancia de este modelo de dominio radica en que los objetos que lo constituyen a partir de su comportamiento harán que al accionar sobre ellos desde la aplicación se logren resultados válidos. Esto es posible a partir de la asignación de las reglas de negocio a dichos objetos, los cuales controlarán que el negocio evolucione según sus reglas.

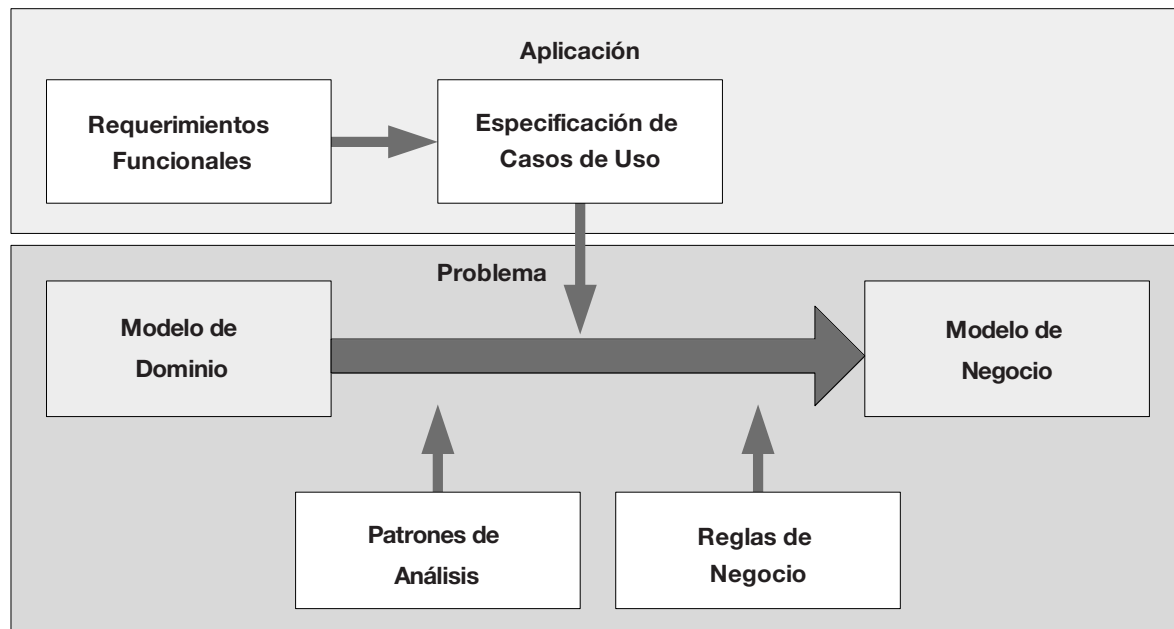


Figura 11.1. Relación entre los modelos de dominio y negocio

Deseamos nombrar claramente a cada uno de estos modelos para evitar confusiones posteriores. El obtenido a partir de los conceptos extraídos del dominio lo llamamos *de dominio* y está compuesto por entidades a las cuales llamaremos *clases*. No define el alcance de cada entidad otorgándoles atributos, solo las vincula

con otras. No se utiliza el mecanismo de herencia ya que se obtiene a partir de un ejercicio de descubrimiento y descomposición [1]. A estas clases se las refina a partir de la asignación de las reglas de negocio y se determina su alcance incorporando los atributos necesarios determinados en la especificación de los casos de uso de la aplicación. En un excelente libro [2], Jill Nicola y sus colaboradores catalogaron patrones de análisis para ser utilizados en la elaboración de este modelo de dominio refinado al que llamamos *modelo de negocio*. En la figura 11.1 se muestran las relaciones entre ambos modelos. Es importante remarcar cuál es el objetivo de la construcción de estos modelos, ¡entender el negocio! Por esta razón forman parte del trabajo con los requerimientos.

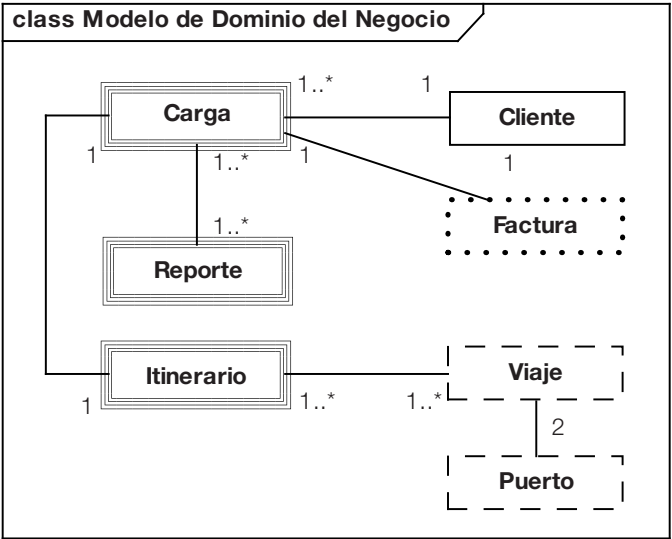


Figura 11.2. Modelo de dominio del ejemplo

En la figura 11.2 se puede observar el modelo de dominio del ejemplo en el cual se ven solo relaciones simples y la cardinalidad asociada. Estas clases aparecen señadas con las mismas tramas de los paquetes en que se subdividió el sistema y se utilizaron estas clases como punto de partida para la descomposición. Recordamos que este ejercicio se lleva adelante según un criterio de cohesión del dominio del problema. Las tareas de construcción de estos modelos se llevan adelante simultáneamente con la construcción de los diagramas de casos de uso. Estos expresan conceptos del problema y los otros la dinámica de la aplicación.

Refinando este modelo con la utilización de los patrones de análisis [2] reconocemos la presencia de patrones, que no son más que conceptos que los desarrolladores de software buscaron por años en el desarrollo de sistemas. Para el modelo del ejemplo los patrones reconocidos se muestran en la tabla 11.1

Tabla 11.1. Patrones de análisis reconocidos en el sistema del ejemplo	
Conceptos refinados	Patrones de Análisis
Persona – Cliente	Actor – Rol
Empresa - Cliente	Actor - Rol
Persona – Empleado	Actor – Rol
Cliente – Orden Servicio	Rol – Transacción
Vendedor Servicios Carga – Orden Servicio	Rol – Transacción
Orden Servicio – Ítem Carga	Transacción Compuesta – Line Ítem

Tabla 11.1. Patrones de análisis reconocidos en el sistema del ejemplo	
Conceptos refinados	Patrones de Análisis
Ítem Carga – Bulto	Line Ítem – Ítem Específico
Orden Servicio – Orden Entrega	Transacción – Transacción Cronológica
Carga – Bulto	Ensamble – Parte
Vendedor Servicios Carga – Reporte	Rol – Transacción
Contenedor - Bulto	Contenedor – Contenido
Carga – Bulto	Ensamble – Parte
Puerto – Lugar Carga y Lugar Descarga	Gran Lugar – Lugar
Etapas - Puerto	Ensamble - parte

Estos patrones también se denominan de colaboración, porque a partir de la colaboración entre los objetos se logrará la validación de las reglas de negocio cuando sean asignadas. Se muestra en la figura 11.3 el modelo resultante para el contexto de las cargas que es el que estamos analizando.

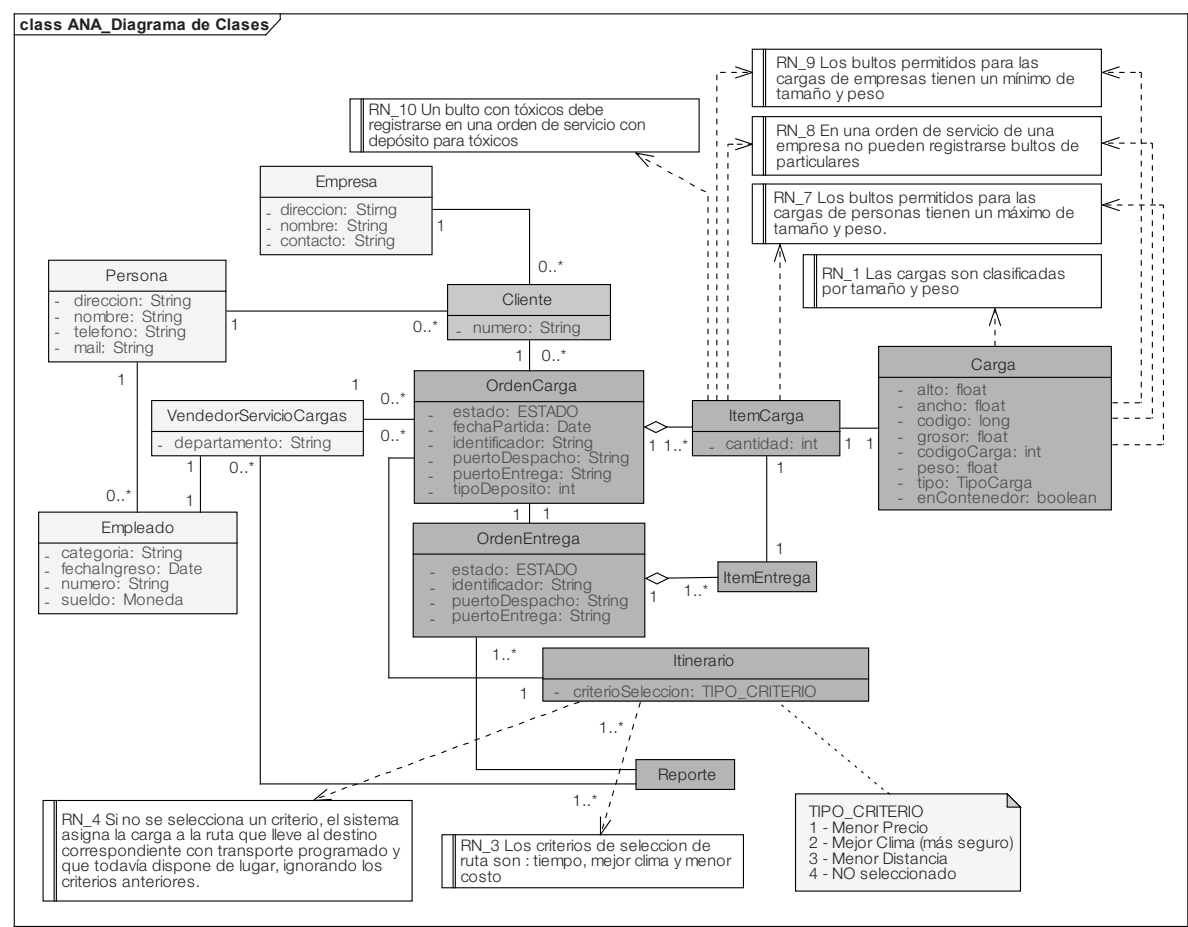


Figura 11.3. Modelo de negocio obtenido refinando el de dominio con los patrones de análisis y la asignación de las reglas para el contexto de cargas

En la figura 11.4 se presenta un diagrama de clases para otro contexto del problema, el relacionado con la logística.

Otra muy buena presentación de patrones de análisis catalogados según el dominio del problema al cual aplican lo hizo Martin Fowler en su libro *Analysis Patterns* [3] del cual pueden extraerse enfoques claros para problemas de diferentes negocios.

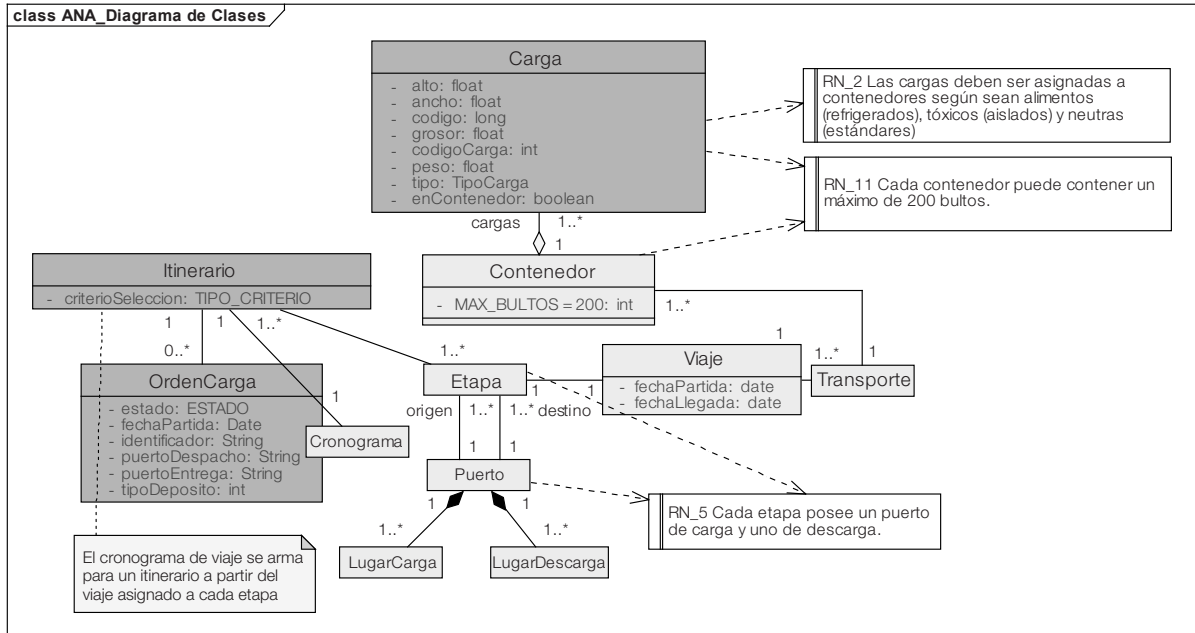


Figura 11.4. Modelo de negocio del aspecto del problema del ejemplo relacionado a los viajes y el transporte

11.1.2 Modelo del comportamiento y control

Todo dominio de cualquier problema está compuesto de conceptos y relaciones entre ellos. La forma en que es posible que estos conceptos se relacionen, los estados que puedan adquirir y cómo pueden reaccionar a determinados estímulos se conocen como *reglas*, las cuales se traducen en el modelo de negocio en validaciones a realizar al momento de establecer una relación entre objetos, crear nuevos objetos, romper con relaciones existentes y asignar valores a sus atributos. El control de la evolución de los eventos que exciten al modelo de negocio a partir de la validación de las reglas determina que el modelo represente o no al problema sobre el cual operará el sistema en desarrollo.

11.1.2.1 Reglas de Negocio – Clasificación y asignación

Por lo tanto es muy importante una correcta asignación y validación de las reglas. En la figura 11.5 se muestran las reglas de negocio relevadas para el sistema del ejemplo.

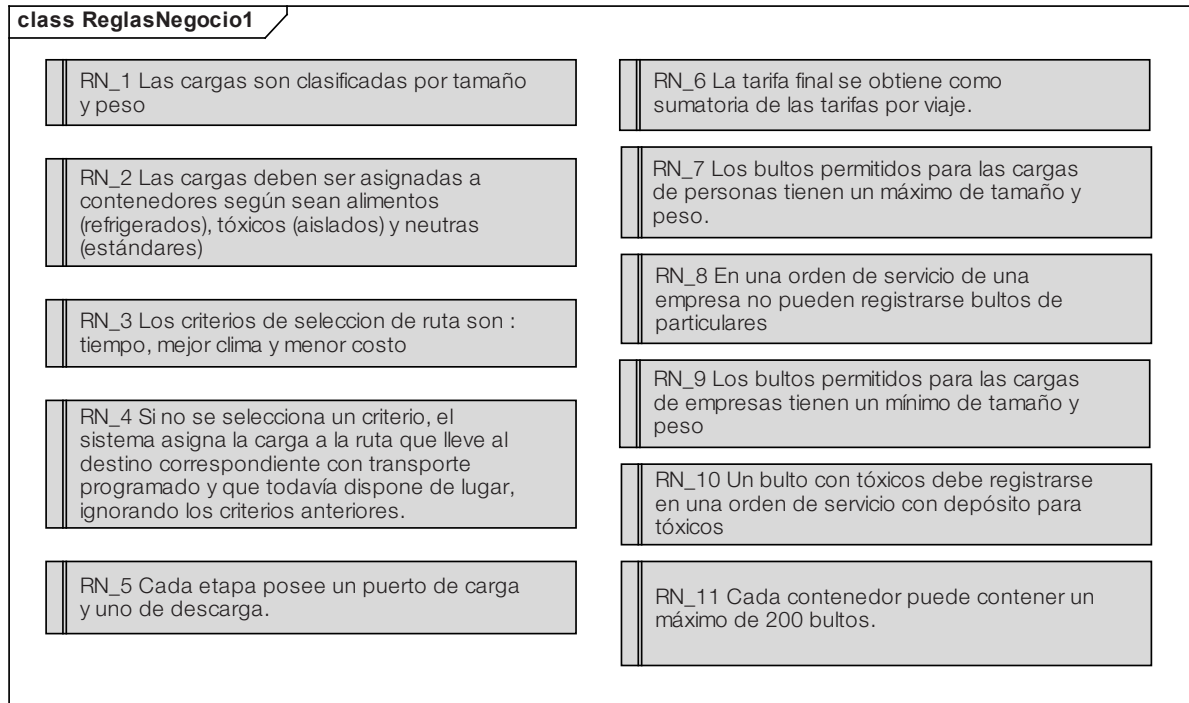


Figura 11.5. Reglas de negocio que afectan al sistema ejemplo en desarrollo

En la figura 11.3 y 11.4 se muestran algunas de ellas asignadas a los objetos que las validarán. Una clasificación de reglas según su tipo, acotada pero rica, es la que se presenta en Streamlined Object Modeling – Patterns, rules and implementations [2]. Los tipos de reglas son los mostrados en la tabla 11.2.

Tabla 11.2. Tipos de reglas de negocios (adaptada de [2])	
Tipos de reglas de negocio	
Tipo de regla	Descripción
Tipo	Determinada por la existencia de diferentes tipos.
Propiedad	Determinada a partir de representación predefinida de datos con formatos.
Multiplicidad	Determinada a partir de la cardinalidad de las relaciones entre objetos.
Estado	Determinada a partir de los estados de los objetos.
Conflicto	Determinada a partir de contradicciones en las relaciones entre objetos.

Las reglas listadas para el modelo del ejemplo son, según esta clasificación, las que se muestran en la tabla 11.3.

Tabla 11.3. Clasificación de las reglas del ejemplo de la figura 11.4	
Reglas de negocio del ejemplo	Tipo de regla
RN_1	Tipo
RN_2	Tipo
RN_3	Tipo
RN_4	Conflicto
RN_5	Multiplicidad
RN_6	Estado
RN_7	Tipo
RN_8	Conflicto
RN_9	Tipo
RN_10	Tipo
RN_11	Multiplicidad

11.1.2.2 Propiedades

Entre las propiedades que deben tener las reglas se encuentran [4]:

- Atómica:** no se puede dividir más sin perder información.
- Inequivoca:** sólo tienen una interpretación obvia.
- Compacto:** normalmente una única frase corta.
- De acuerdo:** en conjunto proporcionan una descripción unificada y coherente.
- Compatible:** comparten las condiciones del resto de las reglas del modelo de negocio.

11.1.2.3 Patrones de implementación

Hay diferentes alternativas de implementación de las reglas de negocio y su validación. Entre las preguntas a hacerse al momento de decidirnos por una de ellas están [4]:

- ¿Quiénes validan las reglas?
 - ¿Cuándo deben validarse las reglas?
 - ¿Qué acciones tomar cuando resultan inválidas?
 - ¿Con qué frecuencia cambian las reglas?
-
- La validación de las reglas debe ser asignada a los objetos que cuentan con la información. En general son los más concretos y pequeños. En la tabla que sigue se agrupa la estrategia de asignación para los patrones de *Streamlined Object Modeling – Patterns, rules and implementations* [2] y los diferentes

tipos de reglas. En las columnas se indican las siglas de los nombres de los patrones y cada uno de los objetos que participan de la validación.

Tabla 11.4. Estrategia de asignación de reglas de negocio a los objetos de los patrones de análisis (adaptada de Streamlined Object Modeling – Patterns, rules and implementations [2])

Regla	Patrón de colaboración											
	A-R	GL-L	I-IE	E-P	C-C	G-M	T-R	T-L	T-IE	TC-LI	LI-IE	TC-T
Tipo	R	L IE	P C	M	R	L	IE	LI		T		
Multiplícidad	R	GL, L	I, IE	E, P	C, C	G, M	T, R	T, L	T, IE	TC, LI	LI, IE	TC, T
Propiedad	R	GL, L IE	E, P	C, C	G, M	R	L	IE			T	
Estado	R	GL, L	I, IE	E, P	C, C	G, M	R	L	IE			T
Conflicto	R	L IE	P C	G, M	R	L	IE			T		

Referencias:

- A – R: actor – rol
- GL – L: gran lugar – lugar
- I – IE: ítem – ítem específico
- E – P: ensamble – parte
- C – C: contenedor – contenido
- G – M: grupo – miembro
- T – R: transacción – rol
- T – L: transacción – lugar
- T – IE: transacción – ítem específico
- TC – LI: transacción compuesta – line ítem
- LI – IE: line ítem – ítem específico
- TC – T: transacción cronológica - transacción

- Las reglas en general son validadas en las transiciones de estados del modelo y el objetivo es controlar la evolución de los estados permitiendo los válidos y prohibiendo los no válidos. A veces es necesario realizar validaciones al momento de establecer relaciones entre objetos, otras veces puede permitirse la asunción de valores no permitidos pero que serán validados al momento en que tengan impacto en el estado del modelo. En este último caso las validaciones se realizan sobre valores ya asignados o relaciones ya establecidas. Este aspecto determina la posterior implementación y condiciona las decisiones de diseño. En el primer caso bastará con interceptar la ejecución de un método, en el segundo podría utilizarse una máquina de inferencia que analice la lógica de la regla para decidir la acción a tomar.
- Las acciones a tomar cuando las reglas son inválidas van desde el disparo de una excepción y su posterior publicación hacia los usuarios hasta el cambio de rumbo en la evolución de un evento de negocio.

- 4. Dependiendo de la frecuencia de cambio de las reglas será la implementación que se decida. Esta puede ir desde estar embebida en el código hasta ser importada desde un archivo para su posterior validación, lo cual implica cambios a la regla sin afectar al código.

11.1.3 Trazabilidad de los requerimientos

En el proceso de especificación y análisis de los requerimientos del sistema, por un lado comenzamos elaborando un listado de requerimientos funcionales que expresan las operaciones que los actores podrán realizar con su uso. Por el otro relevamos cuáles son los actores externos al sistema y cuál es su interés en su uso. Así, a partir del listado y de los actores, identificamos los casos de uso. La relación entre los requerimientos y los casos de uso constituyen el camino de entrada a la implementación del sistema en desarrollo, ya que cambios futuros a los requerimientos se podrán seguir a través de estas relaciones de trazabilidad a efectos de analizar el impacto de dichos cambios. En la figura 11.6 se observa una matriz de estas relaciones entre los requerimientos funcionales y los casos de uso (construida con Enterprise Architect – Sparx Systems). En la figura 11.7 se indica en un esquema esta traza como la T1.

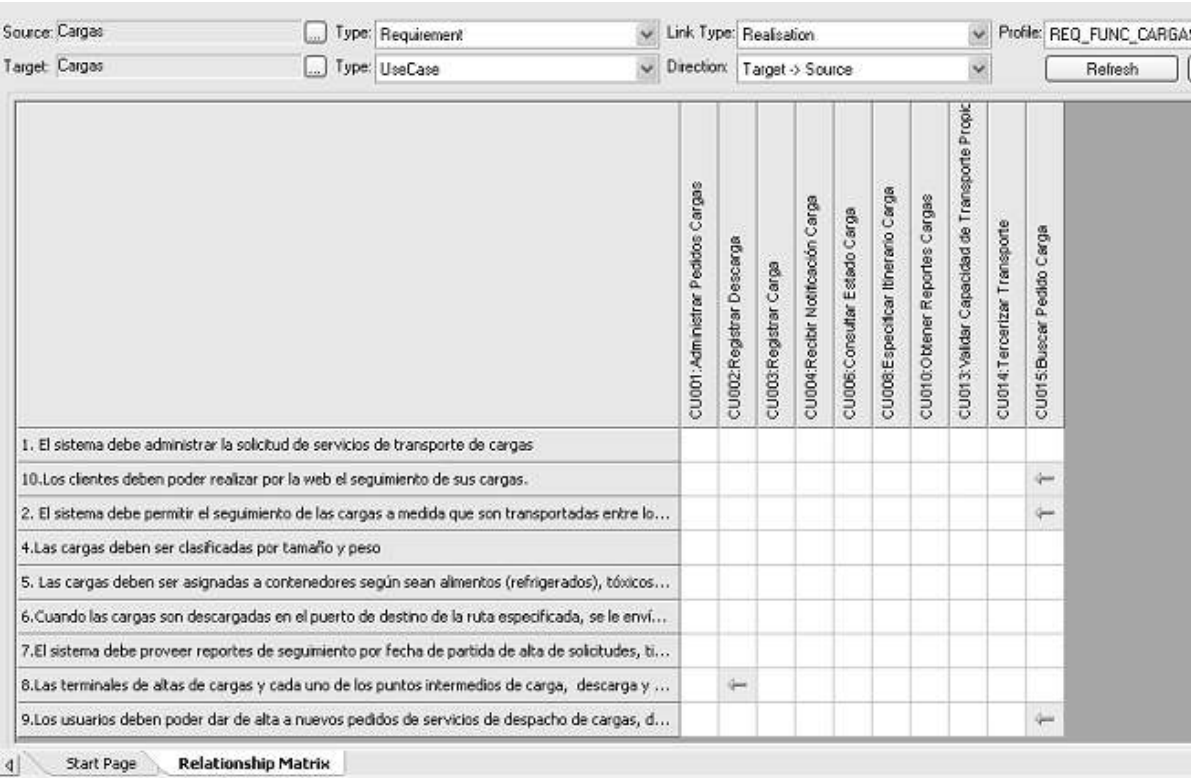


Figura 11.6. Trazas entre los requerimientos funcionales y los casos de uso del módulo de cargas

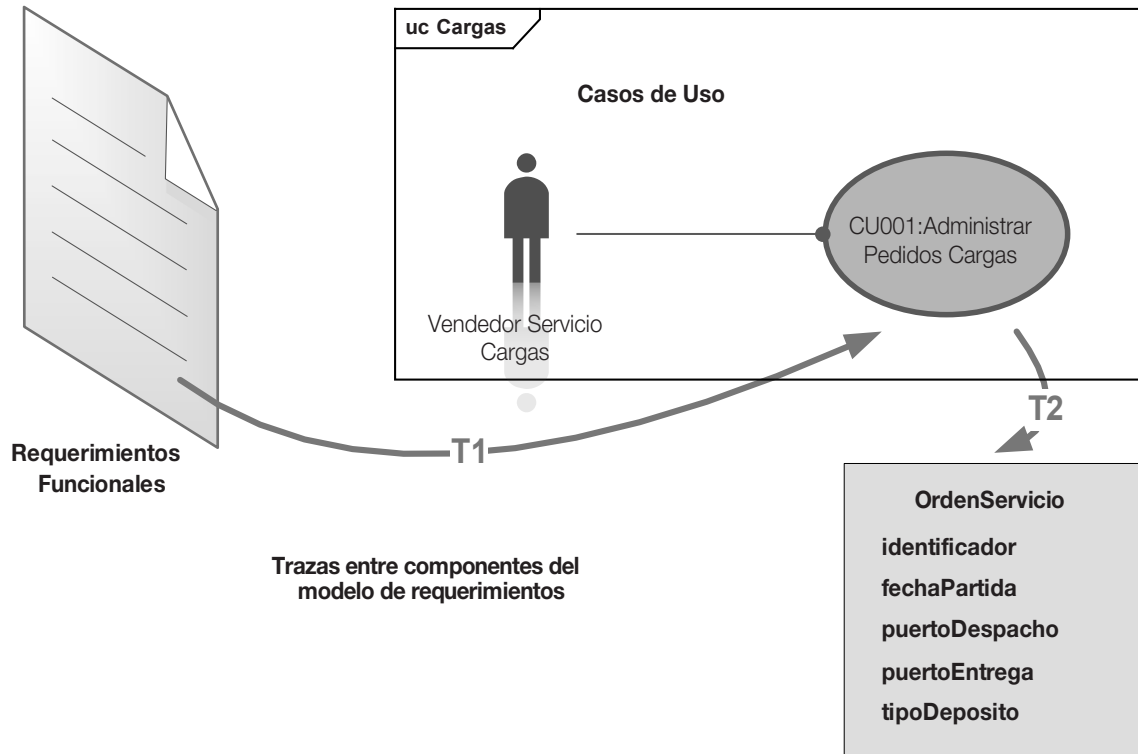


Figura 11.7. Diagrama que muestra las relaciones de trazabilidad entre el listado de requerimientos, los casos de uso y las clases del modelo de dominio

Muchos de los datos manipulados en los casos de uso constituyen atributos de las clases del modelo de dominio elaborado simultáneamente a partir de conceptos extraídos del dominio del problema, como fue descrito anteriormente en este capítulo, por lo tanto es de utilidad contar con una traza que indique estas relaciones. Esta documentación es de utilidad para facilitar la implementación a partir de un mejor entendimiento de los requerimientos. Además de brindar apoyo al momento de administrar los futuros cambios, como describimos anteriormente. Estas relaciones se indican en la figura 11.7 como T2.

11.2 Conclusión

Se presentó el análisis de los requerimientos en relación con el problema sobre el cual el sistema en desarrollo debe operar.

Se construyó un modelo de dominio que refinado con patrones de análisis es utilizado para entender el negocio a partir de la validación de las reglas asignadas a sus objetos.

Con estas construcciones se sientan las bases para el posterior diseño que implementará el sistema en desarrollo, a partir de un modelo que describe el negocio y su evolución.

En el próximo capítulo veremos cómo utilizar las construcciones de los capítulos 10 y 11 para probar (validar y verificar) los requerimientos.

11.3 Contenido de la página web de apoyo



11.3.1 Mapa conceptual

11.3.2 Autoevaluaciones

11.3.3 Presentaciones en Power Point (exclusivo para el docente)

11.4 Referencias

- [1] *Object Oriented Analysis and Design with applications*, 2da edition, Grady Booch, Cummings Press, 1994.
- [2] *Streamlined Object Modeling – Patterns, rules and implementations*, Jill Nicola et ál., PHPTR, 2002.
- [3] *Analysis Patterns*, Martin Fowler, 1997
- [4] *Business Rules and Information Systems: Aligning IT with Business Goals*, Tony Morgan, Addison-Wesley, 2002.

Pruebas a los requerimientos

Contenido

12.1 Pruebas a los requerimientos	180
12.2 Validación	180
12.3 Verificación	187
12.4 Conclusión	196
12.5 Contenido de la página web de apoyo	197
12.6 Referencias	197

Objetivos

- Validar las reglas de negocio utilizando modelos de dominio
- Probar el modelo de dominio utilizando prototipos
- Validar los requerimientos funcionales a través de prototipos
- Probar los requerimientos construyendo y ejecutando casos de prueba

Competencias

- Definir las pruebas para la validación y verificación de los requerimientos de un proyecto de desarrollo

12.1 Pruebas a los requerimientos

Hasta el momento, dentro del ciclo de vida del sistema del caso testigo, disponemos de los siguientes modelos construidos mientras concebimos el sistema a desarrollar:

- Listado de requerimientos
- Diagramas de casos de uso
- Especificación de casos de uso
- Modelo de negocio.

Es el momento de validar que este sistema es el correcto, es el que esperan y necesitan los clientes y usuarios. Por esta razón nos dispondremos a validarlo y a pensar cómo lo verificaremos, es decir cómo probaremos que cubre los requerimientos relevados y que fue construido correctamente.

12.2 Validación

A efectos de determinar si el sistema que estamos concibiendo es el correcto, el que necesitan el cliente y los usuarios realizamos su validación.

Hasta el momento hemos elaborado prototipos de interfaces que forman parte de la aplicación y un modelo de clases con reglas de negocio asignadas que constituye el modelo de negocio. Utilizaremos a ambos en el ejercicio de validación.

12.2.1 Validación de la aplicación

Un modo de validar la aplicación se da a partir de la revisión en detalle de los prototipos de interfaces que forman parte de la especificación de los casos de uso. A continuación describimos el diálogo mantenido por uno de los analistas y uno de los usuarios durante una sesión de validación del caso de uso “CU044: Buscar Orden de Carga” presentado en el capítulo anterior y cuya interfaz propuesta fue la presentada en la figura 10.10.

Analista: Aquí puedes ver cómo será la funcionalidad correspondiente para buscar una carga. ¿Qué piensas?

Usuario: Déjame ver... ¿Qué sería este campo “identificador”?

Analista: El identificador es el número por el cual cada una de las cargas se identifica unívocamente y es necesario para poder encontrar las cargas. Ustedes actualmente ya utilizan un número para identificarlas, ¿lo denominan de alguna forma en particular?

Usuario: Nosotros usamos un número que depende del puerto origen, puerto destino, tipo de carga, la fecha de origen y destino. A este valor lo llamamos número de carga. Creo que se debería poner este texto en lugar de “identificador”. Estamos más familiarizados con esta denominación. ¿Es esto posible?

Analista: Sí, por supuesto. Estoy de acuerdo. ¿Qué opinión te merecen el resto de los campos?

Usuario: Fecha de partida, puerto origen, puerto destino, tipo de carga... el valor que aparece de tipo de carga, ¿qué representa?

Analista: Neutra es el valor que toma por defecto el tipo de carga. Supusimos que la mayor cantidad de cargas eran de tipo neutras, ¿esto es así? La idea es que la mayor cantidad de veces no tengan que modificar este valor.

Usuario: Entiendo lo que dices. Sí, generalmente las cargas son del tipo neutras. Entonces me parece bien que esté esta opción por defecto.

Analista: Perfecto.

Usuario: Y el último campo sería el nombre del cliente. Bien.

Analista: Exactamente. ¿Te parece proseguir con la pantalla donde se presenta el resultado de la búsqueda?

Usuario: Sí, continuemos.

Analista: Bien. Como resultado se presenta el identificador, que previamente pactamos modificar esta leyenda a "número de carga", la fecha de partida y de arribo, el tipo de carga, el puerto origen y destino, la ruta completa que realizará la carga, el cliente y por último la descripción. ¿Qué piensas?

Usuario: Me gusta ver la ruta, lo que está entre paréntesis junto al nombre del puerto es la inicial del país del puerto, ¿verdad?

Analista: Sí, así es.

Usuario: Pero me gustaría poder ver también dónde está actualmente la carga, ¿esto es posible?

Analista: Sí, por supuesto. ¿En qué lugar piensas que es conveniente?

Usuario: Creo que luego de la ruta sería conveniente.

Analista: Perfecto. Lo incorporamos.

Usuario: Otros datos que me interesarían ver son el tipo de cliente y su mail.

Analista: Bueno, lo agregamos a continuación del nombre del cliente, ¿está bien?

Usuario: Sí, sí.

Analista: Perfecto.

Usuario: Algo que me olvidé con respecto a la ruta, ¿sería posible incluir el tipo de ruta que ha sido seleccionada? Esto es, si se ha seleccionado algún criterio de optimización o no.

Analista: Esto lo deberíamos obtener de otro subsistema. Debería confirmarlo. Déjame revisar los datos que podemos obtener de este subsistema.

Usuario: Adelante.

...

Analista: Bueno, efectivamente es posible obtener esa información. Por lo tanto la incorporamos a continuación de la ruta, ¿qué piensas?

Usuario: Sí, me parece bien.

Analista: Excelente. ¿Algo más que te gustaría cambiar o agregar?

Usuario: Creo que con los datos que agregamos estaría bien. Así tendríamos disponible la información relevante que necesitamos cuando buscamos una carga.

Analista: Perfecto. Entonces las pantallas serán así.

Usuario: Bueno, muchas gracias.

Analista: Gracias a ti por el tiempo y la colaboración. Hasta luego.

Usuario: Hasta luego.

Luego de esta charla se logró llevar a cabo la validación de la interfaz correspondiente al caso de uso “CU015: Buscar Orden de Carga”. Se han realizado algunas modificaciones solicitadas por el usuario y se concluyó a la interfaz final validada, como se muestra a continuación en la figura 12.1.

12.2.2 Validación del modelo de negocio

Una vez validada la aplicación necesitamos hacer lo mismo con el modelo de negocio. Ambos modelos, el de casos de uso y el de negocio, representan aspectos diferentes del sistema en construcción. El primero representa la aplicación, el segundo el dominio del problema. Por lo tanto nos resta validar si el modelo de negocio construido representa al dominio. Para esto construiremos test que probarán la dinámica implementada con las reglas de negocio correspondientes [1].

Hemos elegido validar las reglas RN_2 y RN_11 del ejemplo, las cuales se muestran en la figura 12.2. Validaremos que cuando se intente sobrecargar el contenedor, el sistema detectará esta situación y no lo permitirá; también que una carga que contiene alimentos no pueda ser depositada en un contenedor estándar ya que el mismo requiere uno refrigerado.

En la figura 12.3 se ha refinado el diagrama de clases de la figura 12.2 y se muestra la implementación de las reglas de negocio con sus métodos asociados.

class Prototipo de Pantalla - Buscar Pedido de Carga

Buscar Carga

Número de Carga :

Fecha de Partida :

Puerto Origen :

Puerto Destino :

Tipo de Carga :

Neutra

Nombre del Cliente :

Buscar

Cancelar

↓

Carga Buscada

Número de Carga:

2309898

Fecha de Partida:

20/07/2010

Fecha de Arribo:

10/08/2010

Tipo de Carga:

Neutra

Puerto Origen:

Buenos Aires, Argentina

Puerto Destino:

Vila do Conde, Brasil

Tipo de Ruta:

Sin criterio

Ruta:

Buenos Aires (A), Rio Grande (B), Santos (B), Rio de Janeiro (B), Salvador (B), Areia Branca (B), Vila do Conde (B)

Posición Actual:

Puerto Rio Grande, Brasil

Cliente:

Transportes PERIC S.A

Tipo de Cliente:

Baja Carga

Mail del Cliente:

cargasPeric@peric.com

Descripción:

Indumentaria deportiva femenina y masculina

Aceptar

Figura 12.1. Prototipo del caso de uso “Buscar Orden de Carga” validado con el usuario

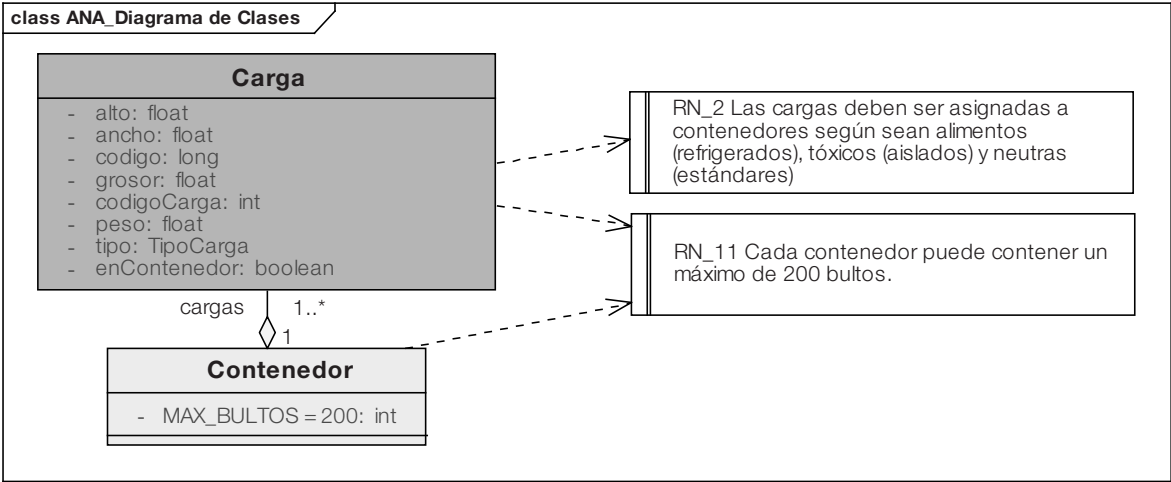


Figura 12.2. Clases y sus reglas de negocio asociadas

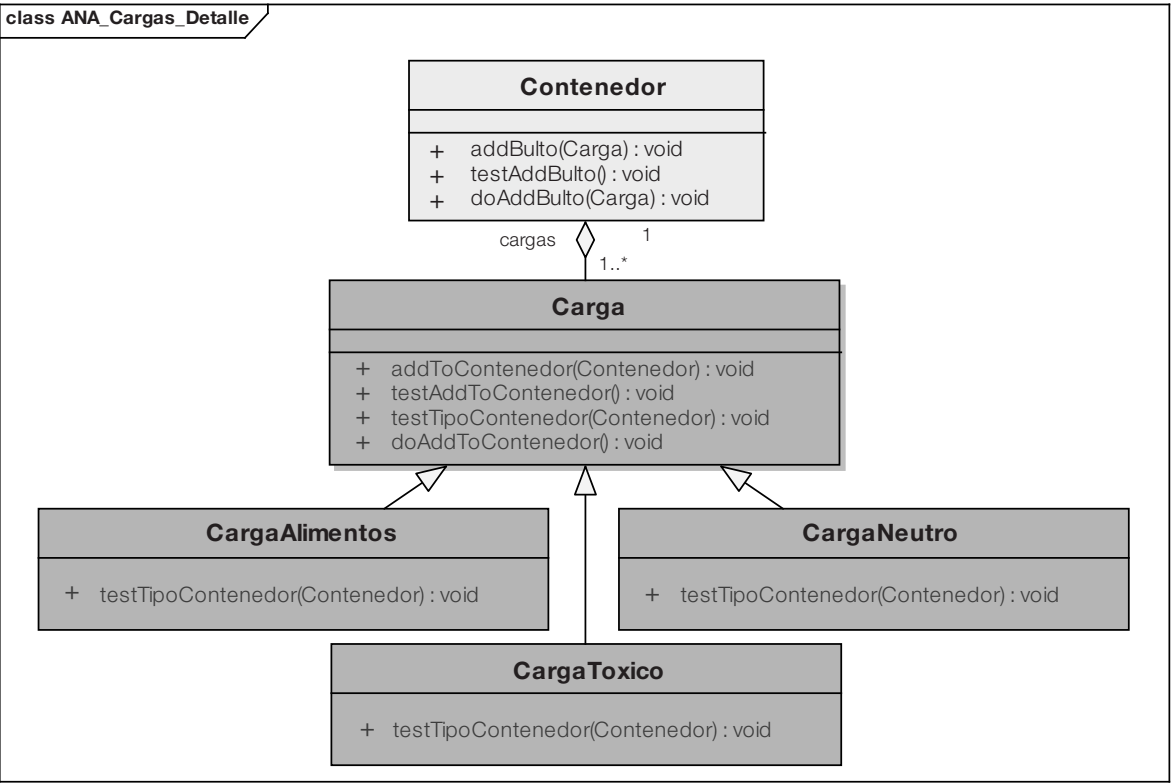


Figura 12.3. Clases de la figura anterior y sus reglas de negocio implementadas

A continuación presentamos el código Java de las validaciones correspondientes. Este código fue escrito a efectos de validar el modelo, sin optimizarlo según criterios de diseño. Este será optimizado en una fase posterior, cuando nos focalicemos en el diseño y nuestra preocupación sea otra. Por ejemplo, que el sistema sea escalable, extensible, fácil de mantener, etc. Seguramente este código será tomado como base de este diseño.

```
class Contenedor{

    private final int MAX_BULTOS = 20;
    private List bultos = new ArrayList();

    public void addBulto(Bulto bulto)throws Exception {
        if(bulto == null){
            throw new Exception("Bulto nulo en contenedor");
        }
        bulto.addToContenedor(this);
    }

    public void testAddBulto()throws Exception {
        if(bultos.size() >= MAX_BULTOS){
            throw new Exception("Contenedor lleno");
        }
    }

    public void doAddBulto(Bulto bulto){
        bultos.add(bulto);
    }
}

class Bulto{

    private boolean enContenedor = false;

    public void addToContenedor(Contenedor contenedor)throws Exception {

        if(contenedor == null){
            throw new Exception("Contenedor nulo en Bulto");
        }
        contenedor.testAddBulto();
        testAddToContenedor();
        doAddToContenedor();
        contenedor.doAddBulto(this);
    }

    public void testAddToContenedor()throws Exception {
        if(enContenedor == true){
            throw new Exception("Bulto ya depositado en
                                contenedor");
        }
    }
}
```

```

    }

    abstract public void testTipoContenedor(Contenedor contenedor)
        throws Exception;

    public doAddToContenedor(){
        enContenedor = true;
    }
}

class BultoAlimento extends Bulto{
    public void testTipoContenedor(Contenedor contenedor)
        throws Exception {
        if(tipoBulto == ALIMENTOS && !contenedor.refrigerado()){
            throw new Exception("Tipo de contenedor no válido para
bulto");
        }
    }
}

```

Los tests para probar estas validaciones, utilizando un *framework* de pruebas del estilo JUnit [2], se listan a continuación:

```

class TestContenedorBultos extends testCase {

    private static Contenedor contenedor =
        new Contenedor(TipoContenedor.ESTANDARD);

    private static float alto  = 1.00;
    private static float alto  = 1.00;
    private static float ancho = 1.00;
    private static float grosor = 1.00;
    private static float peso  = 215.00;
    private static long codigo = 123000;
    private static int codigoCarga = 999;

    public void setup(){
        int cantCarga = 200;
        while(cantCarga >= 0){
            contenedor.addBulto(new Bulto(alto, ancho, grosor,
            peso, codigo, codigoCarga, TipoCarga.NEUTRO));
            cantCarga--;
        }
    }

    public void tearDown(){}
}

```

```

public void testCapacidadContenedor(){

    try{
        contenedor.addToContenedor(new Bulto(alto, ancho, grosor,
                                                peso, codigo, codigoCarga,
                                                TipoCarga.ALIMENTO));

        fail();
    }
    catch(Exception exc){

        assertEquals("Falla contenedor.testAddToContenedor",
                    "Contenedor lleno", exc.getMensaje());
    }
}

public void testTipoContenedor(){

    try{
        contenedor.addToContenedor(new Bulto(alto, ancho, grosor,
                                                peso, codigo, codigoCarga,
                                                TipoCarga.ALIMENTO));

        fail();
    }
    catch(Exception exc){

        assertEquals("Falla bulto.testTipoContenedor",
                    "Tipo de contenedor no válido para bulto",
                    exc.getMensaje());
    }
}
}

```

El modelo de negocio será sometido así a todos los test necesarios a efectos de validar su comportamiento y asegurarnos que representa la dinámica del negocio que los clientes y usuarios nos informaron.

De esta manera nos hemos asegurado que el sistema (aplicación + negocio) en construcción es el correcto.

12.3 Verificación

12.3.1 Casos de pruebas

En este punto continuaremos con la verificación de lo que hemos hecho hasta ahora. Por un lado ya validamos los prototipos de pantallas, los modelos de dominio y negocio construidos, pero aún queda pendiente verificar

los casos de uso que nacieron a partir de los requerimientos iniciales. Esta verificación consiste en constatar que la funcionalidad correspondiente al caso de uso en cuestión cumple con el comportamiento esperado.

Para llevar a cabo esta tarea generaremos casos de prueba a partir de los casos de uso. Pero ¿cómo construiremos los casos de prueba?

Para contestar esta pregunta describiremos los pasos que ejecutaremos para lograr un caso de prueba [4], [5]:

1. Generar todos los escenarios del caso de uso. Para esto hay que tener en cuenta el flujo básico, los flujos alternativos y de validación.
2. Para cada escenario, identificar al menos un caso de prueba y las condiciones que deben cumplirse para que este sea ejecutado.
3. Identificar los datos a utilizar para ejecutar cada una de las pruebas.

Ahora bien, hemos mencionado en reiteradas oportunidades el término “escenario”, pero ¿qué es un escenario exactamente?

Un escenario es un conjunto de condiciones que se consideran el entorno de un flujo del caso de uso. Lo que buscamos aquí es definir un conjunto de condiciones para cada uno de los flujos existentes en el caso de uso.

Inicialmente comencemos construyendo los distintos escenarios de los casos de uso. Estos escenarios corresponden a los diferentes flujos que el usuario del sistema irá recorriendo al utilizar la aplicación. Entonces retomemos el caso de uso construido en el capítulo 10: “Buscar Orden de Carga” y dediquémonos a la construcción de los escenarios recordando que para este caso de uso existen dos flujos alternativos (“A: No existen pedidos de carga que satisfagan la búsqueda” y “B: El usuario cancela la búsqueda”) y un flujo de validación (“A: Los datos ingresados no son correctos”) que deben ser considerados además del flujo básico.

Tabla 12.1. Escenarios correspondientes al caso de uso “Buscar Pedido de Carga”			
Escenarios	Flujos incluidos		
Escenario 1: Búsqueda de pedido de carga exitosa.	Flujo Básico		
Escenario 2: Carga no existente.	Flujo Básico	Flujo Alternativo A	
Escenario 3: Carga no existente y usuario cancela la búsqueda.	Flujo Básico	Flujo Alternativo A	Flujo Alternativo B
Escenario 4: Usuario cancela la búsqueda.	Flujo Básico	Flujo Alternativo B	
Escenario 5: Datos escasos o incorrectos.	Flujo Básico	Flujo de Validación A	
Escenario 6: Datos escasos o incorrectos y el usuario cancela la búsqueda.	Flujo Básico	Flujo de Validación A	Flujo Alternativo B

Debemos recalcar que los escenarios planteados aquí son representativos del caso de uso. Es decir que se consideran los flujos más importantes y críticos de este. Tenemos que tener en cuenta que no es factible el testeo exhaustivo.

El siguiente paso es identificar los casos de prueba, para lo que analizaremos los escenarios definidos en el paso anterior y repasaremos la especificación del caso de uso del capítulo 10. Como mínimo deberíamos generar un caso de prueba por cada escenario, aunque generalmente hay más de uno. Para el planteo de los casos de prueba nos valdremos de una tabla en la que identificaremos cada caso de uso, colocaremos una descripción breve de este, los datos que son necesarios para su ejecución y el resultado esperado. Para confeccionar este cuadro volvamos nuevamente a la especificación del caso de uso para detectar que los campos que se utilizan como entrada del caso de prueba son los mismos que se habían detallado en la parte de definición de datos previamente. Para cada uno de estos campos se ha colocado una V, I o N/A para indicar Válido, Inválido y No Aplica (no es necesario proporcionar el dato). También, notemos que sólo el camino exitoso contiene todas V en sus filas; de esta manera, nos aseguramos de que siempre estemos probando un caso distinto de prueba.

A continuación presentamos el cuadro correspondiente para el caso de prueba que estamos siguiendo.

Tabla 12.2. Cuadro con los estados de las variables del caso de prueba									
Id	Escenario	Id. Carga	Fecha partida	Puerto origen	Puerto destino	Tipo carga	Cliente	Carga registrada	Resultado esperado
1	Escenario 1: Búsqueda de pedido de carga exitosa	V	V	V	V	V	V	V	Visualización del detalle de la carga
2	Escenario 2: Carga no existente	V	V	V	V	V	V	F	Mensaje "Carga no existente"
3	Escenario 3: Carga no existente y usuario cancela la búsqueda	V	V	V	V	V	V	F	Visualización de la pantalla principal de la aplicación
4	Escenario 4: Usuario cancela la búsqueda.	N/A	N/A	N/A	N/A	N/A	N/A	N/A	Visualización de la pantalla principal de la aplicación
5	Escenario 5: Datos escasos o incorrectos	I	I	I	I	I	I	N/A	Mensaje "Datos escasos o incorrectos"
6	Escenario 6: Datos escasos o incorrectos y el usuario cancela la búsqueda	V	N/A	N/A	N/A	N/A	N/A	N/A	Visualización de la pantalla principal de la aplicación

Por último le asignaremos valores a cada uno de los datos que usemos para ejecutar las pruebas. Para hacer esto tenemos que pensar en aquellos valores que produzcan lo que queremos lograr. Es decir que si, por ejemplo, tenemos que pensar en un valor válido para la fecha de partida podríamos utilizar 12/03/2010 mientras que si el dato tendría que ser inválido podríamos usar 35/03/2010.

Veamos cómo queda la tabla para el caso de prueba que estamos utilizando como guía.

Tabla 12.3. Caso de prueba correspondiente al caso de uso “Buscar Orden de Carga”									
Id	Escenario	Id. Carga	Fecha partida	Puerto origen	Puerto destino	Tipo carga	Cliente	Carga registrada	Resultado esperado
1	Escenario 1: Búsqueda de pedido de carga exitosa	10000	12/03/2010	Bs. As.	Río de Janeiro	Neutra	Carlos Pérez	V	Visualización del detalle de la carga
2	Escenario 2: Carga no existente	10000	12/03/2010	Bs. As.	Río de Janeiro	Neutra	Carlos López	F	Mensaje “Carga no existente”
3	Escenario 3: Carga no existente y usuario cancela la búsqueda	10000	12/03/2010	Bs. As.	Río de Janeiro	Neutra	Carlos López	F	Visualización de la pantalla principal de la aplicación
4	Escenario 4: Usuario cancela la búsqueda	N/A	N/A	N/A	N/A	N/A	N/A	N/A	Visualización de la pantalla principal de la aplicación
5	Escenario 5: Datos escasos o incorrectos	10000	35/04/2010	Bs. As.	Bs. As.	Neutra	-	N/A	Mensaje “Datos escasos o incorrectos”
6	Escenario 6: Datos escasos o incorrectos y el usuario cancela la búsqueda	10000	N/A	N/A	N/A	N/A	N/A	N/A	Visualización de la pantalla principal de la aplicación

Ahora continuaremos con un caso de uso más complejo. Repasemos la especificación del caso de uso “Registrar Carga” presente también en el capítulo 10 donde además del flujo básico habíamos presentado dos flujos alternativos (“A: Ocurre un error durante el proceso de almacenamiento de los datos” y “B: El operador

cancela la registraci3n”) y dos flujos de validaci3n (“A: Los datos ingresados son inválidos” y “B: Los datos ingresados ya se encuentran registrados en la aplicaci3n”). Veamos en la siguiente tabla los distintos escenarios posibles.

Tabla 12.4 . Escenarios correspondientes al caso de uso “Registrar Carga”			
Escenarios		Flujos incluidos	
Escenario 1: Registraci3n exitosa.		Flujo Bási­co	
Escenario 2: Error durante la registraci3n.		Flujo Bási­co	Flujo Alternativo A
Escenario 3: Error durante la registraci3n y el usuario cancela la operaci3n.		Flujo Bási­co	Flujo Alternativo A Flujo Alternativo B
Escenario 4: El usuario cancela la operaci3n.		Flujo Bási­co	Flujo Alternativo B
Escenario 5: Los datos son escasos o inválidos.		Flujo Bási­co	Flujo de Validaci3n A
Escenario 6: Datos escasos o incorrectos y el usuario cancela la operaci3n.		Flujo Bási­co	Flujo de Validaci3n A Flujo Alternativo B
Escenario 7: La carga ya se encuentra registrada.		Flujo Bási­co	Flujo de Validaci3n B
Escenario 8: La carga ya se encuentra registrada y el usuario cancela la operaci3n.		Flujo Bási­co	Flujo de Validaci3n B Flujo Alternativo B

Hemos definido los escenarios principales para el caso de uso “Registrar Carga”. Ahora continuaremos estableciendo las condiciones y las salidas esperadas para cada uno de los escenarios.

Tabla 12.5. Cuadro con los estados de las variables del caso de uso “Registrar Carga”										
Id	Escenario	Fecha partida	Puerto origen	Puerto destino	Tipo carga	Cliente	Descripci3n	Carga no registrada	Conexi3n a la base de datos: OK	Resultado esperado
1	Escenario 1: Registraci3n exitosa	V	V	V	V	V	V	V	V	Mensaje “Los datos se han cargado satisfactoria-mente con el nÚmero 100000”

Tabla 12.5. Cuadro con los estados de las variables del caso de uso “Registrar Carga”										
Id	Escenario	Fecha partida	Puerto origen	Puerto destino	Tipo carga	Cliente	Descripción	Carga no registrada	Conexión a la base de datos: OK	Resultado esperado
2	Escenario 2: Error durante la registración	V	V	V	V	V	V	V	F	Mensaje “Ha ocurrido un error durante la registración”
3	Escenario 3: Error durante la registración y el usuario cancela la operación	V	V	V	V	V	V	F	F	Visualización de la pantalla principal de la aplicación
4	Escenario 4: Usuario cancela la operación	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	Visualización de la pantalla principal de la aplicación
5	Escenario 5: Datos escasos o inválidos	V	I	I	V	V	-	N/A	V	Mensaje “Datos escasos o incorrectos”
6	Escenario 6: Datos escasos o incorrectos y el usuario cancela la operación	V	I	I	V	V	-	N/A	V	Visualización de la pantalla principal de la aplicación
7	Escenario 8: La carga ya se encuentra registrada	V	V	V	V	V	V	F	V	Mensaje “La carga ya se encuentra registrada”
8	Escenario 8: La carga ya se encuentra registrada y el usuario cancela la operación	V	V	V	V	V	V	F	V	Visualización de la pantalla principal de la aplicación

Por último incorporamos los valores reales que utilizaremos para ejecutar cada una de las pruebas planteadas en la tabla anterior.

Tabla 12.6. Casos de prueba para el caso de uso “Registrar Carga”

Id	Escenario	Fecha partida	Puerto origen	Puerto destino	Tipo carga	Cliente	Descripción	Carga no registrada	Conexión a la base de datos: OK	Resultado esperado
1	Escenario 1: Registración exitosa	2/2/2010	Río de Janeiro	Bs. As.	Neutra	Cutu S.A.	Indumentaria deportiva	V	V	Mensaje “Los datos se han cargado satisfactoriamente con el número 100000”
2	Escenario 2: Error durante la registración	2/2/2010	Bs. As.	Río de Janeiro	Neutra	Cutu S.A.	Indumentaria deportiva	V	F	Mensaje “Ha ocurrido un error durante la registración”
3	Escenario 3: Error durante la registración y el usuario cancela la operación	12/2/2010	Bs. As.	Río de Janeiro	Neutra	Cutu S.A.	Indumentaria deportiva	V	F	Visualización de la pantalla principal de la aplicación
4	Escenario 4: Usuario cancela la operación	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	Visualización de la pantalla principal de la aplicación
5	Escenario 5: Datos escasos o inválidos	2/2/2010	Aaaaaa (1)	Aaaaaa (1)	Neutra	Galo S.A.	-	N/A	V	Mensaje “Datos escasos o incorrectos”
6	Escenario 6: Datos escasos o incorrectos y el usuario cancela la operación	2/2/2010	Aaaaaa (1)	Aaaaaa (1)	Neutra	Galo S.A.	-	N/A	V	Visualización de la pantalla principal de la aplicación
7	Escenario 8: La carga ya se encuentra registrada	2/2/2010	Río de Janeiro	Bs. As.	Neutra	Cutu S.A.	Indumentaria deportiva	F	V	Mensaje “La carga ya se encuentra registrada”

Tabla 12.6. Casos de prueba para el caso de uso “Registrar Carga”

Id	Escenario	Fecha partida	Puerto origen	Puerto destino	Tipo carga	Cliente	Descripción	Carga no registrada	Conexión a la base de datos: OK	Resultado esperado
8	Escenario 8: La carga ya se encuentra registrada y el usuario cancela la operación	2/2/2010	Río de Janeiro	Bs. As.	Neutra	Cutu S.A.	Indumentaria deportiva	F	V	Visualización de la pantalla principal de la aplicación

(1) Se consideran puertos inválidos

Así obtenemos los casos de prueba para verificar el caso de uso “Registrar Carga”.

12.3.2 Ciclo de vida y herramientas

Las pruebas de funcionalidad generadas a partir de los casos de uso, como acabamos de explicar, se ejecutarán. Luego se recolectarán los resultados y se devolverá a los programadores para que sean corregidos los errores encontrados. Este ciclo es realizado tradicionalmente en forma manual, con la ayuda de alguna planilla de cálculo o herramienta que permite registrar el desarrollo de estas tareas y sus resultados. En la figura 12.4 se muestra el ciclo de vida de las pruebas funcionales. Este ciclo es lento y costoso, propenso a introducir errores secundarios y además con frecuencia irrepetibles.

Como veremos en un próximo capítulo, la dinámica de las pruebas del software ha cambiado. Se han introducido dos conceptos fundamentales que han alterado por completo el ciclo de vida de los proyectos de desarrollo de software. Estos conceptos están basados en la idea de que realizar pruebas cuando el código ya fue escrito no contribuye en nada a la mejora de la calidad de los productos. Ya es tarde, solo aporta alguna métrica acerca de estadísticas de nuestros errores. El objetivo es no cometerlos. Es importante entonces adelantar las tareas vinculadas a las pruebas que permitan generar un criterio de diseño basado en las pruebas y además automatizarlas [6]. Estos temas serán tratados en detalle en un próximo capítulo.

Las herramientas orientadas a realizar pruebas unitarias de tipo xUnit [2] son componentes importantes que permiten automatizar las pruebas. Siguiendo esta línea existe una herramienta conocida como Fitnesse Acceptance Testing Framework [3] que permite implementar pruebas funcionales repetitivas, no manuales, e integrarlas con el resto en servidores de Integración Continua [6]. Esta herramienta permite cargar los casos de prueba elaborados en una planilla de cálculo y dejarlas integradas al resto de las pruebas que se ejecutarán automáticamente. Permite probar cálculos, operaciones con colecciones y flujos de eventos.

En la figura 12.5 se muestra la interfaz de esta herramienta implementada como una Wiki.

Trabajo con tests manuales

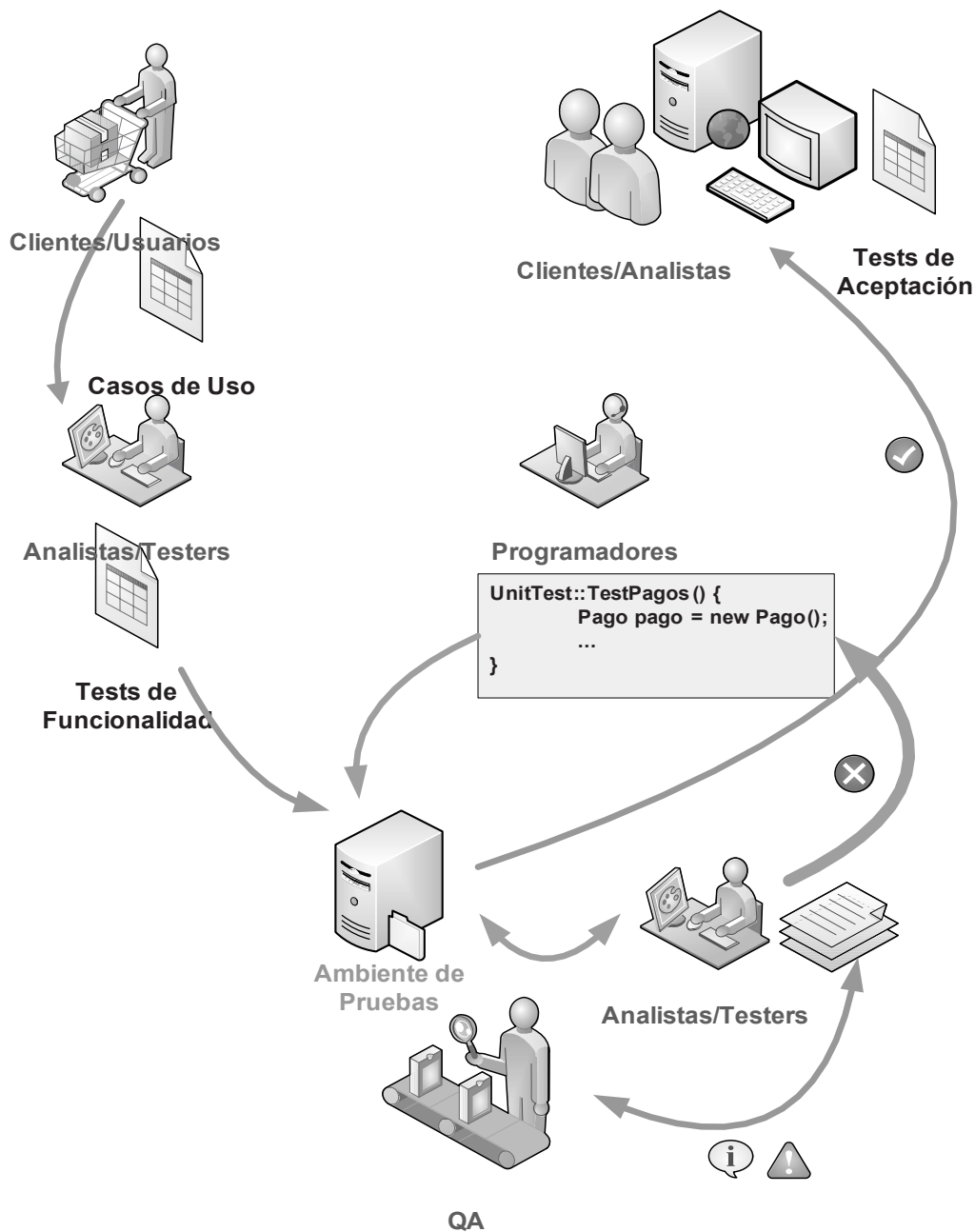


Figura 12.4. Ciclo de vida del proceso manual de pruebas funcionales

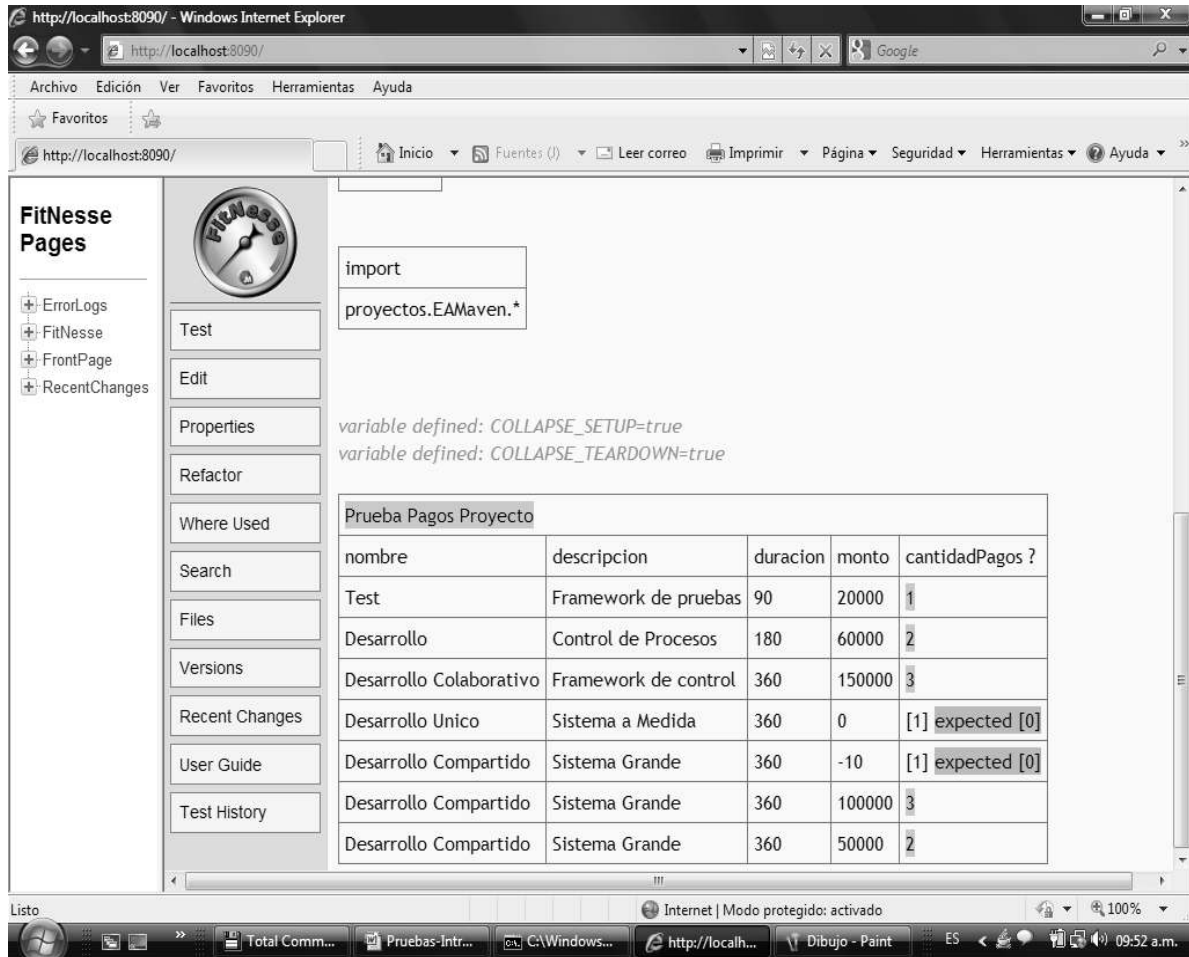


Figura 12.5. Interfaz de usuario del Framework Fitnesse

12.4 Conclusión

Hemos presentado dos actividades centrales en el ciclo de vida del desarrollo de software. La validación del sistema en desarrollo que implica dos tareas separadas. Por un lado la validación de la aplicación y por el otro la del modelo de negocio. Con esto estaremos estableciendo certeza acerca de si estamos construyendo el sistema correcto, el que necesitan y pidieron los usuarios y clientes. Además hemos probado la funcionalidad para establecer si el sistema está bien construido en términos de los requerimientos relevados y acordados. Para ello construimos casos de pruebas derivados de los casos de uso especificados.

12.5 Contenido de la página web de apoyo



12.5.1 Mapa conceptual

12.5.2 Autoevaluaciones

12.5.3 Presentaciones en Power Point (exclusivo para el docente)

12.6 Referencias

- [1] *Streamlined Object Modeling – Patterns, rules and implementations*, Jill Nicola et ál., PHPTR, 2002.
- [2] *xUnit Test Patterns*, Gerard Meszaros, Addison-Wesley, 2007.
- [3] *Fit for Developing Software: Framework for Integrated Tests*, Rick Mugridge, Ward Cunningham, Prentice Hall PTR, 2005.
- [4] *Generating Test Cases From Use Cases*, Jim Heumann, Rational Software, 2001.
- [5] *Tests and Requirements, Requirements and Tests: A Möbius Strip*, Robert C. Martin, Grigori Melnik, IEEE Software, 2008.
- [6] *Calidad en el Desarrollo de Software*, Guillermo Pantaleo, Alfaomega Grupo Editor, 2011.

Arquitectura de software

Contenido

13.1 Definiciones	200
13.2 Selección de componentes	211
13.3 Conclusión	219
13.4 Contenido de la página web de apoyo	219
13.5 Referencias	219

Objetivos

- Aprender el concepto de arquitectura
- Separar la arquitectura de negocio de la técnica
- Aprender el proceso de definición de la arquitectura de negocio a partir de los requerimientos
- Aprender el proceso de definición de la arquitectura técnica a partir de la arquitectura de negocio
- Definir la arquitectura cumpliendo con los atributos de calidad

Competencias

- Definir la arquitectura del software que soportará y servirá de plataforma para la implementación del sistema en desarrollo de un proyecto de software

13.1 Definiciones

13.1.1 El concepto de arquitectura

Se entiende como *arquitectura de un sistema* a las partes que lo componen y a la forma en que estas se relacionan.

Estas partes tienen diferente representación en distintos momentos del proyecto y para los diferentes roles involucrados. Una representación de la arquitectura de una organización fue desarrollada por Zachman [1]. Inspirados en esta última visión presentamos en la tabla 13.1 una matriz en la que se pueden observar las múltiples dimensiones del concepto de arquitectura. El concepto de arquitectura focalizada en el software de los sistemas, con una múltiple representación, se propuso en *The 4+1 View Model of Architecture* [2], donde se componen las siguientes cuatro vistas: lógica, desarrollo, procesos y nodos. En la figura 13.1 se muestra un esquema de esta representación.

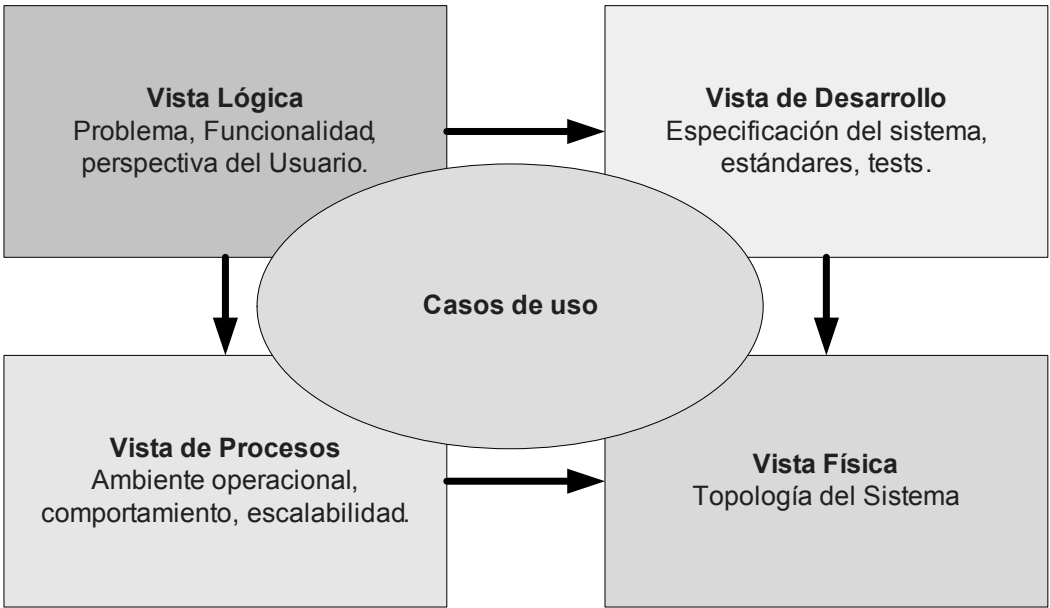


Figura 13.1. Modelo 4 + 1 de la representación de la arquitectura de software

Arquitectura de Sistemas							
Nivel		Qué	Cómo	Dónde	Quién	Cuándo	Por qué
	Modelo Conceptual	Conceptos de negocio	Procesos de negocio	Localizaciones	Roles	Eventos de negocio	Visión, objetivos
	Modelo Organizacional	Entidades conceptuales de negocio	Modelo de negocio	Sistemas de negocio	Flujo de trabajo	Cronograma de negocio	Plan de negocio
							Modelo Organizacional

Tabla 13.1. Matriz de Zachman que expresa la arquitectura de sistemas de una organización

Arquitectura de Sistemas							
	Qué	Cómo	Dónde	Quién	Cuándo	Por qué	
Nivel	Modelo Lógico de Sistema	Modelo lógico	Estilo de arquitectura	Distribución de partes	Interfaces conceptuales de usuarios	Tipo de Procesamiento	Reglas de negocio
	Modelo Técnico de Sistema	Modelo de clases	Modelo de clases con tecnología	Arquitectura de nodos	Look and feel	Estructura de control	Diseño de reglas
	Modelo Técnico de Desarrollo	Representación de datos	Programas	Arquitectura de paquetes	Permisos y actividades	Definición de tiempos	Especificación de reglas
	Modelo de Usuario	Datos manipulados	Funciones de uso	Funcionalidades localizadas	Organización de funcionalidades	Cronograma implementado	Estrategia de trabajo

13.1.1.1 Arquitectura de negocio

Como primera aproximación a la definición de la arquitectura del sistema en desarrollo del caso testigo, comenzamos analizando la estructura de la organización. En la figura 13.2 se muestran las áreas de negocio que la componen. Puede observarse en ella tres áreas de cara al público que se ocupan de la administración de las relaciones con los clientes, la venta de los servicios de transporte de carga y la prestación de servicios posventa a los clientes, como por ejemplo el seguimiento de las cargas correspondientes a las órdenes de cargas contratadas.

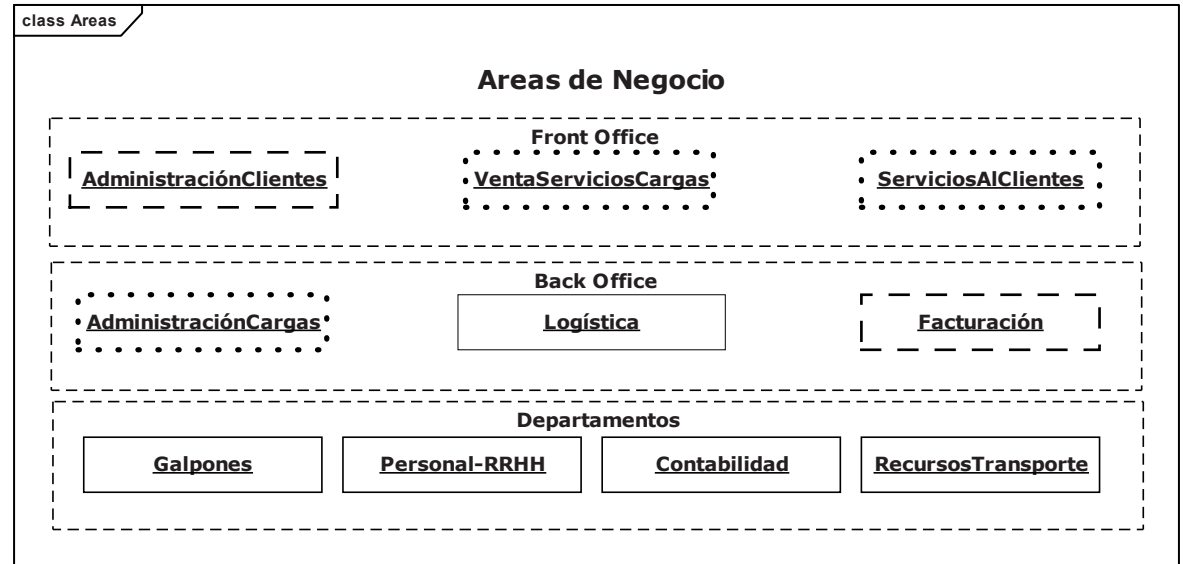


Figura 13.2. Áreas de negocio de la organización cliente del proyecto del caso testigo

Alineados con la definición de arquitectura que establece que esta se da por las partes que la componen y sus relaciones, la estructura y dinámica del negocio aportará una primera aproximación.

También puede verse en la figura 13.2 que existen otras tres áreas de negocio cuyo fin es dar soporte a las anteriores. Estas últimas funcionan puertas adentro y son las que se ocupan de la administración de las cargas asociadas con las órdenes de los servicios vendidos, de la organización y coordinación del transporte de estas cargas y de la facturación de los servicios correspondientes. Por último existen otras tres áreas de servicios generales que soportan el desarrollo del negocio y son la administración de los inmuebles (galpones) donde las cargas se almacenan, los recursos materiales de transporte como camiones y barcos propios, el personal que opera las diferentes etapas del negocio y la contabilidad. Según fue especificado el sistema en desarrollo cubre sólo las áreas que se muestran con tramas punteadas en la figura. Las áreas con tramados de verde en la figura anterior están asociadas al concepto de carga en sus diferentes fases.

Podríamos preguntarnos en este momento si sería factible plantear como alternativa el desarrollo de sistemas separados para cada área o un único sistema. También para el caso de la alternativa de sistemas separados, cómo intercambiarían información estos sistemas, es decir cuál sería su forma de comunicarse.

La respuesta a la primera pregunta debe responderse a partir del modo en que el negocio de cada área cambia. Si bien las áreas componen este negocio, seguramente evolucionan en distintas direcciones. Por lo cual si se optara por un único sistema, este debería realizar cambios para cada uno de los aspectos relacionados a cada área. También seguramente cada área evolucionaría y cambiaría con diferente frecuencia, por lo cual sería un buen criterio seleccionar un sistema para cada aspecto con evoluciones distintas.

A continuación seguiremos analizando el negocio de la organización para volver a esta discusión con mayor información.

En la figura 13.3 hemos representado los dos tipos de servicios que se prestan en esta empresa. Para el cliente que contrata el servicio de cargas solo existe uno. Sin embargo, cuando la empresa ve saturada su capacidad, subcontrata el servicio de transporte de cargas. Por lo tanto para cada tipo de servicio se llevan adelante diferentes tareas, como se muestra en la figura 13.3. Debajo de cada servicio se presentan las funciones o actividades de negocio desarrolladas. A partir de recorrer verticalmente la figura para cada servicio se hallan las funciones o actividades relacionadas. Puede verse que para el caso de la tercerización del transporte no se realiza el ruteo, la administración del transporte, de los galpones, ni de otros recursos. Sí se administra al cliente, se vende el servicio, se factura y se contabiliza.

Seguramente la empresa cuenta con recursos para llevar adelante estas funciones. Entre otras, estos recursos están compuestos por aplicaciones (programas de computadoras). Algunos de estos ya existen y otros serán los que se desarrollarán como parte de este proyecto.

A continuación identificamos los procesos de negocio compuestos por tareas derivadas de las funciones de negocio vistas. En la figura 13.4 se presentan los procesos que componen el negocio dentro del alcance del proyecto de desarrollo.

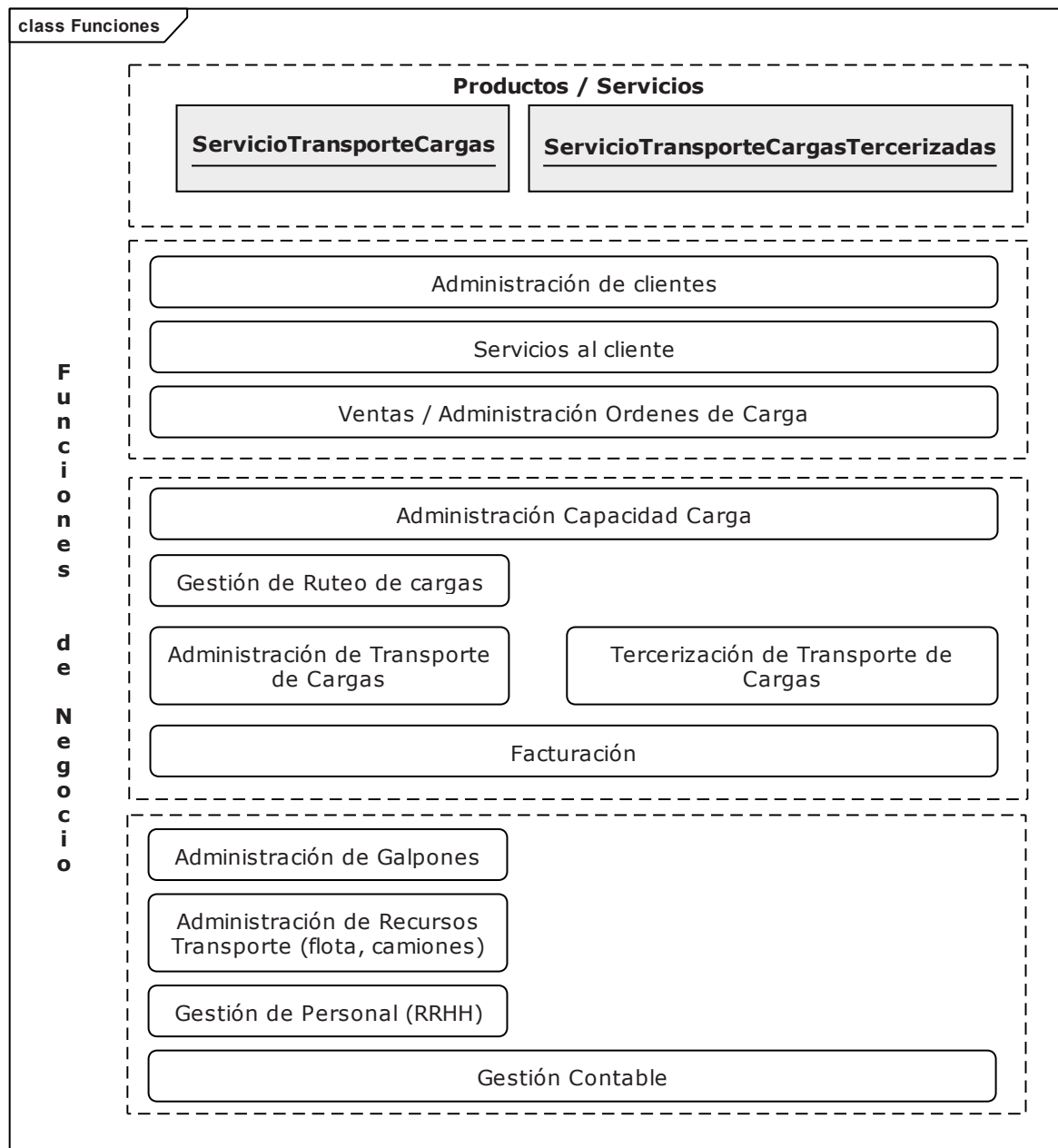


Figura 13.3. Mapeo de servicios prestados y funciones de negocio

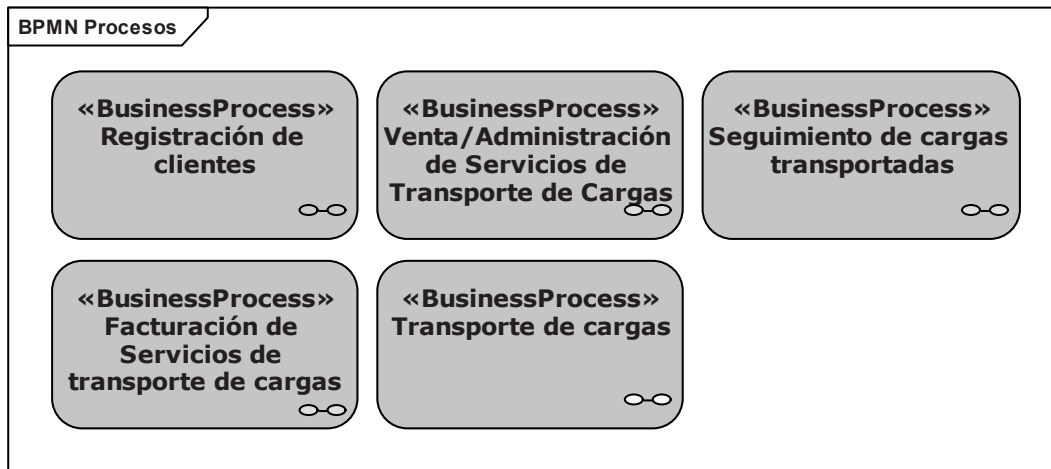


Figura 13.4. Procesos de negocio del sistema en desarrollo

Se observan cinco procesos que forman parte del negocio. Estos son la Registración de los clientes, el cual comienza cuando el cliente ingresa sus datos, continúa cuando presenta documentación en papel, sigue con la validación de esta información y termina cuando finalmente es aceptado como cliente.

El proceso de Venta de Servicios de transporte de cargas comienza cuando un cliente solicita el servicio, se desarrolla con el ingreso de la información del tipo de carga (alimentos, tóxicos, estándar), las dimensiones de los bultos que componen la carga, el puerto de destino y el criterio de selección de ruta. Sigue con la validación de la capacidad de carga de la empresa para decidir si se terceriza o se satisface con transporte propio y finalmente cuando se deja pendiente la orden para que se le asigne un transporte.

El proceso de **Transporte de cargas** comienza con una orden emitida pendiente y se desarrolla a partir de la asignación de la carga correspondiente a un transporte en cada etapa del viaje que la llevará desde el origen al puerto de destino. Finaliza cuando se cubre toda la ruta o itinerario.

El proceso de **Facturación de servicios de transporte de cargas** comienza con la orden de servicio armada, los transportes asignados para la ruta, se desarrolla con el cálculo de cada ítem e impuesto a facturar y finaliza con la emisión de la factura para que el cliente realice el pago correspondiente.

Por último, el proceso de **Seguimiento de cargas** transportadas comienza cuando la carga correspondiente a una orden es despachada, se desarrolla cuando el cliente consulta a lo largo del tiempo la ubicación de dicha carga (en viaje en la etapa tal o en depósito en el puerto tal) hasta que finalmente termina con la carga depositada en el puerto de destino a la espera de ser retirada.

Llevando adelante sus respectivas funciones, cada área contribuye de diferente forma en la realización de cada uno de los procesos analizados. Es decir que estos procesos se construyen con la integración de las distintas funciones de negocio. Si optáramos por la alternativa de un único sistema, las funciones serían módulos del mismo sistema. Si optáramos por la otra alternativa, deberíamos analizar cuáles serían los diferentes sistemas y qué funciones incluirían.

Las funciones vinculadas a la administración de los clientes podríamos especular que se mantendrán relativamente estables. Sin embargo, es probable que la administración de la capacidad pueda variar con el crecimiento de la empresa e incorpore diferentes soluciones, como anexión de inmuebles y transportes no propios, etc.

Las funciones relacionadas con la facturación también variarán pero en diferente dirección. Los cambios de tarifa para los diferentes ítems, los cambios de legislación de impuestos, los cambios en la forma de cálculo de determinados subtotales harán que este módulo esté sometido a cambios de otra frecuencia y naturaleza.

La gestión de la asignación de transportes a las cargas (Logística), con la administración de viajes, seguramente cambiará de una forma también propia.

Por las razones planteadas es que se propone el desarrollo de diferentes aplicaciones o sistemas que resuelvan los diferentes aspectos del negocio.

En la figura 13.5 se muestra un mapeo entre los procesos ya analizados y las aplicaciones que, trabajando en forma integrada, permitirán llevar adelante dicho negocio.

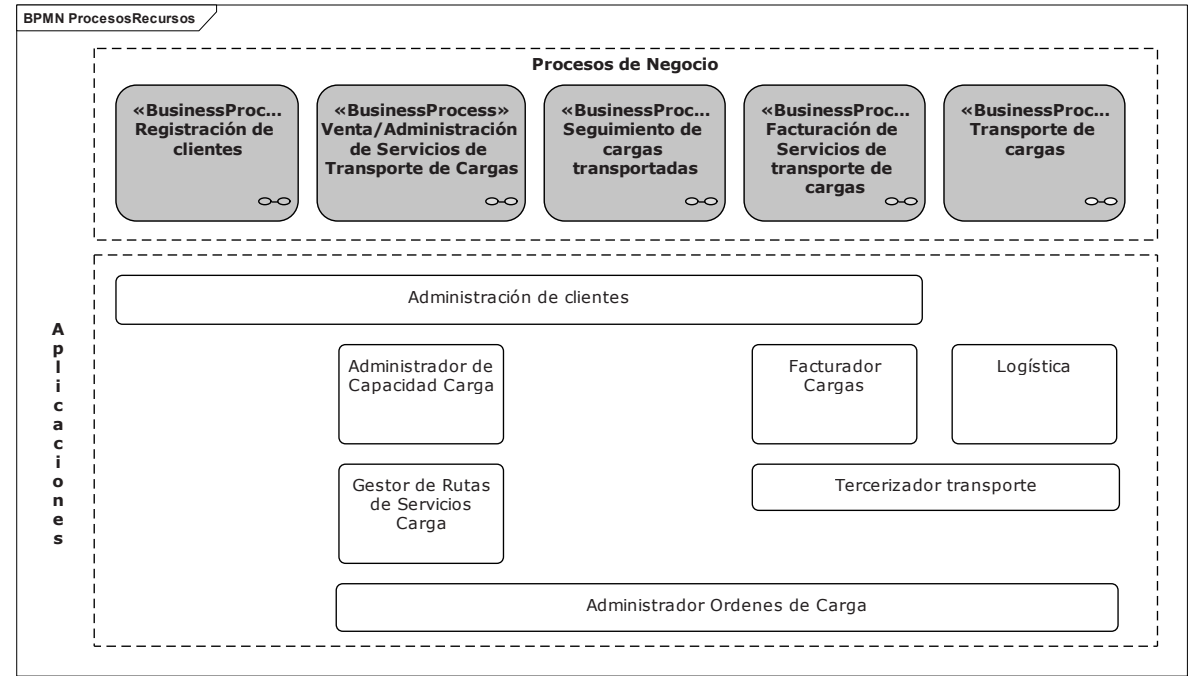


Figura 13.5. Mapeo de procesos a funciones de negocio

Se propone desarrollar una aplicación para la Administración de Clientes. Otra para la realización de la Facturación, que en el caso de no tercerizar el servicio de transporte incluirá el manejo de las tarifas de los viajes. El transporte se gestionará a partir de la aplicación nombrada como Logística y en el caso de la tercerización habrá un módulo (Tercerizador Transporte) que se ocupará de la conexión con el sistema externo de la otra

empresa. Existe una restricción respecto a la integración y es que debe implementarse del modo especificado por la otra empresa, a partir de un API (*Application Programmer Interface*) que oportunamente publicaron.

La aplicación Administrador de Órdenes de Carga realizará la venta de servicios, la administración de órdenes y el seguimiento de las cargas en tránsito. Para analizar la capacidad, esta aplicación invocará servicios del sistema Gestor de Capacidad de Carga, que por ser un sistema propio ya existente se adaptará para que sean factibles estas invocaciones. Más adelante analizaremos este tipo de integración. El servicio de ruteo según los diferentes criterios si bien es un sistema que forma parte de este contexto se desarrollará utilizando el diseño ya existente y se integrará de la misma forma que el anterior. A partir de estas definiciones concluimos que la arquitectura del sistema en desarrollo estará compuesta de varias aplicaciones integradas a partir de la prestación de servicios a las otras. Es decir que tendrá componentes de una arquitectura orientada a servicios (SOA – *Service Oriented Architecture*). Cada una de las aplicaciones tendrá las siguientes características:

- Multiusuario
- Una o más base de datos
- Accesos concurrentes
- Lógica de negocio
- Múltiples interfaces de usuario.

Lo cual las define como aplicaciones de tipo *enterprise* (Empresariales) [3]. Estas aplicaciones poseen una arquitectura en capas verticales y horizontales que analizamos a continuación.

13.1.1.2 Arquitectura conceptual

Las aplicaciones de tipo *enterprise* son aplicaciones Web, por lo que su arquitectura deriva del cliente / servidor, a la que suman una capa de negocio. Estas capas verticales (*tiers*) se disponen en la forma que se indica en la figura 13.6. Este estilo o patrón de arquitectura es nombrado *enterprise* cuando posee las características arriba mencionadas. La mayor parte de las tecnologías poseen un modelo de desarrollo que soporta esta arquitectura [4], [5]. Las capas que se presentan en la figura son Presentación, Negocio y Persistencia. El acceso a la capa de Negocio es a través de una capa no mostrada cuyo objetivo es controlar el acceso al negocio. Esta es conocida como capa de Servicios de Aplicación e implementada mediante un patrón de diseño conocido como *facade* [3], [7].

Además, por estar este tipo de aplicaciones orientadas a resolver problemas que presentan diferentes aspectos de muy distinto nivel de abstracción, es una buena práctica que el diseño de las capas verticales (*tiers*) esté estratificado en capas horizontales (*layers*). Esto permite una serie de beneficios tales como el cambio, la reutilización, el desarrollo simultáneo, la prueba independiente, entre los más destacados. La clave de este patrón de arquitectura es la definición de interfaces estables para cada capa y un criterio de cohesión que determine cuáles componentes comparten una capa. En el caso de la capa de negocio este criterio se deriva directamente de este. En este tipo de patrón, cada capa solicita servicios a la inmediata inferior. Las capas inferiores son más genéricas e independientes y por lo tanto más reutilizables.

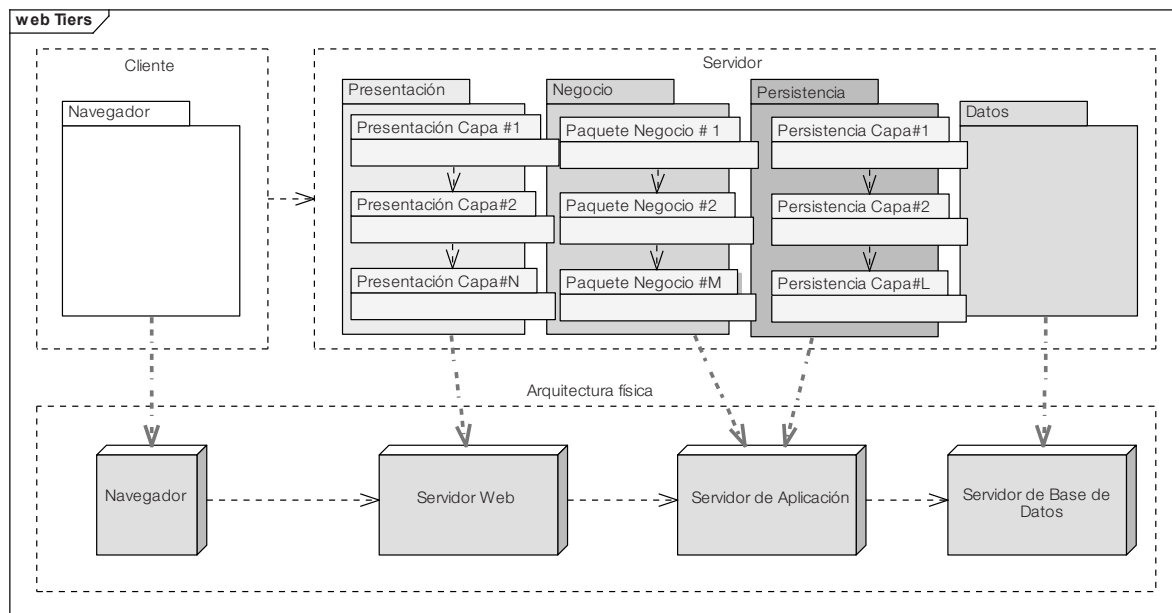


Figura 13.6. Capas verticales y horizontales de la arquitectura conceptual de tipo *enterprise* y la arquitectura física sobre la cual funciona

13.1.1.3 Arquitectura física

Para cada aplicación *enterprise* la arquitectura física está compuesta por los servidores Web, de aplicación y de base de datos, como se muestra en la figura 13.6. Cada uno de estos estará conformado por el hardware y software de base y de aplicación. Podrán ser distribuidos o no, dependiendo de los requerimientos no funcionales del sistema, es decir que funcionarán en nodos separados o en uno mismo. En el caso del sistema en desarrollo estarán distribuidos ya que alguno de sus componentes, como el sistema de transporte de terceros y el de la propia empresa, están instalados en su propio hardware.

La integración de aplicaciones de tipo *enterprise* que utilizan servicios exportados por estas para la implementación de un proceso de negocio se observa en la figura 13.7. En ella se puede ver la capa de aplicación donde se alojan las aplicaciones, la capa de servicios compuesta por los servicios exportados y por último la capa de procesos de negocio donde se integran los servicios anteriores. Las claves de esta integración son la comunicación asincrónica, la diversidad de protocolos y la interoperatividad. Para su implementación se recurrirá a un ESB (*Enterprise Service Bus*) que permitirá la comunicación entre diferentes protocolos con la conversión de datos apropiada y el ruteo de los mensajes a los destinos correspondientes.

El caso del ambiente operacional que permitirá esta integración se muestra esquemáticamente en la figura 13.8.

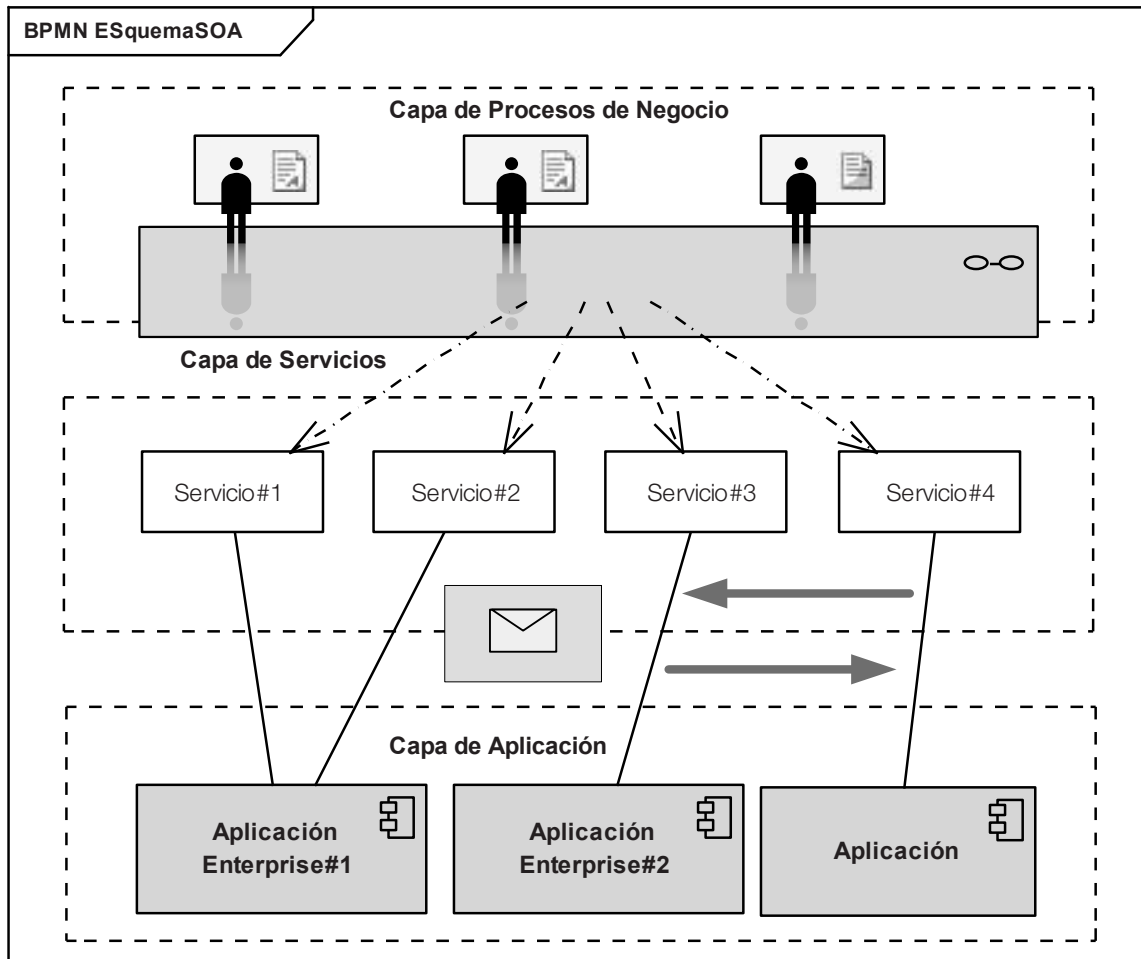


Figura 13.7. Esquema de la arquitectura de tipo SOA

13.1.2 Documentación de la arquitectura

A partir del análisis anterior comenzamos a dar forma a la arquitectura del sistema de administración de cargas. Hemos concluido que estará compuesto por las siguientes aplicaciones *enterprise* a desarrollar Ventas/Administración de Cargas y Facturación. También por las aplicaciones Gestión de Ruteo de Transporte de Cargas y Administración de Capacidad de Transporte de Cargas. La primera será desarrollada con la propiedad de publicación de servicios de ruteo en un ambiente SOA y la segunda será adaptada para que pueda participar del mismo contexto de funcionamiento. Además se adaptará la aplicación *enterprise* de Administración de Transporte de Carga ya existente para que sea factible integrarla mediante la utilización de sus servicios en el mismo contexto de funcionamiento que las anteriores. Debido a que la tercerización del servicio de transporte se realiza a partir de la comunicación con un sistema externo no propio, esta

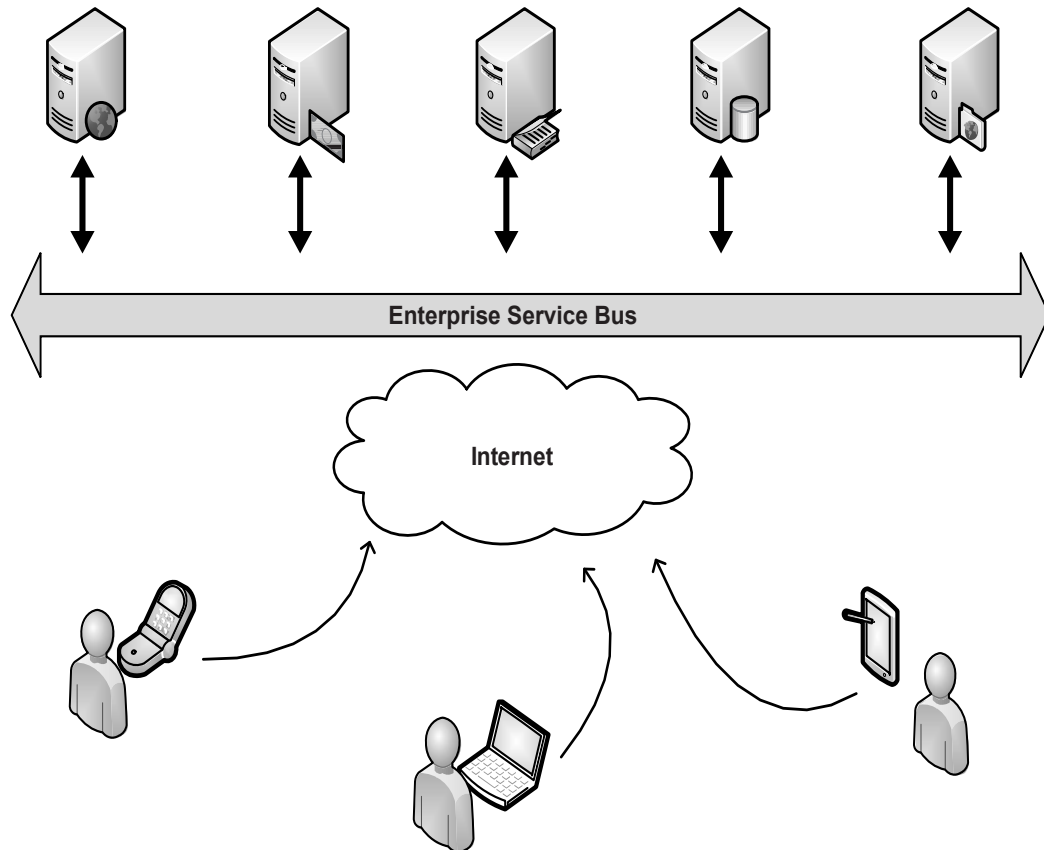


Figura 13.8. Ambiente operacional de sistemas *enterprise* integrados

integración debe hacerse mediante un conector que utiliza el API publicada por la empresa propietaria. De esta definición resulta la arquitectura cuya vista estática se muestra en la figura 13.8 que se describe a continuación.

13.1.3 Vista estática

La arquitectura que se definió para el sistema en desarrollo (figura 13.9) está compuesta de tres capas principales. La capa inferior, nombrada Infraestructura, contiene los recursos de base que proveen los servicios de infraestructura como seguridad, conectividad, contexto de ejecución, persistencia y la conexión con los sistemas externos. De estos deben desarrollarse el último, integrarse la persistencia y configurarse el resto.

La capa intermedia nombrada Aplicación está compuesta por las aplicaciones *enterprise*. Esta a su vez está dividida en las capas verticales (*tiers*) de estas aplicaciones, como se analizó anteriormente.

Por último, la capa superior, nombrada **Negocio**, está compuesta por los procesos de negocio implementados a partir de la integración de los servicios publicados por las aplicaciones de la capa anterior.

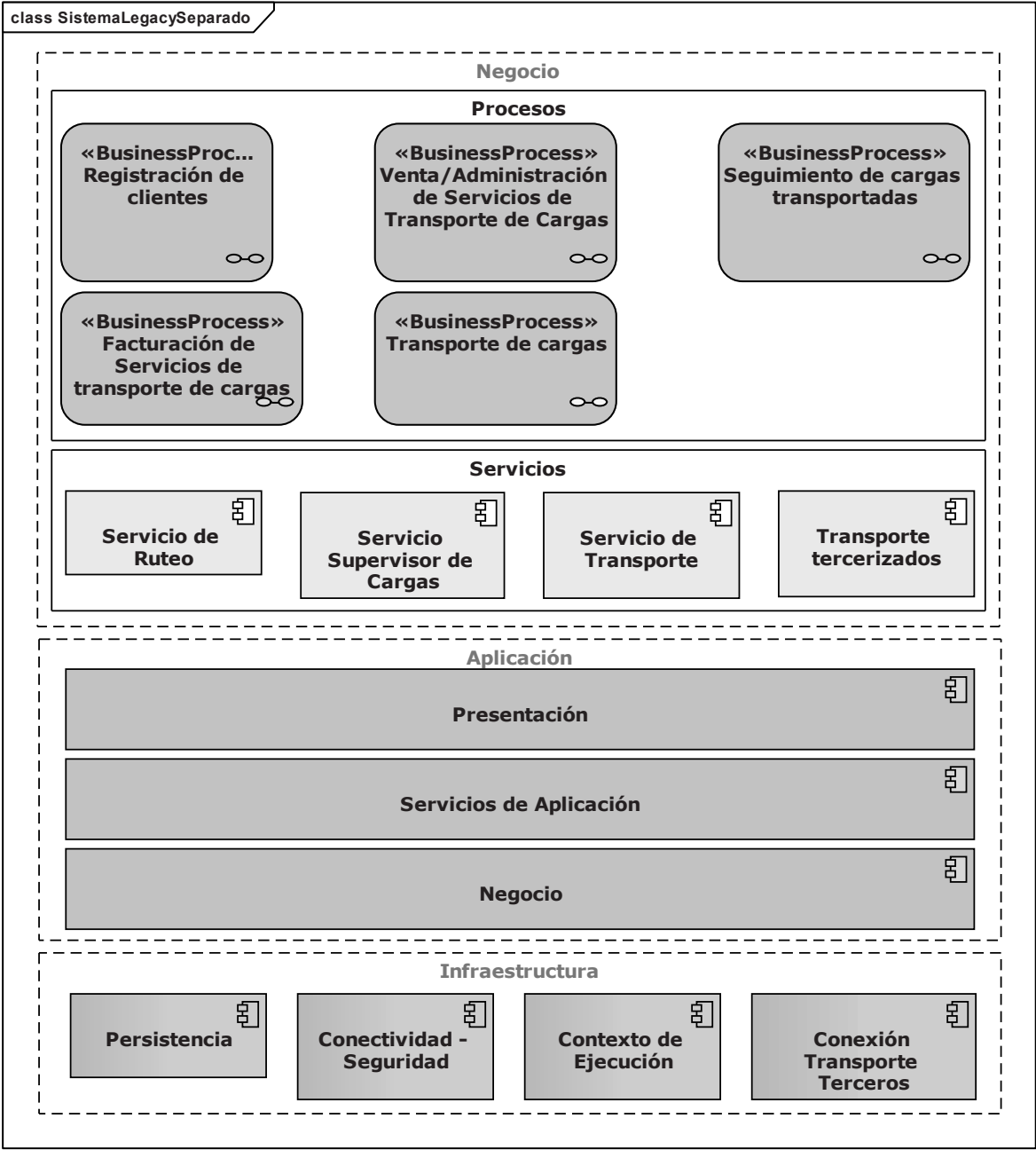


Figura 13.9. Vista estática de la arquitectura

En la figura 13.10 se presenta, a modo de ejemplo, la implementación del proceso de negocio Venta / Administración de Servicios de Transporte de Cargas a partir de los distintos servicios integrados exportados por las aplicaciones.

Hasta aquí hemos definido la arquitectura de componentes, es tiempo de seleccionar las plataformas marco (*frameworks*), software de base y de desarrollo a utilizar. Esta actividad la describimos en la próxima sección.

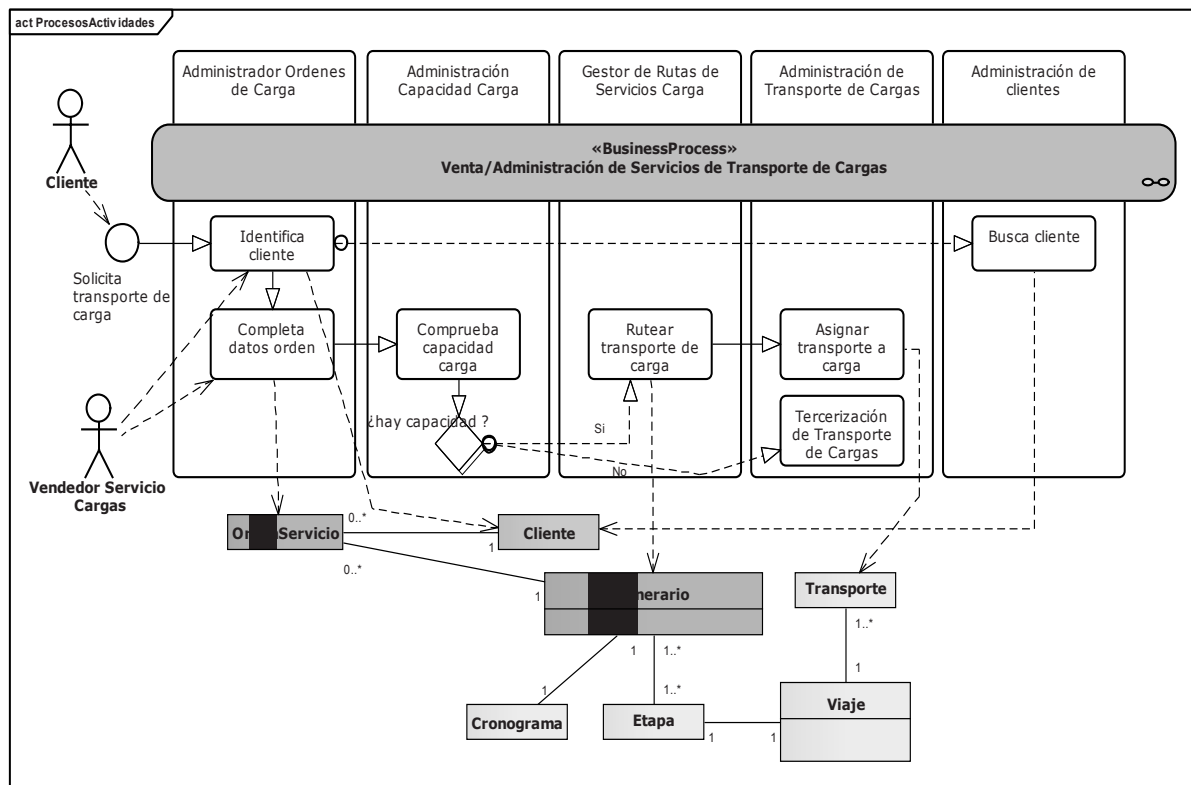


Figura 13.10. Proceso de negocio Ventas/Administración de Servicios de Transporte de Cargas y los servicios aportados por las diferentes aplicaciones que contribuyen a su implementación

13.2 Selección de componentes

13.2.1 Plataformas marco (*Framework*)

Le empresa tiene una historia de proyectos de desarrollos C++ y Java que ejerce influencia a la hora de la elección del lenguaje de programación. Si bien no cuenta con un grupo de expertos, la mayoría de los desarrolladores tiene experiencia en programación Java. Además es política de la empresa promover la utilización de tecnologías “open source” y “free”, con lo cual en el área de desarrollo se ha instalado la idea de convertirse en especialistas en el desarrollo de sistemas en lenguaje Java con la utilización de este tipo de software. Se han desarrollado varias capacitaciones con la participación de la mayoría de los programadores y llevado adelante una serie de proyectos menores a modo de pruebas piloto. Basados en esta definición se seleccionaron las plataformas y herramientas de desarrollo.

Para la implementación de las capas verticales (*tiers*) de las aplicaciones *enterprise* analizamos la selección de diferentes plataformas. Se evaluó cómo implementar la persistencia de los objetos de negocio, para lo cual se manejaron diferentes alternativas. De la misma forma fue necesario definir el uso de alguna plataforma que ayude en la implementación de la capa de presentación. Estas plataformas debían ser integradas a partir de la programación de código o a partir de un servidor de aplicaciones que también debería seleccionarse.

Se definieron cuáles serían los criterios de selección para estas plataformas de software en sus aspectos generales:

- Madurez, estabilidad y fiabilidad del producto
- Cantidad de usuarios en el mercado
- Soporte y / o actividad de la comunidad *open source*
- Productividad del grupo de desarrollo
- Flexibilidad de extensión
- Facilidad de implementar pruebas
- Política de manejo de errores
- Bajo acoplamiento
- Simplicidad en la configuración
- Promoción de buenas prácticas de programación orientada a objetos
- Facilidad de aprendizaje.

Además se impusieron los siguientes requerimientos:

Para el servidor de aplicaciones

- No invasivo del modelo de negocio
- Desacoplamiento y abstracción que permitan fácil integración con el acceso a los datos (persistencia)
- Administración de transacciones
- Definición declarativa de transacciones
- Modelo de programación y soporte para comunicación asincrónica y objetos remotos
- Facilidad de implementación de pruebas unitarias.

Se evaluaron Jboss Microcontainer [8], Spring [9], Pico/Nano [10], Google Guice [11] and Yan/Nuts [12]. Se decidió la utilización de la plataforma Spring.

Para la plataforma de presentación

- Promoción de reutilización de componentes y diseño
- Soporte para validación
- Promoción del modelo MVC
- Facilidad de configuración de acceso a componentes y flujo de navegación
- Facilidad de integración.

Se evaluaron Struts2 [13], Spring MVC [14] y JSF [15]. Se decidió la utilización de Struts2.

Para la plataforma de persistencia

- Mapeo de objetos a tablas
- Administración de cache de instancias en uso
- Administración de concurrencia
- Factibilidad de cargas tardías
- Facilidad de configuración
- Facilidad de integración
- Definiciones declarativas.

Se evaluaron Hibernate [16], iBatis [17] y TopLink [18]. Se decidió por la utilización de Hibernate.

Como software de infraestructura se decidió la utilización de Apache [19], Tomcat-6 [20] y Axis-2 [21] como servidores Web y Web Services.

Los servicios exportados por las aplicaciones *enterprise* serán implementados como Web Services al nivel de la capa de acceso al negocio.

En las figuras 13.11 se muestra la integración de Struts y Spring a partir de su configuración.

En las figura 13.12 se muestra la integración de Hibernate y Spring a partir de su configuración.

En la figura 13.13 se puede ver la infraestructura resultante de las aplicaciones *enterprise* que utilizan estas plataformas.

13.2.2 Comunicación con sistemas externos

La comunicación con los sistemas externos se muestra en la figura 13.14, donde pueden verse algunas de las definiciones ya adelantadas en secciones anteriores. Se adaptará el sistema propio de Transporte para que a través de la exportación de servicios pueda integrarse con el resto de las aplicaciones. Del mismo modo se adaptará el sistema de administración de la capacidad de carga utilizando la misma estrategia.

Para el caso de la comunicación con el sistema de transporte de terceros, se deben utilizar las interfaces por ellos publicadas para la implementación de dicha conexión. Para esto se implementará una capa de anti-corrupción [6] utilizando patrones de diseño del tipo *facade* y *adapter* [7]. Dicho diseño se mostrará en forma detallada en el próximo capítulo. El criterio que nos mueve a desacoplar ambos sistemas es que el de terceros fue desarrollado en otro tiempo, por otra gente y con otros objetivos, y es deseable que el modelo de ese sistema no distorsione al desarrollado en este proyecto.

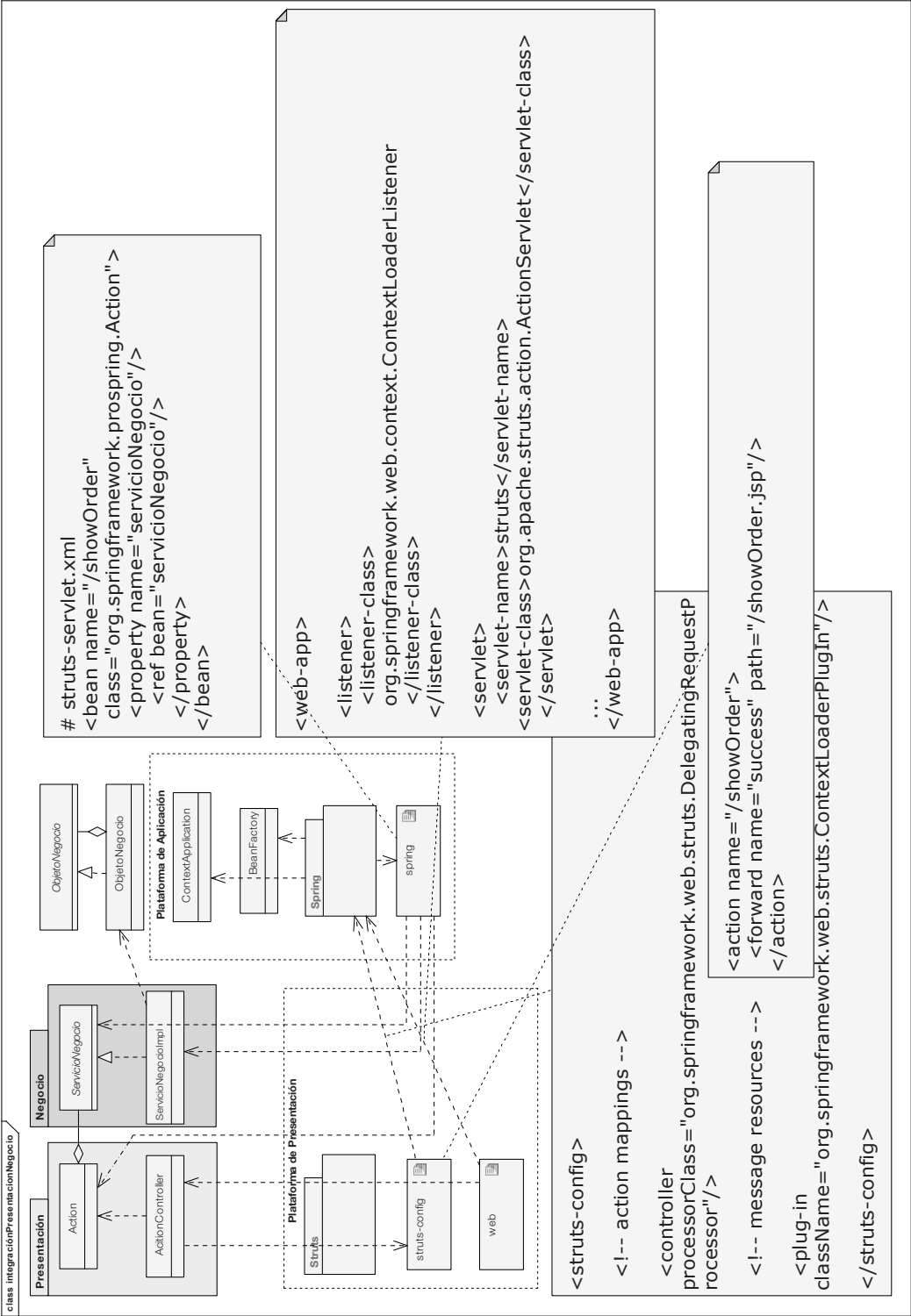


Figura 13.11. Diagrama esquemático que muestra la integración de Struts con Spring

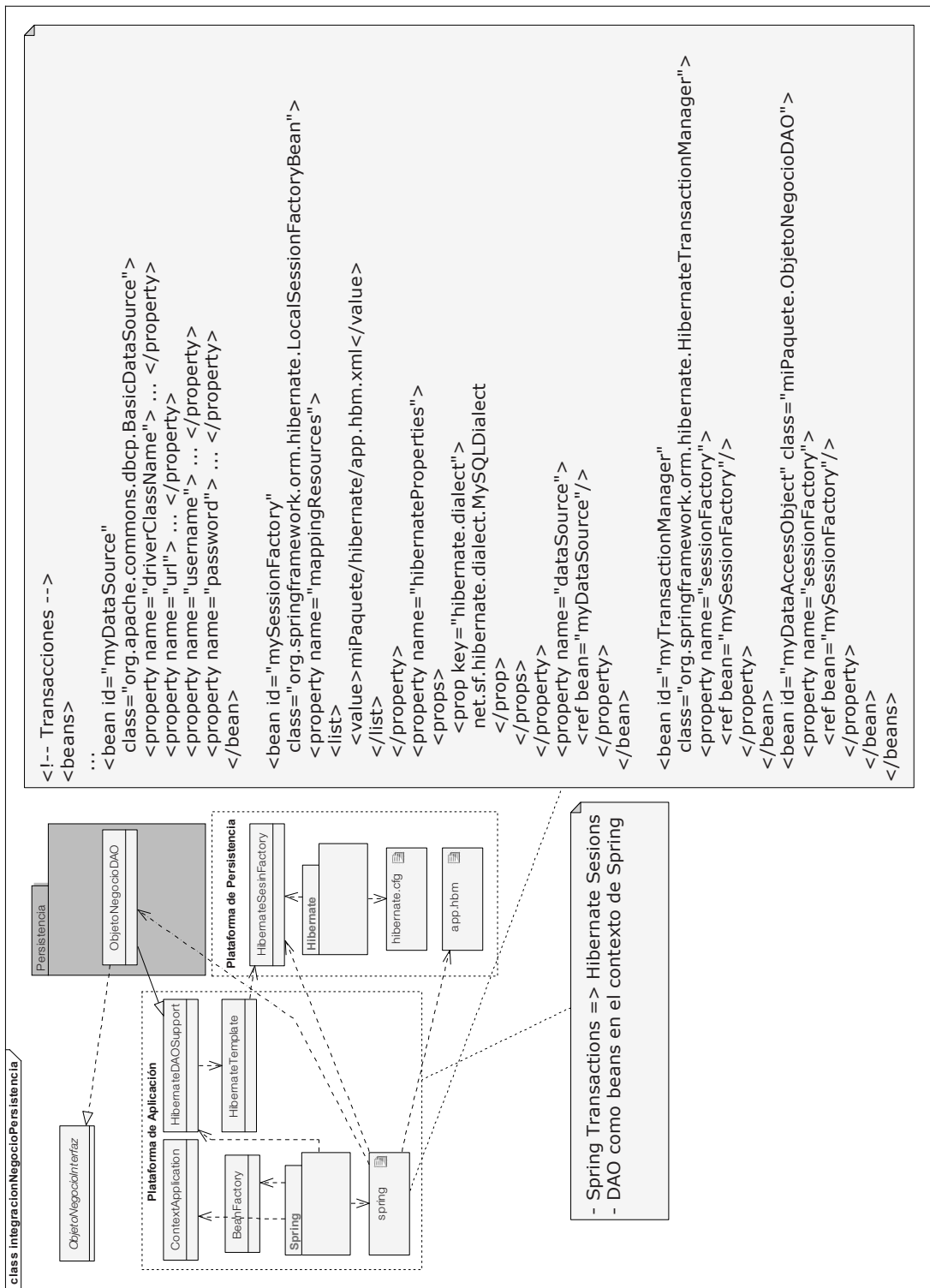


Figura 13.12. Diagrama esquemático que muestra la integración de Spring con Hibernate

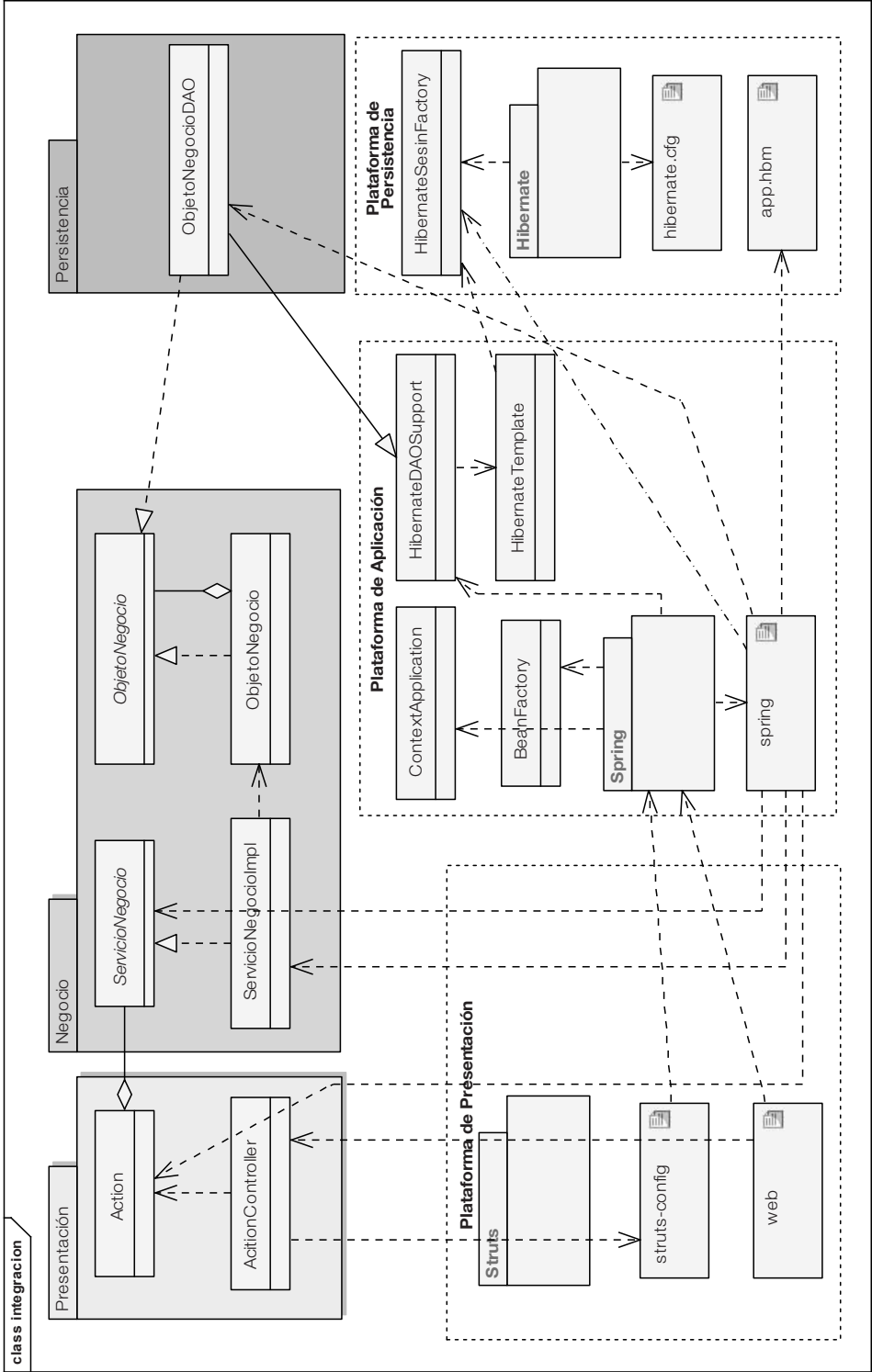


Figura 13.13. Diagrama esquemático de la arquitectura de una aplicación enterprise utilizando Struts, Spring y Hibernate

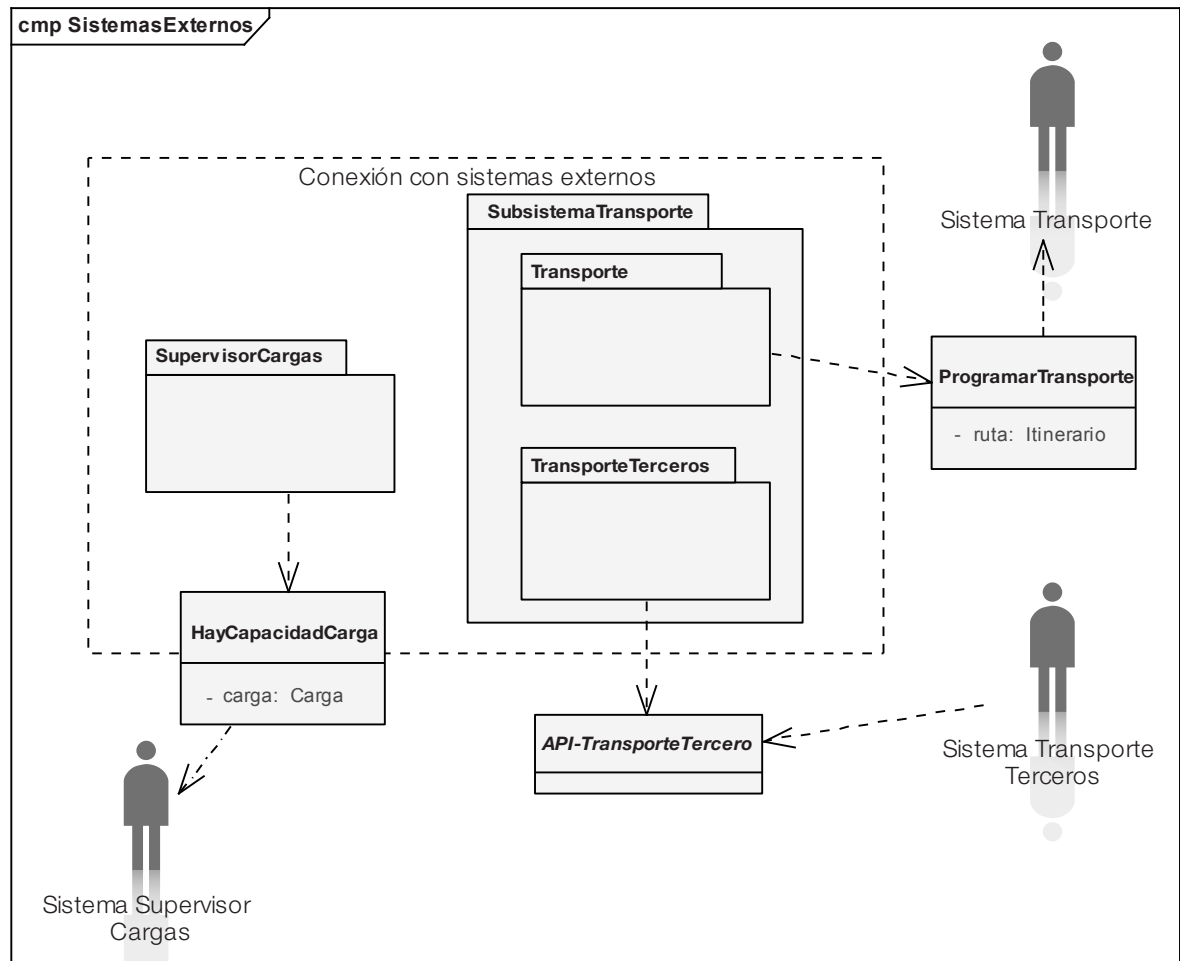


Figura 13.14. Diagrama esquemático que presenta las diferentes formas de comunicación con los sistemas externos

13.2.3 Atributos de calidad

De los requerimientos no funcionales del sistema en desarrollo se derivan los siguientes atributos de calidad, los cuales deben ser garantizados por la arquitectura elaborada:

- Confiabilidad
- Usabilidad
- Seguridad
- Disponibilidad
- Escalabilidad
- Fácil mantenimiento.

A efectos de asegurar la confiabilidad se planificó la realización de pruebas del software en un esquema de Integración Continua con herramientas de tipo xUnit [22]. En un próximo capítulo nos extenderemos en esta forma de trabajo. Para lograr un sistema usable se tendrán en cuenta las buenas prácticas de diseño de interfaces de usuario [23]. A efectos de construir un sistema seguro se utilizará el esquema de autenticación y autorización que ofrece Spring (Acegi Security System) [9]. En relación con la disponibilidad de 7x24x365 requerida se pensó en un esquema de redundancia de hardware que permite al sistema prestar sus servicios sin interrupciones. El énfasis en la escalabilidad se basa en las buenas prácticas de diseño y programación que permitirán al sistema crecer con flexibilidad y la utilización de plataformas maduras que ya fueron usadas en el desarrollo de sistemas de gran volumen. Por último el fácil mantenimiento se logrará a partir de buenas prácticas de diseño, en especial si se introduce la “testeabilidad” como criterio de buen diseño basada en la separación de incumbencias y en la realización de pruebas atómicas e independientes. De esta forma se garantiza un mantenimiento ordenado y sistemático.

13.2.4 Pruebas a la arquitectura

Las pruebas a la arquitectura que se planean realizar por considerarse fundamentales son el análisis de dependencia entre paquetes y la medida de comportamiento en condiciones de carga. A efectos de garantizar la escalabilidad y el fácil mantenimiento es de gran importancia que los paquetes que componen las diferentes partes del sistema posean un grado de abstracción e inestabilidad determinados de acuerdo al objetivo de cada paquete. Para esto se utilizará una herramienta conocida como JDepend [22]. También es importante que no haya dependencias circulares entre dichos paquetes. Para su detección se utilizará iCode [22] como herramienta de análisis. Para la realización de las pruebas de carga se utilizará una herramienta llamada JMeter [22]. En un próximo capítulo presentamos los resultados de estas pruebas.

13.2.5 Herramientas de desarrollo

Entre las herramientas importantes para el desarrollo se seleccionaron las siguientes [22]:

Ambiente de programación: Eclipse, por su madurez, gran comunidad de usuarios, soporte y facilidad de integración con otras herramientas.

Plataformas de prueba: JUnit, DBUnit, ServletUnit/HttpUnit y Test de Watir por sus características que permiten la realización de pruebas sistemáticas y automáticas. FitFitsesse por sus características que permiten la automatización de pruebas de funcionalidad y JMeter para la realización de pruebas de sistema.

Organizador del proyecto: Maven2 por su servicio de construcción de binarios e instalación automática de componentes, por su madurez y comunidad de usuarios, soporte y facilidad de integración con otras herramientas.

Integración Continua: Continuum por su facilidad de configuración y uso.

La selección de las herramientas de desarrollo y soporte además de estar tecnológicamente alineadas con las plataformas seleccionadas están motivadas también por los atributos de calidad del proyecto.

13.3 Conclusión

El proceso de definición de la arquitectura de un sistema es uno de los fundamentales del ciclo de vida del desarrollo. Como fue visto necesita de información de la organización y de las necesidades planteadas para el actual sistema en desarrollo y cómo este funcionará insertado en su ambiente operacional.

Son importantes el contexto de la organización, las plataformas y las herramientas de desarrollo. Las decisiones de diseño que se tomen en este proceso así como la selección del software a utilizar están íntimamente relacionadas.

En el próximo capítulo desarrollaremos los temas vinculados al diseño y la construcción de las diferentes partes del sistema.

13.4 Contenido de la página web de apoyo



13.4.1 Mapa conceptual

13.4.2 Autoevaluaciones

13.4.3 Presentaciones en Power Point (exclusivo para el docente)

13.5 Referencias

- [1] *A framework for information systems architecture*, J. A. Zachman, IBM System Journal, vol. 26, 1987.
- [2] *The 4+1 View Model of Architecture*, I Philippe Kruchten, IEEE Software, 12 (6), November 1995, IEEE, pp.42- 50.
- [3] *Patterns of Enterprise Application Architecture*, Martin Fowler, Addison-Wesley, 2002.
- [4] *The J2EE Tutorial, 2d edition*, Stephanie Bodooff, Eric Armstrong, Jennifer Ball, Debbie Carson, Ian Evans, Dale Green, Kim Haase, Eric Jendrock, Sun, 2004.
- [5] *Applied Microsoft .NET Framework Programming*, Jeffrey Richter, Microsoft, 2002.
- [6] *Domain Driven Design*, Eric Evans, Addison-Wesley, 2004.
- [7] *Design Patterns: Elements of Reusable Object-Oriented Software*, Erich Gamma, Richard Helm, Ralph Johnson, and John M. Vlissides, Addison-Wesley, 1994.
- [8] <http://jboss.org/jbossmc>.
- [9] <http://www.springsource.org/>.
- [10] <http://www.picocontainer.org/>.
- [11] <http://code.google.com/p/google-guice/>.
- [12] <http://yan.codehaus.org/Nuts>.
- [13] <http://struts.apache.org/2.2.1/index.html>.
- [14] <http://www.springsource.org/>.
- [15] <http://www.oracle.com/technetwork/java/javaee/javaserverfaces-139869.html>.
- [16] <http://www.hibernate.org/>.

- [17] <http://ibatis.apache.org/>.
- [18] <http://www.oracle.com/technetwork/middleware/toplink/overview/index.html>.
- [19] <http://httpd.apache.org/>.
- [20] <http://tomcat.apache.org/>.
- [21] <http://ws.apache.org/axis/>.
- [22] *Calidad en el Desarrollo de Software*, Guillermo Pantaleo, Alfaomega, 2011.
- [23] *Quality Attributes of Web Software Applications*, Jeff Offutt, IEEE Software, 2002.

Diseño de software

Contenido

14.1 El proceso de diseño	222
14.2 Desarrollo del Caso Testigo.....	224
14.3 Conclusión	237
14.4 Contenido de la página web de apoyo	237
14.5 Referencias	237

Objetivos

- Aprender el concepto de diseño
- Aprender y utilizar los criterios de buen diseño
- Aprender el concepto de patrones de diseño
- Practicar los conceptos anteriores aplicándolos al caso testigo

Competencias

- Establecer los criterios de diseño que se utilizarán en un proyecto de desarrollo para consolidar y promover los principios con que fue definida la arquitectura

14.1 El proceso de diseño

En este capítulo presentaremos el diseño de algunos aspectos vinculados con el caso testigo que venimos desarrollando en los capítulos anteriores. Enfatizaremos las buenas prácticas de diseño de software y las analizaremos desde la perspectiva de la Ingeniería de Software.

En el capítulo anterior hemos definido la arquitectura del sistema en desarrollo a partir del análisis de las necesidades de los usuarios en el contexto de la empresa. Es tiempo de diseñar el interior de los módulos concebidos como ejercicio de esa definición.

Cualquier gerente de desarrollo desea que el diseño del software de sus proyectos se elabore con criterios que hagan a los productos generados reusables, flexibles para extenderlos, fáciles de probar y mantener, simples de documentar y comunicar, y robustos.

Las buenas prácticas de diseño que presentaremos constituyen una buena parte del cuerpo de conocimiento de la Ingeniería de Software y los beneficios de trabajar utilizándolas son:

- reusables para que los componentes de diferentes proyectos puedan ser compartidos y por lo tanto más robustos al haber sido probados en diferentes contextos;
- fáciles de probar para que cada aspecto o incumbencia pueda ser asociado a un test atómico e independiente y la detección de fallas facilitará el mantenimiento;
- simples de documentar y comunicar para evitar fallas y facilitar que los desarrolladores los utilicen de la mejor manera a partir de la reutilización;
- robustos para que funcionen en los más diversos contextos de ejecución.

Todos estos beneficios se convierten en atributos de calidad que califican a una empresa a partir de los productos por ella producidos.

14.1.1 Criterios de buen diseño

Los criterios mencionados constituyen una guía a la hora de elaborar un diseño para la solución de un determinado problema. Citamos a continuación algunos más generales y luego algunos más concretos. Criterios generales [1], [2]:

14.1.1.1 Economía

Flexibilidad arbitraria por sí misma, características innecesarias, complejidad en el diseño, atributos fuera de proporción con el problema a resolver y un enfoque en la reutilización en lugar de la usabilidad. Todos estos síntomas muestran ausencia de economía.

14.1.1.2 Visibilidad

Este criterio está directamente ligado a la expresividad de los artefactos que describen la arquitectura. Este síntoma de escasa visibilidad dificulta la comunicación del diseño a los desarrolladores. La utilización de patrones de diseño es una forma de hacer más visible un diseño.

14.1.1.3 Espaciamiento

La separación de incumbencias es un claro ejemplo de buen espaciamiento. La separación en paquetes a partir de un criterio de cohesión determinado garantiza un buen espaciamiento. Si el criterio deriva del negocio, tendremos un espaciamiento adecuado desde esa perspectiva, si deriva de cuestiones técnicas relacionadas a los servicios tendremos un buen espaciamiento en la infraestructura.

14.1.1.4 Simetría

Un diseño simétrico es más fácil de comunicar y entender. Además de expresar balance y consistencia, es intuitiva. Un ejemplo de esto podrían ser los patrones creacionales de tipo *factory*, si se ocupan de la creación solamente expresan asimetría. Si además de la creación se ocupan de la destrucción, entonces expresan simetría.

14.1.1.5 Emergencia

Forzar la estructura y el control de forma centralizada genera diseños que no son buenos. Es deseable que estas propiedades surjan, emerjan naturalmente de la alternativa seleccionada acorde al problema a resolver.

Criterios específicos [3]:

14.1.1.6 Inversión en la cadena de dependencia

Este criterio de buen diseño establece que los módulos de alto nivel no deben depender de aquellos de más bajo nivel y que ambos deben depender de abstracciones.

14.1.1.7 Código clausurado ante cambios

Este criterio establece que el código de nuestras clases debe presentarse abierto a la extensión pero cerrado a modificaciones.

14.1.1.8 Principio de substitución

Este criterio establece que los subtipos deben poder ser substituidos por sus tipos base.

14.1.1.9 Segregación de interfaces

Este criterio establece que los componentes cuyas interfaces carezcan de cohesión en sus métodos presentarán una desventaja que se manifestará a través de su uso por clientes con muy diferentes intensiones.

Este principio propone ante estos síntomas romper las interfaces en otras cuyos métodos estén agrupados por algún criterio de cohesión y sean utilizados con una única intensión.

14.1.1.10 Dependencias no cíclicas

Este criterio establece que no es deseable contar en un diseño con dependencias cíclicas que generen dependencias en el desarrollo y las pruebas, además de limitar la posibilidad de reutilización.

Puede verse que todos y cada uno de los criterios mencionados son la base para lograr los beneficios enumerados más arriba. Existen excelentes libros que tratan en detalle los aspectos del diseño de software, ellos son mencionados en las referencias de este capítulo [1] – [7].

14.1.2 Patrones de diseño

Hacia el año 1995 tuvo lugar un suceso de fundamental importancia en el área del desarrollo de software, fue la constitución de una corriente de estudio llamada *Patrones de Diseño*. El primer libro de texto surgido de ese movimiento fue el libro de Gamma y otros [7]. La esencia que motorizó esta escuela de diseño de software fue la catalogación de las diferentes soluciones propuestas a diversos problemas a lo largo de años en distintos contextos. A estas soluciones se las llamó *Patrones*. Esta sistematización del cuerpo de conocimiento relacionado al diseño fue una de las más importantes contribuciones a la Ingeniería de Software de las últimas décadas. En estas soluciones están presentes de forma básica los criterios de buen diseño listados en la sección anterior. Las características salientes de expresar los diseños a través de los patrones son:

- Representan la experiencia y conocimiento de un experto
- Expresan un diseño con un nivel de abstracción superior al de las clases
- Establecen un idioma compacto y expresivo entre los miembros de un grupo de desarrollo
- Descomponen en micro arquitecturas la arquitectura de un sistema

Cuando analicemos alguno de los aspectos del diseño del caso testigo apelaremos al uso de estos patrones.

14.2 Desarrollo del Caso Testigo

Como se deriva de la arquitectura definida en el capítulo anterior, una de las aplicaciones que forman parte del sistema en desarrollo es el Administrador de Cargas. Esta es una aplicación de tipo *enterprise*, como se describió en el capítulo anterior, e incluirá, entre otras, la funcionalidad de búsqueda de cargas a partir de un conjunto de datos asociados tal como se describió en el capítulo de requerimientos a partir del caso de uso CU_015 Buscar Orden carga. Para la realización de la búsqueda, el usuario carga los datos en un formulario (ver figura 10.10) el cual fue implementado en este desarrollo con el nombre de BusquedaCargaForm. La solicitud de búsqueda es implementada en la capa de presentación a través de la clase BusquedaCargaAction. El acceso al negocio de las funcionalidades asociadas a la administración de cargas está expresada por la interfaz de negocio IAminCarga e implementada por la clase AminCarga.

14.2.1 Presentación

En la implementación de la capa de presentación se definió la utilización de la plataforma Struts que debe ser registrada en el archivo de configuración de la aplicación. En el listado que sigue se muestra este registro, el cual será utilizado por el servidor web (Tomcat) para administrar el funcionamiento de esta capa [10].

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE web-app
  PUBLIC "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
  "http://java.sun.com/dtd/web-app_2_3.dtd">
<web-app>
  <servlet>
    <servlet-name>storefront</servlet-name>
    <servlet-class>org.apache.struts.action.ActionServlet</servlet-class>
    <init-param>
      <param-name>config</param-name>
      <param-value>/WEB-INF/struts-config.xml</param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>storefront</servlet-name>
    <url-pattern>/action/*</url-pattern>
  </servlet-mapping>
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>
  <error-page>
    <error-code>404</error-code>
    <location>/common/404.jsp</location>
  </error-page>
  <error-page>
    <error-code>500</error-code>
    <location>/common/500.jsp</location>
  </error-page>
  <taglib>
    <taglib-uri>/WEB-INF/struts-html.tld</taglib-uri>
    <taglib-location>/WEB-INF/struts-html.tld</taglib-location>
  </taglib>
  <taglib>
    <taglib-uri>/WEB-INF/struts-bean.tld</taglib-uri>
    <taglib-location>/WEB-INF/struts-bean.tld</taglib-location>
  </taglib>
  <taglib>
    <taglib-uri>/WEB-INF/struts-logic.tld</taglib-uri>
    <taglib-location>/WEB-INF/struts-logic.tld</taglib-location>
  </taglib>
</web-app>

```

Este es un típico archivo de configuración escrito en formato XML (eXtensible Markup Language), lenguaje orientado a representar datos con estructuras complejas. XML no es solo aplicado en Internet, sino que es un

estándar para el intercambio de información estructurada entre diferentes plataformas. Se puede usar en bases de datos, editores de texto, hojas de cálculo y otros.

14.2.2 Acceso al negocio

La capa de Acceso al Negocio está formada por un conjunto de clases que concebidas a partir de los casos de uso implementan la dinámica de estas funcionalidades sobre el negocio. Suele llamársele *Capa de Servicios* o *Servicios de Aplicación*. Es el nexos entre la Presentación y el Negocio. El listado de más abajo muestra el código de la clase que a partir de la solicitud de búsqueda invoca los servicios de aplicación (IAdminCarga) para que se realice dicha búsqueda.

```
package com.libroIS.struts.carga.action;

import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import org.apache.struts.action.Action;
import org.apache.struts.action.ActionMapping;
import org.apache.struts.action.ActionForm;
import org.apache.struts.action.ActionForward;
import com.libroIS.struts.carga.service.CargaException;
import com.libroIS.struts.carga.service.IConstantes;
import com.libroIS.struts.carga.view.CargaView;
import com.libroIS.struts.carga.form.BusquedaCargaForm;

/**
 * Este Action es invocada por el ActionServlet cuando el usuario
 * realice una búsqueda de cargas. El ActionForm es una instancia de
 * BusquedaCargaForm.
 */

public class BusquedaCargaAction extends Action {

    public ActionForward execute( ActionMapping mapping, ActionForm form,
                                HttpServletRequest request,
                                HttpServletResponse response )
                                throws Exception {

        ActionForward forward = null;
        Collection cargas = null;
        BusquedaCargaForm busquedaForm = (BusquedaCargaForm) form;

        String identificador = busquedaForm.getIdentificador( );
        String fechaPartida = busquedaForm.getFechaPartida( );
        String puertoOrigen = busquedaForm.getPuertoOrigen( );
```

```

String puertoDestino = busquedaForm.getPuertoDestino( );
String tipoCarga     = busquedaForm.getTipoCarga( );
String nombreCliente = busquedaForm.getNombreCliente( );

// se accede al código de acceso al negocio  serviciosCarga =
getServicioBusquedaCarga( );

// se invoca un servicio de negocio  cargas =
serviciosCarga.find(identificador, fechaPartida, puertoOrigen, puertoDestino,
                    tipoCarga, nombreCliente);

// se preparan los recursos de la sesión
HttpSession session = request.getSession(false);
if(session != null) {
    session.invalidate( );
}
session = request.getSession(true);

// se disponen los recursos en la sesión
session.setAttribute( IConstantes.CARGAS, cargas );
forward = mapping.findForward(IConstantes.SUCCESS_KEY );
return forward;
}

private IAdminCarga serviciosCarga;

public void setIAdminCarga(IAdminCarga serviciosCarga){
    this.serviciosCarga = serviciosCarga;
}
public IAdminCarga getIAdminCarga(){
    return serviciosCarga;
}
}

```

Como se decidió utilizar como plataforma a Spring, en la configuración de Struts se debe agregar la indicación de que la solicitud será delegada para su procesamiento a Spring, además del registro del “action” correspondiente con información de la carga de datos para realizar la búsqueda así como de la página de respuesta con el resultado exitoso o erróneo.

```

<struts-config>
. . .
<controller processorClass=
"org.springframework.web.struts.DelegatingRequestProcessor"/>
. . .
<action

```

```

    path="/carga"
    type="com.libroIS.struts.carga.action.BusquedaCargaAction"
    scope="request"
    name="busquedaCargaForm"
    validate="true"
    input="/carga.jsp">
    <forward name="Success" path="/action/visualizarCarga"
    redirect="true"/>
    <forward name="Failure" path="/carga.jsp" redirect="true"/>
</action>
. . .
<struts-config>

```

En la configuración de Spring se debe registrar el “action” y el servicio de negocio asociado para que Spring pueda instanciarla cuando sea invocada desde el controlador correspondiente y se realice la inyección de la dependencia del servicio.

```

<bean name="/busquedaCarga"
    class="com.libroIS.struts.carga.action.BusquedaCargaAction">
    <property name="serviciosCarga" ref="serviciosCarga" />
</bean>

```

La motivación en el uso de Spring como servidor de aplicación está dada entre otras cosas por dotar al sistema en desarrollo de un grado de desacoplamiento importante. Para esto aprovechamos el mecanismo de Inyección de Dependencia que ofrece esta plataforma. De esta forma los diferentes componentes estarán en condiciones de ser reemplazados sin mayor impacto en el código de las diferentes partes que conocen al resto a partir de declaraciones realizadas en interfaces Java.

La clave de este funcionamiento se basa en el uso de patrones de diseño del tipo Service Locator y Object Factory [8]. Estos diseños se basan en un archivo de registro declarativo de componentes disponibles y una clase que administra su creación utilizando el mecanismo de “reflection” y la identificación por nombre de los componentes que son inyectados en el código detrás de la interfaz que implementan.

En el caso de Spring esto se hace en un archivo applicationContext.xml, para conocer la dependencias de los objetos que gestiona.

En una aplicación que utiliza Struts los componentes mencionados son las instancias de las clases Action y de las clases de acceso al negocio [9], [10], [11].

En el listado que sigue se presenta el código de la interfaz de negocio y en la figura 14.1 un esquema de las dependencias de las clases que participan en la solicitud de búsqueda anterior.

```

public interface IAminCarga {

    public Collection findAll() throws CargaException;
    public CargaDTO find(Integer id) throws CargaException;

```

```

    public CargaDTO find(identificador, fechaPartida, puertoOrigen,
        puertoDestino, tipoCarga, nombreCliente)
        throws CargaException;
    public boolean crearCarga() throws CargaException;
    public boolean modificarCarga(Integer id) throws CargaException;
    public boolean eliminarCarga(Integer id) throws CargaException;
}

```

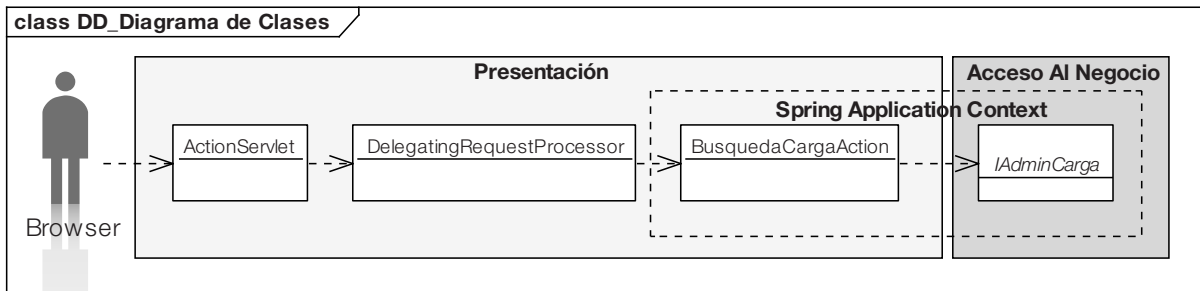


Figura 14.1. Diagrama esquemático que muestra una invocación desde Struts a una clase de tipo “action” y su servicio de negocio asociados registrados en Spring

En la figura 14.2 se presenta un diagrama esquemático de la traza en la implementación del caso de uso utilizado como ejemplo del caso testigo. Se observa la separación de incumbencias lograda a partir del desacoplamiento de las capas de Presentación y Negocio, lograda con la utilización de la plataforma basada en el Patrón de Arquitectura MVC (Model View Controller) y la definición de la interfaz **IAdminCarga** de negocio usada para que la plataforma Spring realice la inyección de dependencia del servicio de negocio implementado por la clase **AdminCarga**.

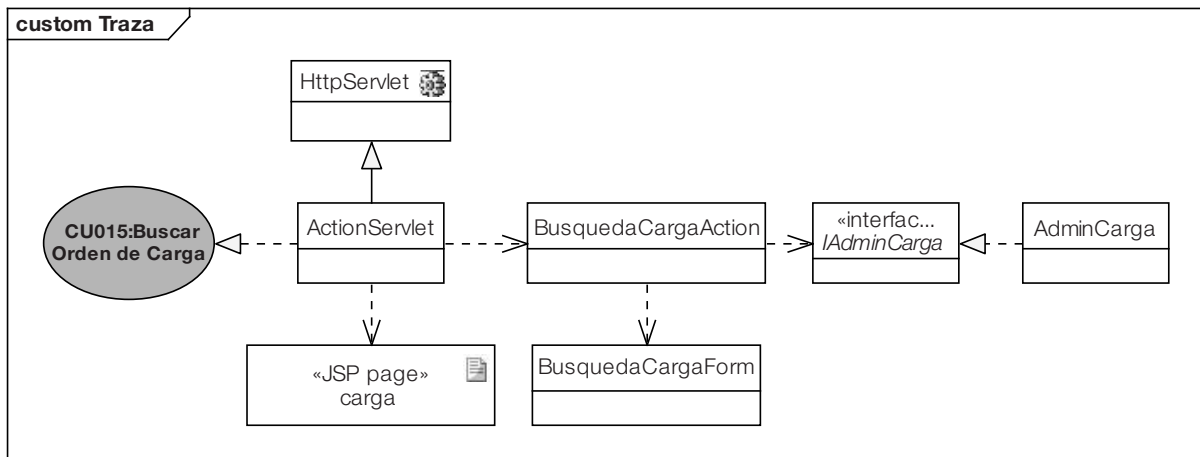


Figura 14.2. Diagrama esquemático que muestra la traza de la implementación del caso de uso CU015 Buscar Orden de Carga hasta la capa de Acceso al Negocio

14.2.3 Negocio

Es importante hacer notar que las clases de negocio fueron concebidas al momento de la construcción del modelo de negocio, como fue descrito en el capítulo 11. Sin embargo en su momento indicamos que su objetivo era conocer y validar el dominio y sus reglas de negocio. En la mayoría de los modelos ocurre que al no estar el objetivo alineado a lograr un buen diseño las clases resultantes y sus relaciones serán candidatas a ser revisadas a la luz de los criterios de buen diseño enumerados al comienzo del capítulo para ser refactorizadas y así cumplir con estos. Los test de validación servirán como guía y reaseguro de que el modelo continúa comportándose según lo establecido por el negocio.

14.2.4 Persistencia

La capa de persistencia de las aplicaciones *enterprise* que forman parte del sistema en desarrollo será administrada por la plataforma Hibernate, como se decidió y fundamentó en el capítulo anterior donde se definió dicha arquitectura. Esta plataforma proveerá los siguientes servicios:

- Mapeo de objetos a tablas relacionales.
- Administración de conflictos por accesos concurrentes.
- Administración de cargas tardías (*Lazy Load*).
- Repositorio temporal de objetos (*Cache*).
- Administración de conexiones a la base de datos.
- Agrupamiento para la ejecución de solicitudes (Unidad de trabajo).

Por su parte la integración con la plataforma Spring proveerá [9]:

- Administración de transacciones en forma programática y declarativa.
- Administración de excepciones.
- Ejecución segura en ambiente multiusuario.
- Clases modelos para el flujo de ejecución.
- Clases de soporte para el acceso a datos DAO (*data access object*).
- Administración de recursos.

Las acciones orientadas al manejo de la persistencia (*save*, *update*, *delete*, *load*) de los objetos son implementadas a través de la interfaz `org.hibernate.Session` y el DAO implementado para cada objeto a persistir. Además los componentes que implementan estas interfaces son administradas por `SessionFactory` de Hibernate. Spring provee una capa abstracta representada en este caso por la clase `HibernateTemplate` que abre y cierra sesiones de Hibernate y convierte las excepciones para su traslado a otras capas. Para configurar Spring en este contexto debe registrarse, como se muestra en el listado que sigue, el nexa con Hibernate.

```
<bean id="hibernateTemplate"
  class="org.springframework.orm.hibernate3.HibernateTemplate">
  <property name="sessionFactory" ref="sessionFactory" />
</bean>
```

Además este nexo debe especificar el modo en que estos componentes serán creados y relacionados, como se muestra en la figura 14.3 y el listado de configuración que sigue.

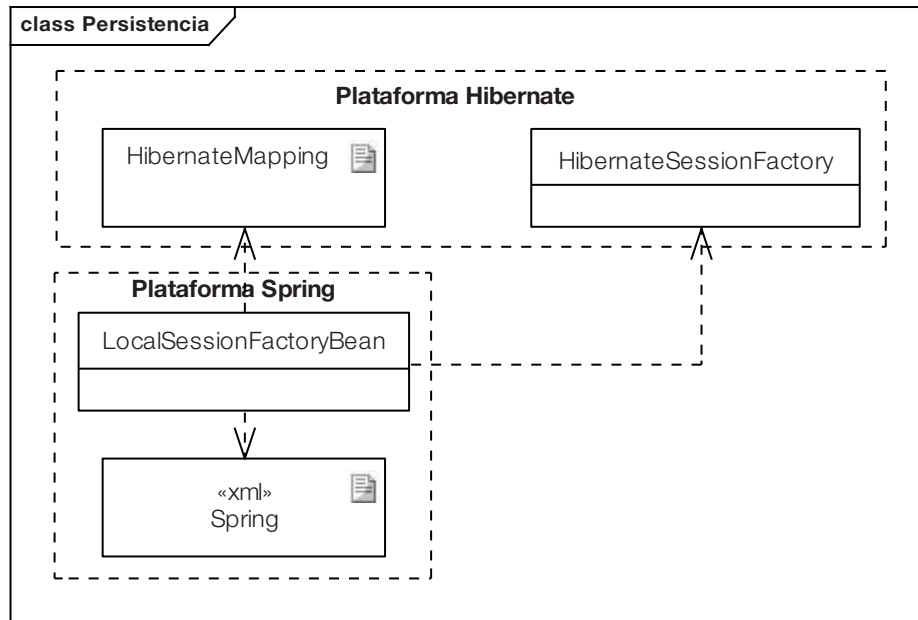


Figura 14.3. Diagrama esquemático que muestra a Spring (LocalSessionFactoryBean) produciendo una HibernateSessionFactory desde un archivo de mapeo

```

<bean id="sessionFactory"
      class="org.springframework.orm.hibernate3.LocalSessionFactoryBean">
  <property name="dataSource" ref="dataSource" />
  <property name="mappingResources">
    <list>
      <value>com/libroIS/cargas/Carga.hbm.xml</value>
      <value>com/libroIS/clientes/Cliente.hbm.xml</value>
      <value>com/libroIS/logistica/Logistica.hbm.xml</value>
    </list>
  </property>
  <property name="hibernateProperties">
    <props>
      <prop key="hibernate.dialect">${hibernate.dialect}</prop>
    </props>
  </property>
</bean>
  
```

En el listado que sigue se muestra la implementación del DAO para la clase de negocio OrdenCarga.

```
public class HibernateOrdenCargaDao implements OrdenCargaDao {
    public HibernateOrdenCargaDao() {}
    . . .
    private HibernateTemplate hibernateTemplate;
    public void setHibernateTemplate(HibernateTemplate template) {
        this.hibernateTemplate = template;
    }
}
```

A efectos de que se realice la inyección de dependencia del `HibernateTemplate`, se debe registrar en la configuración de Spring como sigue:

```
<bean id="ordenCargaDao"
      class="com.libroIS.dao.hibernate.HibernateOrdenCargaDao">
    <property name="hibernateTemplate" ref="hibernateTemplate" />
</bean>
```

La persistencia es administrada a través del componente inyectado, como se muestra en el listado que sigue:

```
public void saveOrdenCarga(OrdenCarga ordenCarga) {
    hibernateTemplate.saveOrUpdate(ordenCarga);
}
```

La búsqueda de las órdenes de carga por fecha y puerto de destino se realiza como se muestra a continuación:

```
public OrdenCarga findOrdenCargaFechaPuerto(Date fecha, String puerto) {

    List results = hibernateTemplate.find("from " + ORDEN_CARGA +
        " where fechaPartida = ? and puertoDestino = ?",
        new Object[] {fecha, puerto});

    return results.size() > 0 ? (OrdenCarga) results.get(0) : null;
}
```

La búsqueda por identificador de un único objeto se realiza así:

```
public OrdenCarga find(Integer id) {
    return (OrdenCarga) hibernateTemplate.load(OrdenCarga.class, id);
}
```

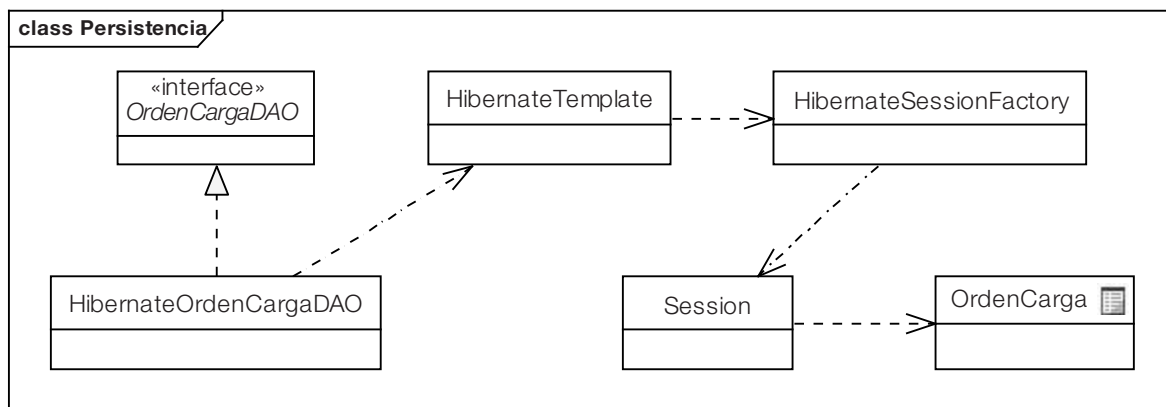


Figura 14.4. Relación entre las clases de persistencia y la plataforma Spring

La administración de las transacciones será delegada por la plataforma Spring a Hibernate a través de la clase `HibernateTransactionManager` que funciona como un patrón de diseño de tipo *facade* [7]. Para esto debe registrarse en la configuración lo siguiente:

```

<bean id="transactionManager"
  class="org.springframework.orm.hibernate3.HibernateTransactionManager">
  <property name="sessionFactory" ref="sessionFactory"/>
</bean>

```

Esta clase realiza la administración de los `commit` y `rollback` a partir de delegaciones a la clase `org.hibernate.Transaction`.

Las transacciones son definidas en forma declarativa y administradas por la propiedad AOP (aspect oriented programming) de la plataforma Spring [9]. Se definió para la mayor parte de ellas un nivel de aislación "read committed" y una política de ejecución entre "requires new", "required" y "supports" [9], lo que significa que ante conflictos por accesos concurrentes las diferentes transacciones no son visibles hasta después de realizado el correspondiente `commit` y que los métodos a ejecutarse, algunos lo harán en una transacción nueva, otros en una nueva o ya abierta mientras que otros lo harán sin importar si hay una transacción abierta o no.

14.2.5 Conexión con sistemas externos

Como analizamos en capítulos anteriores, cuando la capacidad de carga de la empresa se ve saturada, las órdenes nuevas deben tercerizarse. A efectos de integrar el sistema en desarrollo con el sistema del proveedor actual se cuenta con una API entregada por esta otra empresa a efectos de facilitar el desarrollo del software de integración. Debido a la imposibilidad de modificar el sistema de transporte de este tercero se decidió integrarlo utilizando el modo propuesto por las interfaces declaradas en la API. La conectividad con dicho sistema será a través de un patrón *broker* implementado sobre el protocolo RMI (*remote method invocation*). En la figura 14.5 se muestra el diseño, el cual no incluye detalles de la conectividad. Los objetos del lado del sistema externos son remotos. Se decidió asilar dicho sistema detrás de un patrón de tipo *facade* implementado por la clase `TercerizaServiciosTransporte`. Esta clase define el protocolo (conjunto de métodos) a través del cual el

sistema en desarrollo solicitará servicios al sistema externo. Debido a un desajuste entre ambos sistemas, se utiliza un patrón de tipo *adapter* [7] para adaptar a ambos el cual está implementado por la clase Adaptador-ServicioTransportetercero cuyas instancias resuelven las solicitudes adaptando la interfaz del sistema externo al del *facade* del sistema en desarrollo.

14.2.6 Web Service

Como se analizó en el capítulo anterior (figura 13.10) es necesario que la aplicación que administre a los clientes exporte el servicio de búsqueda de clientes. Este servicio estará centrado en una interfaz que llamaremos *Cliente* y cuya definición se muestra en el listado que sigue en formato WSDL (*web service definition language*) [11]. Puede verse que a partir del nombre o número de cliente se obtienen sus datos.

```
<?xml version="1.0" encoding="UTF-8"?>
<wsdl:definitions targetNamespace="http://onjava"
  xmlns:apachesoap="http://xml.apache.org/xml-soap"
  xmlns:onjava="http://onjava"
  xmlns:intf="http://onjava"
  xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
  xmlns:wsdlsoap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <!--WSDL created by Apache Axis version: 1.4 Built on Apr 22, 2006 (06:55:48 PDT)-->
```

<!-- Definición de tipos, pueden trasladarse a un xsd -->

```
<wsdl:types>
  <schema elementFormDefault="qualified" targetNamespace="http://
    onjava" xmlns="http://www.w3.org/2001/XMLSchema">

    <element name="buscaCliente">
      <complexType>
        <sequence>
          <element name="numero" type="xsd:string"/>
          <element name="nombre" type="xsd:string"/>
        </sequence>
      </complexType>
    </element>
    <element name="buscaClienteResponse">
      <complexType>
        <sequence>
          <element name="buscaClienteReturn"
            type="impl:DatosCliente"/>
        </sequence>
      </complexType>
    </element>
  </schema>
</wsdl:types>
```

<!-- Definición de la estructura de los mensajes -->

```

    <wsdl:message name="buscaClienteRequest">
    <wsdl:part element="impl:buscaCliente" name="parameters"/>
    </wsdl:message>
    <wsdl:message name="buscaClienteResponse">

    <wsdl:part element="impl:buscaClienteResponse" name="parameters"/>
    </wsdl:message>

```

<!-- Definición de la interfaz del servicio -->

```

    <wsdl:portType name="Cliente">
    <wsdl:operation name="buscaCliente">
    <wsdl:input message="impl:buscaClienteRequest"
        name="buscaClienteRequest"/>
    <wsdl:output message="impl:buscaClienteResponse"
        name="buscaClienteResponse"/>
    </wsdl:operation>
    </wsdl:portType>

```

<!-- Definición del vínculo con el protocolo -->

```

    <wsdl:binding name="ClienteSoapBinding" type="impl:Cliente">
    <wsdlsoap:binding style="document"
        transport="http://schemas.xmlsoap.org/soap/http"/>
    <wsdl:operation name="buscaCliente">
    <wsdlsoap:operation soapAction=""/>
    <wsdl:input name="buscaClienteRequest">
    <wsdlsoap:body use="literal"/>
    </wsdl:input>
    <wsdl:output name="buscaClienteResponse">
    <wsdlsoap:body use="literal"/>
    </wsdl:output>
    </wsdl:operation>
    </wsdl:binding>

```

<!-- Definición de la implementación del servicio -->

```

    <wsdl:service name="ClienteService">
    <wsdl:port binding="impl:ClienteSoapBinding" name="Cliente">
    <wsdlsoap:address location="http://www.empresa.com:8080/WS_Cliente/services/Cliente"/>
    </wsdl:port>
    </wsdl:service>
</wsdl:definition>

```

Estos servicios básicos serán implementados en la capa de acceso al negocio de la aplicación *enterprise* de administración de clientes.

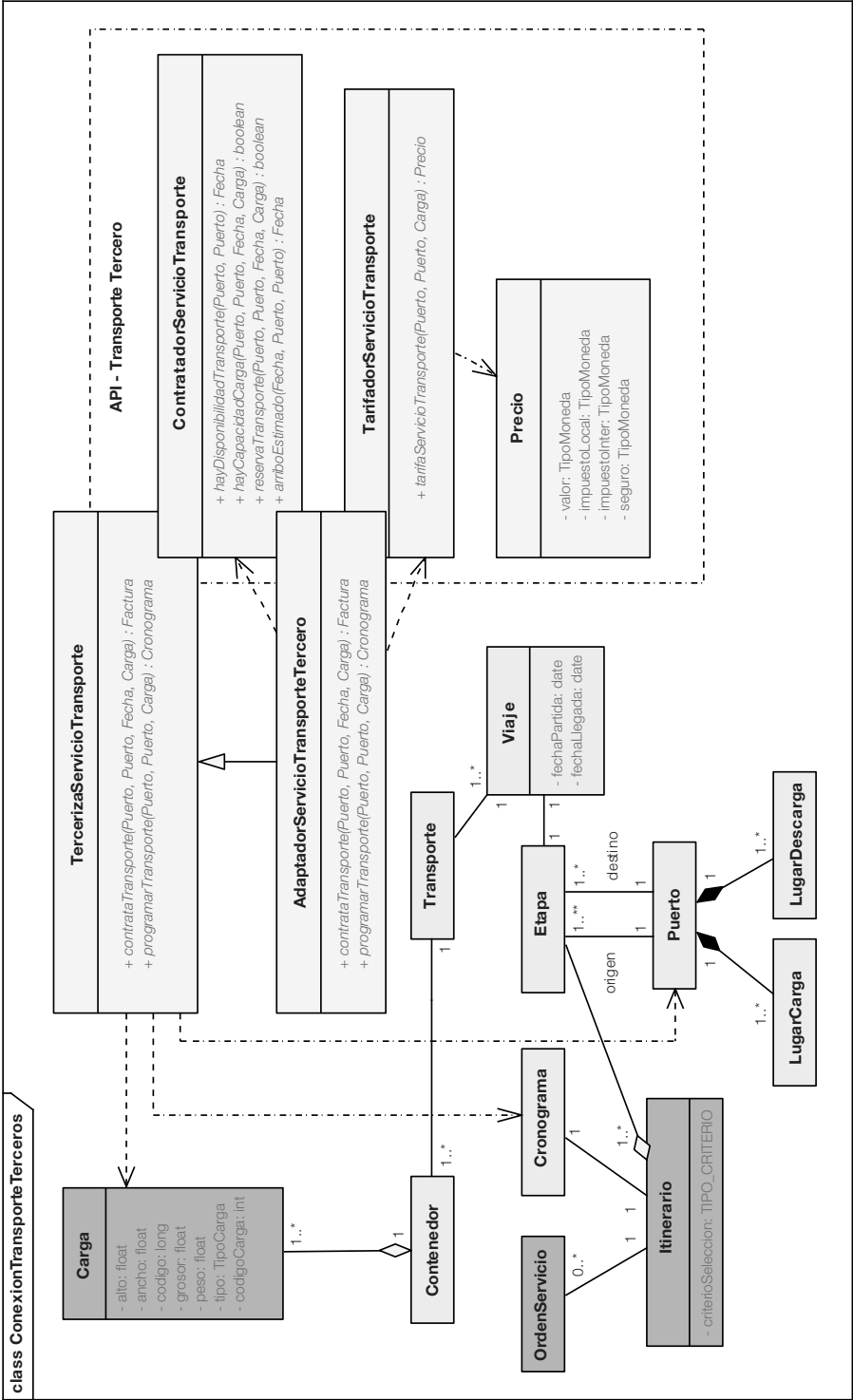


Figura 14.5. Diagrama de clases de la integración con el API del sistema de transporte con terceros

14.3 Conclusión

Hemos presentado algunas de las partes representativas del diseño del sistema en desarrollo a efectos de mostrar cómo impactan los criterios de buen diseño en la calidad de los productos que se generan.

Además, a partir de lo expuesto a lo largo del capítulo en relación con el proyecto, podemos concluir que debido a que las tecnologías modernas implican un gran trabajo de configuración de las diferentes plataformas y herramientas de desarrollo, el trabajo en grupo y el uso de diversas plataformas hacen que el volumen de código generado sea importante, se debe administrar el proyecto en sus aspectos técnicos:

- estandarizando el estilo de codificación;
- implementando pruebas sistemáticas y automáticas;
- contando con herramientas potentes de administración del ambiente de desarrollo.

En próximos capítulos presentaremos cómo realizar estas actividades que constituyen otro capítulo de la Ingeniería de Software.

14.4 Contenido de la página web de apoyo



14.4.1 Mapa conceptual

14.4.2 Autoevaluaciones

14.4.3 Presentaciones en Power Point (exclusivo para el docente)

14.5 Referencias

- [1] *Five Considerations for Software Architecture, Part 1 y 2*, Buschmann, Kevlin Henney, IEEE Software, June 2010.
- [2] *Principles in Practice*, Rebecca J. Wirfs-Brock, IEEE Software, July/August 2009.
- [3] *Agile Principles, Patterns, and Practices in C#*, Martin C. Robert, Martin Micah, Prentice Hall, 2006.
- [4] *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Eric Evans, Addison-Wesley, 2006.
- [5] *Refactoring: Improving the Design of Existing Code*, Martin Fowler, Kent Beck, John Brant, and William Opdyke, Addison-Wesley, 1999.
- [6] *Patterns of Enterprise Application Architecture*, Martin Fowler, Addison-Wesley, 2003.
- [7] *Design Patterns: Elements of Reusable Object-Oriented Software*, Erich Gamma, Richard Helm, Ralph Johnson, and John M. Vlissides, Addison-Wesley, 1994.
- [8] *Core J2EE™ Patterns: Best Practices and Design Strategies, Second Edition*, Deepak Alur, John Crupi, Dan Malks, Prentice Hall PTR, 2003.
- [9] *Spring in action, 2th edition*, Craig Walls, Ryan Breidenbach, Manning Publications, 2008.
- [10] *Struts in action*, Donald Brown, Chad Michael Davis, Scott Stanlick, Manning Publications, 2008.
- [11] *SOA in Practice: The Art of Distributed System Design (Theory in Practice)*, Nicolai M. Josuttis, O'Reilly, 2007.

Métricas de Software

Contenido

15.1 El proceso de evaluación	240
15.2 Primera aproximación al diseño de un sistema	240
15.3 Adentrándonos en los subsistemas	245
15.4 Buenas prácticas de codificación	251
15.5 Conclusión	257
15.6 Contenido de la página web de apoyo	257
15.7 Referencias	258

Objetivos

- Aprender los conceptos y prácticas de evaluación de software
- Aprender técnicas de construcción de indicadores para las métricas
- Conocer algunas herramientas para la obtención de métricas
- Entender la importancia de las revisiones de diseño y código como aseguramiento de la calidad de un sistema en desarrollo

Competencias

- Determinar a partir de métricas los atributos de calidad del diseño y código del software de un proyecto a efectos de mejorar su nivel de calidad

15.1 El proceso de evaluación

Hay diferentes escenarios en los cuales se realiza una evaluación del diseño y código del software de un sistema. Uno de ellos es durante su desarrollo. En esta instancia la motivación es generar un producto de calidad que cumpla con los requerimientos preestablecidos en el proyecto. Hay otro escenario que con frecuencia debemos transitar los ingenieros y es el mantenimiento de sistemas en cuyo desarrollo no hemos participado, en los cuales los desarrolladores ya se mudaron de empleo y no existe documentación más allá del código fuente. En este caso la motivación es estar en condiciones de soportar al sistema haciendo mantenimiento evolutivo, es decir corregir fallas que los usuarios detecten y extender el sistema con nuevas funcionalidades.

15.1.1 Construyendo un sistema

Para proyectos grandes, con un volumen de código importante, es necesario rever y realizar un seguimiento de las decisiones de diseño a lo largo del ciclo de vida del proyecto. Cuando se trabaja con código compartido, tema que trataremos en un próximo capítulo, y además cuando el tiempo pasa y el trabajo se dilata existe la tendencia a resolver diferentes aspectos del desarrollo no siempre alineados con las definiciones del inicio. Entonces es importante la realización de revisiones de diseño y código a lo largo del ciclo de vida completo que permitan reforzar las definiciones de la arquitectura. Estas evaluaciones necesitan de un criterio de selección de los artefactos a revisar ya que sería imposible e improductivo revisar todo el desarrollo [3].

15.1.2 Manteniendo un sistema

Cuando en muchas situaciones algunos desarrolladores son contratados para hacer mantenimiento de un sistema de cuyo desarrollo no han participado y no cuentan con más documentación que el código fuente encaran su trabajo a fuerza bruta. Es decir, navegan el código buscando entender a partir de la funcionalidad explicada por los usuarios los criterios con que fue construido el software del sistema. Esta tarea puede ser muy complicada, lenta y casi siempre infructuosa [2]. Es necesario elaborar un criterio para abordar las diferentes actividades que conduzcan a una comprensión que permita dicho mantenimiento. Existen herramientas que ayudan a desarrollar este criterio que trataremos en este capítulo [1].

15.2 Primera aproximación al diseño de un sistema

Una primera aproximación a la estructura de un sistema está dada por los paquetes que conforman los distintos componentes. Entre los criterios de buen diseño listados en el capítulo 14 incluimos la Inversión en la Cadena de Dependencia [5]. Este criterio establece que los módulos generales, aquellos que definen la arquitectura, deben ser abstractos. Entendemos por *abstractos* elaborados a partir de interfaces y / o clases abstractas. Los módulos concretos, aquellos que encapsulen la implementación, deben estar contruidos por clases concretas, según las definiciones de los módulos abstractos. La distribución de los paquetes idealmente estaría en torno a una recta como se muestra en la figura 15.1 en línea llena.

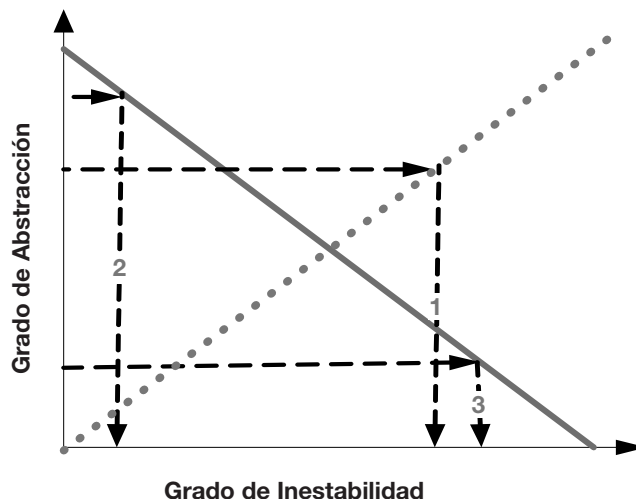


Figura 15.1. Abstracción vs Inestabilidad

En la figura además se muestra mediante una línea de bolitas la distribución típica de los módulos de un sistema construido con algún lenguaje procedural. Debido al pobre encapsulamiento de estos lenguajes comparados con los lenguajes orientados a objetos se aprecia la diferencia en las curvas. Justamente esta inversión es posible gracias al encapsulamiento y al polimorfismo propio de estos últimos lenguajes.

Con las coordenadas número 1 se muestra que, en los desarrollos con menor encapsulamiento, los módulos generales son inestables. Mientras que con las coordenadas número 2 se muestra que al estar invertida la cadena de dependencias, los módulos generales son poco inestables, es decir son estables. Abstracción y generalidad son la misma cosa desde el objetivo buscado, lo mismo que inestabilidad y dependencia.

¿Cuáles son los beneficios logrados a partir de este comportamiento? La estabilidad en la arquitectura definida. Propiedad que no gozan los desarrollos con lenguajes procedurales, ya que la propagación de cambios desde los módulos inestables alcanza a los generales, lo que hace que rara vez los sistemas desarrollados así terminen con la misma arquitectura originalmente definida. Esto es muy importante porque tiene implicancias fundamentales. Los sistemas que posean esta propiedad podrán ser optimizados, cambiados y extendidos con menor esfuerzo y sin propagación de cambios a otras partes del software.

En la figura 15.2 se presenta un diagrama UML conceptual de clases con el fin de ejemplificar un patrón típico en el cual se cumple la Inversión en la Cadena de Dependencia.

Puede verse que este concepto es trasladable a las clases, es decir que al igual que las relaciones entre los paquetes generales y dependientes, también se puede caracterizar a la relación entre clases de la misma forma. En este gráfico observamos claramente que, a partir de definiciones de interfaces estables de las abstracciones, los cambios en las clases dependientes no afectan el nivel de abstracciones el cual define a la arquitectura de los sistemas y da lugar a estructuras estables, como se describió al analizar la figura 15.1.

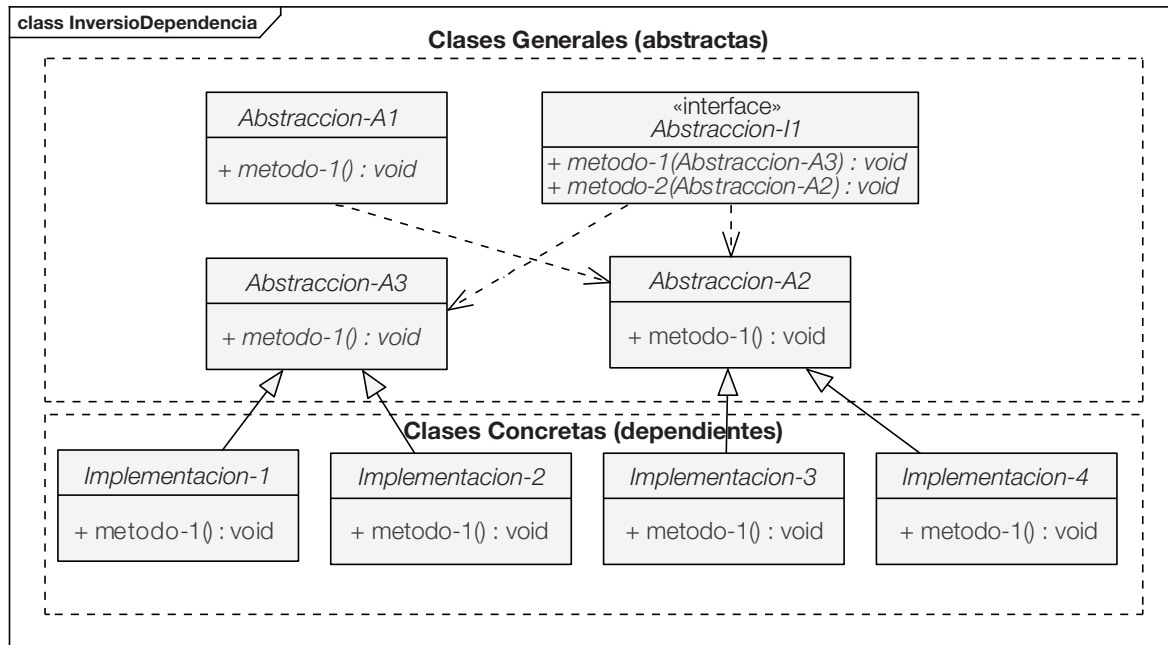


Figura 15.2. Diagrama UML conceptual de un software con la propiedad de Inversión de Dependencia

Esta propiedad puede ser medida en los paquetes de la siguiente forma [4]:

$$GA = CCA / CTC$$

$$GI = CRS / CTR$$

donde

GR: Grado de Abstracción

CCA: Cantidad de Clases Abstractas

CTC: Cantidad Total de Clases (abstractas + concretas)

GI: Grado de Inestabilidad

CRS: Cantidad de Relaciones Salientes

CTR: Cantidad Total de Relaciones (entrantes + salientes).

De esta forma podemos medir la distancia a la recta de los diferentes módulos de un sistema y detectar así cuáles módulos son candidatos a ser estudiados y refactorizados para facilitar la extensión y / o cambios sin propagaciones.

En la figura 15.2.a se muestra la gráfica para la distribución de los paquetes de la aplicación de Administración de Cargas del caso testigo en desarrollo. En la figura 15.2.b se graficó la distancia a la recta. Puede observarse que dos de los módulos se encuentran a una distancia de la recta que denotan indefinición, es decir no son abstractos ni inestables. De esta forma detectamos artefactos a revisar del sistema en desarrollo. Si este fuera un sistema del cual no poseemos mayor documentación, el hallazgo es igualmente importante ya que nos dará pautas acerca del diseño subyacente y la forma de cambiarlo.

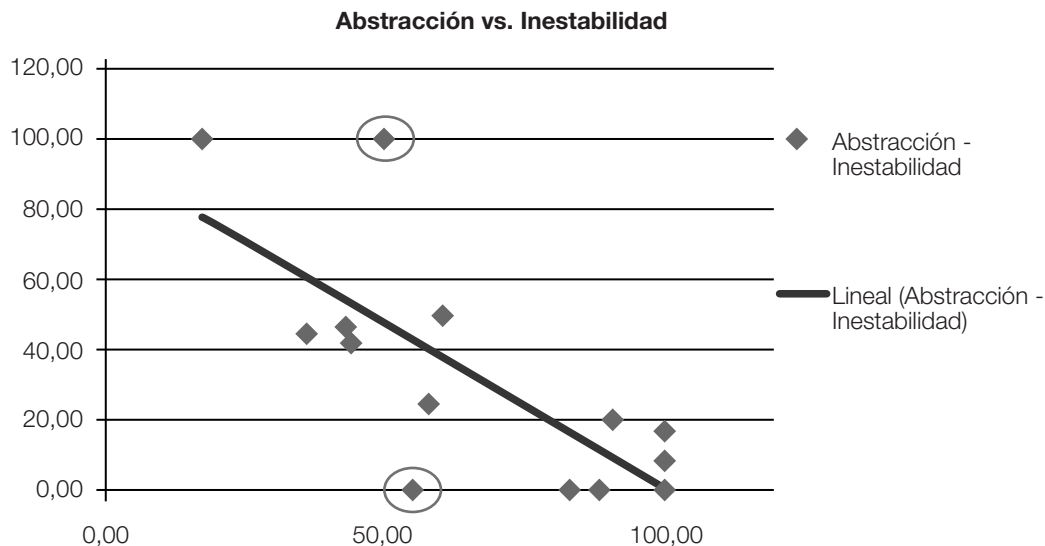


Figura 15.2.a. Distribución de los módulos de la aplicación Administrador de Cargas en torno a la recta de Inversión de Dependencia

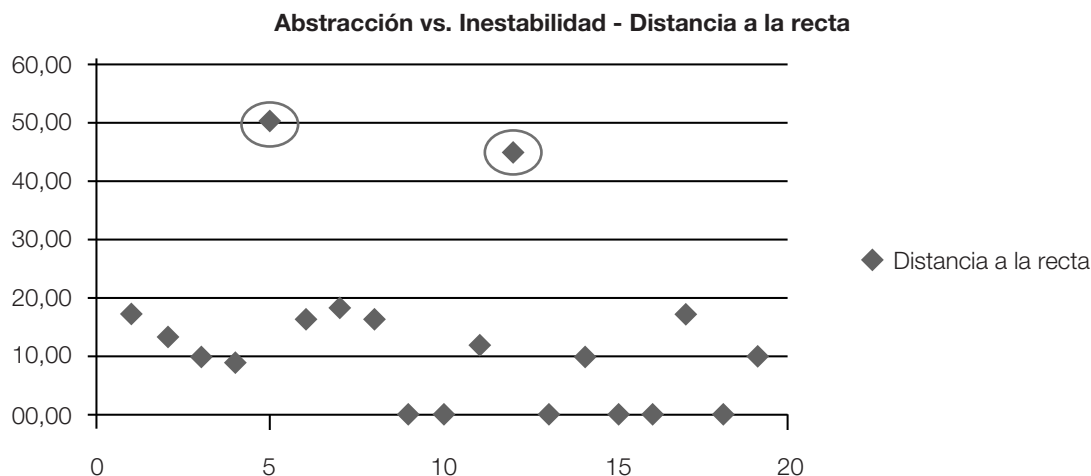


Figura 15.2.b. Distancia a la recta de la distribución de cada módulo

Luego de revisar y analizar estos módulos se concluyó que en un caso la falta de abstracción se debía a un mal diseño de las clases que definían los tipos de datos asociados a un patrón de diseño denominado DTO (*data transfer object*). Estas construcciones se utilizan para desacoplar la capa de presentación de la del negocio y consiste en un contenedor de datos convertidos a un tipo genérico que ambas entiendan, por ejemplo “strings”. Estas clases se habían definido para cada clase cuyos datos necesitaban llegar a la presentación sin ninguna jerarquía ni generalización en su concepción. Fueron reemplazados por una jerarquía de clases donde se generalizó su construcción a partir de la utilización de diccionario de datos (*Map*). El otro caso se debió al mal modelado de las reglas de negocio a validar sobre clases del dominio. Para este caso se refactorizó el

diseño a partir de la implementación de dichas reglas con un patrón de diseño de tipo “strategy”, y se separó así las abstracciones de las implementaciones.

Con ambos cambios se logró que el sistema esté dentro del margen deseado que permita su extensión y su posible posterior optimización sin ninguna propagación y se dejó en condiciones al software del sistema para realizar un posterior mantenimiento más ordenado y eficiente. Una herramienta que ayuda al cálculo de los índices anteriores en desarrollos con lenguaje de programación Java es JDepend [7].

Otro de los criterios de buen diseño que sirve de guía en esta primera aproximación y que de no cumplirse seguramente será un indicativo de problemas es la no existencia de dependencias cíclicas. Las dependencias cíclicas entre paquetes son un signo de falta de independencia de módulos, imposibilidad de probarlos por separado, de reutilizarlos en otros contextos, etc. Si bien su detección no es compleja, en sistemas de gran volumen es una tarea que se vuelve tediosa. Existe una herramienta conocida como inCode [6] que permite, a partir de un complemento para el ambiente de desarrollo Eclipse, detectar estas anomalías en un proyecto. Para la refactorización de estas dependencias cíclicas se debe recurrir por lo general a la incorporación de interfaces a partir de las cuales se logra desacoplar los paquetes con relaciones indeseadas. En la figura 15.3 se muestra la detección de una de estas relaciones para el sistema testigo en desarrollo. Estas métricas fueron obtenidas con un *plugin* de Eclipse desarrollado por Jacopo Malnati [14].

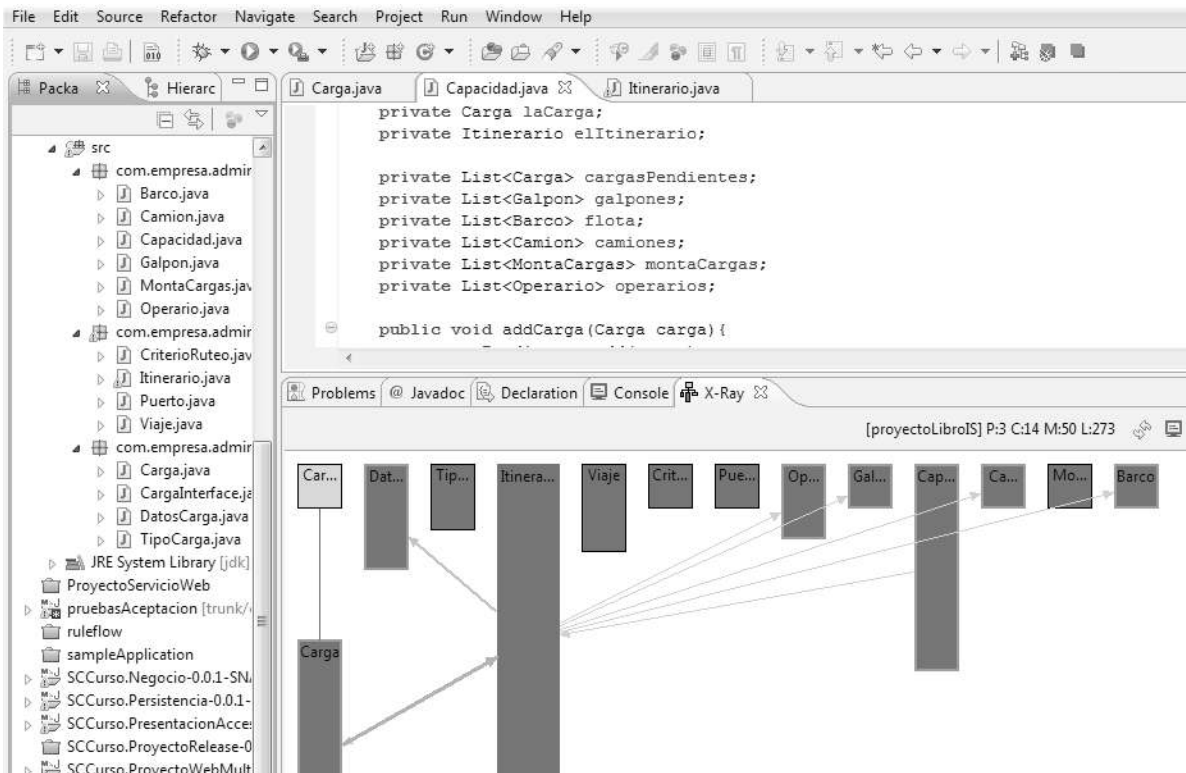


Figura 15.3. Vista de las relaciones circulares entre clases de diferentes paquetes del software de la aplicación Administrador de Cargas

A partir de estos criterios de buen diseño hemos logrado una primera aproximación para mejorar un sistema en desarrollo o entender un sistema a mantener. Una vez que se detectaron las debilidades anteriores y se hayan resuelto tendremos un mejor software y una mayor comprensión de su diseño. Estamos en condiciones de preguntarnos por las clases que conforman los paquetes analizados a efectos de continuar en el camino de mejorar el sistema de que se trate.

15.3 Adentrándonos en los subsistemas

Cuando se utiliza la palabra métrica se está tentado a pensar en números obtenidos a partir de algoritmos complejos alimentados con mediciones tediosas de obtener. Sin embargo, para caracterizar un diseño es posible hacerlo a partir de unos pocos indicadores, los cuales son muy fáciles de construir. Estos indicadores representan algunas de las características deseables de un diseño, es decir están directamente ligados a los criterios de buen diseño presentados al comienzo del capítulo 14. Con relación a los escenarios analizados anteriormente y siguiendo los pasos orientados a construir un procedimiento de análisis de la calidad de un sistema en desarrollo o de uno a mantener, estos indicadores deben ser interpretados en relación con el contexto donde se utilizan. Cuando avancemos en esta presentación volveremos sobre el tema.

Con las métricas que describiremos a continuación buscamos:

- Caracterizar el diseño de sistemas utilizando programación orientada a objetos.
- Evaluar y mejorar el diseño de sistemas.
- Guiar y conducir revisiones de diseño y código como mecanismo de garantía de calidad.

15.3.1 Métricas

En relación con las métricas que propondremos los seguiremos a Michele Lanza y Radu Marinescu [1]. Se necesitan las siguientes definiciones para medir las fortalezas y debilidades de un diseño:

1. Un indicador
2. Una escala
3. Una herramienta para medir (medidor).

Además, clasificaremos los indicadores según califiquen:

- Tamaño y complejidad
- Uso del mecanismo de herencia
- Grado de acoplamiento.

15.3.1.1 Indicadores

Entendemos por *indicador* un número medido o calculado a partir de una medida directa que represente y ponga de manifiesto una característica o un comportamiento determinado del proceso observado.

Tamaño y complejidad

Entre los indicadores utilizados como indicadores de tamaño y complejidad de un software podemos citar a:

- NOP: cantidad de paquetes o *namespaces* en el sistema.
- NOC: cantidad de clases en el sistema.
- NOM: cantidad de métodos por clase.
- LOC: cantidad de líneas de código por método.
- CYCLO: cantidad de decisiones por líneas de código (Complejidad Ciclomática).
- WMC: CYCLO / Métodos por LOC / Método por NOM / clase.
- AMW: CYCLO / Método (average method weight).

Mecanismo de Herencia

Entre los indicadores del uso del mecanismo de herencia en un software podemos citar a:

- ANDC: Cantidad promedio de clases derivadas. Solo se contabilizan clases propias, no de librerías. No se contabilizan interfaces.
 $ANDC = \text{sumatoria de (cantidad de clases por cantidad de herencias por clase)} / NOC.$
- AHH: Profundidad promedio de las jerarquías de herencia.
 $AHH = \text{sumatoria (cantidad de clases "root" por cantidad de niveles de herencia)} / NOC.$

Acoplamiento

Entre los indicadores del acoplamiento de un software podemos citar a:

- CALLS: cantidad de invocaciones a cualquier método de una clase desde diferentes métodos de otras.
- FANOUT: cantidad de clases dependientes (uso, invocaciones).

Estos indicadores serán explicados al aplicarlos a un caso específico en la próxima sección.

15.3.1.2 Escalas

Ya hemos definido los indicadores, con lo cual podemos medir las magnitudes necesarias en el diseño a evaluar. Sin embargo resta definir contra qué valores compararemos estas medidas. Cuáles serán nuestras referencias para calificar a una medida baja, normal o alta.

Para eso en *Object-Oriented Metrics in Practice - Using Software Metrics to Characterize, Evaluate, and Improve the Design of Object-Oriented Systems* [1] se recurrió a una cantidad de proyectos de Java y C++ para utilizarlos como referencia y darle peso estadístico a las medidas que se obtengan. Es decir que los números de los indicadores de un proyecto cualquiera serán comparados con las estadísticas de la tabla 15.1 que se muestra más abajo.

- Distribución Normal
- Valor esperado, promedio E
- Desviación estándar Sigma , 68% de las observaciones en E +- Sigma
- Outlier [E +- Sigma] * 1.5 -> 50% > [E +- S]

Tabla 15.1. Estadísticas obtenidas de 45 proyectos Java y 37 C++, adaptada de [1], donde S = Sigma						
Métrica	Java			C++		
	- S	Promedio	+ S	- S	Promedio	+ S
CYCLO / LOC	0.16	0.20	0.24	0.20	0.25	0.30
LOC / NOM	7	10	13	5	10	16
NOM / NOC	4	7	10	4	9	15
CALLS / NOM	2.01	2.62	3.2	1.17	1.58	2
FANOUT / CALLS	0.56	0.62	0.68	0.20	0.34	0.48
ANDC	0.25	0.41	0.57	0.19	0.28	0.37
AHH	0.09	0.21	0.32	0.05	0.13	0.21

15.3.1.3 Medidor

Utilizaremos como medidor una herramienta llamada inCode [6] la cual ofrece la posibilidad de integración con Eclipse. Estas mediciones se agrupan y presentan en una pirámide que resume los resultados, como se muestra en la figura 15.4.a. En la figura 15.4.b se muestra el resultado de usarla sobre un paquete de la aplicación Administrador de Cargas del sistema del caso testigo.

Interpretación de la Complejidad

CYCLO / LOC = 0.10 evidencia que la complejidad es baja en relación al umbral.
LOC / NOM = 23.0 evidencia que el tamaño de los métodos es grande en relación al umbral.
NOM / NOC = 2.0 evidencia que el tamaño de las clases es bajo en relación al umbral.
NOC/NOP = 3.0 evidencia que la cantidad de clases por paquete es bajo en relación con el umbral.

Interpretación del Acoplamiento

CALLS / NOM: 20.0 evidencia gran acoplamiento.
FANOUT / CALL = 0.34 evidencia que el acoplamiento está concentrado en unas pocas clases.

Interpretación de la Herencia

ANDC = 0.00 muestra uso bajo de herencia.
AHH = 0.00 indica que las jerarquías no son profundas sino horizontales.

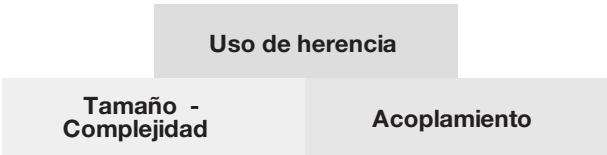
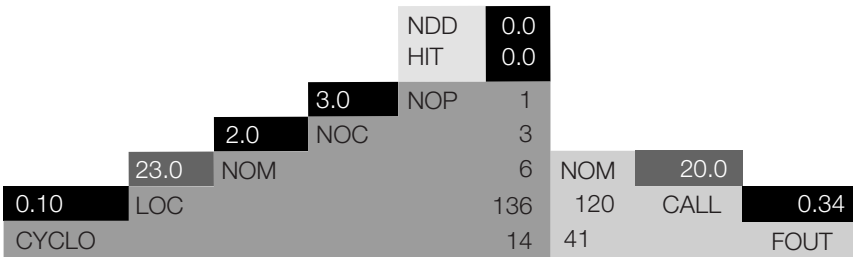


Figura 15.4.a. Pirámide resumen de las métricas definidas, adaptada de [6]



Interpretation of the Overview Pyramid for module serviciosApplication.dto

Class Hierarchies tend to be rather **sparse** (i.e. there are mostly standalone classes and few inheritance relations)

Classes tend to:

- be rather **small** (i.e. have only a few methods);
- be organized in rather **fine-grained packages** (i.e. few classes per package);

Methods tend to:

- be rather **long** yet having a rather **simple logic** (i.e. few conditional branches);
- call **many methods** (high coupling intensity) from **few other classes** (low coupling dispersion);

More details on the Overview Pyramid can be found in the Object-Oriented Metrics in Practice book by contacting us at incode@intooitus.com. For any questions or suggestions, please visit our web site and forum.

Figura 15.4.b. Métricas para el módulo de servicios de aplicación de la aplicación Administrador de Cargas y sus DTO (data transfer objects)

Puede observarse, a partir de los valores obtenidos, la posibilidad de revisión guiada por estos a efectos de refactorizar el diseño y así mejorar la calidad del software en cuestión. De otra forma la revisión sería realizada a ciegas y quizás este paquete nunca hubiese sido seleccionado como candidato a mejorar.

15.3.2 Polimétricas

Uno de los inconvenientes que se busca resolver es la visualización rápida y clara de las métricas obtenidas. Con este fin se ha buscado la forma de representación de los valores citados. Uno de ellos ha sido llamado *Polimétricas* [1] y consiste en una representación gráfica, como se muestra en la figura 15.5.

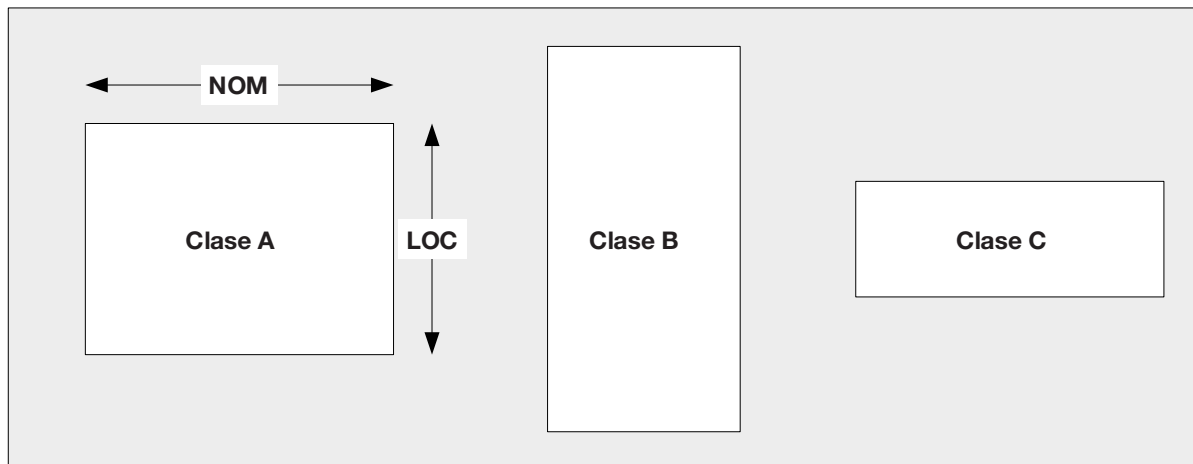


Figura 15.5. Representación visual de clases y sus métodos mediante Polimétricas

Puede verse que el ancho de los rectángulos que representa a una clase es proporcional a la cantidad de métodos de esta, mientras que el alto lo es a la cantidad de líneas de código de dichos métodos. Por lo tanto la clase de la figura nombrada como B posee una cantidad de métodos menor que la nombrada como C, sin embargo los métodos de B son mucho más largos. Esta caracterización busca facilitar el análisis de código de proyectos cuyo código es de gran volumen. Además se puede agregar a esta representación tramas, como se muestra en la figura 15.6. En este caso se presenta un grupo de clases del proyecto de la aplicación Administrador de Cargas utilizando inCode. Las clases pintadas de rayas no presentan problemas, las pintadas de puntos presentan algún problema de diseño a nivel de sus métodos y las pintadas de rayas diagonales tienen dificultades de diseño a nivel de la clase.

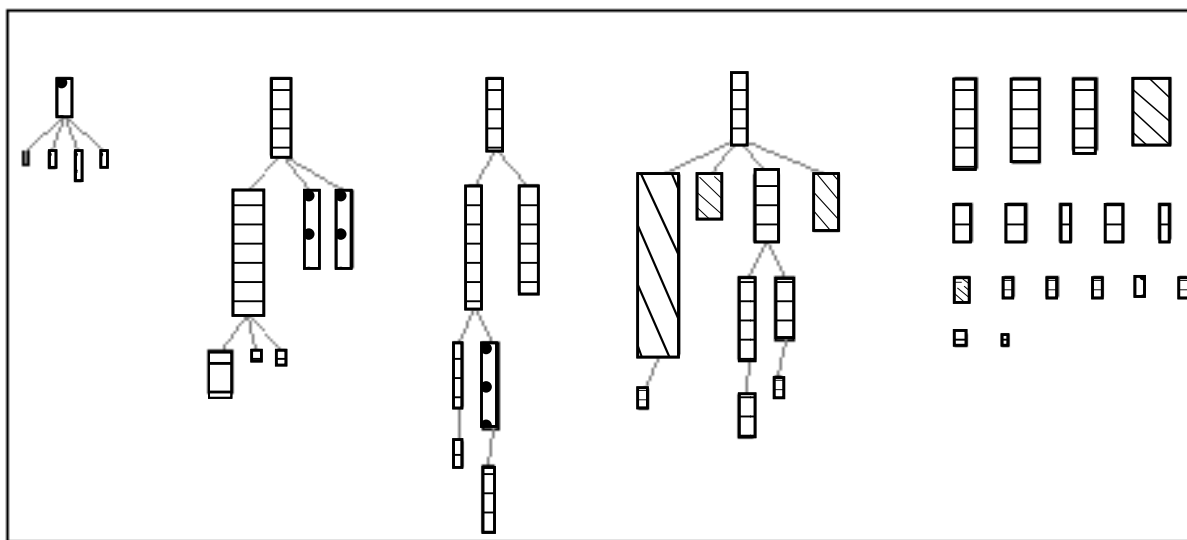


Figura 15.6. Polimétrica de algunas clases del proyecto de la aplicación Administrador de Cargas obtenida con inCode

La detección de estos posibles problemas de diseño y los criterios de clasificación de estos son denominados *desarmonías* y se presentan en la próxima sección.

15.3.3 Desarmonías

Si se logra cumplir con los criterios de buen diseño, cada componente dentro de un sistema estará en armonía con el resto. Por esta razón a los no cumplimientos se los llama *desarmonías*.

A efectos de calificar las violaciones a los criterios de buen diseño analizamos las posibles causas de estas fallas y en función de estas haremos la clasificación [1]. Los criterios de buen diseño promueven:

Armonía de Identidad

Armonía entre la clase que modela un concepto y los datos y métodos que operan sobre esos datos.

Armonía en la Colaboración

Armonía en la forma en que colabora cada clase con el resto a efectos de lograr la funcionalidad requerida.

Armonía en la Clasificación

Cómo cada clase hace uso de los métodos heredados y los propios. Cuál es el comportamiento de cada clase en la jerarquía de herencia a la cual pertenece.

Utilizando estas pautas calificamos sus violaciones y las clasificamos.

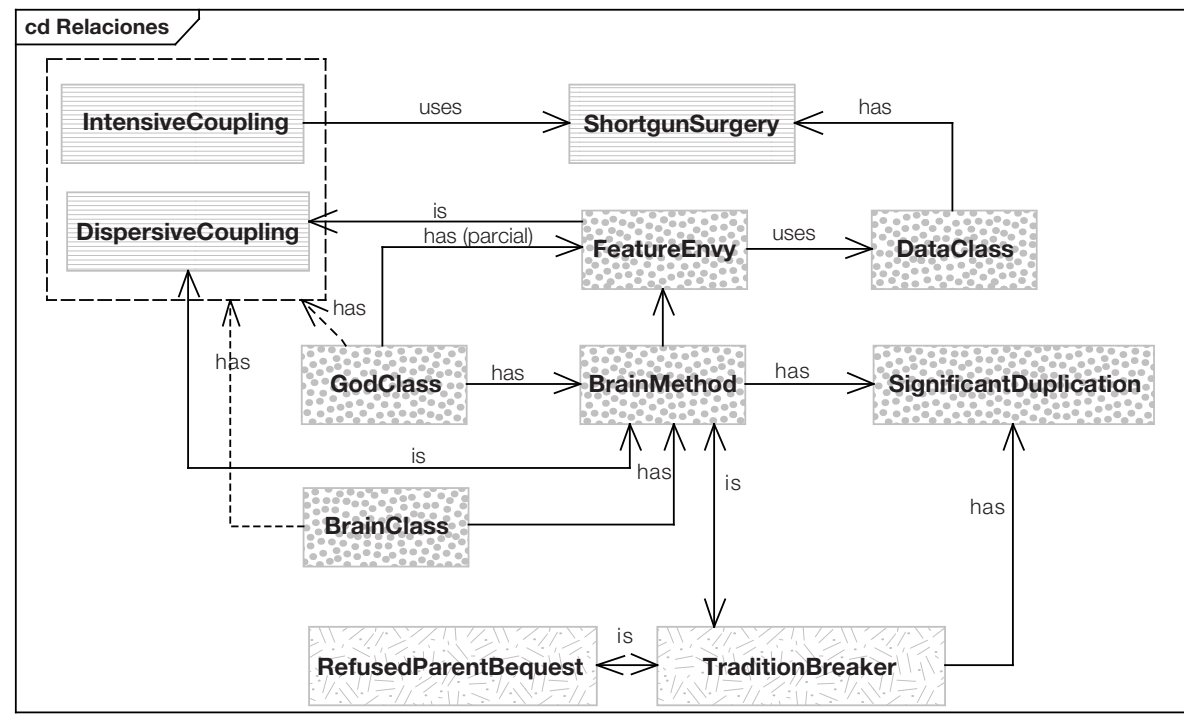


Figura 15.7. Desarmonías de diseño clasificadas según las pautas que violan

En la figura 15.7 se muestran las desarmonías agrupadas y pintadas de diferentes tramas. Las de Identidad aparecen con trama de puntos, las de Colaboración con rayas horizontales y las de Clasificación con trama de cruces. Las estrategias de detección de estas desarmonías se basan en algoritmos que testean combinaciones de las métricas analizadas contra determinados umbrales [1].

A continuación definimos conceptualmente estas desarmonías.

Desarmonías de Identidad

- Brain Class: clases con métodos sobredimensionados.
- Brain Method: métodos sobredimensionados en tamaño y complejidad.
- God Class: clases que realizan varias tareas no cohesivas.
- Feature Envy: clases que acceden a atributos no propios y hacen que otras se conviertan en proveedores de datos sin funcionalidad.
- Data Class: clases sin funcionalidad.
- Significant Duplication: porciones de código duplicado.

Desarmonías de Colaboración

- Intensive Coupling: alto acoplamiento con una cantidad reducida de clases.
- Dispersive Coupling: alto acoplamiento con una cantidad importante de clases.
- Shotgun Surgery: un método de una clase es invocado por muchos métodos de muchas clases.

Desarmonías de Clasificación

- Refused Parent Bequest: las clases derivadas rechazan el comportamiento heredado y definen nuevos métodos sin redefinir los heredados.
- Tradition Breaker: las clases derivadas extienden la interfaz con nuevos métodos que no caracterizan el concepto modelado por la clase base.

La detección de estas desarmonías constituye también una guía para la realización de revisiones de diseño y código a partir de la selección basada en estas detecciones. La herramienta inCode [6] que venimos utilizando permite la detección de algunas de ellas.

15.4 Buenas prácticas de codificación

Un aspecto interesante respecto de las actividades de la Ingeniería de Software es el establecimiento de estándares. En lo que respecta al código es muy conveniente definir un estándar y apegarse a las pautas que este marca. Este aspecto se ha vuelto muy importante a partir del desarrollo con la modalidad de código compartido que hoy día se utiliza mayoritariamente. En un próximo capítulo nos referiremos a ella, sin embargo por el momento rescatamos la esencia de esta forma de trabajo que está dada por un grupo de desarrolladores que escriben código y lo comparten a través de un repositorio de fuentes. La consecuencia de esto es que cada día todos los desarrolladores trabajan sobre su propio código y sobre el del resto de sus colegas. Por esta razón toma importancia un estándar que despersonalice el código y permita adoptar buenas prácticas de codificación.

15.4.1 Estándares

Existen en Internet diferentes estándares de codificación para distintos lenguajes de programación [9], [10]. Cada grupo de desarrollo debería seleccionar uno, adaptarlo a sus criterios y seguirlo. Estos cubren desde disposición de construcciones, tales como:

Si bien las estructuras de decisión deben tener la siguiente forma:

```
if (condición) {sentencias;} ó
if (condición) {sentencias;} else {sentencias;} ó
if (condición) {sentencias;} else if (condición) {sentencias;} else
{sentencias;}
```

La forma elegida es la siguiente:

```
if (condición){  sentencias;
}
else if (otraCondición){  sentencias;
}
else{  sentencias;
}
```

por su claridad y facilidad de manipulación de las diferentes partes.

Pasan por la asignación de nombres:

Nombrar las variables por la combinación de rol y tipo:

```
Point startingPoint, centralPoint;
Name loginName;
```

así se genera código más expresivo.

Hasta llegar a buenas prácticas, tales como:

Las importaciones se deben hacer explícitamente para cada clase

```
import java.util.List;
import java.util.HashSet;
```

y no `import java.util.*;`
así se evitan inclusiones no necesarias.

Después de seleccionar un estándar y adaptarlo a los criterios propios, los desarrolladores deben adoptarlo como propio de manera que cuando un pedazo de código es inspeccionado no se reconozca al autor.

15.4.2 Chequeos de estilo

Para ayudarnos con las adherencias a los estilos de programación definidos existen herramientas que pueden ser incluidas como parte del ambiente de desarrollo o como auditorías a ejecutar luego de que el código es construido, por ejemplo a continuación de la ejecución de los test.

Rules	Violations	Severity
PackageHtml	19	Error
NewlineAtEndOfFile	19	Error
Translation	0	Error
JavadocMethod	714	Error
JavadocType	78	Error
JavadocVariable	183	Error
JavadocStyle	8	Error
ConstantName	11	Error
LocalFinalVariableName	0	Error
LocalVariableName	16	Error
MemberName	20	Error
MethodName	4	Error
PackageName	0	Error
ParameterName	40	Error
StaticVariableName	9	Error
TypeName	0	Error
AvoidStarImport	14	Error
IllegalImport	0	Error

Figura 15.8.a. Listado de parte de las reglas de chequeo de la herramienta Checkstyle

Violation	Message	Line
*	File does not end with a newline.	0
*	Class XmlHelper should be declared as final.	35
*	Missing a Javadoc comment.	35
*	Missing a Javadoc comment.	38
*	'{' is not preceded with whitespace.	38
*	'{' is not followed by whitespace.	38
*	'}' is not preceded with whitespace.	38
*	Line is longer than 80 characters.	42
*	Missing a Javadoc comment.	42
*	Parameter xmlFileName should be final.	42
*	'{' is not preceded with whitespace.	42
*	'try' is not followed by whitespace.	43
*	'{' is not preceded with whitespace.	43
*	'if' construct must use '{ }'s.	44
*	'catch' is not preceded with whitespace.	48
*	'}' is not followed by whitespace.	48
*	'catch' is not followed by whitespace.	48
*	'{' is not preceded with whitespace.	48
*	'+' is not preceded with whitespace.	49

Figura 15.8.b. Listado de detecciones obtenidas a partir de la ejecución de Checkstyle en modo auditoría

Una de las más utilizadas es Checkstyle [11]. En la figura 15.8.a se muestra un listado de parte de las reglas sobre las que trabaja Checkstyle y en la figura 15.8.b un listado de las detecciones obtenidas para un módulo de un proyecto. Estos informes fueron adquiridos con una herramienta de organización de proyectos que integra varias herramientas llamada *Maven* [12]. Trataremos estas cuando desarrollemos el tema de sistematización de las pruebas de un proyecto.

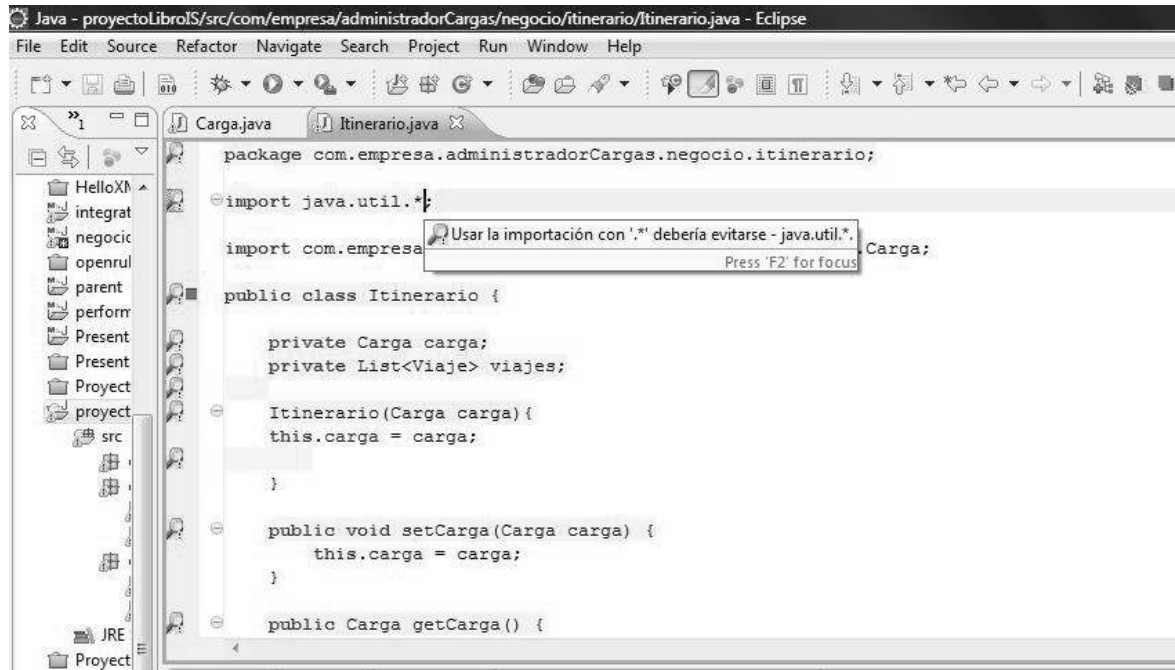


Figura 15.8.c. Ambiente de desarrollo (Eclipse) con herramienta de chequeo de estilo (Checkstyle) integrada

En la figura 15.8.c puede verse un ambiente de desarrollo con Checkstyle integrada a través de un plugin. En este caso las detecciones se realizan mientras el código está siendo construido.

Revisiones de diseño y código

Las revisiones de artefactos asociados a un proyecto de desarrollo de software han mostrado ser el medio más eficiente para la detección y prevención de errores. Algunas estadísticas muestran que del total de errores de un proyecto, el 60% es resuelto con revisiones contra el 40% mediante test [8].

Hay además un valor agregado en estas prácticas y es que brindan la posibilidad de desarrollarlas en forma temprana, de manera que contribuyen a la mejora de la calidad de los productos generados en los proyectos. A diferencia de los test que tradicionalmente son ejecutados una vez construido el código. Cuando en un capítulo próximo tratemos el proceso de pruebas nos volveremos a referir a este tema.

Un aspecto saliente de las revisiones es la selección de los artefactos a revisar. Cuando no se cuenta con la guía proporcionada por las herramientas ya presentadas en este capítulo muchas veces esta selección no es bien realizada. Sin embargo ya contamos con un proceso definido informalmente y la ayuda de las herramientas para elegir los productos a revisar.

1. Objetivos

1. Detectar problemas de diseño y código en forma temprana.
2. Definir y acordar criterios de retrabajo para su resolución.
3. Verificar que se resolvió de acuerdo al criterio acordado.

2. Beneficios

1. Genera datos acerca del producto y el proceso de desarrollo.
2. Genera conocimiento entre miembros del grupo de desarrollo.
3. Aumenta la efectividad de la validación y verificación.
4. Contribuye a la instalación del concepto de calidad.

3. Tipos de revisiones

Las revisiones pueden ser desarrolladas con mayor o menor formalidad. Muchas organizaciones las llevan adelante informalmente con buenos resultados. Otras apelan a una mayor formalidad para generar registros y un seguimiento que permita aprovechar los beneficios derivados de estas prácticas.

15.4.2.1 Formales

Básicamente se caracterizan por contar con roles, responsabilidades y un procedimiento definido.

15.4.2.2 Informales

A diferencia de las anteriores, estas se llevan adelante con roles desdibujados y sin procedimiento.

En la figura 15.9, que sigue, se muestran los roles involucrados en las revisiones formales. Sus responsabilidades son [13]:

- **Moderador:** es la persona que coordina el desarrollo de las sesiones y la intervención de cada uno de los otros roles. Programa la reunión, envía invitaciones, coordina la asistencia de todos, envía el material con los productos a revisar y concentra y almacena los registros. Muchas veces es un miembro del área de calidad.
- **Autor:** es el desarrollador que generó el diseño y / o código a revisar.
- **Inspector:** es el experto que revisa y opina acerca del producto en revisión.
- **Anotador:** es el que se ocupa de registrar los eventos que suceden en las reuniones.
- **Gerente:** es el responsable o un representante del área de desarrollo.
- **Lector:** es el presentador de las diferentes cuestiones (características del proyecto, contexto de ejecución, requerimientos relacionados, producto a revisar) a los asistentes.

Las revisiones formales cuentan con un procedimiento a seguir que se puede resumir en pocas palabras. Los pasos de este procedimiento son:

1. Se programan las sesiones en los proyectos.
2. El moderador releva desde los proyectos el material relacionado a los productos a revisar.

- 3. El moderador organiza las reuniones con fecha, lugar, duración, material a revisar y lista de participantes.
- 4. El moderador envía invitaciones a todos los participantes con alrededor de una semana de antelación. A los revisores les envía el material o la información para que estos puedan acceder al repositorio donde se encuentran los productos a revisar.
- 5. El moderador provee al anotador de las planillas de registro.
- 6. Se desarrolla la reunión con la coordinación de parte del moderador.
- 7. Se compilan en el registro los datos de la sesión.
- 8. Se publican los resultados a los interesados (autor, líder del proyecto, QA).

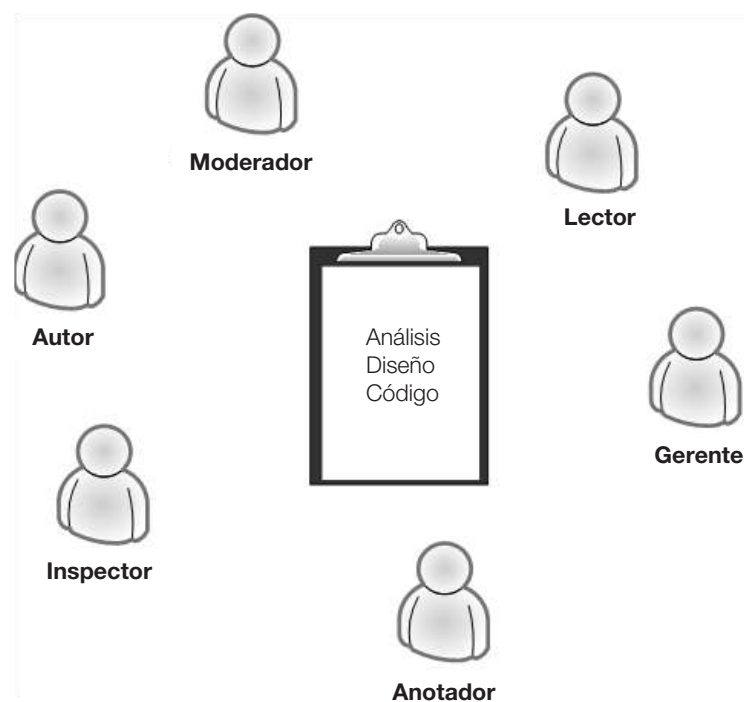


Figura 15.9. Roles involucrados en revisiones formales

A efectos de establecer una clara diferencia entre los tipos de revisiones mostramos en la tabla 15.2 las características salientes de ambos.

Tabla 15.2. Diferencias entre los tipos de revisiones		
Atributo	Formal	Informal
Objetivos	Detectar errores	Detectar errores
	Verificar retrabajo	Discutir alternativas de solución
	Focalizada sobre si o no los productos cubren los requerimientos	Focalizada en demostrar cómo los productos cubren los requerimientos

Tabla 15.2. Diferencias entre los tipos de revisiones		
Atributo	Formal	Informal
Decisiones	Decisiones concensuadas	Decisiones del Autor
Responsable	Moderador entrenado	Autor
Asistentes	Pares con asistencia registrada	Pares y responsables técnicos, sin registrar
Material	Presentado por el Lector	Presentado por el Autor
Métricas	Requeridas	Opcionales
Procedimiento	Formalmente registrado	Informal
Entrenamiento	Requerido para todos los roles	No requerido

15.5 Conclusión

Al momento de evaluar el desarrollo del software de un sistema, ya sea para mejorar el desarrollo presente o para realizar el mantenimiento de un sistema desconocido, es importante seguir un proceso que conduzca el análisis y la resolución de los problemas generados por criterios de buen diseño que se hayan violado.

Debido al volumen de los sistemas actuales es muy importante para que este proceso sea productivo ayudarse con las herramientas adecuadas. Existe una buena cantidad de estas herramientas que son de código abierto y / o gratuitas de muy buen desempeño.

Las revisiones de diseño y código cuando son conducidas por hallazgos obtenidos que siguen el proceso descrito en este capítulo generan muy buenos resultados en la mejora de la calidad del software y constituyen uno de los aspectos de la Ingeniería de Software en los cuales se ha trabajado mucho en la última década y se continúa trabajando hoy día.

15.6 Contenido de la página web de apoyo



15.6.1 Mapa conceptual

15.6.2 Autoevaluaciones

15.6.3 Presentaciones en Power Point (exclusivo para el docente)

15.7 Referencias

- [1] *Object-Oriented Metrics in Practice - Using Software Metrics to Characterize, Evaluate, and Improve the Design of Object-Oriented Systems*, Michele Lanza, Radu Marinescu, Springer, 2006.
- [2] *Working Effectively with Legacy Code*, Michael C. Feathers, Prentice Hall PTR, 2004.
- [3] *Code Complete: A Practical Handbook of Software Construction*, Steve McConnell, Microsoft Press, 2004.
- [4] *Agile Principles, Patterns, and Practices in C#*, Martin C. Robert, Martin Micah, Prentice Hall, 2006.
- [5] *Designing Object-Oriented C++ Applications using the Booch Method*, Robert C. Martin, Prentice Hall, 1995.
- [6] *inCode*, <http://www.intooitus.com/inCode.html>.
- [7] *JDepend*, <http://www.clarkware.com/software/JDepend.html>.
- [8] *Code Complete, 2thd edition*, Steve McConnell, Microsoft Press, 2004.
- [9] *Java Programming Style Guidelines*, <http://geosoft.no/development/javastyle.html>.
- [10] *C++ coding standard*, <http://www.possibility.com/Cpp/CppCodingStandard.html>.
- [11] *CheckStyle*, <http://checkstyle.sourceforge.net/index.html>.
- [12] *Maven2*, <http://maven.apache.org/>.
- [13] *IEEE Standard for Software Reviews*, IEEE Std 1028-1997, IEEE, 1997.
- [14] *X – Ray*, Jacopo Malnati, <http://xray.inf.usi.ch/>.

Pruebas de Software

Contenido

16.1 Definiciones	260
16.2 Niveles y Tipos de pruebas	264
16.3 Estrategias de prueba	270
16.4 Tácticas	273
16.5 Claves del cambio en la forma de trabajo	275
16.6 Ejemplo	281
16.7 Conclusión	283
16.8 Contenido de la página web de apoyo	284
16.9 Referencias	284

Objetivos

- Aprender el concepto de prueba
- Cómo construir pruebas que sirvan para la detección de errores
- Cuáles son los diferentes niveles de prueba de software
- Aprender diferentes estrategias de planteo de las pruebas
- Entender la diferencia entre las formas tradicional y moderna de realizar pruebas
- Qué es un criterio de diseño basado en las pruebas
- Cómo sistematizar y automatizar las pruebas de software

Competencias

- Seleccionar un criterio de prueba para el software de un proyecto

16.1 Definiciones

En este capítulo trataremos temas relacionados a las pruebas de software. Esta disciplina ha sufrido una transformación importante en los últimos 10 años a juicio de los autores en la dirección de una mejora fundamental. En el año 1995, Erich Gamma y Kent Beck desarrollaron una plataforma para implementar pruebas unitarias sistemáticas de código Java llamada JUnit [10]. Consideramos que este desarrollo constituye el hito inicial en el cambio mencionado, el comienzo del cambio que afectó a la prueba de software y la hizo evolucionar desde su forma tradicional a la actual, que se basa en dos conceptos básicos y fundamentales: sistematización de las pruebas y las pruebas como criterio de diseño. En este y el próximo capítulo desarrollamos estos temas [1]. Algunas de las definiciones más precisas de la actividad de prueba de software fueron dadas por Glenford Myers en su libro publicado [2] hace ya 30 años y que aún tienen vigencia a pesar de la evolución en la forma de trabajo y los cambios de la tecnología utilizada.

Algunas de las definiciones conocidas acerca de esta actividad son:

Las pruebas de software constituyen el proceso orientado a demostrar que un programa no tiene errores.

Las pruebas de software son las tareas realizadas para demostrar que un programa realiza las funciones para las cuales fue construido.

Las pruebas de software son las tareas realizadas para probar que un programa realiza lo que se supone debe hacer.

Si bien todas ellas manifiestan una intensión vinculada al objetivo que se busca alcanzar, no son precisas ya que es imposible demostrar que un software no tiene errores, no alcanza con demostrar que realiza las funciones esperadas y aún si las hace, puede contener errores.

Una definición más enfocada es la siguiente:

Las pruebas de software son la ejecución de programas de software con el objetivo de detectar defectos y fallas.

Como se menciona en *The Art of Software Testing* [2], este es un proceso destructivo y sádico.

Según esta definición, una prueba exitosa es aquella que detecta errores y una no exitosa la que no los detecta.

Para llevar adelante esta actividad, planteada en un contexto afectado por un problema psicológico, se requiere un cambio de actitud ya que naturalmente los seres humanos somos constructivos. Esta es la misma razón por la cual no es recomendable que las mismas personas que construyeron el software sean las que lo prueben.

Otras definiciones que conviene hacer para expresar las ideas que siguen con las palabras adecuadas son:

Un error es una equivocación de una persona al desarrollar alguna actividad de desarrollo de software.

Un defecto se produce cuando una persona comete un error.

Una falla es un desvío respecto del comportamiento esperado del sistema, puede producirse en cualquier etapa del ciclo de vida del desarrollo.

Según lo anterior, un defecto es una vista interna de un error ya que lo ven los desarrolladores; mientras que una falla es una vista externa, pues la ven los usuarios.

En lo que sigue utilizaremos en forma indistinta la palabra test para referirnos a las pruebas.

16.1.1 Objetivos y filosofía

Los objetivos y la filosofía con que las empresas de desarrollo realizan las pruebas del software que producen fueron cambiando con el tiempo. Antes de la existencia de Internet, las aplicaciones y productos de software eran utilizados por los usuarios en sus oficinas. Las fallas eran detectadas y a veces reportadas, otras no. La comunicación entre usuarios de productos era mucho menor que hoy día. Un software con fallas muchas veces era utilizado con los perjuicios que ocasionaba durante años. A partir de la utilización masiva de Internet, el software en su mayor parte fue puesto en la vidriera. Usuarios que tratan de utilizar un sitio que aleatoriamente falla después de un par de veces cambian su hábito y reemplazan el uso de dicho sitio por el de otro proveedor. Además, los usuarios están comunicados y los comentarios acerca de escenarios de fallas se propagan rápidamente. Esto hace que el dueño del sitio pierda en corto tiempo una porción importante de sus clientes.

Se concluye de este análisis que como consecuencia de la mayor visibilidad de los productos de software, las pruebas han tomado una importancia mayor en los últimos años en el proceso de desarrollo.

Algunas de las excusas esgrimidas por los desarrolladores de software acerca de que el cliente no aprecia el beneficio de las pruebas, el cliente no entiende de pruebas por lo tanto es muy difícil incorporar en el presupuesto partidas para las pruebas o sea que el cliente no paga por las pruebas, perdieron sentido más que nunca.

Hoy en día, un software con fallas puede arruinar el más productivo de los negocios. Ya no tiene sentido la especulación acerca de la comparación de los costos de invertir en el proceso de pruebas o gastar en el mantenimiento posinstalación [3]. El ejercicio de comparación entre estos costos de mantenimiento y la inversión en pruebas a efectos de acortar el tiempo del retorno de la inversión, es decir que los ingresos superen a los gastos, perdió sentido. Las razones ya las mencionamos, cada vez son menos los errores factibles de ser admitidos en el uso de un producto de software masivo.

Sin embargo, lo analizado no modifica los objetivos de siempre: menores costos, menores tiempos de desarrollo y gran satisfacción del cliente. A la luz de lo expuesto debemos optimizar el proceso de desarrollo en lo referente a las pruebas para alcanzarlos, de forma de generar productos de mayor calidad.

Otro aspecto relevante es el cambio de filosofía de los grupos de desarrollo, donde por las razones analizadas no siempre está instalada la cultura de las pruebas. Implementar pruebas y ejecutarlas suele ser catalogado como un trabajo duro por los desarrolladores. Por lo tanto hizo falta un cambio y este aún está en proceso

de establecimiento, ya que hay empresas que lo adoptaron, otras que no y otras que los están haciendo. Esto último generó la necesidad de convertir a las pruebas en automáticas, fáciles de mantener, vincularlas al desarrollo, robustas y efectivas [4]. En secciones próximas nos referimos a estos temas.

16.1.2 Principios

Más allá de los cambios en la forma tradicional de probar el software hay un conjunto de pautas o principios que deseamos mencionar porque permanecen vigentes desde los días de la primera publicación del libro de Myers [2]. A continuación los presentamos y comentamos a la luz de los cambios mencionados en la disciplina.

1. Una parte necesaria de un test es la definición de los resultados esperados: esta necesidad se plantea a partir de realizar, como parte de la prueba, la comparación con una referencia conocida y considerada correcta. En la figura 16.1, adaptada de *xUnit Test Patterns: Refactoring Test Code* [4], se presenta un esquema donde se muestran las cuatro fases de una prueba, en ella aparece la verificación contra el resultado esperado. También se indican las entradas como puntos de control y las salidas como puntos de observación.

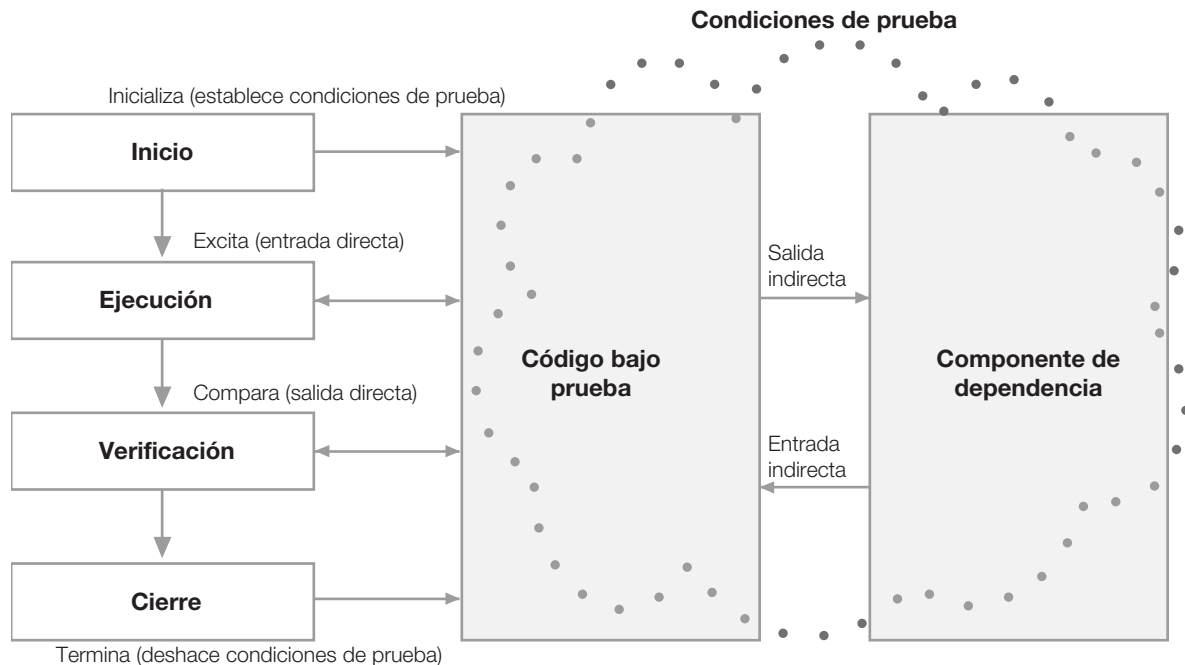


Figura 16.1. Fases de una prueba y su dinámica en relación con el código bajo prueba (adaptada de [4])

2. Un programador debe evitar probar su propio desarrollo: esto es debido al carácter sádico y destructivo mencionado en una sección anterior. Sin embargo, la existencia de herramientas de sistematización de pruebas y la definición de criterios claros de diseño de éstas permiten a los desarrolladores construir pruebas que constituyen un reaseguro al momento de trabajar con código compartido. Esta

modalidad de trabajo será explicada más adelante pero adelantamos que consiste en un repositorio de código al cual todos los desarrolladores aportan y toman el código con el cual trabajan a diario.

3. Una organización no debe probar sus propios desarrollos: la razón es la misma que la anterior, romper con la pertenencia a efectos de diseñar pruebas que cumplan con el objetivo de detectar errores. Sin embargo las empresas, en las cuales se ha empezado a dar el cambio, sus miembros han aprendido que un ambiente de confianza en relación a un sistema en desarrollo o un producto se logra a partir de contar con un conjunto de test bien diseñados y de fácil ejecución.
4. Revise los resultados de los test en profundidad: para evitar una interpretación mala o superficial de estos y la construcción de otros que eventualmente muestren los mismos resultados. Como criterio general los test deberán ser atómicos, independientes, repetibles y fáciles de ejecutar. Todas estas propiedades, como veremos más adelante, las lograremos con la utilización de herramientas de tipo xUnit [4].
5. Los test deben incluir entradas inválidas e inesperadas así como las válidas y esperadas: para probar todos los escenarios posibles de funcionamiento del software bajo prueba. Como se ve en la figura 16.1, en la fase de *setup* es donde se establecen las condiciones vinculadas a estos escenarios.
6. Revisar un programa para verificar que hace lo que se espera que haga es sólo la mitad de la prueba; la otra consiste en comprobar que no haga lo que no se espera: esto se logra de la manera que indica el punto anterior.
7. No tirar los test a la basura a menos que el programa sea basura: porque los test aumentan su valor con el paso del tiempo. Las plataformas de prueba de tipo xUnit promueven la administración del código de prueba de la misma manera que el código productivo en un repositorio de forma que los test forman parte de los activos generados por el proyecto.
8. No planear esfuerzos de pruebas asumiendo que no se encontrarán errores: es no entender el proceso de prueba. Como veremos en el capítulo que sigue, la planificación debe hacerse siguiendo números estadísticos de guía y un criterio para definir el alcance del proceso de prueba que permita establecer los recursos necesarios para la realización de las diferentes actividades del proceso.
9. La probabilidad de encontrar errores en una sección de un programa es proporcional al número de errores ya encontrados en esa sección: porque la experiencia lo indica.
10. El “testing” constituye una tarea creativa e intelectualmente desafiante: esto es así si se afronta como una actividad esencial y no accesorio. Durante mucho tiempo y en diferentes empresas hemos visto que las personas asignadas a las actividades de prueba de software eran los programadores menos capaces. Consideramos que este fenómeno nunca debió producirse pero menos aún hoy día donde, como veremos más adelante, las pruebas serán convertidas en un criterio de diseño. Por lo tanto debemos contar con los mejores desarrolladores trabajando en este aspecto

Si bien los principios continúan siendo válidos, como fue analizado y comentado, los argumentos que los fundamentan han cambiado. No obstante el cambio hacia el nuevo escenario de pruebas sigue estando alineado con los fundamentos expresados por estos principios.

16.2 Niveles y Tipos de pruebas

Las pruebas de software se clasifican comúnmente de acuerdo al nivel del componente que afectan y al aspecto (tipo) del software que se desea probar. Las pruebas realizadas por los programadores de los componentes mínimos de software se denominan test unitarios. El hito a que nos referimos cuando tratamos el tema de evolución de las pruebas fue determinado por JUnit, una herramienta orientada a sistematizar este tipo de pruebas. El concepto seguido se extendió a otros niveles y, como puede verse en la tabla 16.1, hoy día existen herramientas basadas en este que soportan las pruebas de niveles superiores, como las pruebas funcionales, de sistema y de aceptación. Claramente se ve en la tabla que los diferentes niveles de prueba son realizadas con diferentes objetivos, por diferentes roles y en distintos lugares. De esto se deriva que son diseñadas, ejecutadas y analizados sus resultados en diferentes momentos del ciclo de vida del proyecto de desarrollo. Sin embargo, como analizaremos en una próxima sección, la nueva escuela promueve la realización de todas estas lo más temprano posible y con una forma de trabajo muy interdisciplinaria. Puede verse la participación de los desarrolladores en las pruebas de funcionalidad. Esto es necesario para construir el código que permita integrar dichas pruebas a un ambiente de prueba sistematizado con el fin de minimizar el trabajo manual y facilitar la ejecución repetitiva de los test con la menor cantidad de errores introducidos al ejecutarlos y con el mejor aprovechamiento de los recursos posible [1].

Niveles de prueba de software					
Test	Objetivo	Participantes	Ambiente	Método	Herramientas
Unitario	Detectar errores en los datos, lógica y algoritmos	Programadores QA	Desarrollo	Caja Blanca	JUnit, DBUnit, HttpUnit, ServletUnit, Test de Watir
Integración	Detectar errores de interfaces y relaciones entre componentes	Programadores QA	Desarrollo	Caja Blanca Top Down Bottom Up	JUnit, Continuum
Funcional	Detectar errores en la implementación de los requerimientos	Programadores Testers Analistas QA	Desarrollo	Caja Negra	FitNesse
Sistema	Detectar fallas en el cubrimiento de los requerimientos	Testers Analistas QA	Desarrollo	Caja Negra	JUnit, JMeter, FitNesse
Aceptación	Detectar fallas en la implementación del sistema	Testers Analistas Cliente QA	Productivo	Caja Negra	JUnit, JMeter, FitNesse

Referencias de la Tabla 16.1

Herramienta	Referencia
Tests de Watir	http://wiki.openqa.org/display/WTR/Tutorial
HttpUnit	http://httpunit.sourceforge.net/
ServletUnit	http://httpunit.sourceforge.net/doc/servletunit-intro.html
FIT Fitnessse	http://fitnesse.org/
JUnit	http://www.junit.org/
DBUnit	http://www.dbunit.org/
JMeter	http://jakarta.apache.org/jmeter/
Continuum	http://continuum.apache.org/

Tabla 16.2. Algunos tipos de pruebas y sus características

Tipos de Pruebas		
Test	Objetivo	Características
Test de Volumen	Orientado a probar el funcionamiento con datos masivos	Pueden ser diseñados y ejecutados con un prototipo de la arquitectura y verificados cuando el sistema es terminado
Test de Estrés	Orientado a probar el funcionamiento en condiciones de carga	Pueden ser diseñados y ejecutados con un prototipo de la arquitectura y verificados cuando el sistema es terminado
Test de Usabilidad	Orientado a probar la facilidad de uso de las interfaces de usuario	Se suele trabajar con prototipos que permitan flexibilidad ante cambios
Test de Seguridad	Orientado a probar normas de seguridad de acceso	Son ejecutados a medida que se construyen los componentes para validar que siguen la política definida
Test de Performance	Orientado a probar tiempos de respuesta, procesamiento, etc.	Son ejecutados en ambientes iguales al productivo
Test de Instalabilidad	Orientado a probar el procedimiento y datos de configuración e instalación	Son ejecutados en ambientes vírgenes
Test de Documentación	Orientado a probar la legibilidad y completitud de manuales de usuario y técnicos	Se generan consultas al azar para verificar completitud

16.2.1 Métodos de prueba

Cuando el método de prueba prescinde de la estructura interna del software y solo se diseña el test a partir del conocimiento expresado en su especificación esto se conoce como *de Caja Negra*. Por el contrario, cuando el test es concebido a partir del conocimiento interno del software se denomina *de Caja Blanca*.

16.2.2 Caja Negra

Este método de prueba se diseña a partir de la especificación de los requerimientos y de la selección de los datos de entrada. Por esta razón se lo llama a veces *conducido por los datos*. Consiste en la ejecución de test con un conjunto de datos de entrada que ejercitan exhaustivamente al software bajo prueba. Debido a la imposibilidad de probar con las infinitas posibles variantes, se elaboran conjuntos de datos representativos de la totalidad denominados *Clases de Equivalencia*. En la figura 16.2 se presenta un esquema conceptual de este tipo de pruebas. En una sección próxima presentamos a modo de ejemplo una funcionalidad del proyecto del caso testigo donde se utilizan los diferentes métodos de prueba.

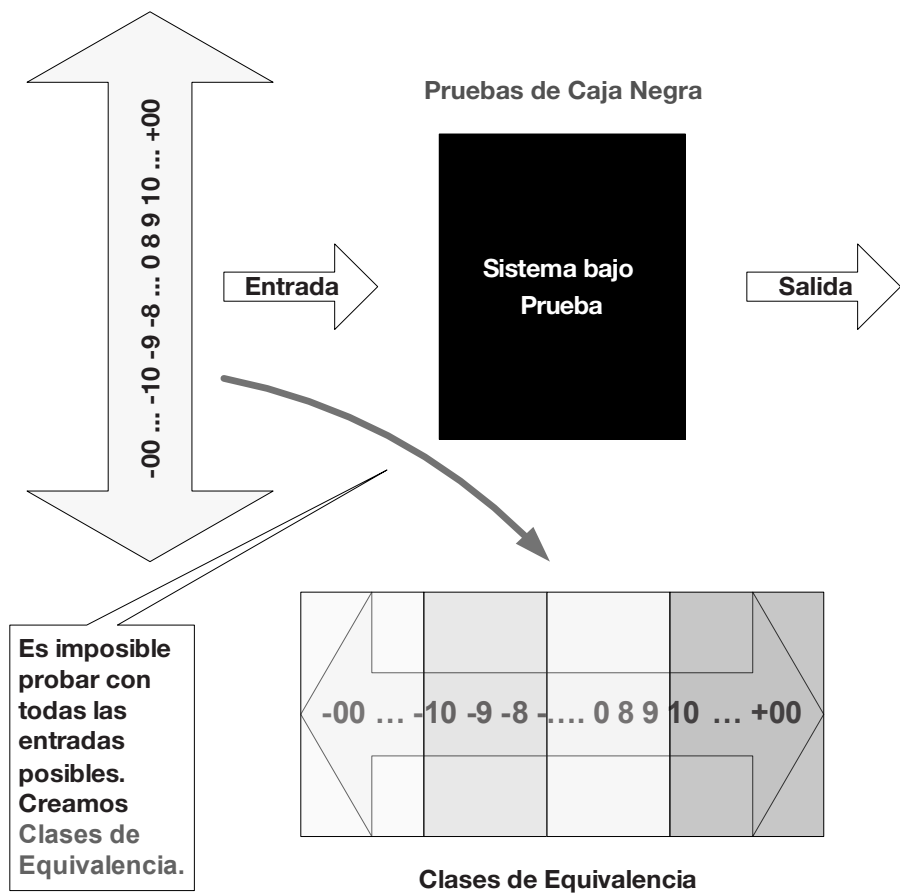


Figura 16.2. Esquema de test de Caja Negra

16.2.3 Caja Blanca

Cuando el método de prueba se diseña a partir de la estructura y lógica del software bajo prueba lo llamamos *de Caja Blanca*. Para su diseño se pueden considerar diferentes criterios los cuales se muestran esquemáticamente en la figura 16.3. Cabe destacar que los valores constantes que aparecen en este código fueron insertados con la intención de facilitar su lectura en el análisis de esta presentación aunque viole la buena práctica de programación que establece no insertar constantes de esta forma.

Un criterio posible es diseñar el test de forma que se ejecuten todos los caminos o bifurcaciones del programa. Este criterio corresponde a los textos a los que se hace referencia de la figura.

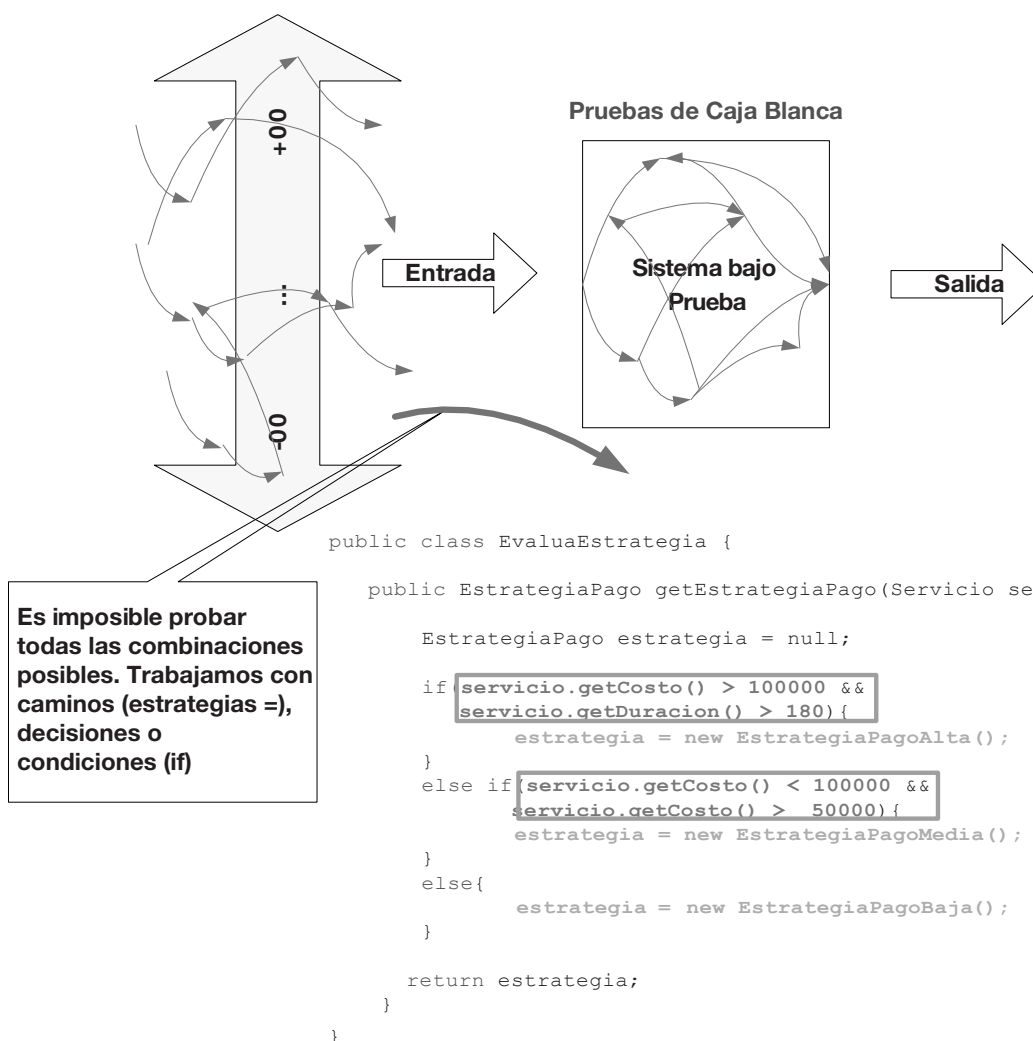


Figura 16.3. Criterios de diseño de test para el método de prueba denominado de Caja Blanca

Con la intención de probar el software con este criterio basta con ejecutar un test con un valor para el costo del servicio mayor que 100.000 y un valor para la duración de 300 días, otro con un valor entre 50.000 y 100.000 y un tercero con un valor menor a 50.000.

En el primer caso la prueba es incompleta ya que si lo correcto fuera un costo que supere un valor de 100.000 o una duración que supere un tiempo en días de 180 no se ejecutaría el camino correspondiente en algunos casos. Sin embargo debería ejecutarse con cualquiera de las condiciones que componen la decisión. Esto es debido a una decisión equivocada tomada a partir de más de una condición. Esta visto entonces que no basta con la prueba de los caminos sino que el criterio debe ampliarse.

Una forma de ampliarlo es diseñar un test para cada decisión. Uno correspondiente a un valor esperado verdadero y otro para falso. De esta forma elegiríamos probar con valores de costo y duración del servicio que generen resultados verdaderos y falsos para cada decisión.

Debido a que una decisión está compuesta de más de una condición y que los resultados lógicos de estas pueden combinarse de diferentes formas enmascarando escenarios, otro criterio más amplio de prueba es diseñar los test en base a las decisiones. Estos criterios son los que se representan en el esquema de la figura 16.3.

Por último nos detendremos en los valores límites. En el caso mostrado de la figura estos valores son los correspondientes a las constantes contra las cuales se realizan las comparaciones. El criterio de diseño de pruebas basado en estos propone un test con datos igual al valor límite, uno con un valor apenas por debajo y otro con un valor apenas por encima de este.

Tabla 16.3.a. Tests derivados de las clases de equivalencia para valores de entrada				
Clases de Equivalencia				
Clases	Condición Entrada	Clase Equivalencia	Test	Id Test
1	Duración del servicio fija Costo del servicio variable	Valor positivo > 0.00	Prueba con entrada costo = 150.000.00 y duración = -100	1
2		Valor cero (0)	Prueba con entrada costo = 0.00 y duración = -100	2
3		Valor < 0	Prueba con entrada costo = -1000.00 y duración = -100	3
4	Duración del servicio fija Costo del servicio variable	Valor positivo > 0.00	Prueba con entrada costo = 150.000.00 y duración = 0	4
5		Valor cero (0)	Prueba con entrada costo = 0.00 y duración = 0	5
6		Valor < 0	Prueba con entrada costo = -1000.00 y duración = 0	6

Tabla 16.3.a. Tests derivados de las clases de equivalencia para valores de entrada

Clases de Equivalencia				
Clases	Condición Entrada	Clase Equivalencia	Test	Id Test
7	Duración del servicio fija Costo del servicio variable	Valor positivo > 0.00	Prueba con entrada costo = 150.000.00 y duración = 100	7
8		Valor cero (0)	Prueba con entrada costo = 0.00 y duración = 100	8
9		Valor < 0	Prueba con entrada costo = -1000.00 y duración = 100	9

Tabla 16.3.b. Tests derivados de las decisiones y condiciones de la estructura y lógica del software a probar

Decisiones / Condiciones				
Decisión	Lógica	Condición a probar	Test	Id Test
1	costo >= 100.000 duración >= 180	true / true	Prueba con entrada costo = 150.000.00 y duración = 200	10
		true / false	Prueba con entrada costo = 150.000.00 y duración = 100	7
		false / true	Prueba con entrada costo = 60.000.00 y duración = 200	11
		false / false	Prueba con entrada costo = 1000.00 y duración = 100	12
2	costo < 100.000 y costo >= 50.000	true / true	Prueba con entrada costo = 60.000.00	11
		true / false	Prueba con entrada costo = 1000.00	12
		false / true	Prueba con entrada costo = 150.000.00	7
		false / false	Prueba con entrada costo = 1000.00	12

Como conclusión podemos utilizar al conjunto de criterios para diseñar los test. Como veremos algunos de ellos producen escenarios de prueba que se solapan con los generados por otros. En la tabla 16.3 a, b y c se muestran los test diseñados para el ejemplo, diseñados de acuerdo a clases de equivalencia para los valores de entrada, decisiones / condiciones y valores límites. Puede verse en la columna de la derecha que los escenarios de prueba no siempre suman un nuevo test.

Tabla 16.3.c. Tests derivados de los valores límites del software a probar				
Valores Límites				
Límite	Valor	Condición a probar	Test	Id Test
1	100.000	=	Prueba con entrada costo = 100.000 y duración = N	12
		>	Prueba con entrada costo = 100.001 y duración = N	13
		<	Prueba con entrada costo = 99.999 y duración = N	14
2	50.000	=	Prueba con entrada costo = 50.000	15
		>	Prueba con entrada costo = 50.001 y duración = N	16
		<	Prueba con entrada costo = 49.999 y duración = N	17
3	180	=	Prueba con entrada costo = 180 y costo = M	18
		>	Prueba con entrada costo = 181 y costo = M	19
		<	Prueba con entrada costo = 179 y costo = M	20

16.3 Estrategias de prueba

Como en toda actividad, para llevar a cabo las pruebas de software es necesario contar con una estrategia. Es difícil plantear una cuando aún restan tratar temas que hacen a su elección. Sin embargo recomendaremos algunas pautas mínimas que constituyen la esencia de un software bien probado.

Si el proyecto está en marcha recomendamos revisar algunas de las siguientes situaciones [4] que ponen de manifiesto falencias en las actividades de prueba:

El número de veces que deben ser modificados los test y la cantidad de errores encontrados en las pruebas de funcionalidad realizadas por usuarios en las pruebas de aceptación. Esto afecta a la funcionalidad, la calidad, los recursos y el costo de los productos generados.

Los cambios causan fallas en los test o directamente no compilan. Es decir que estos dependen y son frágiles frente a cambios en interfaces, datos, funcionalidad, fecha, hora, accesos a servidores, etc.

Los indicadores anteriores muestran que su proyecto necesita de una estrategia que defina los objetivos de la actividad ya que a la luz de las situaciones anteriores estos no se están logrando por falta de claridad en su definición o por la aplicación de malas tácticas.

Entre las preguntas que deben responderse para la definición de la estrategia mencionada están además las siguientes:

- Qué clase de test automatizar
- Cuáles herramientas utilizar para la automatización
- Cómo administrar las condiciones de prueba
- Cómo asegurar que el sistema será eficientemente probado.

16.3.1 Propiedades básicas

Alineados con los principios ya enumerados al comienzo del capítulo y en busca de establecer la estrategia definimos las siguientes propiedades básicas que deberán cumplir los test del proyecto:

Atómicos: prueban una única funcionalidad o aspecto del sistema.

Independientes: ejecución aislada, sin dependencia de datos, ambiente u otros test.

Separación de incumbencias: código de prueba por un lado y productivo por el otro.

Repetibles: producen los mismos resultados en múltiples ejecuciones sin intervención manual.

Fáciles de ejecutar y mantener: ejecución automática ligada al proceso de construcción de binarios y formando parte del repositorio del proyecto.

16.3.2 Evolución de las pruebas con el desarrollo

Las pruebas del software en desarrollo pueden ser planeadas y ejecutadas de diversas formas. Entre estas formas mencionaremos top down, bottom up e incrementales.

16.3.2.1 Top Down

Consiste en desarrollar y probar las capas de nivel superior antes de las de nivel inferior. Para que esto sea posible es necesario definir interfaces claras para los niveles inferiores y la implementación de un software

que emule el comportamiento de los niveles aún no implementados. Estos programas o componentes son conocidos como Stubs (reemplazo del objeto real por otro que genera las entradas indirectadas deseadas). A medida que el desarrollo avanza se quitan estos componentes y se reemplazan por el software desarrollado para ese nivel. En la figura 16.4 se muestra un esquema de esta forma de trabajo.

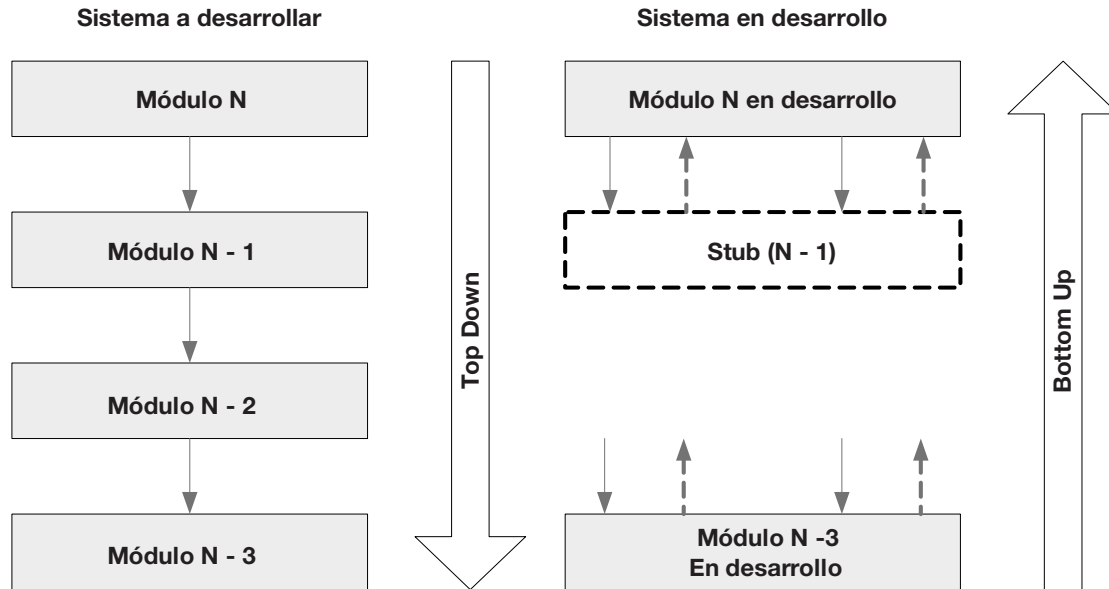


Figura 16.4. Esquema del desarrollo y de la prueba de un software en forma top down y bottom up

Entre las desventajas de esta forma de trabajo podemos mencionar:

- Se retrasa la prueba del procesamiento real realizado generalmente en módulos de más bajo nivel.
- Desarrollar Stubs que emulen a los módulos es mucho trabajo.

16.3.2.2 Bottom Up

Consiste en desarrollar y probar las capas de nivel inferior antes de las de nivel superior. Las pruebas comienzan en el más bajo nivel con la implementación de algoritmos que realizan procesamiento. Se escriben test que dan el contexto de ejecución a los módulos, se prueban los módulos, se desarrolla e integran funcionalidades del módulo superior y se repite este ciclo.

En la figura 16.4 se muestra un esquema de esta forma de trabajo.

Entre las desventajas de esta forma de trabajo podemos mencionar:

- Hasta que se logra un nivel determinado, la aplicación no es visible.
- Problemas asociados a volumen, recursos y tiempo se prueban en etapas tardías.

16.3.2.3 Incrementales

En cualquiera de las formas de trabajo anteriores, cada vez que una parte del software es terminada y probada, resta aún la integración con las partes ya desarrolladas a efectos de hacer crecer el sistema. Este trabajo de integración suele ser complejo y generador de muchos errores. Sin embargo es necesario a la hora de contar con dos componentes de software que forman parte de un mismo sistema. Una forma de realizar este trabajo que minimiza el esfuerzo y evita errores es el trabajo con código compartido y el testeo continuo, ya que distribuye las pruebas de integración en la integración diaria del código compartido. En una sección próxima trataremos en detalle estos aspectos.

16.4 Tácticas

Existe un conjunto de buenas prácticas de pruebas que al igual que los criterios de buen diseño de software ya vistos son más abstractos que los patrones. Poseen la misma relación que en el diseño entre los criterios de buen diseño y los patrones de diseño. Estos criterios fueron recopilados en *xUnit Test Patterns: Refactoring Test Code* [4] y de ellos se derivarán algunas de las tácticas que presentaremos en formato patrón de prueba.

1. Escriba el test primero, antes que el código que se desea desarrollar / probar.
2. Diseñe el software para que haya testeabilidad.
3. Utilice interfaces públicas siempre que pueda, en lugar de accesos no visibles o recursos privados.
4. Comunique la intención de la prueba, para facilitar la lectura y el mantenimiento.
5. No modifique el software bajo prueba, de esta forma se podrá garantizar que se prueba lo desarrollado y lo que irá al ambiente productivo.
6. Mantenga los test independientes para eliminar dependencias y facilitar el aislamiento de errores.
7. Aíse al software bajo prueba para no depender del ambiente.
8. Minimice el solapamiento de test para facilitar el mantenimiento.
9. Minimice el código no probado para evitar errores en pruebas funcionales y en ambiente productivo.
10. Mantenga la lógica de prueba fuera del código productivo para evitar fallas en producción.
11. Verifique una condición a la vez para facilitar el aislamiento de errores con sus fases de establecimiento de condiciones, ejecución, verificación de resultados y desarmado de condiciones.
12. Pruebe incumbencias por separado para facilitar el aislamiento de errores y el mantenimiento de los test.
13. Balancee el esfuerzo de prueba en relación con el desarrollo para que sea sustentable.

Los patrones de prueba pueden clasificarse [4] como:

Establecimiento y Administración de Condiciones de Prueba: reúne los patrones de establecimiento y restablecimiento de las condiciones de prueba tales como frescas y transitorias, creadas para cada test cuando son ejecutados. Condiciones frescas y persistentes funcionan en escenarios donde se prueban componentes acoplados por algún medio persistente. Condiciones compartidas funcionan en escenarios donde se busca optimizar la ejecución de test cuando se usa la táctica de frescas y persistentes. Necesitan código que se ocupe de establecerlas y desarmarlas. Estos patrones tienen gran influencia en el rendimiento a la hora de realizar y ejecutar tests en forma masiva.

Verificación de Resultados: comparación de resultados esperados contra resultados obtenidos a partir de Asserts. Se comparan estados y comportamiento como se muestra en la figura 16.5.a.

Organización de Test: los test de un proyecto pueden ordenarse por clase, por funcionalidad o por condición de prueba.

Pruebas con Base de Datos: orientados a probar el software de interacción con la base sin la base (Fake-Object), probar *store procedures* y a desacoplar la base de datos (inyección de dependencia).

Diseño para Testeabilidad: utilizar mecanismos de desacoplamiento que permitan probar todos y cada uno de los aspectos del software bajo prueba, como la inyección de dependencias y las fábricas de objetos.

Pruebas con Dobles (Imitadores): para reemplazar componentes de los cuales se depende y mejorar repetibilidad o *performance* (Fake Object), para controlar entradas indirectas (TestStub), para verificar salidas indirectas (MockObject), para verificar comportamiento a través de salidas indirectas (SpyTest) y para reemplazar subsistemas con los que se interactúa y obtener entradas indirectas (TestDouble).

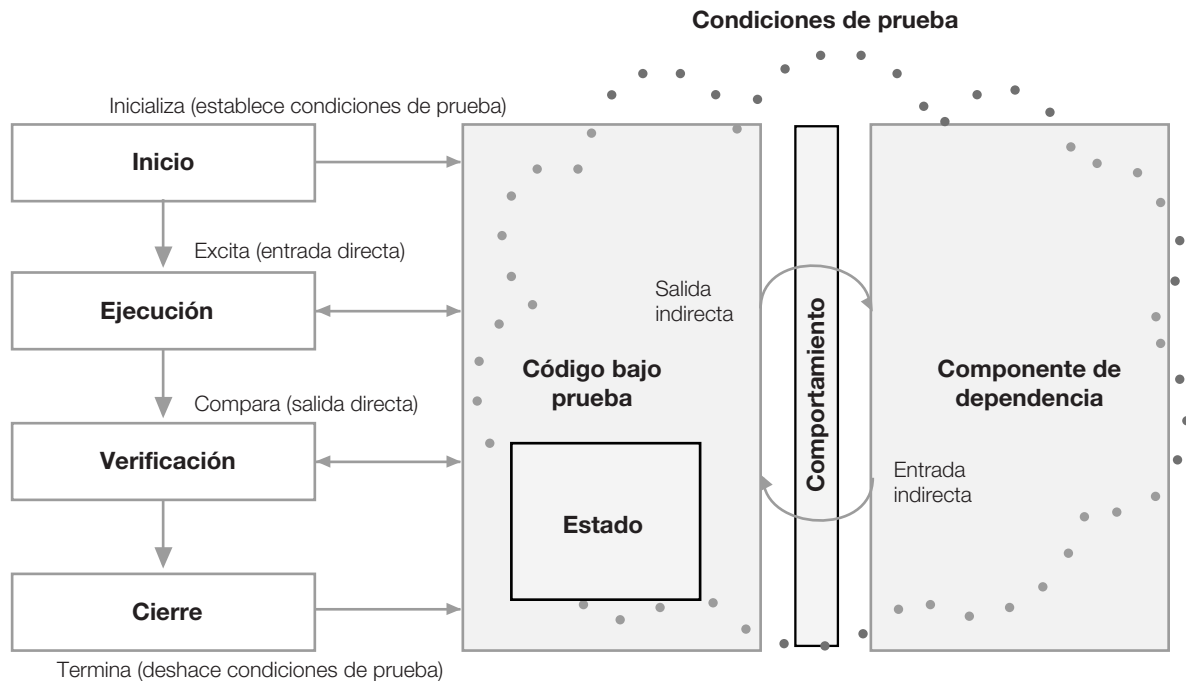


Figura 16.5.a. Esquema de prueba que muestra la verificación de estado y comportamiento del software bajo prueba. Un patrón de prueba orientado a la verificación del comportamiento es SpyTest, un tipo de Test con Dobles

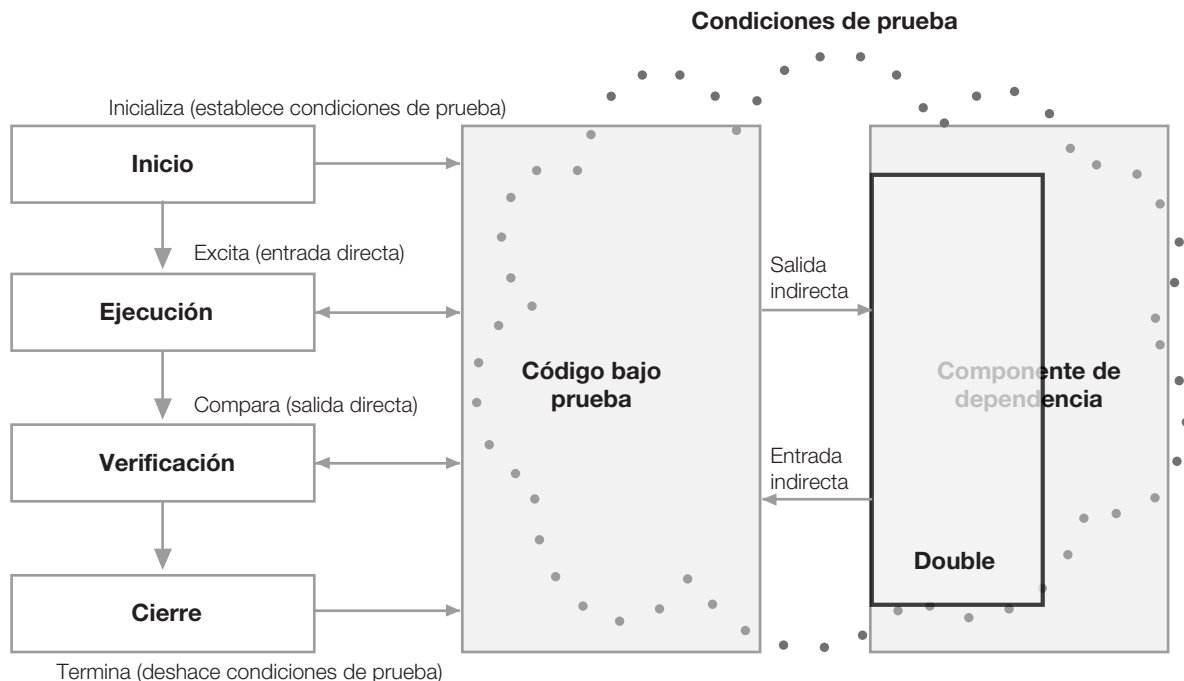


Figura 16.5.b. Esquema de prueba que muestra el reemplazo del componente del cual depende el software bajo prueba por un Doble. Los patrones orientados a realizar este tipo de pruebas son Fake Object, Mock Object, Spy Test y Test Stub

16.5 Claves del cambio en la forma de trabajo

Como fue adelantado en una sección anterior, las claves de la evolución de la forma tradicional de probar software a la moderna son las siguientes:

- Automatización
- Prueba como criterio de diseño.

Ambas son claves por las siguientes razones. La automatización porque facilita la ejecución repetitiva de los test con un aprovechamiento muy bueno de los recursos. Para esto es condición necesaria haber sistematizado el diseño y ejecución de los test, por ejemplo a través del uso de herramientas de tipo xUnit en forma integrada a un repositorio de código fuente como veremos más adelante. Por lo tanto, si los test son fáciles de ejecutar, se ejecutan; y esto es lo que sucede.

Debido a la estrategia de adelantar las tareas de prueba en el ciclo de vida de los proyectos, como se analizó anteriormente, se presenta la posibilidad de condicionar el diseño a las pruebas. De esta forma nace un nuevo criterio de diseño y es el que desacopla componentes a efectos de separar incumbencias desde la perspectiva de facilitar la realización de pruebas.

16.5.1 Razones para automatizar las pruebas

Como se analizó en el capítulo 12 y se esquematizó en la figura 12.6, el ciclo manual de pruebas (forma de trabajo tradicional) es largo, lento y caro [1]. Esta es una de las razones por las cuales se desea contar con un ciclo automatizado. Entre otras se encuentran además [4], [5]:

- El ciclo de prueba manual es muy largo.
- El proceso de prueba manual es propenso a errores.
- Liberar a la gente para realizar tareas creativas en lugar de ejecutar test, tarea tediosa que puede ser realizada por una computadora.
- Generar un ambiente de confianza soportado por los **test ejecutándose en forma continua, de manera que cualquier cambio en el código que genere una falla será detectado y reportado rápidamente.**
- Obtener realimentación de forma temprana y con alta frecuencia basado en las razones del punto anterior.
- Generar conocimiento del sistema en desarrollo a partir de los test ya que se promueve la **participación de desarrolladores, analistas y testers en el diseño y construcción en etapas tempranas.**
- Generar documentación del código consistente basado en los test.
- Generar una mejor utilización de los recursos a partir de una forma de trabajo que implica la erogación de menores costos.

16.5.1.1 Adelanto de las pruebas en el ciclo de vida

Por las razones mencionadas y la estrategia que planteamos, el adelanto de las tareas relacionadas a las pruebas en el ciclo de vida de los proyectos es importante. En la figura 16.6 se muestra un esquema de la forma tradicional de trabajo y en la figura 16.7 el equivalente para la forma de trabajo que llamamos *moderna*. Se observa que las pruebas unitarias y de integración fueron adelantadas de forma de realizarlas a medida que avanza el desarrollo con cada iteración [1]. De esta forma se promueve el trabajo de los diferentes roles de manera muy conectada, la influencia de las pruebas en el diseño, la participación de los analistas y testers en la construcción de pruebas de funcionalidad utilizando herramientas de tipo xUnit que permitirán sistematizar los test para luego automatizarlos como describiremos en el próximo capítulo. Básicamente se trata de volcar los test pensados y diseñados en planillas de cálculo en test con formato xUnit para que sea factible su ejecución por parte de una herramienta [5].

16.5.1.2 Obstáculos para automatizar las pruebas

En una organización en la cual se plantee realizar el cambio de la forma de trabajo mencionada se encontrarán seguramente algunos de los siguientes obstáculos:

Actitud de los programadores que consideran la elaboración de los test como trabajo adicional de menor valor que el diseño y codificación del código productivo.

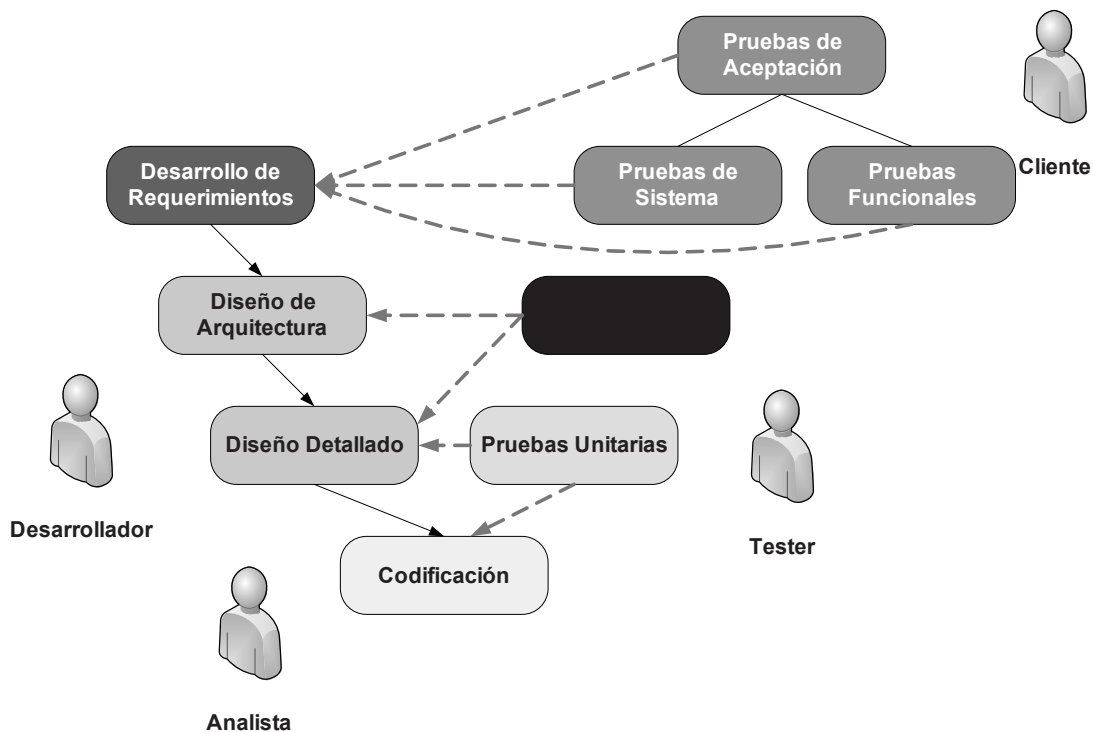


Figura 16.6. Forma de trabajo tradicional en tareas de prueba de software

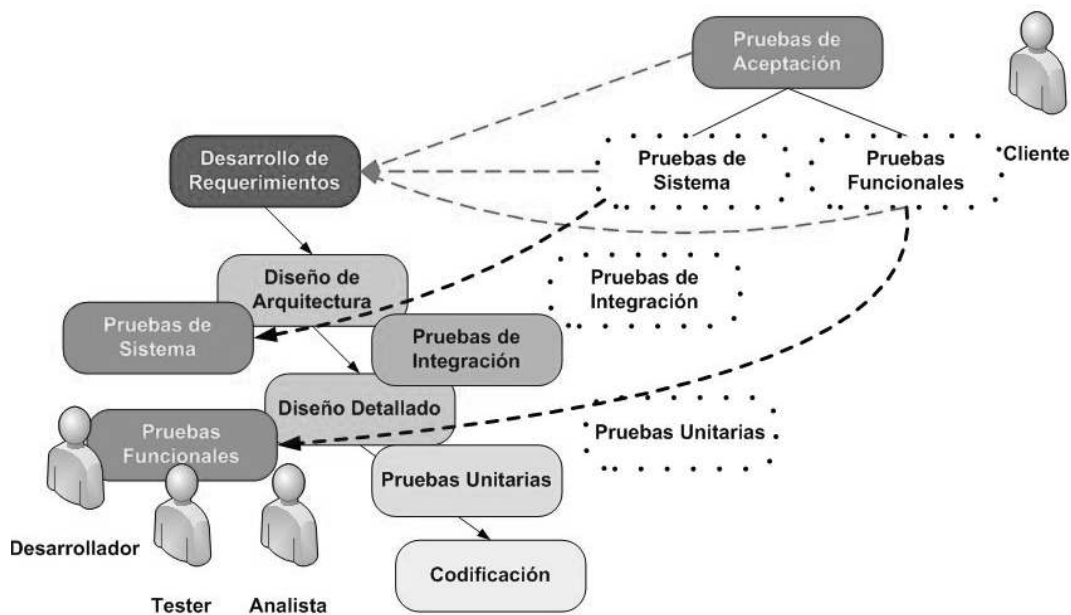


Figura 16.7. Forma de trabajo moderna en tareas de prueba de software donde se ve el adelanto de las tareas vinculadas a las pruebas realizadas en forma conjunta por los diferentes roles

La oposición al cambio y el temor al aprendizaje de la nueva forma. **La excusa de que el código siempre cambia** y entonces se necesita un esfuerzo adicional para el mantenimiento de los test. **La existencia de sistemas legacy** con la imposibilidad de plantear test para ellos y el **temor al cambio de los viejos hábitos**.

Todos estos son obstáculos fácilmente removibles [1], [4].

16.5.2 Pruebas sistemáticas

16.5.2.1 Trabajo con código compartido

Debido a las características del software de gran parte de los sistemas que hoy día se construyen, el modelo de trabajo es el que se muestra en la figura 16.8.

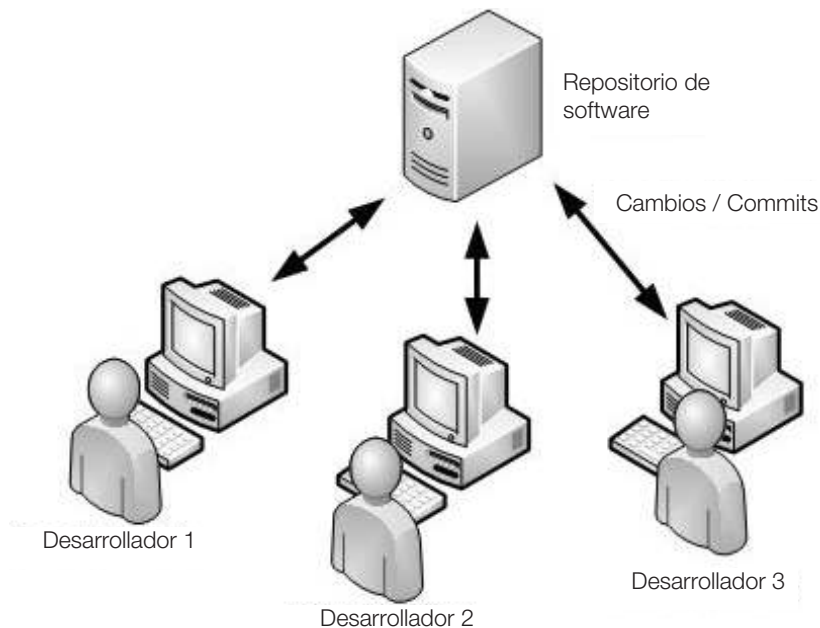


Figura 16.8. Modelo de trabajo con código compartido

Este esquema de trabajo consiste en un grupo de desarrolladores que escriben código en sus respectivas estaciones de trabajo. Cada uno de ellos está desarrollando componentes de software que dependen de los que el resto desarrollan simultáneamente. Por esta razón, comparten el código que cada uno de ellos escribe y prueba a través de un repositorio común de fuentes [7]. Esta dinámica de trabajo consiste en que cada uno adapta su estación de trabajo con los cambios generados con la mayor frecuencia posible (diariamente), desarrolla sus componentes, los prueba escribiendo además los test correspondientes e incorpora al repositorio todo este código. La condición que permite a los desarrolladores incorporar nuevo código al repositorio es la ejecución exitosa de los nuevos y viejos test, de sus test y de los del resto de los desarrolladores. De esta forma se mantiene un repositorio de código que además de constituir el único punto de acceso al código, tiene un nivel de calidad garantizado por los test. También, como se desprende de lo dicho, la integración de los diferentes

componentes se distribuye en tareas diarias en lugar de hacerlo cuando los componentes están terminados con las consabidas dificultades que esto presenta y el tiempo adicional que esto implica. Para que la ejecución de estos test pueda llevarse a cabo de la manera descrita es necesario contar con una plataforma que permita sistematizar dichos test. Estas herramientas, en su mayoría de tipo xUnit, se describen a continuación.

16.5.2.2 Herramientas de tipo xUnit

Cuando la plataforma para Java fue concebida se le llamó JUnit [7], a partir de entonces las diferentes plataformas construidas para los diferentes lenguajes o tecnologías que siguieron la filosofía de JUnit eligieron nombrar a las suyas reemplazando la “J” de Java por alguna letra o sigla vinculada. Como fueron mencionadas en la tabla 16.1 existen varias de ellas. Además de las allí mencionadas existen para lenguajes como C++, PHP, C# y otros. Las pautas seguidas por los autores de JUnit en su diseño fueron:

- Permitir la realización de pruebas de ejecución automática una vez construidas para facilitar su ejecución continua.
- Permitir la realización de pruebas repetitivas para generar seguridad.
- Una herramienta que haga a los test fáciles de construir y ejecutar por los desarrolladores, por lo tanto debe ser construida en el mismo lenguaje de desarrollo y así evitar mudar de tecnología a la hora de codificar un test.
- Una herramienta que verifique resultados y los informe.
- Una herramienta que manipule los casos de prueba como si fueran objetos.

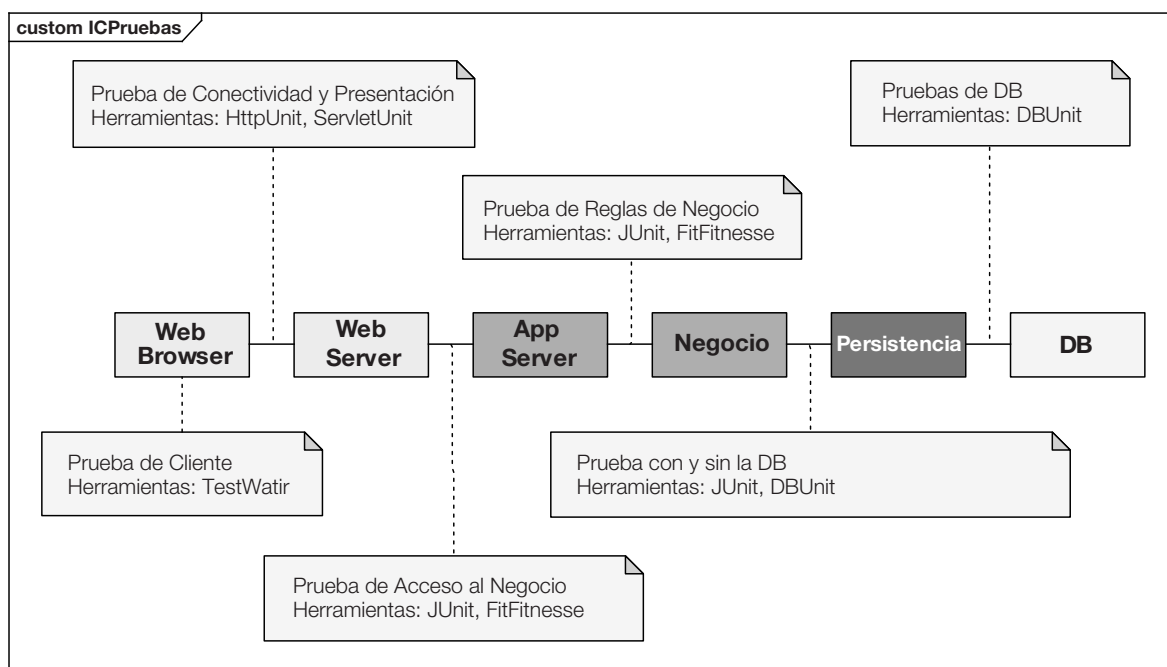


Figura 16.9. Herramientas de prueba de tipo xUnit utilizadas en las diferentes capas de una aplicación *enterprise*

En la figura 16.9 se presenta en forma esquemática algunas de las herramientas de tipo xUnit utilizadas en las pruebas de las diferentes capas de una aplicación *enterprise*. Todas ellas presentan una arquitectura que sigue los lineamientos de JUnit y promueve una forma de trabajo como la que se describió.

16.5.2.3 Las pruebas como criterio de diseño

Se define el *grado de prueba* (*testability*) de un software como la facilidad que presenta para ser probado. Según el esquema de pruebas presentado en la figura 16.1, para que un software presente esta propiedad debe ser **controlable y observable**, es decir debe tener puntos de control donde los test invoquen la funcionalidad del software y puntos de observación en los que los test verifiquen los resultados obtenidos contra los esperados. Según las recomendaciones realizadas en secciones anteriores, deben ser **atómicos e independientes** para detectar fallas localizadas que puedan ser fácilmente reproducibles y asignadas a un error determinado. Estas propiedades deseables de los test deben ser logradas en el proceso de diseño a través del uso de criterios como la **separación de incumbencias** que garantice la posibilidad de plantear test atómicos por funcionalidad y a través del **desacoplamiento** que asegure la independencia.

En la figura 16.10.a se observa un esquema del módulo de ruteo de cargas del sistema del caso testigo. Este módulo concebido de forma atómica y monolítica presentó gran dificultad a la hora de su prueba. Hasta que se decidió refactorizarlo, como se muestra en la figura 16.10.b, separando las funcionalidades de enfoque de la zona que incluye el puerto de origen y destino, detección y ubicación de los puertos en la zona donde la empresa presta servicios y por último el cálculo del itinerario según el criterio seleccionado. De esta forma se detectaron y fijaron rápidamente los errores que provenían de la ubicación de los puertos con servicios.

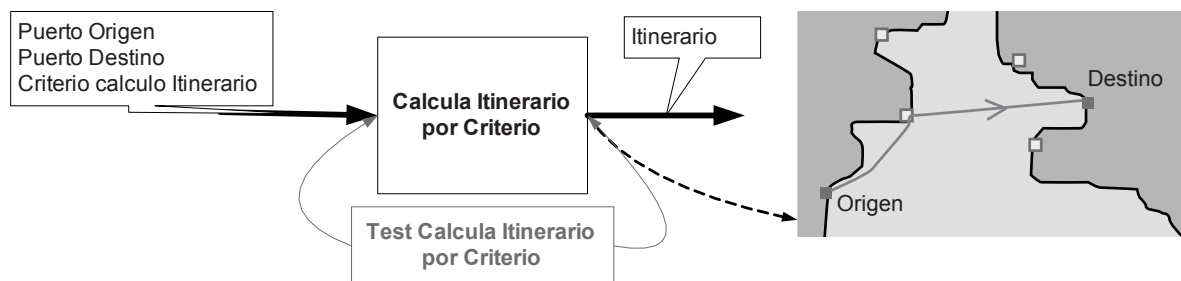


Figura 16.10.a. Esquema del módulo original del ruteo de cargas del sistema del caso testigo como un subsistema atómico y de muy difícil prueba

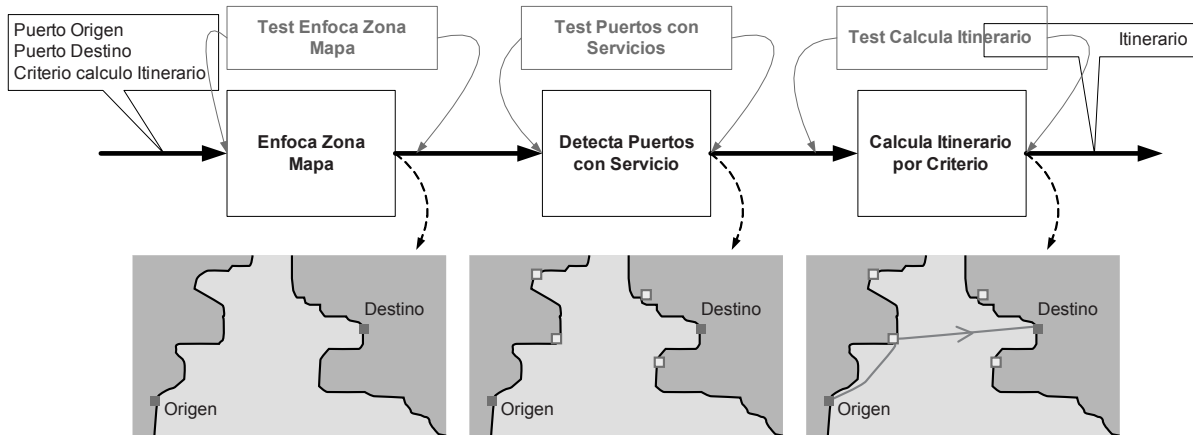


Figura 16.10.b. Esquema del módulo del ruteo de cargas del sistema del caso testigo modificado, con funcionalidades desacopladas con el criterio de facilitar las pruebas

16.6 Ejemplo

Se muestran a continuación tres test escritos para probar la funcionalidad asociada a los pagos de los diferentes tipos de servicios de transporte de cargas, cuestión que sirvió en una sección anterior para diseñar las pruebas de caja blanca y negra. En este caso se utiliza JUnit como plataforma de prueba. Los servicios mencionados poseen una estrategia de pago en forma intrínseca, la prueba que se plantea es el funcionamiento de la selección de esta estrategia a partir de servicios con diferentes valores y duración. En el listado que sigue se observa que las clases que implementan los test extienden a `TestCase` de la plataforma JUnit.

```
import junit.framework.Test;
import junit.framework.TestCase;
import junit.framework.TestSuite;

public class TestAdminPagosServicio extends TestCase{

    static private Servicio ServicioLocal; static private Servicio
    ServicioDoméstico; static private Servicio ServicioGlobal;

    static private boolean isFixtureSetup = false;
```

Se han declarado tres instancias de tipo estáticas de la clase `Servicio`, las cuales serán los objetos sobre los que realizan las pruebas.

En el listado de abajo se presenta el método `setUp` redefinido donde se establecen las condiciones de prueba, en este caso se crean y persisten las instancias de los servicios. Se observa que se instancia un servicio que por sus características comprende tres (3) pagos, uno de dos (2) pagos y uno de uno (1).

```

//-----
// se establecen las condiciones de prueba
//-----
protected void setUp() {

    if(isFixtureSetup == false){
        TransactionFactory.TRANSACCION.openBusinessTransaction();
        setCondiciones();
        try {
            TransactionFactory.TRANSACCION.commitBusinessTransaction();
        }
        catch (EnterpriseException e) {
            fail("Falla el commit de la UoW en setUp");
        }
        TransactionFactory.TRANSACCION.closeBusinessTransaction();
        isFixtureSetup = true;
    }
    TransactionFactory.TRANSACCION.openBusinessTransaction();
}

protected void setCondiciones() {
    Calendar cal = Calendar.getInstance();
    java.sql.Date fecha = new java.sql.Date(cal.getTimeInMillis());
    ClientePersona cliente = null;
    cliente = new ClientePersona("Andres", "andresp@empresa.ar");
    cliente.setTransaccion(TransactionFactory.TRANSACCION.getTransaccion());
    ServicioLocal = new Servicio(cliente, 90, 20000, fecha);
    ServicioLocal.setTransaccion(TransactionFactory.TRANSACCION.getTransaccion());
    assertTrue(ServicioLocal != null);
    ServicioDoméstico = new Servicio(cliente, 180, 60000, fecha);
    ServicioDoméstico.setTransaccion(
        TransactionFactory.TRANSACCION.getTransaccion());
    assertTrue(ServicioDoméstico != null);
    ServicioGlobal = new Servicio(cliente, 360, 150000, fecha);
    ServicioGlobal.setTransaccion(
        TransactionFactory.TRANSACCION.getTransaccion());
    assertTrue(ServicioGlobal != null); }

```

El método suite listado a continuación agrupa los test en la plataforma y por último run los ejecuta.

```

//-----
// se agrupan los test a ejecutar
//-----
public static Test suite() {
    return new TestSuite(TestAdminPagosServicio.class); }
//-----

```



```
// se ejecutan los test
//-----
public static void main (String[] args) {
    junit.textui.TestRunner.run(suite()); }
}
```

Cada uno de los test que se listan a continuación invoca la funcionalidad del cálculo de los pagos de los servicios y verifica la cantidad de pagos contra el resultado esperado en cada caso mediante el método `assertEquals`.

```
//-----
// test que prueban los pagos de los servicios
// y verifican los resultados
//-----
public void testCalcularPagosServicioChico() {

    ServicioLocal.calcularPagos();
    Collection<Pago> pagosServicioLocal = ServicioLocal.getPagos();
    assertEquals(new Integer(pagosServicioLocal.size()), new Integer(1));
}

public void testCalcularPagosServicioMedio() {

    ServicioDoméstico.calcularPagos();
    Collection<Pago> pagosServicioDoméstico = ServicioDoméstico.getPagos();
    assertEquals(new Integer(pagosServicioDoméstico.size()), new Integer(2));
}

public void testCalcularPagosServicioGrande() {

    ServicioGlobal.calcularPagos();
    Collection<Pago> pagosServicioGlobal = ServicioGlobal.getPagos();
    assertEquals(new Integer(pagosServicioGlobal.size()), new Integer(3));
}
}
```

16.7 Conclusión

Hemos tratado en este capítulo los temas relacionados a las pruebas de software y presentado conceptos fundamentales: niveles, tipos y métodos de prueba así como los principios de prueba que se mantienen vigentes.

También hemos notado que debido a los cambios en las prácticas de desarrollo generados por los cambios en el modelo de desarrollo hemos enfatizado la evolución desde la forma tradicional a la moderna de realizar las pruebas. La primera situaba estas actividades en la secuencia de tareas luego de la codificación y la nueva

promueve la realización adelantada en el ciclo de vida de los proyectos de estas actividades con la participación de los diferentes roles.

Las claves de esta nueva forma de probar el software se basa en dos conceptos fundamentales: la automatización y la prueba como criterio de diseño. Para que la primera sea factible es necesario sistematizar las pruebas y para la segunda que estén presentes los test, que deben tener las propiedades de ser atómicos e independientes lo cual se logra incorporando las pruebas como criterio de diseño. Ambos aspectos fueron tratados en este capítulo. En el capítulo próximo trataremos la automatización en el proceso de pruebas de software.

16.8 Contenido de la página web de apoyo



16.8.1 Mapa conceptual

16.8.2 Autoevaluaciones

16.8.3 Presentaciones en Power Point (exclusivo para el docente)

16.9 Referencias

- [1] *Calidad en el Desarrollo de Software*, Guillermo Pantaleo, Alfaomega, 2011.
- [2] *The Art of Software Testing, First and Second Edition*, Glenford J. Myers, John Wiley & Sons, Inc., 1979 y 2004.
- [3] *Software Verification and Validation for Practitioners and Managers, Second Edition*, Steven R. Rakitin, Artech House, 2001.
- [4] *xUnit Test Patterns: Refactoring Test Code*, Gerard Meszaros, Addison-Wesley, 2007.
- [5] *Fit for Developing Software: Framework for Integrated Tests*, Rick Mugridge, Ward Cunningham, Prentice Hall PTR, 2005.
- [6] *Software Configuration Management Patterns: Effective Teamwork, Practical Integration (Software Patterns Series)*, Stephen P. Berczuk (Author), et ál., 2008.
- [7] *Plataforma JUnit*, <http://www.junit.org/>.

Proceso de pruebas de software

Contenido

17.1 El proceso de prueba de software	286
17.2 Automatización de las pruebas	291
17.3 Integración continua	293
17.4 Conclusión	301
17.5 Contenido de la página web de apoyo	301
17.6 Referencias	301

Objetivos

- Entender el esfuerzo del proceso de prueba de software
- Cómo planificar las tareas de pruebas en un proyecto de desarrollo
- Hasta cuándo probar
- Por qué y qué automatizar en el proceso de prueba
- Cómo organizar la automatización de las pruebas
- Aprender el concepto y las prácticas de Integración continua

Competencias

- Establecer la forma en que será probado el software de un proyecto, qué alcance tendrán las pruebas, cuáles herramientas se utilizarán y qué recursos se necesitarán para estas tareas

17.1 El proceso de prueba de software

Uno de los resultados más positivos de la evolución del proceso de pruebas analizado en el capítulo anterior fue la instalación de la cultura de la prueba en los grupos de desarrollo que optaron por el cambio. Esto puede ser graficado como que el esfuerzo de los desarrolladores por implementar determinada funcionalidad no se acaba con una bella interfaz de usuario ni con un flujo de control bien logrado, ni con la persistencia precisa de los datos manipulados, ni con la aislación de las transacciones asociadas; sino con todo esto terminado, compilado, probado por un conjunto de test y subido al repositorio del proyecto. Este ciclo que comienza con el refinamiento de requerimientos, pasa por el análisis y diseño detallado de estos hasta la codificación y prueba que muestran el funcionamiento esperado genera una sensación de seguridad en los desarrolladores que se transmite a la organización y está sustentada por el conjunto de test que en todo momento determina la calidad del desarrollo hasta ese momento. Este ambiente de confianza permite realizar un mejor trabajo por parte de todos los involucrados, incluyendo a los clientes. Este logro no es gratuito, fue necesario contar con herramientas que permitieran a los programadores escribir los test en el mismo lenguaje de programación del proyecto así como también contar con plataformas del estilo xUnit que permitieran ejecutar los test tantas veces como fuera necesario con el mínimo esfuerzo. Las organizaciones que han logrado incorporar estos cambios generan productos de mejor calidad a menor costo y además sus desarrolladores pueden dedicarse a tareas más creativas que suman valor a los productos de software generados al ser liberados del trabajo tedioso y repetitivo de la ejecución de pruebas manuales.

17.1.1 Plan de pruebas

Para la realización de cualquier tarea es importante el ejercicio de planificación para llevarlas adelante sin omisiones y en forma ordenada y controlada [4].

Las pruebas de software son uno de los ingredientes que ayudarán a garantizar la calidad de los productos generados. Los otros son las revisiones, de análisis, diseño y código, y el desarrollo de procesos pensados y definidos de forma inteligente para cada proyecto que permitan recorrer el ciclo de vida del proyecto de manera controlada.

Este ejercicio de planificación, como en todos los casos, será más formal cuanto mayor sea el volumen del proyecto. Este ejercicio será volcado en un plan de pruebas cuyo documento complementará el del proyecto.

Como todo plan deberá contemplar:

- Los recursos humanos necesarios para la realización de las tareas planificadas.
- La infraestructura de hardware y software; es decir que debe incluir la definición del ambiente en el cual se realizarán las pruebas.

Además deberá incluir:

- La estrategia de diseño y ejecución de las pruebas.
- Prioridades.
- Objetivos.
- Acciones a tomar ante las fallas detectadas.

- Modelo de Informes de resultados.
- Criterios de comienzo y finalización.
- Coordinación de tiempos con el resto de las tareas del proyecto.
- Métricas a recolectar.
- La asignación de roles a efectos de determinar quienes son los responsables del diseño, construcción, ejecución y análisis de los resultados de las pruebas.

17.1.1.1 Caso testigo

En el proyecto del caso testigo se decidió lo siguiente respecto de los ítems de la planificación de las pruebas mencionados:

Los test unitarios y de componentes serán diseñados, construidos y ejecutados por los desarrolladores. Para esto el analista, un tester contratado a una empresa de servicios de pruebas, y el líder técnico definieron los criterios con que estos test serán implementados (decisiones, condiciones, clases de equivalencia y valores límites).

Los test de funcionalidad y aceptación serán diseñados y construidos por el analista que trabajará conjuntamente con los desarrolladores y contará con la participación de los involucrados por parte del cliente. Serán ejecutados en forma automática durante el desarrollo y manualmente por el analista y el tester contratado durante la entrega y aceptación de los productos del proyecto.

Para la realización de las pruebas se dispuso la instalación de un servidor con todo el software de base necesario para la realización de cada uno de los test. En una próxima sección se detallan estos componentes. Los desarrolladores ejecutan los test unitarios en sus estaciones de trabajo. Se decidió utilizar herramientas de tipo xUnit en todas las fases de prueba, incluyendo Fit-Fitnesse para las pruebas funcionales, los test de Watir para la capa de presentación y JMeter para las pruebas de sistema. Se construyeron planillas para las pruebas manuales en base a modelos estándares predefinidos en la organización.

Las pruebas se trataron con la misma prioridad que los requerimientos en todas sus fases, desde su diseño hasta la corrección de los errores asociados. Las fallas detectadas se priorizaban y se sumaban a la tareas de la próxima iteración del ciclo de vida del proyecto.

Con la terminación de cada iteración de duración fija, se comienza la ejecución de los test mientras se planifica y refina la próxima iteración. Los informes que se utilizan son los generados por las herramientas de prueba que son recopilados a partir de la ejecución de un servidor de integración continua como se describe en una próxima sección.

Las métricas son recolectadas desde los informes mencionados y básicamente están orientadas a determinar la estabilidad del proyecto a partir de la cantidad de errores detectados y corregidos a lo largo del ciclo de vida.

17.1.2 Qué probar

Además de probar todas las funcionalidades implementadas haciendo un análisis conducido por los casos de uso en la construcción de los casos de test, deberán probarse aspectos de las aplicaciones asociadas

a los requerimientos no funcionales. Algunos de estos requerimientos pueden ser más críticos que otros y en ese caso deberá invertirse mayor esfuerzo en su prueba. Una práctica común es guiarse a partir de una clasificación o taxonomía de atributos de calidad y de los riesgos generados por las posibles fallas en estos a la hora de establecer una estrategia de prueba [3]. Otra práctica en la utilización de taxonomías es utilizar la generación de fallas y guiarse por el impacto de los efectos de ellas sobre el sistema en desarrollo. En la tabla que sigue, tabla 17.1, se presenta una clasificación no exhaustiva como guía de selección de pruebas a realizar sobre un software en desarrollo

Tabla 17.1. Clasificación de fallas guía de aspectos a probar			
Alcance	Error	Falla	Guía de prueba
Interfaces de usuario	Inconsistencia en la estructura	Confusión al utilizar la aplicación	Pruebas sistemáticas de interfaces de aplicación
	Falta de controles	Imposibilidad de acceder a determinada funcionalidad	Pruebas sistemáticas de interfaces de aplicación
	Acciones incorrectas	Flujo de trabajo erróneo	Cubrimiento de flujos básicos y alternativos
Manejo de errores	Falta de detalle en la publicación	Dudas al momento de tomar decisiones	Cubrimiento de casos negativos y definición de excepciones de negocio
Requerimientos	Omisión	Funcionalidad no implementada	Probar trazabilidad
	Ambigüedad	Comportamiento indefinido y confuso	Cubrimiento exhaustivo con casos negativos y positivos
	Inconsistencia	Comportamiento contradictorio	Cubrimiento exhaustivo con casos negativos y positivos
	Falta de rigurosidad en la especificación	Comportamiento aleatorio	Probar con tablas de decisiones y estados
Diseño	Acoplamiento	Dificultad de probar aspectos aislados y de reusar módulos	Revisiones de criterios de diseño (inversión de dependencia)
	Falta de flexibilidad ante cambios	Dificultad de incorporar cambios y nuevas funcionalidades	Revisiones de criterios de diseño (clausura de código)
	Dependencias cíclicas	Dificultad de fijar errores y hacer pruebas independientes	Revisiones de criterios de diseño (dependencias)

Tabla 17.1. Clasificación de fallas guía de aspectos a probar			
Alcance	Error	Falla	Guía de prueba
Funcionalidad	Requerimientos incompletamente o mal implementados	Función del negocio imposible de realizar	Trazabilidad
Uso	Sin reglas de usabilidad	Sistema no usado por los usuarios	Pruebas funcionales manuales
Mantenimiento	Dificultad de hacer correcciones	Mayores tiempos de corrección de fallas	Revisiones de criterios de diseño (inversión de dependencia)
Portabilidad	Poco portable	Difícil instalación, configuración y adaptación a otro ambiente operacional	Revisiones de criterios de diseño (arquitectura)

Dependiendo del análisis del impacto que estas fuentes de error tengan sobre los productos del proyecto, se decidirá invertir mayor esfuerzo en las pruebas orientadas a garantizar la calidad del aspecto relacionado.

17.1.3 Hasta cuándo probar

Algunas de las pautas para determinar hasta cuándo probar los productos generados en un proyecto de desarrollo de software surgen de *A Practitioner's Guide to Software Test Design* [3]:

Cubrimiento: determinada a partir de un análisis del software y la ayuda de alguna herramienta que permita medir dicho cubrimiento como Clover [1].

Frecuencia de detección de errores: determinada a partir de la ejecución continua de pruebas.

Costo: determinada a partir de los recursos disponibles para el proyecto.

Consenso: determinada a partir del acuerdo entre los miembros del grupo de desarrollo, incluyendo a los involucrados por parte del cliente, y en base a la información técnica y al análisis de los resultados de los test.

Decisiones corporativas: determinada a partir de decisiones corporativas relacionadas al negocio.

Si bien estas pautas siempre contribuyen e influyen la decisión de hasta cuándo probar el software, puede elaborarse una estimación a partir de números provenientes de estadísticas y formas de trabajo asociadas a las pruebas [2] que indican que después de realizar revisiones de código aún permanecen sin detectar 5 errores cada 100 líneas. Supongamos que se ha fijado como objetivo detectar y corregir el 98% de los errores de codificación y el 95% de diseño.

Para una aplicación de 1.000.000 líneas los errores residuales estimados según estas estadísticas serán:
Errores estimados: $1.000.000 / 100 * 5 = 50.000$.

Además estas estadísticas muestran una distribución de los errores cometidos por tarea del ciclo de vida que es el que se muestra en la tabla que sigue.

Tabla 17.2. Porcentajes de errores cometidos en proyectos de desarrollo de software por tarea realizada, adaptada de Applied Software Measurement: Assuring Productivity and Quality [7]

Distribución de errores por tareas	
Requerimientos	20 %
Diseño	25 %
Codificación	35 %
Pruebas	8 %
Documentación	12 %

A partir de estos porcentajes se pueden obtener las siguientes cantidades para nuestro sistema:

- Errores residuales de codificación: $50.000 \times 35 / 100 = 17.500$

Según los objetivos fijados, deberán detectarse y corregirse:

- Errores residuales de codificación: $17.500 \times 98 / 100 = 17.150$

Con respecto al diseño:

- Errores residuales de diseño: $50.000 \times 25 / 100 = 12.500$

Según los objetivos fijados, deberán detectarse y corregirse:

- Errores residuales de diseño: $12.500 \times 95 / 100 = 11.875$

Entonces, se probará hasta detectar y corregir esa cantidad de errores o hasta que se trabaje el tiempo equivalente al 50% del tiempo que demandó la codificación. Este porcentaje ha mostrado ser apropiado en la mayoría de los casos de sistemas de mediana complejidad. Todo esto con la condición previa de haber diseñado los test según los criterios analizados y de haber realizado revisiones de código como actividades de aseguramiento de la calidad dentro del proyecto.

El resto de los errores provenientes de las otras actividades serán detectados en las sesiones de revisión programadas como actividades a lo largo del ciclo de vida del proyecto. Estas actividades deberán ser más frecuentes y rigurosas cuanto mayor sea el volumen del proyecto. Se recomienda una revisión de la especificación de los requerimientos, una de la arquitectura y revisiones periódicas de diseño y código.

Dependiendo de las características del proyecto y de su volumen y complejidad se planificará el número de estas revisiones y se extenderán las ejecuciones de los test con la guía anterior.

17.1.3 1 Métricas

Una métrica de utilidad vinculada con la evolución de las pruebas es la cantidad de fallas o defectos detectados y la cantidad de corregidos.

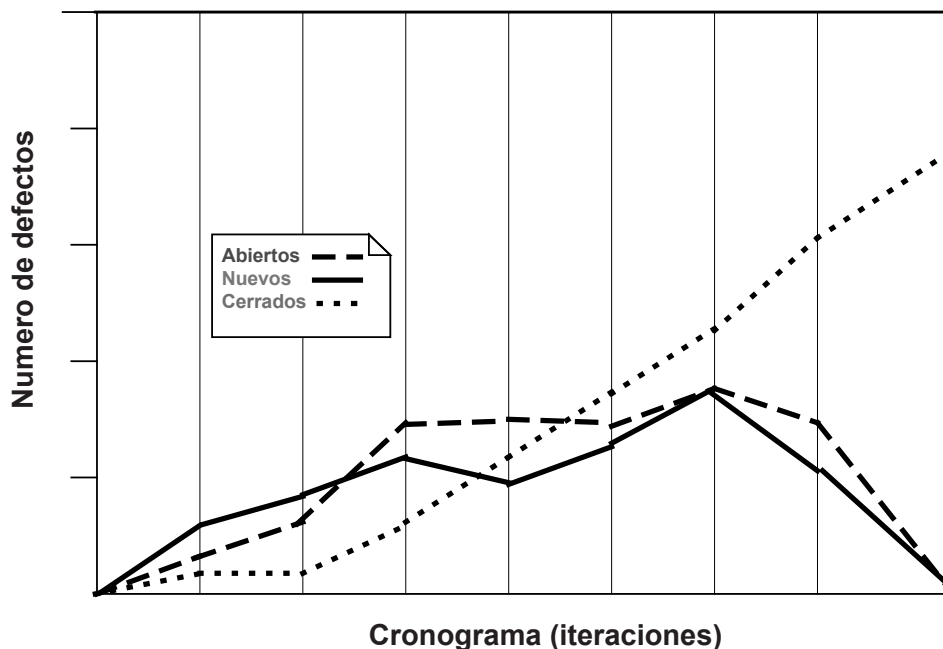


Figura 17.1. Evolución de la cantidad de defectos nuevos detectados y corregidos a lo largo del ciclo de vida de un proyecto

La evolución conjunta de estas cantidades muestra la tendencia de la calidad de los productos que se generan en el proyecto. Como se muestra en la figura 17.1, cuando los corregidos superan a los nuevos, es un indicativo de la estabilidad del proyecto. Si esto no fuera así, estaríamos en presencia de un proyecto en el cual la cantidad de fallas nuevas detectadas está por encima de las corregidas hacia el final de este. Esta última situación estaría mostrando un proyecto con serios inconvenientes y fuera de control.

17.2 Automatización de las pruebas

17.2.1 Por qué automatizar las pruebas

La necesidad de automatizar las pruebas radica en un mejor aprovechamiento de los recursos y en la confianza que genera en los desarrolladores y por lo tanto en la empresa para la cual trabajan la posibilidad de ejecutar repetitivamente los test con cada cambio del código del proyecto. Esta posibilidad facilita la respuesta a la pregunta: ¿qué grado de avance posee el proyecto? Ya que lo concretamente desarrollado y probado

es lo que puede observarse a través de la ejecución de un servidor de pruebas e integración que funcionan continuamente.

17.2.2 Qué debería automatizarse

El objetivo es contar con la mayor cantidad y tipo de pruebas automatizadas. En casi todos los niveles de prueba es factible automatizar algunos test, otros aún no son posibles ya que las herramientas no están maduras y no son confiables y otros son necesariamente realizados en forma manual ya que necesitan de la participación de alguno de los roles interactuando con otros, como ser las pruebas exploratorias o las de aceptación en las cuales participan los analistas, los tester y los involucrados por parte de los clientes. Entre las pruebas a automatizar están entonces:

- Pruebas unitarias y de componentes.
- Pruebas de funcionalidad sin interfaces de usuario.
- Pruebas de sistema con interfaces de usuario.

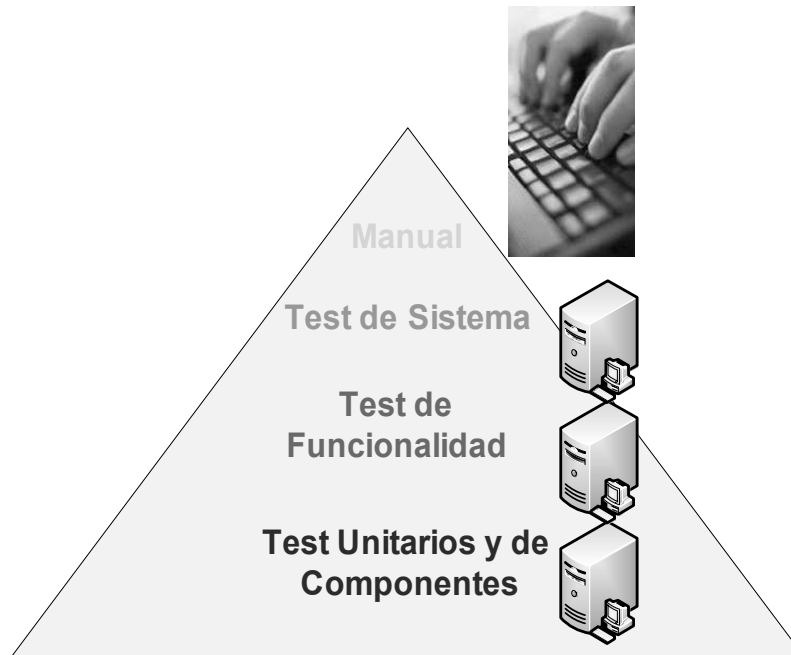


Figura 17.2. Esquema de la pirámide de pruebas donde se muestran los aspectos a automatizar

Las pruebas mencionadas que no admiten automatización debido a la necesidad de la participación de los diferentes roles son:

- Pruebas de uso, orientadas a la utilización de las aplicaciones.
- Pruebas exploratorias, orientadas al aprendizaje y la comprensión de diferentes aspectos de las aplicaciones y el negocio.

- Pruebas que no fallarán, debido a que constituyen esfuerzo mal gastado.
- Tareas únicas de fácil ejecución manual y difícil automatización.

17.2.2.1 Estrategia para comenzar la automatización

Debido a que el desarrollo de software que utiliza pruebas automatizadas y criterios de diseño que incluyen las pruebas implican una forma de trabajo diferente a la tradicional, debe planificarse este cambio cuidadosamente para que sea adoptado en los grupos de desarrollo. Algunos pasos de esta planificación deberían ser:

Capacitación a analistas, tester y programadores.

- Seleccionar una forma de trabajo cohesiva e integrada entre todos los desarrolladores.
- Seleccionar herramientas que faciliten el conjunto de actividades.
- Desarrollar proyectos pilotos donde se experimente con la nueva forma de trabajo.
- Institucionalizar la nueva forma de trabajo al resto de los proyectos.

17.3 Integración continua

17.3.1 Fases del flujo de trabajo

Si bien hemos resuelto alguno de los problemas planteados con relación a las pruebas del software, aún restan otros para los que propondremos ahora una solución. Por cierto, propusimos la mejora de la calidad de nuestros diseños incorporando las pruebas como criterio y sistematizamos dichas pruebas con la utilización de las herramientas de tipo xUnit. Veamos entonces una forma de automatizarlas y de extraer indicadores que nos guíen en la selección de productos de software (diseño y código) a revisar. En la figura 17.3 se muestra un esquema de un ambiente de trabajo de Integración Continua [1]. La dinámica de la forma de trabajo se indica en los pasos que se listan a continuación.

17.3.1.1 Pasos

1. Un desarrollador realiza un commit (cambios) sobre el repositorio (SCM) mientras el administrador de integración continua (IC) lo consulta por cambios con una frecuencia determinada.
2. Después del commit el administrador de CI detecta el cambio, toma del repositorio las últimas versiones y ejecuta los scripts que integran todo el software (compila y linkea), ejecuta los test, toma métricas y elabora informes.
3. El administrador de IC informa por mail acerca de los resultados a los miembros del grupo de desarrollo de los resultados del Build.
4. El administrador de IC continúa consultando al repositorio (SCM) con la frecuencia determinada.

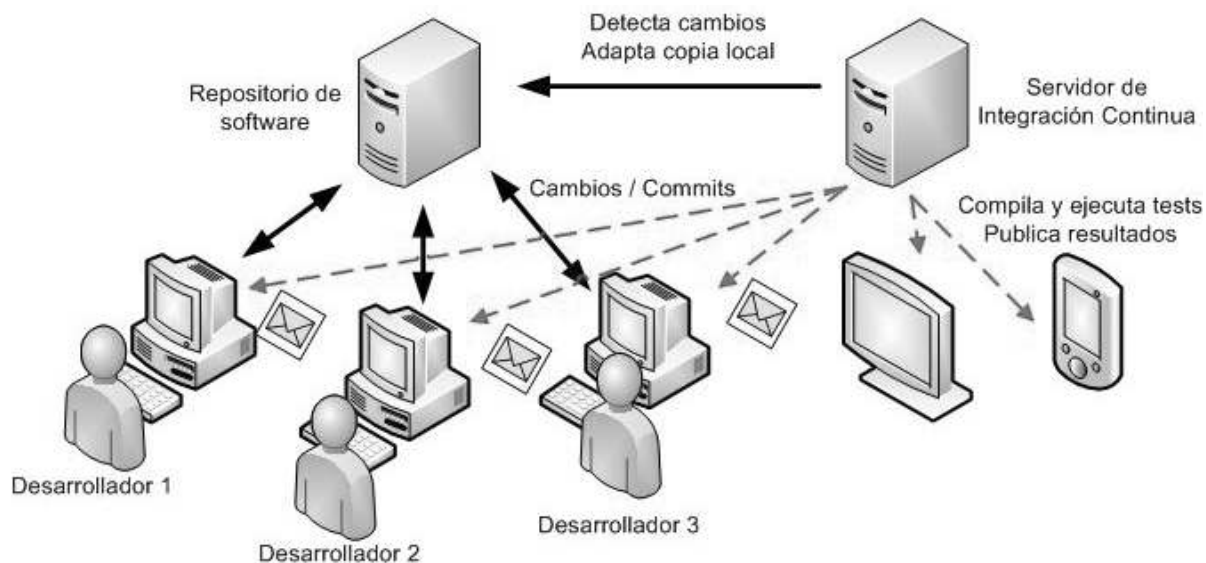


Figura 17.3. Esquema de un ambiente de trabajo que utiliza Integración Continua

17.3.1.2 Principios

Los criterios sobre los que se fundamenta esta arquitectura se resumen en los principios mostrados en la tabla 17.3, adaptados de *Continuous Integration: Improving Software Quality and Reducing Risk* [6], y están agrupados por componente del esquema de trabajo y tarea a realizar sobre ellos. Se observa que todos y cada uno de estos principios está orientado a la incorporación de buenas prácticas que eviten problemas conocidos y cometidos en todo grupo de desarrollo. Apuntan a organizar el ambiente de desarrollo de forma ordenada, clara y segura.

En la figura 17.4, de más abajo, se muestra la nueva forma de trabajo en la que se puede observar la influencia de las pruebas durante el desarrollo (relación más gruesa en el dibujo entre analistas, tester y programadores) y además se ha incorporado un nuevo componente en este escenario. A efectos de sistematizar y automatizar las pruebas para contar con una realimentación temprana y con alta frecuencia aparece un servidor de integración continua que ante cada cambio en el repositorio actualiza su ambiente, construye los binarios, ejecuta los test e informa de los resultados. Además construye informes de calidad de diseño, código y resultados obtenidos de las pruebas [1].

Trabajo con tests automatizados

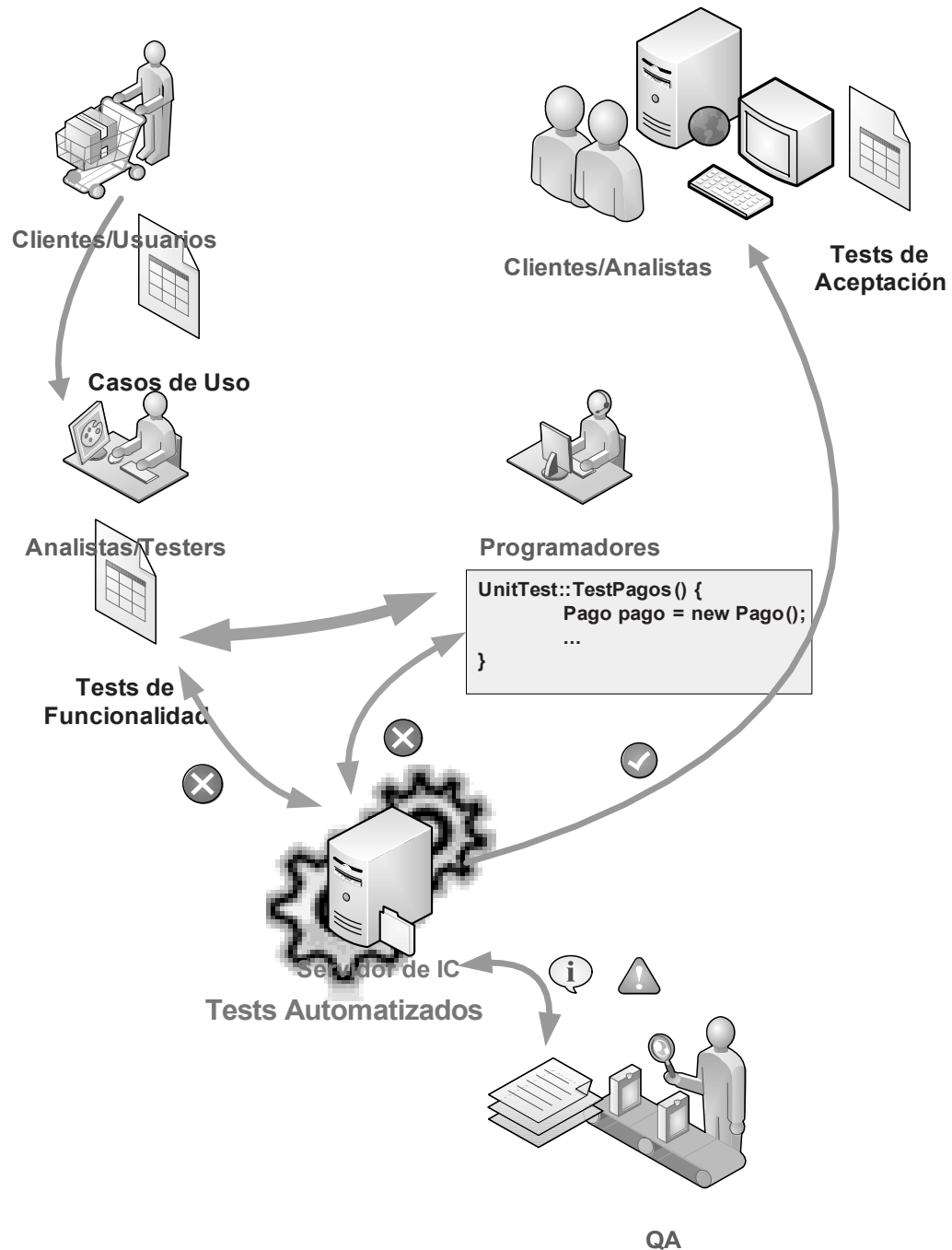


Figura 17.4. Flujo de trabajo utilizando integración

Tabla 17.3. Listado de principios de Integración Continua	
Principios de Integración Continua	
Repositorio	Construya binarios con cada cambio
	Haga commit con frecuencia
	Corrija errores que generan fallas de los test en forma inmediata
	No haga commit de código no probado
	Nombre cada entregable, indique con cuáles componentes fue construido y genere capacidad para volver atrás
Construcción de binarios	Automatizar
	Separar del IDE (<i>integrated development environment</i>)
	Ejecute comandos simples
	Crear estructura de directorios consistente
	Automatice la integración de la base de datos
	Utilice una copia privada de la base de datos
Pruebas y revisiones	Automatice y categorice los test
	Ejecute los test en momentos diferentes
	Escriba test para fijar errores
	Haga a los test repetibles
	Controle el cubrimiento de los test
	Defina estándares de la organización para los diseños y el código mantenidos por QA
	Promueva el uso de estándares ejecutando revisiones guiadas por métricas
	Tome métricas sobre el diseño y el código
Integración Continua	Utilice un servidor independiente para la integración continua
	Provea información para la publicación de resultados a los involucrados
	Publique resultados de la integración en forma continua
	Genere informes con cada ejecución y guárdelos
Informes	Construya informes y publíquelos a los desarrolladores y a QA
	Promueva la lectura de los informes

17.3.2 Infraestructura

En la tabla 17.4 pueden verse algunas de las posibles herramientas para conformar una plataforma de Integración continua. La presentada es solo una posible arquitectura usada por nosotros en algunos proyectos donde se utilizó tecnología Java. La misma es la utilizada en el proyecto del caso testigo. Una referencia donde pueden encontrarse vínculos a otras herramientas es el libro de Duvall [6].

Tabla 17.4. Herramientas según su rol conformando una plataforma de Integración Continua		
Descripción de herramientas por rol		
Rol	Herramienta	Propósito
Repositorio	Subversion 1.4.2	Administración del repositorio
		Organización del proyecto
Construcción de binarios	Maven 2.0.9	Definición de ciclo estándar de generación
		Generación de binarios
		Definición de directorios
		Administración de dependencias
		Administración de repositorios de binarios
		Generación automática de entregables
		Integración de plugins de informes
	Ant 1.7.1	Ejecución de scripts de construcción
Pruebas y métricas para revisiones	JUnit 3.8.1	Pruebas automáticas y sistemáticas
	JDepend	Analizar la relación entre paquetes
	ChecStyle	Chequear los estilos de programación
	PMD/CPD	Detectar problemas de programación
	Clover	Controlar el cubrimiento de los test
	JavaDoc	Documentar código del sistema y test
	Doxygen	Documentar diseño desde el código
	JIRA 3.13	Seguimiento de problemas y errores
	Enterprise Architecture	Documentación del proyecto
Integración Continua	Cruise Control 2.7.3	Detección de cambios en el repositorio
		Actualización de su ambiente de trabajo
		Ejecución de las fases de construcción de Maven
		Informar a involucrados de los resultados
		Almacenar historia de ejecuciones
Informes	Tomcat 5.5	Publicar informes con resultados

En las referencias al final del capítulo se presentan vínculos a los sitios Web de las herramientas mencionadas.

17.3.3 Resultados

Un aspecto de gran importancia que deseamos volver a mencionar es la posibilidad que ofrece Maven y Cruise Control de generar un sitio para el proyecto donde se publican los informes con resultados y métricas.

Este sitio es el punto de referencia de los involucrados en el proyecto así como para los miembros del área de calidad de la organización, ya que ellos encontrarán información de gran utilidad para realizar su trabajo con relación al proyecto.

En la figura 17.5 se presenta una imagen de la portada de uno de estos sitios.

Es importante tener en cuenta que la información allí publicada es actualizada con cada construcción de binarios, por lo que constituye la fuente de información más actualizada y precisa del proyecto. Estas herramientas permiten acoplarles módulos para la generación de informes de revisión de estilo de código, diseño, documentación de código, cubrimiento de los test, y otros. Toda esta información es actualizada y completa, la cual contribuye a la programación de las actividades de revisión de productos de software.

De esta forma hemos logrado los objetivos iniciales planteados en el comienzo del capítulo en relación con la visibilidad y la actualidad de la información deseable a efectos de respondernos la pregunta: ¿qué hemos desarrollado y qué falta desarrollar?

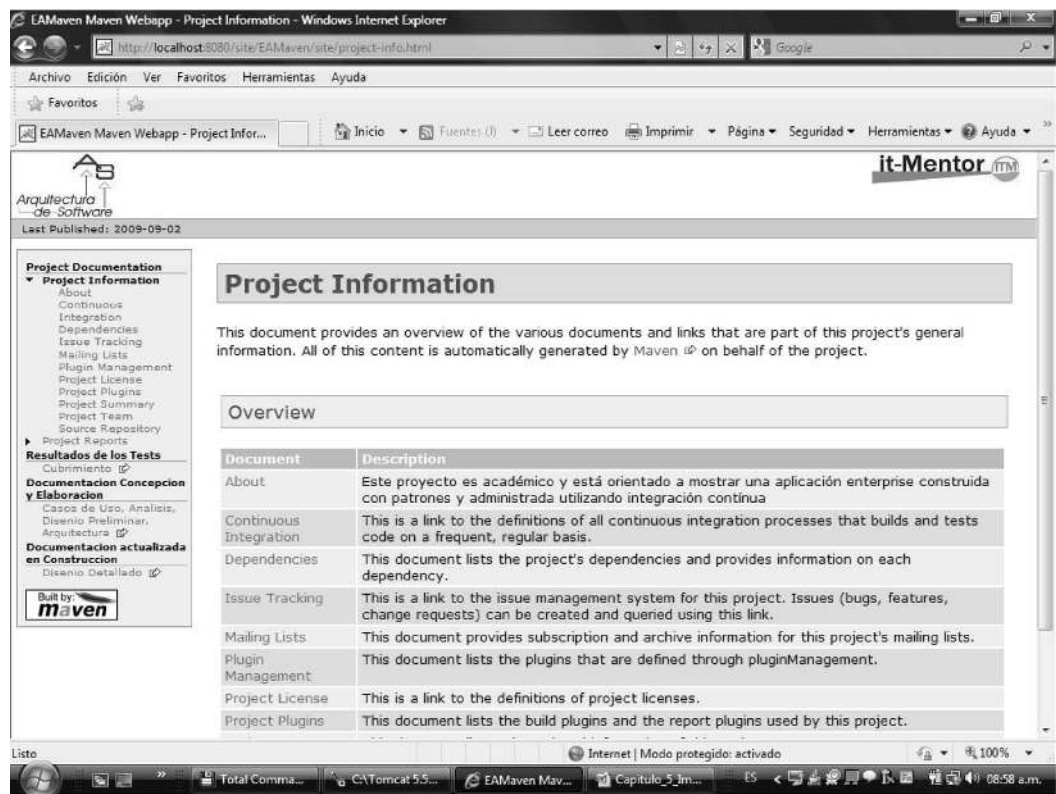


Figura 17.5. Sitio de un proyecto Maven2

En la figura 17.6 se muestra una vista del mismo sitio correspondiente al informe de cubrimiento de los test construida con Clover [14].

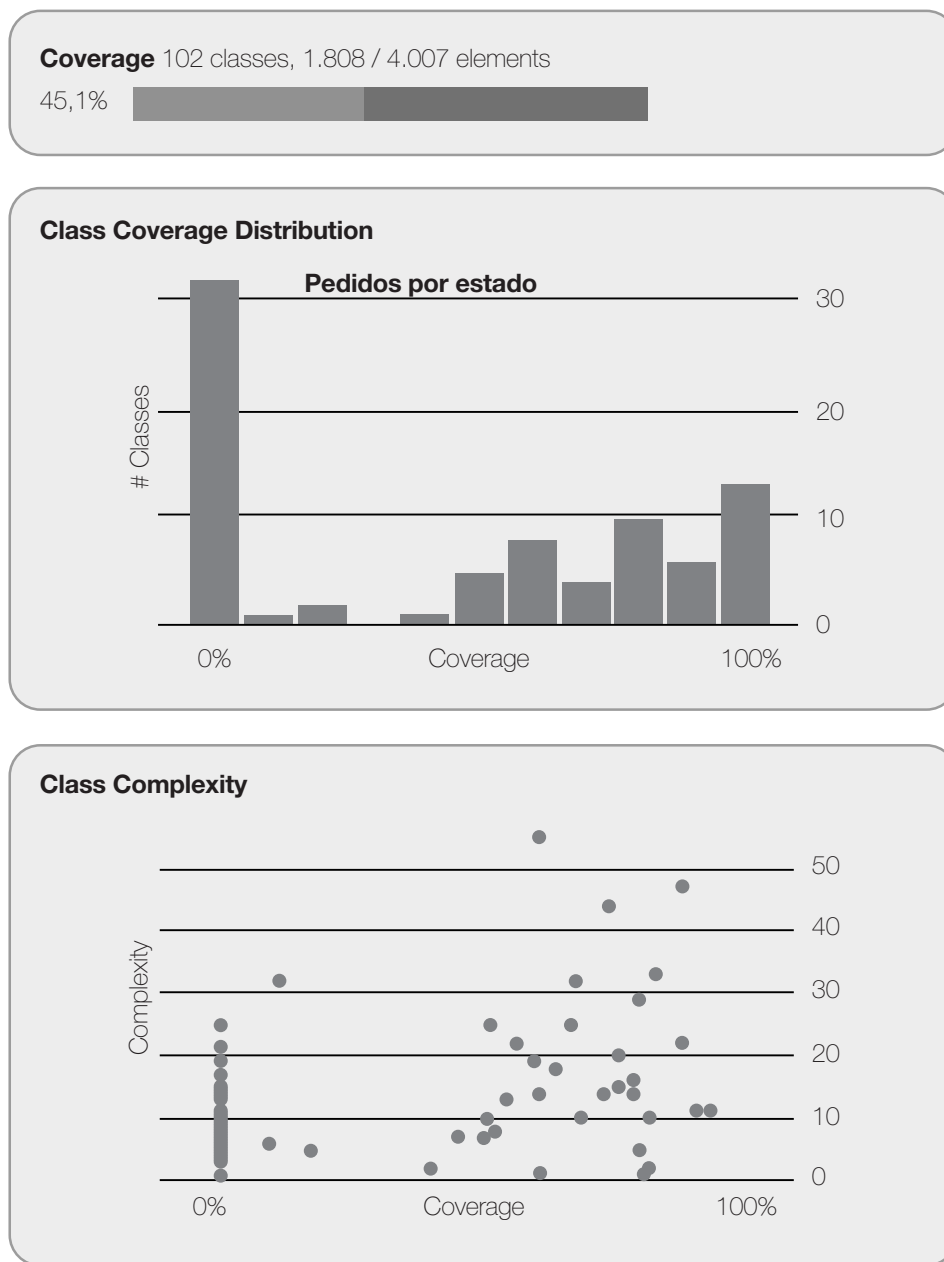


Figura 17.6. Vista del sitio construido con Maven2 donde se muestra la cobertura de los test, elaborada con Clover [14]. En ella se observa el cubrimiento porcentual de los test y la distribución de las clases de acuerdo a su cubrimiento y complejidad

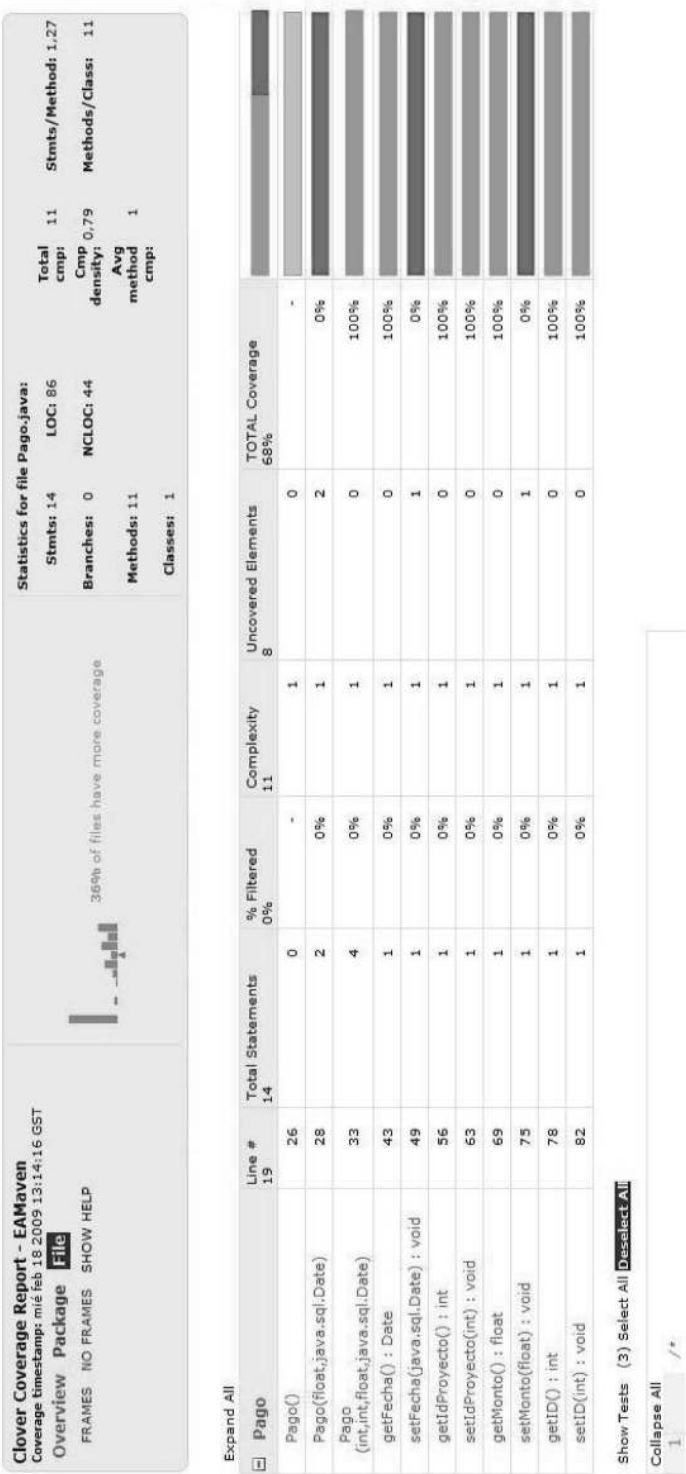


Figura 17.7. Vista generada por Clover [14] donde se muestra el cubrimiento de los test para los métodos de la clase Pago del proyecto del caso testigo

17.4 Conclusión

La realización de las pruebas del software de un proyecto de desarrollo debe constituirse en una fuente de reaseguro de que el proyecto evoluciona de manera estable. Para ello es necesario diseñarlas, construirlas y ejecutarlas de manera temprana en el ciclo de vida. Por eso se propone concebir las pruebas como criterio de diseño que permita mejorar la calidad de los productos generados y contar con la posibilidad de ejecutarlas con frecuencia y con mínimo esfuerzo. Utilizando herramientas de tipo xUnit y la forma de trabajo descripta en este capítulo como Integración Continua se logra este objetivo. Como hemos analizado no se trata de mudar las herramientas por otras más nuevas sino de adoptar una nueva forma de trabajo que afecte a todos los miembros del grupo de desarrollo. Este proceso de pruebas de software garantiza el cumplimiento de los principios listados en el capítulo anterior como fundamentales y vigentes desde hace mucho tiempo en relación con las actividades que garantizan productos de buena calidad.

17.5 Contenido de la página web de apoyo



17.5.1 Mapa conceptual

17.5.2 Autoevaluaciones

17.5.3 Presentaciones en Power Point (exclusivo para el docente)

17.6 Referencias

- [1] *Calidad en el Desarrollo de Software*, Guillermo Pantaleo, Alfaomega, 2011.
- [2] *The Art of Software Testing, First and Second Edition*, Glenford J. Myers, John Wiley & Sons, Inc., 1979 y 2004.
- [3] *A Practitioner's Guide to Software Test Design*, Lee Copeland, Artech House, 2004.
- [4] *Software testing and Continuous Quality Improvement*, 3rd edition, William E. Lewis, CRC Press, 2009.
- [5] *Fit for Developing Software: Framework for Integrated Tests*, Rick Mugridge, Ward Cunningham, Prentice Hall PTR, 2005.
- [6] *Continuous Integration: Improving Software Quality and Reducing Risk*, Paul M. Duvall with Andrew Glover and Steve Matyas, 2008.
- [7] *Applied Software Measurement: Assuring Productivity and Quality*, Capers Jones, 1996.

Herramientas

1. *Subversion*, <http://subversion.tigris.org/>.
2. *Ant*, <http://ant.apache.org/>.
3. *Maven*, <http://maven.apache.org/>.
4. *Eclipse*, <http://www.eclipse.org/>.
5. *Junit*, <http://junit.sourceforge.net/>.
6. *Tomcat*, <http://tomcat.apache.org/>.

7. *CruiseControl*, <http://cruisecontrol.sourceforge.net/>.
8. *CheckStyle*, <http://checkstyle.sourceforge.net/>.
9. *JavaDoc*, <http://java.sun.com/j2se/javadoc/>.
10. *Doxygen*, <http://www.stack.nl/~dimitri/doxygen/>.
11. *PMD/CPD*, <http://pmd.sourceforge.net/cpd.html>.
12. *JDepend*, <http://clarkware.com/software/JDdepend.html>.
13. *Enterprise Architecture*, <http://www.sparxsystems.com.ar/>.
14. *Clover*, <http://www.atlassian.com/software/clover/>.
15. *JIRA*, <http://www.atlassian.com/software/jira/>.

Gestión de proyectos
Estimación y planificación

Contenido

18.1 Gestión de proyectos de desarrollo de software	304
18.2 Estimación	304
18.3 Planificación	307
18.4 Conclusión	322
18.5 Contenido de la página web de apoyo	322
18.6 Referencias	322
18.7 Apéndice A – Cuestionario guía de relevamiento preliminar	323

Objetivos

- Cómo se venden los proyectos de desarrollo de software
- Estimaciones iniciales y propuesta técnico económica
- Por qué y cómo establecer acuerdos
- Cómo planificar cuando se utilizan metodologías conducidas por los planes
- Cómo planificar cuando se utilizan metodologías ágiles

Competencias

- Elaborar una propuesta técnico económica para un proyecto de desarrollo de software
- Planificar un proyecto de desarrollo de software

18.1 Gestión de proyectos de desarrollo de software

La gestión de los proyectos de desarrollo de software comprende la estimación, la planificación y el seguimiento. En este capítulo tratamos la estimación y planificación de los proyectos. En el capítulo próximo el seguimiento y en el otro mostramos estos conceptos aplicados al desarrollo del sistema del caso testigo.

18.2 Estimación

La gestión de un proyecto típico de desarrollo de software se caracteriza por tener tres etapas bien diferenciadas. La primera es la de Estimación (preventiva), dominada por la elaboración y el acuerdo de la propuesta Técnico Económica que la empresa desarrolladora propondrá a la empresa cliente a efectos de lograr que el desarrollo del sistema en cuestión le sea asignado. La segunda es la Planificación del proyecto y comienza cuando la propuesta es aceptada y se formalizan los acuerdos para el inicio del proyecto. Esta etapa tiene lugar durante los primeros días del proyecto. La tercera es la de Seguimiento y se desarrolla durante todo el ciclo de vida del proyecto [12]. En este capítulo describimos los objetivos y las actividades que se llevan adelante para lograrlos en cada una de las etapas mencionadas. También tratamos en forma separada algunos aspectos de la gestión de proyectos de desarrollo según se utilicen métodos conducidos por los planes o ágiles. En el próximo capítulo tratamos estos temas aplicados al proyecto del caso testigo.

18.2.1 Venta de proyectos de desarrollo de software

Por lo general los proyectos de desarrollo de software de sistemas se plantean y evolucionan de una manera determinada que es la que analizaremos. Un cliente posee determinadas necesidades y problemas que piensa podrían resolverse con la utilización de un sistema que piden se les provea. La mayor parte de los casos este sistema no existe en el mercado como tal y por lo tanto debe construirse, en algunos casos integrando partes o sistemas ya desarrollados. Algunas veces el cliente posee un sistema ya obsoleto, que por cambios del negocio ya no cubre todas sus necesidades y desea uno nuevo que sea una extensión del anterior. En algunos casos las empresas clientes poseen un departamento de sistemas con el cual se deben tratar algunos de los aspectos relacionados al proyecto de desarrollo, en otros el cliente no posee conocimiento técnico alguno.

Como vemos, la variedad de escenarios es amplia. Sin embargo todos poseen un elemento común: la incertidumbre acerca del sistema a construir. Lo interesante del caso es que las empresas clientes que perciben esta falta de certidumbre entienden que la forma de eliminarla es solicitando algunos números vinculados al proyecto de manera rápida, es decir lo antes posible. Las empresas desarrolladoras deben responder a esta solicitud con un precio y un tiempo de desarrollo.

La cuestión central es que para determinar ambos números es necesario conocer la información acerca del problema a resolver que será relevada y analizada durante el ciclo de vida del desarrollo, es decir cuando el proyecto haya comenzado. Por lo tanto cómo se construye una propuesta con información desconocida, que no sea riesgosa para ninguna de las partes y que sea razonable, es decir que permita a la empresa cliente organizar un presupuesto y la participación de sus miembros en el proyecto y que permita a la empresa desarrolladora organizar el proyecto, el grupo de desarrollo, la infraestructura de desarrollo y los aspectos financieros. Todo esto con la poca información de que se dispondrá al momento de la propuesta.

Este escenario es así en la Ingeniería de Software porque los sistemas a desarrollar si bien poseen aspectos comunes son todos diferentes, por lo tanto la incertidumbre inicial se irá achicando recién a medida que evolucione el proyecto pero debemos acordarlos en medio de esta incertidumbre aún no achicada.

El desarrollo de productos de software posee menos incertidumbre, ya que en general son desarrollados por sus mismos dueños y por lo tanto no está presente la desconfianza de dos partes que negocian. El proyecto se estructura en base a la inversión que la empresa está dispuesta a hacer en el producto. Sin embargo somos conscientes de que esta es una visión simplificada de la cosa, no ignoramos que muchas veces el contexto de los proyectos es más complejo de los aquí descriptos. Pero aún con esta simplificación pensamos que el descripto es lo suficientemente ilustrativo para poner a los lectores en clima. En la figura 18.1 se puede ver el llamado *cono de incertidumbre* en el cual se observa el nivel de falta de certeza a lo largo del ciclo de vida de un proyecto. Cabe mencionar que los hitos mencionados en la figura corresponden a una metodología no iterativa sino en cascada, sin embargo en los primeros momentos la incertidumbre es la misma. La incertidumbre inicial es de cuatro (4) veces la duración de la planificación final.

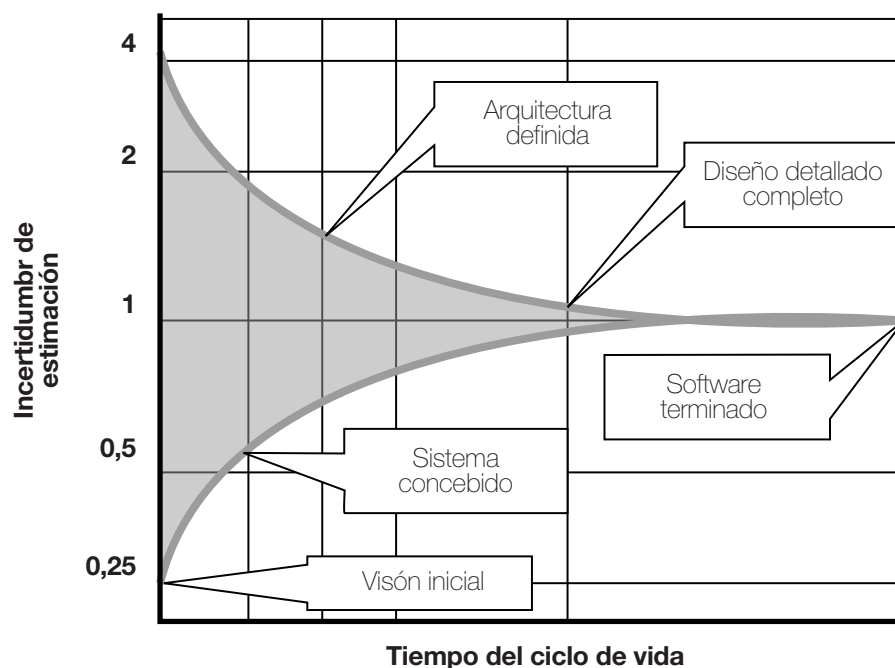


Figura 18.1. Cono de incertidumbre de un proyecto de desarrollo de software, se observa la variación en función de los hitos del proyecto

18.2.1.1 Estimaciones iniciales

La forma de trabajo cuando un cliente solicita una propuesta de desarrollo es la asignación de un número reducido de personas al proyecto en esta etapa de preventa. El objetivo es, como contamos más arriba, elaborar una propuesta técnico económica. Dependiendo de las características del problema se trabajará más o

menos tiempo para lograr hacer una presentación al cliente. Se acostumbra asignar un analista y un líder de proyecto que, trabajando con los clientes y futuros usuarios, releven la información necesaria para dimensionar el proyecto. Este trabajo es realizado a riesgo de la empresa desarrolladora de no ser contratada. En el caso de serlo, el costo de esta propuesta será pagada por el proyecto.

En el apéndice de este capítulo se muestra un cuestionario guía para determinar algunas de las características determinantes de un proyecto de desarrollo en su etapa de preventa.

18.2.1.2 Propuesta técnico/económica

Para la elaboración de esta propuesta se deben relevar requerimientos preliminares, detectar riesgos preliminares, definir la infraestructura inicial del proyecto para su ambiente de desarrollo y productivo, se debe estimar el alcance y esfuerzo, y volcar esto en una planificación inicial.

Esta planificación inicial establecerá números iniciales que el cliente tomará como definitivos. Estos son el alcance (requerimientos), el precio y el tiempo de desarrollo.

La dificultad de proponer esto radica en que los requerimientos no están lo suficientemente refinados y por lo tanto son expresados con cierto grado de ambigüedad, lo que implica que cuando haya que acordar el alcance una vez refinados dependerá de la interpretación de quien los lea. El precio fue acordado por el esfuerzo que fue determinado en base a los requerimientos ambiguos que mencionamos, lo que le transfiere una baja certeza, y por último el tiempo de desarrollo también depende de los requerimientos anteriores con la misma incertidumbre. Dependiendo de cómo se acuerde entre ambas partes la realización del proyecto, será el impacto que esta incertidumbre tendrá sobre ambas empresas.

18.2.1.3 Acuerdos

Existe en la literatura un tratamiento típico de este tema al cual se lo nombra como el *triángulo de control* de los proyectos. Los vértices del triángulo son el precio, el alcance y el tiempo de desarrollo del proyecto. En todos los textos se mencionan los tres como posibles variables de control de que dispone el que gestiona el proyecto. En todos los textos se dice que al menos una de estas magnitudes debe ser variable para que sea posible gestionar el proyecto. Sin embargo al momento de acordar los proyectos es muy difícil lograr que una de estas permanezca sin fijar, por lo tanto en la mayoría de los casos se cumple lo que los textos cuentan, en algún momento los proyectos se vuelven incontrolables. En las secciones que siguen describimos dos formas diferentes de acordar el proyecto entre las partes.

18.2.2 Metodología conducida por los planes

Negociación de contrato

Los acuerdos en las metodologías conducidas por los planes se basan en contratos que expresan las obligaciones y los derechos que las dos partes, empresa desarrolladora y cliente, tendrán con relación al proyecto de desarrollo. Una cuestión central en esta forma de acuerdo es que ambas partes durante el ciclo de vida del

proyecto defenderán sus intereses cuando estos se vean afectados por la incertidumbre analizada y hagan valer lo expresado en el contrato. Por lo tanto cuando el tiempo pase y se detecte que no es suficiente para construir el software que cubra todos los requerimientos se tratará de quitar del alcance alguno de estos o alargar el tiempo de desarrollo a cambio de un mayor precio. Estos planteos serán rechazados por la empresa cliente.

Cuando se refinan los requerimientos los usuarios de los clientes pedirán agregados y cambios a los requerimientos iniciales para que sean incorporados al proyecto sin alteración alguna de la planificación original. Estos planteos serán rechazados por la empresa desarrolladora. El contrato es el instrumento que se elige para resolver estas cuestiones de confrontación de intereses. El proyecto transcurrirá en medio de estas disputas. La gestión por parte de la empresa desarrolladora tendrá como objetivo cumplir con los hitos fijados en la planificación, controlando el trabajo realizado por los desarrolladores y mostrando dicho trabajo realizado, mientras se negocian las confrontaciones mencionadas. En algunos casos estas se dirimirán replanificando parte del proyecto, es decir cambiando algunos tiempos, como resultado del acuerdo de la realización de cambios a lo ya acordado.

18.2.3 Metodologías ágiles

Acuerdo de colaboración

En las metodologías ágiles la gestión que se propone necesita de un acuerdo y no de un contrato para defender intereses. Para que esto sea posible la empresa desarrolladora debe convencer a la empresa cliente de las ventajas que esta forma de trabajo conlleva si se realiza bien. En lugar de fijar las tres variables de control mencionadas, se fijan dos y se deja una libre. Se establece un tiempo para la duración del proyecto y un precio a pagar. El alcance se deja abierto y se establece que este será maximizado, es decir que el cliente al cabo del tiempo que dure el proyecto y pagando una cantidad fija sabe que puede esperar un sistema que implementará la solución a los requerimientos previamente priorizados. De esta manera la empresa desarrolladora gestiona el proyecto de forma de realizar todas las tareas con el objetivo de que los entregables cubran la mayor cantidad de requerimientos según prioridades determinadas por el cliente. El interés de ambas partes es, en todo momento durante el ciclo de vida, maximizar los productos de software que se generen. Como se intuye de este planteo, para que este escenario sea posible se necesita una gran confianza mutua para afrontar así la incertidumbre derivada de los proyectos de desarrollo como se analizó en una sección anterior. La clave de esta forma de trabajo está en alinear los intereses en lugar de contraponerlos.

18.3 Planificación

Una vez que el proyecto fue asignado, la empresa desarrolladora comenzará con la planificación del proyecto con la participación de los miembros de la empresa cliente. Este trabajo de planificación tiene distinto volumen e intensidad según sea la metodología seleccionada, conducida por los planes o ágil, para llevar adelante el proyecto. En el primer caso se avanzará en todos los aspectos de la planificación hasta el detalle [1], [11], en el segundo se planificarán algunos aspectos necesarios y se dejarán otros para cuando surja la necesidad de ocuparse de ellos [3]. Sin embargo deseamos enfatizar que siempre es necesario realizar el ejercicio de planificación y volcar su resultado en un plan de proyecto con mayor o menor formalidad. Este plan de proyecto

constituye un aspecto diferenciador entre ambos grupos de metodologías. En un caso se construye un plan que guiará a todos los involucrados en todo el ciclo de vida, que expresa los objetivos a seguir y que en raras ocasiones se replanificará para cambiarlo. En el otro, el plan es solo una guía superficial, un mapa sin detalles del cual se cambiará lo necesario cuando sea necesario. Esta necesidad surgirá de un seguimiento cercano del proyecto siempre siguiendo el objetivo de maximizar los requerimientos que se satisfaga.

18.3.1 Metodología conducida por los planes

18.3.1.1 Plan de Proyecto

Un plan de proyecto es el componente común a todas las formas de trabajo conocidas, conducidas por los planes y ágiles. Es conveniente separar el documento que lo representa del plan en sí mismo. Queremos rescatar el ejercicio de planificación frente al documento, sin embargo uno es consecuencia del otro. Muchos miembros de la comunidad informática que adhieren a alguna metodología ágil reniegan de la documentación al verla como artefactos innecesarios, consumidores de esfuerzo y / o basura al decir de Lean Programming [13]. Esto es consecuencia de que existen malas interpretaciones por parte de los usuarios de las metodologías y malas publicaciones por parte de quienes las promueven [6]. Si un proyecto no es trivial y demanda el esfuerzo de por lo menos un par de desarrolladores y un cliente externo, es necesario contar con un documento que refleje el ejercicio de planificación mencionado. Los proyectos que arrancan con sus desarrolladores escribiendo código por lo general fracasan, de la misma forma que fracasan aquellos cuyos miembros del grupo de desarrollo se dedican a elaborar prolijos y atractivos documentos en lugar de que estos sean una herramienta que ayude en la planificación y posterior gestión.

No tiene mayor sentido gastar un mes de un proyecto de dos meses y dos desarrolladores elaborando documentos previamente al desarrollo como tampoco sirve arrancar escribiendo código en un proyecto con un grupo de desarrollo de siete personas que deberán trabajar por el término de ocho meses.

Elaborado con el criterio anterior, el plan de proyecto no debe ser un conjunto de anotaciones formales sino una referencia donde se vuelquen los criterios de conducción del proyecto y una herramienta de gestión.

En el capítulo 5, Metodologías Conducidas por los Planes, se presentó un modelo de un Plan de Proyecto. A continuación elaboraremos los aspectos claves del ejercicio de planificación y en el próximo capítulo describiremos cómo estos procedimientos fueron aplicados al proyecto del caso testigo.

18.3.1.2 Estrategia de conducción del proyecto

Básicamente la meta del ejercicio de planificación es: **obtener un cronograma que muestre la distribución del esfuerzo estimado según la estrategia elaborada teniendo en cuenta los riesgos detectados y analizados.**

En la figura 18.2 se muestra en forma esquemática la dinámica del ejercicio de planificación, con la cual nos ayudaremos para describir cómo lo llevamos a cabo.

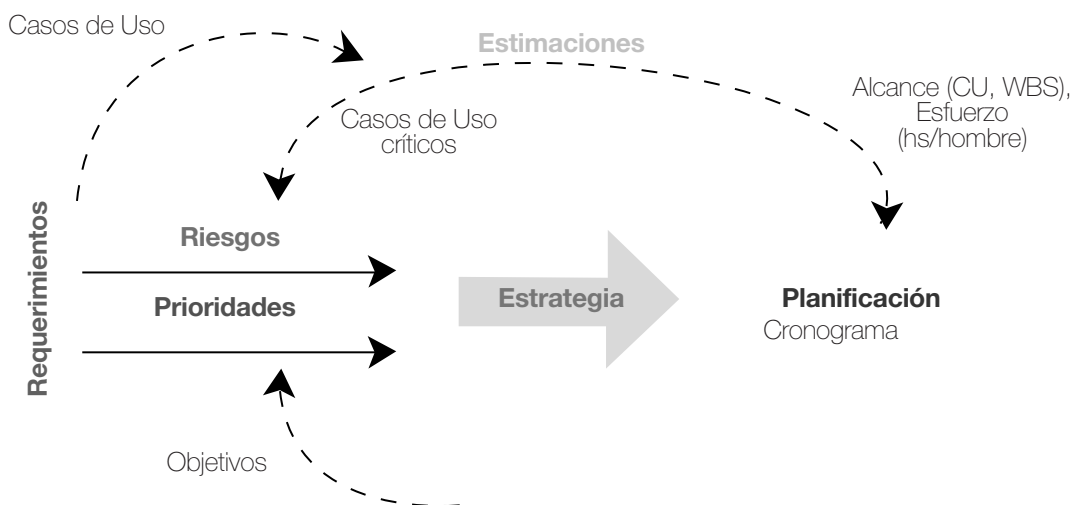


Figura 18.2. Esquema de la dinámica del ejercicio de planificación de proyectos de desarrollo de software

18.3.1.3 Visión y Objetivos

Es muy probable que la mayor parte de los desarrolladores nunca hayan preguntado a los involucrados por parte de la empresa cliente de un proyecto de desarrollo ¿por qué trabajaremos en este proyecto? ¿Por qué construiremos este sistema? ¿Para qué lo construiremos?

Sin embargo, estas son las respuestas que se necesitan a la hora de establecer una visión para el proyecto. Si en un proyecto de desarrollo se avanza sin responder estas preguntas, si se asume que la idea del sistema a construir nació porque sí, por generación espontánea, seguramente este tendrá graves problemas en su evolución.

Estas preguntas ayudan a definir una visión de la cual derivarán objetivos y prioridades y será esta visión a la que se recurrirá cuando se pierda el rumbo en cualquier momento del ciclo de vida del proyecto.

Un ejemplo de diferentes visiones son “construir el mejor producto del mercado en un determinado segmento” y “captar cuanto antes una porción inicial de un segmento de mercado”. Para lograr cualquiera de los objetivos anteriores se deberá trabajar de diferente manera, con distinta organización y criterio, y se deberá invertir recursos de manera diversa. Por esta razón es importante contar y elaborar una visión para el proyecto, que deberá ser compartida por todos los involucrados en el proyecto.

A efectos de construir esta visión se deben analizar y documentar los aspectos siguientes:

Requerimientos del negocio: proveen la fundamentación y referencia para todos los requerimientos a cubrir por el sistema a implementar. Estos serán provistos por los gerentes de desarrollo, patrocinadores, gerentes de producto y / o el departamento de marketing. Aportan la respuesta a las siguientes preguntas: ¿Por qué se debe desarrollar el producto / sistema? ¿Para qué desarrollar el producto / sistema?

Estado actual: proporcionar una descripción general de los antecedentes y / o de las situaciones que conducen a la conclusión de que este producto / sistema debe ser construido.

Oportunidades de negocio: describir las oportunidades existentes en el mercado si se tratara de un producto o de la posibilidad de resolver los problemas dentro de una organización. Si es un producto puede incluir una breve evaluación comparativa de los ya existentes y las posibles soluciones, lo que indica por qué el producto propuesto es un potencial negocio. Si es un sistema se deben identificar los problemas que actualmente no se pueden resolver, y cómo el sistema se ajusta a la estrategia del negocio dentro de la organización [14].

Objetivos y criterios de éxito: describir los objetivos empresariales más importantes del producto de una manera que sea cuantitativo y medible. Debe incluir el valor agregado para el cliente y / o usuarios. Deben incluirse beneficios económicos y de eficiencia operativa generados por el producto. Se debe definir cómo se medirán los efectos generados por los productos y un procedimiento de medida de si los objetivos de negocio se han cumplido.

Problemas y necesidades de los usuarios: describir los problemas actuales y cómo el producto y / o sistema los resolverá parcial o totalmente. También describir las necesidades de los clientes típicos o de los segmentos del mercado.

Riesgos: listar los principales riesgos del negocio asociados con el desarrollo del producto y / o sistema, entre ellos: la competencia del mercado, tiempo de salida al mercado, penetración en el mercado, etc. Además de detectarlos debe analizarse su impacto y planificarse su mitigación.

Características salientes de la solución: se debe establecer una visión a largo plazo para el sistema que se construirá para lograr los objetivos de negocio. Esta visión proporcionará el marco dentro del cual se tomarán las decisiones durante el transcurso del ciclo de vida del producto y cómo este se integrará a la arquitectura técnica de la organización [14]. Además esta incluirá las características más importantes que lo diferencian del resto en el mercado donde se comercializará o en la organización donde funcionará.

Supuestos y dependencias: registrar todas las condiciones y subordinaciones del proyecto tales como hitos de negocio, cambios en las tecnologías involucradas, evolución de las relaciones con proveedores, etc.

Alcance inicial y posterior: definir el alcance de la versión inicial y las posteriores en base a las prioridades establecidas.

Involucrados: listar a todos los involucrados (individuos, grupos u organizaciones que participen en el proyecto y que se ven afectados por sus resultados o pueden influir en su resultado) con el proyecto y determinar sus intereses, expectativas, prioridades y restricciones que imponen.

Prioridades del proyecto: es útil establecerlas para las siguientes dimensiones: tiempo, características del producto, calidad, recursos humanos y costo. Para cada una de ellas calificarlas como positivas (conductor), negativas (restricción) o equilibrantes (variable libre a controlar) de acuerdo a su influencia en el proyecto.

Ambiente operacional: describir el ambiente operacional definido por los requerimientos no funcionales. Identificar el hardware y software de base necesarios para que el producto pueda funcionar.

Esta visión compartida volcada en un documento de visión debe ser acordada por todos los involucrados al momento de acordar el desarrollo del proyecto. Como veremos en el capítulo que sigue, es lo que marca el rumbo en la evolución del ciclo de vida del proyecto.

18.3.1.4 Requerimientos y prioridades

Como se analizó en el capítulo 10, Relevamiento de requerimientos, de nada sirve una bolsa de estos sin prioridad alguna. Un atributo importante del relevamiento de los requerimientos es establecer prioridades para cada uno de ellos en base a los objetivos derivados de la visión. Entre los criterios utilizados para este fin se analizaron los siguientes [12]:

Valoración financiera: aporta una perspectiva del flujo de dinero a lo largo del ciclo de vida del proyecto en función de la posibilidad de instalar componentes que comiencen a generar recursos por nuevos ingresos, ingresos retenidos y eficiencia operacional. Todo esto teniendo en cuenta los recursos para financiar el desarrollo del software.

Deseo de los miembros de la empresa cliente: aporta la visión de los diferentes usuarios a partir de los problemas que el sistema resuelve y por lo tanto genera mejores condiciones de uso. También de los involucrados no usuarios que administran el negocio.

Valor agregado al negocio: ayuda a establecer prioridades basadas en el valor que aporta el producto y / o sistema en la administración del negocio, lo que permite realizar nuevas funcionalidades y, por lo tanto, prestar nuevos y mejores servicios de negocio.

Aporte de conocimiento del negocio: desarrollando en forma temprana componentes esenciales para el negocio y no superficiales los desarrolladores comprenden más rápidamente las reglas del negocio. Así facilitan el desarrollo del resto de los componentes.

Riesgo de desarrollo: adelanta el desarrollo de los componentes más complejos y evita llegar a momentos del ciclo de vida cercanos a la finalización con riesgos altos. Esto debe balancearse con el valor agregado como se mostró en el capítulo 10.

18.3.1.5 Estimaciones de alcance y esfuerzo

Las estimaciones de alcance y esfuerzo de un proyecto de desarrollo de software son un tema abierto acerca del cual los miembros de la comunidad informática continúan discutiendo y formando opiniones diversas. Recomendando leer el libro de Steve McConnell [9] para contar con una visión abarcativa de los problemas relacionados. Independientemente del método que se utilice para hacer estimaciones, se deben tener en cuenta los siguientes aspectos:

- Las estimaciones no son ni objetivos ni compromisos de negocio.
- Las estimaciones se hacen con información reducida.
- Las estimaciones como tal tienen límites de incertidumbre.
- Las estimaciones deben ser presentadas como estimaciones.

- Las estimaciones deben ser construidas por todos los involucrados.
- Las estimaciones teóricas deben ser ajustadas a la realidad del proyecto.
- Las estimaciones de organizaciones que conservan sus grupos de desarrollo formados y aprenden de sus errores son cada vez más precisas.
- Las estimaciones cerradas de proyectos vendidos con alcance fijado, costo fijado y planificación (tiempo) fijada conducen a proyectos fallidos.
- Las estimaciones realizadas para un proyecto trabajado en forma iterativa (alcance variable de acuerdo a prioridades, costo fijo y planificación fija) deben ser comunicadas y explicadas a la organización cliente como tal, de lo contrario se presentarán conflictos cuyo desenlace puede ser la cancelación del proyecto.

De todas estas recomendaciones nos extenderemos en un par de ellas.

Las estimaciones teóricas deben ser ajustadas a la realidad del proyecto

Las estimaciones realizadas con la mayoría de los métodos dan como resultado una probabilidad de cumplir con los tiempos de desarrollo de un 50%. De allí que es costumbre agregar buffers temporales que amortigüen los posibles problemas y representen una probabilidad cercana al 90% [11]. ¿Cuáles son las fuentes de estos posibles problemas?

En la figura 18.3 (adaptada de *Software Estimation: Demystifying the Black Art* [9]) se presenta la cuantificación de los posibles problemas generada a partir de un gran número de proyectos.

Estos factores influyen las estimaciones además, dependiendo del tamaño del proyecto, de manera opuesta a factores similares en proyectos de desarrollo en otras ingenierías. Se pone de manifiesto en el desarrollo de software que no funciona en proyectos de esta naturaleza la economía de escala. Es decir cuanto más grande y mayor cantidad más complejo y por lo tanto más caro [9].

Los factores que influyen de manera que las estimaciones al 50% serán holgadas (acortamiento de la planificación) deben conservarse a partir de acentuar la causa de dicho factor. Los factores que influyen de manera que las estimaciones al 50% serán insuficientes (alargamiento de la planificación) son donde debe focalizarse el trabajo para contrarrestar dicha influencia. Contra la influencia de algunos factores podremos trabajar para minimizar su efecto, contra la de otros no, sin embargo es importante que su impacto no crezca.

Las planificaciones no pueden acortarse sumando gente a los proyectos

Si bien es una problemática conocida, cuando se perciben retrasos en las planificaciones se sigue sugiriendo sumar desarrolladores a los proyectos. Una planificación no puede reducirse más allá de un 75% a partir del agregado de miembros al grupo de desarrollo. Existe una llamada “zona imposible” de evolución de los proyectos de desarrollo cuyo límite está dado por el número mencionado [9]. Este hecho se muestra en la figura 18.4 donde se observa que por efecto de la mayor dificultad en gestionar el trabajo de un grupo grande, la planificación en lugar de reducirse aumenta cuando se sobrepasa en 5 a 7 la cantidad de miembros de un grupo de desarrollo.

Factores de ajuste de las estimaciones

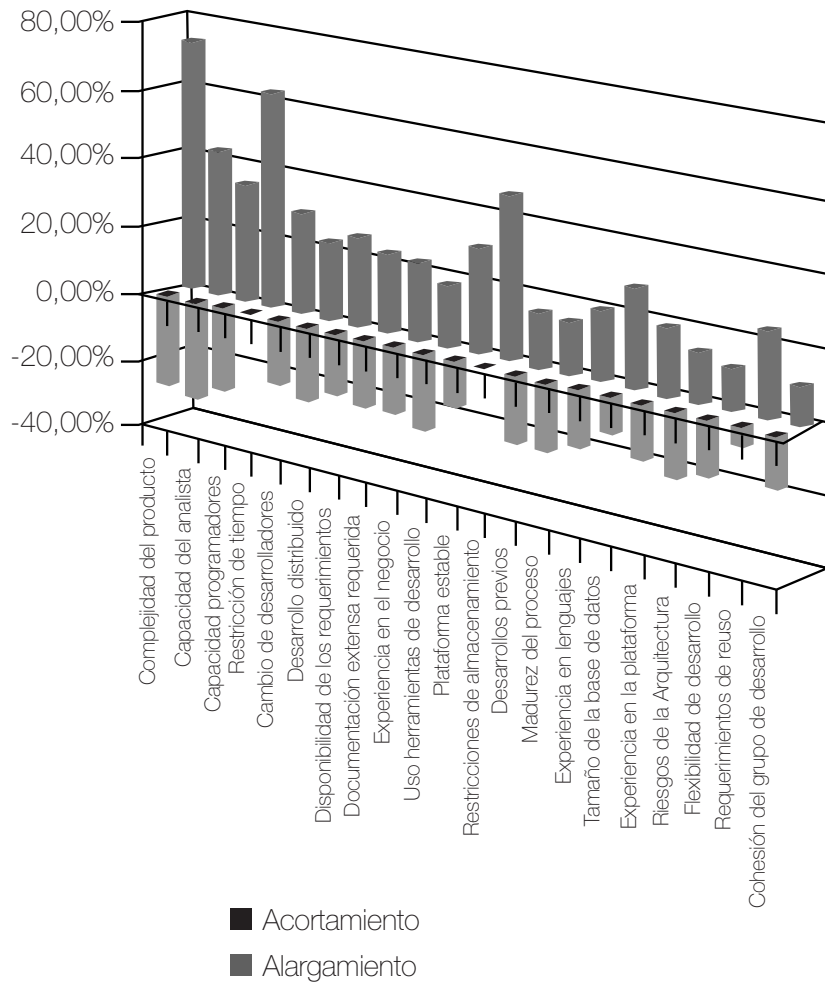


Figura 18.3. Factores que influyen en las estimaciones de esfuerzo

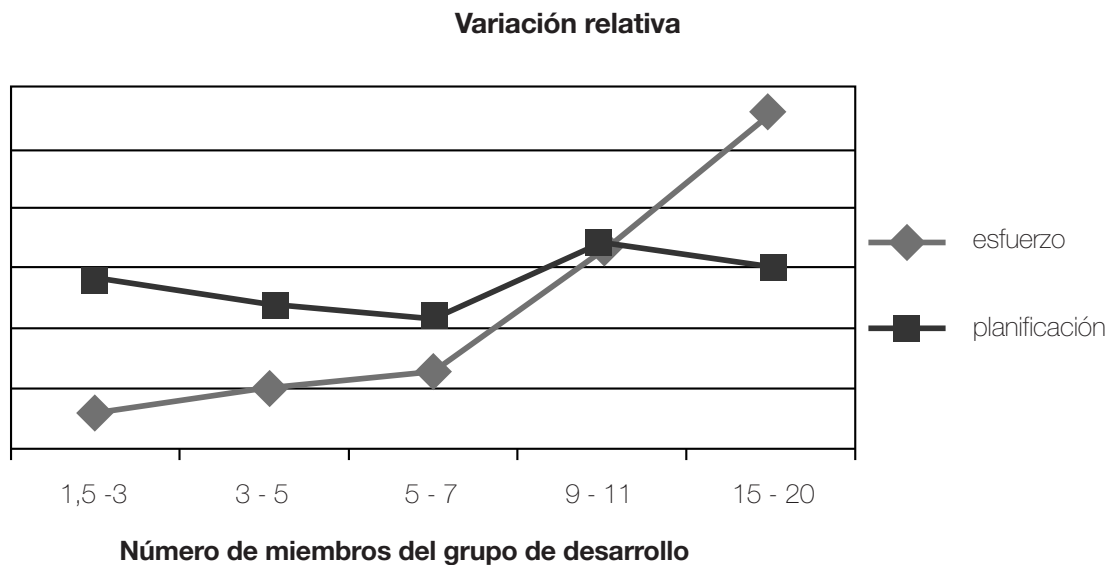


Figura 18.4. Variación del esfuerzo y la planificación con el número de miembros de un grupo de desarrollo

El proceso de estimación comienza con la estimación de alcance y a partir de los resultados de esta se realiza una estimación del esfuerzo para ese alcance. A continuación se describen dos métodos diferentes, más información detallada puede encontrarse en *Software Estimation: Demystifying the Black Art* [9].

Métodos de estimación

Las estimaciones se realizan en un proyecto al momento de dimensionarlo para elaborar una propuesta técnica económica como analizamos en una sección anterior y a lo largo del ciclo de vida de este para evaluar el avance y eventualmente realizar reestimaciones cuando se realizan replanificaciones. En esos distintos momentos del proyecto la información con que se cuenta, es decir el nivel de incertidumbre, es diferente, lo cual habilita a utilizar distintos métodos de estimación.

Estimaciones a partir de casos de uso

Alcance: consiste en determinar la cantidad de casos de uso, interfaces con sistemas externos y reportes con que contará el sistema a construir. Los sistemas pueden descomponerse en estos tipos de componentes y pueden contarse a partir de un relevamiento preliminar de requerimientos funcionales y no funcionales. Debido a la alta granularidad de esta unidad de medida (caso de uso) este método de estimación es utilizado en etapas tempranas (preventa, concepción) de los proyectos, cuando la poca información es con todo lo que se cuenta.

Esfuerzo: esta estimación consiste en cuantificar cuánto tiempo demanda la implementación de los tres tipos de componentes mencionados para diferentes grados de complejidad. Las empresas que mantienen a sus desarrolladores trabajando juntos aprenden estos números de referencia los cuales son muy valiosos [12] como se muestra en la tabla 18.1.

Esfuerzo de referencia			
Tareas	Casos de Uso [complejidad]		
	Baja	Media	Alta
Análisis / Diseño Detallado	8	16	32
Desarrollo	32	64	128
Pruebas	16	32	64
Totales [horas]	56	112	224
Tareas	Interfaces [complejidad]		
	Baja	Media	Alta
Análisis / Diseño Detallado	8	16	32
Desarrollo	16	32	68
Pruebas	8	16	34
Totales [horas]	32	64	134
Tareas	Reportes [complejidad]		
	Baja	Media	Alta
Análisis / Diseño Detallado	4	8	16
Desarrollo	16	24	40
Pruebas	8	12	20
Totales [horas]	28	44	76

A partir de la cantidad de componentes a construir y del esfuerzo necesario se obtiene el esfuerzo total para el sistema expresado en horas hombre. Cabe mencionar, como se rotula en la figura, que los números mostrados corresponden a las tareas que en el Proceso Unificado, como se mencionó en el capítulo 5, Metodologías conducidas por los planes, es el trabajo de las iteraciones de la fase de Construcción.

Distribución del esfuerzo en el ciclo de vida: para obtener el esfuerzo de la totalidad de las fases se extrapolan estos resultados sabiendo que estadísticamente los proyectos poseen una distribución de tiempo y esfuerzo como la que se muestra en la siguiente figura 18.5.

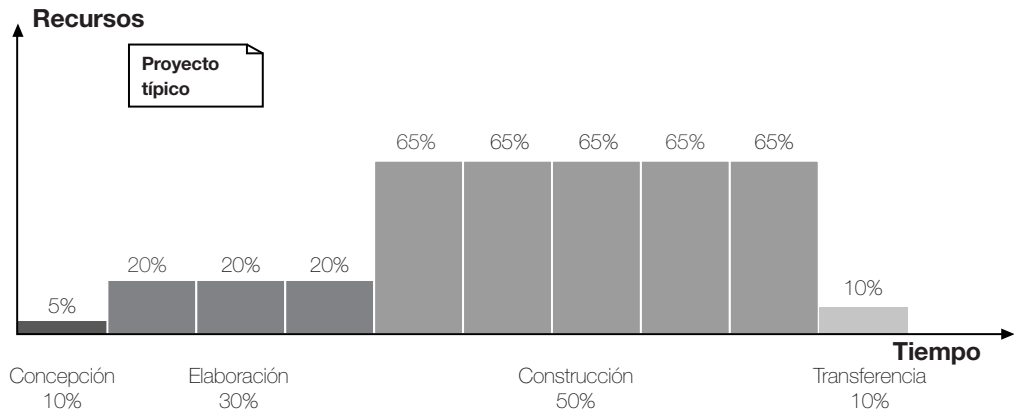


Figura 18.5. Patrón de un proyecto típico de desarrollo con duración porcentual de tiempo y ocupación de recursos humanos por fase (Adaptado de *Managing Iterative Software Development Projects* [1])

De esta forma se obtiene una distribución del esfuerzo en base a las estimaciones de alcance iniciales. Una buena práctica es afectar a estos resultados por los factores analizados en una sección anterior. Cuando el proyecto avanza y la incertidumbre se achica se pueden aplicar otros métodos que demandan información más detallada pero que obviamente generan resultados más precisos.

Estimaciones a partir de puntos de función

Un método de estimación de alcance son los *puntos de función*, inventado por Albrecht [15] cuando trabajó para IBM en 1979. Este método fue concebido como una forma independiente del lenguaje de programación de medir el tamaño del software. En la actualidad este método es adaptado, informado y publicado por un organismo internacional, el International Function Point Users Group (IFPUG), en cuyo sitio puede encontrarse información detallada (www.ifpug.org).

Alcance: la cantidad de puntos de función en un software depende de la cantidad de entradas externas (pantallas con edición), salidas externas (pantallas informativas y reportes), consultas externas (transacción de negocio), archivos lógicos internos (tabla en una base de datos) e interfaces con sistemas externos (otros sistemas con los que interactúa). También en este caso los diferentes componentes del sistema se clasifican de acuerdo a su complejidad como se hacía en el método de los casos de uso. En la tabla 18.2 se muestran estos números.

Tabla 18.2. Puntos de función por tipo de componente (adaptada de *Software Estimation: Demystifying the Black Art* [9])

Puntos de Función			
Componente	Baja Complejidad	Media Complejidad	Alta Complejidad
Entradas externas	N * 3	N * 4	N * 6
Salidas externas	N * 4	N * 5	N * 7
Consultas externas	N * 3	N * 4	N * 6
Archivos lógicos internos	N * 4	N * 10	N * 15
Interfaces con sistemas externos	N * 5	N * 7	N * 10

Si bien ambos métodos tratan de mensurar los mismos aspectos de los sistemas, este requiere un análisis más detallado al ser necesario el número de entidades de un modelo de dominio (archivos lógicos internos). Por esta razón decíamos que este método es más preciso pero necesita de mayor información, razón por la cual es adecuado para ser aplicado en fases más avanzadas del proyecto que la preventa. Los números obtenidos a partir de la Tabla 18.2 se denominan *puntos de función no ajustados*. Existen 14 factores con los que se ajustan estos números de acuerdo a características del sistema en desarrollo tales como si existe o no comunicación remota de datos, complejidad de procesamiento y otras [15].

Esfuerzo: los puntos de función así calculados pueden convertirse a horas hombre y así proceder de la misma forma que con el método anterior. Existen tablas para los lenguajes más usados que cuantifican el

número de líneas de código por punto de función. También tablas que traducen el número de líneas de código a horas hombre [9].

18.3.1.6 Administración de riesgos

Definiendo los riesgos de un proyecto como *aquellas cuestiones que con cierta probabilidad se podrían convertir en obstáculos para lograr los objetivos*, es importante comenzar a trabajar en forma temprana. En algunos libros de administración de proyectos [8] se presentan listados de riesgos que pueden ser utilizados como guía. No debe olvidarse que son una guía y que posiblemente no concuerde con los riesgos de su proyecto. El trabajo con riesgos es otra de las tareas en la cual es importante contar con cierta madurez para hacer visibles posibles fuentes de riesgo que por diferentes razones no es fácil de explicitar.

Las fuentes de riesgos son en general debidas a:

- Requerimientos: la complejidad y la falta de disponibilidad constituyen una fuente de riesgo común en muchos proyectos.
- Tecnología: la falta de experiencia en su uso o la falta de madurez de alguna tecnología son fuente de riesgos.
- Organización desarrolladora: falta de apoyo de la gerencia o falta de compromiso para con el proyecto según fue planificado al momento de competir para ganarlo.
- Organización cliente: falta de experiencia en el involucramiento en proyectos de desarrollo.
- Grupo de desarrollo: problemas generados por grupos de desarrollo recientemente formados para el proyecto, sin trabajos anteriores compartidos.

Los riesgos son importantes porque el líder conducirá la gestión del proyecto a partir de su administración, en el día a día, y además condicionarán la construcción de la estrategia a seguir para dicha gestión [1], [8], [11].

La administración de riesgos incluye las siguientes etapas:

1. **Detección:** en ella se detectan los obstáculos posibles del proyecto en términos de las causas orígenes mencionados.
2. **Análisis:** en esta etapa se asigna a cada riesgo una probabilidad de suceder y un nivel de impacto sobre el proyecto.
3. **Planificación:** a partir del análisis anterior se planifican las acciones a llevar adelante para evitar que finalmente estos riesgos se conviertan en problemas y además bajen su probabilidad de ocurrencia.
4. **Seguimiento:** finalmente se realiza el seguimiento de los efectos de las acciones implementadas para bajar los riesgos y eventualmente realizar nuevas acciones.

En el próximo capítulo mostraremos la administración de los riesgos del proyecto del caso testigo.

Tareas, tiempos y recursos

La distribución de tareas a lo largo del tiempo del ciclo de vida se basará en un WBS (*Work Breakdown Structure*) realizado a partir de la planificación. En general las tareas dependerán de otras y viceversa. Las

tareas asociadas al desarrollo de determinados requerimientos (casos de uso) con mayor prioridad serán ubicadas primero y las de menor prioridad luego.

A cada tarea se le asignarán responsables dentro del grupo de desarrollo de manera de administrar los recursos humanos de acuerdo a los roles que desarrollará cada uno. De esta forma se construye un cronograma que, expresado en un diagrama de Gantt, tiene una estructura como la que se muestra a continuación en la figura 18.6.a, tareas correspondientes a la estimación, planificación y gestión de un proyecto, y en la figura 18.6.b las correspondientes al desarrollo. Estas imágenes fueron obtenidas utilizando la herramienta Project de Microsoft.

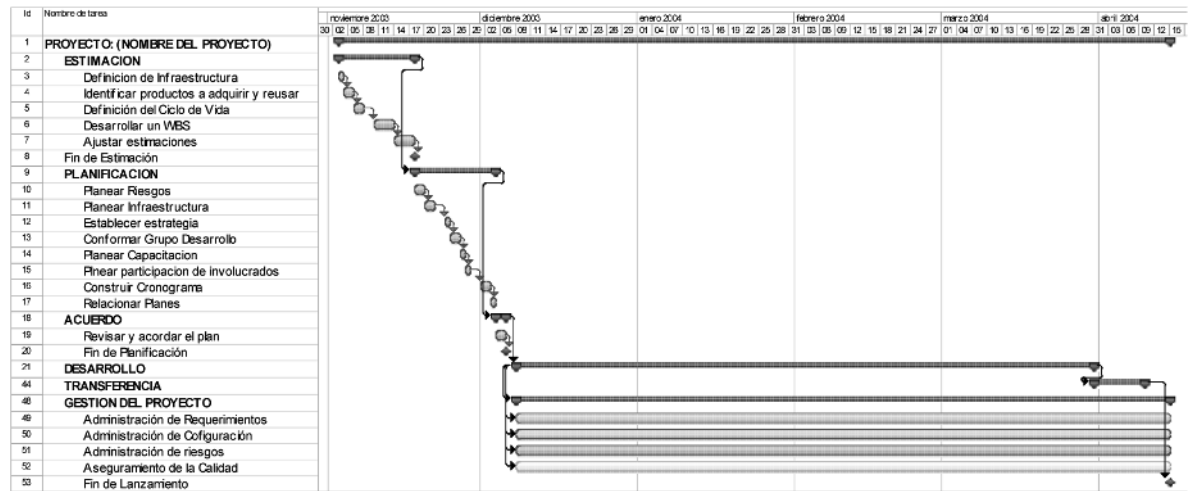


Figura 18.6.a – Cronograma desplegado que muestra las tareas asociadas a la planificación y gestión del proyecto

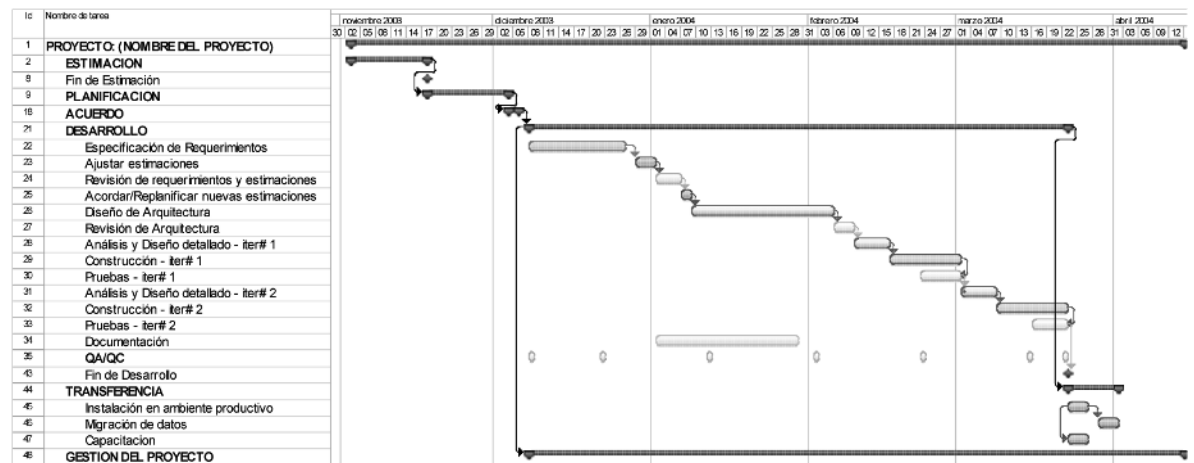


Figura 18.6.b. Cronograma desplegado que presenta las tareas asociadas al desarrollo

Puede verse que este cronograma cuenta con una única iteración para las fases de concepción, elaboración y transferencia; y con dos iteraciones para la fase de construcción. También puede observarse que ambas iteraciones poseen tareas de pruebas del software solapadas con la construcción. Otra cuestión a mencionar es que se han planificado sesiones de QA (revisiones de diseño y código) en forma periódica para todo el desarrollo.

Unidades de trabajo

Un grupo de N miembros seguramente podrá ser dividido en Unidades de Trabajo, compuestas cada una por dos o más desarrolladores. Estas unidades trabajarán en diferentes tareas en forma simultánea.

Un aspecto de interés es la asignación de tareas a los miembros del grupo de desarrollo en cada Unidad de Trabajo que puede hacerse en base a criterios técnicos relacionados con el conocimiento y las habilidades de las personas o en función del negocio.

Si se hace según el primero habrá casos de uso que serán implementados por personas habilidosas en tecnologías de presentación, en tecnologías de persistencia, y por personas que programarán la capa de negocio.

Si se hace según el segundo criterio habrá casos de uso implementados integralmente por determinadas personas.

La primera es utilizada en muchas software *factories* y se basa en la eficiencia, la segunda se basa en el conocimiento que los desarrolladores adquieren de aspectos diferentes del negocio.

18.3.2 Metodologías ágiles

El proceso de planificación de Scrum, según [3], implica responder las siguientes tres preguntas:

- ¿Qué pueden esperar los patrocinadores del proyecto que haya cambiado cuando el proyecto termine?
- ¿Qué progreso se habrá logrado en el final de cada Sprint?
- ¿Por qué el proyecto es una inversión de valor para los patrocinadores y por qué creen que los que proponen el proyecto pueden ofrecer los beneficios previstos?

18.3.2.1 Visión

Para responder la primera y la tercera de estas preguntas es necesario elaborar una visión para el proyecto. Si bien las metodologías ágiles expresan en su manifiesto “Working software over comprehensive documentation” (Trabajar en el software antes que en una documentación completa) también promueven el ejercicio de planificación continua, para lo cual se necesitan objetivos a alcanzar derivados de una visión del proyecto. La elaboración de una visión compartida para el proyecto es una tarea común a todas las metodologías.

A efectos de compartirla (comunicarla) es necesario documentar esta visión. Los puntos sobre los que esta se construye son los mismos que ya describimos en este capítulo.

18.3.2.2 Acuerdos

Los proyectos de software, todos independientemente de la metodología seleccionada para llevarlo adelante, son vendidos y existe como tal dos partes. Una parte compradora o patrocinante del proyecto y una vendedora o empresa desarrolladora. A diferencia de las metodologías conducidas por los planes, los acuerdos en estas metodologías se plantean como acuerdos en base a un alineamiento de intereses en lugar de contratos con confrontación de intereses. Pero más allá del acuerdo los patrocinantes necesitan los mismos dos números de siempre: cuánto tiempo demandará el proyecto y cuál será el costo. La propuesta técnico económica se elabora de la misma forma, sin embargo se acuerda trabajar un tiempo determinado y cobrar por ello una cantidad determinada maximizando los requerimientos a implementar. En este último punto radica la diferencia.

Cabe destacar que en la fase de preventa son utilizadas las mismas preguntas preliminares ya presentadas, pues las mismas son adecuadas independientemente de la metodología que se elija utilizar.

18.3.2.3 Product Backlog

La planificación de un proyecto se apoya en un listado de requerimientos (*User Stories*) y tareas a realizar que se dio en llamar *Product Backlog*. Este consiste en un listado en cuya elaboración participa activamente el *Product Owner* (involucrado por parte del cliente) y cuenta además con las prioridades de cada ítem listado. En este se incluyen requerimientos y tareas, es decir que es una mezcla de WBS y listado de requerimientos. Constituye una vista de todo lo que el proyecto involucra, de allí su utilización como mecanismo de planificación. Existen herramientas que facilitan este proceso, entre ellas Pivotal Tracker [16], GreenHopper [17].

En la figura 18.7 se muestra en forma esquemática un Product Backlog y sus componentes básicos y los Sprints Backlog correspondientes. También se indica que el proyecto será entregado en dos entregas (*release*) las cuales fueron planeadas al comienzo del proyecto.

Plan del proyecto

El plan del proyecto no es definido por ninguna de estas metodologías en lo formal, aunque casi todas ellas y en especial Scrum (orientada a la gestión) promueven que se planifiquen los siguientes puntos de una manera resumida y sencilla [4], [8].

En dicho plan se busca expresar las mínimas definiciones que establezcan:

- Las ideas básicas acerca de la veracidad de llevar adelante el proyecto de la forma que se describe a efectos de generar confianza en los patrocinadores.
- Una enumeración de los riesgos y la planificación de las acciones a tomar frente a cada uno de ellos.
- La constitución del grupo de desarrollo con la asignación de los roles de Gerente de Desarrollo, *Scrum Master* (líder del proyecto), *Product Owner* (definidor del producto) y el resto de los miembros del grupo.
- La definición de las actividades de los roles y la forma de trabajo.
- La definición de las buenas prácticas en cada uno de los aspectos de interés para la evolución del proyecto.
- La sincronización de las tareas de QA con las del desarrollo.

Respecto de volcar las definiciones anteriores en un documento, depende del volumen del proyecto, del número de involucrados y de la visión y pedido del cliente. Habrá casos en que estas definiciones estarán documentadas y otras no. En estos últimos casos el plan del proyecto lo constituyen la Visión Compartida y el Product Backlog.

Todas las tareas de planificación son realizadas en lo que podríamos llamar el *Sprint cero*, ya que sin esta planificación es imposible comenzar con el primer Sprint del proyecto.

Si bien estos aspectos no son cubiertos por estas metodologías, en estos momentos iniciales del proyecto deben formularse definiciones acerca de la arquitectura técnica del sistema a desarrollar.

18.3.2.4 Release

A partir de objetivos derivados de la visión y de las prioridades correspondientes se planifican los entregables (*release*) que el proyecto tendrá a lo largo de su ciclo de vida. Cada uno estará compuesto por un número determinado de Sprints (iteraciones) de duración fija. Por lo tanto, el tiempo de la planificación es conocido de antemano.

18.3.2.5 Sprint

Cada iteración es planificada a partir de la estimación del esfuerzo necesario de cada uno de los ítems a desarrollar del Product Backlog. Este ejercicio consiste en extraer del Product Backlog e incluir en el Sprint Backlog aquellos ítems de mayor prioridad. Se conforma el Sprint de acuerdo a la cantidad de Story Points (medida de esfuerzo) que el grupo estima podrá desarrollar en el transcurso del Sprint (normalmente de 20 días). En este ejercicio de planificación se refinan los requerimientos realizando el análisis de cada uno [3].

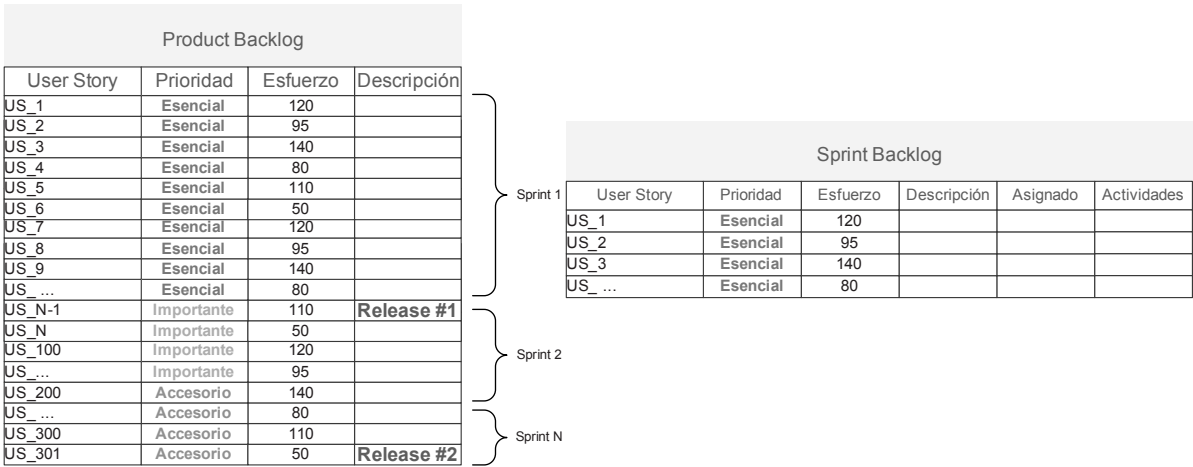


Figura 18.7. Diagrama esquemático que muestra el Product Backlog de un proyecto reflejando su planificación en Releases y Sprints

18.3.2.6 Estimación de esfuerzo

Las estimaciones de esfuerzo son realizadas por los desarrolladores al momento de planificar el Sprint [8]. Las estimaciones se basan en la experiencia previa en desarrollos con las mismas características. Se refuerza una vez más la bondad de contar con desarrolladores que hayan participado en proyectos anteriores en la empresa y en el mismo grupo. Una técnica adecuada para organizar estas estimaciones es conocida como *Delphi* [9] y consiste en reuniones donde un coordinador releva las estimaciones del grupo y presenta los valores facilitando la discusión a efectos de hacerlas converger. Este proceso se realiza en forma iterativa y es adecuado para su utilización en la planificación de los Sprints.

18.4 Conclusión

Se presentaron dos formas de gestionar proyectos de software en lo que hace a la estimación y planificación de proyectos.

Como se analizó hay cuestiones comunes a todos los proyectos independientemente de la metodología de desarrollo o gestión que se elija. Los proyectos deben venderse y para ello es necesario elaborar una propuesta técnico económica para lo cual se debe achicar la incertidumbre. Por lo tanto es necesario construir una visión de la cual se desprendan los objetivos que sirvan de rumbo a seguir en el ciclo de vida del proyecto. Para ello propusimos una serie de claves a analizar y conclusiones que ayudan en dicha construcción.

También tratamos las actividades y los activos fundamentales a realizar en la planificación de los proyectos para las dos formas de trabajo, conducida por los planes y ágiles. En el próximo capítulo veremos cómo se acentúan las diferencias entre ambas formas al tratar el seguimiento de los proyectos.

18.5 Contenido de la página web de apoyo



18.5.1 Mapa conceptual

18.5.2 Autoevaluaciones

18.5.3 Presentaciones en Power Point (exclusivo para el docente)

18.6 Referencias

- [1] *Managing Iterative Software Development Projects*, Kurt Bittner, Ian Spence, Addison-Wesley, 2006.
- [2] *The Unified Software Development Process*, Jacobson, Ivar, et ál., Addison-Wesley, 1998.
- [3] *Agile Project Management with Scrum*, Ken Schwaber, Microsoft Press, 2004.
- [4] *Planning Extreme Programming*, Kent Beck, Martin Fowler, Addison-Wesley, 2000.

- [5] *Agile Management for Software Engineering: Applying the Theory of Constraints for Business Results*, David J. Anderson, Eli Schragenheim, Prentice Hall PTR, 2003.
- [6] *SDLC 3.0 – Beyond a tacit understanding of Agile*, Mark Kennaley, Fourth Medium Press, 2010.
- [7] *Balancing Agility and Discipline: A Guide for the Perplexed*, Barry Boehm, Richard Turner, Addison-Wesley, 2003.
- [8] *Agile Estimating and Planning*, Mike Cohn, Prentice hall, 2005.
- [9] *Software Estimation: Demystifying the Black Art*, Steve McConnell, Microsoft Press, 2006.
- [10] *Software Project Survival Guide*, Steve McConnell, Microsoft Press, 1996.
- [11] *Software Project Survival Guide: How to Be Sure Your First Important Project Isn't Your Last*, Steve McConnell, Microsoft Press, 1997.
- [12] *Calidad en el Desarrollo de Software*, Guillermo Pantaleo, Alfaomega 2011.
- [13] *Lean Software Development: An Agile Toolkit*, Mary Poppendieck, Tom Poppendieck, Addison-Wesley, 2002.
- [14] *Enterprise Architecture as Strategy*, J. Ross, P. Weill, D. Robertson, Harvard Business Press, 2006.
- [15] *Measuring Application Development Productivity*, Proceedings of the Joint SHARE/GUIDE/IBM Application Development Symposium, Albrecht, Allan J., October 1979: 83–92.
- [16] *Pivotal Tracker*, <http://www.pivotaltracker.com/>.
- [17] *GreenHopper*, <http://www.atlassian.com/software/greenhopper/>.

18.7 Apéndice A – Cuestionario guía de relevamiento preliminar

18.7.1 Aspectos vinculados al contexto

1. El desarrollo que se solicita es:
 - Proyecto integral
 - Proyecto desde la arquitectura
 - Proyecto de construcción (codificación)
 - Proyecto de mantenimiento.
2. ¿Cuál es el dominio del negocio donde el desarrollo del proyecto funcionará?
3. Establecer cuáles son los objetivos que se pretenden alcanzar con el software a desarrollar.

18.7.2 Reingeniería de sistemas (Sistemas a ser reemplazados)

18.7.2.1 Aspectos funcionales del negocio

1. Describa el objetivo del proyecto en términos del área de negocio.
2. Describa la funcionalidad existente en términos de módulos y flujo de información del negocio entre cada módulo.
3. ¿Serán agregadas nuevas funcionalidades en el nuevo sistema? Describa las nuevas funcionalidades, por módulo.

18.7.2.2 Aspectos no funcionales

1. ¿Existen condicionamientos de hardware y /o software en términos de base de datos, sistema operativo, lenguaje de programación, tipo de browser, etc.?
2. ¿Es esta una aplicación distribuida?
3. ¿Es esta una aplicación que debe funcionar en Internet? ¿En una intranet y /o en una extranet? ¿Qué browser debe utilizarse para conectarse a ella?
4. ¿Hay algún requerimiento no funcional?
 - Velocidad
 - Adaptabilidad
 - Eficiencia: (capacidad, restricción de tiempos, etc.)
 - Seguridad y auditoria
 - Tamaño.

18.7.2.3 Aspectos de arquitectura y tecnología

1. ¿Cuáles fueron la tecnología y las herramientas de desarrollo con que se construyó el sistema existente?
2. ¿Existen restricciones de seguridad de algún tipo especificadas?
3. ¿Hay algún *framework* que deba ser usado?

18.7.2.4 Aspectos de interfaces

1. Liste y describa las interfaces del sistema según su tipo (batch u online, sincrónica o asincrónica, etc.) y las operaciones asociadas en las transacciones de negocio.
2. Describa el ambiente en el cual funcionan estos sistemas externos (hardware, OS, localización, etc.).
3. ¿Existe algún estándar corporativo para el intercambio de datos o nosotros debemos proponer uno (XML Web Services, etc.)?
4. ¿Existe una herramienta ya definida para la implementación de estas interfaces?
5. ¿Es posible reutilizar algunas de las interfaces existentes o alguna de sus partes?

18.7.2.5 Aspectos vinculados al alcance

1. ¿Existen módulos en el sistema ya existente que pueden ser reutilizados?
2. ¿Cuántas líneas de código fuente tiene el sistema existente?
3. ¿Cuántas pantallas de entrada tiene el sistema existente?
4. ¿Cuántas pantallas, reportes y consultas tiene el sistema existente?
5. ¿Cuántos files y tablas son accedidas por cada proceso que forman parte del sistema actual?
6. ¿Cuántos componentes deben ser rehechos, cuántos módulos?
7. ¿Cuántos tipos de usuarios deben acceder al Nuevo sistema?
8. ¿Cuántos en forma simultánea?

18.7.2.6 Migración de datos

1. ¿Cuál es la base de datos actual y en qué ambiente está funcionando (OS, hardware)?
2. ¿Se debe conservar la estructura del modelo de datos o se cambiará a uno Nuevo?
3. ¿Cuentan con modelo de datos (Viejo y nuevo) disponible a través de una herramienta?
4. ¿Cuentan con un modelo lógico de datos que describa los diferentes conceptos del negocio?

18.7.2.7 Aspectos vinculados al proyecto

1. ¿Existe presupuesto para todos los ítems de software y hardware involucrados?
2. ¿Existen limitaciones de Presupuesto de algún tipo?
3. ¿Existe una fecha fijada para la terminación del proyecto?
4. ¿Con qué tipo de soporte se contará durante el tiempo que demande el desarrollo del Nuevo sistema?
¿Habrá disponibilidad de personas por parte del cliente, tales como usuarios del sistema a reemplazar, expertos del negocio, etc.?

18.7.3 Sistemas a desarrollar en proyectos integrales

18.7.3.1 Aspectos funcionales del negocio

1. Describa el objetivo del proyecto en términos del área de negocio.
2. Describa la funcionalidad existente en términos de módulos y flujo de información del negocio entre cada modulo.
3. ¿Serán agregadas nuevas funcionalidades al negocio con el sistema? Describa las nuevas funcionalidades, por módulo de negocio.
4. En forma resumida indique cuál es la visión estratégica del negocio a partir del sistema que se solicita.
5. Objetivos del software y sus beneficios por contar con el nuevo sistema.
6. Enumere los problemas actuales por no contar con el sistema.
7. ¿Existe un sistema en la actualidad que cubra todo o en parte su negocio?

18.7.3.2 Aspectos no funcionales

1. ¿Existen condicionamientos de hardware y /o software en términos de base de datos, sistema operativo, lenguaje de programación, tipo de browser, etc.?
2. ¿Es esta una aplicación distribuida?
3. ¿Es esta una aplicación que debe funcionar en Internet? ¿En una intranet y /o en una extranet? ¿Qué browser debe utilizarse para conectarse a ella?
4. ¿Hay algún requerimiento no funcional?
 - a. Velocidad
 - b. Adaptabilidad
 - c. Eficiencia: (capacidad, restricción de tiempos, etc.)

- d. Seguridad y auditoria
- e. Tamaño.

18.7.3.3 Aspectos de arquitectura y tecnología

1. ¿Existe una tecnología ya seleccionada por la compañía en forma corporativa con la cual debe construirse el sistema?
2. ¿Existen restricciones de seguridad de algún tipo especificadas?
3. ¿Hay algún *framework* que deba ser usado?

18.7.3.4 Sistemas relacionados ya existentes

1. ¿Existen sistemas ya existentes con los cuales el nuevo deberá interactuar?
2. Liste y describa las interfaces de los sistemas según su tipo (batch o online, sincrónica o asincrónica, etc.) y las operaciones asociadas en las transacciones de negocio.
3. Describa el ambiente en el cual funcionan estos sistemas externos (hardware, OS, localización, etc.)
4. ¿Existe algún estándar corporativo para el intercambio de datos o nosotros debemos proponer uno (XML Web Services, etc.)?
5. ¿Existe una herramienta ya definida para la implementación de estas interfaces?
6. ¿Es posible reutilizar algunas de las interfaces existentes o alguno de sus partes?

18.7.3.5 Aspectos de alcance

1. En forma resumida indique cuáles son las expectativas respecto del sistema a desarrollar.
2. ¿Debería el sistema cubrir todos los aspectos del negocio o solo algunos? ¿Cuáles?
3. Enumere, en orden de prioridad para su negocio, los módulos mencionados en el apartado Aspectos Funcionales del Negocio.
4. Especificar las características principales y los aspectos sobresalientes del sistema que se solicita, de forma que se pueda obtener una visión clara del software a construir.
5. Definir los límites del proyecto de manera que no queden dudas respecto a lo que el sistema abarcará y contemplará. El alcance propuesto debe permitir alcanzar los objetivos del proyecto.
6. Si esta en condiciones de hacerlo, indique:
 - Una breve descripción del alcance.
 - Una lista exhaustiva de todas aquellas funciones que, acordadas con el usuario, deberán ser contempladas por el sistema.
 - Las funciones excluidas son todas aquellas funciones que luego de haber sido identificadas en el relevamiento y verificadas con el usuario se acordó que no formarán parte del sistema. Entre estas es particularmente valioso incluir aquellas funciones “linderas” a las funciones incluidas que no formarán parte del alcance, como así también algunas funciones cuyos resultados son utilizados por el requerimiento en cuestión y también tareas relacionadas con la implantación de la solución que pudieran dar lugar a duda de no puntualizarlas en este ítem.

7. Si ya se ha realizado un relevamiento de la funcionalidad del sistema a desarrollar, es de gran ayuda que nos lo hagan conocer. Ej. Casos de uso, modelo de datos, diagramas de contexto, diagramas de flujo de datos, etc.

18.7.3.6 Migración de datos

1. ¿Deberá migrarse alguna base de datos para el nuevo sistema?
2. ¿Cuál es la base de datos actual y en qué ambiente está funcionando (OS, hardware)?
3. ¿Se debe conservar la estructura del modelo de datos o se cambiará a uno Nuevo?
4. ¿Cuentan con modelo de datos (Viejo y nuevo) disponible a través de una herramienta (Erwin, Designer Oracle, etc.)?
5. ¿Cuentan con un modelo lógico de datos que describan los diferentes conceptos del negocio?

18.7.3.7 Aspectos vinculados al proyecto

1. ¿Existe presupuesto para todos los ítems de software y hardware involucrados?
2. ¿Existen limitaciones de Presupuesto de algún tipo?
3. ¿Existe una fecha fijada para la terminación del proyecto?
4. ¿Con qué tipo de soporte se contará durante el tiempo que demande el desarrollo del Nuevo sistema?
¿Habrá disponibilidad de personas por parte del cliente, tales como usuarios del sistema a reemplazar, expertos del negocio, etc.?

Gestión de proyectos
Monitoreo y Control

Contenido

19.1 Monitoreo y control de proyectos	330
19.2 Conclusión	337
19.3 Contenido de la página web de apoyo	338
19.4 Referencias	338

Objetivos

- Cómo realizar el seguimiento de proyectos utilizando metodologías conducidas por los planes
- Cómo realizar el seguimiento de proyectos utilizando metodologías ágiles
- Entender el estado y avance de los proyectos
- Qué registros construir y qué reuniones realizar

Competencias

- Gestionar un proyecto de desarrollo de software en base a su previa planificación

19.1 Monitoreo y control de proyectos

Cuando el proyecto ya fue acordado se comienza con el desarrollo de las tareas que fueron planificadas, como se describió en el capítulo anterior. A partir de este momento del ciclo de vida debe realizarse un seguimiento que consiste en monitorear cada una de las actividades y controlar su evolución tomando las acciones que se decidan necesarias. En este capítulo presentamos el seguimiento de los proyectos desde la perspectiva de los dos tipos de metodologías de trabajo [1], [6] que venimos analizando a lo largo de los capítulos anteriores.

19.1.1 Metodologías conducidas por los planes

Se planificó que el seguimiento del proyecto se haría a partir de reuniones de estado a las cuales asistirían los miembros del grupo de desarrollo y reuniones de avance a las cuales asistirían los responsables (gerentes y líderes) involucrados en el proyecto [1], [2]. La frecuencia de dichas reuniones se determina en función del volumen del proyecto, de la cantidad de personas, de si el proyecto es remoto o localizado, etcétera.

19.1.1.1 Seguimiento del estado

El líder del proyecto facilita y conduce la realización de las diferentes tareas atendiendo diariamente la evolución de los riesgos detectados y analizados.

En general, una vez por semana coordina una reunión del grupo de desarrollo donde se analizan diferentes aspectos del proyecto que van desde lo técnico, pasando por cuestiones de relación con los usuarios y clientes, asuntos personales de los miembros, hasta cualquier otro tema vinculado a la evolución del proyecto. Esta reunión puede demandar unas dos o tres horas para un grupo de siete personas. Cuando se tratan los problemas surgidos es conveniente asignar un responsable dentro del grupo y una fecha estimativa de solución. Un aspecto importante de todas estas reuniones orientadas al seguimiento es que deben estar encadenadas entre ellas [5]. De nada sirve una serie de reuniones aisladas, razón por la cual se deben tratar los temas de la reunión anterior a efectos de concluir su cierre antes de tratar los nuevos temas. Cuando en el próximo capítulo mostremos cómo aplica esto al proyecto del caso testigo, utilizaremos un informe documentado orientado a vincular esta serie de reuniones. Entre los problemas típicos que surgen y deben resolverse se cuentan los listados más abajo en la tabla 19.1.

19.1.1.2 Seguimiento del avance

De la misma forma que mencionamos para las reuniones anteriores, la frecuencia de estas depende de las mismas variables. Estas reuniones están orientadas a monitorear y controlar el avance del proyecto con la participación de ambas partes. Para un proyecto mediano, una frecuencia mensual suele ser común. A ella asisten los responsables del proyecto por ambas partes, los gerentes responsables de ambas partes y miembros del grupo de desarrollo a designar dependiendo de los temas que se vayan a tratar. Puede ser el analista si se trata de acordar cambios a los requerimientos, puede ser el líder técnico si se trata de informar o acordar cuestiones de la tecnología.

En estas reuniones se tomas decisiones, por lo cual los asistentes deben tener poder de decisión para alterar la planificación original y eventualmente de ser necesario replanificar tareas del proyecto que pueden afectar los costos.

Se analizan e informan los avances, las horas invertidas versus las estimadas, los pagos de acuerdo al contrato y la evolución futura del proyecto.

Se revisan los hitos, los entregables, la calidad de los productos generados a partir de los resultados de las pruebas del software y el logro de los objetivos planteados en la planificación.

Tabla 19.1. Problemas típicos tratados en las reuniones de estado	
Tipo de problema	Categoría
Falta de capacitación	Recursos
Falta de infraestructura	Recursos
Falta de recursos humanos	Recursos
Retraso en cronograma	Gestión
No acuerdo en requerimientos	Acuerdos
No acuerdo en tecnología	Acuerdos
No acuerdo en metodología	Acuerdos
Perdida de visión compartida con el cliente	Acuerdos
Conflicto personal entre involucrados	Grupo Desarrollo
No acuerdo con la forma de trabajo	Grupo Desarrollo
Lugar físico de trabajo inadecuado	Grupo Desarrollo
Ambiente de trabajo inadecuado	Grupo Desarrollo

Es aconsejable hacer coincidir algunas de estas reuniones con las fechas planificadas para hitos del proyecto, como la finalización de fases, iteraciones y entregas parciales.

19.1.1.3 Revisión de fases

Una de las fallas más comunes cuando se trabaja de esta forma es confundir el cronograma de la planificación con la evolución real del proyecto. Como se describió en el capítulo 5, Metodologías conducidas por los planes, cada período del ciclo de vida del proyecto tiene objetivos bien determinados. Por ejemplo, la Concepción tiene como meta achicar los riesgos del negocio a partir del relevamiento de requerimientos y la Elaboración, los riesgos asociados a la tecnología proponiendo y testeando una arquitectura para el sistema en construcción. Estas fases no terminan porque lo especifique un cronograma, sino porque se logran estos objetivos.

En estas revisiones se comparan los resultados obtenidos con los esperados para [1]:

- Los requerimientos implementados.
- Los tiempos invertidos.
- Las pruebas de aceptación.
- La evolución de los riesgos.

Entre los problemas típicos que se plantean en estas revisiones suelen aparecer los siguientes:

- Inestabilidad por indefiniciones en la arquitectura.
- Inestabilidad en los requerimientos por fallas en el relevamiento o por indefiniciones por parte de los usuarios.
- Alcance subdimensionado detectado al refinar los requerimientos.
- Falta de recursos en la implementación del proceso de pruebas.
- Mucho retrabajo como consecuencia de cualquiera de las causas anteriores.

Como conclusión de la revisión de estas fases [1] debe tenerse en cuenta que:

- Las Fases NO terminan si NO se alcanzan los objetivos asociados a los hitos de terminación.
- Una Fase exitosa (objetivos alcanzados) puede significar la cancelación del proyecto. Ej.: una Concepción puede mostrar que el proyecto es inviable en las condiciones planteadas.

Estas cuestiones son presentadas y analizadas en las reuniones de avance del proyecto con la participación de los miembros del grupo de desarrollo y demás involucrados que sean necesarios para un tratamiento amplio de todos los temas.

19.1.1.4 Revisión de iteraciones

Si bien estas evaluaciones están ligadas al desempeño del grupo de desarrollo, son buenas oportunidades para compartirlas con los involucrados por parte del cliente en el trabajo diario del proyecto y corregir desajustes que de persistir pueden hacer que el proyecto tenga una evolución no deseada.

En estas revisiones se comparan los resultados obtenidos con los esperados para [1]:

- Los requerimientos implementados de acuerdo a sus prioridades.
- Los tiempos invertidos en cada una de las tareas de desarrollo.
- La cantidad de errores nuevos, fijados y corregidos.
- La cantidad de cambios solicitados, analizados e implementados.
- La evolución de los riesgos.
- Los objetivos planteados para la iteración.

Los problemas típicos que surgen en estas evaluaciones son:

- No lograr los objetivos por falta de claridad en sus planteos.
- Esfuerzo insuficiente en el proceso de pruebas hace crecer los errores no corregidos.

- Mala administración del tiempo al no contar con prioridades para cada requerimiento.
- Criterios poco claros de evaluación (falta claridad en la evidencia a generar).
- Alcance planificado muy grande o muy chico.
- Planeamiento muy detallado o de muy alto nivel.
- Cronogramas muy ajustados por presión del negocio.
- Pérdida de foco del grupo de desarrollo por dedicación a múltiples tareas.

Como conclusión de la revisión de estas iteraciones [1] debe tenerse en cuenta que:

- Una iteración exitosa siempre implica la continuación del proyecto o su terminación según lo acordado.
- Una iteración fallida puede implicar la cancelación o extensión del proyecto.
- Cuando se trabaja con iteraciones de duración fija, estas terminan en la fecha prefijada.

19.1.1.5 Replanificación

Como planteamos en los capítulos 4 y 5, de Metodologías, y en el capítulo 18 de Gestión de proyectos, esta forma de trabajo basa su planificación en una cantidad de requerimientos a implementar a lo largo de un tiempo estimado y por cuyo trabajo se cobrará una cantidad especificada en el presupuesto expresado en la propuesta técnico - económica. Sin embargo, cuando los requerimientos son refinados, durante el ciclo de vida del proyecto, se obtiene información que en general determina que las estimaciones deberán ser ajustadas ya que el desarrollo requerirá de un mayor esfuerzo. Esta negociación es permanente y presenta dos alternativas, que son: cancelar requerimientos para que el desarrollo encaje en el tiempo estimado y presupuesto fijado o extender el tiempo del proyecto acompañado de un crecimiento correspondiente en el presupuesto.

Existen otras causas que pueden llevar a un cambio en la planificación, sin embargo esta es la más importante porque implica una discusión de cuestiones cuyas causas son a veces de difícil fundamentación. Por el contrario la demora en la entrega de componentes de infraestructura, la deserción de miembros del grupo de desarrollo, la implementación fallida de determinadas cuestiones técnicas, la falta de un referente del negocio con suficientes conocimientos y otros son problemas comunes que afectan a los proyectos pero que son entendidos aunque no siempre aceptados por las distintas partes. Pero la citada anteriormente como más importante está ligada a la esencia misma de la forma de trabajo.

En la figura 19.1 se presenta en forma esquemática diferentes estrategias de replanificación de una tarea [7].

Por Absorción: No se altera la planificación original y se absorbe la realización de las tareas retrasadas en el tiempo asignado a futuras tareas. Una causa posible para la aplicación de esta estrategia es la pérdida de un miembro del grupo de desarrollo por ausencia por enfermedad y su posterior recuperación con posibilidad de trabajo simultáneo con el resto del grupo.

Por Desplazamiento: La realización de las tareas atrasadas es asignada a tiempos futuros en la planificación del proyecto, con lo cual resulta cambiada la planificación original en una cantidad de tiempo igual a la necesaria para realizar las tareas atrasadas. Una causa posible para la aplicación de esta otra estrategia de

replanificación es el suceso de inconvenientes surgidos en el uso de determinada tecnología que requirió un aprendizaje previo por única vez.

Por Afectación Porcentual: La planificación se ve afectada a lo largo de todo el proyecto en un porcentaje igual a las tareas atrasadas. Una causa posible para su aplicación es la mala estimación previa del esfuerzo necesario y / o contar con desarrolladores con poca experiencia.

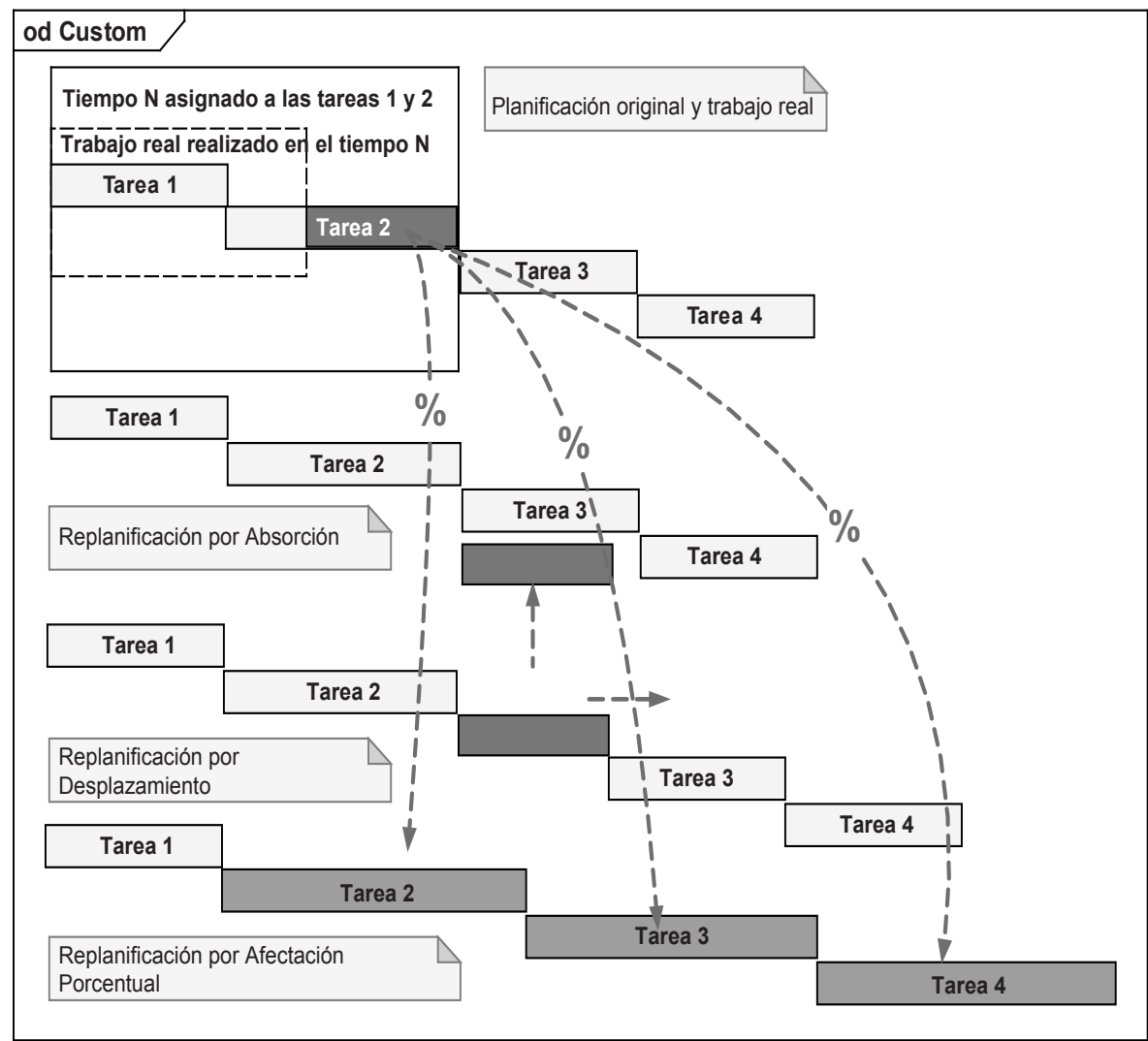


Figura 19.1. Esquema que muestra las posibles formas de replanificación y cómo resulta afectado el tiempo total del proyecto

19.1.2 Metodologías ágiles

Un aspecto clave de las metodologías ágiles es la gran visibilidad de todos los sucesos en la evolución de los proyectos. Por esta razón el seguimiento se basa fuertemente en realizar en forma disciplinada las actividades que generen la visibilidad necesaria.

El otro componente fundamental de una buena gestión son las acciones orientadas a la adaptación del proyecto a los cambios. Por esta razón es importante la realización de un ejercicio de planificación continua que permita afrontar estos cambios [3], [4]. En adelante asumimos que se trabaja gestionando el proyecto según Scrum.

A esta visibilidad y adaptación contribuyen tareas ineludibles como la adaptación diaria del Sprint Backlog con el estado de avance de las diferentes tareas. La adaptación del Product Backlog con los pedidos de cambio y la realización de las reuniones diarias y de revisión de los Sprints donde se tomen las decisiones que conducen el proyecto.

Esto es así ya que la no adaptación del Sprint Backlog, por ejemplo, quita visibilidad al proyecto, oculta el verdadero progreso y deforma la estimación de la velocidad de desarrollo.

Otro aspecto importante del seguimiento de los proyectos es la forma en que se comunican al resto de la empresa los avances y logros del proyecto. Muchas veces es necesario ser cuidadoso, sobre todo cuando las gerencias poseen una visión más tradicional de la gestión de los proyectos de desarrollo. En estas situaciones es imprescindible la traducción del Scrum al formato y al idioma tradicional de la gestión para publicar el proyecto de manera adecuada [6].

19.1.2.1 Daily Scrum

El seguimiento más cercano comienza con las reuniones informales del grupo de desarrollo en forma diaria llamadas *Daily Scrum*.

Esta es una reunión que busca la comunicación estrecha y la generación de un ambiente de confianza dentro del grupo de desarrollo. En estas se comparten dudas y conocimientos; se refuerzan actitudes de colaboración y se comparten aciertos. Son coordinadas por el Scrum Master y sus características salientes son:

- Frecuencia diaria.
- Frente al pizarrón utilizado para el proyecto.
- Con una duración que no exceda los 15 minutos.
- Con los miembros del grupo parados.
- Con horario y puntualidad administrados.
- Cada miembro a su turno describe sus actividades del día anterior y las programadas para el presente.
- Cada miembro adapta el estado de las Stories en desarrollo en las cuales están trabajando.
- El Scrum Master compila este avance en el Sprint Backlog a efectos de mantenerlo adaptado.

19.1.2.2 Sprint review meeting

Un componente importante de cada Sprint es la validación del logro de los objetivos a partir de una demostración al Product Owner. Esta demostración es una forma objetiva y concreta de mostrar el logro de los objetivos. Los motivos para incluir dicha demostración al final de cada Sprint son:

- Generar confianza en el grupo a partir del reconocimiento del Product Owner y el resto de la empresa.
- Generar un aprendizaje en equipo del negocio sobre el que se está trabajando.
- Informar a todos los involucrados en el proyecto.
- Generar un ambiente similar a una entrega aunque no lo sea a efectos de crear compromiso y actitud de los desarrolladores.
- Obligar a demostrar con hechos la implementación de los requerimientos sobre los que se trabajó.
- Reforzar la confianza en el Product Owner y en los patrocinantes acerca de la factibilidad del proyecto.

Estas presentaciones deben constituir un mecanismo de utilidad al proyecto y no un gasto inútil de esfuerzo. Por esta razón se recomienda [6]:

- Introducir a los presentes en el proyecto, sobre todo a aquellos que no estén directamente relacionados y familiarizados.
- No gastar esfuerzo extra en demostraciones estéticas.
- Ser eficiente y no invertir en aspectos burocráticos.
- Centrarse en el valor agregado al negocio y no en cuestiones técnicas.
- Permitir interactuar a los presentes con el producto.

19.1.2.3 Sprint retrospectives

Entre los aspectos importantes de estas metodologías se encuentra la promoción de la mejora continua. Con este fin se realiza al finalizar cada Sprint una reunión en la que se analiza al comportamiento del grupo en dicha iteración. Los objetivos de esta autocrítica son:

- Lograr instalar el aprendizaje en grupo.
- Generar el espacio para hacer consultas y reconocer errores.
- Generar un ambiente de análisis para hablar de las oportunidades de mejoras.

Entre las recomendaciones para la organización de estas reuniones se encuentran [6]:

- La duración no debe exceder un medio día.
- Debe asegurarse la participación de todos los involucrados.
- Realizarlas en un lugar alternativo al de trabajo para evitar dispersiones.
- Asignar a alguien el rol de anotador.
- El Scrum Master en carácter de líder del grupo debe hacer un resumen del Sprint pasado.
- Cada asistente a su turno hace el suyo.
- Analizar las diferencias de velocidad en el desarrollo con respecto a otros Sprints.
- Listar las mejoras propuestas para el próximo Sprint.

19.1.2.4 Informes

A efectos de realizar un seguimiento de la forma en que el Sprint afectó al proyecto se acostumbra listar el Product Backlog en el inicio de los Sprints y de esa forma poner en evidencia las Stories implementadas en ellos. De esta manera se informan también los cambios introducidos.

19.1.2.5 Product Backlog Burn down chats

Este informe consiste en un gráfico de dos dimensiones y puede mostrar la evolución del trabajo pendiente en base a los Stories no implementados del Product Backlog o del Sprint. En el eje vertical del gráfico se representa la cantidad de esfuerzo pendiente mientras que en el eje horizontal los días del proyecto o del Sprint según sea el caso. Debido a que con el transcurso del tiempo la cantidad de trabajo pendiente debe decrecer a medida que el proyecto avanza, la variación de la gráfica consistirá en la unión a través de segmentos de recta de los puntos de una serie temporal decreciente. En la figura 19.2 se muestra en forma esquemática un ejemplo de este informe.

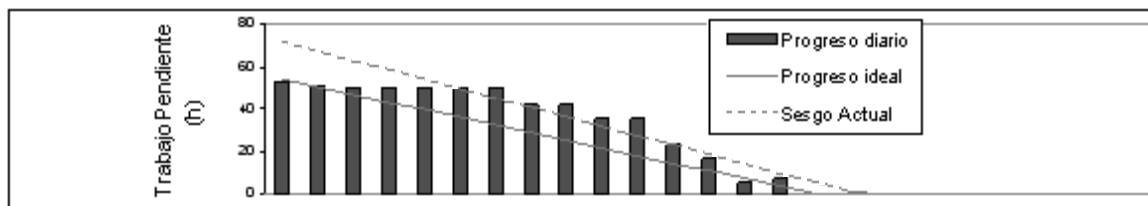


Figura 19.2. Product Backlog Burn down chat para un Sprint de un proyecto

Este diagrama es útil para predecir cuándo se completará todo el trabajo ya que a partir de la velocidad de desarrollo de los últimos Sprints se puede trazar una recta cuya pendiente depende de la velocidad mencionada y así en el punto de intersección con el eje horizontal determinar el tiempo de terminación estimado.

En el próximo capítulo aplicamos estos conceptos al desarrollo del sistema del proyecto del caso testigo.

19.2 Conclusión

En este capítulo se presentaron dos formas diferentes de enfocar el seguimiento de los proyectos. No se buscó hacer una comparación entre ambos sino mostrar los mecanismos utilizados por las distintas formas de trabajo. Está claro que cualquiera sea la elegida, el monitoreo y control de los proyectos es de fundamental importancia para su evolución.

Por un lado se trabaja gestionando riesgos a efectos de facilitar el desarrollo del proyecto y negociando la replanificación del proyecto en base a la cancelación de requerimientos o la extensión del tiempo.

Por otro se trabaja generando la mayor visibilidad posible y maximizando los requerimientos a implementar en cada una de las iteraciones (Sprints) de tiempo y presupuesto fijo.

19.3 Contenido de la página web de apoyo



19.3.1 Mapa conceptual

19.3.2 Autoevaluaciones

19.3.3 Presentaciones en Power Point (exclusivo para el docente)

19.4 Referencias

- [1] *Managing Iterative Software Development Projects*, Kurt Bittner, Ian Spence, Addison-Wesley, 2006.
- [2] *The Unified Software Development Process*, Jacobson, Ivar, et ál., Addison-Wesley, 1998.
- [3] *Agile Management for Software Engineering: Applying the Theory of Constraints for Business Results*, David J. Anderson, Eli Schragenheim, Prentice Hall PTR, 2003.
- [4] *SDLC 3.0 – Beyond a tacit understanding of Agile*, Mark Kennaley, Fourth Medium Press, 2010.
- [5] *Calidad en el Desarrollo de Software*, Guillermo Pantaleo, AlfaOmega, 2011.
- [6] *Scrum and XP from the Trenches - How we do Scrum*, Henrik Kniberg, 2007, Free Online Version, Support this work, buy the print copy: <http://infoq.com/minibooks/scrum-xpfrom-the-trenches>.
- [7] *Software Project Survival Guide*, Steve McConnell, Microsoft Press, 1996.

Gestión del proyecto del caso testigo
Planificación

Contenido

20.1 Preventa	340
20.2 Metodología conducida por los planes	345
20.3 Metodología ágil	353
20.4 Conclusión	358
20.5 Contenido de la página web de apoyo	359
20.6 Referencias	359
20.7 Apéndice - Visión del proyecto	359

Objetivos

- Cómo se realiza la pre venta del proyecto utilizando metodologías conducidas por los planes
- Cómo se realiza la pre venta del proyecto utilizando metodologías ágiles
- Cómo se acuerda el proyecto según la metodología de desarrollo utilizada
- Cómo se realizan las estimaciones y la planificación según la metodología de desarrollo utilizada

Competencias

- Elaborar una propuesta técnico económica para un proyecto de desarrollo de software
- Planificar un proyecto de desarrollo de software

20.1 Preventa

Según fue narrado en la presentación del caso testigo, la dirigencia de la empresa de transporte de cargas, en adelante TransCargas, espera de la empresa desarrolladora, en adelante SoftDesa, una propuesta técnico económica lo antes posible. Este proyecto es de interés para SoftDesa ya que se encuentran en un período de expansión de los mercados verticales para los que desarrolla sistemas, y la logística y transporte está en sus planes desde hace tiempo. Por lo tanto no es un proyecto más para la empresa. A partir del primer contacto, del cual surgió la información volcada en la presentación del caso, ha habido algunas charlas entre gerentes de ambas empresas y han acordado la elaboración de la propuesta de parte de SoftDesa para evaluar la posibilidad del desarrollo del sistema.

20.1.1 Propuesta técnico económica

Para la elaboración de la propuesta técnico económica SoftDesa ha asignado un analista (Mary) y un líder de proyecto (Juan), más el apoyo técnico de los líderes técnicos que sea necesario. En una reunión mantenida entre ambos profesionales y el gerente de desarrollo (Enrique) les fueron comunicadas algunas pautas para el trabajo que deben hacer. Una, quizás la más importante, es que la propuesta debe estar lista para ser revisada por la gerencia y luego ser entregada a TransCargas en los próximos diez días hábiles. Además fueron contactados vía mail con los referentes del negocio que TransCargas asignó para atender sus consultas. La empresa acostumbra trabajar en este tipo de compromisos por ese lapso de tiempo y de ganar el proyecto, el costo de la propuesta es recuperado ya que se le factura al cliente como parte del proyecto. En los casos en que los proyectos no les son asignados, este costo es contabilizado como una pérdida. Es parte del riesgo que las empresas deciden correr al participar del negocio de desarrollo de software en estos escenarios de desarrollo de sistemas a medida.

En los meses previos a este evento, los miembros de SoftDesa, como parte de las mejoras que de tanto en tanto promueven, han estado analizando y discutiendo acerca de la forma de trabajo del área de desarrollo y la relación entre esta y la forma en que se venden los proyectos. En estas reuniones han participado Mary, Juan y un par más de personas técnicas representando a los desarrolladores; también Enrique y el gerente comercial (Patricio) representando a la gerencia. La cuestión es que, a pesar de no haber llegado a un acuerdo ni conclusión, Mary y Juan piensan que este proyecto podría ser una oportunidad para generar un cambio.

La empresa posee una tradición de desarrollos que utilizan metodologías en cascada, es decir no iterativas. A pesar de haber incorporado tecnologías orientadas a objetos, aún no han logrado adaptarse a la dinámica de su funcionamiento. Además se ha incorporado a la empresa un grupo de desarrolladores con alguna experiencia en metodologías ágiles que promueven su uso sin mucho éxito. Solo han logrado que se les permita utilizar Extreme Programming en un pequeño desarrollo interno.

20.1.1.1 Relevamiento preliminar

Mary y Juan comienzan trabajando en colaboración con los dos referentes de la empresa TransCargas relevando requerimientos preliminares para lo cual utilizan el cuestionario del apéndice del capítulo 18, Gestión de proyectos – Estimación y planificación. La intención es conocer las necesidades y los problemas de la empresa y comenzar a construir una visión para el proyecto. Para esto solicitan una cita con los gerentes propietarios de

TransCargas a efectos de conocer de primera mano la opinión de quienes conducen la empresa. La meta de ellos es que la propuesta esté basada en una visión compartida que aunque sea temprana sirva de sustento al proyecto y ayude a establecer objetivos y prioridades. La experiencia en proyectos anteriores les ha enseñado que los proyectos que se piensan desde un inicio de esta manera, más allá de los inconvenientes propios de siempre, generalmente llegan a buen puerto. En todo caso cuando el proyecto comience se refinará y ajustará esta visión previa. Entre ambos, cada vez que elaboran y procesan la información de las reuniones que van teniendo con los miembros de TransCargas, intercambian opiniones que les ayuda a listar algunos riesgos preliminares del posible proyecto.

Como resultado de estos días de trabajo la propuesta va tomando forma. De esta se presenta en el apéndice de este capítulo un resumen de la visión elaborada para el proyecto [6]. En la tabla 20.1 se resume la visión según los involucrados y en la tabla 20.2 las prioridades del proyecto [5]. Los requerimientos preliminares son los listados en el capítulo 10, Relevamiento de requerimientos, solo que en este caso sin refinar

Tabla 20.1. Visión de los involucrados en el proyecto				
Involucrados	Valor mayor	Expectativas	Interés mayor	Restricciones
Directivos de TransCargas	Nuevos servicios	Mejor posicionamiento para captar nuevos clientes	Contrato con empresa de computación	Recursos humanos disponibles para el proyecto
Empleados de TransCargas	-	Mejores condiciones de trabajo	-	-
Clientes de TransCargas	Seguimiento de cargas en tránsito	-	Menores costos	-
Directivos de SoftDesa	Proyecto exitoso	-	Ingreso al mercado del software de logística	Recursos humanos disponibles para el proyecto
Desarrolladores de SoftDesa	Participación en un proyecto importante, novedoso y tecnológicamente interesante	Lograr el desempeño del grupo de desarrollo que ayudaron a definir y planificar	Lograr el reconocimiento en la empresa como miembros de un proyecto exitoso	-

Tabla 20.2. Prioridades del proyecto			
Dimensión	Conductores	Restricciones	Grados de libertad
Cronograma	La primera versión debe estar terminada en 6 meses	-	-
Alcance	Los requerimientos esenciales deben estar todos implementados	-	Según las prioridades asignadas, el seguimiento es altamente prioritario frente a los otros requerimientos.

Tabla 20.2. Prioridades del proyecto			
Dimensión	Conductores	Restricciones	Grados de libertad
Calidad	-	-	Las funcionalidades de seguimiento deben estar disponibles todo el tiempo
Personal	-	No se puede disponer de más recursos humanos que los asignados	-
Costo	-	No debe superar el costo planificado	-

En la tabla 20.3 se muestra un listado de los riesgos preliminares detectados al momento de la construcción de la propuesta [6]. Estos serán utilizados para armar una estrategia que será volcada en la propuesta y servirá como base para la que se elabore cuando el proyecto comience.

Estrategia

A partir de la visión acordada y los riesgos detectados se tratará de obtener acuerdos en los que haya más de un involucrado por parte de TransCargas. Se utilizará al capataz como referente de los usuarios dentro de la empresa tanto para obtener requerimientos como para la realización de capacitaciones tempranas en el uso del sistema a medida que se desarrollen las iteraciones.

Se contratarán dos desarrolladores con experiencia intermedia para cubrir la falta de recursos propios. Se pondrá énfasis en el desarrollo temprano de la funcionalidad orientada a la realización del seguimiento y se armarán demostraciones con la asistencia de miembros de clientes potenciales y de la empresa de computación.

Se trabajará en forma iterativa con gran visibilidad a efectos de corregir cualquier desviación que pueda tomar el proyecto en su evolución.

A partir de una estimación de esfuerzo realizada según el alcance definido por los requerimientos con el soporte de algunos de los líderes técnicos surgió que el proyecto demandará un tiempo de desarrollo de ocho (8) meses con un grupo de cinco (5) personas. Con estas estimaciones se elaboró el presupuesto que se incluyó en la propuesta técnico económica.

Además, la propuesta incluyó recomendaciones acerca de la implementación con tecnología de algunas plataformas de desarrollo como Spring, Hibernate, Struts y otras. También se dimensionó la infraestructura de hardware necesaria para el ambiente operacional del sistema la cual formó parte de la propuesta.

Finalmente se incluyó una propuesta de trabajo que implicó la participación de los involucrados de ambas empresas, el ciclo de vida que tendría el proyecto y los compromisos de ambas partes.

Motivado por la idea de cambio promovida por los desarrolladores y después de largas discusiones con la gerencia se plantea la posibilidad de llevar adelante el proyecto según alguna forma de trabajo conducida por

los planes o por una forma ágil. Se acuerda que en la decisión final serán tenidos en cuenta aspectos técnicos y de gestión propios de la empresa y además la conformidad y los acuerdos que puedan establecerse con la empresa cliente.

Tabla 20.3. Listado de los riesgos detectados y su nivel de exposición					
ID	Enunciado del riesgo		Probabilidad	Pérdida	Exposición
	Condición	Consecuencia			
1	Los usuarios idóneos no colaboren con el proyecto	Indefiniciones en el relevamiento de requerimientos y retraso de acuerdos	0,75	8,00	6,00
2	Dificultades en la introducción del sistema en la empresa para el uso de parte de los usuarios	Retrasos en lograr el funcionamiento pleno del sistema en producción	0,75	6,00	4,50
3	Falta de desarrolladores en condiciones de formar parte del grupo de desarrollo	Sobrecarga de tareas de los desarrolladores y alargamiento del cronograma	0,75	6,00	4,50
4	Restricciones de tiempo condicionan el contrato con la empresa de computación	Presión de carácter político al no poder cambiar este hito impuesto por terceros	0,75	6,00	4,5
5	Falta de experiencia de SoftDesa en el desarrollo iterativo	Retrasos en el cronograma	0,75	4,00	3,00
Exposición - Sub total principales 5 riesgos					22,50
6	Falta de experiencia de TransCargas en la compra de proyectos de desarrollo	Retraso en el cronograma por no cumplimiento de compromisos en término	0,75	4,00	3,00
7	Compromisos asumidos por el presidente no se cumplan	Retrasos por falta de recursos y / o definiciones	0,25	8,00	2,00
8	Disponibilidad del software de base fuera de tiempo	Retrazo en la transferencia del proyecto	0,50	4,00	2,00
Exposición - total de todos los riesgos					29,50

A partir de este punto describiremos en paralelo y a lo largo de todo este capítulo dos escenarios diferentes con dos finales para cada uno. Ambos corresponden a una forma de trabajo distinta.

Cada uno de los dos escenarios comienza, como se describe a continuación, con la negociación con el cliente.

20.1.1.2 Contrato

Después de haber seleccionado trabajar según una metodología conducida por los planes, cuyo proceso de selección se describe más abajo, se propone al cliente el contrato que regulará el proyecto [4].

En este se especifica que por la construcción del sistema con los N requerimientos incluidos en la propuesta se trabajará un tiempo T y se cobrará un precio P . Se dispone además que en caso de tener que renegociar el contrato, esto solo se hará de común acuerdo.

Se establece que en caso de conflictos este contrato es el instrumento que se usará para resolver las obligaciones y derechos de ambas partes.

Observaciones: TransCarga no es una empresa habituada a la compra de proyectos de software, lo cual implica que no tiene conocimiento de la incertidumbre inherente a todo proyecto de desarrollo de software presente en el contexto y al momento en que se firma un contrato de inicio de un proyecto. Por lo tanto el contrato que se firma aparenta ser como cualquiera de los contratos que habitualmente firma con sus proveedores de bienes y servicios del negocio al cual se dedican.

20.1.1.3 Acuerdo colaborativo

Después de haber seleccionado trabajar según una metodología ágil, cuyo proceso de selección se describe más abajo, se propone al cliente el acuerdo colaborativo que regulará el proyecto [4].

Este establece que se trabajará por un tiempo T , que se cobrará un precio P y que a lo largo de la duración del proyecto se harán todos los esfuerzos necesarios a partir de un trabajo conjunto entre los involucrados de ambas partes para que el sistema que se entregue después del tiempo T tenga implementado la mayor cantidad de requerimientos posible de los N relevados u otros que surjan, siempre de acuerdo a las prioridades fijadas por TransCargas.

Observaciones: TransCarga no es una empresa habituada a la compra de proyectos de software, por lo tanto se niega en primera instancia a firmar un contrato que no especifica concretamente el alcance del producto por el que está pagando. Los miembros de SoftDesa se topan con el primer escollo en su camino hacia la agilidad. Se dan cuenta que para vender un proyecto de esta forma hace falta generar un ambiente de mayor confianza con el cliente. Por esta razón deciden dedicar tiempo extra a realizar exposiciones a los miembros de TransCargas para informarlos acerca de las dificultades que presenta la incertidumbre en los proyectos de software. También qué riesgos implica para ambas partes firmar un contrato en base a información preliminar y finalmente inducirlos y convencerlos de las bondades de esta forma de trabajo frente a la regida por los planes y contratos. Cuando el proyecto parece irremediabilmente perdido, reciben una llamada telefónica de uno de los gerentes de la empresa cliente y luego un mail en el que aceptan acordar el proyecto de esta forma.

Sin saberlo, SoftDesa hizo un planteo correcto en un aspecto esencial de los proyectos de desarrollo. Vinculó la forma de trabajo del proyecto al modo en que propuso venderlo a su cliente. Es común observar en el mercado casos en los que los proyectos son comprados y vendidos de una forma totalmente ajena a como evolucionarán. Las metodologías no son sólo algo que afecta a los desarrolladores, sino al proyecto en su totalidad.

20.2 Metodología conducida por los planes

20.2.1 Selección de la metodología

Existían tres miradas diferentes en SoftDesa acerca de la forma de trabajo en los proyectos de desarrollo. Las diversas miradas eran sostenidas por personas con diferentes experiencias y estudio, con diferentes roles y distinto poder de decisión. Además de la opinión de estas personas, influía la historia de la empresa y los proyectos exitosos y malogrados que habían sido llevados adelante en los últimos años. A continuación describimos cuáles eran estas opiniones y cómo se decidió utilizar una metodología conducida por los planes para el proyecto del caso testigo [1].

20.2.1.1 Participantes

Gerente general (Pablo): Licenciado en administración de empresas, con un pasado de vendedor de software exitoso para una multinacional, de reflejos rápidos ante cambios del negocio, intuitivo, con un excesivo gusto por el protagonismo y exposición. Una falencia, no delega responsabilidades a los roles definidos.

Gerente comercial (Patricio): Licenciado en ciencias económicas, postgrado en administración de empresas con especialidad en el software, de gran visión para los negocios, con una gran capacidad de trabajo.

Gerente de desarrollo (Enrique): Ingeniero en informática, trabajó en administración de sistemas gran parte de su carrera, escasa experiencia en desarrollo, gran capacidad de organización.

Líder de proyecto (Juan): desarrollador de experiencia y muy buena comunicación con sus colegas, ingeniero en informática. Representaba a los líderes de proyecto y analistas en la reunión.

Líder técnico (Germán): desarrollador de experiencia, certificado en múltiples tecnologías, buena capacidad docente. Representaba a los programadores en la reunión.

Las tres miradas instaladas en la empresa hasta el momento eran:

Propuesta de utilización de metodologías ágiles: la mayoría de los desarrolladores; Germán y Juan lo consideraban posible.

Propuesta de utilización de una forma de trabajo controlada: algunos analistas, Pablo y Enrique.

Propuesta de utilización de una forma de trabajo con delegación: los líderes de proyecto y algunos analistas, Juan, Germán y Patricio.

20.2.1.2 Reunión de discusión

Pablo – *Este es el momento propicio para definir una metodología de desarrollo ya que la empresa pretende encarar este proyecto con todas las definiciones, recursos y recaudos para que este salga bien.*

Ya hemos tenido muchas de estas reuniones y no podemos dilatar esta definición.

Como algunas vez les contamos con Enrique, nosotros estamos convencidos de que hace falta un organización que nos permita controlar de cerca los proyectos y el trabajo de cada desarrollador.

Germán – *La opinión de ustedes, la clase dirigente de la empresa, nos es compartida por la mayoría de los miembros de esta organización.*

Los programadores de todos los proyectos esperan que haya una definición al respecto. Todos ustedes conocen la opinión de la mayoría de ellos, una metodología ágil es lo que consideran indicado. Las publicaciones cuentan que este es el sesgo en la industria hoy día y no podemos ir a contramano.

Patricio – *Es verdad lo que los programadores expresan pero solo a medias. Las metodologías ágiles necesitan de desarrolladores más experimentados y con más conocimientos que los que nosotros mayoritariamente tenemos. Además necesitamos hacer un proyecto interno o no crítico primero antes de lanzarnos a utilizar estas metodologías. Corremos el riesgo de desaprovechar todo lo bueno que promueven y malograr su uso en la empresa por una primera mala experiencia.*

Juan – *Todos nosotros conocemos la amplitud mental de la dirigencia de esta empresa al consultarnos acerca de su propia organización y valoramos esto. Sin embargo no compartimos su opinión de instalar una forma de trabajo en cascada con control vertical sobre los miembros de los proyectos. Está probado que aún utilizando esta forma de trabajo hubo proyectos en el pasado que fracasaron. Proponemos utilizar una metodología que sin ser ágil promueva la toma de decisiones en los proyectos, que sea iterativa e incremental y que sirva de paso intermedio al futuro uso de alguna metodología ágil. Aún sin ser ágil deberemos adecuar algunos procesos actuales que de continuar como están nos generarán inconvenientes.*

Enrique - *¿Cuáles?*

Juan – *Cómo se escalan los problemas de los proyectos, cómo se acuerdan algunas cuestiones con los clientes, cómo se asignan recursos para las pruebas y cómo se deciden la aceptación o rechazo de cambios a los requerimientos. Es decir los asociados a los canales de comunicación y niveles de decisión.*

Pablo – *Bien, creo que esta propuesta ya la habíamos analizado, me gustaría conocer qué significa decisión por parte de los desarrolladores. Pero estoy de acuerdo que parece ser la adecuada por el momento.*

Juan – *Significa que hay cuestiones que Enrique deberá delegar en los líderes y miembros de los grupos de proyecto, esto es lo que los sajones llaman “empowerment”.*

20.2.2 Estimación y planificación del proyecto

20.2.2.1 Estimación de esfuerzo

Del refinamiento de los requerimientos y su trazabilidad a casos de uso y otros componentes surgieron las siguientes cantidades para el alcance del sistema [6]:

Cantidad de casos de uso: 16 de complejidad media y 2 de baja.

Cantidad de interfaces con otros sistemas: 2 de complejidad alta y 1 de complejidad media.

Cantidad de reportes: 10 de complejidad media.

Trasladadas estas cantidades al esfuerzo correspondiente a partir del esfuerzo unitario expresado en la tabla 18.1 se obtienen los siguientes números para el esfuerzo de **construcción**:

Casos de uso: $(16 \times 112 + 2 \times 56)$ horas hombre = 1904 horas hombre.

Interfaces con otros sistemas: $2 \times 134 = 268$ horas hombre.

Reportes: $10 \times 44 = 440$ horas hombre.

Extrapolamos estos números a la totalidad del proyecto según el criterio expresado en la figura 18.5 y obtenemos una duración total para el proyecto de 5224 horas hombre. Afectada esta estimación por los factores listados en la figura 18.3, los cuales suman un 20% en la dirección del alargamiento, obtenemos como estimación para el total de la planificación 6269 horas hombre.

Asumiendo meses de 160 horas de trabajo por persona esta planificación tendría como resultado final que en promedio trabajarán 5 personas durante 8 meses.

Esta planificación no es factible para el proyecto ya que según las prioridades fijadas en 6 meses deberían contar con la funcionalidad asociada al seguimiento. El proyecto ya está 15 días atrasado pues los 6 meses eran contados desde el momento en que se hizo el primer contacto con TransCargas. Además se necesitan unos 15 días adicionales con el sistema funcionando para que la aseguradora haga sus pruebas y lo certifique a efectos de que la empresa de computación asigne el servicio. Esta es una de las razones por las cuales se decidió y se propuso una estrategia diferente que consiste en desdoblar el proyecto en dos, un primer entregable con la implementación de los requerimientos de alta prioridad, es decir aquellos vinculados al seguimiento, y un segundo con el resto. Los números de esta alternativa, la cual fue acordada con TransCargas, son los siguientes:

Entregable 1

Alcance

Cantidad de casos de uso: 5 de complejidad media y 2 de baja.

Cantidad de interfaces con otros sistemas: 0.

Cantidad de reportes: 2 de complejidad media.

Esfuerzo de construcción

Casos de uso: $(5 \times 112 + 2 \times 56)$ horas hombre = 672 horas hombre.

Interfaces con otros sistemas: 0.

Reportes: $2 \times 44 = 88$ horas hombre.

Esfuerzo total sin afectar: 1520 horas hombre.

Esfuerzo total afectado: 1748 horas hombre.

Planificación: 4 meses con 3 personas en promedio.

Esta alternativa deja para el segundo entregable el resto de la funcionalidad. Además permite organizar al grupo de desarrollo con los recursos disponibles al momento del comienzo del proyecto e incorporar desarrolladores adicionales que se liberarán de los proyectos que van terminando mientras tanto.

20.2.2.2 Entregables

Los dos entregables según la estrategia establecida incluirán lo siguiente:

Entregable 1:

- Alta y modificación de clientes
- Alta de servicio de carga
- Registro de cargas
- Registro de descargas
- Consulta de estado de cargas
- Notificación de arribo de cargas.

Entregable 2:

- Administración de clientes por historia, tipo y región
- Especificación de itinerario y ruteo de cargas
- Validación del la capacidad de transporte propio de carga
- Tercerizar transporte de cargas
- Funcionalidad asociada a la facturación de los servicios
- Funcionalidad asociada a la conexión con el sistema de transporte de terceros
- Funcionalidad asociada a la programación de viajes
- Reportes varios de cargas, clientes, facturación e itinerarios.

20.2.2.3 Administración de riesgos

La gestión del proyecto será conducida por la mitigación de los riesgos detectados al comienzo, los cuales serán adaptados con una frecuencia de 15 días. Las acciones orientadas a mitigarlos, los responsables de llevarlas adelante y las fechas esperadas de implementación de dichas acciones se muestran en el listado 20.4 que sigue. El líder del proyecto realizará un seguimiento diario de los riesgos incorporando nuevos y supervisando las acciones planificadas.

Tabla 20.4. Acciones planificadas para la mitigación de los riesgos					
ID	Enunciado del riesgo		Acciones		
	Condición	Consecuencia	Acción	Responsables	Fecha
1	Los usuarios idóneos no colaboren con el proyecto	Indefiniciones en el relevamiento de requerimientos y retraso de acuerdos	A través de Rodríguez acceder a los diferentes usuarios y enfatizar la importancia de su aporte. Planificar reuniones de relevamiento	Juan y Mary	Lo antes posible

Tabla 20.4. Acciones planificadas para la mitigación de los riesgos

ID	Enunciado del riesgo		Acciones		
	Condición	Consecuencia	Acción	Responsables	Fecha
2	Dificultades en la introducción del sistema en la empresa para el uso de parte de los usuarios	Retrasos en lograr el funcionamiento pleno del sistema en producción	Como parte de la misma relación con los usuarios, desarrollar capacitaciones y participación de ellos en las pruebas	Juan y Mary	Planificar para hacerlo cuando se liberen los primeros entregables
3	Falta de desarrolladores en condiciones de formar parte del grupo de desarrollo	Sobrecarga de tareas de los desarrolladores y alargamiento del cronograma	Coordinar con Enrique y los responsables de recursos humanos la contratación de dos desarrolladores para el segundo entregable del proyecto	Juan, Enrique	Un mes antes del comienzo del segundo entregable
4	Restricciones de tiempo condicionan el contrato con la empresa de computación	Presión de carácter político al no poder cambiar este hito impuesto por terceros	Coordinar reuniones con Pablo y Patricio para mostrar los avances del proyecto en ambas empresas	Juan, Enrique, Pablo, Patricio	Con frecuencia quincenal mientras dure el primer entregable
5	Falta de experiencia de SoftDesa en el desarrollo iterativo	Retrasos en el cronograma	Realizar un seguimiento cercano del proyecto con mucha visibilidad y capacitaciones necesarias	Juan, Mary y Germán	Todo el tiempo del ciclo de vida del proyecto
6	Falta de experiencia de TransCargas en la compra de proyectos de desarrollo	Retraso en el cronograma por no cumplimiento de compromisos en término	Mantener informados a Rodríguez y Jorge y avisar con antelación de por lo menos una semana de los hitos del proyecto	Juan, Enrique	Todo el tiempo del ciclo de vida del proyecto

Tabla 20.4. Acciones planificadas para la mitigación de los riesgos					
ID	Enunciado del riesgo		Acciones		
	Condición	Consecuencia	Acción	Responsables	Fecha
7	Compromisos asumidos por el presidente no se cumplan	Retrasos por falta de recursos y / o definiciones	Establecer acuerdos y compromisos en presencia de otros responsables como Jorge por ejemplo.	Enrique	Todo el tiempo del ciclo de vida del proyecto
8	Disponibilidad del software de base fuera de tiempo	Retrazo en la transferencia del proyecto	Planificar con antelación de por lo menos dos meses la compra y distribwución de la infraestructura necesaria	Juan, Germán, Enrique	Una vez definida la arquitectura del sistema

20.2.2.4 Estrategia del proyecto [7]

Como fue determinado al realizar la estimación del esfuerzo necesario, se acordó con los responsables de TransCargas la realización de dos entregables para el proyecto. El primero orientado a lograr el objetivo de ganar el contrato con la empresa de computación, para lo cual es necesario contar con el seguimiento de las cargas. Por esta razón se planificó desarrollar las funcionalidades de cada paquete que permita instalar el sistema parcialmente y cubrir de esta manera los requerimientos asociados. En un segundo entregable se completará el resto de los requerimientos. De esta forma también se contará para el segundo con desarrolladores nuevos y con un par adicionales que se liberarán de otros proyectos ahora en curso.

Como fue expresado en la estimación de esfuerzo esta planificación permitirá un desarrollo en los tiempos impuestos por el negocio.

A la vez se trabajará mitigando los riesgos detectados, como se mostró en la planificación de las acciones correspondientes, las cuales apuntan a mantener una gran visibilidad y un contacto estrecho con el cliente.

20.2.2.5 Cronograma del proyecto

La planificación anterior para el entregable primero se distribuyó en el tiempo y se asignaron recursos a cada una de las tareas incluidas. En la figura 20.1 se presenta parte de dicha distribución donde además de las tres iteraciones programadas para la fase de construcción puede verse la actividad de QA asociada al proyecto que se desarrollará en forma periódica a través de las revisiones del proceso seguido y de los productos generados. También se analiza en estas sesiones la evolución de los errores fijados y corregidos.

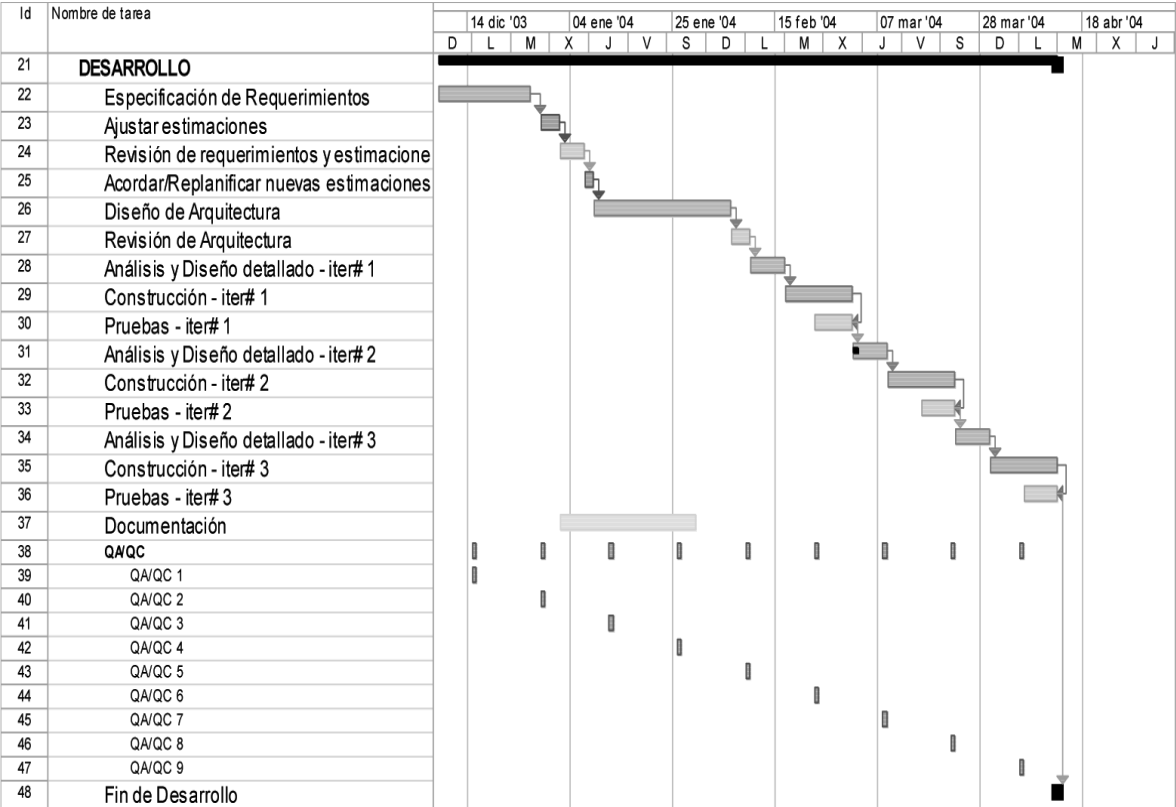


Figura 20.1. Cronograma del proyecto donde se muestra la distribución de las tareas e hitos de la planificación

20.2.2.6 Organigrama del proyecto

Respecto de la organización del proyecto en sus aspectos vinculados a la comunicación entre los involucrados y asignación de roles se decidió una organización como la que se muestra en la figura 20.2.

Como se observa en la figura se han indicado con líneas punteadas dos ámbitos diferentes, uno el interno a SoftDesa y otro a TransCargas. Ambos incluyen a los involucrados en el proyecto por las dos partes. Juan y Rodríguez como responsables, facilitadores y coordinadores del proyecto en ambas partes constituyen el nexo. El seguimiento del proyecto se realiza como se indica en una sección próxima de acuerdo a lo indicado en la figura.

20.2.2.7 Ciclo de vida

De las estimaciones de esfuerzo, la disponibilidad de recursos y la forma de trabajo seleccionada se derivó el ciclo de vida del proyecto. En la figura 20.3 se muestra la distribución de recursos y asignación de tiempos a cada una de las fases del ciclo de vida.

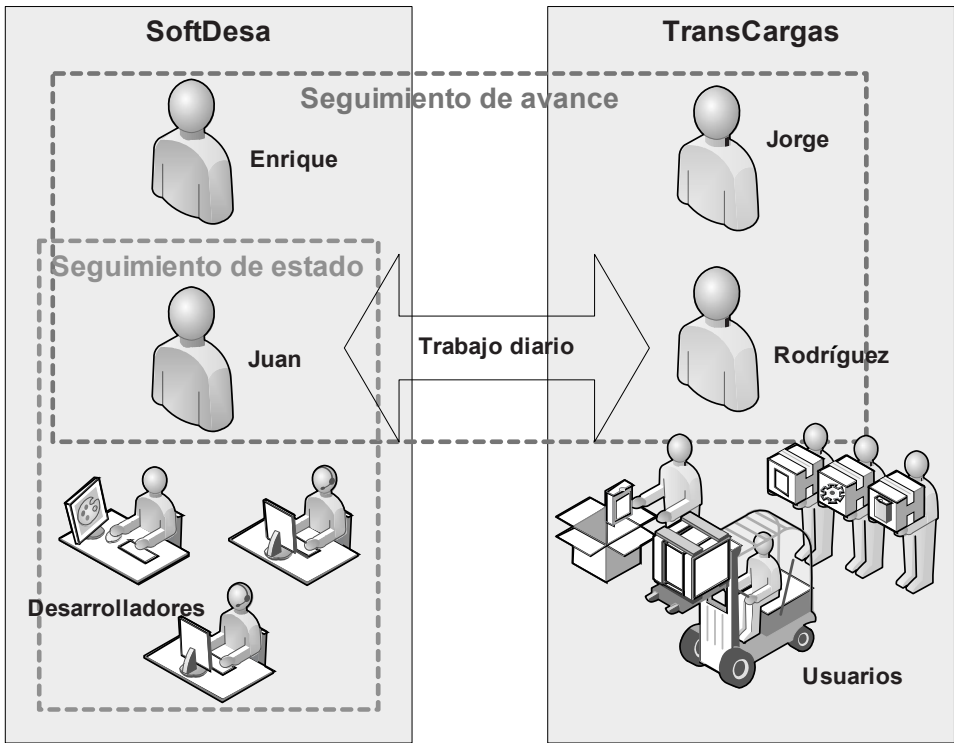


Figura 20.2. Organigrama del proyecto para la metodología conducida por los planes seleccionada

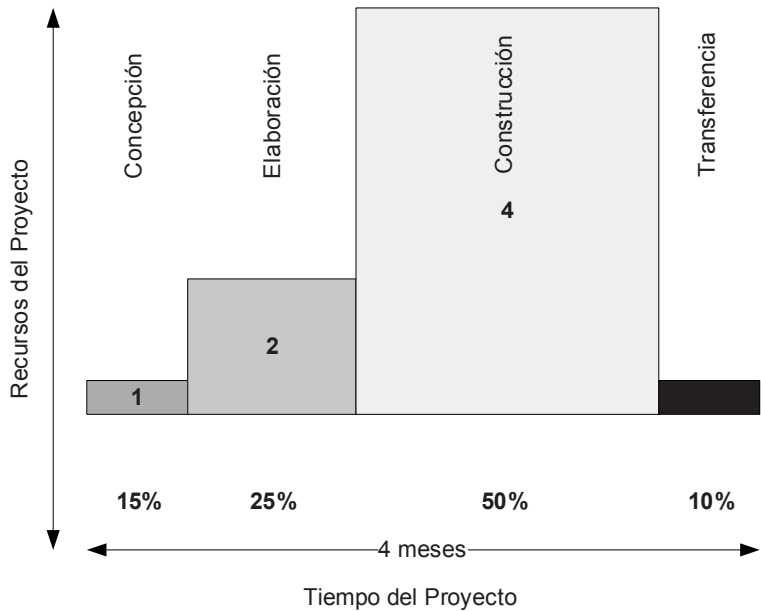


Figura 20.3. Distribución de los recursos (desarrolladores) y tiempos del proyecto a lo largo de su ciclo de vida. Estos recursos no incluyen al líder del proyecto

20.2.2.8 Plan de calidad

Basado en las definiciones anteriores al proyecto realizadas en la SoftDesa con el objetivo de mejorar el proceso de desarrollo se planificaron las tareas del proyecto. Se armaron *check list* de productos y procesos. Estas revisiones serían llevadas adelante por un analista de otro proyecto mientras se decidía la asignación de esa persona al rol de QA. Se decidió revisar los acuerdos de requerimientos, plan del proyecto y resultados de las pruebas de aceptación.

Respecto del resto de las pruebas se asignaron tiempos del cronograma para la implementación de las pruebas unitarias, de componentes, algunas funcionales y las de sistema sobre una plataforma de tipo xUnit.

20.2.2.9 Seguimiento

Se acordó entre ambas empresas realizar un seguimiento quincenal a través de reuniones de avance según el organigrama que se mostró en la figura 20.2. El seguimiento interno del grupo de desarrollo se llevará adelante en reuniones informales diarias coordinadas por el líder técnico (Germán) y semanales a cargo del líder del proyecto (Juan). En el próximo capítulo se presenta y analiza un informe del seguimiento del proyecto.

20.3 Metodología ágil

20.3.1 Selección de la metodología

Ya hemos presentado cuál es la situación en la empresa al momento de encarar esta decisión respecto de la existencia de tres opiniones diferentes. También quiénes eran los que las sostenían y qué rol cumplían en la organización.

20.3.1.1 Participantes

Los participantes de la reunión decisoria fueron los mismos que en el caso anterior, lo que variará es el desenlace de dicha reunión y que por supuesto tendrá consecuencias diferentes en el desarrollo del proyecto.

20.3.1.2 Reunión de discusión

Pablo – *Este es el momento propicio para definir una metodología de desarrollo ya que la empresa pretende encarar este proyecto con todas las definiciones, los recursos y recaudos para que éste salga bien.*

Ya hemos tenido muchas de estas reuniones y no podemos dilatar esta definición.

Como algunas vez les contamos con Enrique, nosotros estamos convencidos de que hace falta un organización que nos permita controlar de cerca los proyectos y el trabajo de cada desarrollador.

Germán – *La opinión de ustedes, la clase dirigente de la empresa, no es compartida por la mayoría de los miembros de esta organización.*

Los programadores de todos los proyectos esperan que haya una definición al respecto. Todos ustedes conocen la opinión de la mayoría de ellos, una metodología ágil es lo que consideran indicado. Las publicaciones cuentan que éste es el sesgo en la industria hoy día y no podemos ir a contramano.

Patricio – *No estoy plenamente convencido de que estemos en condiciones en la actualidad de adoptar una metodología ágil. Sin embargo mi experiencia me indica que la motivación de los desarrolladores al tomar la empresa una decisión alineada a sus expectativas redundará en un esfuerzo adicional por parte de todos en los proyectos. Si encauzamos toda esa energía podremos dar un impulso a los proyectos que de otra manera será difícil de lograr.*

Juan – *Está claro que eso es así, sin embargo deberemos capacitar a todos los desarrolladores no solo en aspectos técnicos sino que deberemos inducir a muchos en una metodología ágil que creen conocer pero en la cual no cuentan con experiencia real en proyectos.*

Enrique - *¿En qué aspectos no técnicos?*

Juan – *En elaborar diálogos inteligentes con el cliente a efectos de aprovechar el trabajo compartido en relevar requerimientos y planificar pruebas de funcionalidad, en autogestionarse en el trabajo diario y en elaborar un modelo técnico de desarrollo.*

Pablo – *Bien, apostaremos a la motivación de la gente, queremos ver los proyectos revitalizados.*

Juan – *Debemos planificar cuidadosamente entonces las capacitaciones e inducciones.*

20.3.2 Estimación y planificación del proyecto

La selección de una metodología ágil para el proyecto condujo a utilizar una combinación de Extreme Programming para los aspectos técnicos y Scrum para la gestión. A efectos de planificar la organización del proyecto fueron necesarias varias reuniones de inducción a la forma de trabajo para los miembros de TransCargas y capacitaciones para los miembros de SoftDesa.

Un aspecto al cual estas metodologías parecen no dar mayor importancia por no tratarlo en sus bases es la planificación y la definición de una arquitectura como pasos previos al desarrollo de las iteraciones (Sprints) [2], [3], [4]. En el proyecto estas tareas se desarrollaron en lo que se llamó *Sprint cero*.

Cabe mencionar que la visión y las estimaciones iniciales fueron las mismas que las descriptas para la otra metodología.

Esta planificación reforzó la estrategia esencial que se trabajaría durante 4 meses con un promedio de tres personas para maximizar la cantidad de los requerimientos (Stories) implementados de acuerdo a las prioridades fijadas por el cliente (TransCargas) a través de sus miembros involucrados en el proyecto. Esto para el primer entregable, para el segundo la estrategia sería repetida.

20.3.2.1 Organigrama y forma de trabajo

Para la organización del proyecto se eligió la misma estructura de roles que propone Scrum [2]. Esa se muestra en la figura 20.4. En ella deseamos enfatizar la incorporación del Product Owner al grupo de proyecto. Éste es el que define el sistema a construir, establece las prioridades, adapta el Product Backlog y acuerda con el Scrum Master las decisiones operativas del proyecto. También este rol representa al cliente, en este caso TransCargas, en las verificaciones de las implementaciones de los diferentes *Stories* y coordina la participación de los usuarios en las pruebas de aceptación.

Para el seguimiento se adoptó la forma de reuniones diarias informales (*Daily Scrum*) en el ámbito del grupo de desarrollo.

Para la evaluación de los Sprints en su final se realizará una revisión (*Sprint Review*) donde se analiza el desempeño del grupo y se acuerdan modificaciones en actividades puntuales.

La forma de trabajo que se definió se apoya en el código compartido, la refactorización y la programación de test de tipo xUnit. También se eligió trabajar realizando integración continua.

20.3.2.2 Construcción del Product Backlog

Como parte de la planificación del proyecto se construyó el Product Backlog en reuniones mantenidas por el grupo del proyecto [2]. Rodríguez tuvo una participación activa en la definición del sistema a pesar de ser su primera participación en un proyecto de estas características. Además contó con el apoyo de Jorge, quien motivó al resto de los usuarios para que tuvieran una actitud colaborativa a la hora de transmitir el conocimiento que ellos poseen del negocio. Una porción de este Product Backlog se muestra en la tabla 20.5. En ella se observan además del nombre de cada Story y su descripción, una serie de otros atributos que deseamos mencionar.

- El **estado** que será cambiado cuando cada una de estas *Stories* sea asignada a alguno de los Sprints y sea seleccionada por alguno de los desarrolladores para ser implementada.
- El **tamaño** que surgió de la misma reunión de planificación con el grupo de desarrolladores y fue asignado por consenso después de un análisis del alcance de cada Story y la experiencia previa en desarrollos similares.
- La **prioridad** que fue fijada por Rodríguez que necesitó varias consultas con Jorge y otros miembros de su empresa. Desde SoftDesa se sugirió utilizar los criterios que se utilizaron en el capítulo 10, Relevamiento de requerimientos, para establecer dichas prioridades [7].
- La **verificación** que fue acordada dentro del grupo pero sugerida por los desarrolladores como una forma objetiva de comprobar que la Story fue implementada como se especificó.

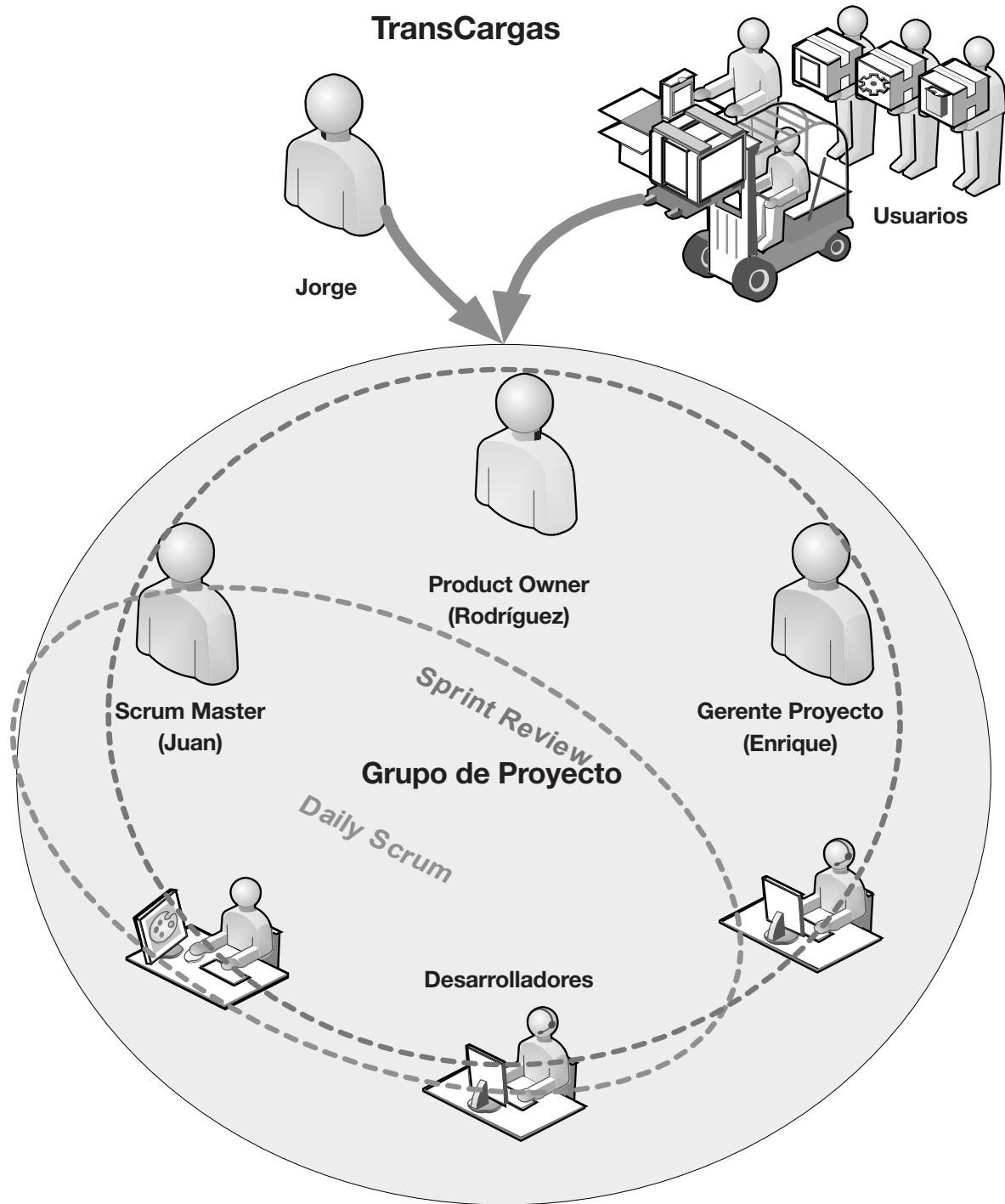


Figura 20.4. Organigramma del proyecto para la metodología ágil seleccionada

Tabla 20.5. Parte del Product Backlog del proyecto del caso testigo

ID	Nombre de la Story	Estado	Tamaño	Sprint	Prioridad	Descripción	Verificación
1	Administración de servicios de cargas.	Planeado	50	1	1	Alta, bajas y modificaciones.	Test de ABM utilizando DBUnit
2	Búsqueda de servicios de carga por diferentes criterios.	Planeado	30	1	4	Requiere definición con Rodríguez y usuarios.	Test de búsqueda utilizando Fitnesse
3	Administración de clientes.	Planeado	40	1	2	Alta, bajas y modificaciones.	Test de ABM utilizando DBUnit
4	Búsqueda de clientes.	Planeado	30	1	4	Requiere definición con Rodríguez y usuarios.	Test de búsqueda utilizando Fitnesse
5	Registro de cargas.	Planeado	40	1	2	Requiere definición con Rodríguez y usuarios.	Test de búsqueda utilizando Fitnesse
6	Registro de descargas.	Planeado	40	1	2	Requiere definición con Rodríguez y usuarios.	Test de búsqueda utilizando Fitnesse
7	Captura de información de estado de cargas.	Planeado	40	1	2	Requiere interacción con sistema de transporte.	Diseñar caso de prueba
8	Reporte de cargas en tránsito.	Planeado	30	1	3	Requiere definición con Rodríguez y usuarios.	Diseñar caso de prueba
9	Reporte de cargas por cliente.	Planeado	30	1	3	Requiere definición con Rodríguez y usuarios.	Diseñar caso de prueba

20.3.2.3 Planificación de los Sprints

A partir de contar con una planificación para los entregables del proyecto y una definición de la arquitectura del sistema, se planificó el primer Sprint [2]. Esta planificación se realizó de la misma manera que la planificación del Product Backlog, en este caso conducida por las prioridades definidas en aquella ocasión. Se seleccionaron Stories y se refinaron a efectos de desglosarlas en las tareas necesarias para su implementación. El Product Backlog del primer Sprint se muestra en el listado 20.6. Se observa la relación entre las Stories del

Sprint y las del Product Backlog en su identificación. Las tareas asociadas aparecen en el listado con subíndices referidos a la misma Story.

En el listado también aparecen los días cubiertos por el Sprint sobre el cual se irán asentando los progresos en la implementación de cada tarea como analizaremos en el próximo capítulo cuando presentemos el seguimiento del proyecto.

Tabla 20.6. Product Backlog del primer Sprint del proyecto del caso testigo									
Días del Sprint			5		Esfuerzo		Días pendientes de implementación...		
Sesgo en base a los últimos Sprints			5	Días	Totales	120	0		
Tarea			ID	Responsable	Estado	Est.	1	2	3 4 5
Construcción del modelo de dominio de cargas			1.1		Planeado	10			
Relevamiento y asignación de reglas al modelo de dominio			1.2		Planeado	15			
Diseño y construcción del modelo de datos			1.3		Planeado	10			
Implementación de los test de validación de las reglas			1.4		Planeado	15			
Diseño y construcción de la pantalla para la configuración del filtro de búsqueda de servicios de carga			2.1		Planeado	7			
Diseño y construcción de la pantalla para la visualización de las búsquedas de servicios de cargas			2.2		Planeado	8			
Diseño y construcción del controlador de búsquedas			2.3		Planeado	8			
Implementación de los test de búsquedas			2.4		Planeado	7			
Análisis de las interfaces de conexión			7.1		Planeado	10			
Diseño del conector			7.2		Planeado	10			
Codificación del conector			7.3		Planeado	10			
Prueba del conector			7.4		Planeado	10			

20.4 Conclusión

Se presentaron dos formas diferentes de afrontar un proyecto de desarrollo de software ejemplificado sobre el caso testigo. Se trató de ilustrar los distintos aspectos típicos de los proyectos en relación con las empresas clientes y desarrolladoras.

La problemática planteada en este ejemplo trasciende el proyecto de este caso y con seguridad estará presente en todos los proyectos de desarrollo manifestándose de manera parecida.



20.5 Contenido de la página web de apoyo

20.5.1 Mapa conceptual

20.5.2 Autoevaluaciones

20.5.3 Presentaciones en Power Point (exclusivo para el docente)

20.6 Referencias

- [1] *Managing Iterative Software Development Projects*, Kurt Bittner, Ian Spence, Addison-Wesley, 2006.
- [2] *Agile Project Management with Scrum*, Ken Schwaber, Microsoft Press, 2004.
- [3] *Planning Extreme Programming*, Kent Beck, Martin Fowler, Addison-Wesley, 2000.
- [4] *SDLC 3.0 – Beyond a tacit understanding of Agile*, Mark Kennaley, Fourth Medium Press, 2010.
- [5] *Agile Estimating and Planning*, Mike Cohn, Prentice hall, 2005.
- [6] *Software Project Survival Guide*, Steve McConnell, Microsoft Press, 1996.
- [7] *Calidad en el Desarrollo de Software*, Guillermo Pantaleo, AlfaOmega 2011.

20.7 Apéndice - Visión del proyecto

20.7.1 Requerimientos del negocio

- El sistema deseado debe desarrollarse porque no se cuenta con uno que permita la administración integrada del negocio de prestación de servicios de transporte de cargas.
- Debe desarrollarse para facilitar la interacción con la empresa aseguradora, para que los clientes puedan consultar el estado de la carga por la Web, para disponer de un control preciso de la ubicación de las cargas en viaje, para integrar el funcionamiento con el transporte de terceros y propio, y para lograr capturar como cliente a la empresa multinacional de equipos de computación.

20.7.1.1 Estado actual

Actualmente este negocio se administra en forma manual, con el soporte de utilitarios de oficina.

- Los clientes no pueden consultar el estado de sus envíos, solo a partir de llamadas telefónicas a la empresa que hace lo mismo con los barcos y las terminales de los puertos.
- Los precios de los servicios de carga de terceros no están disponibles *on line*, por lo cual la solicitud de carga se realiza en dos pasos. Uno al contratar el servicio y otro a las horas, al contar con los valores provistos por las transportadoras.
- La empresa aseguradora no tiene acceso a los contratos en forma *on line*, por lo cual se demora un mínimo de 24 horas para asegurar las solicitudes registradas.

- La planificación de las rutas se realiza actualmente en forma manual, por lo que se debe recibir la información de precios y clima antes de poder sugerir alternativas a los clientes.

20.7.1.2 Oportunidades de negocio

- El contrato con la empresa de computación es una oportunidad en momentos de saturación del mercado por competencia.
- La informatización permitirá ofrecer servicios a clientes a los que actualmente no se tiene llegada.

20.7.1.3 Objetivos de negocio y criterios de éxito

- Lograr el contrato con la empresa de computación.
- Capturar nuevos clientes que permitan incrementar los ingresos en un 25%.

20.7.1.4 Necesidades de clientes / mercado

- Muchas empresas medianas y grandes han implementado seguimiento de cargas en tránsito y consulta por parte de los clientes.
- Muchas otras están evaluando implementarla.
- Es necesario mayor agilidad en el despacho de las cargas al momento de contratación del servicio.
- Es necesario una integración completa con las porciones mercerizadas de los servicios.

20.7.1.5 Riesgos de negocio

- Al no contar con una integración ágil con terceros, no obtener tratos diferenciales de costos.
- No poder acceder a clientes grandes por falta de seguimiento de cargas en tránsito.
- No lograr el contrato con la empresa de computación.
- No lograr integrar el sistema al negocio actualmente administrado manualmente.

20.7.2 Visión de la solución

La solución debe proveer una administración integrada del negocio para facilidad del trabajo de los empleados y para eficiencia operacional de la empresa. Además debe permitir ofrecer servicios similares a la competencia, lo cual en estos momentos significa seguimiento por la Web de las cargas en tránsito.

20.7.2.1 Propósito, intención

Las empresas y particulares deben encontrar en la empresa un proveedor de servicios integrales y seguros de transporte de cargas, con visibilidad y precios competitivos.

20.7.3 Características salientes

20.7.3.1 Descripción general del sistema a desarrollar

Las características salientes del sistema a desarrollar para soportar este negocio son:

- Administrar el servicio de transporte de cargas.
- Seguimiento de cargas en tránsito entre los diferentes puertos del itinerario seguido.
- Facturar el servicio de transporte de cargas.
- Clasificación de las cargas por tamaño y peso.
- Asignación de cargas a contenedores según diferentes tipos.
- Notificación de llegada con cronología y factura.
- Informes de seguimiento.
- Acceso en las terminales de carga y descarga.
- Administración de repositorio de clientes.
- Interacción con sistema de transporte de terceros.

El sistema proveerá funcionalidades para soportar la venta de servicios de transporte, carga y descarga, aseguramiento de cargas. También proveerá el soporte para la facturación de los servicios prestados.

20.7.3.2 Beneficios esperados

- Captura de nuevos clientes.
- Mayor eficiencia operacional.
- Baja de costos.

20.7.3.3 Supuestos y dependencias

- Se asume que la empresa cuenta con el financiamiento para el desarrollo del sistema.
- Se asume que los miembros de la empresa asignados al proyecto colaborarán de la mejor forma con el proyecto en lo que hace al conocimiento del negocio.
- Se asume que de implementar en tiempo y forma el seguimiento de las cargas en tránsito se adjudicará el contrato con la empresa de computación.
- Se asume que la llegada a los nuevos clientes cuando esté desarrollada la funcionalidad del seguimiento está garantizada a partir de los contactos previamente establecidos.

20.7.4 Alcance y limitaciones

20.7.4.1 Alcance de la versión inicial

El alcance del sistema a desarrollar incluirá para los diferentes módulos las siguientes funcionalidades:

- Alta de cargas en forma manual.

- Seguimiento de cargas en tránsito.
- Informes básicos de soporte a la administración.

Este alcance surge de las prioridades establecidas para el proyecto según se analizó en el capítulo 10.

20.7.4.2 Alcance de versiones siguientes

El alcance de las versiones siguientes incluirá las siguientes funcionalidades:

- Administración de clientes.
- Administración automática de cargas (contenedores por tipo, itinerarios según criterios y organización de volúmenes).
- Facturación de servicios.
- Integración con transporte de terceros.
- Informes complejos.

20.7.4.3 Limitaciones y exclusiones

Ninguna.

20.7.5 Contexto del proyecto

20.7.5.1 Perfiles de los involucrados

La tabla fue incluida en el texto de la sección Relevamiento preliminar del presente capítulo.

20.7.5.2 Prioridades del proyecto

La tabla fue incluida en el texto de la sección Relevamiento preliminar del presente capítulo.

20.7.5.3 Ambiente operacional

- El sistema debe funcionar con acceso a la Web.
- El sistema debe satisfacer una disponibilidad de 7x24.
- Las terminales de carga y descarga deben contar con acceso al sistema.
- Debe integrarse con los sistemas de organización de volúmenes de cargas y transporte propio y de terceros.

Gestión del proyecto del caso testigo
Seguimiento

Contenido

21.1 Metodología conducida por los planes	364
21.2 Metodología ágil	371
21.3 Conclusión	375
21.4 Contenido de la página web de apoyo	375
21.5 Referencias	376

Objetivos

- Mostrar cómo se realiza el seguimiento del proyecto en las metodologías conducidas por los planes
- Mostrar cómo se realiza el seguimiento del proyecto en las metodologías ágiles
- Escenificar el cierre de un proyecto fallido y uno exitoso utilizando metodologías conducidas por los planes
- Escenificar el cierre de un proyecto fallido y uno exitoso utilizando metodologías ágiles

Competencias

- Gestionar un proyecto de desarrollo de software en base a su previa planificación

21.1 Metodología conducida por los planes

21.1.1 Monitoreo y control del proyecto

El seguimiento del proyecto fue llevado adelante de la forma que se definió en la planificación presentada en el capítulo anterior para la metodología conducida por los planes. Durante todo el ciclo de vida del proyecto el trabajo de la gestión fue dirigida por la administración de los riesgos y consistió en renegociar cada vez que se presentaba un desvío de la planificación [1], [3], [4]. Como mencionamos en el capítulo anterior esta negociación consistió en analizar, optar y acordar una de las dos alternativas posibles para estas metodologías: a) achicar el alcance del proyecto a efectos de terminar dentro del tiempo y presupuesto programados o b) agregar iteraciones que permitan en un tiempo y presupuesto mayor implementar el alcance original.

El mecanismo utilizado para realizar el seguimiento del proyecto fueron reuniones de estado y avance. A continuación se presentan algunas evidencias de dicho seguimiento.

21.1.1.1 Evolución del estado

Juan, miembro de SoftDesa y líder del proyecto, semanalmente se reunió con los desarrolladores y volcó el desarrollo de dichas reuniones en simples informes de estado de los cuales reproducimos uno a continuación. Los informes consistieron en un par de páginas a modo de minutas que contenían la siguiente información [5]:

Actividades planificadas – como se muestra en la tabla 21.1 se analizaban las tareas de la iteración presente y su grado de avance que están identificadas con los mismos identificadores del cronograma.

Tabla 21.1. Listado de actividades planificadas del informe de estado		
Estado de las actividades planeadas		
id. Actividad	Estado	Observación
Crono_30	En desarrollo	Se están analizando las posibles causas de los errores aleatorios que fueron detectados.
Crono_31	En desarrollo	Se está actualizando el modelo de datos en base a las nuevas clases aparecidas por cambios en el modelo de dominio.
Crono_32	Planificada	Se investigan un par de técnicas para lograr mayor eficiencia en las búsquedas.
Crono_33	Planificada	Se está completando el listado de test con los usuarios.
Crono_38	Planificada	Se está planificando con la gente de QA la próxima revisión de código.

Este listado generaba una evidencia que luego era compartida por los miembros de todo el grupo cuando Juan distribuía esta minuta por mail.

Como en todo proyecto de desarrollo, todo el tiempo surgían tareas no planificadas. Juan creyó importante hacerlas visibles a efectos de mantener el tiempo de los miembros del grupo bajo control y administrar la realización de estas tareas en forma ordenada. En el listado 21.2 se observa una lista de estas tareas.

Tabla 21.2. Listado de tareas no planificadas correspondientes al informe de una de las semanas del proyecto			
Estado de las actividades no planeadas			
Actividad	Estado	Origen	Descripción
ANP_23	Pendiente	Pedido de Rodríguez a partir de sugerencia de un usuario.	Capacitación de los usuarios en la ejecución de los test de funcionalidad.
ANP_24	En desarrollo	Conclusión del análisis de la migración de la DB.	Implementación de queries SQL para la búsqueda de clientes especiales.
ANP_25	Pendiente	Pedido de la gerencia general de SoftDesa como resultado de la reunión entre ejecutivos de ambas empresas no involucrados en el proyecto.	Instalar una demo en el servidor de TransCargas para ser utilizado por el departamento de marketing en la captación de clientes.
ANP_26	En desarrollo	Conclusión de la última revisión de diseño y código.	Implementar capacitación para desarrolladores de nivel iniciados.
ANP_27	En desarrollo	Conclusión de la última reunión de refinamiento de requerimientos.	Colaborar con un usuario en la búsqueda de documentación en papel de remitos modelos de diferentes épocas.

Es de notar, como se mencionó en el capítulo donde se trató el monitoreo y control del proyecto, que este seguimiento no puede consistir en fotos tomadas con cierta frecuencia del estado, sino que se necesita la película que muestre la evolución del ciclo de vida [5]. Así lo entendió Juan, que por esta razón incluyó una forma de encadenar las reuniones a través de sus informes. Esta forma consistió en registrar los problemas surgidos desde la última reunión pero también los de reuniones anteriores a efectos de seguirlos hasta su cierre. En el listado 21.3.a se presentan los problemas nuevos y en el listado 21.3.b los anteriores.

Tabla 21.3.a. Listado de problemas surgidos la última semana en el proyecto					
Problemas nuevos surgidos					
id. Problema	Tipo	Asignado a	Descripción	Acción	Fecha cierre
PR_54	Técnico	Joaquín	La plataforma de persistencia genera errores en casos de datos masivos.	Investigar, cambiar modo de uso y escribir test.	Una semana a partir de la fecha.
PR_55	Técnico	María	El parseo de los archivos xml debe ser resuelto con mayor eficiencia.	Cambiar el diseño y escribir algunos test.	Una semana a partir de la fecha.

Tabla 21.3.a. Listado de problemas surgidos la última semana en el proyecto					
Problemas nuevos surgidos					
id. Problema	Tipo	Asignado a	Descripción	Acción	Fecha cierre
PR_56	Cliente	Mary	EL nuevo diseño de la pantalla de búsquedas no fue aceptado.	Reunirse con los usuarios y relevar la razón.	Próximo lunes.
PR_57	Cliente	Juan	Los usuarios representando a los operadores del puerto no colaboran.	Solicitar a Rodríguez nuevos contactos.	Cuanto antes.
PR_58	Gerencia	Juan	EL servidor de pruebas aún no fue instalado.	Gestionar con la gerencia la provisión del servidor.	Cuanto antes.

Tabla 21.3.b. Listado de problemas surgidos con anterioridad a la última semana en el proyecto				
Problemas anteriores a este informe				
id. Problema	Tipo	Asignado a	Estado	Fecha cierre
PR_49	Técnico	Joaquín	Pendiente	Próximo viernes
PR_50	Cliente	Jorge	En desarrollo	Lo antes posible
PR_51	Cliente	Mary	Pendiente	Próximo martes
PR_52	Gerencia	Juan	Pendiente	Lo antes posible
PR_53	Grupo	María	En desarrollo	En 10 días

Juan pensó, con buen criterio, que una métrica a tomar de los proyectos de la empresa para evaluar su gestión sería el tiempo que demoraban los problemas en ser resueltos. Estos informes les proporcionarían las mediciones para sus métricas.

21.1.1.2 Evolución del alcance

De manera similar al seguimiento realizado al nivel del grupo de desarrollo fue planeado el del proyecto al nivel de los involucrados por parte de ambas empresas. Para esto se acordó la realización quincenal de reuniones entre Juan, Rodríguez, Enrique y Jorge. En ellas se evaluaron temas que necesitaban de poder de decisión, como por ejemplo aquel que trataba la realización de una iteración adicional [4], [5]. A continuación se muestra un extracto del informe que Juan y Enrique elaboraban para cada una de estas reuniones y luego compartían con Jorge y Rodríguez.

Tabla 21.4. Avance y desvíos del proyecto desde su planificación								
Grado de avance								
Fase	Duración				Esfuerzo		Desvío	
	Estimado		Real		Estimado	Real	de Tiempo	de Esfuerzo
	inicio	fin	inicio	fin				
Concepción	8/12	26/12	8/12	4/01	200 horas	400 horas	10 días	200 horas
Elaboración	9/01	5/02	19/01	25/02	480 horas	720 horas	10 días	240 horas
Construcción 1	17/02	1/03	10/03	20/03			0 días	0 horas
Construcción 2	2/03	22/03	-	-	-	-	-	-
Construcción 3	23/03	12/04	-	-	-	-	-	-
Transferencias	13/04	22/04	-	-	-	-	-	-

Puede verse en la tabla 21.4 los registros del esfuerzo gastado y del comportamiento de la evolución real del proyecto comparada con la planificación. En las dos primeras fases se muestran las iteraciones adicionales que fueron incluidas. Una debida a la falta de enfoque en la concepción y el trabajo con las prioridades de los requerimientos y otra en la definición de la arquitectura. Cuando se narra el cierre del proyecto en una próxima sección se describen las causas de estas iteraciones agregadas.

Tabla 21.5. Listado de los entregables del proyecto acordados durante la planificación				
Entregables				
Nombre de Entregable	Fecha de entrega		Estado	Observaciones
	Estimada	Real		
Especificación de requerimientos	26 de diciembre	4 de enero	Entregado	Acuerdo logrado luego de iteración adicional
Especificación de arquitectura	5 de febrero	25 de febrero	Entregado	Acuerdo logrado luego de iteración adicional
Pruebas de aceptación	9 de febrero	25 de febrero	Entregado	Ok
Primer entregable en línea para probar	1 de marzo	20 de marzo	Entregado	Errores operativos resueltos
Informe con errores corregidos	10 de marzo	30 de marzo	Entregado	Ok
Segundo entregable en línea para probar	22 de marzo	-	-	-
Informe con errores corregidos	31 de marzo	-	-	-

Tabla 21.5. Listado de los entregables del proyecto acordados durante la planificación				
Entregables				
Nombre de Entregable	Fecha de entrega		Estado	Observaciones
	Estimada	Real		
Tercer entregable en línea para probar	12 de abril	-	-	-
Informe con errores corregidos	20 de abril	-	-	-
Resultados de las pruebas de aceptación	30 de abril	-	-	-
Sistema instalado	2 de mayo	-	-	-

Los entregables del proyecto, entre ellos el producto de trabajo de las primeras iteraciones, fue seguido como se muestra en la tabla 21.5 la cual forma parte del informe elaborado por Enrique y Juan al finalizar la primera iteración de la fase de construcción. De la misma forma que con los entregables se registraron los pagos asociados a los diferentes hitos del ciclo de vida del proyecto. Esto se muestra en la tabla 21.6.

Tabla 21.6. Hitos y pagos del proyecto					
Facturación					
#pago	Hito	Fecha	Importe	Estado	Observaciones
1	Acuerdo de requerimientos	4/01	10 000 + adicional	Hecho	Iteración adicional de costo compartido
2	Acuerdo del plan de proyecto	10/01	10 000	Hecho	Ok
3	Primer entregable	20/03	15 000 + adicional	Hecho	Iteración adicional
4	Segundo entregable	-	15 000	Pendiente	-
5	Tercer entregable	-	15 000	Pendiente	-
6	Instalación	-	25 000	Pendiente	-

Como todos los temas que requieren de aprobaciones que eventualmente implican mayores tiempos del proyecto y por lo tanto mayor costo, estos fueron tratados a este nivel y fueron registrados por los informes que se elaboraron para estas reuniones de avance. Uno de esos temas son los cambios a los requerimientos, los cuales se registraron como se muestra en la tabla 21.7. Estas decisiones de aprobación o rechazo de los cambios referidos eran fundamentados por análisis de impacto de estos cambios previamente elaborados por Mary y Juan.

Cambios a los requerimientos			
id. Cambio	Responsable de aprobación	fecha de aprobación	documento de impacto relacionado
Cbio_23	Rodríguez - Jorge	27/12	DocImp_005
Cbio_38	Rodríguez – Jorge	10/02	DocImp_007
Cbio_39	Rodríguez – Jorge	10/02	DocImp_007
Cbio_40	Rodríguez - Jorge	10/02	DocImp_007

Los problemas surgidos en el proyecto y que debían escalarse porque su solución no estaba dentro de las atribuciones de Juan y Rodríguez eran tratados en estas reuniones y se registraban de la misma forma que Juan hacía en sus informes de estado, es decir que se anotaban los anteriores y los nuevos para darle continuidad a su tratamiento. Las tablas correspondientes no se muestran por cuestiones de espacio.

Un aspecto de importancia fue la administración de los riesgos del proyecto. Juan, de acuerdo con Enrique, decidió hacerlos públicos, compartirlos con el cliente, y por lo tanto fueron también incluidos en estos informes. Su evolución se muestra en la figura que sigue, 21.1.

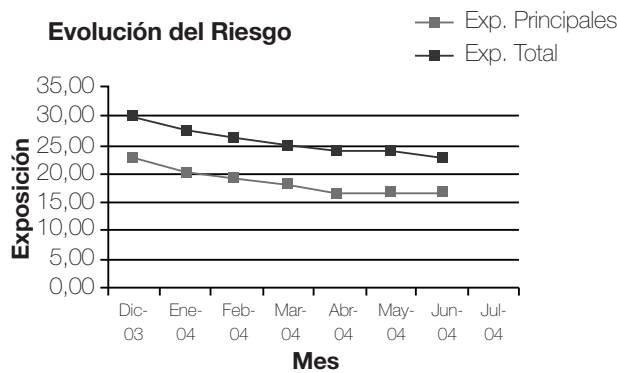


Figura 21.1. Evolución de los riesgos del proyecto a lo largo de su ciclo de vida

Por último fue incluido un listado de los compromisos y acuerdos con que debería contar el proyecto en diferentes momentos de su ciclo de vida. Al incluirlos se hicieron visibles y se tenían presentes cada vez que se reunían.

Acuerdos y compromisos				
Acuerdo / Compromiso	Acordado (sí/no)	Responsable de aprobación	Acción	fecha
Requerimientos	Sí	Rodríguez - Jorge	-	4/01
Plan	Sí	Rodríguez - Jorge	-	10/01
Infraestructura	Sí	Rodríguez - Jorge	-	10/01
Pruebas compartidas	No	Gerencias	Explicar desajustes	Lo antes posible

21.1.2 Cierre del Proyecto

Como adelantamos en un capítulo anterior describimos dos posibles terminaciones del proyecto que describen escenarios diferentes.

21.1.2.1 Cierre I

Después de largas negociaciones no fue posible acordar una replanificación que dejará satisfechos los intereses de ambas partes. Los clientes se sintieron estafados, su percepción fue que SoftDesa les cobraba por un trabajo que no había realizado y que el sistema implementado hasta entonces no cumplía y no cumpliría con sus necesidades. Esto fue causado por incumplimientos en los hitos y entregables del proyecto según se había planificado. Esta situación llevó a la cancelación del proyecto. Las explicaciones de carácter técnico respecto de las dificultades por las que pasó el proyecto no fueron suficientes, ya que los miembros de TransCargas no las entendieron.

En reuniones internas a posteriori con el fin de capitalizar lecciones aprendidas se concluyó que:

El trabajo con los requerimientos durante la concepción del sistema se realizó sin focalizarse en los objetivos del proyecto, razón por la cual se relevaron aspectos no importantes del problema a resolver que estaban fuera del alcance del proyecto.

La definición de la arquitectura fue la correcta pero se demoró más tiempo del disponible, razón por la cual la primera iteración de la codificación se realizó con una arquitectura inestable lo que generó gran cantidad de errores de difícil resolución. Esta falencia se debió a la inclusión, en parte, de requerimientos técnicos no funcionales que no se derivaron del problema sino del sobredimensionamiento basados en buenos criterios de diseño pero ajenos al problema.

Se gastó tiempo en la elaboración de documentos innecesarios, ya que al momento de la posible replanificación y cancelación del proyecto estos no solo no habían aportado nada al grupo de desarrollo sino que nunca se habían utilizado.

21.1.2.2 Cierre II

Debido a la falta de madurez en la utilización de formas de trabajo iterativo gestionado por los riesgos y focalizado en objetivos fue necesario corregir algunos desvíos detectados en la primera fase del proyecto. La falta de prioridades de algunos requerimientos de acuerdo a los objetivos hizo que el tiempo asignado al relevamiento se gastara sin centrar el esfuerzo en achicar los riesgos del negocio en los aspectos relacionados al alcance. Esta situación no fue percibida hasta que el cronograma marcó que el tiempo había pasado y aún faltaba entender cuestiones importantes del negocio. Fue necesario replanificar el proyecto agregando una iteración a esta fase para cerrar estas cuestiones. La negociación que se llevó adelante en esa oportunidad fue soportar de común acuerdo y en forma compartida el costo de esta nueva iteración por ambas partes debido a la responsabilidad de los involucrados de ambas partes.

Otra negociación fue llevada adelante como consecuencia de los errores introducidos por la inestabilidad de la arquitectura previamente a su modificación. En esa oportunidad fue necesario explicar a TransCargas que

algunos de los pedidos técnicos de sus miembros fueron las causas de esta inestabilidad. La iteración adicional introducida para su reparación alargó los tiempos del proyecto y su costo. El líder y el gerente de desarrollo tuvieron que invertir gran parte de su tiempo en estas negociaciones continuas, que trabajando de esta forma constituyen la esencia de la gestión.

En una de las reuniones de estado, uno de los desarrolladores críticos de esta metodología mencionó la documentación redundante e inútil que se estaba construyendo, lo cual fue resuelto de común acuerdo por los miembros del grupo a partir de la revisión y reducción de la documentación generada.

El proyecto finalizó en el tiempo planificado más las extensiones mencionadas debido a las replanificaciones, ambas empresas quedaron conformes con el proyecto.

21.2 Metodología ágil

21.2.1 Monitoreo y control del proyecto

El seguimiento del proyecto fue llevado adelante de la forma que se definió en la presentación realizada a TransCargas al momento de la propuesta técnico económica. Durante todo el ciclo de vida del proyecto el trabajo de la gestión fue conducido por la administración del Product Backlog y de los Sprints Backlog y consistió en optimizar la cantidad de Stories implementadas en cada iteración [2], [3]. Como mencionamos en el capítulo anterior esta gestión se basó en establecer prioridades por parte del Product Owner (Rodríguez) para cada Story y acordar la factibilidad y la forma de desarrollo con el Scrum Master (Juan). Al final de cada Sprint se volvían a priorizar las Stories que no pudieron ser implementadas con las seleccionadas para el nuevo Sprint.

El mecanismo utilizado para realizar el seguimiento del proyecto fueron reuniones diarias (Daily Scrum) y revisiones de las iteraciones (Sprint Review – Sprint Retrospective). A continuación se describen algunas de estas reuniones e informes elaborados.

21.2.1.1 Daily Scrum

Estas reuniones informales diarias constituyen el nexo más fuerte entre los miembros del grupo de desarrollo y se desarrollan frente al pizarrón de la sala del proyecto, preferentemente parados y donde se tratan los diversos temas. Se recomienda que su duración no exceda los 15 minutos [6]. A continuación reproducimos una de estas reuniones.

Juan – Buenos días, como están ustedes. Veo que todos han adaptado el grado de avance de sus tareas, ¡bien! Germán les contará acerca de un tema que necesitamos resolver. Germán...

Germán – Sí, necesitamos resolver dos cuestiones. Vamos con la primera, la plataforma de persistencia está generando errores que no entendemos y alguien tiene que investigar el asunto a la luz de los resultados de los últimos test. Creo que en un par de días podríamos corregir esta situación.

Juan – OK, ¿quién se hará cargo del tema? ¿María...?

María – Prefiero ocuparme del otro tema, del que me comentaste ayer y Germán nos contará ahora.

Joaquín – Bueno, en ese caso yo me hago cargo así Marcos puede terminar con los test de funcionalidad que Mary le especificó.

Juan – Bien, asignado. Puedes contar con cuatro días, pero cuanto antes termines mejor.

Germán – El segundo asunto tiene que ver con el parser de los archivos xml, necesitamos mejorar los tiempos. Si te vas a hacer cargo, María, cuando terminemos la reunión te paso la documentación.

María – Sí, dale.

Juan – Mary, ¿quieres contarnos algo acerca de tu reunión de ayer con Rodríguez?

Mary – Ahora no, prefiero que esté presente Rodríguez.

Marcos – Juan, no te olvides que necesito acceso al servidor de TransCargas para ejecutar los queries.

Juan – Ok, en un rato lo gestiono. Mary, entonces mañana lo vemos cuando esté Rodríguez acá. Gracias señores, a trabajar.

21.2.1.2 Revisión de Sprints – Review, Retrospective

Al finalizar cada uno de los Sprints el grupo de desarrollo realizó reuniones de evaluación ejecutando las acciones necesarias para la verificación de las Stories implementadas (Review) [6]. También se desarrollaron reuniones con el objetivo de capitalizar las lecciones aprendidas y generar mejoras dentro del grupo de desarrollo (Retrospective). A continuación reproducimos parte de una de estas reuniones:

Review

Juan – Buenos días, ¿cómo están ustedes? Esperemos que los miembros de soporte informático hayan habilitado esta computadora y podamos desde acá visualizar y probar la iteración.

Germán – Podremos, ayer por la tarde yo probé y ya estaba habilitada.

Juan – Bien, Mary, ¿podrás conducir la ejecución de los casos de prueba?

Mary – Sí, claro. Rodríguez acérquese así podemos comenzar.

Germán – Mary, probemos la Story de Marcos en último lugar ya que está demorado.

Mary – Ok, como pueden ver estoy accediendo a la búsqueda de cargas....

Dos horas después...

Rodríguez - Bien, me gustó cómo finalmente implementaron estas pantallas, es mucho más fácil el manejo de los datos del cliente. En este próximo Sprint entonces podríamos cambiar también las pantallas que ya habíamos visto de las búsquedas y de esa forma tendremos mayor uniformidad y menos confusión.

Juan - Bien, pero eso llevará tiempo adicional, tendremos que dejar de hacer algo de lo ya planificado para introducir este cambio.

Rodríguez - Veámoslo, pero ese cambio es importante, la gente de marketing y Jorge ya me lo habían pedido el mes pasado.

Retrospective

Juan - Buenos días, ¿cómo están ustedes? Bueno, como es costumbre propongo que cada uno haga su balance y luego abramos la discusión y acuerdo de posibles cambios. Germán...

Germán - Esta vez quiero marcarles a todos una falencia que todos han tenido y quizás podamos encontrar entre todos las causas. De una u otra forma han violado en algún lugar de su código las definiciones que habíamos hecho para la arquitectura. En la próxima revisión de diseño y código podrán acceder a los detalles. Por ejemplo Marcos no respetó la forma de acceso a la capa de persistencia y terminó escribiendo código que acopla la presentación con los mappers. Y así hay otros ejemplos...

Marcos - Sí, ese fue un error que cometí por falta de claridad mientras contaba con poco tiempo para terminar esa tarea.

María - A mí me pasó algo parecido en la forma de uso del parser de los archivos xml. Creo que nos hace falta una documentación que todos podamos acceder mientras desarrollamos.

Germán - Ustedes mismos consideraron trabajo desperdiciado elaborar ese documento. Si ahora hace falta es tiempo de construirlo.

Mary - Por mi parte creo que las pruebas de funcionalidad implementadas fueron todo un éxito ya que nos permitieron detectar unos cuantos errores, pero nos llevó mucho tiempo diseñarlas utilizando Fitnesse.

Joaquín - Eso fue porque no conocíamos la plataforma, creo que desde ahora las haremos de una manera mucho más eficiente.

Juan - Rodríguez, algo que aportar...

Rodríguez - Yo quiero pedirles que para la próxima vez que haya que ejecutar pruebas y participen los usuarios de mi empresa, antes los capaciten para que podamos aprovechar mejor el tiempo. Además yo les prometí que era una tarea fácil y ellos podrían hacerla... pero si se sienten incapaces será difícil volver a contar con ellos.

Juan - Bien, le asignaremos tiempo del proyecto a hacer esta capacitación, Mary y María podrían hacerse cargo.

Cambiando de tema, veo que la velocidad de desarrollo del grupo se mantiene como pueden ver en el reporte (figura 21.2) que les publiqué en la red interna al proyecto. Si nos mantenemos así y ordenamos algunos de estos temas que tratamos podremos avanzar cómodos hacia el próximo Sprint. Gracias por su participación, mañana por la mañana nos reunimos para terminar de planearlo.

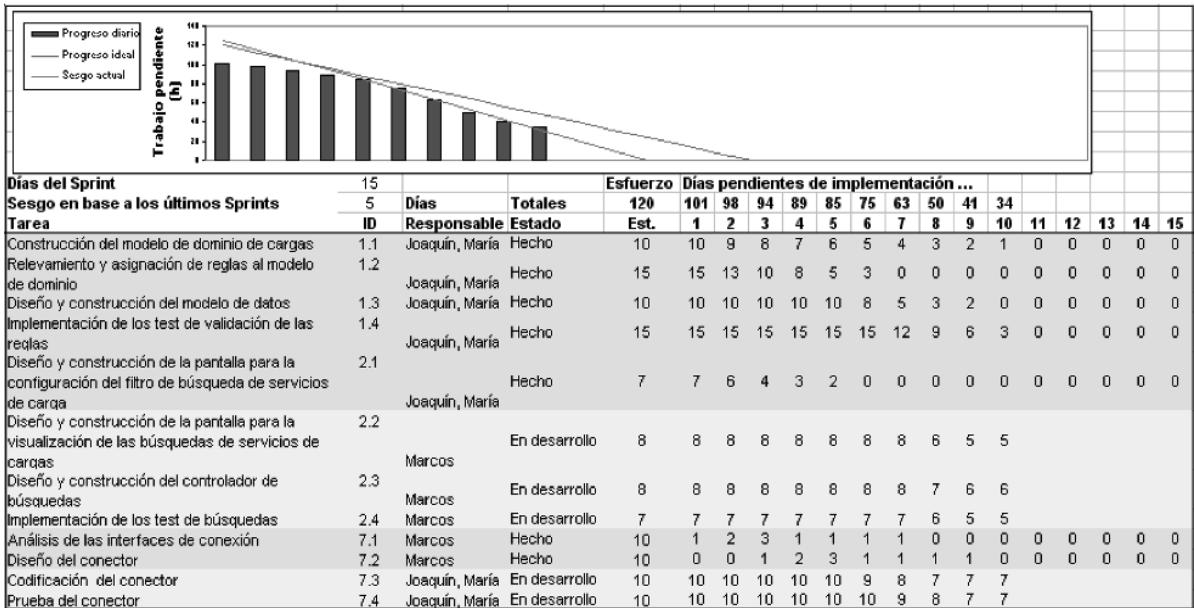


Figura 21.2. Informe de avance del proyecto en función del trabajo pendiente en un Sprint (Burn Down Chart)

21.2.2 Cierre del Proyecto

Como hicimos con la metodología conducida por los planes, describimos dos posibles terminaciones del proyecto que describen escenarios diferentes para esta metodología ágil.

21.2.2.1 Cierre I

Cuando el proyecto promediaba se planteó una discusión acerca de la velocidad de avance del proyecto y las tareas de retrabajo necesarias debido a la aparente falta de planificación inicial. La cuestión central era que si se extrapolaban las Stories (requerimientos) pendientes a la velocidad promedio de desarrollo del proyecto ni siquiera los priorizados como esenciales estarían todos implementados.

Esto fue producto de una escasa planificación inicial por considerar a ésta un gasto de esfuerzo innecesario al confundir el ejercicio de planificación con el documento que eventualmente lo contiene. Hubo también que hacer importantes cambios ya que por no haber definido una arquitectura, por la misma razón que no fue planificado adecuadamente el proyecto, se gastó tiempo que había sido planeado para el desarrollo de distintas funcionalidades.

Otra de las falencias que el proyecto evidenció fue la falta de un modelo de desarrollo, es decir un proceso que permita analizar e integrar el dominio con la tecnología.

Por todas estas causas cuando el tiempo del proyecto terminó, el producto generado estaba lejos de cumplir con las pautas acordadas.

21.2.2.2 Cierre II

Cuando el proyecto promediaba se evidenciaron algunas falencias en la evolución del proyecto y en la definición de la arquitectura. Esto fue debido a que en el lanzamiento del proyecto inmediatamente se afectó a los desarrolladores a escribir código sin un tiempo para hacer definiciones que fueron calificadas como gasto cuando en realidad eran una inversión. Se trataba nada menos que de planificar los entregables del proyecto y de definir la arquitectura del sistema. Se acordó dedicar un tiempo equivalente a un Sprint para esto y el proyecto continuó a partir de las definiciones realizadas.

La carencia de un modelo de desarrollo afectó todo el proyecto, desde el primero hasta el último de los Sprints. Fue una buena lección aprendida, agilidad no significa no ocuparse de las cuestiones técnicas del proyecto como lo es un proceso de evolución de los componentes del sistema a lo largo del ciclo de vida del proyecto. A pesar de esto el proyecto finalizó con todos los requerimientos priorizados como esenciales y el 70% de los importantes implementados. En estos días se está acordando un Sprint adicional para la implementación del resto de los importantes y algunos de los accesorios.

21.3 Conclusión

Se presentó el seguimiento del proyecto desde la perspectiva de las metodologías analizadas en el libro para el proyecto del caso testigo. También dos cierres diferentes para cada caso. En ambos se eligieron las causas conocidas que constituyen las falencias comunes que muchas veces ocasionan terminaciones no buenas como las descritas [3].

21.4 Contenido de la página web de apoyo



21.4.1 Mapa conceptual

21.4.2 Autoevaluaciones

21.4.3 Presentaciones en Power Point (exclusivo para el docente)

21.5 Referencias

- [1] *Managing Iterative Software Development Projects*, Kurt Bittner, Ian Spence, Addison-Wesley, 2006.
- [2] *Agile Project Management with Scrum*, Ken Schwaber, Microsoft Press, 2004.
- [3] *SDLC 3.0 – Beyond a tacit understanding of Agile*, Mark Kennaley, Fourth Medium Press, 2010.
- [4] *Software Project Survival Guide*, Steve McConnell, Microsoft Press, 1996.
- [5] *Calidad en el Desarrollo de Software*, Guillermo Pantaleo, Alfaomega, 2011.
- [6] *Scrum and XP from the Trenches - How we do Scrum*, Henrik Kniberg, InfoQ.com, 2007.

Calidad de procesos
y productos de software

Contenido

22.1 Calidad de procesos y productos de software	378
22.2 Métricas en el software	385
22.3 Conclusiones	394
22.4 Contenido de la página web de apoyo	394
22.5 Referencias	395

Objetivos

- Qué determina la calidad de un proceso de desarrollo de software
- Qué determina la calidad de un producto de software
- Cuál es el vínculo entre ambos conceptos
- Qué significa verificación y qué validación
- Qué es un checklists
- Cómo se construyen métricas de medida de calidad

Competencias

- Desarrollar criterios de calidad orientados a los productos generados y los procesos llevados adelante en los proyectos de desarrollo de software

22.1 Calidad de procesos y productos de software

Hasta aquí hemos estado trabajando con los requerimientos, realizado el diseño del software, tocado el tema de la codificación y las pruebas del software y por último hemos hablado sobre la estimación, planificación y seguimiento de los proyectos. Mencionamos diversos aspectos y poco nos hemos referido al término calidad, ¿antes no debíamos tenerla en cuenta? El concepto de calidad está presente en cada una de las áreas que hemos trabajado y en cada uno de los productos que generamos (documentación o código). Por lo tanto, la calidad está presente durante todo el proceso de desarrollo de software. Aunque muchas veces se evalúa la calidad del producto de software en etapas tardías, por ejemplo luego de ser construido. Por lo general, cuanto más tarde se evalúe la calidad más costosa será la verificación y su corrección. Supongamos que se lleva a cabo una entrega al cliente de un producto de software de baja calidad. El mantenimiento de este producto será más costoso que para un producto con calidad.

En este capítulo describiremos los aspectos de calidad de procesos y productos de software. Pero antes de comenzar con esto definiremos el término calidad.

Genéricamente la *calidad* se define como la propiedad o el conjunto de propiedades inherentes a algo que permiten juzgar su valor. Definición extraída del diccionario de la lengua española RAE [1].

Ahora sí veamos en el siguiente apartado el concepto del término calidad dentro del contexto del software.

22.1.1 El concepto de calidad aplicado al software

Para comprender el concepto de calidad aplicado al software tenemos que tener en cuenta los dos aspectos que lo vinculan. El producto de software es generado como resultado de la ejecución de una serie de actividades definidas por procesos específicos. Por lo tanto, un aspecto es el proceso mencionado y el otro el producto de software en sí mismo. Ambos componentes forman parte del producto final y por eso se analizarán en los siguientes apartados.

22.1.1.1 Calidad de productos

Para arribar al concepto partamos desde el origen del producto de software, es decir desde los requerimientos y necesidades del usuario y cliente. A partir de aquí elaboramos el producto intentando cumplir con las expectativas del usuario y cliente. Entonces, si tenemos que lograr que el software cumpla con esas expectativas, debemos tener esto en cuenta para juzgar su valor. Así el grado de cuán cerca de lograrlo estamos representa la calidad del producto generado. Por lo tanto, es sumamente importante la satisfacción del usuario.

Para continuar me gustaría que pensemos al producto de software desde dos perspectivas. Por un lado podríamos posicionarnos en el lugar del usuario que aprovecha las funcionalidades de la aplicación. Por ejemplo, si estamos ante un editor de texto las opciones de guardar el documento y darle formato al texto son funciones de la aplicación. Pero ¿cómo se guarda el documento? Y aquí es necesario que la persona que desarrolló esta funcionalidad nos cuente cómo lo hizo, qué diseño empleó, qué funciones le brindaba el lenguaje que utilizó, qué parámetros eran necesarios, etc. Es decir que, por otro lado, estamos posicionados en el

interior de la aplicación para analizar su código. A la calidad asociada a los aspectos internos de la aplicación, como el diseño del código, la legibilidad del código, etc. se la denomina “Calidad interna”. Ahora nos podemos imaginar que la calidad relacionada con las características externas de la aplicación, como ser, la interfaz de usuario, se denomina “Calidad externa”.

Ambos aspectos hacen a la calidad integral del producto de software. De todas maneras, la calidad interna no es negociable, mientras que sí lo es la calidad externa. Pero ¿por qué decimos que la calidad interna no es negociable? Por un lado, es difícil que el usuario o cliente comprenda la necesidad, por ejemplo, de un buen diseño; y por otro lado, un mal diseño difícilmente pueda generar un producto con buena calidad externa.

22.1.1.2 Calidad de procesos

Hemos definido hasta aquí el concepto de calidad de producto y queda pendiente especificar el concepto de calidad de procesos. Pero antes de introducirnos en ello, definiremos a qué nos referimos con el término *proceso* en este contexto. Un *proceso* es un conjunto de actividades que se realizan o suceden para lograr un objetivo determinado. Generalmente el proceso se extiende en el tiempo y puede tener estados. Para finalizar la comprensión definamos el concepto de *actividad*. Una *actividad* es un conjunto de tareas que se realizan o suceden para lograr un objetivo determinado y no se prolonga en el tiempo.

Dicho esto, podemos concluir que la calidad de procesos está definida por la calidad de las actividades individuales y éstas como conjunto, es decir como parte del proceso que conforman.

Pensemos, por ejemplo, en una empresa que fabrica camisas. El proceso de fabricación de camisas consta de varias actividades. Comienza con el corte de la tela, continúa con la costura y por último el ensamble final de la prenda. La calidad en cada una de las actividades y la coordinación entre ellas hacen a la calidad en el proceso de fabricación. Por ejemplo, si los cortes realizados para las mangas de una camisa son diferentes unos con otros, las mangas de la camisa no quedarán iguales, lo que daría una calidad pobre al producto final. Lo mismo podría ocurrir si se utilizan dos mangas de distinta talla para el ensamblado de una camisa.

¿Y qué ocurre en el desarrollo de software? En el desarrollo de software sucede algo similar. Si el proceso de elicitación de requerimientos no define correctamente las necesidades del cliente, muy difícilmente cumpla con sus expectativas y provoque el rechazo del producto. Imaginemos ahora una falla en el proceso de diseño de la arquitectura de la aplicación. Posiblemente esto genere un retrabajo (significando un retraso) o una aplicación poco modificable o, hasta incluso, un producto de baja eficiencia. Es decir que en todos estos casos el producto final tendrá poca calidad debido a las fallas en el proceso de desarrollo. Para intentar resolver estos inconvenientes han surgido las verificaciones y validaciones sobre los procesos que buscan mejorar su ejecución.

22.1.2 Validación y verificación

En el apartado anterior comentamos que se definen procesos para lograr un objetivo determinado que puede ser un producto de software. Por lo tanto puede interesarnos comprobar la ejecución de ese proceso de acuerdo a cómo fue definido. Cuando nos referimos a llevar a cabo una comprobación, estamos pensando en verificar si el proceso se siguió de acuerdo a cómo había sido definido.

22.1.2.1 Checklists

Los checklists (listas de control) pueden ser empleados para la validación y verificación de los procesos y productos de software. Los checklist se conforman de una lista de ítems que deben ser verificados sobre el producto o proceso sobre el cual se esté ejecutando. Cada uno de estos ítems puede estar formulado como preguntas que se contestarán durante la ejecución de la validación. Estas preguntas podrán ser contestadas con “sí”, “no” o “n/a”, tengamos en cuenta que no se desea utilizar demasiado tiempo para la ejecución. También puede anexarse un responsable a cada una de las no conformidades halladas y acciones para su resolución.

Para concluir, podemos decir que los checklist representan una herramienta rápida que lista los ítems que deben ser verificados para validar un producto o proceso.

22.1.3 Ejemplo de uso

Tomemos como base la especificación de requerimientos presentada en el Anexo I del Capítulo 10. Ahora imaginemos que queremos validar ese documento. Para ello ejecutaremos un checklist sobre el documento.

El modelo de checklist a ser ejecutado se presenta a continuación en la tabla 22.1.

Tabla 22.1. Checklist de especificación de requerimientos					
Id	Checklist ERS	S/N/NA	Acciones	Responsable	Fecha
1	¿Está definido el objetivo del documento de especificación de requerimientos?				
2	¿Han sido identificados los objetivos del documento de especificación de requerimientos?				
3	¿Han sido identificados los participantes de la especificación de requerimientos?				
4	¿Los requerimientos funcionales han sido listados?				
5	¿Los requerimientos no funcionales han sido listados?				
6	¿Se han identificado las interfaces con otros sistemas?				
7	¿Se han identificado los actores del sistema?				
8	¿El modelo de actores se encuentra presente en el documento de especificación de requerimientos?				
9	¿Han sido identificadas las funciones excluidas de la aplicación?				

Tabla 22.1. Checklist de especificación de requerimientos					
Id	Checklist ERS	S/N/NA	Acciones	Responsable	Fecha
10	¿Están los casos de uso listados?				
11	¿Los casos de uso están empaquetados en paquetes lógicos definidos?				
12	¿Los requerimientos han sido priorizados?				
13	¿Los requerimientos son unívocamente identificables?				
14	¿Cada uno de los requerimientos es preciso y no ambiguo?				
15	¿Cada uno de los requerimientos es verificable o factible de ser testado?				
16	¿Cada uno de los requerimientos es correcto?				
17	¿El origen de los requerimientos está identificado claramente?				
18	¿Los requerimientos de restricciones de hardware y software están especificados?				
19	¿Fue construido un glosario?				

Procedamos ahora a ejecutar el checklist junto con el analista que es el responsable de este documento.

Validador: ¿Estás listo para comenzar con la validación del documento especificación de requerimientos?

Analista: Por supuesto, adelante.

Validador: Bien. Empecemos por el primer punto, ¿está definido el objetivo del documento de especificación de requerimientos?

Analista: Sí, como puedes observar existe un apartado que se denomina “Objetivo” donde justamente se describe el fin del documento.

Validador: Perfecto. Continuemos. ¿Han sido identificados los documentos relacionados?

Analista: Sí, nuevamente el documento cuenta con una sección exclusiva con este objetivo. Los documentos listados en este caso son el documento de visión y la propuesta económica.

Validador: Muy bien. Con respecto a los requerimientos, ¿se identificaron los requerimientos funcionales?

Analista: Los requerimientos se han listado en la sección Requerimientos funcionales. He confeccionado una tabla con el detalle de los requerimientos funcionales.

Validador: Perfecto. Pasemos a los requerimientos no funcionales, ¿has podido identificarlos?

Analista: Así es. Cada uno de los requerimientos no funcionales ha sido descrito junto a su código de identificación.

Validador: Bien, los requerimientos no funcionales son importantes mencionarlos dado que la persona que lleve a cabo las pruebas deberá tenerlos en cuenta también.

Analista: Sí, así es.

Validador: Pasemos ahora a hablar de las interfaces con otros sistemas, ¿has identificado alguna?

Analista: Claro que sí. Yo he identificado como interfaces externas al sistema de transporte a terceros, el sistema de transporte y el sistema supervisor de cargas.

Validador: El listado es correcto, me gustaría saber si has detallado cada uno de estas interfaces.

Analista: No, eso no lo he hecho. Pero me parece que sería importante para que no existan dudas a qué sistema estoy haciendo referencia. Lo incorporo en el mismo apartado.

Validador: Pues sí, también creo lo mismo. Entonces continuemos con los actores del sistema, ¿qué actores has podido identificar?

Analista: Los actores que he identificado en este sistema son el vendedor de servicio de cargas, el cliente, el sistema de transporte, el servicio de transporte a terceros, el sistema supervisor de cargas y por último el operador de carga.

Validador: Excelente, no se te ha pasado por alto ningún actor. De todas maneras me gustaría saber si has podido detallar brevemente cada uno de ellos.

Analista: No, tú sabes que eso se me pasado por alto. He incluido el modelo de actores, pero no una descripción de cada uno de ellos.

Validador: Bueno, me has ahorrado la próxima pregunta, pero con respecto a la descripción ¿puedes incluirla?

Analista: Sí, por supuesto.

Validador: Muchas gracias. Sigamos si te parece bien con la próxima pregunta.

Analista: Adelante.

Validador: ¿Has podido identificar alguna función excluida del sistema a desarrollar?

Analista: Sí, he incluido un listado dentro de la sección de funciones excluidas.

Validador: Entramos al tema casos de uso, ¿están listados?

Analista: Aquí me pareció mejor incluir el diagrama de los casos de uso donde muestro la relación con los actores intervinientes.

Validador: Bueno, eso es importante también. Pero es necesario incluir un listado con los casos de uso, su identificador y una breve descripción de cada uno de ellos. A veces también suele ser útil indicar el requerimiento del cual se ha desprendido. Aquí, como estamos ante un sistema con pocos requerimientos y casos de uso, esto podríamos no incluirlo. Pero sí sería imprescindible en un sistema de mayor magnitud.

Analista: Nuevamente creo que estás en lo correcto. Realizaré el listado de los casos de uso junto con una descripción de cada uno de ellos.

Validador: Bueno, continuando con el tema de los casos de uso, ¿han sido empaquetados?

Analista: Sí, así es. Se han empaquetado teniendo en cuenta los siguientes paquetes: Cargas, Roles, Facturación, Clientes y Logística.

Validador: Muy bien. ¿Los requerimientos han sido priorizados?

Analista: Pues sí, estos han sido priorizados por el cliente en función de sus necesidades.

Validador: Bien, ahora debería verificar si los requerimientos son precisos y no ambiguos.

Analista: Adelante.

El validado lleva a cabo un control de los requerimientos...

Validador: Bueno, los requerimientos son precisos y no son ambiguos. Asimismo he verificado que los requerimientos son factibles de ser testeados y son correctos.

Analista: Perfecto.

Validador: Sí, podemos continuar con el próximo punto. ¿Se ha identificado el origen de los requerimientos?

Analista: Habíamos hablado que los requerimientos han sido todos originados por el cliente, pero no he dejado constancia de ello en el documento.

Validador: Bueno, sería preciso que sea incorporado el origen para cada uno de los requerimientos listado.

Analista: No hay problema, serán mencionados.

Validador: Perfecto. Ahora me gustaría saber si los requisitos de software y hardware han sido especificados en el documento.

Analista: No, las necesidades de hardware y software no han sido listadas. Pero realmente considero que es necesario incorporarlas.

Validador: Así es. Entonces finalmente arribamos al último punto. ¿Has realizado un glosario?

Analista: No, no consideré que sea necesario, pero si te parece puedo revisarlo nuevamente luego de realizar los cambios que coordinamos.

Validador: Creo que sería apropiado y conveniente esto. En estas condiciones podemos dar por concluida la validación y si te parece bien volvemos a realizar la revisión el 30 de marzo.

Analista: Me parece perfecto. Muchas gracias.

Validador: A ti, por el tiempo.

Luego de realizar la validación, se ha completado el checklist y el resultado se presenta en la tabla 22.2.

Tabla 22.2. Checklist de especificación de requerimientos ejecutado					
Id	Checklist ERS	S/N/NA	Acciones	Responsable	Fecha
1	¿Está definido el objetivo del documento de especificación de requerimientos?	S	-	-	-
2	¿Han sido identificados los documentos relacionados?	S	-	-	-
3	¿Han sido identificados los participantes de la especificación de requerimientos?	S	-	-	-
4	¿Los requerimientos funcionales han sido listados?	S	-	-	-
5	¿Los requerimientos no funcionales han sido listados?	S	-	-	-
6	¿Se han identificado las interfaces con otros sistemas?	S	Incorporar una descripción de cada una de las interfaces mencionadas.	Analista	30/03/2011
7	¿Se han identificado los actores del sistema?	S	Incorporar una descripción de cada uno de los actores del sistema.	Analista	30/03/2011
8	¿El modelo de actores se encuentra presente en el documento de especificación de requerimientos?	S	-	-	-
9	¿Han sido identificadas las funciones excluidas de la aplicación?	S	-	-	-
10	¿Están los casos de uso listados?	N	Incorporar un listado de los casos de uso y una breve descripción de los mismos.	Analista	30/03/2011

Tabla 22.2. Checklist de especificación de requerimientos ejecutado

Id	Checklist ERS	S/N/NA	Acciones	Responsable	Fecha
11	¿Los casos de uso están empaquetados en paquetes lógicos definidos?	S	-	-	-
12	¿Los requerimientos han sido priorizados?	S	-	-	-
13	Los requerimientos, ¿son unívocamente identificables?	S	-	-	-
14	¿Cada uno de los requerimientos es preciso y no ambiguo?	S	-	-	-
15	¿Cada uno de los requerimientos es verificable o factible de ser testeado?	S	-	-	-
16	¿Cada uno de los requerimientos es correcto?	S	-	-	-
17	¿El origen de los requerimientos está identificado claramente?	N	Incorporar el origen de cada uno de los requerimientos.	Analista	30/03/2011
18	¿Los requerimientos de restricciones de hardware y software están especificados?	N	Se listarán los requerimientos de hardware y software.	Analista	30/03/2011
19	¿Fue construido un glosario?	N	Analizar los términos que serían apropiados incluir.	Analista	30/03/2011

22.2 Métricas en el software

A efectos de calificar y clasificar un proceso o un producto de software según su calidad debemos establecer categorías y criterios para ubicarlos en ellas. Por lo tanto definiremos parámetros de calidad y luego formas de obtener los números asociados. A estos números los llamamos *medidas* o *métricas*. En función del valor de estos parámetros calificaremos la calidad de los productos y procesos de software.

22.2.1 Las medidas como mecanismo de aseguramiento y control de calidad

22.2.1.1 Medidas como verificación de aseveraciones

Las medidas de determinados parámetros asociados a productos o procesos de software permiten, como en otras disciplinas, hacer afirmaciones sustentadas en observaciones provenientes de la realidad. En la figura 22.1 se muestra un esquema para graficar una aseveración muy utilizada en el área del desarrollo de software que dice que “la calidad de un producto depende de la calidad del proceso usado para su producción”. Esta afirmación es indiscutida, sin embargo si deseamos corroborarla debemos sustentarla con medidas que muestren este comportamiento.

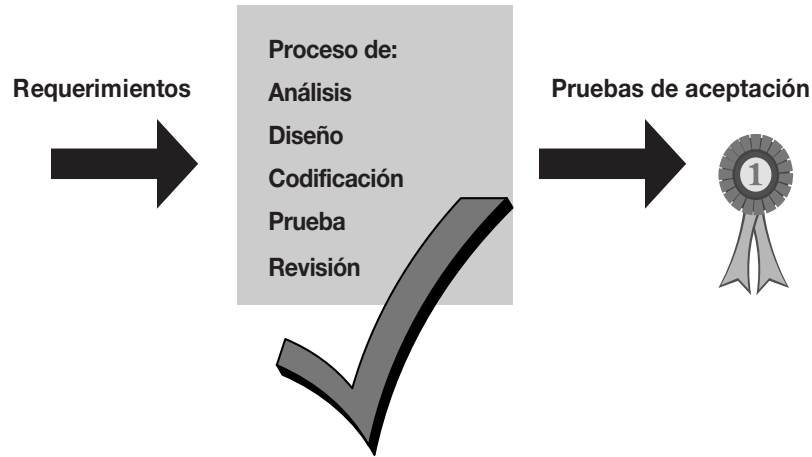


Figura 22.1. Esquema de la hipótesis “a mejor proceso mejor producto”

Para sustentar la afirmación anterior debemos, por ejemplo, medir la efectividad de los procesos involucrados en la generación y la calidad de los productos y correlacionar su variación. Por un lado, podríamos medir el cubrimiento de los test ejecutados, el número de revisiones de diseño y código durante el desarrollo y por otro, la cantidad de fallas del producto generado por puntos de función o líneas de código. Cuanto más eficiente muestren los números al proceso, menor será la cantidad de fallas. De esta forma estaríamos validando la afirmación inicial, de lo contrario la rechazaríamos, o buscaríamos otros parámetros a medir a efectos de modificar la aseveración. Si resulta verdad, las medidas sobre el proceso se convertirán en indicadores guías a efectos de garantizar la calidad de los productos construidos.

22.2.1.2 Modelo conceptual y modelo operacional

El proceso de definir medidas comienza con el análisis del fenómeno a medir. Esta parte del proceso de medidas opera sobre los conceptos asociados a este fenómeno y a las posibles alternativas de calificación y clasificación. A partir de este análisis surgen preguntas guías que conducen la transformación de este análisis conceptual al campo operacional en el cual se definen las medidas necesarias, los indicadores a visualizar o formas de presentación, el procedimiento de recolección de los datos, el proceso y los criterios de análisis y la decisión basada en la información y el procedimiento de almacenamiento. En una próxima sección trataremos cómo estas definiciones aplican a la obtención de medidas de productos y medidas en la mejora de procesos.

22.2.1.3 Indicadores y herramientas

Las medidas que se recolecten deberán ser las adecuadas a efectos de reflejar el fenómeno estudiado. Para esto es importante que posean las escalas adecuadas, la frecuencia de observación y las unidades apropiadas, y que sean relacionadas de la forma correcta. Si son medidas puntuales que han sido obtenidas según el procedimiento correcto, si son estadísticas que provienen de una población que le de validez estadística.

A efectos de obtener indicadores que permitan visualizar los valores de las medidas y su evolución de forma tal que respondan a las preguntas deberemos combinar las observaciones obtenidas. Diferentes herramientas

permiten representaciones distintas tales como los de tipo checklist ya vistos, histogramas, Pareto, de dispersión, de control, series temporales, etc. [4], [7], [8]. Algunos indicadores implementados con estas herramientas se muestran en la figura 22.2.

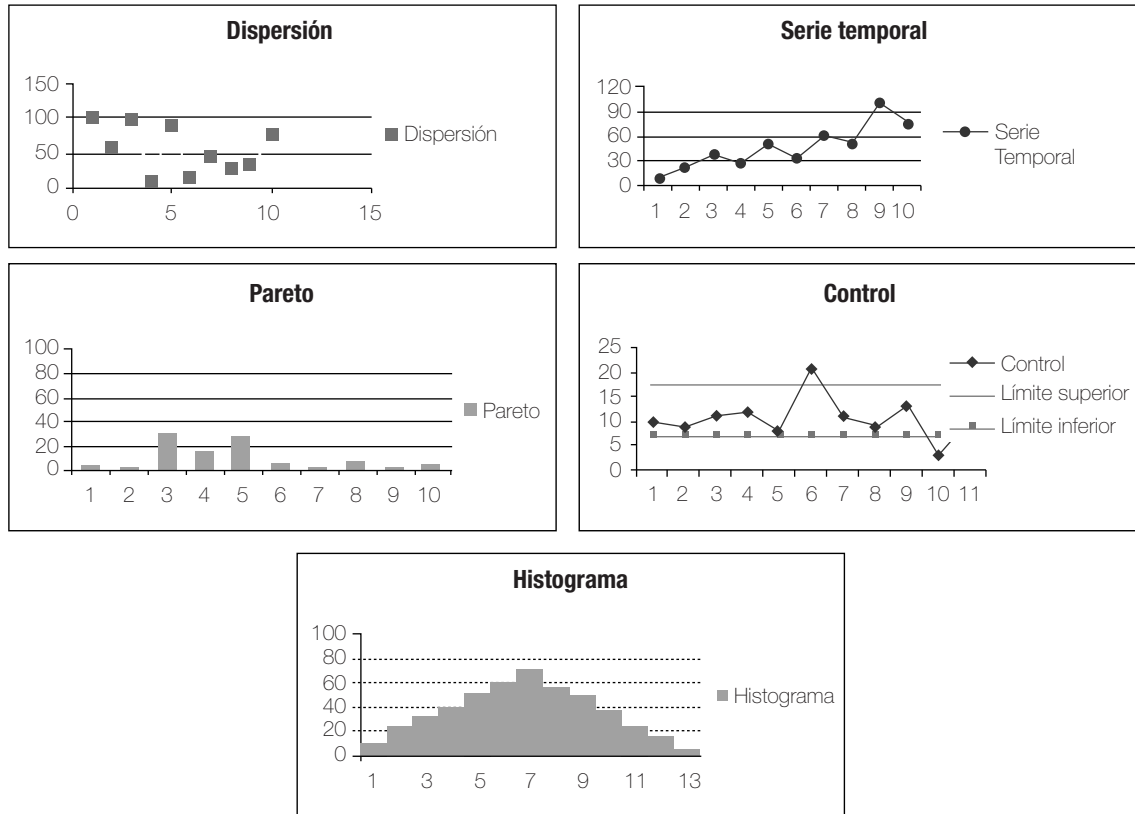


Figura 22.2. Diferentes tipos de indicadores para la visualización de las medidas

22.2.2 Medidas de la calidad de productos de software

Entre las medidas utilizadas para caracterizar un producto se encuentran el tamaño, la complejidad, las características de diseño, el nivel de calidad expresado de una forma preestablecida, etc.

Las medidas son recolectadas para cuantificar tanto atributos internos como externos de calidad, como por ejemplo la satisfacción del cliente. Algunas de estas medidas son:

- Tiempo medio entre fallas
- Densidad de errores
- Problemas del cliente
- Satisfacción del cliente.

El primero mide el tiempo entre dos fallas consecutivas, el segundo el número de errores por alguna unidad que puede ser puntos de función o líneas de código. Ambos cuantifican la calidad interna del producto construido. Los dos restantes lo hacen con los atributos externos [4].

22.2.2.1 Atributos internos

El tiempo medio entre fallas es costoso de ensayar pero muchas veces es necesario, sobre todo en sistemas críticos de monitoreo de parámetros de salud, transporte de larga distancia, sistemas de alarmas y sistemas que en general deben funcionar en forma interrumpida.

Otra forma de expresar atributos de calidad internos es a partir de la cantidad de defectos por unidad de líneas de código. Tradicionalmente esta era una práctica común que se vio afectada hace algunos años con la utilización de plataformas de terceros para la construcción del software propio por parte de las empresas. Estas plataformas suman código no escrito por los desarrolladores, sino por un proveedor que a su vez está proveyendo un producto que posee su propio nivel de calidad. Este atributo lo expresamos así:

$$\text{DLC} = \text{Cantidad defectos} / \text{Líneas de código}$$

Una alternativa es utilizar como unidad de medida los puntos de función.

22.2.2.2 Atributos externos

Los problemas del cliente se cuantifican como la cantidad de problemas por usuario que utiliza el sistema bajo medida.

$$\text{Problemas del cliente (PC)} = \text{cantidad de problemas} / \text{cantidad usuarios mes}$$

Esta medida se puede vincular con los atributos externos. En *Metrics and Models in Software Quality Engineering* [4] se vinculan los problemas generados por los defectos (D) y problemas debidos a otras causas (NDPC) tales como el uso, la documentación poco clara, etc., con la satisfacción del cliente. En la figura 22.3 que sigue se muestra este vínculo a través de un diagrama de Venn donde la satisfacción del cliente (SC) es expresada en términos de los defectos (D) y las otras causas (NDPC).

$$\text{SC} = 1 - \text{PC} = 1 - (\text{D} \cup \text{NDPC}) = (1 - \text{D}) \cap (1 - \text{NDPC})$$

La satisfacción del cliente se mide con frecuencia en una escala de los siguientes niveles:

- Muy satisfecho
- Satisfecho
- Neutro
- Insatisfecho
- Muy Insatisfecho.



Figura 22.3. Relación entre las métricas de calidad internas y externas, adaptada de Metrics and Models in Software Quality Engineering [4]

Y los criterios de encuestas preguntan acerca de la capacidad, funcionalidad, facilidad de uso, rendimiento, fiabilidad, capacidad de instalación, mantenimiento, documentación e información, servicio en general para IBM (CUPRIMDSO) [5] y de la funcionalidad, uso, fiabilidad, rendimiento y servicio para Hewlett-Packard (FURPS) [5].

22.2.3 Medidas de la calidad de procesos de software

22.2.3.1 Medidas asociadas al desarrollo

Una de las métricas más comunes y necesarias en ciclos de vida iterativos es la evolución de los errores detectados en el código mientras se suceden las iteraciones. La visualización de los errores detectados y cómo éstos van siendo corregidos determina la estabilidad de un proyecto. En la figura 22.4 se observa dicha evolución para un proyecto estable en el que los errores pendientes de corrección tienden a bajar a medida que el proyecto avanza. Este comportamiento permite establecer que el producto a instalar hacia el final de la planificación será estable.

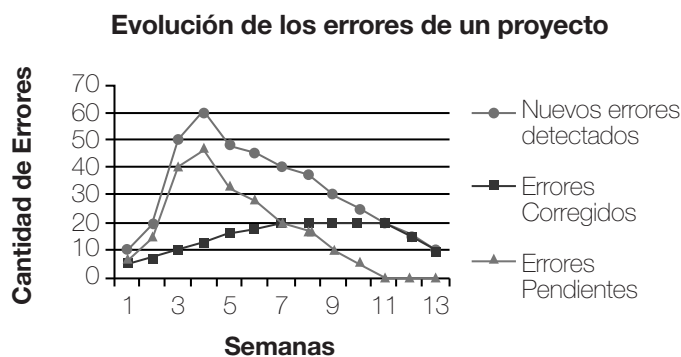


Figura 22.4. Evolución de los errores en función del ciclo de vida de un proyecto

22.2.3.2 Medidas asociadas a la mejora de procesos

Para la evaluación de los procesos de una organización un enfoque muy utilizado y que sirve de marco al plan de medidas y su análisis es el denominado GQM (Goal Question Metric) [6].

La estrategia con la cual se plantean las medidas a recolectar en este marco es que estén alineados los objetivos de medida a los de negocio. Como se muestra esquemáticamente en la siguiente figura 22.5 este modelo consta de tres niveles. Uno es el conceptual o de negocio, el segundo es el operacional y el tercero es el cuantitativo asociado con las medidas y sus indicadores.

Una guía de cómo llevar adelante la planificación y ejecución de un plan de medidas en este marco es:

1. Definir de forma clara los objetivos de negocio de la organización.
2. Generar preguntas acerca de estos objetivos que ayuden a visualizar las condiciones de sus logros. Dar respuesta a estas preguntas son los objetivos de medida.
3. Especificar las medidas necesarias que deben recolectarse para responder a esas preguntas, los indicadores construidos con ellas y el proceso de seguimiento y conformidad entre los resultados obtenidos y las metas buscadas.
4. Desarrollar mecanismos para la recolección de datos y su análisis al momento para aportar realimentación a los proyectos.
5. Recopilar, validar, analizar y almacenar los datos.
6. Analizar los datos recopilados y almacenados para evaluar la conformidad con los objetivos y formular recomendaciones para futuras mejoras.

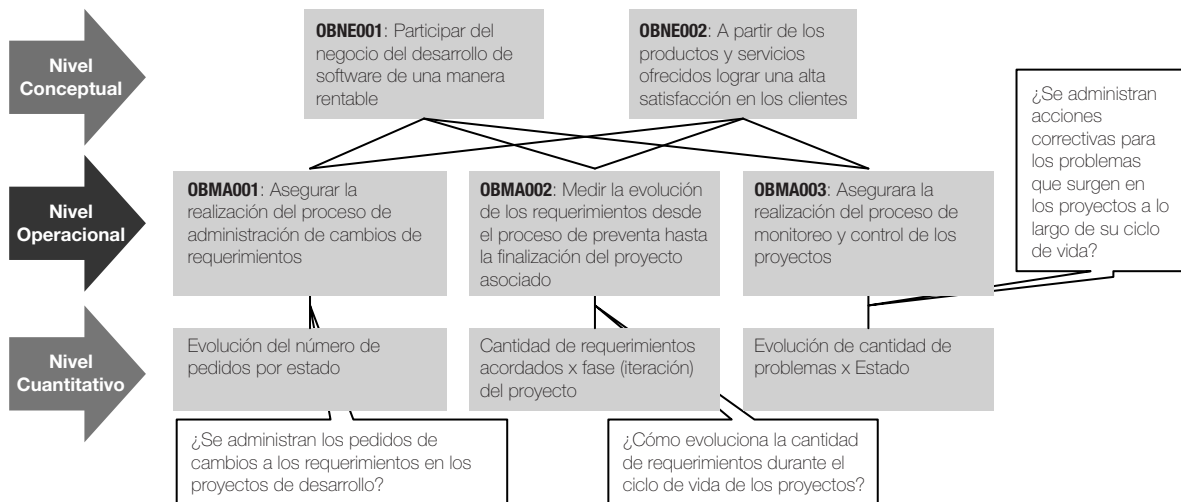


Figura 22.5. Diagrama esquemático que muestra los niveles del modelo GQM. El nivel superior dado por el negocio, el intermedio y bajo por la operatividad

Debe destacarse que los objetivos son la generación de un conjunto de números que ayuden a planificar y gestionar proyectos, mejorar procesos y planificar la organización a partir de la información provista por estas

medidas. En este sentido, y dependiendo de la madurez de la empresa, el plan de medidas tendrá como propósito obtener información que ayude a una de las siguientes propiedades:

1. **Adherencia:** inicialmente se organizarán las medidas con el objetivo de medir la incorporación y el cumplimiento de nuevos procesos y procedimientos que fueron definidos.
2. **Estabilidad:** una vez establecidos los procesos definidos se buscará medir su estabilidad a efectos de tomar acciones para que se comporten de forma estable.
3. **Capacidad:** cuando estos procesos se comporten en forma estable se buscará evaluar el cubrimiento de los objetivos planteados por éstos. Para eso se obtendrán las medidas que se definan a tal fin.
4. **Comportamiento:** en una etapa posterior se tomarán medidas orientadas a evaluar la eficiencia y eficacia de los procesos anteriores.
5. **Mejora:** por último, llegado a este estadio, se continuará con el proceso de medidas y análisis a efectos de lograr una mejora continua de los procesos definidos para la organización.

Ejemplo de plan de medidas en el marco GQM

Este ejemplo corresponde a la empresa SoftDesa, organización dedicada al desarrollo de software del caso testigo que venimos analizando a lo largo de los capítulos anteriores.

Objetivos del plan: Los objetivos de este plan son, en esta primera versión, establecer un proceso de medidas y análisis que permita obtener indicadores de la adherencia de los procesos definidos. Será llevado adelante durante el proceso de mejoras orientado a la certificación CMMi2. En versiones posteriores serán incorporados nuevos objetivos de medida orientados a hacer estos procesos estables, capaces, bien comportados y continuamente mejorables.

Objetivos de negocio (nivel conceptual): de las reuniones con los dirigentes de la empresa fueron recolectados los siguientes objetivos de negocio:

- **OBNE001:** Participar del negocio del desarrollo de software de una manera rentable.
- **OBNE002:** A partir de los productos y servicios ofrecidos lograr una alta satisfacción de los clientes.

Estos objetivos implican el desarrollo de los proyectos dentro de los tiempos y presupuestos planificados. De manera que para lograr estos objetivos se deben hacer buenas estimaciones de esfuerzo, para lo cual se deben relevar los requerimientos necesarios.

Los proyectos deben ser gestionados con una visibilidad importante a efectos de tomar acciones correctivas rápidas que impidan grandes desvíos.

Objetivos de medidas (nivel operacional): Los objetivos de medida que se establecen en este plan surgen de los objetivos de negocio listados y de las falencias observadas en el proceso de desarrollo de software actualmente instalado en la empresa en relación con las aseveraciones anteriores acerca de las condiciones necesarias para el cumplimiento de los objetivos de negocio. Las falencias conocidas pero no mensuradas que atentan contra el logro de los objetivos planteados se pueden expresar en forma resumida de la siguiente manera:

- Falta de visibilidad en el acuerdo de requerimientos.

- Falencias en el relevamiento de requerimientos.
- Falta de mecanismos claramente establecidos de seguimiento y control de los proyectos.
- Falta de disponibilidad de información para toma de decisiones.

Los objetivos de medida derivados de los de negocio anteriores y de las falencias a analizar y resolver se definieron como:

- **OBMA001:** Asegurar la realización del proceso de administración de cambios de requerimientos.
- **OBMA002:** Medir la evolución de los requerimientos desde el proceso de preventa hasta la finalización del proyecto asociado.
- **OBMA003:** Asegurar la realización del proceso de monitoreo y control de los proyectos.

Las preguntas que se plantean para responder son las siguientes:

1. ¿Se administran los pedidos de cambios a los requerimientos en los proyectos de desarrollo?
2. ¿Cómo evoluciona la cantidad de requerimientos durante el ciclo de vida de los proyectos?
3. ¿Se administran acciones correctivas para los problemas que surgen en los proyectos a lo largo de su ciclo de vida?

A partir de estas preguntas se definen las medidas y los indicadores correspondientes los cuales se muestran en las figuras 22.6, 22.7 y 22.8.

Medidas e indicadores (nivel cuantitativo)

Evolución del número de pedidos x estado: este indicador muestra los valores absolutos de los pedidos de cambios de requerimientos según su estado al momento de ser recolectados los datos. La toma de datos se realiza en forma periódica a intervalos de 10 días con lo cual se espera visualizar en este indicador la evolución entre estados, lo que indica la realización de actividades vinculadas al análisis de impacto, el acuerdo, la asignación, implementación y cierre de cada uno de los pedidos de cambios de requerimientos. En la figura 22.6 se observa un indicador de esta medida.

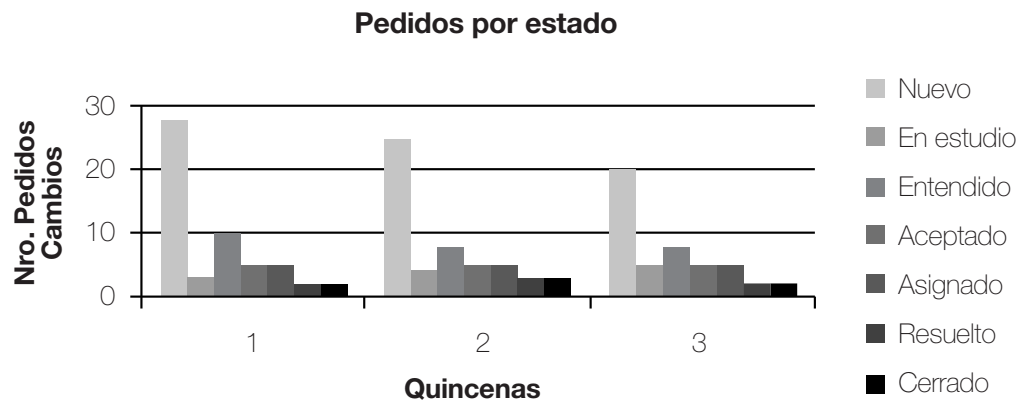


Figura 22.6. Indicador de la medida de pedidos de cambio a los requerimientos

Cantidad de requerimientos acordados x fase (iteración) del proyecto: con este indicador se busca visualizar la evolución de los requerimientos acordados a lo largo del proyecto desde la elaboración de la propuesta técnica en la fase de preventa y la finalización del proyecto. A partir de los resultados obtenidos con este indicador se definirán las acciones que se tomarán orientados a disminuir la diferencia entre los requerimientos iniciales (fase de preventa) y aquellos relevados durante las demás fases del proyecto. En la figura 22.7 se observa un indicador de esta medida.

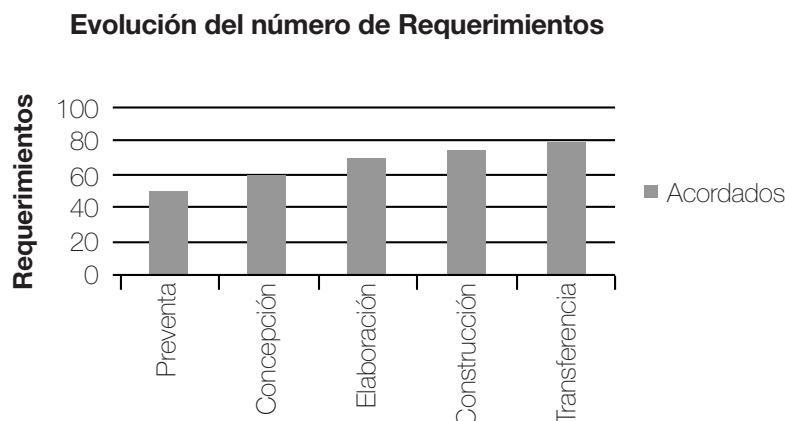


Figura 22.7. Indicador de la medida de evolución de la cantidad de requerimientos por fase del proyecto

Evolución de cantidad de problemas x estado: este indicador muestra los valores absolutos de los problemas surgidos en el proyecto según su estado al momento de ser tomados los datos. La toma de datos se realiza en forma periódica en concordancia con las reuniones de estado y avance definidas en el plan de proyecto correspondiente. En la figura 22.8 se observa un indicador de esta medida.

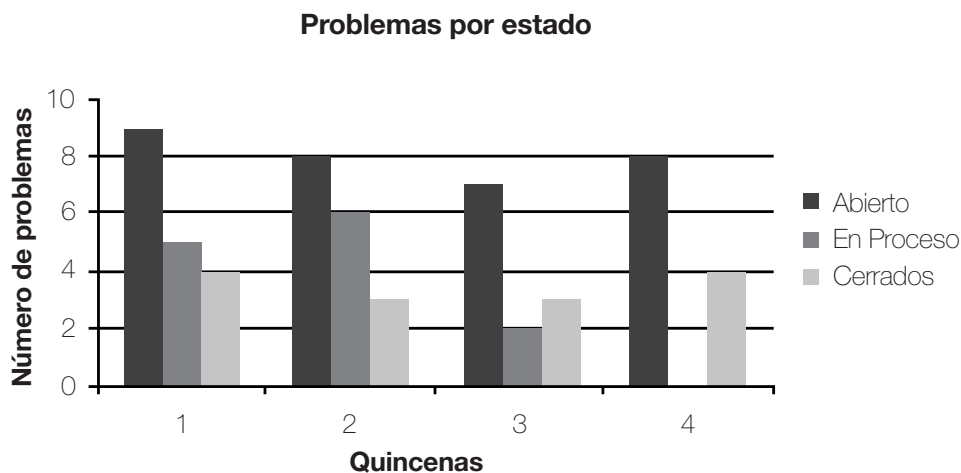


Figura 22.8. Indicador de la medida de problemas por estado del proyecto a lo largo de su ciclo de vida

Con el análisis de estos indicadores se logró obtener información que mostró los siguientes resultados:

1. Existían falencias en el relevamiento y análisis de requerimientos en las primeras fases de los proyectos. Ésta era la razón del crecimiento en la cantidad cuando los proyectos avanzaban. Para resolver esto se asignó más tiempo a las tareas vinculadas al trabajo con los requerimientos además de capacitar a los analistas en estas tareas.
2. Los pedidos de cambio a los requerimientos no se analizaban con suficiente rigor, se definieron procedimientos de análisis de impacto que permitieron priorizar y programar el tratamiento a dichos pedidos. Los pedidos de cambios eran excesivos debido a la falta de sistematización en su tratamiento.
3. Algunos de los problemas detectados nunca cambiaban de estado debido a que no se escalaban algunos problemas surgidos en los proyectos a la gerencia lo cual determinaba que éstos no fueran resueltos con las consiguientes consecuencias de retrasos en los tiempos de los proyectos. Se ajustaron las revisiones de los problemas en las reuniones de evaluación.

Estas medidas sirvieron para cuantificar algunas de las falencias conocidas y hacer visibles otras, de manera de adoptar soluciones que permitieron corregirlas.

22.3 Conclusiones



Actualmente más empresas buscan distinguirse por la calidad de sus productos. Como hemos comentado, la calidad de los productos de software generados depende directamente del proceso definido para obtenerlo. Aunque desde el punto de vista de los usuarios, la calidad es una cualidad propia del producto.

Por último, la calidad externa es negociable y no así la calidad interna, dado que un producto con baja calidad interna difícilmente consiga una buena calidad externa. Mientras que un producto con buena calidad interna puede o no tener calidad externa.

Muy probablemente el cliente no recuerde si el producto de software con baja calidad fue entregado a tiempo o si el producto de alta calidad fue entregado tarde mientras que les agrade el producto que están utilizando [3].

Hemos analizado también en este capítulo la importancia central que tienen las métricas a efectos de cuantificar y calificar la calidad de los productos y procesos de software. Sustentan las afirmaciones acerca de diferentes cuestiones y constituyen una guía de ayuda en la toma de decisiones tanto en la calificación de productos como en la mejora de los procesos que los producen.

22.4 Contenido de la página web de apoyo

22.4.1 Mapa conceptual

22.4.2 Autoevaluaciones

22.4.3 Presentaciones en Power Point (exclusivo para el docente)

22.5 Referencias

- [1] *Diccionario de la lengua española*, Real Academia Española (RAE).
- [2] *Ingeniería de Software. Un enfoque práctico*, Quinta Edición, Roger S. Presuman, McGraw-Hill, 2002.
- [3] *Software Survival Guide*, Steve McConnell, Microsoft Press, 1998.
- [4] *Metrics and Models in Software Quality Engineering*, Second Edition, Stephen H. Kan, Addison-Wesley, 2002.
- [5] *Quality Software Project Management*, Robert T. Futrell, Donald F. Shafer, Linda I. Safer, Prentice Hall PTR, 2002.
- [6] *The Goal Question Metric Approach*, Basili, Victor; Gianluigi Caldiera, H. Dieter Rombach, 1994.
- [7] *Practical Guide to Software Quality Management*, Second Edition, John W. Horch, Artech House, 2003.
- [8] *Software Metrics: A Rigorous and Practical Approach*, Revised, Norman E. Fenton, Shari Lawrence Pfleeger, PWS Publishing Company, 1997.

Organización del área de
calidad de una empresa

Contenido

23.1 Introducción	398
23.2 Origen	398
23.3 Implementación del área de calidad	399
23.4 Conclusiones	414
23.5 Contenido de la página web de apoyo	415
23.6 Referencias	415

Objetivos

- Cuáles son los organigramas alternativos
- Qué roles deben definirse
- Cuáles son las actividades a llevar a cabo
- Qué recursos son necesarios

Competencias

- Establecer criterios de calidad en una organización dedicada al desarrollo de software para llevar adelante sus proyectos

23.1 Introducción

Podemos hablar de calidad en todas las áreas de los negocios pasando por la industria automotriz hasta la industria del software. Esto indica que el concepto de calidad en las organizaciones data de hace mucho tiempo. En este sentido podemos mencionar la evolución del concepto de calidad partiendo de la calidad de producto, de proceso, y finalizando con el concepto de la calidad total (Total Quality Control, TQC). Inicialmente se hablaba de un control de calidad intentando detectar los defectos, posteriormente se buscó asegurar la calidad por medio de la prevención de los defectos y por último mencionamos la calidad total apoyada sobre la mejora continua.

La evolución que mencionamos en el párrafo anterior trajo como consecuencia la creación de áreas, dentro de las compañías, que tienen como principal objetivo la administración de la calidad. En este contexto es que surgen las áreas de aseguramiento de la calidad de servicios (SQA).

En este capítulo no se intenta dar fórmulas mágicas en respuesta a todos los problemas que presenta una organización. Simplemente se pretende guiar al lector en el proceso de creación del área de calidad.

23.2 Origen

Ahora bien, ¿cuáles son los síntomas que indican la necesidad de un área de aseguramiento de calidad en una organización? Podemos mencionar algunos puntos para guiar al lector, como ser: baja calidad de los productos generados (por ejemplo una gran cantidad de fallas del producto, errores detectados por el cliente), la falta de procesos o la ausencia de control en la organización. Es importante que se comprenda que los síntomas dependen del contexto de la organización: su tamaño, su cultura, su estructura, la situación del/los país/es con el/los cual/es interactúa.

23.2.1 Primeros pasos

El primer gran paso es detectar que existe un problema. El segundo, identificar que la solución a ese problema es la creación de un área de aseguramiento de la calidad en la organización. En este punto, y antes de comenzar con la mención de los primeros pasos a llevar a cabo, es importante recalcar que para lograr con éxito la creación del área y sus objetivos es crucial contar con el apoyo de la alta gerencia.

Pues bien, para llegar a aquí seguramente se ha llevado a cabo una evaluación de la situación, se han identificado los principales problemas y se ha elaborado un diagnóstico que nos ha revelado la necesidad de un área de calidad. Asimismo es conveniente tener presentes las ventajas y desventajas de la incorporación del área de calidad en la organización. Es clave la lista de ventajas para la gerencia dado que con su apoyo será posible conseguir el tiempo y los recursos para la implementación del área.

Nos encontramos frente al requisito de constituir un área de calidad dentro de una empresa que ya está en funcionamiento, que tiene una cultura y un flujo de trabajo ya establecido. Por lo tanto, a continuación es conveniente describir la estrategia y las tácticas para establecer la dirección en la cual se trabajará. Aquí es crucial

definir la dirección del cambio alineada con los objetivos de negocio de la organización. En tanto a continuación elaboraremos la planificación que nos guiará durante todo el proceso hasta completar la implementación.

Seguramente la inserción de esta nueva área provoque cambios dentro de la organización y muchas veces los cambios son resistidos por sus miembros. Algunas de las situaciones frecuentes que pueden presentarse son la falta de consenso del problema a resolver, la dirección de la solución y la resistencia al cambio. Para resolver esto es importante comprender el estado actual, el contexto de la organización e instaurar la necesidad del cambio. Una vez establecida la necesidad del cambio se deben presentar las alternativas posibles de solución y transmitir que es posible mejorar. El apoyo de la gerencia y de la organización completa durante esta fase es relevante para el avance del proceso. Cuando decimos la palabra *apoyo* nos estamos refiriendo a un entendimiento y acompañamiento durante esta etapa.



Figura 23.1. Aspectos a considerar para lograr la implementación del área de calidad

Una vez que todos los escollos han sido resueltos y hemos llegado hasta aquí, comenzamos a implementar el área de calidad: desarrollo de tácticas, de planes detallados, asignación de responsabilidades y autoridad. En esta etapa también es importante tener en cuenta que el nivel de resistencia continúa presente: falta de una clara visión del cambio y los obstáculos en el logro de la solución, falta de confianza y temores acerca del plan elaborado. Para reducir la resistencia en esta etapa es clave contar con el compromiso del área de desarrollo de software, seguir trabajando alineados con los objetivos de la organización y llevar a cabo una capacitación sobre los nuevos procesos implementados con el fin de comprender los beneficios de los cambios. En la figura 23.1 se esquematizan los aspectos a considerar en la implementación descrita.

En este punto podemos concluir que con el fin de mejorar el producto final, brindándole mayor calidad al cliente, es preciso un cambio que deberá tener relación directa con los objetivos del negocio de la empresa, para contar con el apoyo de la gerencia.

En el siguiente apartado detallaremos el proceso de implementación del área de calidad.

23.3 Implementación del área de calidad

Encarar la implementación del área de calidad puede resultar en fracaso dependiendo del modo en el que se lleve a cabo. Por un lado, si intentamos ejecutar la implementación en forma paralela en todos los proyectos de la organización, sin tener en cuenta el estado de cada uno de ellos, probablemente en algunos tenga éxito y en otros fracase. Por otro, si intentamos seleccionar un aspecto e implementarlo en todos los proyectos de

la empresa en forma paralela nuevamente en algunos proyectos será exitosa la situación pero en otras fallará. En ambos casos para lograr casos de éxito se deberá entender el contexto del proyecto y a partir de eso implementar lo que se requiera. Veamos algunas estrategias que han tenido éxito.

- Escenario de aplicación: sólo proyectos nuevos
En este contexto la implementación se realizará sólo sobre los proyectos nuevos. De esta manera se evitan los problemas que podrían presentarse en proyectos avanzados en el ciclo de vida de desarrollo y se gana en que el proyecto arranca desde un comienzo con “la nueva forma de trabajo”. La desventaja de esta aproximación es que los proyectos iniciados no podrán gozar de los beneficios de los nuevos elementos. A menos que no existan proyectos iniciados previamente.
- Escenario de aplicación: sólo un aspecto a la vez
En este contexto se seleccionará un aspecto y será implementado en todos los proyectos (nuevos y en progreso) pero siempre teniendo en cuenta la fase en la cual se halla cada uno de ellos. Para lograr casos satisfactorios es importante comprender los beneficios y perjuicios que podría traer consigo la incorporación del aspecto en cuestión. Tengamos en cuenta que hay aspectos que son independientes de la fase del proyecto y otros, como por ejemplo su configuración, que comenzarlos en medio de un proyecto puede ser altamente problemático.

La ventaja de esta aproximación es que los proyectos pueden beneficiarse del elemento introducido en forma temprana y permite observar los beneficios durante el resto del proyecto. La desventaja es que en los proyectos nuevos sólo se percibirán los beneficios del elemento introducido. A menos que la organización no cuente con proyectos nuevos.

- Escenario de aplicación: implementación combinada
Las aproximaciones mencionadas previamente forman parte de un extremo. Será conveniente el empleo de la primera alternativa cuando no existan proyectos iniciados previamente. Mientras que será preferible implementar la mejora de un aspecto a la vez cuando no haya proyectos nuevos. Pero sabemos que esto en las organizaciones reales no sucede y tenemos nuevos proyectos y proyectos que ya han comenzado. Por lo tanto, la mejor alternativa es la implementación combinada que tiene en cuenta el estado de cada uno de los proyectos para llevar a cabo la implementación.

Para proyectos nuevos se implementará la mayor cantidad de aspectos posibles. Preferiblemente se implementarán todos los aspectos.

Para proyectos ya iniciados se realizará un análisis y se incluirán aquellos aspectos que sean necesarios y logren beneficios para el proyecto. Por ejemplo, si un proyecto se ha iniciado recientemente quizás sea conveniente la incorporación de la administración de la configuración. Mientras que si el proyecto se encuentra en la fase final, incluir la administración de la configuración traerá consigo un gran retrabajo que demorará otras tareas primordiales en esa fase.

Finalmente, queremos remarcar que la implementación que mejor se adapta a cada organización depende directamente de su contexto y aquella que tiene en cuenta la fase en la cual se encuentra cada uno de los proyectos que serán afectados.

23.3.1 Alternativas de organigrama

Cuando mencionamos la implementación del área de calidad en la sección Primeros pasos hemos hecho mención implícita a la ubicación del área de calidad dentro de la organización. Esta ubicación varía en función de su contexto.

Independientemente de ello, hay ciertas condiciones que deben cumplirse para que las acciones del área de calidad sean efectivas. Dentro de las condiciones necesarias podemos mencionar (adaptado de *Practical Guide to Software Management*, John W. Horch):

- El área de calidad debe reportar a niveles organizacionales superiores de decisión.
- El área de calidad debe ser independiente del/las área/s a la/s cual/es monitorea. Supongamos el siguiente organigrama que se muestra en la figura 23.2:



Figura 23.2. Área de calidad reportando al área a la cual monitorea (adaptado de *Practical Guide to Software Management*, John W. Horch)

Notablemente esta ubicación dentro de la organización no será útil para el área de calidad. Probablemente los problemas encontrados por el área no traspasen las barreras del área de desarrollo de software y, por lo tanto, no se tomen las acciones correctivas apropiadas.

Teniendo en cuenta los puntos mencionados previamente podríamos establecer el organigrama de la figura 23.3. Aquí el área de calidad se encuentra en el mismo nivel organizacional que el resto de las áreas de procesamiento de datos. Entonces el gerente del área de calidad es un par del gerente del área que se está monitoreando. Asimismo hay un gerente superior a ambos que será quien resolverá los inconvenientes que no puedan ser resueltos por los gerentes involucrados.

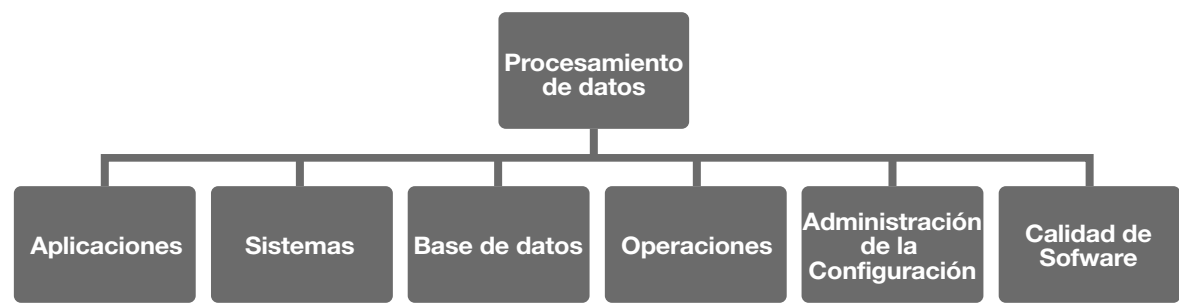


Figura 23.3. Área de calidad al mismo nivel (adaptado de Practical Guide to Software Management, John W. Horch)

Las ventajas de este organigrama son las siguientes:

- Las áreas que deberán ser monitoreadas por el área de calidad son pares entre sí.
- Cuando surja algún conflicto ente el área de calidad y algún área monitoreada, habrá un único superior que será el responsable para mediar en el conflicto suscitado.
- Los miembros del área de calidad están al alcance del resto de los equipos en caso que lo requieran.

Organizaciones con un sistema de calidad maduro y fuerte presentan organigramas como el de la figura 23.4. Aquí el área de calidad de software es una extensión del área de calidad.

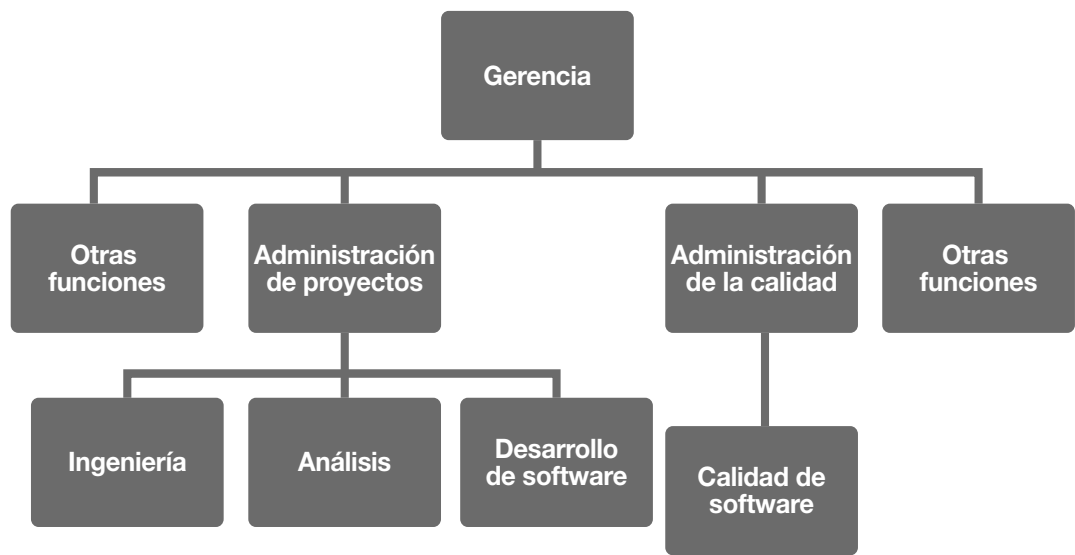


Figura 23.4. Organigrama alternativo (adaptado de Practical Guide to Software Management, John W. Horch)

Como puede apreciarse, el área de calidad reporta directamente a la alta gerencia. Esto puede ser un inconveniente si la comunicación entre el área de calidad y las áreas a las cuales monitorea no es buena y fluida. Otra cuestión que podría plantearse es el “excesivo” poder que tiene el área de calidad. Por lo tanto el éxito de este organigrama dependerá de la interacción entre esta área y aquellas que monitorea y del aparente poder que ésta podría tener.

Como se ha comentado previamente, dada la dependencia que existe entre las tareas que el área de calidad monitorea y que el área de desarrollo ejecuta es importante mantener cierta proximidad para trabajar en forma conjunta cómodamente. Pero es imprescindible que el área de calidad sea independiente administrativamente y en forma financiera de las áreas de monitorea.

23.3.2 Roles

Si bien es cierto que cada uno de los miembros de la organización es responsable por la calidad, esto en la práctica no es así. En las organizaciones la responsabilidad recae sobre el gerente de la empresa, quien a su vez delegará las responsabilidades a los gerentes de las distintas áreas. Así los diferentes miembros de la organización poseen distintas responsabilidades en relación a su rol.

Antes de adentrarnos en las características de cada uno de los roles nos gustaría mencionar una serie de responsabilidades que es importante que todos los integrantes del área de calidad conozcan para desarrollar adecuadamente su labor, como ser: revisar documentación, tener conocimiento sobre las normas aplicadas a los proyectos, saber cómo comunicar los hallazgos a las gerencias que correspondan, hacer seguimiento de las no conformidades y controlar el cumplimiento de los procesos definidos. En relación a los procesos podemos mencionar los procesos para la administración de la configuración, de los requerimientos, de las métricas y del seguimiento de los proyectos, entre otros.

Ahora sí veamos los distintos roles que pueden formar parte del área de calidad, no nos olvidemos que los roles presentes dependerán del tamaño de la organización (no es lo mismo un área de calidad en una empresa de 50 empleados que en una organización de 10.000 empleados) y de su estructura (como hemos visto en el apartado anterior, hay distintas alternativas de organigrama).

Analizador de defectos: Será la persona responsable de asegurar que el proceso de pruebas ha sido ejecutado apropiadamente. Dentro de esta actividad se verificarán que los casos de prueba planteados satisfacen el plan de pruebas, se revisará el resultado de las pruebas procurando asegurar que cada uno de los errores encontrados tenga el tratamiento adecuado. Asimismo debe asegurar que se efectúen los adecuados test de regresión en los casos que correspondan procurando a su vez que los test no sean incompletos o redundantes. Recordemos que el objetivo de las pruebas es encontrar errores. Es responsabilidad de este rol la revisión de todas las fases y tipos de pruebas que se ejecuten, como por ejemplo, las pruebas funcionales, de sistema y de integración, entre otras.

Este rol será fundamental previo a la entrega del producto al cliente, es decir antes de la ejecución de los pruebas de aceptación, dado que debe asegurar que se hayan ejecutado todas las pruebas necesarias, que se hayan corregido los errores detectados y que la administración de la configuración sea la apropiada (con esto queremos decir que se haya trazado una línea base con la información correspondiente). Con to-

das las tareas mencionadas el analizador de defectos guiará el proceso hasta las pruebas con el cliente y su satisfacción.

Revisor de diseño: Es el responsable de asegurar que las revisiones de diseño de software se realicen en tiempo y forma. Con respecto a las revisiones informales no es necesario que participen, pero es crucial que pueda asegurar que éstas se efectúan. En relación a las revisiones formales, sí es necesaria su presencia para asegurar que las decisiones tomadas se cumplan posteriormente.

Revisor de documentación: Es el responsable de realizar las revisiones de la documentación generada durante el desarrollo de los proyectos. Es quien debe poder asegurar que la información necesaria está presente y de forma adecuada, es decir que se adapta a su destinatario. En caso de encontrar alguna falla, ésta deberá ser corregida por el responsable del documento.

Dentro de la documentación más importante que el revisor controlará podemos mencionar las planificaciones de desarrollo. Con respecto a éstas, se podrá hacer un seguimiento cooperando para su cumplimiento y reportando cualquier desviación. Otro documento de relevancia a ser revisado es aquel que contiene los detalles de los requerimientos. Es importante que se verifique que los requerimientos sean necesarios, correctos, completos, medibles, no ambiguos y consistentes.

Por último, queremos mencionar las revisiones a la documentación del usuario. El revisor deberá analizar los manuales para verificar si son útiles para el usuario y si la información necesaria está presente en ellos.

Este rol también cuenta con la responsabilidad de asegurarse que las revisiones se realicen en tiempo y forma, conforme a un calendario acordado.

Educador: Es el responsable de que cada una de las personas de la organización conozca sus tareas y responsabilidades en relación a la calidad. Para cumplir esta meta podrá coordinar un plan de capacitaciones. Aunque generalmente las organizaciones acostumbran a asignar las actividades de capacitación a un equipo externo mediante videos previamente preparados para tal fin o a personas internas que tienen experiencia sobre el tema. Más allá de cómo se lleven a cabo las capacitaciones, nos gustaría reforzar su importancia para los integrantes de la organización: conocer las tareas permite a los empleados mejorar su trabajo diario y, por lo tanto, aumentar la calidad del producto final.

Miembro del comité de control de cambios: Será la persona que forma parte del comité y será quien monitoree los aspectos de calidad de los cambios. Recordemos que el comité de control de cambios es la autoridad que decide si los cambios son llevados a cabo o no.

Por todo lo mencionado hasta aquí, esperamos que el lector haya podido comprender que las personas que conforman el área de calidad no escriben documentos ni corren los test. Los roles buscan establecer que esas actividades se ejecuten y se realicen de tal manera que sea posible asegurar que el resultado obtenido cumple con los requerimientos, por ejemplo. Debe revisar, inspeccionar, evaluar, medir y reportar. Y hasta aquí llega su alcance. Con esto queremos decir que las acciones a tomar a partir del reporte del área deben ser consensuadas y aprobadas por la gerencia y ejecutadas por el área pertinente.

23.3.3 Actividades

Adentrémonos un poco en las actividades del área de calidad para poder comprender la necesidad de su existencia. Para ello, plantearemos algunos escenarios diferentes para comprender cómo actúa el área de calidad en cada caso.

Comencemos pensando en un desarrollo en cascada, ver figura 23.5.

Como ya hemos visto, el primer hito que nos interesa en este modelo es la definición de software, donde el responsable de calidad deberá revisar la documentación generada durante la fase, como por ejemplo el documento de visión. Asimismo deberá asegurar el cumplimiento de los procesos definidos por medio de la verificación de los procedimientos establecidos. Una vez que el hito de la fase se haya conseguido, se deberá validar que ésta se ha ejecutado satisfactoriamente de acuerdo a lo definido y por lo tanto se procederá a la siguiente, de definición de los requerimientos donde el responsable de calidad debe revisar que el proceso definido para este fin se lleva a cabo adecuadamente, para lo cual puede, al finalizar la fase, ejecutar el checklist de los procesos correspondientes y la fase. Asimismo, para asegurar la necesidad, completitud, no ambigüedad y la consistencia de los requerimientos podrá revisar el documento generado con los requerimientos relevados.

A continuación nos centraremos en la definición de un diseño de arquitectura, por lo cual el trabajo del responsable de calidad será asegurar que las revisiones de diseño se lleven a cabo y además participará en aquellas revisiones formales para hacer el seguimiento correspondiente. En caso de que se genere un documento, éste deberá ser auditado por el área de calidad. Asimismo deberá hacer el seguimiento de las no conformidades encontradas en las fases anteriores, si es que existieran. Cuando la fase de diseño se encuentre finalizada se deberá ejecutar el checklist correspondiente a esta fase y en caso de que no se encuentre ninguna observación bloqueante, se procederá con las siguientes etapas. Durante la fase de codificación, los desarrolladores codificarán en base a lo definido en las fases previas. Nuevamente la documentación generada deberá ser validada así como los procesos. Cuando los documentos generados y los procesos hayan sido verificados, se procederá a la etapa de pruebas, en la que los responsables ejecutarán las pruebas pactadas y reportarán las fallas detectadas. Los errores detectados serán corregidos por el equipo de desarrollo a lo largo de esta fase. El área de calidad es responsable de asegurar el cumplimiento del proceso definido para esta etapa y la calidad del producto final.

Una vez que el producto se encuentre listo para ser implementado se ejecutarán los checklist correspondientes a la fase y en caso de no hallar observaciones bloqueantes se dará inicio a las tareas de la siguiente etapa.

Posteriormente nos encontramos con la fase de aceptación e implementación, donde el área de calidad deberá asegurar que se han ejecutado las tareas de configuración correspondientes, que la documentación generada para el cliente es adecuada y que el proceso de implementación y de aceptación se sigue de acuerdo a lo definido. Una vez que la implementación y las pruebas de aceptación han finalizado, el área de calidad ejecutará la validación correspondiente al fin de la fase y en caso de no encontrar observaciones bloqueantes se proseguirá a la fase de mantenimiento.

A lo largo de la fase de mantenimiento el área de calidad es la responsable de asegurar que los procesos definidos se ejecutan de acuerdo a lo definido, tanto para el mantenimiento del producto como para el caso de presentarse algún inconveniente con éste.

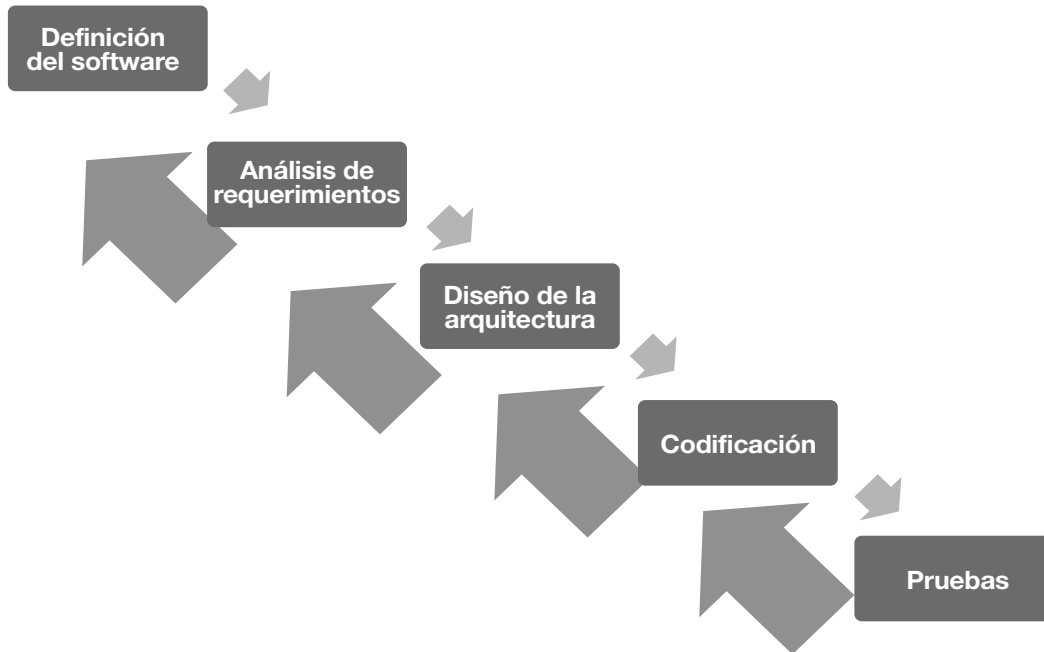


Figura 23.5 - Escenario desarrollo en cascada

Como el lector habrá podido observar durante cada una de las fases el área de calidad debe asegurar la ejecución de los procesos de acuerdo a lo definido y validar la documentación generada con el fin de verificar su calidad. Luego, **recomendará**, de acuerdo a los hallazgos, continuar con la próxima fase o no. Hemos destacado el vocablo “recomendar” dado que no será quien decida si se prosigue o no. Esta decisión será tomada en forma conjunta con la gerencia que corresponda.

Le recomendamos al lector dirigirse a la figura 23.6 para observar que el área de calidad forma parte activa de todas las instancias del desarrollo de software, incluso durante el proceso de adquisición de un software externo. Aquí es crucial que se asegure que éste cuente con toda la documentación necesaria para su adecuado uso dentro de la organización.

Hasta aquí hemos estudiado las actividades del área de calidad en un escenario con desarrollo en cascada. Pero ¿qué sucede si estamos ante un desarrollo iterativo incremental? Como podemos ver en la figura 23.7 las múltiples iteraciones. En cada una de éstas se ejecutarán las tareas de calidad. Es decir que si se desarrollan tres iteraciones distintas de relevamiento de requerimiento, el área de calidad deberá ejecutar el checklist de la especificación de requerimientos en las tres, así como también el checklist de los procedimientos correspondientes para asegurar su cumplimiento y al terminar cada una de las iteraciones deberá validar la finalización para poder proseguir con la siguiente. Estas tareas en lugar de realizarlas una sola vez, como sucedía para el desarrollo en cascada, se ejecutarán tres veces, en este caso. En general, las tareas de calidad se ejecutarán tantas veces como se ejecuten las tareas en el proyecto.

Para las metodologías de desarrollo de software tradicionales podríamos concluir que la documentación se deberá validar a medida que es generada y los procesos se revisarán de acuerdo a lo definido y por lo general cuando estén finalizados para proseguir a la siguiente fase.

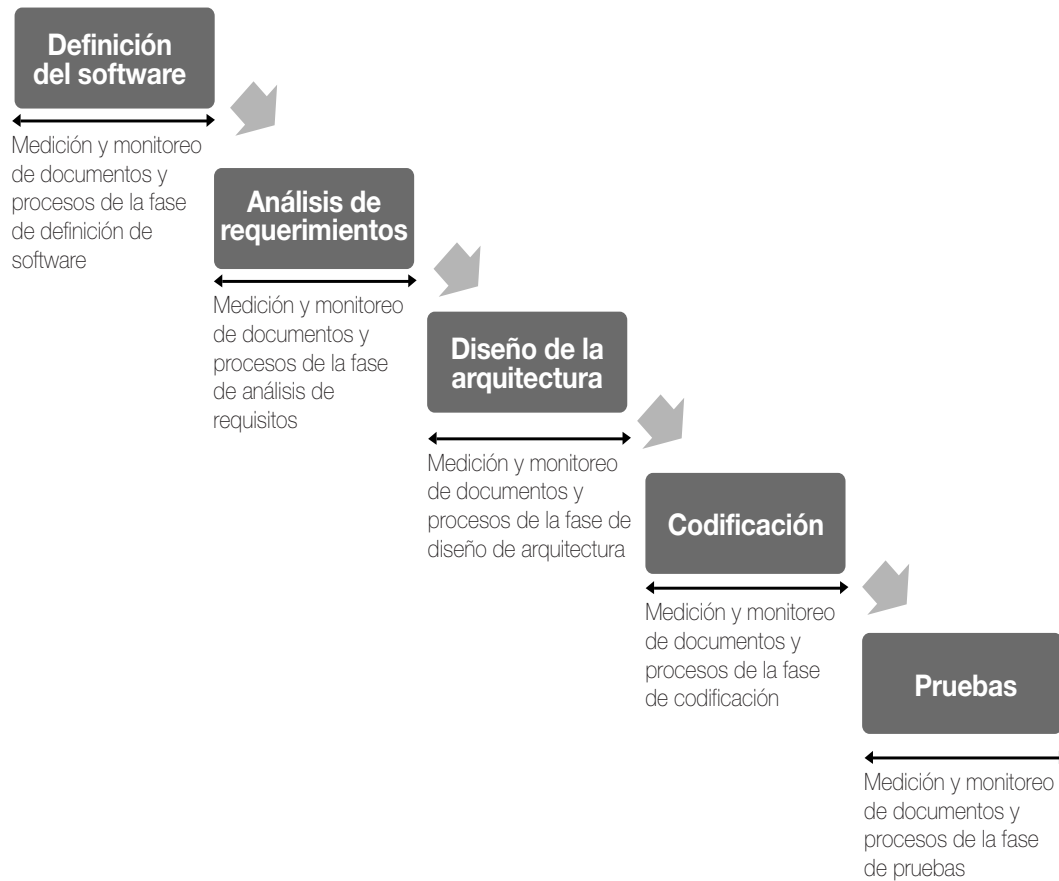


Figura 23.6 - Escenario desarrollo en cascada junto a las tareas del área de calidad

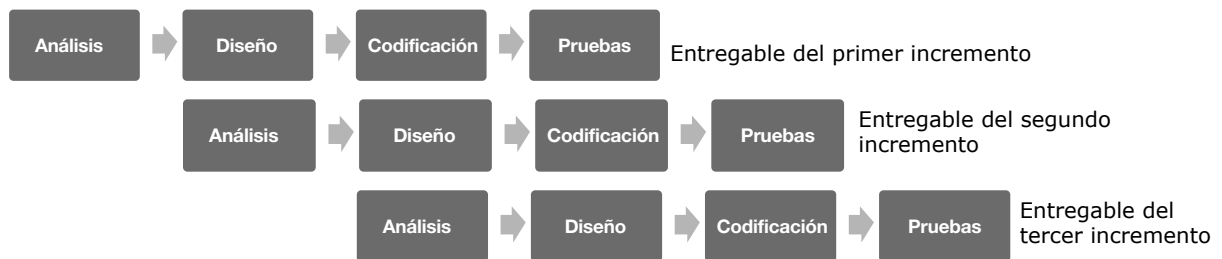


Figura 23.7. Escenario desarrollo incremental

Pues bien, hemos estado mencionando en varias oportunidades las mediciones y el monitoreo que lleva a cabo el área de calidad, pero ¿qué hacemos con las mediciones recolectadas y las observaciones detectadas?

Los resultados de las mediciones deben ser procesados para informar a la gerencia de lo relevante. En relación a las observaciones se deben informar aquellas que sean necesarias y además es pertinente hacer un seguimiento de ellas para conseguir que sean solucionadas.

Pero eso no es todo. Las mediciones recolectadas nos permiten detectar estados y mejorar los procesos definidos.

Analicemos ahora cuáles son las actividades de un área de calidad en un contexto ágil. Recordando la forma de trabajo de XP/Scrum, por ejemplo, nos vienen a la cabeza los Sprints, cada uno de los cuales busca obtener una versión funcional del producto en su finalización y lograr un software de mayor calidad. Con este mismo fin se utilizan diversas prácticas en el contexto ágil. Por ejemplo:

- Programación de a pares (pair programming): mejora la calidad del código desarrollado.
- Desarrollo guiado por las pruebas (test driven development): el desarrollado será guiado por las pruebas, haciendo código testeable.
- Pruebas automáticas
 - Pruebas luego del desarrollo: Las pruebas se elaboran y ejecutan luego que el código esté desarrollado.
 - Pruebas antes del desarrollo: Las pruebas se elaboran antes del desarrollo, por lo tanto se realizan teniendo en cuenta los requerimientos. Esto a su vez permite un entendimiento de los requisitos y un diseño evolutivo.
- Diseño simple: mejora la calidad del diseño y brinda facilidad para ser modificado en el futuro. El hallazgo de mayor cantidad de errores brinda mayor calidad al producto final.
- Integración continua: permite encontrar errores en el producto en forma frecuente y temprana.
- Refactor: permite mantener el código actualizado de acuerdo a las necesidades. Brinda calidad dado que el diseño no se degrada.

¿Cómo participa el área de calidad en estas tareas? De la misma forma que en las otras metodologías, realizando revisiones, relevando métricas y aportando una visión objetiva.

Como el lector habrá podido observar, hemos estado mencionando diversas prácticas que mejoran la calidad interna. Quizás se estén preguntando qué sucede con la calidad externa. Pues bien, recordarán que hemos dicho que la calidad externa es negociable, pero no lo es la calidad interna. En relación a la calidad externa es importante saber si el cliente realmente está en condiciones y desea pagar los costos implicados. Muchas veces, sobre todo en períodos de crisis, el cliente prefiere mantener un cierto nivel de calidad y disminuir los costos. Esto puede sonar un poco extraño, hemos estado mencionando continuamente la importancia de la calidad y ahora me dicen que puede no ser tan relevante. Bueno, no es tan así, veamos un ejemplo. Supongamos que deseen comprar una pelota de fútbol para jugar con vuestros amigos una vez a la semana. Muy probablemente no quieran gastar mucho, dado que simplemente quieren una pelota para jugar una vez por semana con sus amigos. Entonces si la pelota es muy costosa, probablemente no la adquieran. Realmente no les importa que la calidad no sea excelente a cambio de comprarla a un costo razonable. Pues bien, nuestro cliente está en la misma situación que nosotros con la pelota.

Hemos visto que algunas de las actividades del área de calidad pueden ser desarrolladas con la participación de personas del área de desarrollo de software. Esto reducirá el posible miedo o la desconfianza hacia el área que los monitoreará e instaura en algunos casos un sentimiento de propiedad por parte de los miembros del equipo de desarrollo. Tengamos en cuenta que en la implementación del área de calidad varios elementos del proceso de desarrollo pueden ser modificados, como ser el ciclo de vida de desarrollo y los procesos involucrados. En relación a los procesos involucrados podemos mencionar la elaboración de una nomenclatura común estableciendo, por ejemplo, un cierto patrón para los nombres de los documentos: CódigoDelProyec-

to_NombreDelCliente_NombreDelDocumento_Versión. Otro ejemplo frecuente es la definición de un formato unificado para los documentos: todos deberán tener un cierto encabezado y pie de página, una cierta carátula y otros patrones que se pactarán buscando de esta forma lograr una estandarización de la documentación.

El área de calidad participará en la definición de los procesos de desarrollo así como en los relacionados a la administración de la configuración, las métricas, la administración de los proyectos, la preventa, la apertura del proyecto, la planificación, el monitoreo, el cierre y el mantenimiento del producto final. Es decir que realiza un trabajo transversal que afecta los diferentes procesos de la organización.

Así como hemos mencionado que hay diversas alternativas para la ubicación del área de calidad en una organización en función de su contexto, algo similar ocurre con las actividades del área. Hay actividades que toda área de calidad poseerá pero puede suceder que este área tenga a su cargo tareas que podrían corresponder a la administración de la configuración o la capacitación, por ejemplo. Idealmente existirá un área para la administración de la configuración y la capacitación, pero si estamos ante una empresa pequeña puede suceder que no contemos con los recursos necesarios y asignemos estas tareas de la mejor forma para la organización.

23.3.4 Recursos

Para llevar a cabo las actividades el área de calidad se vale de diversos recursos. Desde los más básicos, como las planillas de cálculo, hasta sofisticadas herramientas para la administración de métricas.

Entre los elementos básicos podemos comenzar mencionando un activo muy útil y poderoso: el **checklist**, descripto y utilizado en el capítulo 22 a modo de guía para la revisión del documento de especificación de requerimientos. Queremos aclarar al lector que el checklist en este caso se ha ejemplificado para un documento, pero puede ser usado, de igual forma, para revisar un proceso. Esta herramienta proporciona una forma de registro del estado de los datos para su posterior análisis, permite mantener un histórico de la evolución y confirmar las reglas definidas.

Otra planilla utilizada es la de **seguimiento de no conformidades**. Este recurso permite mantener en forma centralizada las observaciones realizadas en las auditorías y hacer un seguimiento de los problemas encontrados hasta su resolución. En la tabla 23.1 se puede ver un ejemplo de la plantilla mencionada, las no conformidades descriptas son aquellas encontradas durante la auditoría del capítulo 22.

Tabla 23.1. Plantilla de seguimiento de no conformidades							
Fecha de alta	Nombre del checklist	Nro. ítem en checklist	Proceso o producto	Estado	Acción	Responsable	Fecha de resolución
30/03/11	CHK_ERS_Cargas_3.0	#6	producto	Abierto	Incorporar una descripción de cada una de las interfaces mencionadas.	Analista	15/04/11

Tabla 23.1. Plantilla de seguimiento de no conformidades

Fecha de alta	Nombre del checklist	Nro. ítem en checklist	Proceso o producto	Estado	Acción	Responsable	Fecha de resolución
30/03/11	CHK_ERS_Cargas_3.0	#7	producto	Abierto	Incorporar una descripción de cada uno de los actores del sistema.	Analista	15/04/11
30/03/11	CHK_ERS_Cargas_3.0	#10	producto	Abierto	Incorporar un listado de los casos de uso y una breve descripción de éstos.	Analista	15/04/11
30/03/11	CHK_ERS_Cargas_3.0	#17	producto	Abierto	Incorporar el origen de cada uno de los requerimientos.	Analista	15/04/11
30/03/11	CHK_ERS_Cargas_3.0	#18	producto	Abierto	Se listarán los requerimientos de hardware y software.	Analista	15/04/11
30/03/11	CHK_ERS_Cargas_3.0	#19	producto	Abierto	Analizar los términos que serían apropiados incluir.	Analista	15/04/11

Además de los elementos mencionados existen otros recursos utilizados para la recolección y medición de métricas relacionadas con la calidad. Por ejemplo, podemos mencionar actas o formularios de seguimiento, diagramas de dispersión, gráficos, histogramas, diagramas de Pareto, diagramas de flujo y de causa y efecto. Estos mecanismos fueron presentados en el capítulo 22.

Dentro de las herramientas de software podemos mencionar las utilizadas para el seguimiento de las no conformidades. Aquí se utilizan las mismas herramientas que para el reporte de errores del software. Muchos de los programas dedicados al reporte de errores tienen opciones para la carga de no conformidades de las auditorías realizadas por el área de calidad.

En relación al tamaño del área de calidad podemos decir que es directamente proporcional a la de la organización, en primer lugar. Asimismo estará relacionado con la estructura de la organización, como hemos mencionado previamente.

23.3.5 Ejemplo de uso

Volviendo sobre el ejemplo guía recordemos que la organización que llevará a cabo el desarrollo del producto para la administración del transporte de cargas no contiene un área de calidad. De hecho, la gerencia ha manifestado que no se considera necesario argumentando que la forma de trabajo es flexible y cómoda

para los empleados. Pero, como se ha mencionado, los desarrolladores, analistas y líderes de proyecto se encuentran con una gran sobrecarga de trabajo realizando múltiples y variadas tareas. Esta situación ha generado un malestar entre los empleados de la organización que ha causado la necesidad de un cambio en la forma de trabajo. El empleado de mayor antigüedad en la empresa y el que tiene una relación fluida con la gerencia, Juan, que recordemos tiene asignado el rol de líder de proyecto, ha resuelto comunicarle a ésta la situación actual. Lo cual ha originado la sorpresa de la gerencia, Enrique, que luego de varios “me cuesta creer lo que estás mencionando” y ante la insistencia del líder, ha tomado el tema para analizarlo más profundamente.

Sin salir del asombro, la gerencia ha repasado los hechos ocurridos en los últimos tiempos: la gran cantidad de problemas reportados por parte de los clientes en aplicaciones productivas, la consecuente interrupción durante el ciclo de vida de los proyectos en progreso. “¿Será necesario que contemos con más recursos?”. ,

Finalmente han optado por monitorear el trabajo diario de los empleados para hallar la mejor solución al contexto actual.

Cuando efectivamente la gerencia entró en contacto con el trabajo diario de los empleados detectaron la coyuntura comentada por Juan, el líder de proyecto, e incluso comprendieron que la solución no era tan simple como la “adquisición de nuevos recursos”.

La gerencia, Pablo y Enrique, se propuso encarar un plan de acción junto con el líder de proyecto que podría retroalimentarlos en forma más concreta y real sobre la situación. En primera instancia se listaron los síntomas para descubrir finalmente las causas reales. Dentro de éstos podemos nombrar el malestar y la sobrecarga de los empleados, el solapamiento de las tareas y la desorganización en la forma de trabajo. El análisis de estas cuestiones en conjunto concluyó en una serie de causas relevantes que requerían una solución: falta de recursos, carencia de una forma de trabajo establecida, ausencia de control sobre las tareas y elementos generados (documentación y productos de software). En relación a cada uno de los puntos mencionados se buscará una solución: han decidido incorporar más recursos a la empresa y han conversado sobre los otros puntos.

Pablo: ¿Cómo piensan que sería bueno solucionar el problema de la ausencia de una forma de trabajo y de los controles?

Juan: Con respecto a la forma de trabajo, podríamos trabajar en conjunto con alguno de los chicos para definirla. Tanto ellos como yo estamos día a día luchando con los problemas y podríamos buscar la forma de optimizar el flujo de trabajo.

Enrique: Me parece una buena idea. Después podrían armar un documento para comunicárselo al resto de los integrantes, ¿no?

Juan: Bueno, yo no sé si eso será lo más adecuado. Quizás sería mejor se pudiera realizar alguna charla o capacitación.

Pablo: Pero me están diciendo que hay muchísimo trabajo, que están sobrecargados y ¿me propones hacer una capacitación que vaya Dios a saber cuánto tiempo llevará? La verdad es que no me parece lo mejor. Tendríamos que analizarlo en detalle cuando tengamos algo en concreto.

Juan: Eso podríamos platicarlo más adelante, pero tengan en cuenta que hay capacitaciones que son importantes para los empleados y más si está explicando cómo se modificará el flujo de trabajo diario de ellos. No es lo mismo que leer un documento, tendríamos que pensar en un espacio para las dudas y otro tipo de cuestiones que puedan suscitarse.

Igual, insisto, podríamos platicarlo más adelante, si les parece.

Enrique: Pues sí, analicémoslo dentro de un tiempo. Concentrémonos en establecer la forma de trabajo... y ¿quién hará los controles que mencionan? la idea sería hacer controles entre ustedes, ¿verdad?

Juan: Aquí entramos en un asunto más complicado. Lo ideal es que la persona que elabora un documento no lo verifique, sino que la validación sea llevada a cabo por otra persona. Lo mismo sucede con el código y las funcionalidades de las aplicaciones. En general la calidad de los productos generados se está deteriorando por la forma de trabajo que estamos adquiriendo debido a la situación.

Yo aquí había pensado en un cambio distinto. Me gustaría crear un área de calidad para definir los procesos adecuados a nuestro contexto desde la forma de trabajo diaria hasta la forma de comunicación con la gerencia, el monitoreo de las tareas...

Pablo: Disculpa que te interrumpa, pero ¿de qué forma piensas que un área de calidad podría ayudarnos? Realmente no veo lo que dices.

Juan: Es un punto de vista distinto al nuestro que nos puede ayudar a mejorar la calidad de los productos internamente y de cara al cliente. Nos ayudará a establecer los procesos, a comunicarlos, a hacer un seguimiento de los proyectos ordenado y mantenerlos informados a ustedes sobre su estado. Podríamos hacerla en forma progresiva la implementación del área.

Enrique: ¿De qué forma nos mantendría informados? Actualmente también nos informan semanalmente el estado de los proyectos. Considero que sería tener más personas en el medio. Ahora podemos manejarnos con libertad. Podríamos asignar las tareas de control de la documentación y de los procesos dentro del proyecto al líder, ¿no les parece mejor?

Juan: Sinceramente no me parece lo mejor. Un área de calidad analiza las métricas que recolecta y a partir de ellas se podrán tomar decisiones para mejorar. Si los líderes de proyecto llevamos adelante nuestros propios controles sucederá que con el afán de querer entregar algo rápidamente no se lleve a cabo el respectivo control. Pasado un tiempo, caeríamos en la misma situación que nos encontramos ahora.

Enrique: Eso dependerá de la conducta del líder de proyecto.

Juan: El problema es que estamos cometiendo el mismo error que si le pedimos a un desarrollador que pruebe su propio código. Los controles tienen que hacerse en forma cruzada para que ocurran conflictos de intereses. Además estamos pensando en incorporar más tareas a las que ya tenemos, cuando hemos estado platicando por la sobrecarga del trabajo.

Pablo: Bueno, en este punto estoy de acuerdo. Podríamos pensar en otra persona. De todas maneras, hay un tema que me preocupa más. Hemos estado recibiendo repetidas quejas de los clientes sobre los

productos que entregamos. Incluso, nuestro cliente más informado nos ha comunicado que no concretará el próximo proyecto hasta tanto no se solucionen los errores de la aplicación que tienen en producción. No podemos perder un proyecto por semejante situación. Hasta me atrevería a decir que puede recurrir a otra empresa de software en el futuro. Lo cual sería aún más grave. Es nuestro mejor cliente, debemos cuidarlo.

Juan: Yo estoy totalmente de acuerdo contigo. Es imprescindible que mejoremos la calidad de nuestros productos. Sobre todo pensemos que el proyecto de administración de cargas será instalado en varios clientes, posiblemente.

Pablo: Así es.

Juan: Pues una vez más les sugiero la creación del área de calidad. Podremos definir procesos para mejorar la calidad en todos los proyectos. Comenzando desde la preventa hasta el mantenimiento del producto y pasando por la elicitación de requerimientos, el diseño, el desarrollo, las pruebas y la fase de aceptación.

Como podrán apreciar son varios los aspectos que estaríamos cubriendo con la creación de esta área. ¿No les parece?

Enrique: Quizás tienes razón. Deberíamos analizar e investigar tu propuesta para entender en detalle las funciones del área. Deberíamos encontrar una forma de implementación que no sea un cambio tan radical para la organización.

Juan: Por supuesto. Hay varias alternativas de implementación.

Pablo: Perfecto, nos quedamos con los temas para estudiar cada uno de ellos. Muchas gracias por tu colaboración y opinión en esta charla.

Juan: No hay problema. Estamos al habla.

Posteriormente al análisis efectuado por la gerencia, se resolvió la creación del área de calidad. Ha sido una decisión difícil, la gerencia no comprendía realmente los beneficios y optó por contratar a una persona que tuviera experiencia en el área para que trabaje con otros en la implementación del área.

Entonces para comenzar con el proceso de implementación del área de calidad cooperará un integrante con poder de decisión del área de desarrollo, que en este caso será el líder de proyecto y el sponsor del proceso para comenzar a definir la planificación.

La planificación deberá tener en cuenta el contexto de la organización. A grandes rasgos los ítems que deberían considerarse se apilan en la figura 23.8.

Análisis de la situación**Asignación de responsabilidades****Definición de la estrategia, alcance del área de calidad**

- En este punto es importante tener en cuenta el cambio organizacional que provocará la incorporación de este área.
- La estrategia deberá estar alineada con los objetivos de la organización.

Definición de los procesos

- La cantidad de procesos a definir quedará determinada por el alcance que se le quiera dar al área.
- Es importante la participación del área de desarrollo.

Capacitación

- Capacitar a las personas que sea necesario, es decir que estén involucradas con el proceso incorporado.
- Comunicar a las personas que sean afectadas de forma indirecta por los procesos incorporados.

Implementación

- El área de calidad guiará la implementación de los procesos incorporados.

Figura 23.8. Hitos principales de la planificación

23.4 Conclusiones

La incorporación de un área de calidad en una organización implica un cambio cultural en la organización que involucra a todos sus miembros. No es una tarea simple, requiere de mucho esfuerzo y es conveniente una implementación progresiva para corregir los desvíos y lograr la mayor cantidad de ventajas posibles.

Ahora bien, hemos platicado que la calidad es responsabilidad de todos los integrantes de la organización. Pero entonces podrían estar preguntándose para qué necesitamos un área de calidad si cada una de las personas hace bien su trabajo, con calidad y no comete errores. Como el lector podrá apreciar, en este caso, no tendría ningún sentido la existencia del área. Asimismo se puede observar que la cantidad de actividades a ser ejecutadas por el área no es muy elevada, lo que refuerza una vez más la duda sobre la existencia. Pero si nos detenemos a pensar un instante que los errores son inherentes al ser humano, las personas pueden poner un gran esmero en hacer las cosas bien, pero no somos infalibles. Estos errores, cometidos por el humano, pueden provocar defectos en el producto generado. Entonces es aquí en donde la presencia del área de calidad cobra sentido y se torna imprescindible.

Por último, queremos remarcar que es frecuente encontrarse con áreas de calidad que no cuentan con la suficiente autoridad y poder para llevar a cabo un cambio importante. Por este motivo es primordial contar con el apoyo de la gerencia, si es que realmente se desea mejorar.

23.5 Contenido de la página web de apoyo

23.5.1 Mapa conceptual

23.5.2 Autoevaluaciones

23.5.3 Presentaciones en Power Point (exclusivo para el docente)

23.6 Referencias



- [1] http://es.wikipedia.org/wiki/Historia_de_la_calidad.
- [2] *Ingeniería de Software. Un enfoque práctico*, Quinta Edición, Roger S. Presuman, McGraw-Hill, 2002.
- [3] *Practical Guide to Software Management*, John W. Horch, Second Edition, Artech House, 2003.
- [4] *Calidad en el desarrollo de Software*, Guillermo Pantaleo, Capítulos 1 y 3. Alfaomega, 2011.
- [5] *Metrics and Models in Software Quality Engineering*, Second Edition, Stephen H. Kan, Addison-Wesley, 2002.
- [6] *Pattern of Agile Practice Adoption. The technical cluster*, Arm Elssamadisy, Forwards by Ron Jeffries and Craig Larman, C4 Media, 2007.
- [7] *Scrum and XP from the Trenches*. How we do Scrum, Henrik Kniberg, C4 Media, 2007.

Planificación de las actividades de control de calidad de un proyecto de desarrollo

Contenido

24.1 Introducción	418
24.2 Plan de calidad	420
24.3 Conclusiones	432
24.4 Contenido de la página web de apoyo	432
24.5 Referencias	432
24.6 Apéndice – Ejemplo del caso testigo	432

Objetivos

- Cómo es el plan de calidad de un proyecto
- Cuáles son los objetivos
- Cuál es el flujo de trabajo
- Cómo aplica al caso testigo

Competencias

- Realizar las tareas asociadas al control de calidad en un proyecto de desarrollo de software

24.1 Introducción

En este capítulo nos centraremos en la planificación de las tareas de control del área de calidad para un proyecto de desarrollo. En el capítulo anterior hemos platicado sobre la implementación del área de calidad en una organización, las pautas a tener en cuenta, los roles dentro del área y los recursos de los cuales se vale para llevar a cabo las tareas. Se supone que el lector ha comprendido el entorno de trabajo y las tareas a cargo del área de calidad y que está en condiciones de adentrarse en los detalles que involucran a ésta dentro de un proyecto de desarrollo.

Asumimos que el programa de calidad llevado adelante por el área de calidad está establecido y en aplicación. ¿Qué aspectos debe cubrir este programa en relación con los proyectos? Como expresamos en el capítulo anterior, los recursos invertidos en el aseguramiento de la calidad dependerán del contexto en el cual se desarrolle el proyecto y de sus características. Sin embargo hay un conjunto esencial de aspectos en el que se deberá trabajar con mayor o menor esfuerzo, estos son [1], [2], [3]:

- Estándares
- Revisiones
- Pruebas
- Análisis de fallas
- Administración de la configuración
- Capacitación
- Administración de proveedores
- Administración de riesgos.

Cada uno de estos aspectos afectará a la calidad de los **procesos y los productos por ellos generados**. Cada uno es importante tanto para construir software con criterios de calidad preestablecidos como para mantenerlos a lo largo del ciclo de vida del proyecto, cada uno es importante por diferentes razones, a saber [1]:

Estándares: el desarrollo de software hoy día, como hemos mencionado en capítulos anteriores, se realiza como una actividad compartida por miembros de un grupo. El producto que se construye está formado por partes que son construidas y compartidas por los desarrolladores, por lo tanto debe establecerse una forma común de elaborar y documentar los artefactos de software. De lo contrario sería imposible intercambiar estas partes para que cada uno le agregue funcionalidad y lo haga crecer. Como todas las ingenierías, la de software ha sistematizado sus mecanismos. Por esta razón se trabaja con estándares de codificación para los diferentes lenguajes de programación, con estándares de especificación y diseño utilizando UML, etc.

Revisiones: como se mencionó en el capítulo en el que se trabajó con la validación y verificación del software, un importante porcentaje de los errores residentes en el software de un proyecto son extraídos a partir de las revisiones de los productos. Por lo tanto, estas revisiones se aplican a la especificación de requerimientos (ERS), de arquitectura (EDA), al diseño detallado (EDD), al código y a los diferentes planes. Consiste en reuniones en las que expertos del proyecto y de afuera expresan su opinión acerca del producto revisado. Con la revisión de la evidencia generada en los proyectos y la ayuda de checklist se revisan los procesos y procedimientos definidos a efectos de detectar y corregir no adherencias o no conformidades a la forma de trabajo.

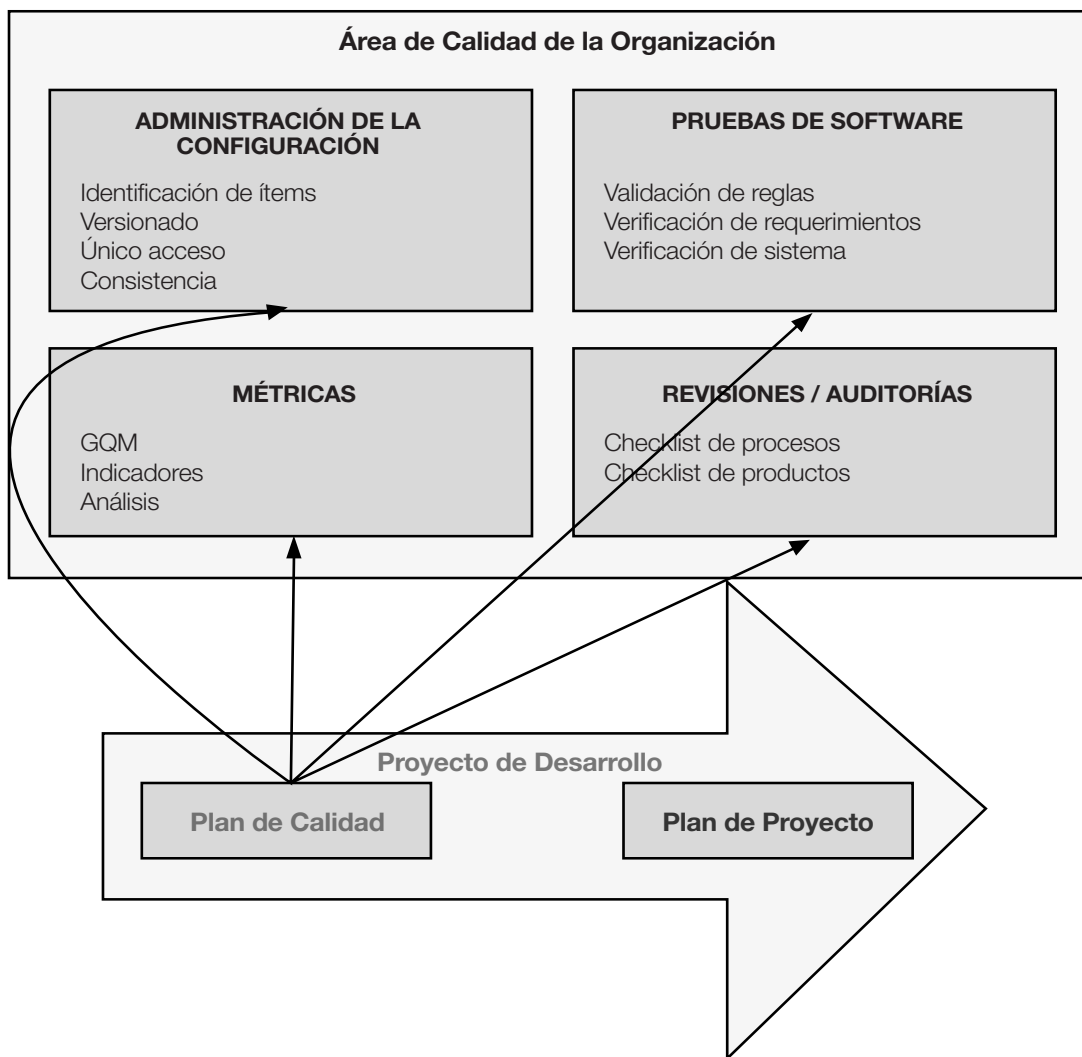


Figura 24.1. Plan de Calidad de un proyecto de desarrollo y su vínculo con el área de calidad de la organización

Pruebas: las pruebas del software son el medio a través del cual se verifica la correcta y completa implementación de los requerimientos. Como vimos en capítulos anteriores con la utilización de pruebas de diferentes niveles se cubren los distintos aspectos del sistema en desarrollo.

Análisis de fallas: este proceso busca dar racionalidad y fijar estrategias de detección, fijación y corrección de los errores. También contribuye a la planificación del retrabajo asociado a su corrección. Si se acompaña con la recolección de métricas asociadas permite planificar acciones de prevención a partir de la implementación de buenas prácticas en el desarrollo.

Administración de la configuración: la ubicación de todos los activos (código fuente, diagramas, planes, etc.) en un repositorio asegura su versionado y una única fuente de provisión para todos los involucrados en los proyectos con la consiguiente consistencia de los productos y activos. De esta forma se cuenta con ítems con historia y además con la garantía de que son los mismos para todo el grupo de proyecto.

Capacitación: los conocimientos necesarios para el trabajo en cualquiera de los aspectos vinculados a las tareas de aseguramiento de la calidad en los proyectos deben impartirse para que todos los involucrados participen desde su rol con la parte que les toca. De esta forma, a través de la capacitación se busca generar un ambiente colaborativo a efectos de lograr los objetivos de calidad definidos para los proyectos.

Administración de proveedores: los proveedores externos que contribuyan, por ejemplo, con el desarrollo de determinados componentes que se integrarán con los productos construidos por el grupo de desarrollo deben ser administrados. La razón directa es que sus productos y procesos impactarán en la calidad de los construidos dentro de la organización. En esta área de procesos deberán administrarse estándares comunes, planificación de tiempos, pruebas de aceptación, etcétera.

Administración de riesgos: los riesgos o posibles problemas que se conviertan en obstáculos para cumplir con los objetivos del proyecto deberán administrarse. Como se expuso en un capítulo anterior cuando se trataron los temas relacionados a la gestión de los proyectos, el área de calidad ayudará en la planificación y el seguimiento de los riesgos auditando su gestión.

En la figura 24.1 se presenta un esquema que muestra cómo el plan de calidad de un proyecto de desarrollo de software se construirá a partir de lo que el proyecto demande y en base a las definiciones realizadas en el establecimiento del área de calidad de la organización.

A continuación, en la próxima sección presentamos las partes que componen el plan de calidad de un proyecto de desarrollo y luego cómo aplica éste al proyecto del caso testigo del libro.

24.2 Plan de calidad

Más allá de la existencia del área de calidad en la organización, es preciso planificar la calidad para cada uno de los proyectos que se inician. Con el fin de organizar las tareas de calidad y las pautas que se tendrán en cuenta durante el proyecto en relación a ésta es que se desarrolla el plan de calidad que se adaptará en función del contexto del proyecto (tipo de proyecto, tamaño y área de negocio) y de la organización. Un modelo genérico de este plan es el siguiente:

1. Propósito

Objetivo del plan.

2. Referencias

Listado de documentos, proyectos, estándares e información adicional de referencia del plan de calidad.

3. Destinatarios

Personas que recibirán el plan de calidad.

4. Administración

En esta sección se describirá la organización, las tareas y responsabilidades de la administración de la calidad de la organización en relación al proyecto en cuestión:

- 4.1 Organización
- 4.2 Roles y Responsabilidades
- 4.3 Tareas.

5. Documentación

En esta sección se describirá la documentación que se elaborará y se utilizará durante cada una de las fases del proyecto en cuestión:

- 5.1 Propósito
- 5.2 Documentación mínima
- 5.3 Especificación de requerimientos
- 5.4 Especificación de diseño
- 5.5 Plan de validación
- 5.6 Reporte de validación
- 5.7 Documentación de usuario
- 5.8 Plan de administración de la configuración
- 5.9 Otra documentación.

6. Estándares, prácticas, convenciones y métricas

En esta sección se identifican los estándares, las métricas e indicadores junto con su interpretación a utilizar en el proyecto:

- 6.1 Propósito
- 6.2 Documentación, convenciones, estándares
- 6.3 Estándares de pruebas, convenciones y prácticas
- 6.4 Métricas.

7. Revisiones y auditorías

En esta sección se establecen todos los productos y procesos a ser auditados. Asimismo se define el momento en el cual se realizarán las revisiones:

- 7.1 Propósito
- 7.2 Requerimientos mínimos
 - 7.2.1 Revisión de los requerimientos
 - 7.2.2 Revisión del diseño
 - 7.2.3 Revisión de la validación del software
 - 7.2.4 Auditoría funcional
 - 7.2.5 Revisión del plan de administración de la configuración
 - 7.2.6 Postmortem
- 7.3 Otras auditorías y revisiones.

8. Pruebas

En esta sección se hace referencia a las pruebas de software. Es común que se elabore un plan de pruebas y aquí se haga la referencia a dicho plan.

9. Reportes y plan de acción

En esta sección se establece la forma de trabajo en relación a los reportes y los planes de acción:

9.1 Forma de trabajo

9.2 Responsabilidades.

10. Herramientas, técnicas y metodologías

En esta sección se mencionan las herramientas, técnicas y metodologías utilizadas para llevar a cabo las tareas asociadas al plan de calidad.

11. Capacitación

En esta sección se hace referencia a las capacitaciones a ser desarrolladas en relación a la calidad para el proyecto en cuestión.

12. Administración de riesgos

En esta sección se hace mención de los riesgos del proyecto en relación a la calidad.

El listado mencionado como modelo del Plan de Calidad ha sido adaptado del libro *Practical Guide to Software Quality Management*, de John W. Horch [1].

24.2.1 Ejemplo del caso testigo

A continuación mostraremos un ejemplo del plan de calidad para el proyecto de “Administración de Cargas” que ha sido de guía a lo largo de todo el libro.

Para la elaboración del plan se han tomado las siguientes consideraciones:

- El proyecto evolucionará utilizando una metodología de desarrollo tradicional guiada por los planes.
- La metodología será iterativa e increment

Plan de calidad
Proyecto: Administración de Cargas
Cliente: Empresa Cliente

Propósito

El presente plan incluye las pautas que se van a emplear en la totalidad de las fases del ciclo de vida del software que se desarrolla en el proyecto “Administración de Cargas”. Se considera desde la fase de concepción hasta la fase de transferencia. El propósito es asegurar que los procesos y productos que se listan a continuación satisfagan la definición del proceso de desarrollo.

- Proceso Fase de Concepción
- Proceso Fase de Elaboración
- Proceso Fase de Construcción
- Proceso Fase de Transferencia
- Proceso de Iteración
- Proceso Administrar Cambios de Requerimientos
- Proceso Administrar Configuración
- Proceso Planificación
- Proceso Implementación de Medidas y Análisis
- Especificación de Requerimientos (ERS)
- Plan de Proyecto (PP)
- Plan de Iteración (PI)
- Plan de Administración de la Configuración (PSCM)
- Plan de la Calidad (PQA)
- Plan de pruebas (PT)
- Análisis de Impacto de Cambios de Requerimientos (AI)

Referencias

Los documentos relacionados con el presente plan se listan a continuación:

- Plan de Pruebas (PT)
- Plan de Administración de la configuración (PSCM)
- Plan de Proyecto (PP)
- Plan de Iteración (PI)

Destinatarios

Los destinatarios del presente documento son:

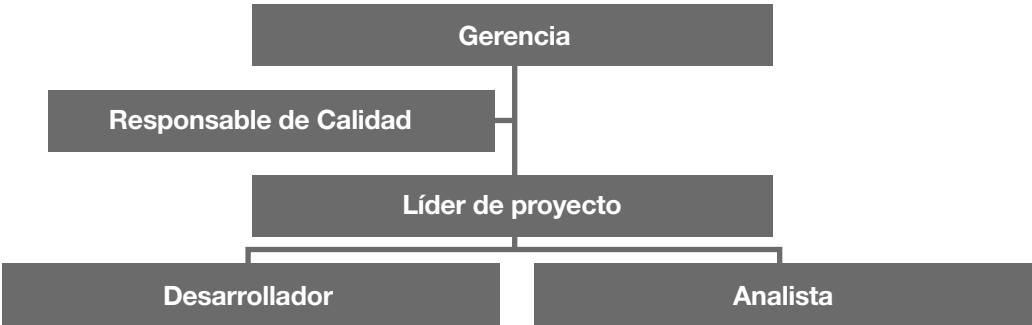
- El líder de proyecto
- El responsable del área de calidad

Administración

A continuación se describirá la organización, las tareas y responsabilidades de la administración de la calidad dentro de la organización y en particular para el proyecto “Administración de Cargas”.

Organización

A continuación se presenta el organigrama de la organización.



Roles y Responsabilidades

A continuación se identifican y asignan las responsabilidades asignadas a cada miembro del proyecto en relación a la calidad.

Rol	Nombre	Responsabilidades
Líder de proyecto	Juan	- Mantener actualizada la información y las medidas definidas para el proyecto. - Escalar las no conformidades detectadas en los recursos con el proyecto.
Analista	Mary	- Elaborar y ejecutar los test funcionales y aprobar el release previa liberación al cliente. - Ser criterioso para decidir si una funcionalidad está en condiciones de ser liberada al cliente, ya sea en test o en producción.
Desarrollador	Germán	- Cumplimiento de la calidad interna pactada del producto generado. - Elaborar y ejecutar los test unitarios.
Miembro del área de calidad	Carla	- Asegurar el cumplimiento de los procesos y productos establecidos. - Informar la ausencia de solución de las no conformidades y la falta de ejecución de acciones correctivas a la gerencia. - Escalar las no conformidades detectadas en los recursos con el proyecto a la gerencia.

Tareas

A continuación se listan las tareas del área de calidad para cada una de la/s iteración/es de las etapas del ciclo de vida del proyecto “Administración de Cargas”.

Etapa/Iteración	Responsable	Criterio de entrada
Concepción - Producto		
Ejecutar el checklist del plan de proyecto	Líder de proyecto / Miembro de calidad	En la etapa promedio y/o al finalizar el plan y previo a comprometerlo o entregarlo al cliente
Corregir las observaciones encontradas en el plan del proyecto.	Líder de proyecto	A partir del resultado del checklist
Ejecutar el checklist del plan de iteración	Líder de proyecto / Miembro de calidad	Al promediar y/o finalizar el plan y previo a comprometerlo o entregarlo al cliente
Corregir las observaciones encontradas en el plan de iteración.	Líder de proyecto	A partir del resultado del checklist
Ejecutar el checklist del plan de calidad	Líder de proyecto / Miembro de calidad	Al promediar y/o finalizar el plan y previo a comprometerlo o entregarlo al cliente
Corregir las observaciones encontradas en el Plan de calidad.	Líder de proyecto	A partir del resultado del checklist.
Elaborar el plan de prueba	Líder de proyecto	A partir de los casos de uso definidos
Ejecutar el checklist del plan de administración de la configuración	Líder de proyecto / Miembro de calidad	Al promediar y/o finalizar el plan y previo a comprometerlo o entregarlo al cliente
Corregir las observaciones encontradas en el plan de pruebas.	Líder de proyecto	A partir del resultado del checklist.
Ejecutar el checklist del plan de administración de la configuración	Líder de proyecto / Desarrollador/ Miembro de calidad	Al promediar y/o finalizar el plan y previo a comprometerlo o entregarlo al cliente
Corregir las observaciones encontradas en el plan de administración de la configuración	Líder de proyecto / Desarrollador	A partir del resultado del checklist
Ejecutar el checklist de la especificación de requerimientos	Líder de proyecto /Analista/ Miembro de calidad	Al promediar y/o finalizar la especificación de requerimientos y previo a comprometerla o entregarla al cliente

Etapa/Iteración	Responsable	Criterio de entrada
Corregir las observaciones encontradas en la especificación de requerimientos	Líder de proyecto / Analista	A partir del resultado del checklist
Ejecutar el checklist del análisis de impacto de requerimientos	Líder de proyecto / Analista / Miembro de calidad	De existir, al promediar y/o finalizar el análisis de impacto de requerimientos y previo a comprometerlo o entregarlo al cliente
Corregir las observaciones encontradas en el análisis de impacto de los requerimientos	Líder de proyecto / Analista	A partir del resultado del checklist
Ejecutar el checklist del análisis de impacto de la configuración	Líder de proyecto / Miembro de calidad	De existir, al promediar y/o finalizar el análisis de impacto de requerimientos y previo a comprometerlo o entregarlo al cliente
Concepción - Procesos		
Ejecutar el checklist de la Iteración	Líder de proyecto / Miembro de calidad	Al finalizar la iteración
Corregir las observaciones encontradas en la iteración	Líder de proyecto	A partir del resultado del checklist
Ejecutar el checklist del proceso de administración de la configuración	Líder de proyecto / Miembro de calidad	Al promediar la fase y antes de finalizarla
Corregir las observaciones encontradas en el proceso de administración de la configuración	Líder de proyecto	A partir del resultado del checklist
Ejecutar el checklist del proceso de administrar cambios de requerimientos	Líder de proyecto / Miembro de calidad	Al promediar la fase y previo a finalizarla
Corregir las observaciones encontradas en el proceso de administrar cambios de requerimientos	Líder de proyecto	A partir del resultado del checklist
Ejecutar el checklist del proceso implementar medidas	Líder de proyecto / Miembro de calidad	Al término de la iteración
Corregir las observaciones encontradas en el procedimiento de medidas	Líder de proyecto	A partir del resultado del checklist

Etapa/Iteración	Responsable	Criterio de entrada
Ejecutar el checklist de la fase	Líder de proyecto / Miembro de calidad	Antes del término de la fase
Corregir las observaciones encontradas en la fase	Líder de proyecto	A partir del resultado del checklist
Ejecutar el checklist del proceso de planificación	Líder de proyecto / Miembro de calidad	Al concluir la iteración
Corregir las observaciones encontradas en el proceso de planificación	Líder de proyecto	A partir del resultado del checklist
Ejecutar el checklist del proceso administrar cambios de configuración	Líder de proyecto / Miembro de calidad	Al promediar la fase y previo a su término
Corregir las observaciones encontradas en el proceso administrar cambios de configuración	Líder de proyecto	A partir del resultado del checklist

 Esto debería completarse para cada una de las fases del proceso de desarrollo: elaboración, construcción y transferencia. Aquí sólo se ha completado una fase a modo de ejemplo para el lector.

Estándares

En esta sección se identifican los estándares a utilizar durante el proyecto “Administración de Cargas”. Asimismo se especifican los checklist de productos y de procesos a emplear en las auditorías.

Estándar

Los estándares y las convenciones a utilizar en el presente proyecto son los siguientes:

Estándar	Descripción
Documentos	
DOC001_PlanProyecto	Plan de proyecto
DOC002_Estimaciones	Planilla de estimaciones
DOC003_Cronograma_Concepción DOC004_Cronograma_Elaboración DOC005_Cronograma_Construcción DOC006_Cronograma_Transferencia	Cronogramas de iteración de acuerdo a la etapa a la que pertenece
DOC007_PlanPruebas	Plan de pruebas

Estándar	Descripción
DOC008_Planificación	Planilla de planificación
DOC009_PlanIteración	Plan de iteración
DOC010_ReporteAvanceSemanal	Reporte de avance semanal
DOC011_MinutaReunión	Minuta de reunión
DOC012_Visión	Documento de visión
DOC013_TablaGestión	Tabla de gestión
DOC014_Glosario	Glosario
DOC015_AnálisisImpactoConfiguración	Análisis de impacto de cambios de configuración
DOC016_PorcentajeAdherencia	Planilla de porcentaje de adherencia a productos y procesos
DOC017_PropuestaComercial	Propuesta comercial utilizada en los cambios de alcance
DOC018_EvaluacionIteración	Evaluación de fin de iteración
DOC019_MinutaSeguimiento	Minuta de seguimiento
DOC020_CierreProyecto	Cierre de proyecto
DOC021_IdentificacionItemsConfiguración	Planilla de identificación de ítems de configuración
DOC022_AnálisisMedidas	Planilla de análisis de medidas
DOC023_ERS	Especificación de requerimientos de software
DOC024_EspecificacionCasoDeUso	Especificación de casos de uso
DOC025_AnálisisImpactoRequerimiento	Análisis impacto de cambio de requerimiento
DOC026_PlanCM	Plan de Administración de la configuración
DOC027_ReporteAuditoriaCM	Reporte de auditoría de la administración de la configuración
DOC028_ReporteEstadoConfiguración	Reporte del estado de la configuración
DOC029_PlanQA	Plan de calidad
Checklist de proceso	
CHK001_FaseConcepción	Checklist de la fase de concepción
CHK002_FaseElaboración	Checklist de la fase de elaboración
CHK003_FaseConstrucción	Checklist de la fase de construcción
CHK004_FaseTransferencia	Checklist de la fase de transferencia

Estándar	Descripción
CHK005_AdministrarConfiguración	Checklist del proceso administrar configuración
CHK006_ImplementacionMedidas	Checklist de la implementación de medidas
CHK007_Iteracion	Checklist de la iteración
CHK008_AdministrarCambiosRequerimientos	Checklist del proceso administrar cambios de requerimientos
CHK009_AdministrarCambiosConfiguración	Checklist del proceso administrar cambios de configuración
CHK010_Planificación	Checklist del proceso planificación
Checklist de producto	
CHK011_ERS	Checklist de la especificación de requerimientos de software
CHK012_PlanProyecto	
Checklist del plan de proyecto	
CHK013_PlanQA	Checklist del plan de calidad
CHK014_PlanCM	Checklist del plan de administración de la configuración
CHK015_PlanIteracion	Checklist del plan de iteración
CHK016_AnalisisImpactoRequerimiento	Checklist del análisis de impacto de cambios de requerimientos
CHK017_AnalisisImpactoConfiguración	Checklist del análisis de impacto de cambios de configuración
CHK018_PlanPruebas	Checklist del plan de pruebas

Revisiones y auditorías

En esta sección se listan los productos y procesos a auditar.

Productos a auditar

Los productos a auditar son los siguientes:

- ERS
- Plan de proyecto
- Plan de calidad
- Plan de iteración
- Plan de pruebas
- Plan de administración de la configuración

- Documento de análisis de impacto de cambios en los requerimientos.

Procesos a auditar

Los procesos a auditar se listan a continuación:

- Fase de concepción
- Fase de elaboración
- Fase de construcción
- Fase de transferencia
- Administración de la configuración
- Implementación de medidas
- Administración de cambios de requerimientos
- Planificación.

Pruebas

Las tareas que serán ejecutadas durante el proyecto en relación a las pruebas se encuentran detalladas en el plan de pruebas del proyecto “Administración de Cargas” que se menciona dentro de las referencias.

Reportes y plan de acción

Las revisiones y auditorías ejecutadas a lo largo del proyecto serán registradas y documentadas adecuadamente por medio de los artefactos mencionados en las secciones anteriores. Los resultados de las auditorías serán plasmados en reportes y comunicados a todos los involucrados. En función de los resultados de las auditorías se planificarán acciones correctivas orientadas a la mejora de los productos y procesos.

Forma de trabajo

En una primera instancia el responsable de calidad ejecuta la auditoria y detecta no conformidades, acciones correctivas y determina una fecha límite para su resolución.

Posteriormente, el resultado de las revisiones es enviado por el responsable de calidad al responsable del objeto de revisión (Líder de Proyecto / Desarrollador o Analista Funcional de acuerdo al caso) y al Líder de Proyecto.

En una segunda instancia el responsable de calidad verifica la resolución de las no conformidades. Luego el resultado de la segunda revisión se remite al responsable del objeto de revisión y al Líder de Proyecto. En caso de existir no conformidades sin solucionar el responsable de calidad las eleva a la gerencia.

Plan de acción

El responsable de calidad definirá en primera instancia las acciones a tomar por cada una de las no conformidades halladas. Pero puede suceder que el responsable del objeto en cuestión no esté de acuerdo con estas acciones o con la corrección. La situación de conflicto se resolverá de la siguiente manera:

- En una primera instancia, el responsable de calidad y el responsable del objeto platicarán con el fin de llegar a un acuerdo.
- En una segunda instancia, el responsable del objeto elevará el problema a su nivel inmediato superior de acuerdo al organigrama definido previamente. Luego la persona inmediata superior platicará con el responsable de calidad con el fin de arribar a un acuerdo.
- En última instancia, el nivel superior en el organigrama, la gerencia en este caso, resolverá el conflicto suscitado.

Administración de riesgos

Los riesgos en relación a las tareas de calidad serán listados en la planilla de riesgos que se encuentra dentro del plan de proyecto de “Administración de Cargas”. Asimismo se mencionará el plan de mitigación y contingencia.

24.2.2 Objetivos

Como hemos visto en el apartado previo, es preciso planificar la calidad para los proyectos de la organización. La planificación de las tareas de control de calidad para un proyecto nos asegurará la validación de la documentación generada, de los procesos ejecutados y del producto producido.

Con la validación de la documentación se genera un ambiente de confianza debido a que han sido elaborados todos los documentos necesarios para el proyecto y éstos son correctos.

Con la validación de los procesos nos aseguramos haber trabajado de la forma que había sido definida a partir del análisis orientado a elaborar productos de la calidad buscada.

Por último con la validación de los productos nos aseguramos que el producto que será entregado al cliente tenga calidad interna y externa.

24.2.3 Flujo de trabajo

Desde la fase de preventa el área de calidad se encuentra involucrada asegurando que los procesos definidos se lleven adelante de acuerdo a lo establecido.

En una fase posterior, cuando el proyecto es aprobado y comienzan las tareas de apertura el área de calidad participa de la reunión interna de iniciación del proyecto. Durante el proceso de planificación se elabora el plan de calidad del proyecto teniendo en cuenta su contexto. El plan de calidad está sincronizado y es consistente con el plan de proyecto. Asimismo, para elaborar la planificación del proyecto deberán incluirse las tareas de control del área de calidad. Dado que el área de calidad tiene incumbencia en todos los proyectos de la organización, es oportuno coordinar con los miembros del área de calidad las tareas relacionadas. De esta manera, nos aseguramos la disponibilidad de los recursos cuando sean necesarios.

A lo largo del proyecto, el área de calidad ejecuta las revisiones a los documentos generados y a los procesos ejecutados y en ejecución. En caso de detectar algún tipo de disconformidad, deberá ser registrada en la

herramienta correspondiente, reportada al área adecuada (puede ser una gerencia de proyectos o la gerencia de la organización) y monitoreada hasta su resolución.

En el apéndice de este capítulo se presenta un listado de las tareas de calidad asociadas a las del proyecto para el caso testigo del libro.

24.3 Conclusiones

Para obtener productos con calidad es necesario considerar las tareas pertinentes al área desde el inicio del proyecto. La calidad tiene que ser planificada para que puedan desarrollarse las tareas correspondientes con los recursos y la infraestructura necesaria y de manera sincronizada con la del plan del proyecto. Es preciso tener en cuenta el contexto del proyecto para en función de ello seleccionar las tareas a ser llevadas a cabo por el área de calidad. Las principales que se deberán considerar son las relacionadas con las auditorías y revisiones, las pruebas y la capacitación.

24.4 Contenido de la página web de apoyo



24.4.1 Mapa conceptual

24.4.2 Autoevaluaciones

24.4.3 Presentaciones en Power Point (exclusivo para el docente)

24.5 Referencias

- [1] *Practical Guide to Software Quality Management*, Second Edition, John W. Horch, Artech House, 2003.
- [2] *Ingeniería de Software. Un enfoque práctico*, Quinta Edición, Roger S. Pressman, McGraw-Hill, 2002.
- [3] *Software Quality Assurance Plan*, IEEE Standard, 1995.

24.6 Apéndice – Ejemplo del caso testigo

Presentamos a continuación la planificación de las tareas del área de calidad dentro de una planificación para el proyecto “Administrador de

Id		Tarea	Responsable
Primera Iteración			
Fase de Concepción			
001	Kick off interno		Líder de proyecto
002	Kick off con el cliente		Líder de proyecto
003	Elaborar plan de iteración		Líder de proyecto
004	Ejecutar checklist plan de iteración		Responsable de calidad
005	Correcciones por la ejecución del checklist del plan de iteración (***)		Líder de proyecto
006	Elaborar plan de proyecto		Líder de proyecto
007	Ejecutar checklist plan de proyecto		Responsable de calidad
008	Correcciones por la ejecución del checklist del plan de proyecto (***)		Líder de proyecto
009	Elaborar plan de pruebas		Líder de proyecto
010	Ejecutar checklist plan de pruebas		Responsable de calidad
011	Correcciones por la ejecución del checklist del plan de pruebas (***)		Líder de proyecto
012	Elaborar plan de calidad		Líder de proyecto
013	Ejecutar checklist plan de calidad		Responsable de calidad
014	Correcciones por la ejecución del checklist del plan de calidad (***)		Líder de proyecto
015	Elaborar plan de administración de la calidad		Líder de proyecto
016	Ejecutar checklist del plan de administración de la configuración		Responsable de calidad
017	Correcciones por el checklist del plan de configuración (***)		Líder de proyecto
018	Elaborar documento de especificación de requerimientos		Analista
019	Ejecutar el checklist del documento de especificación de requerimientos		Responsable de calidad
020	Correcciones por el checklist del documento de requerimientos (***)		Analista
021	Elaborar documento de análisis de impacto de requerimientos		Líder de proyecto
022	Ejecutar checklist del documento de análisis de impacto de requerimientos		Responsable de calidad
023	Correcciones por el checklist del documento de análisis de impacto de requerimiento (***)		Líder de proyecto
024	Actualizar planificación por los cambios en los requerimientos		Líder de proyecto
025	Ejecutar checklist del proceso de administración de cambio de requerimientos		Responsable de calidad
026	Correcciones por el checklist del proceso de administración de cambio de requerimientos (***)		Analista

Id	Tarea	Responsable
027	Elaborar documento de análisis de impacto de la configuración	Líder de proyecto
028	Ejecutar el checklist del documento de análisis de impacto de la configuración	Responsable de calidad
029	Correcciones por el checklist del documento de análisis de impacto de la configuración (***)	Líder de proyecto
030	Actualizar planificación por el impacto de los cambios en la configuración	Líder de proyecto
031	Ejecutar checklist de implementar medidas	Responsable de calidad
032	Correcciones por el checklist de implementar medidas (***)	Líder de proyecto
033	Ejecutar checklist de planificación	Responsable de calidad
034	Correcciones por el checklist de planificación (***)	Líder de proyecto
035	Ejecutar el checklist del proceso administrar requerimientos	Responsable de calidad
036	Correcciones por el checklist del proceso de administrar requerimientos (***)	Analista
037	Ejecutar checklist del proceso de administración cambios de la configuración	Responsable de calidad
038	Correcciones por el checklist del proceso de administración de cambios de la configuración (***)	Líder de proyecto
039	Ejecutar checklist de la fase	Responsable de calidad
040	Correcciones por el checklist de la fase (***)	Líder de proyecto
Fase de Elaboración		
041	Actualización del plan de proyecto (*)	Líder de proyecto
042	Ejecutar el checklist del plan de proyecto	Responsable de calidad
043	Correcciones del checklist del plan de proyecto (***)	Líder de proyecto
044	Actualizar plan de la iteración (*)	Líder de proyecto
045	Ejecutar checklist del plan (**)	Responsable de calidad
046	Correcciones del checklist del plan (***)	Líder de proyecto
047	Actualizar el plan de calidad (*)	Líder de proyecto/ Responsable de calidad
048	Ejecutar checklist del plan (**)	Responsable de calidad
049	Correcciones del checklist del plan (***)	Líder de proyecto/ Responsable de calidad
050	Actualizar el plan de administración de la configuración (*)	Líder de proyecto
051	Ejecutar el checklist del plan de administración de la configuración (**)	Responsable de calidad

Id	Tarea	Responsable
052	Correcciones por el checklist del plan de administración de la configuración (***)	Líder de proyecto
053	Actualizar la especificación de los requerimientos (*)	Analista
054	Ejecutar checklist del documento de la especificación de los requerimientos (**)	Responsable de calidad
055	Correcciones por el checklist del documento de especificación de requerimientos (***)	Analista
056	Elaborar casos de uso	Analista
057	Elaborar documento de análisis de impacto de cambio de requerimientos (*)	Líder de proyecto
058	Ejecutar checklist del documento de análisis de impacto de cambio de requerimientos (**)	Responsable de calidad
059	Correcciones por el checklist del documento de análisis de impacto de cambio de requerimientos (***)	Líder de proyecto
060	Ejecutar checklist del proceso de administración de cambio de requerimientos (**)	Responsable de calidad
061	Correcciones por el checklist del proceso de administración de requerimientos (***)	Analista
062	Elaborar documento de análisis de impacto de cambio de configuración	Líder de proyecto
063	Ejecutar checklist del documento de análisis de impacto de cambio de configuración (**)	Responsable de calidad
064	Correcciones por el checklist del documento de análisis de impacto de cambio de configuración (***)	Líder de proyecto
065	Ejecutar checklist de la administración de la configuración (**)	Responsable de calidad
066	Correcciones por el checklist de la administración de cambio de la configuración (***)	Líder de proyecto
067	Ejecutar checklist de la implementación de las medidas (**)	Responsable de calidad
068	Correcciones por el checklist de la implementación de las medidas (***)	Líder de proyecto
069	Ejecutar checklist de la planificación (**)	Responsable de calidad
070	Correcciones por el checklist de la planificación (***)	Líder de proyecto
071	Ejecutar checklist de la fase (**)	Responsable de calidad
072	Correcciones por el checklist de la fase (***)	Líder de proyecto
Fase de Construcción		
073	Actualizar plan de proyecto(*)	Líder de proyecto
074	Ejecutar checklist del plan de proyecto (**)	Responsable de calidad
075	Correcciones por ejecutar el checklist del plan de proyecto (***)	Líder de proyecto

Id	Tarea	Responsable
076	Actualizar plan de iteración (*)	Líder de proyecto
077	Ejecutar checklist del plan de la iteración (**)	Responsable de calidad
078	Correcciones por el checklist del plan de la iteración (***)	Líder de proyecto
079	Actualizar plan de calidad (*)	Líder de proyecto / responsable de calidad
080	Ejecutar el checklist del plan de calidad (**)	Responsable de calidad
081	Correcciones por el checklist del plan de calidad (***)	Líder de proyecto
082	Actualizar plan de administración de la configuración (*)	Líder de proyecto
083	Ejecutar el checklist del plan de administración de la configuración (**)	Responsable de calidad
084	Correcciones por el checklist del plan de administración de la configuración (***)	Líder de proyecto
085	Actualizar documento de especificación de requerimientos (*)	Analista
086	Ejecutar el checklist del documento de especificación de requerimientos (**)	Responsable de calidad
087	Correcciones por el checklist del documento de especificación de requerimientos (***)	Analista
088	Elaborar documento de análisis de impacto de cambio de requerimientos (*)	Líder de proyecto
089	Ejecutar el checklist del documento de análisis de impacto de cambio de requerimientos (**)	Responsable de calidad
090	Correcciones por el checklist del documento de análisis de impacto de cambio de requerimientos (***)	Líder de proyecto
091	Ejecutar checklist del impacto de cambio de requerimientos (**)	Responsable de calidad
092	Correcciones por el checklist del impacto de cambio de requerimientos (***)	Líder de proyecto
093	Elaborar documento de análisis de impacto de cambio de configuración (*)	Líder de proyecto
094	Ejecutar el checklist del documento de análisis de impacto de cambio de configuración (**)	Responsable de calidad
095	Correcciones por el checklist del documento de análisis de impacto de cambio de configuración (***)	Líder de proyecto
096	Ejecutar checklist del impacto de cambio de la configuración (**)	Responsable de calidad
097	Correcciones por el checklist del impacto de cambio de la configuración (***)	Líder de proyecto
098	Ejecutar checklist de la implementación de las medidas (**)	Responsable de calidad
099	Correcciones por el checklist de la implementación de las medidas (***)	Líder de proyecto
100	Ejecutar checklist de la planificación (**)	Responsable de calidad

Id	Tarea	Responsable
101	Correcciones por el checklist de la planificación (***)	Líder de proyecto
102	Ejecutar checklist de la fase (**)	Responsable de calidad
103	Correcciones por el checklist de la fase (***)	Líder de proyecto
Fase Transferencia		
104	Tareas de transferencia al cliente	Líder de proyecto
105	Ejecutar checklist de las tareas de transferencia (**)	Responsable de calidad
106	Correcciones por el checklist de las tareas de transferencia (***)	Líder de proyecto
107	Elaborar documento de análisis de cambio de requerimientos (*)	Analista
108	Ejecutar el checklist del documento de análisis de cambio de requerimientos (**)	Responsable de calidad
109	Correcciones por el checklist del documento de análisis de cambio de requerimientos (***)	Analista
110	Ejecutar checklist del proceso de análisis de cambio de requerimientos (**)	Responsable de calidad
111	Correcciones por el checklist del proceso de análisis de cambio de requerimientos (***)	Analista
112	Elaborar documento de análisis de impacto de cambio de la configuración (*)	Líder de proyecto
113	Ejecutar el checklist del documento de análisis de impacto de cambio de la configuración (**)	Responsable de calidad
114	Correcciones por el checklist del documento de análisis de impacto de cambio de la configuración (***)	Líder de proyecto
115	Ejecutar checklist del proceso de análisis de impacto de cambio de la configuración (**)	Responsable de calidad
116	Correcciones por el checklist del proceso de análisis de impacto de cambio de la configuración (***)	Líder de proyecto
117	Ejecutar checklist de la implementación de las medidas (**)	Responsable de calidad
118	Correcciones por el checklist de la implementación de las medidas (***)	Líder de proyecto
119	Ejecutar checklist del proceso de planificación (**)	Responsable de calidad
120	Correcciones por el checklist del proceso de planificación (***)	Líder de proyecto
121	Ejecutar checklist de la fase (**)	Responsable de calidad
122	Correcciones por el checklist de la fase (***)	Líder de proyecto
123	Ejecutar checklist de la iteración	Responsable de calidad
124	Correcciones por el checklist de la iteración	Líder de proyecto

(*) Si el plan acordado cambia por replanificaciones

(**) Si el plan acordado cambia por replanificaciones

(***) Si existen no conformidades para ser solucionadas

Las tareas mencionadas se han efectuado sólo para una iteración, pero el lector deberá tener en cuenta que si el proyecto cuenta con 3 iteraciones esto deberá realizarse por cada una de ellas. Aquí tampoco se ha tenido en cuenta que hay muchas tareas que pueden hacerse en paralelo. Pero esto deberá ser considerado en función del contexto.

Estándares de calidad de software

Contenido

25.1 Introducción	440
25.2 Normas y modelos de referencia	440
25.3 Norma ISO 9001-2000	444
25.4 Modelo CMMi	446
25.5 Enfoque y alcance de aplicación	448
25.6 Conclusiones	448
25.7 Contenido de la página web de apoyo	450
25.8 Referencias	450

Objetivos

- Qué son las normas y modelos de referencia
- ISO 9001- 2000
- Modelo CMMI
- Cuáles son los enfoques y el alcance de la aplicación

Competencias

- Desarrollar criterios de selección de modelos de calidad a utilizar en organizaciones dedicadas al desarrollo de software

25.1 Introducción

Las normas y los modelos de referencia han surgido al comprender la creciente expansión del desarrollo del software y entender la necesidad de establecer estándares y buenas prácticas que puedan ser aplicadas en las organizaciones que desarrollen esta actividad.

En un principio se pensaba el desarrollo de software como otro desarrollo industrial, como ser la fabricación de autos, por ejemplo. Con el correr de los años se han comprendido las diferencias entre los procesos y se han ajustado los modelos acorde a las necesidades. El desarrollo de software es un producto intelectual, lejos de poder ser interpretado como parte de una producción en serie. Por lo tanto cuando se intentan aplicar los mismos conceptos se fracasa.

En este capítulo intentaremos transmitirle al lector los conceptos relevantes de una serie de normas y modelos de referencia del software [2], [3]. Si el lector lo desea puede acudir a las fuentes para mayor detalle al respecto.

25.2 Normas y modelos de referencia

A continuación mencionaremos los modelos de referencia y las principales normas y haremos un breve repaso de los aspectos importantes de cada una de ellas. En los siguientes apartados entraremos en más detalle sobre aquellas normas y modelos que consideramos más enfocados al desarrollo de software.

- **CMM** (Capability Maturity Model), 1993
 - **Organización:** SEI (Software Engineering Institute)
 - **Información:** <http://www.sei.cmu.edu>
 - **Estructura**
 - **Niveles**
 1. Inicial
 2. Repetible
 3. Definido
 4. Gestionado
 5. Optimizado
 - **Áreas de proceso**
- **CMMi** (Capability Maturity Model Integration), 2001
 - **Organización:** SEI (Software Engineering Institute)
 - **Información:** <http://www.sei.cmu.edu>
 - **Estructura**
 - **Vistas por Niveles y Capacidades**
 - **Áreas de proceso**
 - **Grupos de Áreas de Procesos**
 - Ingeniería (7 áreas de procesos)
 - Proyectos (5 áreas de procesos)
 - Organizativas (6 áreas de procesos)

- **TSP** (Team Software Process)
 - **Organización:** SEI (Software Engineering Institute)
 - **Información:** <http://www.sei.cmu.edu/library/abstracts/books/201731134.cfm>

- **ISO/IEC 15504** – Information Technology – Process Assessment, 1998 –2006
 - **Organización:** ISO (International Standard Organization)
 - **Información:** <http://www.isospice.com/>
 - **Características salientes**
 - Establece un marco y los requisitos para cualquier proceso de evaluación de procesos y proporciona requisitos para los modelos de evaluación a ser utilizados.
 - Proporciona también requisitos para cualquier modelo de evaluación de organizaciones.
 - Proporciona guías para la definición de las competencias de un evaluador de procesos.
 - Actualmente tiene 10 partes: 1-7 completadas y 8-10 en fase de desarrollo.
 - Comprende: evaluación de procesos, mejora de procesos, determinación de capacidad.
 - Proporciona en su parte 5 un modelo de evaluación de procesos para los procesos de ciclo de vida del software fijados en el estándar ISO/IEC 12207 que define los procesos del ciclo de vida del desarrollo, mantenimiento y operación de los sistemas de software.

- **ISO/IEC 12207** Information Technology / Software Life Cycle Processes, 1995-2008
 - **Organización:** ISO (International Standard Organization)
 - **Información:** <http://www.iso.org/iso/home.htm>
 - **Estructura**
 - **Procesos principales**
 - Adquisición
 - Suministro
 - Desarrollo
 - Operación
 - Mantenimiento
 - **Procesos de soporte**
 - Documentación
 - Gestión de la configuración
 - Aseguramiento de calidad
 - Verificación
 - Validación
 - Revisión conjunta
 - Auditoría
 - Resolución de problemas
 - **Procesos de la organización**
 - Gestión
 - Infraestructura
 - Mejora
 - Recursos Humanos

- **PSP** (Personal Software Process), 1995
 - **Organización:** W. Humphrey
 - **Información:** <http://www.sei.cmu.edu/tsp/>

- **Estructura**
 - **Niveles**
 - Nivel 2 - Inicial:
 - Seguimiento y control de proyectos.
 - Planeación de los proyectos.
 - Nivel 3 - Repetible:
 - Revisión entre colegas.
 - Ingeniería del producto de software.
 - Manejo integrado del software.
 - Definición del proceso de software.
 - Foco del proceso de software.
 - Nivel 4 - Definido:
 - Control de calidad.
 - Administración cuantitativa del proyecto.
 - Nivel 5 - Controlado:
 - Administración de los cambios del proceso.
 - Administración del cambio tecnológico.
 - Prevención de defectos.
 - **Fases**
 - PSP0: proceso base, registro de tiempos, registro de errores, estándar de tipo de errores [Proceso personal de arranque].
 - PSP0.1: estándar de codificación, medición de tamaño, propuesta de mejoramiento del proceso (PIP) [Proceso personal de arranque].
 - PSP1: estimación del tiempo, reporte de pruebas [Proceso personal de administración].
 - PSP1.1: planeación de actividades, planeación de tiempos [Proceso personal de administración].
 - PSP2: revisión de codificación, revisión del diseño [Proceso personal de calidad].
 - PSP2.1: formatos de diseño [Proceso personal de calidad].
 - PSP3: desarrollo en ciclos [Proceso cíclico].
- **ITIL** (Information Technology Infrastructure Library), 1989 - 2007
 - **Organización:** United Kingdom's Office of Government Commerce (OGC)
 - **Información:** <http://www.itil-officialsite.com/home/>
 - **Estructura**
 - **Áreas de aplicación (versión 1)**
 - **Administración de Servicios**
 1. Service Support
 2. Service Delivery
 - **Guías Operacionales**
 3. ICT Infrastructure Management
 4. Security Management
 5. The Business Perspective
 6. Application Management
 7. Software Asset Management
 - **Guías de Implementación**
 8. Planning to Implement Service Management

- 9. ITIL Small-Scale Implementation
- **Fases (versión 3)**
 - 1. ITIL Service Strategy
 - 2. ITIL Service Design
 - 3. ITIL Service Transition
 - 4. ITIL Service Operation
 - 5. ITIL Continual Service Improvement

Con el tiempo y por diferentes razones surgieron otros modelos inspirados en los anteriores que buscaron cubrir necesidades puntuales como por ejemplo el MOPROSOFT (Modelo de Procesos para la Industria del Software) concebido en México. Este modelo fue concebido con el objetivo de ser aplicado a las empresas pequeñas y medianas dedicadas al desarrollo de software. Está inspirado en los niveles 2 y 3 del modelo CMM y en las normas ISO/IEC 15504.

Algunos de los criterios con los que fue elaborado este modelo son los siguientes:

- Procesos estratificados según el organigrama típico de este tipo de organizaciones (gerencia alta, gerencia media y operación).
- La gerencia alta tiene como actividades centrales definir las estrategias de la organización y promover las mejoras continuas.
- La gerencia media es la encargada de proveer los recursos a los proyectos y monitorear el cumplimiento de las planificaciones orientadas a cumplir con los objetivos estratégicos.
- La operación es la encargada de llevar adelante los proyectos.
- Los procesos deben ser definidos de forma de mantener entre ellos una relación integradora.
- El proceso de administración de proyectos es atómico.
- La ingeniería de productos es desarrollada con el soporte de otros procesos orientados a garantizar y controlar su calidad (verificación, validación, documentación y control de la documentación).
- Se le asigna especial atención a la administración de los recursos que aportan al conocimiento de la organización: productos generados por proyectos, datos de los proyectos, mediciones, documentación de procesos y datos relevados de su implementación en los proyectos así como lecciones aprendidas.
- **MOPROSOFT** (Modelo de Procesos para la Industria del Software), 2005
 - **Organización:** Asociación Mexicana para la Calidad en Ingeniería de Software
 - **Información:** <http://www.comunidadmoprosoft.org.mx/>
 - **Estructura**
 - Categoría Alta Dirección (DIR)**
Comprende la definición de las políticas, los objetivos de negocio y las estrategias para su logro incorporando la información de la evolución de la organización a efectos de promover la mejora continua.
 - **Gestión de Negocio:** establecer la visión y los objetivos de negocio que fijen el contexto para las actividades de la gerencia media y los proyectos.
 - Categoría Gerencia (GER)**
Comprende la gestión de los procesos, proyectos y recursos de la organización de manera alineada con las políticas definidas por la gerencia alta. Informa a la gerencia alta acerca de la evolución de la organización.

- **Gestión de Procesos:** promover la definición e implementación de los procesos de la organización a partir de las necesidades expresadas por el plan estratégico. Coordinar y colaborar en la mejora de los procesos definidos.
- **Gestión de Proyectos:** asegurar el desarrollo de los proyectos de manera alineada a los objetivos de la organización expresados en su plan estratégico.
- **Gestión de Recursos:** administrar los recursos de la organización proveyendo a los proyectos. Promover y facilitar la administración del capital de conocimiento de la organización. Las actividades de este proceso son soportadas por tres subprocesos:
 - Recursos Humanos y Ambiente de Trabajo
 - Bienes Servicios e Infraestructura
 - Conocimiento de la Organización.

Categoría Operación (OPE)

Comprende las prácticas de gestión y técnicas de los proyectos de desarrollo.

- **Administración de Proyectos Específicos:** estimar, planificar y monitorear el desarrollo de los proyectos de desarrollo y la generación de los productos esperados.
- **Desarrollo y Mantenimiento de Software:** desarrollo de las actividades del ciclo de vida de productos nuevos o modificados (análisis, diseño, construcción, integración y pruebas) alineadas con los requerimientos del proyecto.

También surgieron organizaciones que orientaron sus actividades en torno a los modelos de calidad mencionados, como es el caso de European Software Institute (ESITecnalia).

Este es un centro tecnológico privado creado en 1993 por la Comisión Europea con el apoyo del Gobierno Vasco y de empresas europeas líderes en el ámbito de las Tecnologías de la Información y la Comunicación. La principal actividad del ESITecnalia es ayudar a la industria del software en sus objetivos de crear productos de software de buena calidad. ESI ofrece servicios de auditoría, consultoría, formación y certificación, así como soporte tecnológico (<http://www.esi.es/>).

25.3 Norma ISO 9001-2000

La norma ISO 9000 está compuesta por un conjunto de estándares internacionales para sistemas de calidad, diseñados para la gestión y aseguramiento de la calidad [4], [5], [6]. Dentro de esta norma encontramos la norma ISO 9001 que puede ser empleada para el proceso de desarrollo de software. Está centrada en la eficiencia del sistema de gestión de la calidad para cumplir con los requerimientos del cliente. La norma ISO 9001-2000 es el resultado de una revisión de la norma realizada en el año 2000 que incluyó distintas críticas de los usuarios y se orientó a una aproximación de gestión basada en el proceso considerando los requisitos para una mejora continua del sistema de calidad y las necesidades de los involucrados.

Asimismo, en el año 2008 esta norma ha pasado por una nueva revisión generando la norma ISO 9001-2008. Esta nueva versión no introduce nuevos requerimientos sino que clarifica algunas cuestiones de la versión 2000 y aumenta la compatibilidad con la norma ISO 14001-2004.

Esta norma se utiliza como guía para las etapas de compra, abastecimiento, desarrollo, operación, mantenimiento del software y servicios relacionados.

Esta norma se divide en cinco secciones:

- Sistema de gestión de la calidad
 - Establece requisitos generales acerca de la forma de implementación, documentación, mantenimiento y mejora del sistema de gestión de la calidad en la organización.
- Responsabilidad de la gestión
 - Establece el compromiso que debe asumir la dirección dentro de los procesos, de la organización y con el cliente. Asimismo se extiende el compromiso a la planificación, que puede darse a nivel organizativo y a nivel de proyecto/producto. Por último, se menciona la responsabilidad que tiene la dirección para asegurar una adecuada comunicación y asignación de autoridad y responsabilidades.
- Gestión de los recursos
 - Establece la importancia de disponer de los recursos (personas e infraestructura) necesarios y adecuados para el desarrollo de los procesos definidos y asegurar su implementación, mantenimiento y mejora continua.
- Realización del producto
 - Esta sección es el esqueleto de la norma, por lo tanto detallaremos cada uno de los aspectos:
 - Planificación de la realización del producto: determina los aspectos a tener en cuenta para llevar a cabo la planificación del producto de software. Se establece que la forma en la cual se planificará quedará determinada por la metodología que la organización utilice.
 - Procesos relacionados con el cliente: determina la necesidad de especificar los requerimientos del cliente y el análisis de factibilidad y de riesgos de éstos. Diseño y desarrollo: involucra la etapa de diseño, desarrollo y pruebas del producto, y hace mención a la forma en la cual se llevan a cabo estas fases. Se incluyen revisiones de código, verificaciones y validaciones, además de la planificación y ejecución de las tareas que forman parte del diseño, desarrollo y pruebas del software. Como parte de las pruebas se listan las unitarias, de integración y sistema, de calificación y de aceptación.
 - Compras: determina los aspectos a ser considerados en el caso de una adquisición de software (proceso de compras, control del producto comprado, información de las compras y verificación de los productos comprados).
 - Producción y prestación del servicio: determina los aspectos a tener en cuenta durante la fase de implantación y mantenimiento del servicio. Entre los puntos más relevantes se menciona la disponibilidad del servicio, de la información, etc.
 - Control de los dispositivos de seguimiento y de medición: establece que se debe llevar a cabo tareas de seguimiento y medición para brindar evidencia de la conformidad del producto con los requerimientos determinados.
- Medición, análisis y mejora
 - Establece los procesos de seguimiento, medición, análisis y mejora como fin para demostrar el cumplimiento de los requerimientos del producto, del proceso de gestión de la calidad y de la mejora continua de la eficacia del sistema de gestión de calidad.

25.4 Modelo CMMi

Por un lado, el modelo CMMi provee un marco de referencia para los procesos de desarrollo de software [1]. Este marco proporciona una base para la evaluación de la madurez de las organizaciones y una guía para implementar una estrategia de mejora continua de los procesos. Tengamos en cuenta que la definición de los procesos en una organización dependerá de sus características, tamaño, del tipo de proyectos que desarrolle, etc. Por otro lado, CMMi provee dos representaciones para su marco: por niveles y por área de proceso. Cada una de las representaciones brinda diferentes ventajas que la organización deberá tener en cuenta a la hora de seleccionar una representación.

- Por niveles
 - Provee una secuencia de mejoras, comienza con prácticas básicas y evoluciona a prácticas avanzadas. Cada uno de los niveles se utiliza como base para el próximo.
 - Permite la comparación entre empresas por medio de niveles de madurez.
 - Provee una migración simple de SW-CMM a CMMi.
 - Provee una forma simple de obtener el resultado de un Appraisal y permite la comparación entre empresas.
- Por área de proceso
 - Permite seleccionar el orden en el cual se mejoran los objetivos de negocio y mitiga las áreas de riesgo de la organización.
 - Permite la comparación entre dos organizaciones por áreas de proceso.
 - Provee una migración fácil del Estándar Provisorio de la Alianza de Industrias Electrónicas (EIA/IS) 731 a CMMi.
 - Organización de las áreas de proceso similar a las definidas por la Organización Internacional de Estandarización y la Comisión Internacional de Electrotécnica (ISO/IEC) 15504. Esto permite realizar fácilmente comparaciones.

Ambas representaciones poseen componentes en común, como ser áreas de proceso, objetivos y prácticas genéricas, objetivos y prácticas específicas, productos de trabajo, sub-prácticas, notas y referencias. Estos componentes se relacionan entre sí. Para el esquema por áreas de proceso se podrán evaluar las áreas en forma independiente, así cada una de ellas podrá obtener los niveles de capacidad individualmente. Para el esquema por niveles, los de madurez corresponden a distintas áreas de proceso que deben ser tratadas en cada uno de los niveles. La representación por área de proceso maneja 5 niveles de madurez, cada uno de ellos contiene una serie de áreas de proceso con objetivos genéricos y específicos que deben ser cumplimentados para alcanzar el nivel en cuestión:

- **Inicial**
 - En este nivel se suponen procesos ad hoc y caóticos, que difícilmente son repetibles. El fracaso o éxito de los proyectos depende de la capacidad y responsabilidad de los empleados que los conforman. Asimismo, se supone que frecuentemente los proyectos se excedan en tiempo y costo.
- **Administrado**
 - Para las organizaciones que se encuentran en este nivel se puede asegurar que los requerimientos son administrados y los procesos establecidos son planificados, ejecutados, medidos y controlados.

- **Definido**

- Las organizaciones ubicadas en este nivel han cumplimentado los objetivos del nivel 2 y 3. Aquí los procesos son bien definidos y entendidos, y se describen en estándares, procedimientos, herramientas y métodos. En el nivel 2 los procesos y procedimientos pueden variar en cada instancia específica del proceso, mientras que en el nivel 3 se supone una coherencia organizacional que impide este tipo de situaciones. A su vez, los procesos en este nivel son más detallados y rigurosos que en el nivel 2.

- **Cuantitativamente administrado**

- En este nivel las organizaciones han cumplido los objetivos correspondientes al nivel 2, 3 y 4, y los objetivos genéricos para los niveles 2 y 3. Se manejan estadísticas de los procesos que buscan detectar el origen de los desvíos. La diferencia más importante con el nivel 3 es que aquí los procesos son cuantitativamente predictivos debido a que son controlados estadísticamente, es decir que se cuenta con previsibilidad estadística de los resultados.

- **Optimización**

- Las organizaciones ubicadas en este nivel han alcanzado los objetivos específicos correspondientes para los niveles 2, 3, 4 y 5, y los objetivos genéricos para los niveles 2 y 3. Aquí los procesos son mejorados continuamente teniendo en cuenta el entendimiento cuantitativo de desviación de éstos.

La mejora de los procesos se encara desde el punto de vista incremental y de innovación tecnológica. De esta manera los procesos son continuamente mejorados en función de los resultados estadísticos obtenidos, es decir que no solo se detecta la causa de los desvíos, como en el nivel 4, sino que también se busca el cambio en el proceso para su mejora.

Hasta aquí hemos descripto cada uno de los modelos, pero aún no mencionamos cuáles son las áreas de procesos involucradas. Estas áreas de proceso pueden ser agrupadas en 4 categorías:

- **Administración de proceso**

- Contempla las tareas que afectan en forma transversal al proyecto, definiciones, planificaciones, implementación, integración, monitoreo, control, mediciones y mejora de procesos. Contiene las siguientes áreas:
 - Enfoque de los procesos de la organización
 - Definición de los procesos de la organización
 - Capacitación
 - Desempeño de los procesos de la organización
 - Innovación e implementación organizacional.

- **Administración de proyecto**

- Contempla las tareas de administración de proyecto relacionadas con la planificación, el monitoreo y control de proyecto. Contiene las siguientes áreas:
 - Planificación de proyecto
 - Monitoreo y control del proyecto
 - Administración de los proveedores
 - Administración integral de los proyectos
 - Administración de los equipos de proyectos
 - Administración de riesgos
 - Administración de proveedores integrada

- **Administración de la calidad del prIngeniería**
 - Contempla las tareas de diseño, desarrollo y validación. Contiene las siguientes áreas:
 - Desarrollo de los requerimientos
 - Administración de los requerimientos
 - Solución técnica (diseño)
 - Integración del producto
 - Verificación
 - Validación.
- **Soporte**
 - Contempla las tareas de soporte para el desarrollo del producto y el mantenimiento. Contiene las siguientes áreas:
 - Administración de la configuración
 - Aseguramiento de la calidad de producto y proceso
 - Medidas y análisis
 - Entorno organizacional para integración
 - Análisis de decisiones y resolución
 - Análisis causal y resolución.

A continuación, en la figura 25.1, se presenta un diagrama con los componentes de CMMi en donde se intenta expresar la relación entre las áreas de proceso, los objetivos genéricos, las prácticas genéricas, los objetivos específicos y las prácticas específicas. El lector podrá apreciar que los objetivos y las prácticas genéricas son compartidos por todas las áreas de procesos involucradas, mientras que los objetivos y prácticas específicas son exclusivos de cada área.

25.5 Enfoque y alcance de aplicación

Para la aplicación de cada una de las normas y los estándares es fundamental entender el contexto de la organización y llevar a cabo la implementación en función de ello [2], [3]. A simple vista puede apreciarse que no es lo mismo la aplicación de CMMi en una organización pequeña que en una empresa multinacional. Pero no sólo la cantidad de empleados es un factor importante, sino también el alcance que se quiera dar, los objetivos de negocio, la cultura de la empresa, etc. El alcance de aplicación es un aspecto a definir y a tener en cuenta para la implementación. Por ejemplo, pensemos en una PYME que planea implementar CMMi. Es muy probable que sólo considere un área de desarrollo y limite así el alcance. La cantidad de proyectos y personas involucradas se acota, así como también el alcance en la definición de los procesos, los aspectos a ser considerados y los costos. También es factible, dependiendo del contexto, quitar el área de proveedores. Entonces reducimos aún más el alcance.

25.6 Conclusiones

El éxito en la implementación de las normas y los estándares de software en una organización está ligado a cómo se adapta a sus características, al apoyo gerencial y en menor medida al contexto externo de la empresa.

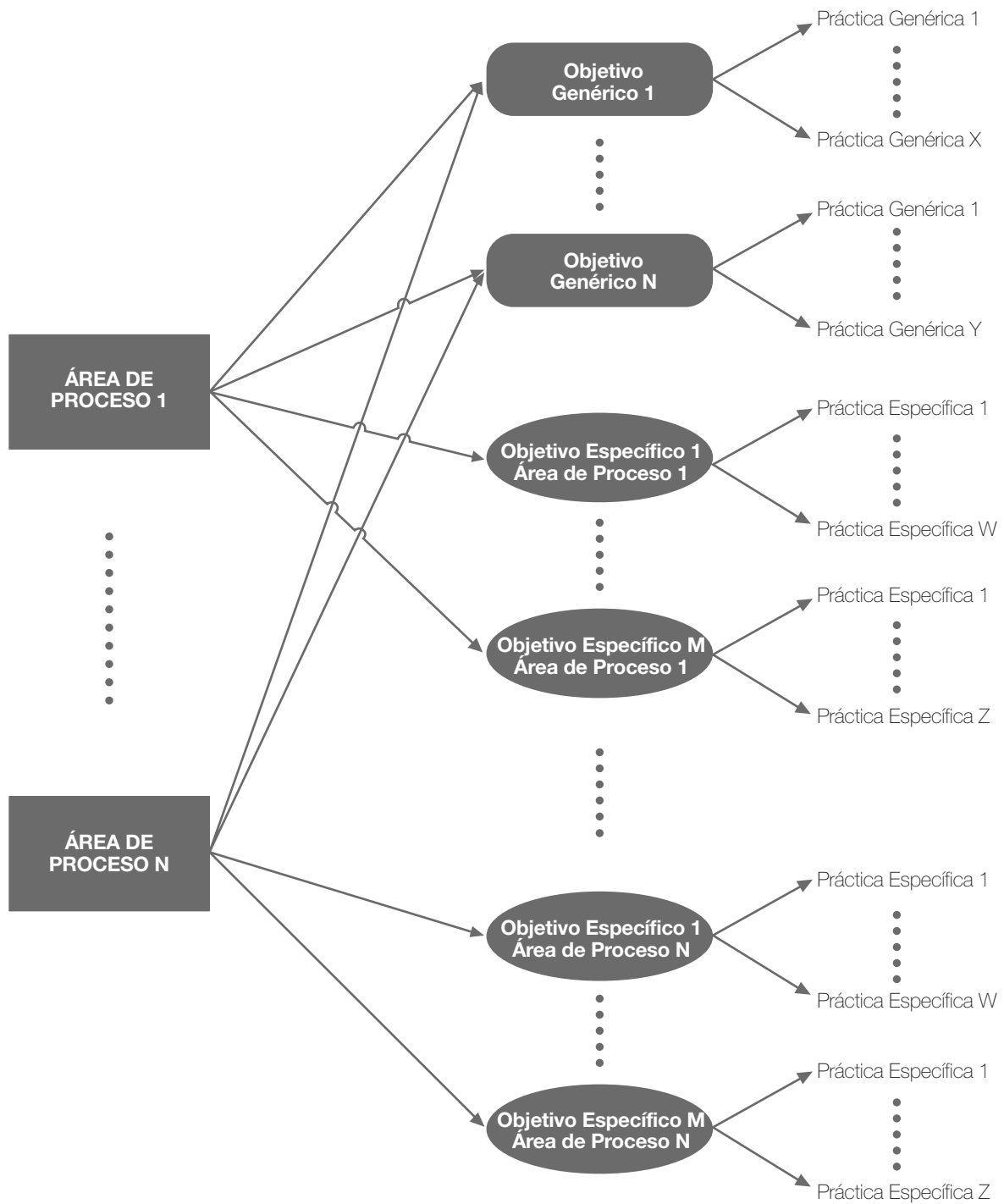


Figura 25.1. Modelo de componentes de CMMi

Existe una amplia variedad de guías y normas para el desarrollo de software que pueden ser aplicadas para mejorar la actividad. Seleccionar aquella que mejor se adapte a las necesidades requiere de un análisis y del apoyo de expertos.

25.7 Contenido de la página web de apoyo



25.7.1 Mapa conceptual

25.7.2 Autoevaluaciones

25.7.3 Presentaciones en Power Point (exclusivo para el docente)

25.8 Referencias

- [1] *Capability Maturity Model Integration (CMMi)*, Versión 1.1, Carnegie Mellon Software Engineering Institute, August 2002.
- [2] *Ingeniería de Software. Un enfoque práctico*, Quinta Edición, Roger S. Presuman, McGraw-Hill, 2002.
- [3] *Calidad en el desarrollo de Software*, Guillermo Pantaleo, Capítulo 1, Alfaomega, 2011.
- [4] *International Organization for Standardization 9001*, 2000, <http://www.iso.org/>.
- [5] *ISO/IEC 12207:2008, Systems and software engineering, Software life cycle processes*, <http://www.iso.org/>.
- [6] *ISO/IEC 15504:2006, Software Process Improvement Capability Determination (SPICE)*, <http://www.iso15504.es/>.

Contenido

26.1 Acerca de la tecnología	452
26.2 Acerca de la forma de trabajo	452
26.3 Acerca de la calidad	453
26.4 Acerca de la Ingeniería de Software	453
26.5 Contenido de la página web de apoyo	454
26.6 Referencias	454

Objetivos

- Qué concluimos acerca de la tecnología
- Qué concluimos acerca de la forma de trabajo
- Qué concluimos acerca de la calidad
- Qué concluimos acerca de la ingeniería de Software

Hemos respondido de esta forma las preguntas que nos hiciéramos en el prefacio al comenzar el libro. Los 25 capítulos que acaban de ser leídos fueron concebidos y elaborados con el objetivo planteado de escribir un libro de consulta que cubriera todos los temas de interés y de utilidad para aquellos involucrados de alguna manera con las actividades de la Ingeniería de Software. Se trató de incluir las cuestiones que a nuestro criterio deben ser aprendidas y maduras por los estudiantes y profesionales. Fueron tratadas con el nivel que nos pareció adecuado en el sentido de transmitir información y conocimiento sin dejar de mantener relacionados los diferentes temas por ahondar en los detalles. Como expresamos al comienzo, la mayor parte de los temas pueden ser profundizados a partir de la lectura de las referencias citadas en cada capítulo. A modo de cierre deseamos concluir tratando a cada uno de los aspectos de la Ingeniería de Software según el sesgo que siguen hoy día.

26.1 Acerca de la tecnología

Si bien existen y existirán sistemas con las más variadas arquitecturas, la integración de sistemas a partir de las arquitecturas orientadas a servicios (SOA) es el paradigma instalado [1]. La integración ha dado en realidad un paso más y los diferentes nodos de dichas arquitecturas se han difundido en la llamada “nube” (Cloud Computing) [2]. Los lenguajes dinámicos (Ruby, Python, JavaScript) han crecido en su utilización frente a los estáticos (C++, Java) [3].

C++ se ha enfocado en el desarrollo de sistemas embebidos y en la generación de código eficiente a partir de templates en lo que se llama *programación genérica* [4]. Por su parte, con Java se busca mantener el desarrollo de sistemas Enterprise en su versión JEE y, por otro, el desarrollo de sistemas embebidos en telefonía celular [5]. En relación a este último punto y debido al uso masivo de los celulares y el desarrollo de sistemas emergen nuevos conceptos que plantean nuevas reglas de negocio, por ejemplo los monederos virtuales.

Han aparecido lenguajes orientados a dominios [6] y las bases de datos distribuidas [7] han comenzado a ser utilizadas a partir de la robustez alcanzada por esta tecnología.

26.2 Acerca de la forma de trabajo

Indudablemente los últimos años han sido marcados por el avance de las metodologías ágiles, como consecuencia de contextos y requerimientos vertiginosamente cambiantes. XP y Scrum son las más utilizadas [8]. Sin embargo, como lo hemos tratado en los capítulos del libro, estas metodologías deben repensarse e incluir aspectos técnicos del modelo de desarrollo. Éstas se han ocupado de la gestión de los proyectos y sólo han propuesto buenas prácticas en lo técnico. Las metodologías conducidas por los planes siguen utilizándose con falencias en la adaptación a los proyectos en los cuales se aplican [8].

26.3 Acerca de la calidad

Los modelos de referencia utilizados por las organizaciones a efectos de trabajar en un marco de administración de la calidad tienen como estándar de hecho a CMMi. Sin embargo existe una gran cantidad de estos modelos que dependen de los países y regiones de que se trate. Un aporte interesante al aseguramiento de la calidad de procesos y productos provino en los últimos años de las metodologías ágiles. La forma de trabajo conocida como *integración continua* y sus promotores han generado una gran cantidad de herramientas que promueven el trabajo integrado entre los diferentes roles del grupo de desarrollo que mejora así la calidad en los proyectos de desarrollo [9].

26.4 Acerca de la Ingeniería de Software

La Ingeniería de Software debe administrar los procesos de gestión y desarrollo de los proyectos de desarrollo de software por lo que se ve impactada por todos los cambios ocurridos en cada uno de los aspectos anteriores. Si bien la tecnología no es su esencia debe ocuparse de su administración. Si bien la forma de trabajo es sólo una parte de ella los cambios en éstas afectan su visión de la administración del ciclo de vida. Si bien la calidad es un tema en sí mismo condiciona la concepción y elaboración de los procesos del desarrollo de software. Por todo esto la Ingeniería de Software debe aglutinar, compatibilizar y hacer consistente la dinámica de cada uno de los aspectos mencionados.

Por lo antedicho en los próximos años esta disciplina deberá compatibilizar la utilización de nuevas arquitecturas y de lenguajes dinámicos con formas de trabajo ágiles y deberá garantizar la calidad de los productos así generados a partir de nuevos y mejores procesos de prueba y revisión de software. Un aporte interesante en esta dirección son los trabajos que se realizan en métricas aplicadas al código para caracterizar el diseño y el código resultante [10]. Estos indicadores facilitan la organización de las sesiones de revisión a partir de la selección de los componentes cuyas métricas evidencien posibles problemas.

Otro problema a resolver vinculado a la forma de trabajo es hacer compatible el trabajo integrado de los diferentes roles con la importante especialización requerida que establecen las nuevas tecnologías.

La posibilidad de contar con herramientas de desarrollo y soporte que ofrecen nuevas funcionalidades constituye una ayuda pero también en muchos casos desafíos. Por ejemplo contamos con herramientas como GIT [11] que permite administrar repositorios de software distribuido, pero a la vez debemos utilizarlos de la mejor manera para aprovechar todas sus prestaciones y que su mal uso no se convierta en un obstáculo.

Por último, la integración de los componentes de software con las plataformas de hardware sobre las que funcionan, la administración de proyectos en organizaciones cada vez más grandes y sirviendo negocios cada vez más complejos así como la realimentación provista por la operación de estos sistemas hace necesario que la Ingeniería de Software pueda integrarse con otras para abarcar estos escenarios. Modelos como ITIL [12] son cada vez más utilizados en la administración de la tecnología informática los cuales incluyen todos los aspectos de la Ingeniería de Software.

26.5 Contenido de la página web de apoyo



26.5.1 Mapa conceptual

26.5.2 Autoevaluaciones

26.5.3 Presentaciones en Power Point (exclusivo para el docente)

26.6 Referencias

- [1] *SOA in practice, the art of Distributed System Design*, Nicolai Josuttis, O'Reilly, 2007.
- [2] *Cloud Computing and SOA Convergence in Your Enterprise: A Step-by-Step Guide*, David S. Linthicum, Addison-Wesley, 2009.
- [3] www.tiobe.com
- [4] *Presentation Materials: C++ and Beyond 2010*, Andrei Alexandrescu, Scott Meyers, and Herb Sutter, Artima Publishing, 2011.
- [5] *Java Must 'Seize the Lead in the Cloud,'* Rod Johnson, JAX 2011, <http://www.Jaxconf.com>.
- [6] *Domain-Specific Languages*, Martin Fowler, Addison-Wesley Signature Series, 2010.
- [7] *Proyecto Apache Cassandra*, <http://cassandra.apache.org/>.
- [8] *SDLC 3.0 – Beyond a tacit understanding of Agile*, Mark Kennaley, Fourth Medium Press, 2010.
- [9] *Fit for Developing Software: Framework for Integrated Tests*, Rick Mugridge, Ward Cunningham, Prentice Hall PTR, 2005.
- [10] *Object-Oriented Metrics in Practice - Using Software Metrics to Characterize, Evaluate, and Improve the Design of Object-Oriented Systems*; Michele Lanza, Radu Marinescu, Springer, 2006.
- [11] *GIT – the fast version control system*, <http://git-scm.com/>.
- [12] *The official introduction to the ITIL service lifecycle*, TSO, 2007.